LLM Systems from Scratch

Building, Training, Serving, and Teaching Language Models to Reason

A systems-first textbook with labs

Prepared for Utkarsh Kumar

Edition 1.0 November 2026

Preface

This book is the result of folding two code-first journeys into one, and then pushing both of them down a layer, into the machine.

The first journey goes from raw text to a working GPT: tokenisation, attention, the transformer block, pretraining, and fine-tuning for classification and for following instructions. The second starts from a small pretrained model and turns it into a *reasoning* model: generating text efficiently, grading answers with verifiers, spending more compute at inference time, scoring and refining answers, training with reinforcement learning on verifiable rewards, and distilling the result into something cheaper. These two journeys overlap: both build a text-generation loop, both talk about sampling, both load pretrained weights, both evaluate models. Here each idea appears once, where it fits best, and later chapters refer back to it.

The third journey is the reason this book exists. Implementing an LLM from scratch shows you *what* the model computes. A systems engineer also needs to know *where the time, memory, and bandwidth go*, and how real training and serving stacks are built around that answer. So every chapter carries a systems layer:

- **Systems Lens** boxes connect each modelling idea to its cost on real hardware: arithmetic intensity, memory traffic, kernel fusion, communication volume.
- **Napkin Math** boxes do the back-of-the-envelope calculations with real accelerator numbers, so you build intuition for orders of magnitude.
- Whole chapters cover topics that the from-scratch books leave out: distributed training (Chapter 8), serving engines with continuous batching and paged KV caches (Chapter 12), quantisation and speculative decoding (Chapter 13), the infrastructure behind RL for reasoning (Chapter 19), and writing GPU kernels in Triton (Appendix A).

How to read this book

The book has five parts. Part I sets up vocabulary and a performance mental model. Part II builds GPT-2 and then a modern Qwen3-style architecture from scratch. Part III trains them on one device and then on many, and fine-tunes them. Part IV is about inference: how a single request is generated, how a server multiplexes thousands of them, and how to make each token cheaper. Part V turns a base model into a reasoning model and studies the systems that make that training possible.

If you already know transformers well, you can skim Part II for the Systems Lens boxes and the labs. If your goal is inference serving, a fast path is Chapter 2 → Chapter 6 → Part IV →

Chapter 15 → Chapter 19.

The grey tags in the margin (for example LLM 3.5 or RM E.3) mark which section of the two source books a passage covers: LLM is *Build a Large Language Model (From Scratch)* and RM is *Build a Reasoning Model (From Scratch)*. The roadmap that follows the table of contents lists the full mapping, so you can check that nothing has been left out.

Labs

Every chapter ends with a lab. Labs live in `code/labs/labNN_*` and share the `llmsys` package in `code/llmsys`. Each lab has a `starter.py` with `TODO`s, a reference `solution.py`, and a test file. The tests run on a laptop CPU in under a minute with tiny model configurations:

```
cd code
pip install -r requirements.txt
LAB_IMPL=starter  python -m pytest labs/lab04_attention   # grade your work
LAB_IMPL=solution python -m pytest labs/lab04_attention   # check the reference
```

Each lab also has a *Measurements* section. Those experiments are meant to be run on real hardware, ideally a GPU, with real checkpoints such as GPT-2 and Qwen3-0.6B, and they ask you to explain what you see using the cost models from the chapter. Numbers in this book are either derived analytically from published hardware specifications or explicitly labelled as measured; your own measurements are the ones that count.

Conventions

Tensors are written with their shapes, as in $\mathbf{X} \in \mathbb{R}^{B \times T \times d}$, where B is the batch size, T the sequence length, and d the model width d_{model}. We write L for the number of layers, H for query heads, G for key/value heads, d_h for the head dimension, d_{ff} for the MLP width, and V for the vocabulary size. "FLOPs" counts floating-point operations, with a multiply–add counted as two. Bandwidths are in bytes per second and use decimal prefixes ($1\,\text{TB/s} = 10^{12}\,\text{B/s}$).

Contents

Preface **ii**

Roadmap and Coverage Map **xii**

I Foundations **1**

1 The Landscape: LLMs, Reasoning Models, and the Systems View **2**
1.1 What is a large language model? . . . 2
1.2 What LLMs are used for . . . 4
1.3 The stages of building and using an LLM . . . 5
1.4 The transformer architecture . . . 6
1.5 Training data at scale . . . 7
1.6 A closer look at GPT . . . 8
1.7 From LLMs to reasoning models . . . 9
1.8 Three levers for improving reasoning . . . 10
1.9 Pattern matching versus logical reasoning . . . 11
1.10 Simulating reasoning without explicit rules . . . 11
1.11 Why reasoning models, and why from scratch? . . . 12
1.12 The systems view: where the FLOPs and bytes go . . . 13
1.13 A roadmap for this book . . . 16
1.14 Summary . . . 17
1.15 Exercises . . . 19
1.16 Lab 1: A napkin-math calculator for LLMs . . . 20

2 PyTorch and the Machine Underneath **22**
2.1 What is PyTorch? . . . 22
2.2 Tensors . . . 24
2.3 Models as computation graphs . . . 26
2.4 Automatic differentiation . . . 27
2.5 Implementing multilayer neural networks . . . 27
2.6 Setting up efficient data loaders . . . 28
2.7 A typical training loop . . . 29
2.8 Saving and loading models . . . 30
2.9 Training on GPUs . . . 31
2.10 The machine underneath: how a GPU is organised . . . 33
2.11 The roofline model . . . 34

2.12 The execution model: asynchrony, overhead and compilation 36
2.13 Measuring and profiling 38
2.14 Summary 39
2.15 Exercises 40
2.16 Lab 2: PyTorch essentials and an empirical roofline 41

II Building a Large Language Model 43

3 Text Data, Tokenization, and the Input Pipeline 44
3.1 From symbols to vectors: embeddings 44
3.2 Tokenizing text 45
3.3 Converting tokens into token IDs 46
3.4 Adding special context tokens 47
3.5 Byte pair encoding 47
3.6 Tokenizers in the wild 51
3.7 Data sampling with a sliding window 52
3.8 Creating token embeddings 54
3.9 Encoding word positions 54
3.10 The input pipeline at scale 56
3.11 Putting it together: from a file to a (B, T, d) tensor 58
3.12 Summary 59
3.13 Exercises 60
3.14 Lab 3: Tokenizers and the input pipeline 61

4 Attention Mechanisms, from Dot Products to FlashAttention 63
4.1 The problem with modeling long sequences 63
4.2 Capturing data dependencies with attention 64
4.3 Self-attention without trainable weights 65
4.4 Self-attention with trainable weights 66
4.5 Hiding future words with causal attention 68
4.6 Extending single-head attention to multi-head attention 70
4.7 The cost of attention 72
4.8 IO-aware attention: the online softmax and FlashAttention 74
4.9 Attention at inference time: a preview of the KV cache 77
4.10 Beyond dense multi-head attention 78
4.11 Summary 79
4.12 Exercises 79
4.13 Lab 4: Attention, from dot products to IO-aware tiling 81

5 Implementing a GPT Model 83
5.1 Coding an LLM architecture 83
5.2 Normalizing activations with layer normalization 86
5.3 A feed-forward network with GELU activations 88
5.4 Adding shortcut connections 90
5.5 Connecting attention and linear layers in a transformer block 92
5.6 Coding the GPT model 93
5.7 Generating text 96

5.8 Accounting for every FLOP and byte . . . 97
5.9 Summary . . . 100
5.10 Exercises . . . 100
5.11 Lab 5: Assemble GPT-2 and account for every parameter, FLOP and byte . . . 102

6 Modern Architectures: Qwen3 from Scratch 104
6.1 From GPT-2 to Qwen3 . . . 104
6.2 RMSNorm . . . 105
6.3 Gated feed-forward networks: SwiGLU . . . 107
6.4 Rotary position embeddings . . . 108
6.5 Grouped-query attention . . . 110
6.6 The Qwen3 transformer block . . . 112
6.7 The main model . . . 113
6.8 KV cache . . . 114
6.9 The tokenizer . . . 115
6.10 Using the model . . . 116
6.11 Scaling up: larger Qwen3 models . . . 117
6.12 Beyond dense models: mixture of experts . . . 119
6.13 Summary . . . 121
6.14 Exercises . . . 122
6.15 Lab 6: Qwen3 from scratch, and planning its memory . . . 123

III Training 125

7 Pretraining on Unlabeled Data 126
7.1 Evaluating generative text models . . . 126
7.2 Training an LLM . . . 130
7.3 Decoding strategies to control randomness . . . 131
7.4 Saving and loading model weights . . . 133
7.5 Loading pretrained weights from OpenAI . . . 135
7.6 Bells and whistles for the training loop . . . 136
7.7 Where the memory goes . . . 139
7.8 Mixed precision . . . 140
7.9 Trading compute for memory, and measuring efficiency . . . 141
7.10 Scaling laws and compute budgeting . . . 143
7.11 Training stability at scale . . . 144
7.12 Summary . . . 144
7.13 Exercises . . . 145
7.14 Lab 7: An instrumented pretraining loop . . . 146

8 Distributed Training Systems 148
8.1 The problem in numbers . . . 148
8.2 Communication fundamentals . . . 149
8.3 Data parallelism . . . 152
8.4 ZeRO and fully sharded data parallelism . . . 155
8.5 Tensor parallelism . . . 157
8.6 Pipeline parallelism . . . 159

8.7 Sequence and context parallelism for long contexts . . . 160
8.8 Expert parallelism . . . 161
8.9 Putting it together: 3D and 4D parallelism . . . 161
8.10 From scratch to production APIs . . . 163
8.11 Reliability at scale . . . 164
8.12 Summary . . . 164
8.13 Exercises . . . 165
8.14 Lab 8: Distributed training from scratch . . . 166

9 Fine-Tuning for Classification 168
9.1 The fine-tuning landscape . . . 168
9.2 Preparing the dataset . . . 169
9.3 Creating data loaders . . . 171
9.4 Initializing a model with pretrained weights . . . 173
9.5 Adding a classification head . . . 175
9.6 Calculating the classification loss and accuracy . . . 177
9.7 Fine-tuning the model on supervised data . . . 178
9.8 Using the LLM as a spam classifier . . . 180
9.9 From one classifier to a fleet . . . 181
9.10 Summary . . . 184
9.11 Exercises . . . 185
9.12 Lab 9: Classification fine-tuning and padding-aware batching . . . 186

10 Instruction Fine-Tuning and Parameter-Efficient Adaptation 188
10.1 Introduction to instruction fine-tuning . . . 188
10.2 Preparing a dataset for supervised instruction fine-tuning . . . 189
10.3 Organizing data into training batches . . . 191
10.4 Creating data loaders for an instruction dataset . . . 194
10.5 Loading a pretrained LLM . . . 195
10.6 Fine-tuning the LLM on instruction data . . . 195
10.7 Extracting and saving responses . . . 196
10.8 Evaluating the fine-tuned LLM . . . 197
10.9 Conclusions and what comes next . . . 198
10.10 Parameter-efficient fine-tuning with LoRA . . . 198
10.11 Summary . . . 204
10.12 Exercises . . . 205
10.13 Lab 10: The SFT data path, packing, LoRA and QLoRA . . . 206

IV Inference and Serving 208

11 Inference Fundamentals: From Logits to Tokens per Second 209
11.1 From a prompt to a response . . . 209
11.2 Setting up the environment . . . 210
11.3 Hardware: what you need and why . . . 211
11.4 Preparing inputs: tokenizers and chat templates . . . 212
11.5 Loading pretrained weights . . . 213
11.6 The autoregressive loop, one step at a time . . . 214

11.7 A minimal streaming generator . . . 216
11.8 KV caching . . . 218
11.9 Compiling the model . . . 219
11.10Where the time goes: launch overhead and CUDA graphs . . . 221
11.11Anatomy of a decode step: the roofline view . . . 222
11.12Measuring inference: metrics and honest benchmarks . . . 224
11.13Summary . . . 225
11.14Exercises . . . 226
11.15Lab 11: Three decoders and a roofline . . . 227

12 Batching and Serving Engines **229**
12.1 Latency, throughput, and why batching helps . . . 229
12.2 Running batched generation . . . 230
12.3 Padding and attention masks . . . 231
12.4 Batching across the reasoning workflow . . . 233
12.5 Single-sequence versus batched execution . . . 234
12.6 From static to continuous batching . . . 235
12.7 Paged KV memory . . . 236
12.8 Sharing KV state: prefix caching and copy-on-write . . . 237
12.9 Chunked prefill and the token budget . . . 239
12.10Scheduling, admission, and preemption . . . 239
12.11Disaggregating prefill and decode . . . 240
12.12Anatomy of a production engine . . . 241
12.13Building a mini engine . . . 243
12.14Simulating serving policies at scale . . . 244
12.15Summary . . . 245
12.16Exercises . . . 246
12.17Lab 12: A paged, continuous-batching engine . . . 247

13 Making Inference Cheaper and Deploying It **249**
13.1 Where the cost of a token goes . . . 249
13.2 Quantization fundamentals . . . 251
13.3 Speculative decoding . . . 256
13.4 Splitting a model across GPUs for inference . . . 261
13.5 Structured output and constrained decoding . . . 262
13.6 The serving stack around the engine . . . 263
13.7 A chat interface for our models . . . 264
13.8 Summary . . . 268
13.9 Exercises . . . 268
13.10Lab 13: Quantize, speculate, and serve . . . 269

V Reasoning Models **272**

14 Evaluating LLMs and Reasoning Models **273**
14.1 Four ways to measure a language model . . . 273
14.2 Verifiers: grading by computation . . . 275
14.3 Loading a pretrained model to generate answers . . . 275

14.4 A generation wrapper . . . 276
14.5 Extracting the final answer . . . 277
14.6 Normalizing the extracted answer . . . 278
14.7 Verifying mathematical equivalence . . . 280
14.8 Grading answers . . . 281
14.9 The MATH-500 benchmark . . . 282
14.10Evaluating the model end to end . . . 283
14.11Multiple-choice accuracy . . . 286
14.12Verifiers beyond math . . . 288
14.13Preferences and leaderboards . . . 288
14.14LLM-as-a-judge . . . 290
14.15Designing evaluation infrastructure . . . 291
14.16Summary . . . 292
14.17Exercises . . . 292
14.18Lab 14: An evaluation harness for reasoning models . . . 294

15 Inference-Time Scaling **295**
15.1 Spending compute at answer time . . . 295
15.2 Loading the model and a flexible generator . . . 296
15.3 Chain-of-thought prompting . . . 297
15.4 Temperature scaling . . . 298
15.5 Top-p (nucleus) sampling . . . 300
15.6 Other decoding controls . . . 302
15.7 Self-consistency . . . 302
15.8 Executing parallel samples efficiently . . . 305
15.9 Compute-optimal test-time scaling . . . 307
15.10Summary . . . 308
15.11Exercises . . . 308
15.12Lab 15: Sampling, self-consistency, and shared-prefix decoding . . . 309

16 Self-Refinement and Scoring **311**
16.1 Scoring and iteratively improving responses . . . 311
16.2 Loading a model whose logits we can see . . . 312
16.3 A rule-based score . . . 312
16.4 Token probabilities . . . 313
16.5 From probabilities to log-probabilities . . . 314
16.6 Scoring confidence with log-probabilities . . . 315
16.7 Learned scorers: reward models . . . 317
16.8 Self-refinement through iterative feedback . . . 318
16.9 Coding the refinement loop . . . 319
16.10The cost of thinking sequentially . . . 321
16.11Beyond a single chain: search . . . 322
16.12Summary . . . 322
16.13Exercises . . . 322
16.14Lab 16: Scoring responses and score-gated refinement . . . 323

17 Reinforcement Learning with Verifiable Rewards: GRPO **325**
17.1 Reinforcement learning for LLMs . . . 325

17.2 RLVR with GRPO . . . 329
17.3 Loading a pretrained model . . . 331
17.4 Loading a MATH training subset . . . 332
17.5 Sampling rollouts . . . 332
17.6 Calculating rewards . . . 334
17.7 From rewards to advantages . . . 334
17.8 Scoring rollouts with sequence log-probabilities . . . 335
17.9 From advantages to a policy update: the GRPO loss . . . 337
17.10Putting the stages into one function . . . 338
17.11The GRPO training loop . . . 339
17.12Loading and evaluating checkpoints . . . 341
17.13Where the memory goes: a GRPO step at scale . . . 342
17.14Summary . . . 343
17.15Exercises . . . 343
17.16Lab 17: GRPO from scratch . . . 345

18 Improving GRPO: Diagnostics, Stability and Its Descendants 347
18.1 From a working loop to a robust one . . . 347
18.2 Tracking GRPO performance metrics . . . 348
18.3 Tracking more advanced metrics . . . 350
18.4 Stabilising updates with clipped policy ratios . . . 352
18.5 Controlling drift with a KL penalty . . . 355
18.6 Adding an explicit format reward . . . 358
18.7 The systems cost of each knob . . . 362
18.8 Summary . . . 362
18.9 Exercises . . . 364
18.10Lab 18: making GRPO robust . . . 365

19 RL Training Systems 367
19.1 The anatomy of an RL step . . . 367
19.2 Models, memory, and placement . . . 370
19.3 Weight synchronisation and resharding . . . 371
19.4 Synchronous versus asynchronous RL . . . 373
19.5 The long-tail problem . . . 374
19.6 Packing the scoring and training passes . . . 376
19.7 Train/inference mismatch . . . 378
19.8 Configuring the rollout engine for RL . . . 380
19.9 Multi-turn and agentic rollouts . . . 381
19.10Reward infrastructure . . . 381
19.11A tour of open RL frameworks . . . 383
19.12Checkpointing and fault tolerance . . . 384
19.13Putting it together: a design worksheet . . . 384
19.14Summary . . . 385
19.15Exercises . . . 385
19.16Lab 19: Building blocks of an RL training system . . . 386

20 Distillation for Efficient Reasoning 389
20.1 What distillation is . . . 389

20.2 Generating a distillation dataset . . . 391
20.3 Loading the distillation data . . . 393
20.4 Building training examples . . . 393
20.5 Loading the student . . . 396
20.6 The training and validation losses . . . 396
20.7 The distillation training loop . . . 398
20.8 Evaluating the distilled model . . . 399
20.9 Soft distillation as a systems problem . . . 401
20.10 Training on long traces . . . 402
20.11 Efficient reasoning: making the student cheaper to run . . . 403
20.12 Future directions for reasoning models . . . 404
20.13 Conclusions . . . 405
20.14 Summary . . . 405
20.15 Exercises . . . 406
20.16 Lab 20: Distilling a reasoning teacher . . . 407

A GPU Kernels with Triton 409
A.1 Why write a kernel at all? . . . 409
A.2 The Triton programming model . . . 410
A.3 Fused softmax and RMSNorm . . . 412
A.4 Tiled matrix multiplication . . . 415
A.5 A fused W8A16 GEMM . . . 417
A.6 FlashAttention forward in Triton . . . 418
A.7 A paged-attention decode kernel . . . 420
A.8 Numerics inside kernels . . . 421
A.9 Under the hood: from Python to SASS . . . 422
A.10 Triton and CUDA: a translation table . . . 423
A.11 Measuring kernels . . . 423
A.12 Summary . . . 424
A.13 Exercises . . . 424
A.14 Lab A: GPU kernels with Triton . . . 426

B Lab Environment, Hardware, and Checkpoints 428
B.1 Repository layout . . . 428
B.2 Installation . . . 428
B.3 Which hardware for which lab . . . 429
B.4 Getting checkpoints and datasets . . . 429
B.5 Reproducibility and measurement hygiene . . . 430

C References and Further Reading 431

D Solutions to Exercises 440

Roadmap and Coverage Map

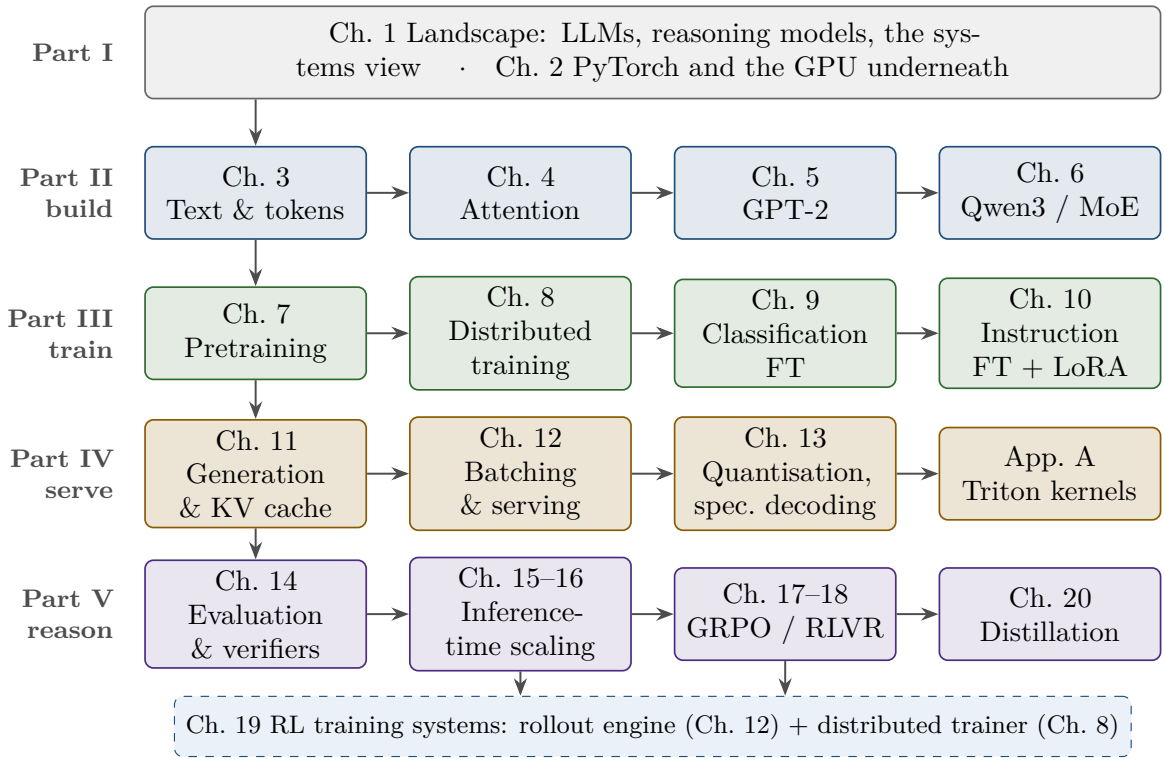


Figure 1: How the book fits together. Each row is a part of the book, read left to right; the first column of each part builds directly on the part above it. Chapter 19 combines the serving engine of Part IV with the distributed trainer of Part III to run RL at scale.

Coverage of the source books

The table below maps every chapter and appendix section of the two source books to the place where this book covers it. LLM is *Build a Large Language Model (From Scratch)* (Raschka, 2024); RM is *Build a Reasoning Model (From Scratch)* (Raschka). The material is reorganised so that overlapping topics appear once. For example, LLM 5.3 (temperature and top-k) and RM 4.4–4.5 (temperature and top-p) are both developed in Chapters 7 and 15, and each later chapter builds on the earlier one. The margin tags inside the chapters give the precise locations.

Source	Topic	Covered in
Build a Large Language Model (From Scratch)		
LLM 1.1–1.7	What an LLM is, applications, stages, transformer, datasets, GPT, roadmap	Chapter 1

Source	Topic	Covered in
LLM 2.1–2.8	Embeddings, tokenisation, token IDs, special tokens, BPE, sliding-window sampling, token & positional embeddings	Chapter 3
LLM 3.1–3.6	Long-sequence problem, attention, simplified and trainable self-attention, causal masking, dropout, multi-head attention	Chapter 4
LLM 4.1–4.7	GPT architecture, LayerNorm, GELU MLP, shortcuts, transformer block, GPT model, generation	Chapter 5
LLM 5.1–5.2	Generation loss, cross entropy, perplexity, train/val losses, training loop	Chapter 7
LLM 5.3	Temperature, top-k, modified generation function	Chapter 7 (and Chapter 15)
LLM 5.4–5.5	Saving/loading weights, loading OpenAI GPT-2 weights	Chapter 7
LLM 6.1–6.8	Fine-tuning categories, spam dataset, loaders, classification head, loss/accuracy, training, inference	Chapter 9
LLM 7.1–7.9	Instruction data, prompt format, batching/collate, loaders, fine-tuning, response extraction, LLM-judge evaluation	Chapter 10
LLM App. A	PyTorch: tensors, autograd, models, data loaders, training loop, save/load, GPUs, DDP	Chapter 2
LLM App. B	References and further reading	Appendix C
LLM App. C	Exercise solutions	Appendix D (our own exercises)
LLM App. D	Warmup, cosine decay, gradient clipping, modified training function	Chapter 7
LLM App. E	LoRA: idea, dataset, initialisation, parameter-efficient fine-tuning	Chapter 10
Build a Reasoning Model (From Scratch)		
RM 1.1–1.7	Defining reasoning, training pipeline, training vs inference techniques, pattern matching vs logic, roadmap	Chapter 1
RM 2.1–2.9	Text generation, environment, hardware, inputs, loading models, sequential generation, KV cache, compilation	Chapter 11
RM 3.1–3.9	Math verifier, answer extraction, normalisation, symbolic equivalence, grading, MATH-500 evaluation	Chapter 14
RM 4.1–4.6	Inference-time scaling, chain-of-thought, temperature, top-p, self-consistency	Chapter 15
RM 5.1–5.8	Scoring, rule-based scores, token probabilities, log-probs, confidence, self-refinement	Chapter 16
RM 6.1–6.12	RLHF vs RLVR, GRPO, rollouts, rewards, advantages, log-probs, loss, training loop, checkpoints	Chapter 17
RM 7.1–7.6	Metrics, advantage/entropy tracking, clipped ratios, KL term, format reward, GRPO variants	Chapter 18
RM 8.1–8.10	Distillation data, formatting, filtering, training, evaluation, future directions	Chapter 20
RM App. A	References and further reading	Appendix C
RM App. B	Exercise solutions	Appendix D (our own exercises)
RM App. C	Qwen3 source code: RMSNorm, FFN, RoPE, GQA, block, model, KV cache, tokenizer	Chapter 6
RM App. D	Larger dense Qwen3 models, checkpoints, reasoning variants	Chapter 6

Source	Topic	Covered in
RM App. E	Batching: padding, masks, batched evaluation/sampling/rollouts/distillation	Chapter 12
RM App. F	Evaluation methods: answer-choice accuracy, verifiers, leaderboards, LLM-as-a-judge	Chapter 14
RM App. G	Chat interface: single-turn and multi-turn	Chapter 13
New systems material (not in either source book)		
—	GPU architecture, roofline, numerics, profiling	Chapter 2
—	FlashAttention, data pipelines, activation memory, MoE, context extension	Chapters 3 to 6
—	Mixed precision, checkpointing, scaling laws, MFU	Chapter 7
—	Collectives, DDP, ZeRO/FSDP, tensor/pipeline/sequence/expert parallelism	Chapter 8
—	Packing, QLoRA, multi-LoRA serving	Chapter 10
—	Continuous batching, PagedAttention, prefix caching, chunked prefill, disaggregation	Chapter 12
—	Quantisation, speculative decoding, parallel inference, deployment	Chapter 13
—	RL frameworks, async RL, weight sync, rollout systems	Chapter 19
—	Triton kernels	Appendix A

Part I

Foundations

Chapter 1

The Landscape: LLMs, Reasoning Models, and the Systems View

Large language models are, at their core, a remarkably small amount of code wrapped around a remarkably large amount of arithmetic. The model definition of a modern open-weight LLM fits on a few printed pages; the computation needed to give those pages meaning (trillions of training tokens, tens of thousands of accelerator-days, petabytes of memory traffic for every day of serving) is what turns an architecture diagram into a product. This book takes both halves seriously. We will write every component of an LLM and of a *reasoning* model from scratch, and at every step we will ask the questions a systems engineer asks: *where do the FLOPs go, where do the bytes go, what is the bottleneck, and what would we change to remove it?*

This opening chapter sets up the vocabulary. We define what an LLM is and what it is used for, walk through the stages of building one (pretraining, fine-tuning, preference tuning), introduce the transformer and the GPT family, and then extend the story to reasoning models: models that spend extra tokens on intermediate steps and that are improved with inference-time scaling, reinforcement learning, and distillation. Throughout, we attach a cost model to every stage, so that by the end of the chapter you can estimate, to within a factor of two, what it costs to train, fine-tune, post-train, and serve a model of a given size. Figure 1.1 shows the plan.

1.1 What is a large language model?

LLM 1.1

A *language model* assigns probabilities to sequences of text. Write a piece of text as a sequence of discrete units $x_1, x_2, \ldots, x_T$ called *tokens* (we will define tokens precisely in Chapter 3; for now think "words or pieces of words"). By the chain rule of probability, any distribution over sequences factorises as

$$p(x_1, \ldots, x_T) \;=\; \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}). \tag{1.1}$$

An *autoregressive* language model learns the right-hand side directly: given a prefix, it outputs a probability distribution over the next token. A *large* language model (LLM) is such a model implemented as a deep neural network with a very large number of trainable parameters (hundreds of millions to hundreds of billions) and trained on a very large corpus (hundreds of billions to tens of trillions of tokens). Both meanings of "large" matter, and, as we will see in Section 1.12, they are tied together by the compute budget.

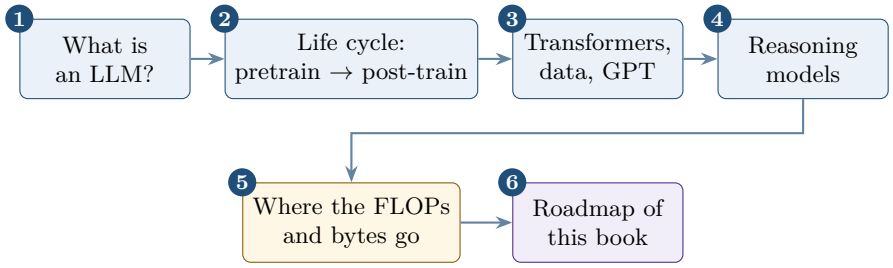


Figure 1.1: Roadmap of Chapter 1. Stages 1–4 build the conceptual picture shared by the two source books; stage 5 adds the cost model that the rest of the book refines; stage 6 lays out how the chapters fit together.

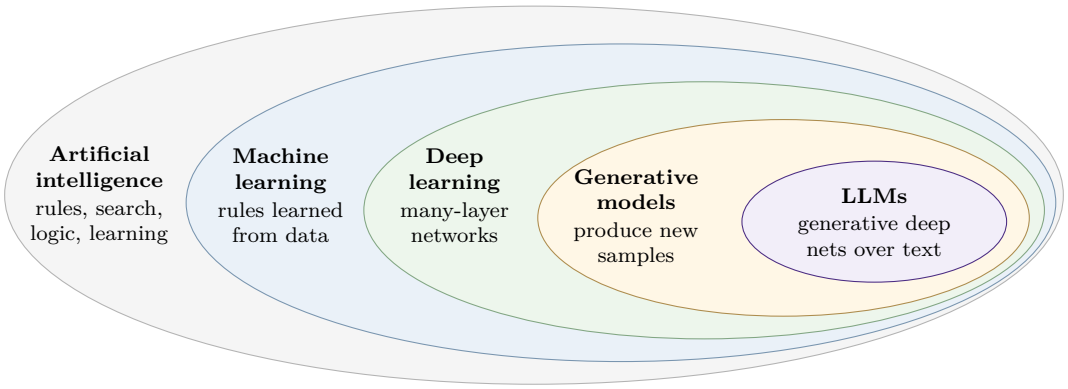


Figure 1.2: LLMs as a nested specialisation. Each ring narrows the previous one: machine learning learns its rules from data, deep learning learns its *features* too, generative models produce new samples, and LLMs are generative deep networks over text.

The training signal is almost embarrassingly simple: take real text, hide the next token, and penalise the model by the negative log-probability it assigned to the token that actually appeared. Summed over a corpus, this is the cross-entropy loss

$$\mathcal{L}(\theta) \;=\; -\frac{1}{N}\sum_{t} \log p_\theta(x_t \mid x_{<t}), \tag{1.2}$$

which we implement in Chapter 7. That such a plain objective yields systems that translate, summarise, write code and solve competition mathematics is one of the central empirical surprises of the last decade. The explanation usually offered is that predicting the next token *well* across the whole distribution of human text requires modelling syntax, facts, styles, and (to some degree) the processes that generated the text, including step-by-step problem solving.

Modern LLMs are built on the *transformer* architecture (Section 1.4), whose key ingredient, *self-attention*, lets every position in a sequence draw information from every earlier position with a learned, data-dependent weighting. Because they generate new content, LLMs are also called *generative AI*. Figure 1.2 places them in the wider field.

The distinction between the inner rings is worth making concrete. Consider building a spam filter. A classical machine-learning pipeline asks a human to engineer features (counts of words like “free” or “winner”, the fraction of upper-case characters, the number of links) and fits, say, a logistic regression on those features. A deep-learning pipeline instead feeds the raw text (as token ids) into a neural network and lets the early layers *learn* whatever features are useful. Both still need labelled examples of spam and non-spam; what changes is who designs the

representation. LLMs push the idea further: during pretraining they need no labels at all, because the next token *is* the label. We will build exactly this spam filter on top of a pretrained GPT in Chapter 9.

Systems Lens: "Large" is a statement about bytes

Parameter count is the number every model card leads with, but for a systems engineer it is a proxy for three more operational quantities: (1) *weight bytes*, which set the minimum memory footprint and, during decoding, the minimum memory traffic per generated token; (2) *FLOPs per token*, roughly $2N$ for a forward pass over a model with N parameters; and (3) *optimizer state bytes* during training, roughly $16N$ with mixed-precision AdamW. A 7B-parameter model is 14 GB of bf16 weights, about 14 GFLOP per generated token, and about 112 GB of training state before a single activation is stored. Most of this book's systems content is about managing those three numbers.

1.2 What LLMs are used for

LLM 1.2

Because nearly every knowledge task can be phrased as "read some text, write some text", LLMs have an unusually wide application surface:

- **Generation and transformation:** drafting prose and code, translation, rewriting for tone or audience, summarisation of long documents.
- **Understanding and classification:** sentiment analysis, intent detection, spam and abuse filtering, routing tickets (Chapter 9 implements this).
- **Assistants and chat:** multi-turn conversational agents that answer questions, follow instructions, and call tools (Chapters 10 and 13).
- **Knowledge work in specialised domains:** search and question answering over legal, medical or scientific corpora, often combined with retrieval.
- **Reasoning-heavy tasks:** competition mathematics, program synthesis, planning in agentic systems, where the reasoning methods of Part V matter most.

Why build or adapt your own model rather than call a hosted API? Four reasons recur. *Quality*: models specialised for a domain (finance, medicine, code) frequently beat larger general models on in-domain tasks. *Privacy and control*: sensitive data need not leave the organisation, and the owner decides when the model changes. *Cost and latency*: a small model on local hardware, or even on a phone or laptop, can be cheaper and faster than a round trip to a large hosted model. *Understanding*: you cannot debug, optimise, or extend what you do not understand, which is the motivation for building everything from scratch in this book.

Napkin Math: API vs. self-hosting, first order

Suppose a workload needs 10^9 generated tokens per day from an 8B-parameter model. At roughly $2N = 16$ GFLOP per token, that is 1.6×10^{19} FLOP/day. One H100 at a realistic 30–50% of its 989 TFLOP/s bf16 peak delivers 2.6–4.3×10^{19} FLOP/day, so on *compute* alone a single GPU suffices. But decoding is memory-bound (Section 1.12): at batch size 1 each token must stream all 16 GB of weights, capping a single sequence at about

3.35 TB/s/16 GB ≈ 200 tokens/s. Reaching $10^9/86400 \approx 11{,}600$ tokens/s requires batching on the order of a hundred concurrent sequences. Whether that is achievable depends on KV-cache memory and on the serving engine, the subject of Part IV. The economics of self-hosting are therefore a *batching* problem more than a FLOPs problem.

1.3 The stages of building and using an LLM

LLM 1.3; RM 1.2

1.3.1 Pretraining: learning from raw text

The first and by far the most expensive stage is *pretraining*. A randomly initialised network is trained with the next-token objective (1.2) on a huge corpus of raw, unlabelled text (web pages, books, encyclopaedias, code, scientific articles), usually after filtering and deduplication. Because the labels are derived from the data itself, this is called *self-supervised* learning. The result is a *base* or *foundation* model. A base model is a superb text *completer*: give it the beginning of a paragraph and it continues plausibly. It also shows *in-context learning*: shown a few input–output examples in the prompt (*few-shot*), or merely a description of the task (*zero-shot*), it can often perform a task it was never explicitly trained for. What it does not reliably do is follow instructions or hold a conversation; it continues text, and an instruction is just more text to continue.

Some pipelines distinguish a *mid-training* phase (continued pretraining on code, mathematics, long documents, or higher-quality data near the end of pretraining). We fold it into pretraining: the objective is the same and only the data mixture changes.

1.3.2 Post-training: fine-tuning and preference tuning

Fine-tuning continues training a pretrained model on a smaller, usually labelled, dataset. Two families matter in this book:

- **Classification fine-tuning**: the data are (text, class label) pairs; the output layer is replaced by a small classification head (Chapter 9).
- **Instruction fine-tuning**, also called supervised fine-tuning (SFT) or instruction tuning: the data are (instruction, desired response) pairs, and the model keeps its language-modelling head but learns to produce responses (Chapter 10).

Production assistants then apply *preference tuning*: the model is nudged towards responses that humans (or a model trained on human judgements) prefer. The classic recipe is reinforcement learning from human feedback (RLHF), which trains a reward model from human rankings and optimises the LLM against it with PPO; we study it in Chapter 17. Lighter-weight alternatives such as Direct Preference Optimization (DPO) optimise the same kind of preference data without an explicit RL loop. Pretraining gives the model raw linguistic competence; SFT teaches it the *format* of being helpful; preference tuning shapes *which* of many acceptable responses it prefers (tone, safety, concision).

Even an instruction- and preference-tuned model is not yet a product. A chat application adds a system prompt, conversation-history management, tool calling, safety layers and a serving stack; we build a minimal chat interface in Chapter 13. Figure 1.3 summarises the life cycle together with its typical compute cost.

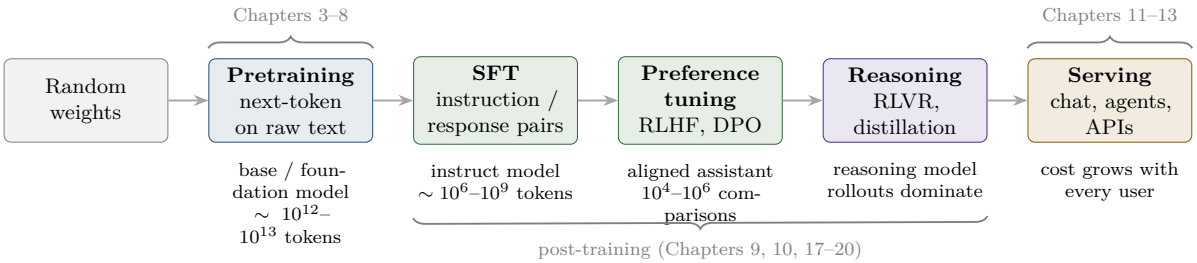


Figure 1.3: The LLM life cycle as used in this book. Pretraining consumes the bulk of training compute; each post-training stage changes behaviour with orders of magnitude less data; serving cost is paid continuously and, for a popular model, eventually dominates everything else.

Key Idea: Post-training changes behaviour, not knowledge

A useful mental model: pretraining stores most of what the model *knows*; post-training mostly selects *how* that knowledge is expressed. This is why SFT on a few tens of thousands of good examples can turn a completer into an assistant, and why reasoning-focused RL on comparatively few problems can unlock long chains of thought that were latent in the base model.

1.4 The transformer architecture

LLM 1.4

The transformer was introduced in 2017 by Vaswani et al. (“Attention Is All You Need”) for machine translation. The original model has two halves. An *encoder* reads the entire source sentence and turns each token into a context-dependent vector; a *decoder* generates the target sentence one token at a time, attending both to the tokens it has already produced and to the encoder’s vectors. Both halves are stacks of identical layers whose central operation is *self-attention*: each position computes a weighted average of the representations of other positions, with weights that depend on the content of both. Self-attention is what lets a model resolve that “it” in “the trophy did not fit in the suitcase because it was too big” refers to the trophy, by learning which earlier tokens are relevant to each later one. We derive it carefully in Chapter 4.

Subsequent work split the architecture into three families (Fig. 1.4):

- **Encoder-only** models such as BERT are trained to reconstruct masked words using context from *both* sides. They excel at understanding tasks (classification, retrieval embeddings) but do not naturally generate text.

- **Decoder-only** models such as the GPT family are trained to predict the next token using only the *left* context, enforced by a causal mask. They generate text naturally and, at scale, turned out to be good at understanding tasks too. Almost every modern LLM, including both models we build (GPT-2 and Qwen3), is decoder-only.

- **Encoder–decoder** models such as T5 keep both halves and remain popular for sequence-to-sequence tasks like translation and speech recognition.

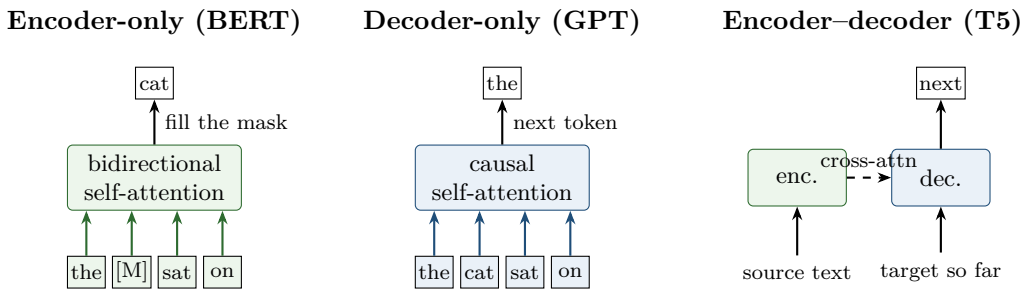


Figure 1.4: The three transformer families. Encoders see the whole input at once; decoders see only the past, which is exactly what is needed to generate text one token at a time.

Systems Lens: Why decoder-only won, from a systems angle

Decoder-only models have one stack, one objective and one data format (a stream of tokens), so every token of every document is a training example: a sequence of length T yields T next-token predictions in a single forward pass thanks to the causal mask. The same weights and the same kernels serve pretraining, fine-tuning, and inference, and the causal structure makes the *KV cache* possible: keys and values of past tokens never change, so they can be stored and reused (Chapter 11). An encoder's bidirectional attention would invalidate such a cache every time a token is appended. Simplicity of the training pipeline and cacheability at inference time are two underrated reasons for the dominance of the decoder-only design.

Zero-shot and few-shot. Encoder–decoder and encoder-only models were typically fine-tuned per task. GPT-3 popularised a different mode of use: describe the task in the prompt (zero-shot), or prepend a handful of solved examples (few-shot), and let the model continue. Few-shot prompting costs *prefill* compute and KV-cache memory proportional to the length of the examples on every request, which is why prefix caching (Chapter 12) is valuable for workloads with shared few-shot prefixes.

1.5 Training data at scale

LLM 1.5

The capabilities of LLMs are inseparable from the scale of their training data. GPT-3, the 175B-parameter model behind early ChatGPT-era products, is a useful historical reference point because its data mixture was published (Table 1.1). Two details are instructive. First, the mixture is *weighted*: high-quality sources such as Wikipedia and books are sampled more often than their raw size would suggest, so the model sees them for several epochs while seeing most of the web data less than once. Second, the model was trained on 300 billion tokens, fewer than the roughly 500 billion available, a choice later work on compute-optimal scaling (Chapter 7) would revisit: the Chinchilla analysis suggests about 20 tokens per parameter, and current open models are trained far beyond that (Qwen3 and Llama-3-class models on tens of trillions of tokens) because extra training compute is repaid by cheaper inference with a smaller model.

A token is not a word. With GPT-2's byte-pair-encoding tokenizer, English prose averages roughly 0.75 words per token, and code or non-English text usually needs more tokens per word. Tokens are the unit in which every cost in this book is measured: training compute scales with tokens processed, context windows are measured in tokens, and serving is priced per token.

Source	Tokens (B)	Share of mix	Character
Common Crawl (filtered)	410	60%	web pages, filtered for quality
WebText2	19	22%	pages linked from upvoted posts
Books1	12	8%	long-form prose
Books2	55	8%	long-form prose
Wikipedia	3	3%	encyclopaedic
Total available / trained	≈500 / 300		

Table 1.1: GPT-3's published pretraining mixture (Brown et al., 2020). Shares do not track sizes: small high-quality sources are upsampled.

Napkin Math: How big is a pretraining corpus on disk?

A 15-trillion-token corpus, typical for a 2025-era open model, tokenised with a vocabulary of about 152k (too large for 16-bit ids) and stored as `uint32` occupies $15 \times 10^{12} \times 4 = 60\,\text{TB}$. GPT-3's 300B training tokens with a 50,257-entry vocabulary fit in `uint16`: 600 GB. At, say, 2 GB/s of aggregate read bandwidth per training node, streaming 60 TB once takes about 8.3 node-hours, negligible compared with the compute, *provided* the data pipeline is prepared offline (tokenised, sharded and shuffled) rather than tokenised on the fly (Chapter 3).

The cost of the compute itself is dominated by the model and token count. Using the standard estimate (derived in Chapter 7) that training costs about $6ND$ FLOPs for N parameters and D tokens, GPT-3 required $6 \times 175 \times 10^9 \times 300 \times 10^9 \approx 3.15 \times 10^{23}$ FLOPs. Section 1.12 turns that into GPU-days.

1.6 A closer look at GPT

LLM 1.6

GPT ("generative pretrained transformer") models are decoder-only transformers trained with next-token prediction. Architecturally, GPT-3 is a scaled-up GPT-2, which is a scaled-up GPT-1: more layers, wider layers, more heads, longer context, more data. The decoder stack takes token ids, maps them to vectors, passes them through L identical transformer blocks, and projects the final vectors back to a score (*logit*) for every vocabulary entry.

Generation is *autoregressive* (Fig. 1.5): the model predicts a distribution over the next token, one token is chosen (greedily or by sampling), appended to the input, and the process repeats. Every generated token therefore requires a full forward pass through the network, and the t-th token depends on all $t-1$ before it. This sequential dependence is the single most important fact about LLM inference performance: it prevents parallelism across the output sequence and makes the per-token latency floor equal to the time to stream the model's weights through the chip (Chapter 11).

Although GPT was trained only to predict the next token, sufficiently large GPT models can translate, answer questions, and summarise without task-specific training. Capabilities that appear only at scale and were not targeted by the objective are often called *emergent*. The term is debated (some apparent emergence is an artefact of discontinuous metrics), but the practical point stands: a single pretrained model is a general substrate from which many tasks can be elicited, by prompting or by light fine-tuning.

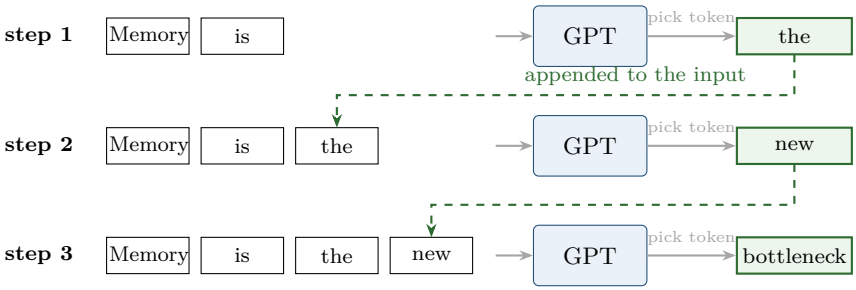


Figure 1.5: Autoregressive generation. Each step runs the network on the sequence so far and appends one token, which becomes input to the next step. Steps cannot run in parallel; only the work *within* a step can.

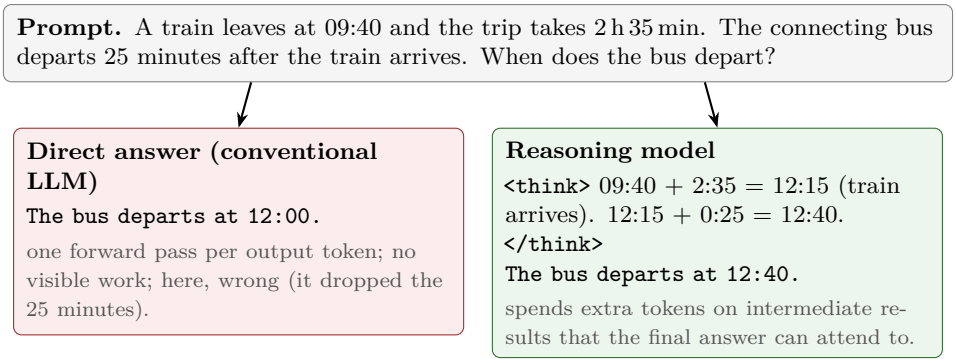


Figure 1.6: Direct answering versus generating intermediate steps. The reasoning trace turns one hard prediction ("final time") into several easy ones, each conditioned on written-down partial results.

1.7 From LLMs to reasoning models

RM 1.1

We use "reasoning" in an engineering sense: an LLM *reasons* when it generates intermediate steps before committing to a final answer. A *reasoning model* is an LLM that has been improved, by prompting, decoding procedures or training, to produce such intermediate steps, usually because doing so increases accuracy on multi-step tasks such as mathematics, coding and logic puzzles. The intermediate steps may be visible, as in classic chain-of-thought (CoT) prompting, or wrapped in special delimiters such as `<think>...</think>` and hidden from the end user. Either way, the model spends *tokens* on thinking. Figure 1.6 contrasts the two behaviours on a small example.

Why should writing things down help a next-token predictor? A transformer performs a bounded amount of computation per generated token: a fixed number of layers, each with a fixed width. Problems whose answer requires more sequential computation than one forward pass provides cannot be solved "in one go", but they can be solved if the model is allowed to write intermediate results into its own context and read them back with attention. In this view, the generated tokens act as a scratchpad and a variable-length compute budget. It also tells us the price: *reasoning is paid for in generated tokens*, the most expensive tokens in the system because they are produced sequentially.

Pitfall: LLM reasoning is probabilistic

A theorem prover or a symbolic solver applies fixed rules and returns the same, guaranteed-sound derivation every time. An LLM generates its steps by sampling from learned token

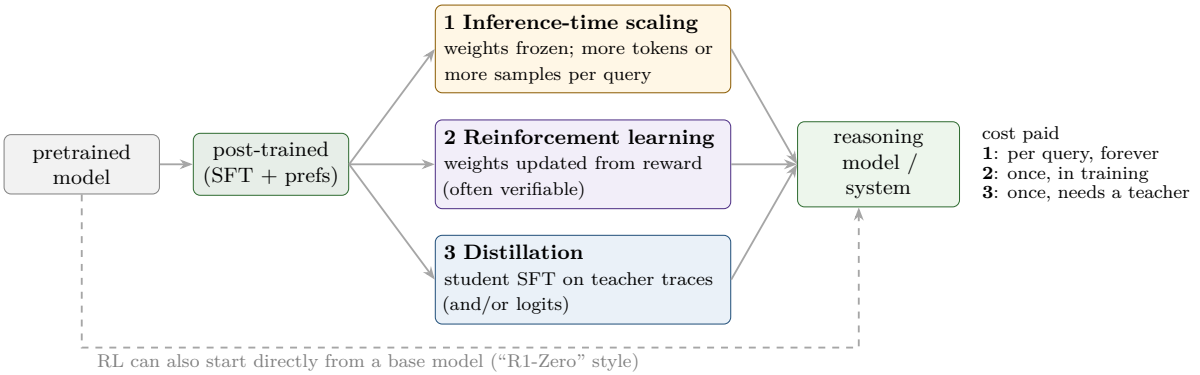


Figure 1.7: The three families of reasoning techniques and where their cost is paid. Inference-time scaling moves cost to serving; RL and distillation move it to training.

distributions. Its traces can look rigorous and still contain an invalid step, and two samples for the same prompt can disagree. Much of Part V (verifiers, self-consistency, scoring, RL with verifiable rewards) exists to manage this randomness rather than to eliminate it. Whether and how such traces resemble human reasoning remains an open question; we treat "reasoning" and "thinking" purely as operational terms.

1.8 Three levers for improving reasoning

RM 1.3

Reasoning models entered the mainstream with OpenAI's o1 (September 2024), presented as models that "spend more time thinking" before they answer, and became broadly reproducible with DeepSeek-R1 (January 2025), whose open weights and technical report described how to train such a model. The techniques fall into three groups, all applied *on top of* a model that has already gone through pretraining and usually post-training (Fig. 1.7):

1. **Inference-time compute scaling** (also called test-time scaling) improves answers *without changing weights*, by spending more computation per query: chain-of-thought prompting, sampling many answers and voting (self-consistency), scoring and refining answers, search. Chapters 15 and 16 implement these.

2. **Reinforcement learning** updates the weights so that the model is more likely to produce outputs that earn high reward. For reasoning, the reward is usually *verifiable*, for instance "the final answer matches the reference" or "the code passes the tests" (RL with verifiable rewards, RLVR). Chapters 17 to 19 implement GRPO and the systems around it.

3. **Distillation** transfers reasoning behaviour from a strong *teacher* model to a smaller *student*, typically by supervised fine-tuning the student on reasoning traces generated by the teacher. In classical deep learning, distillation also matches the teacher's output distribution (logits); in LLM practice, "distillation" often means SFT on teacher-generated text. Chapter 20 implements both variants.

Deeper Dive: RL for reasoning vs. RLHF

Both RLHF and RL for reasoning use the same machinery: sample responses from the current policy, score them, update the policy towards higher-scoring responses. They

differ in where the score comes from. In RLHF, the reward is a learned model of *human preferences*, which is subjective, can be gamed ("reward hacking"), and must be trained on human comparisons. In RLVR the reward is computed by a *program*: a math checker, a unit-test runner, a format validator. Programmatic rewards are cheap, objective and hard to fool on their own terms, but they exist only for tasks whose answers can be checked automatically. This is one reason reasoning models are strongest in mathematics and code.

Systems Lens: The three levers have different cost profiles

Inference-time scaling multiplies serving cost: self-consistency with 16 samples multiplies decode work by 16 (though the prompt's KV cache can be shared, Chapter 15), and every query pays it. RL moves the cost into training, but the training loop is dominated by *generation* (rollouts), so an RL trainer is really an inference engine and a trainer glued together (Chapter 19). Distillation is the cheapest at serving time, since a small student serves fast, but requires a teacher and a large offline generation job to produce traces. Choosing among them is a systems trade-off between one-time and per-query cost.

1.9 Pattern matching versus logical reasoning

RM 1.4

Pretraining teaches a model to reproduce the statistical regularities of its corpus. When asked "What is the capital of Germany?", a model answers "Berlin" not by deduction but because the association is overwhelmingly strong in its training data. We call this *pattern matching*: completing text according to learned correlations. Much text has premise–conclusion structure, so pattern matching can *look like* inference.

Logical reasoning, by contrast, derives conclusions from premises by rules, tracks which statements depend on which, and notices contradictions. Consider the prompt:

> Every metal is solid at room temperature. Mercury is a metal. Is mercury solid at room temperature?

In a *closed-world* reading (only the prompt's premises count), the valid conclusion is "yes". In an *open-world* reading that admits background knowledge, the conclusion contradicts the fact that mercury is a liquid at room temperature, so a careful reasoner should flag the inconsistency and reject or weaken the first premise ("*most* metals are solid…"). Figure 1.8 draws the dependency structure. A purely statistical model does not maintain such a structure explicitly; it predicts whichever continuation is most probable given its training distribution, which may or may not track the contradiction.

Rule-based expert systems that did maintain such structure were widely deployed in the 1980s and 1990s and survive in safety-critical niches. They lost ground because the rules must be hand-written and the systems are brittle outside their rule base. Neural models learn their "rules" from data and are flexible, at the price of guarantees.

1.10 Simulating reasoning without explicit rules

RM 1.5

Ask a strong conventional (non-reasoning) chat model the mercury question and it will very likely answer correctly, explaining that mercury is the exception. Does that mean it performed logical inference? Not necessarily. Text of the form "all X are Y, *except* …" and specific

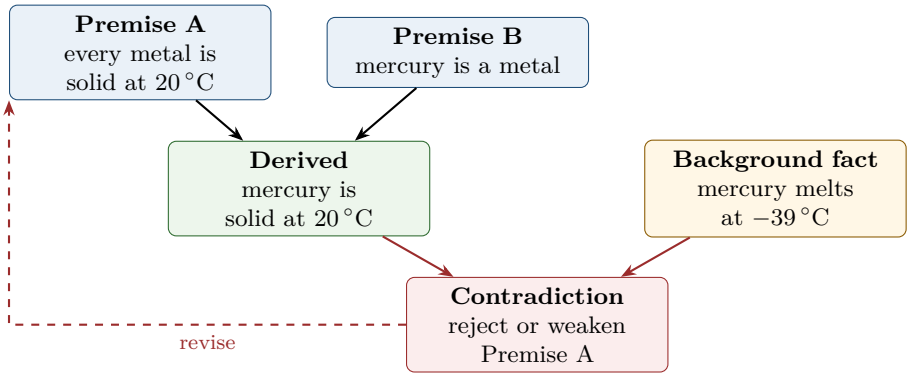


Figure 1.8: Explicit reasoning maintains a dependency graph: conclusions point back to the premises that produced them, so a conflict with background knowledge can be traced to a premise. LLMs carry no such graph; whatever consistency they show must come from patterns in data or from generated intermediate text.

facts about mercury are abundant in training data, so the correct continuation is also a high-probability continuation. The model *simulates* reasoning by pattern matching over a vast corpus of reasoning-shaped text. This works remarkably well at scale, and it degrades in predictable situations: when the problem is *novel* (no close analogue in training data), when it requires *many dependent steps* (each with some error rate, so errors compound), and when the correct answer runs *against* a strong surface association.

Seen this way, LLMs sit on a spectrum. Even before dedicated reasoning models, LLMs could produce intermediate steps when prompted; reasoning models are LLMs in which that behaviour has been made more reliable and more extensive through the levers of Section 1.8. None of them executes an internal rule engine. They improve the odds that the generated trace is correct and that the final answer follows from it.

Napkin Math: Why long chains are fragile

If each step of a derivation is independently correct with probability q, a k-step chain is fully correct with probability q^k. With $q = 0.98$, a 10-step chain succeeds $0.98^{10} \approx 82\%$ of the time and a 50-step chain only $0.98^{50} \approx 36\%$. Sampling n independent chains and taking a majority vote (Chapter 15) or training the model to raise q (Chapter 17) are the two complementary remedies, and both cost compute: the first at every query, the second once.

1.11 Why reasoning models, and why from scratch?

RM 1.6

Since early 2025, improving reasoning has been among the most active areas in LLM research, and the major model providers have converged on models that decide how long to think: short answers for simple requests, extended reasoning for hard ones. That design choice matters because reasoning is not free and not always useful (Table 1.2). For summarisation, translation, or factual lookup, extended thinking adds latency and cost and can even hurt, as models "overthink" simple problems. For mathematics, coding, planning and multi-step analysis it can be decisive.

	Reasoning helps	**Reasoning is wasteful**
Task type	multi-step math, proofs, code generation and debugging, puzzles, planning, tool selection in agents	summarisation, translation, retrieval-style QA, short classification, chit-chat
Cost	10–100× more output tokens; higher latency; more KV memory per request	n/a
Failure modes	verbose, can overthink easy inputs	errors on multi-step problems

Table 1.2: When to pay for reasoning. Production systems increasingly route requests or let the model choose its thinking budget.

Napkin Math: What a thinking budget costs in serving

Take Qwen3-8B in bf16 on one H100. Its KV cache needs $2 \times 36 \times 8 \times 128 \times 2 = 147{,}456$ bytes per token (Table 1.3). A direct 200-token answer to a 300-token prompt occupies about 70 MiB of KV cache; a 4,000-token reasoning trace pushes that to about 600 MiB, reducing the number of concurrent sequences that fit in the GPU's spare memory by roughly 8.5×. At batch size 1 the memory-bound decode floor is 16.4 GB/3.35 TB/s ≈ 4.9 ms per token, so the trace alone costs at least 20 seconds of wall-clock time versus 1 second for the direct answer. Reasoning models are a stress test for every serving optimisation in Part IV.

Why build all this from scratch, when production frameworks exist? Because the methods are young and changing quickly, and the only durable understanding comes from implementing them. Reasoning-model training recipes in particular mix ideas from inference (sampling, batching, KV caches), RL (policy gradients, baselines, KL control), and distributed systems (rollout workers, weight synchronisation). Implementing the pieces makes each of these visible, and makes the design space of production systems such as vLLM, SGLang, veRL and OpenRLHF legible.

1.12 The systems view: where the FLOPs and bytes go

This section is the first installment of a cost model we refine throughout the book. It needs only three facts, each derived properly later.

1. **Forward FLOPs per token** $\approx 2N$ for a dense model with N (non-embedding) parameters: every weight participates in one multiply and one add. Attention adds a term proportional to context length (Chapter 5).

2. **Training FLOPs** $\approx 6ND$ for D training tokens: the backward pass costs about twice the forward pass (Chapters 2 and 7).

3. **Memory**: weights take $N\times$(bytes per parameter); mixed-precision AdamW training state takes $\approx 16N$ bytes; the KV cache takes $2\,L\,H_{kv}\,d_h$ values per token of context (Chapter 11).

Table 1.3 applies these formulas to the model family used in Part V. Two features stand out. First, KV-cache bytes per token are small and nearly constant across sizes (thanks to grouped-query attention, Chapter 6), so at long contexts the cache, not the weights, limits how many requests fit in memory. Second, single-sequence decoding speed is set by *bandwidth*: the

Model	Params (unique)	GFLOP/token (fwd, 4k ctx)	KV/token (bf16)	KV @ 4k ctx (bf16, 1 seq)	Decode tok/s, B=1 (H100 bound)
Qwen3-0.6B	596M	2.1	112 KiB	448 MiB	2,016
Qwen3-1.7B	1.72B	4.4	112 KiB	448 MiB	857
Qwen3-4B	4.02B	10.5	144 KiB	576 MiB	387
Qwen3-8B	8.19B	17.6	144 KiB	576 MiB	197
Qwen3-14B	14.8B	31.3	160 KiB	640 MiB	111
Qwen3-32B	32.8B	72.6	256 KiB	1,024 MiB	50

Table 1.3: Napkin numbers for the dense Qwen3 family, produced by the calculator you build in Lab 1 (Section 1.16). "Decode tok/s" is the roofline upper bound for a single sequence: weight bytes plus KV bytes divided by 3.35 TB/s. Real engines reach a large fraction of it at batch 1; throughput beyond it requires batching.

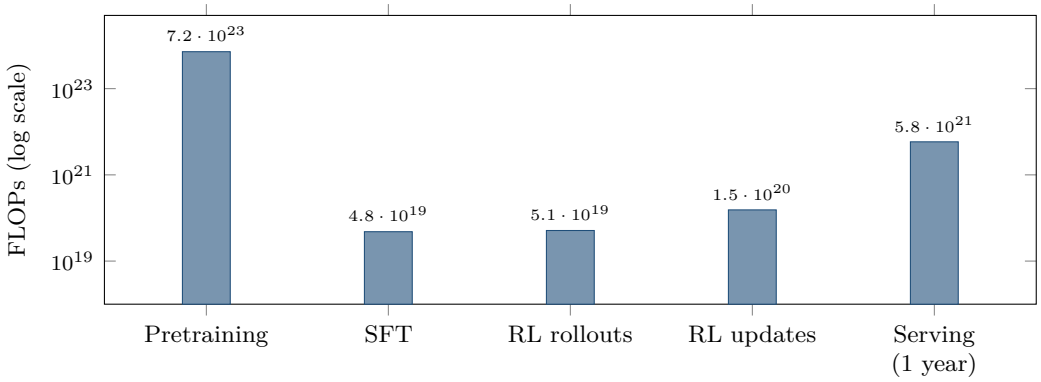


Figure 1.9: Order-of-magnitude compute across the life cycle of an 8B model under the stated assumptions (computed, not measured). Pretraining dominates training compute, but a year of moderate serving already exceeds every post-training stage. FLOPs also understate RL and serving costs, which are bandwidth-bound and run far below peak FLOP/s.

smallest model streams its 1.2 GB of weights about 2,000 times per second, and nothing short of batching, quantisation (Chapter 13) or speculative decoding can beat that bound.

Napkin Math: GPT-3 in GPU-days

$C = 6ND = 6 \times 175 \times 10^9 \times 300 \times 10^9 = 3.15 \times 10^{23}$ FLOPs. One H100 at 40% model FLOPs utilisation (MFU) sustains $0.4 \times 989 \times 10^{12} \approx 4 \times 10^{14}$ FLOP/s, so training takes $3.15 \times 10^{23} / (3.96 \times 10^{14}) \approx 8.0 \times 10^8$ GPU-seconds $\approx$ 221,000 GPU-hours: nine days on 1,024 H100s. The same job on A100s (312 TFLOP/s) needs about 700,000 GPU-hours. The original 2020 run used V100s, roughly an order of magnitude slower per chip than an H100 for this workload.

To see how cost is distributed across the life cycle, Fig. 1.9 applies the same formulas to a hypothetical 8B-parameter model with illustrative but realistic budgets: 15T pretraining tokens; 1B SFT tokens; an RL phase with 10^5 prompts $\times$ 8 rollouts $\times$ 4,000 tokens, counting generation ($2N$ per token) and training ($6N$ per token); and serving 10^9 tokens per day for a year.

Systems Lens: FLOPs are not time

Figure 1.9 counts FLOPs, but stages differ wildly in how efficiently they turn FLOPs into progress. Pretraining runs large dense matrix multiplications and can sustain 40–50%

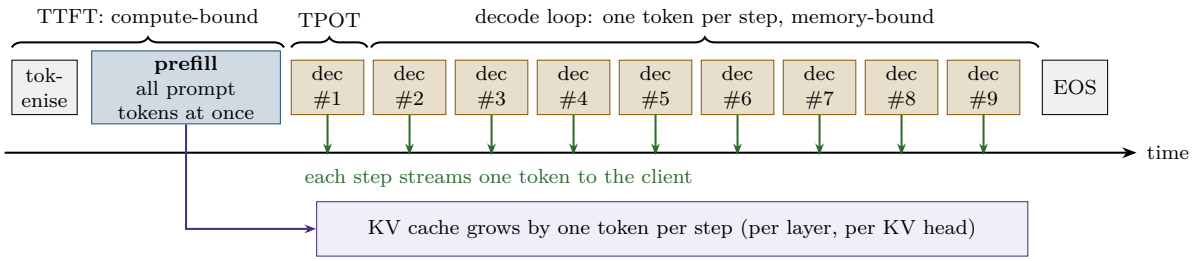


Figure 1.10: Life of one request. Prefill processes the whole prompt in parallel and fills the KV cache; decode then emits one token per forward pass, reading all weights and the growing cache each time. Serving systems (Chapter 12) interleave the prefill and decode steps of many requests.

MFU on well-tuned clusters. Decoding (serving, and RL rollouts) is dominated by matrix–vector products whose arithmetic intensity is near 1 FLOP/byte at batch 1, versus the ∼295 FLOP/byte an H100 needs to be compute-bound (Chapter 2); without large batches, utilisation can fall to a few percent. That is why RL rollouts often take the majority of wall-clock time in reasoning training despite being a small FLOP fraction, and why serving engines obsess over batching.

1.12.1 Anatomy of a single request

It is worth following one chat request through a serving system now, because the vocabulary returns in every later part of the book (Fig. 1.10). The text is *tokenised* on the CPU. The prompt's tokens are then processed in a single parallel forward pass, the *prefill*, which computes and stores the keys and values of every prompt token in the *KV cache* and produces the logits for the first output token. Prefill is a sequence of large matrix–matrix products and is compute-bound; its duration sets the *time to first token* (TTFT). After that, the engine enters the *decode* loop: each iteration feeds only the newest token, attends to the cached keys and values, samples one token, appends its own key and value to the cache, and streams the token back to the client after detokenisation. Decode iterations are matrix–vector products, memory-bound, and their duration sets the *time per output token* (TPOT), also called inter-token latency. A reasoning model simply has a very long decode phase, much of it inside `<think>` tags.

Table 1.4 condenses the resulting picture. It is the most important table in this chapter: nearly every optimisation in the book is an attack on one cell.

Deeper Dive: Scaling laws in one paragraph

Empirically, pretraining loss falls as a smooth power law in parameters, data and compute (Kaplan et al., 2020). Hoffmann et al. (2022, "Chinchilla") refit these laws and found that, for a fixed compute budget $C \approx 6ND$, loss is minimised when parameters and tokens grow in roughly equal proportion, at about 20 tokens per parameter. A 70B model is then "compute-optimal" at about 1.4T tokens. Production models deliberately overshoot (D/N in the hundreds or thousands) because a smaller model trained longer is cheaper to *serve*, and lifetime serving cost dominates (Fig. 1.9). Chapter 7 derives the compute-optimal allocation.

Workload	Dominant operation	Usual bottleneck	Main remedies (chapter)
Pretraining	large GEMMs, fwd + bwd	compute; then interconnect	mixed precision, kernel fusion, FlashAttention, parallelism (7, 8)
Fine-tuning	same, smaller scale	memory capacity (optimizer state)	LoRA and QLoRA, ZeRO and FSDP, checkpointing (10, 8)
Prefill	GEMMs over prompt	compute	FlashAttention, chunked prefill, prefix caching (4, 12)
Decode	GEMVs + attention over cache	memory bandwidth; KV capacity	batching, paged KV, GQA, quantisation, speculative decoding (12, 13)
RL rollouts	decode at scale	bandwidth; long-tail lengths	inference engines inside the trainer, async RL (19)
Evaluation	batched decode + checking	bandwidth; verifier CPU time	batching, caching, sandboxed parallel verifiers (14)

Table 1.4: What limits what. Numbers in the last column are chapter references.

1.12.2 The hardware zoo

Most LLM work today runs on NVIDIA GPUs (A100, H100/H200, B200), whose programming model, CUDA, is the one we use in Appendix A. Google's TPUs organise compute as large systolic matrix units fed by high-bandwidth memory and connect chips in a torus with dedicated inter-chip links. A third family of *dataflow* or near-memory architectures, including Tenstorrent's grids of RISC-V-controlled Tensix cores, Cerebras's wafer-scale engine and Groq's deterministic LPU, trades the GPU's cache hierarchy for explicitly scheduled on-chip SRAM and data movement. The vocabulary of this book (FLOPs, bytes, arithmetic intensity, bandwidth, interconnect, overlap) applies unchanged to all of them; only the constants and the knobs differ. Figure 1.11 sketches the common abstraction.

1.13 A roadmap for this book

LLM 1.7; RM 1.7

The two books this text is built from follow complementary arcs. The first builds a GPT-style model in three stages: (1) data preparation and the attention mechanism, then the full

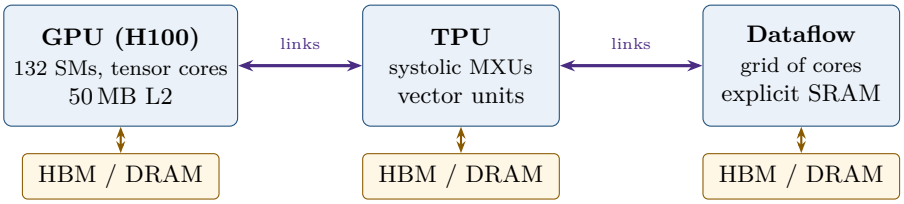


Figure 1.11: Three accelerator families, one abstraction. Performance analysis in this book reduces any of them to compute throughput, memory capacity and bandwidth, and interconnect bandwidth and latency.

architecture; (2) pretraining, evaluation and loading pretrained weights; (3) fine-tuning for classification and for instruction following. The second starts from an existing pretrained (and post-trained) model and adds reasoning: it builds a text-generation pipeline, an evaluation harness with a math verifier, then inference-time techniques (chain of thought, sampling, self-consistency, self-refinement), then training techniques (RL with GRPO, and distillation). We merge the two arcs and wrap them in a systems layer (Fig. 1.12).

How to read this book. The chapters are written to be read in order, but there are two shortcuts. A reader mainly interested in reasoning can read Chapters 1 and 2, skim Chapters 5 and 6, and jump to Part IV. A reader mainly interested in systems can read the Systems Lens boxes, the napkin-math boxes, and Chapters 2, 8, 12, 13 and 19 plus Appendix A. Margin tags such as "LLM 3.5" or "RM 6.8" indicate which section of the source books a section covers.

Labs. Each chapter closes with a lab in `code/labs/`. A lab has a `starter.py` with `TODO`s, a reference `solution.py`, and tests that run on a laptop CPU in seconds using tiny model configurations; most labs also ship a `run.py` or `bench.py` meant for a real GPU and real checkpoints. Tests are graded with `pytest` (Appendix B). The shared library `llmsys` contains the reusable pieces (model definitions, KV caches, sampling, training loops) built up over the first chapters.

1.14 Summary

- An LLM is a deep neural network, almost always a decoder-only transformer, trained to predict the next token on a massive text corpus; "large" refers to both parameter count and data size.
- Building an LLM proceeds through pretraining (self-supervised, by far the most compute), then post-training: classification or instruction fine-tuning and preference tuning. A chat product adds orchestration on top.
- The transformer's self-attention lets each position weigh the relevance of earlier positions. Decoder-only models generate autoregressively, one forward pass per token, which makes decoding inherently sequential and memory-bound.
- Reasoning models generate intermediate steps before the answer. They are improved by inference-time scaling (no weight changes, per-query cost), reinforcement learning (often

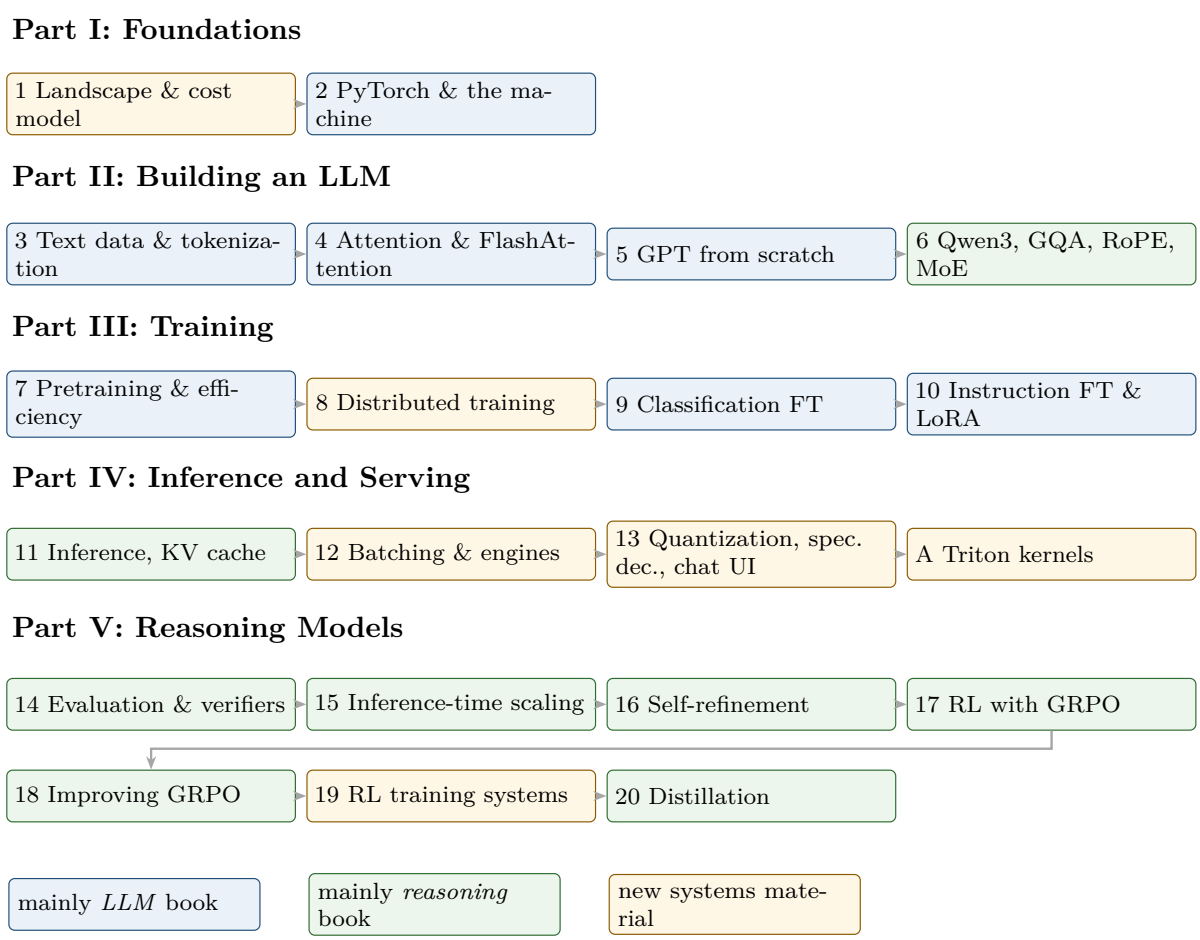


Figure 1.12: How the chapters fit together. Blue chapters follow the LLM-from-scratch arc, green chapters the reasoning-model arc, amber chapters are systems material added in this book. Every chapter ends with a lab; every source-book section is tagged in the margin where it is covered.

with verifiable rewards), and distillation from stronger teachers.

- LLMs simulate reasoning through learned patterns rather than explicit rules; this is powerful at scale but fragile for novel, long, or counter-intuitive problems.
- A three-line cost model ($2N$ FLOPs/token forward, $6ND$ training FLOPs, $16N$ bytes of training state plus a per-token KV cache) is enough to estimate most budgets to within a small factor.

1.15 Exercises

Exercise 1.1 (*Chain rule*). Show that any joint distribution over length-T token sequences can be written as (1.1). Then explain why a model trained on (1.2) with a causal mask produces T training signals from one sequence of length T, whereas a masked-language model that masks 15% of tokens produces only about $0.15\,T$.

Exercise 1.2 (*Perplexity intuition*). A model achieves an average cross-entropy of 2.0 nats per token on held-out text. What is its perplexity? Interpret the number as an "effective branching factor". What loss would a model that predicts uniformly over a 50,257-token vocabulary have?

Exercise 1.3 (*Weighted mixtures*). Using Table 1.1, compute how many epochs GPT-3 made over Wikipedia and over Common Crawl during its 300B-token run. Why might a practitioner deliberately repeat small, high-quality sources?

Exercise 1.4 (*Your own compute budget*). You have 64 H100s for 30 days and expect 35% MFU. Using $C = 6ND$ and the Chinchilla heuristic $D \approx 20N$, what model size and token count are compute-optimal? If you instead plan to serve the model heavily, argue whether to choose a smaller N and larger D.

Exercise 1.5 (*Memory-bound decoding*). Derive the batch-1 decode bound in Table 1.3 for Qwen3-4B at a 4k context by hand. Then compute the bound for the same model with int4 weights (0.5 bytes/parameter) and a bf16 KV cache. Which term now dominates the bytes moved per step?

Exercise 1.6 (*Critical batch size*). For a decode step of a dense model with N parameters stored in b bytes each, ignoring the KV cache, show that the step becomes compute-bound when the batch exceeds approximately $B^\star = b \cdot \mathrm{FLOP/s}_{\mathrm{peak}}/(2 \cdot \mathrm{bytes/s}_{\mathrm{peak}})$. Evaluate $B^\star$ for an H100 in bf16 and compare with the lab's numerical result. Why does a long context increase the effective $B^\star$?

Exercise 1.7 (*Reasoning tokens and concurrency*). A server has 40 GB of HBM free for KV cache and serves Qwen3-8B. Requests have 500-token prompts. How many concurrent requests fit if responses average 300 tokens? If they average 6,000 tokens of thinking? Express the answer as the throughput penalty of turning on thinking, assuming decode is memory-bound and throughput scales with batch size.

Exercise 1.8 (*Error compounding*). Under the independent-step model of Section 1.10, suppose a problem needs 30 steps and each is right with probability 0.97. Compute the single-chain success rate. Assuming chains are independent and wrong answers are spread over many distinct values, estimate how many samples are needed for a majority vote to be right with probability above 0.9. Discuss why the independence assumption is optimistic.

Exercise 1.9 (*Pattern matching probe*). Design three prompts that separate pattern matching from logical reasoning: one closed-world syllogism with a false premise, one novel puzzle unlikely to appear in training data, and one where the answer contradicts a strong surface association. Predict how a base model, an instruction-tuned model and a reasoning model would differ, and how you would score them automatically.

Exercise 1.10 (*Lever selection*). A company must improve accuracy on an internal code-review task. It has a mid-sized open model, a GPU budget for one week of training, and strict latency targets in production. Rank inference-time scaling, RL, and distillation for this setting and justify the ranking using the cost profiles of Section 1.8.

1.16 Lab 1: A napkin-math calculator for LLMs

Lab 1: A napkin-math calculator for LLMs

Code: `code/labs/lab01_napkin`

Goals

Turn the cost model of Section 1.12 into code and verify it against reality: exact parameter counts of real `nn.Module`s, FLOP counts from PyTorch's FLOP counter, and byte counts of real KV-cache buffers. You will reuse this calculator in nearly every later chapter.

Setup

```
cd code && pip install -r requirements.txt
pytest -q labs/lab01_napkin                    # tests your starter.py
LAB_IMPL=solution pytest -q labs/lab01_napkin  # reference solution
```

The calculator works directly on the configuration dictionaries in `llmsys.gpt` and `llmsys.qwen3`, so every formula can be checked against a model built from the same dictionary.

Tasks

1. `gpt_param_count(cfg)`: exact parameter count of `GPTModel(cfg)`, including token and position embeddings, Q/K/V (with optional biases), output projection, MLP, both LayerNorms per block, the final norm, and the separate output head. Check that GPT-2 small without weight tying has 163,009,536 parameters, and that removing the duplicated head recovers the familiar "124M".
2. `qwen3_param_count(cfg)`: the same for `Qwen3Model`: grouped K/V projections, QK-norm, three SwiGLU matrices, RMSNorms, and tied versus untied embeddings.
3. `forward_flops_per_token(cfg, context)`: linear-layer FLOPs plus attention FLOPs. The test multiplies your per-token value by the sequence length and compares it with `torch.utils.flop_counter.FlopCounterMode`, which must match *exactly*.
4. `training_flops`, `training_days`, `training_state_bytes`: the $6ND$ rule, wall-clock

days at a given MFU, and the 16-bytes-per-parameter rule.

5. `kv_cache_bytes`: must equal `StaticKVCache(...).nbytes()` for the same shapes.
6. `decode_step_roofline`, `critical_batch`, `max_batch_for_kv`: a roofline lower bound for one decode step, the smallest batch that makes it compute-bound, and the largest batch whose KV cache fits.

Measurements

Run `python labs/lab01_napkin/report.py` to regenerate Table 1.3 for the GPT-2 and Qwen3 families. Then use `-hw A100-80GB-SXM` and `-hw RTX-4090` and note how the critical batch size tracks the ratio of peak FLOP/s to bandwidth (the *ridge point*, Chapter 2) rather than the model. Expect critical batches of a few hundred on datacentre GPUs.

Deliverables

All tests passing; a one-page table for five model sizes on two GPUs; a paragraph explaining which resource (compute, HBM capacity, HBM bandwidth) limits batch-1 decoding, large-batch decoding, and pretraining.

Stretch goals

Add mixture-of-experts support (active versus total parameters, Chapter 6); add an int4/fp8 weight option and recompute the decode bound; add a `tensor_parallel` argument that divides weight and KV bytes per GPU and adds an all-reduce term per layer (Chapter 8).

Chapter 2

PyTorch and the Machine Underneath

Every model in this book is a PyTorch program, and every PyTorch program is a stream of *kernels* launched onto an accelerator. This chapter covers both levels. The first half is a working introduction to PyTorch: tensors, automatic differentiation, modules, data loaders, the training loop, checkpoints, and running on one or several GPUs. A reader fluent in PyTorch can skim it, but should read the Systems Lens boxes, which connect each API to what happens on the hardware. The second half opens the box: how a GPU is organised, why most kernels are limited by memory bandwidth rather than arithmetic, how the *roofline model* predicts which, how PyTorch's asynchronous execution model hides (or fails to hide) latency, and how to measure all of this honestly. These are the tools we use to reason about every optimisation in later chapters.

2.1 What is PyTorch?

LLM A.1

PyTorch is an open-source Python library for deep learning, originally released by Meta AI in 2016 and now governed by the PyTorch Foundation. It has been the most widely used framework in machine-learning research for years, and it is the substrate of nearly every open LLM codebase, from Hugging Face Transformers to vLLM, SGLang, Megatron-LM and veRL. Its popularity comes from a design that feels like ordinary Python: you write imperative code, run it line by line, inspect any value, and get gradients for free.

2.1.1 The three core components

LLM A.1.1

It is helpful to see PyTorch as three libraries in one (Fig. 2.2):

1. A **tensor library**: an array library in the spirit of NumPy, whose arrays can live on CPUs, NVIDIA and AMD GPUs, or Apple silicon, with a vast set of operations implemented as optimised kernels.

2. An **automatic differentiation engine** (*autograd*): it records the operations applied to tensors and computes gradients by traversing that record backwards.

3. A **deep-learning library**: modules (layers), loss functions, optimisers, data loading and

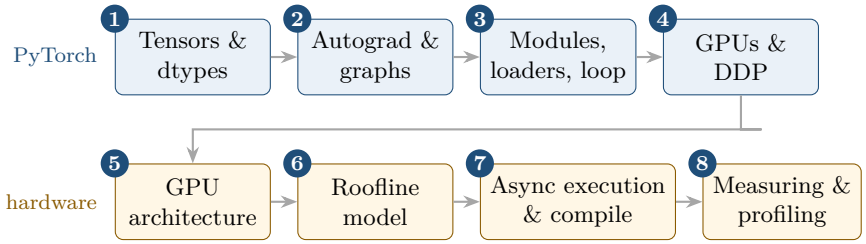


Figure 2.1: Roadmap of Chapter 2: four stages of PyTorch fundamentals followed by four stages that explain what those programs do on the hardware.

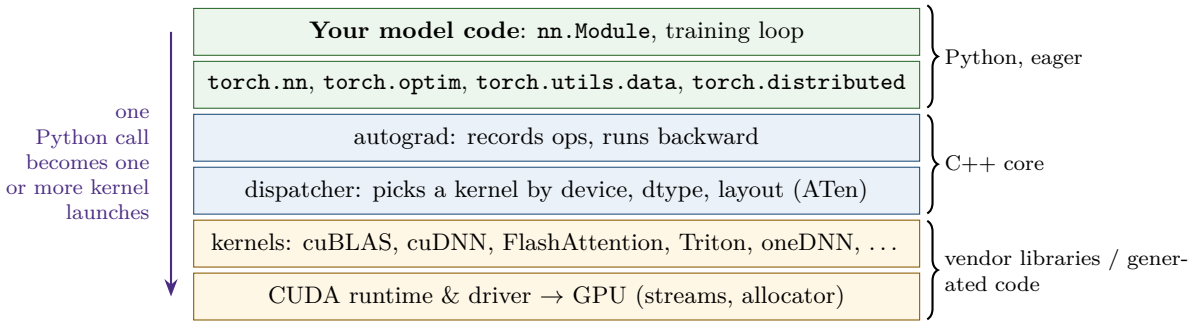


Figure 2.2: The PyTorch stack. A single Python-level operation passes through autograd and the dispatcher before a device kernel is enqueued. Each layer adds a few microseconds of overhead, which is why tiny operations are slow in eager mode and why kernel fusion and compilation matter (Section 2.12).

distributed training utilities built on top of the first two.

2.1.2 Deep learning in one paragraph

LLM A.1.2

Artificial intelligence aims at systems that perform tasks associated with human intelligence; machine learning is the subfield that achieves this by learning from data; deep learning is the part of machine learning that uses neural networks with many layers, which learn both the features and the decision rule from raw inputs. Training a deep network follows a fixed recipe: a model with parameters θ maps inputs to predictions; a loss function measures how wrong the predictions are on labelled training data; gradients of the loss with respect to θ are computed by backpropagation; and an optimiser nudges θ against the gradient. After training, the model is used for *inference* on new inputs. Everything in this book, including LLM pretraining and RL, is an instance of this recipe with a particular model, loss and data stream.

2.1.3 Installing PyTorch

LLM A.1.3

PyTorch is installed with `pip` or `conda`. CPU-only wheels work everywhere; GPU builds must match the CUDA (or ROCm) version supported by the installed driver, and the selector on `pytorch.org` prints the right command. Two lines confirm what is available:

Listing 2.1: Checking the installation and available accelerators.

```
import torch
print(torch.__version__)
print("CUDA:", torch.cuda.is_available(),
      "| MPS:", torch.backends.mps.is_available())    # Apple silicon GPU
if torch.cuda.is_available():
    p = torch.cuda.get_device_properties(0)
```

```
    print(p.name, f"{p.total_memory/2**30:.0f} GiB,", p.multi_processor_count, "SMs"
    )
```

Apple-silicon Macs expose their GPU through the `mps` backend, which is convenient for the small experiments in this book but lacks some kernels and all of the CUDA-specific tooling; we write code that selects a device with `llmsys.utils.get_device()` and runs on whatever is present. Appendix B gives a complete environment recipe, including cloud GPUs.

2.2 Tensors

LLM A.2

2.2.1 Scalars, vectors, matrices and beyond

LLM A.2.1

A tensor is an n-dimensional array with a single element type. Its *rank* (or order) is the number of dimensions: a scalar has rank 0, a vector rank 1, a matrix rank 2, and LLM activations are usually rank 3 with shape `(batch, sequence, features)` or rank 4 inside attention, `(batch, heads, sequence, head_dim)`. Creating tensors is straightforward:

Listing 2.2: Creating tensors of different ranks.

```
s = torch.tensor(3)                        # rank 0, shape ()
v = torch.tensor([1.0, 2.0, 3.0])          # rank 1, shape (3,)
m = torch.tensor([[1, 2], [3, 4]])         # rank 2, shape (2, 2), int64
x = torch.zeros(8, 1024, 768)              # rank 3: (batch, seq, d_model)
print(x.ndim, x.shape, x.dtype, x.device, x.numel())
```

2.2.2 Tensor data types

LLM A.2.2

PyTorch infers the data type (*dtype*) from the input: Python integers become `torch.int64`, Python floats become `torch.float32`. Token ids are integers (`int64` by default, since embedding lookups expect it); everything that is learned or computed is floating point. Converting is explicit: `x.to(torch.bfloat16)` or `x.float()`. The default `float32` is a sensible balance of precision and cost on CPUs, but on modern accelerators most LLM arithmetic runs in 16-bit or even 8-bit formats, which we examine in Section 2.2.4.

2.2.3 Common tensor operations

LLM A.2.3

A handful of operations recur throughout the book:

Listing 2.3: Operations used constantly in LLM code.

```
a = torch.arange(6.0).reshape(2, 3)      # [[0,1,2],[3,4,5]]
a.view(3, 2)                             # reinterpret same memory (needs contiguity)
a.reshape(3, 2)                          # view if possible, else copy
a.T                                      # transpose of a 2-D tensor (a view)
a.transpose(0, 1)                        # swap any two dims (a view)
a @ a.T                                  # matrix multiplication, (2,3)@(3,2) ->
    (2,2)
q = torch.randn(4, 12, 128, 64)          # (batch, heads, seq, head_dim)
scores = q @ q.transpose(-2, -1)         # batched matmul -> (4, 12, 128, 128)
x = torch.randn(2, 5, 8)
x.mean(dim=-1, keepdim=True)             # reductions along a dimension
torch.softmax(x, dim=-1)                 # normalise the last dimension
torch.cat([x, x], dim=1)                 # (2, 10, 8)
x[:, -1, :]                              # last position of every sequence, (2, 8)
```

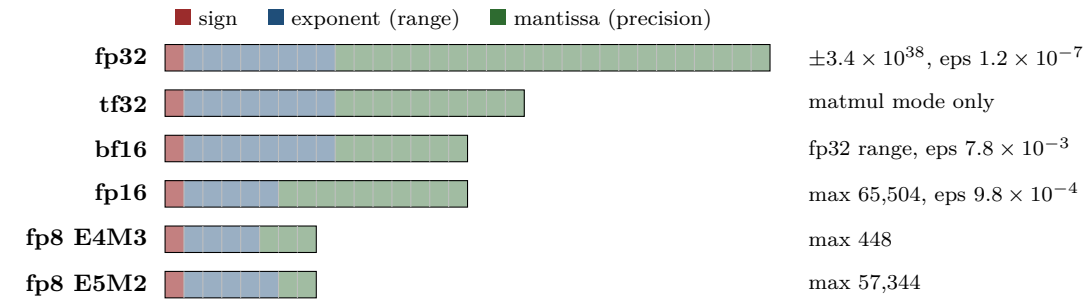


Figure 2.3: Bit layouts of the floating-point formats used for LLMs. bf16 keeps fp32's eight exponent bits and sacrifices mantissa; fp16 does the opposite. Fewer bits mean fewer bytes to move and more tensor-core throughput: on an H100, dense bf16 matmul peaks at about 989 TFLOP/s and fp8 at about twice that.

Systems Lens: Shapes, strides and the cost of a "free" transpose

A tensor is a pointer into a block of memory (its *storage*) plus a shape and a *stride* per dimension: the number of elements to skip to move one step along that dimension. A contiguous 2×3 tensor has strides $(3, 1)$; its transpose is the *same storage* with shape $(3, 2)$ and strides $(1, 3)$. That is why `.T`, `.transpose`, slicing and `.view` are free: they allocate nothing and launch no kernel. The bill arrives later. A kernel that reads a non-contiguous tensor makes strided, uncoalesced memory accesses, and some operations (`.view` after a transpose, many custom kernels) refuse non-contiguous inputs, forcing a `.contiguous()` copy. In attention code, the `transpose(1, 2)` between `(B,T,H,D)` and `(B,H,T,D)` layouts is the classic source of hidden copies. Lab 2 asks you to detect views versus copies by comparing storage pointers.

2.2.4 Number formats

Floating-point formats trade *range* (exponent bits) against *precision* (mantissa bits) (Fig. 2.3). The formats that matter for LLMs are:

- **fp32** (1 sign, 8 exponent, 23 mantissa bits): the reference format; master weights and optimiser states are usually kept in fp32.
- **tf32**: not a storage format but a tensor-core *compute mode* that reads fp32 inputs and multiplies them with a 10-bit mantissa, enabled in PyTorch by setting `torch.backends.cuda.matmul.allow_tf32` to `True`.
- **fp16** (1/5/10): good precision but a narrow range (maximum 65,504), so gradients can underflow or overflow; training in fp16 needs *loss scaling* (Chapter 7).
- **bf16** (1/8/7): the same range as fp32 with much less precision (machine epsilon 2^{-7}). Its range makes it the default for LLM training and inference.
- **fp8** in two flavours, E4M3 (max 448, more precision) and E5M2 (max 57,344, more range), supported by Hopper and later GPUs for matmuls with per-tensor or per-block scaling factors (Chapter 13).

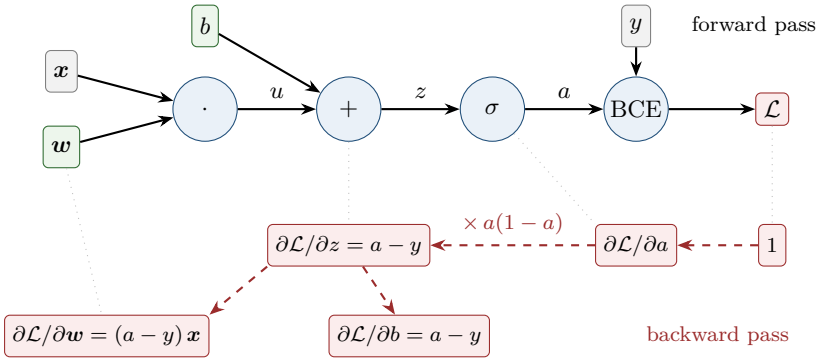


Figure 2.4: Computation graph for logistic regression. Solid arrows: forward pass (tensors flow left to right). Dashed arrows: the backward pass multiplies local derivatives from the loss back to the leaves that require gradients ($\boldsymbol{w}$ and b, green). For sigmoid followed by BCE, the product simplifies to $\partial\mathcal{L}/\partial z = a - y$.

Napkin Math: What a dtype buys you

Halving the bytes per element halves memory capacity and traffic for every tensor stored in that format, and on tensor cores roughly doubles peak matmul throughput (H100 SXM, dense: tf32 ≈495, bf16 ≈989, fp8 ≈1,979 TFLOP/s). A 7B model's weights take 28 GB in fp32, 14 GB in bf16, 7 GB in fp8, and 3.5 GB in int4. Precision is the cost: bf16's relative spacing of $2^{-7} \approx 0.8\%$ means that adding a small update to a large weight can round to no change at all. This is why optimisers keep fp32 master weights even when the forward pass runs in bf16 (Chapter 7).

2.3 Models as computation graphs

LLM A.3

A neural network is a composition of operations, and it is useful to draw it as a directed acyclic graph whose nodes are operations and whose edges carry tensors: the *computation graph*. Take logistic regression on one example, the smallest model with every ingredient of a deep network:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b, \qquad a = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \mathcal{L} = -\big[y \log a + (1 - y) \log(1 - a)\big]. \tag{2.1}$$

Listing 2.4: Logistic regression forward pass; autograd records the graph as it runs.

```
import torch.nn.functional as F
x = torch.tensor([1.5, -0.5])        # input features
y = torch.tensor(1.0)                # true label
w = torch.tensor([0.4, 0.9], requires_grad=True)
b = torch.tensor(0.1, requires_grad=True)
z = x @ w + b                        # weighted sum
a = torch.sigmoid(z)                 # predicted probability
loss = F.binary_cross_entropy(a, y)
print(loss.grad_fn)                  # <BinaryCrossEntropyBackward0 ...>
```

PyTorch builds this graph *dynamically* ("define-by-run"): each operation on a tensor with `requires_grad=True` appends a node, recorded in the result's `grad_fn`, that knows how to compute its local derivative. Figure 2.4 shows the graph for Listing 2.4 together with the backward pass that the next section runs over it.

2.4 Automatic differentiation

LLM A.4

Training needs the gradient of the loss with respect to every parameter. By the chain rule, the gradient of $\mathcal{L}$ with respect to $\boldsymbol{w}$ is a product of local derivatives along the path from $\boldsymbol{w}$ to $\mathcal{L}$. *Reverse-mode* automatic differentiation evaluates that product from the loss backwards, which costs about the same as one more forward pass no matter how many parameters there are. This is the right mode for a scalar loss and millions of parameters.

PyTorch exposes reverse mode in two ways. `torch.autograd.grad` returns gradients of an output with respect to chosen inputs; `.backward()` accumulates gradients into the `.grad` field of every leaf that requires them:

Listing 2.5: Two ways to get gradients from the graph of Listing 2.4.

```
from torch.autograd import grad
gw = grad(loss, w, retain_graph=True)     # keep the graph for a second call
gb = grad(loss, b, retain_graph=True)
loss.backward()                           # fills w.grad and b.grad (and frees the
    graph)
print(w.grad, b.grad)                     # equal to gw[0], gb[0]
print((a - y).detach() * x)               # the analytic gradient for w
```

Three details cause most autograd bugs. First, gradients *accumulate*: `.backward()` adds to `.grad`, so training loops must zero gradients every step (or rely on accumulation deliberately for gradient accumulation, Chapter 7). Second, the graph is freed after one backward pass unless `retain_graph=True`. Third, operations under `torch.no_grad()` or on `.detach()`ed tensors are not recorded, which is what we want during evaluation and generation.

Systems Lens: Why backward costs twice the forward, and where activation memory comes from

For a linear layer $\mathbf{Y} = \mathbf{X}\mathbf{W}$ with $\mathbf{X} \in \mathbb{R}^{m\times k}$ and $\mathbf{W} \in \mathbb{R}^{k\times n}$, the forward pass costs $2mkn$ FLOPs. Backward needs two products of the same size: $\partial\mathcal{L}/\partial\mathbf{X} = \mathbf{G}\mathbf{W}^\top$ and $\partial\mathcal{L}/\partial\mathbf{W} = \mathbf{X}^\top\mathbf{G}$, where $\mathbf{G} = \partial\mathcal{L}/\partial\mathbf{Y}$, for $4mkn$ FLOPs. Hence forward plus backward $\approx 3\times$ forward, and with $2N$ FLOPs per token forward we get the $6N$ FLOPs per training token used throughout the book. The second product needs $\mathbf{X}$, the layer's *input*, so autograd must keep every such input alive from the forward pass until its backward step: these *saved activations* are why training memory grows with batch size $\times$ sequence length $\times$ depth, and why activation checkpointing (recomputing them during backward, Chapter 7) trades FLOPs for memory.

2.5 Implementing multilayer neural networks

LLM A.5

Real models are built by subclassing `torch.nn.Module`. A module registers its parameters (as `nn.Parameter` attributes or submodules) in `__init__` and defines the computation in `forward`. Calling the module runs `forward` and, via hooks, integrates with autograd, device movement, `state_dicts` and more.

Listing 2.6: A multilayer perceptron as an `nn.Module`.

```
import torch.nn as nn

class MLP(nn.Module):
```

```
    def __init__(self, in_dim, out_dim, hidden=(30, 20)):
        super().__init__()
        layers, prev = [], in_dim
        for h in hidden:
            layers += [nn.Linear(prev, h), nn.ReLU()]
            prev = h
        layers.append(nn.Linear(prev, out_dim))   # outputs logits
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

torch.manual_seed(123)                    # reproducible random initialisation
model = MLP(50, 3)
print(sum(p.numel() for p in model.parameters() if p.requires_grad))   # 2213
```

The count is $(50 \cdot 30 + 30) + (30 \cdot 20 + 20) + (20 \cdot 3 + 3) = 2{,}213$: each `nn.Linear(in, out)` has an out $\times$ in weight matrix and an `out`-dimensional bias. The weights are initialised randomly (by default from a scaled uniform distribution), so fixing the seed makes runs reproducible.

Two habits are worth forming early. First, the network returns *logits*, unnormalised scores, not probabilities: PyTorch's `cross_entropy` applies a numerically stable log-softmax internally, and applying softmax yourself first is both wasteful and less accurate. Call `torch.softmax(logits, dim=-1)` only when you actually need probabilities, for instance for sampling. Second, wrap inference in `torch.no_grad()` (or `torch.inference_mode()`), so that no graph is recorded and no activations are kept:

Listing 2.7: Inference without building a graph.

```
X = torch.rand(4, 50)
with torch.no_grad():
    logits = model(X)
    probs = torch.softmax(logits, dim=-1)   # rows sum to 1
```

2.6 Setting up efficient data loaders

LLM A.6

PyTorch separates *what* the data are from *how* they are batched. A `Dataset` implements `__len__` and `__getitem__(i)`, returning one example. A `DataLoader` draws indices (shuffled or not), fetches examples (optionally in worker processes), collates them into batch tensors, and yields them:

Listing 2.8: A map-style dataset and its loader.

```
from torch.utils.data import Dataset, DataLoader

class ToyDataset(Dataset):
    def __init__(self, X, y):
        self.features, self.labels = X, y
    def __getitem__(self, i):
        return self.features[i], self.labels[i]
    def __len__(self):
        return self.labels.shape[0]

```

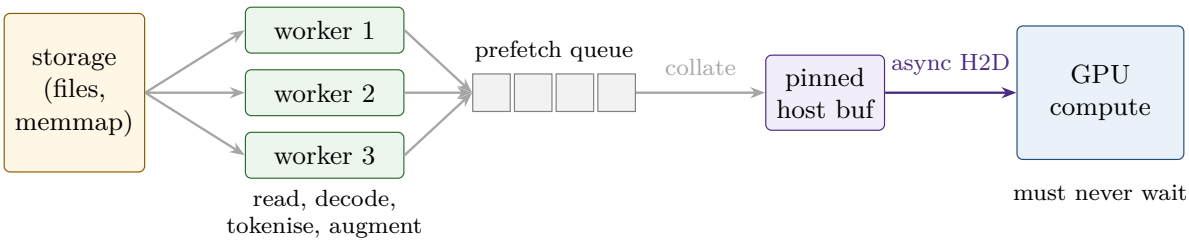


Figure 2.5: A data pipeline. Worker processes produce batches ahead of time into a bounded queue; with `pin_memory=True` batches land in page-locked host memory so the host-to-device copy can be asynchronous and overlap with compute.

```
train_loader = DataLoader(ToyDataset(X_train, y_train), batch_size=2,
                          shuffle=True,      # new order every epoch
                          drop_last=True,    # avoid a tiny final batch
                          num_workers=0)     # 0 = load in the main process
for epoch in range(2):
    for xb, yb in train_loader:
        ...
```

`shuffle=True` matters for SGD, which assumes batches are roughly independent samples; `drop_last=True` avoids a final, much smaller batch whose noisy gradient can destabilise training (and which would also break fixed-shape compilation). `num_workers` controls parallel loading.

Systems Lens: Feeding the accelerator

A DataLoader with `num_workers=k` forks k processes (side-stepping Python's global interpreter lock), each producing whole batches, and keeps `prefetch_factor` batches per worker in flight (Fig. 2.5). The goal is simple: the GPU should never wait for input. For LLM pretraining the per-batch CPU work is tiny if, and only if, tokenisation happened offline: a batch is then just a slice of a memory-mapped array of token ids (`llmsys.data.TokenMemmapDataset`, Chapter 3). Two further flags matter on GPUs: `pin_memory=True` allocates batches in page-locked memory, which the DMA engine can copy from directly, and `x.to(device, non_blocking=True)` then makes the copy asynchronous so it overlaps with the previous step's compute. In Jupyter notebooks, multi-process loading is unreliable; use `num_workers=0` there and in tiny experiments, where process start-up dominates.

2.7 A typical training loop

LLM A.7

With a model, a loss and a loader, the training loop has the same skeleton in every chapter of this book (Fig. 2.6):

Listing 2.9: The canonical training loop.

```
model = MLP(in_dim=2, out_dim=2)
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
for epoch in range(3):
    model.train()                           # enable dropout etc.
    for xb, yb in train_loader:
        logits = model(xb)                  # 1. forward
        loss = F.cross_entropy(logits, yb) # 2. loss
```

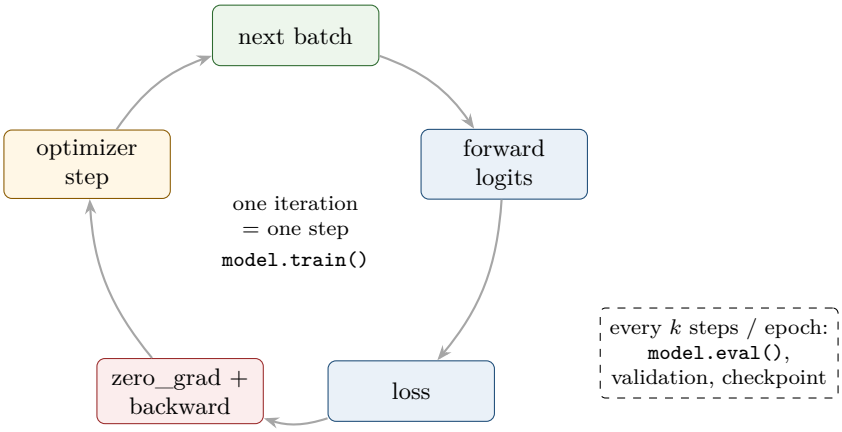


Figure 2.6: The training loop. Each iteration loads a batch, runs forward, computes the loss, clears stale gradients, runs backward, and steps the optimiser. Evaluation and checkpointing happen periodically with `model.eval()` and `torch.no_grad()`.

```
        optimizer.zero_grad()                # 3. clear old gradients
        loss.backward()                      # 4. backward: fill .grad
        optimizer.step()                     # 5. update parameters
    model.eval()                             # disable dropout for evaluation
    print(epoch, compute_accuracy(model, test_loader))

@torch.no_grad()
def compute_accuracy(model, loader):
    correct = total = 0
    for xb, yb in loader:
        correct += (model(xb).argmax(dim=-1) == yb).sum().item()
        total += yb.numel()
    return correct / total
```

A few remarks generalise to LLM training. `model.train()` and `model.eval()` switch the behaviour of layers such as dropout; forgetting `eval()` during generation makes outputs random for the wrong reason. `optimizer.zero_grad()` can sit before `backward()` or after `step()`, but must happen once per optimiser step; `zero_grad(set_to_none=True)` (the default) frees the gradient tensors instead of writing zeros. The learning rate is the most important hyperparameter, and in Chapter 7 we add warmup, cosine decay and gradient clipping to this loop. Evaluating on a held-out set separates fitting from generalisation; for LLMs we will track validation loss and, later, task accuracy.

2.8 Saving and loading models

LLM A.8

A module's `state_dict()` is an ordered mapping from parameter (and buffer) names to tensors. Saving and restoring it is the standard way to checkpoint:

Listing 2.10: Saving and restoring weights (and, for resuming, optimiser state).

```
torch.save(model.state_dict(), "model.pth")
model = MLP(2, 2)                                  # same architecture
model.load_state_dict(torch.load("model.pth", weights_only=True))

# resuming training needs the optimiser state too (e.g. Adam moments)
```

```
torch.save({"model": model.state_dict(), "opt": optimizer.state_dict(),
            "step": step}, "ckpt.pth")
```

`torch.save` uses Python's pickle format, which can execute arbitrary code on load, so the flag `weights_only=True` should be used for files you did not create. The ecosystem has largely moved to **safetensors** for distributing weights: a simple header plus raw tensor bytes, safe to load and memory-mappable, so a 16 GB checkpoint can be opened instantly and paged in lazily. Both the GPT-2 and Qwen3 loaders in `llmsys` read safetensors files.

Napkin Math: Checkpoint sizes

A training checkpoint holds far more than the weights. For a 7B model trained with mixed-precision AdamW, the fp32 master weights and two Adam moments total $7 \times 10^9 \times 12 = 84$ GB, plus 14 GB of bf16 weights if stored. At 2 GB/s of write bandwidth to a parallel file system, saving takes about 50 s, so frequent synchronous checkpointing at scale wastes real GPU time. Sharded and asynchronous checkpointing (Chapters 7 and 8) exist for this reason.

2.9 Training on GPUs

LLM A.9

2.9.1 Computations on GPU devices

LLM A.9.1

Every tensor lives on a device, and an operation runs on the device of its inputs. Moving a tensor is explicit, and mixing devices in one operation is an error:

Listing 2.11: Device placement rules.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
a = torch.tensor([1., 2., 3.]).to(device)
b = torch.tensor([4., 5., 6.]).to(device)
print(a + b)            # computed on the GPU; result lives there too
c = torch.tensor([1., 1., 1.])
# a + c  -> RuntimeError: tensors on different devices (cuda:0 and cpu)
model = model.to(device)  # moves all parameters and buffers in place
```

On a machine with several GPUs, `"cuda:0"`, `"cuda:1"`, ... name individual devices, and the environment variable `CUDA_VISIBLE_DEVICES` restricts and renumbers what a process can see.

2.9.2 Single-GPU training

LLM A.9.2

Moving the loop of Listing 2.9 to a GPU needs three changes: move the model once, move each batch inside the loop, and leave everything else alone. For a tiny MLP the GPU will not be faster (each kernel does almost nothing and launch overhead dominates), but for anything matrix-heavy, including every LLM, the speedups are one to two orders of magnitude over a CPU.

Listing 2.12: The training loop on a GPU.

```
model = MLP(2, 2).to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
for epoch in range(3):
    model.train()
    for xb, yb in train_loader:
```

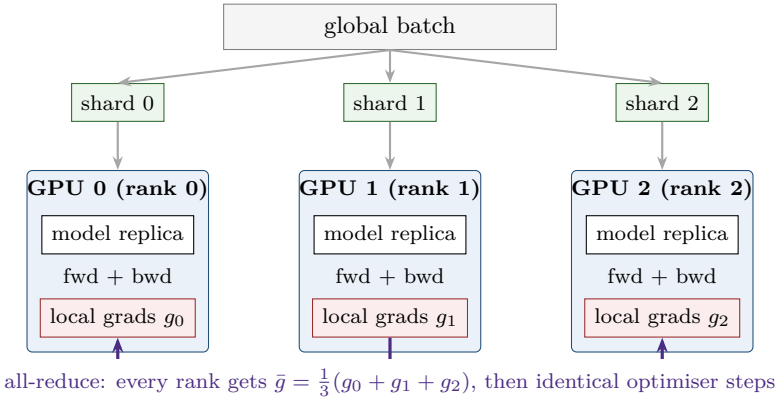


Figure 2.7: Distributed data parallelism. A `DistributedSampler` gives each rank a disjoint shard; each rank computes gradients on its shard; an all-reduce averages them; identical updates keep replicas identical.

```
        xb, yb = xb.to(device, non_blocking=True), yb.to(device, non_blocking=True)
        loss = F.cross_entropy(model(xb), yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

2.9.3 Training with multiple GPUs

LLM A.9.3

The simplest way to use several GPUs is *data parallelism* with PyTorch's `DistributedDataParallel` (DDP). One process is launched per GPU; each holds a full replica of the model and receives a different slice of every global batch. After the backward pass, gradients are averaged across processes with an *all-reduce* collective, so every replica applies the same update and the replicas stay identical (Fig. 2.7). With P GPUs, an epoch takes roughly $1/P$ of the time, minus communication overhead.

The code changes are modest. A process group is initialised with a rendezvous address, the model is wrapped in `DDP`, and the loader uses a `DistributedSampler`. The complete script is `labs/lab02_pytorch_roofline/ddp_demo.py`; its heart is:

Listing 2.13: The essential pieces of DDP training (excerpt of `ddp_demo.py`).

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler

def setup(rank, world_size):
    os.environ.setdefault("MASTER_ADDR", "localhost")    # rendezvous host
    os.environ.setdefault("MASTER_PORT", "29512")        # any free port
    backend = "nccl" if torch.cuda.is_available() else "gloo"
    dist.init_process_group(backend, rank=rank, world_size=world_size)
    if backend == "nccl":
        torch.cuda.set_device(rank)

def worker(rank, world_size, epochs):
    setup(rank, world_size)
    torch.manual_seed(0)                          # same init on every rank
    model = build_mlp(2, [16, 16], 2).to(device)
```

```
    ddp = DDP(model, device_ids=[rank] if device.type == "cuda" else None)
    sampler = DistributedSampler(ds, num_replicas=world_size, rank=rank)
    loader = DataLoader(ds, batch_size=16, sampler=sampler, pin_memory=True)
    for epoch in range(epochs):
        sampler.set_epoch(epoch)               # different shuffle each epoch
        for x, y in loader:
            loss = F.cross_entropy(ddp(x.to(device)), y.to(device))
            opt.zero_grad(); loss.backward(); opt.step()
    dist.destroy_process_group()

torch.multiprocessing.spawn(worker, args=(world_size, 3), nprocs=world_size)
```

In practice, scripts are usually launched with `torchrun --nproc_per_node=N script.py`, which spawns the processes and sets `RANK`, `LOCAL_RANK`, `WORLD_SIZE`, `MASTER_ADDR` and `MASTER_PORT`. DDP requires real processes, so it does not work from inside a Jupyter notebook. The NCCL backend implements the collectives on NVIDIA GPUs over NVLink and InfiniBand; the Gloo backend runs them over TCP and is what the lab uses on CPU.

Systems Lens: What DDP actually does during backward

DDP does not wait for the full backward pass and then all-reduce everything. It registers autograd hooks that fire as each parameter's gradient becomes ready, groups gradients into *buckets* (25 MB by default), and launches an asynchronous all-reduce per bucket while backward continues on earlier layers. Communication thus overlaps with computation. A ring all-reduce moves about $2(P-1)/P$ times the gradient size per GPU, so for a 1B-parameter model with bf16 gradients each step moves about 4 GB per GPU; over NVLink at hundreds of GB/s this hides easily behind compute, over 25 Gb/s Ethernet it does not. DDP also assumes every replica fits the full model, gradients and optimiser state on one GPU. Chapter 8 removes that assumption with ZeRO/FSDP, tensor and pipeline parallelism, and derives these communication costs properly.

2.10 The machine underneath: how a GPU is organised

We now switch from the API to the hardware. The details differ across generations and vendors, but the organisation of an NVIDIA datacentre GPU is representative (Fig. 2.8); we use the H100 SXM as the running example.

Streaming multiprocessors. The chip consists of many identical *streaming multiprocessors* (SMs), 132 on an H100 SXM. Each SM runs many threads concurrently, organised in *warps* of 32 threads that execute the same instruction in lock-step (SIMT), and groups of warps form *thread blocks* that are scheduled onto one SM and can share fast on-chip memory. Each SM contains scalar/vector units and *tensor cores*, specialised units that compute small matrix products (for example $16 \times 8 \times 16$ tiles) in one instruction. Nearly all of an H100's 989 TFLOP/s of bf16 throughput comes from tensor cores; the non-tensor fp32 throughput is more than an order of magnitude lower. A kernel that cannot be phrased as matrix multiplication cannot use most of the chip's FLOPs.

The memory hierarchy. Data move through a hierarchy whose levels trade capacity for bandwidth and latency: per-thread *registers* (256 KB per SM in total); per-SM *shared memory*/L1 (up to about 228 KB of configurable shared memory per SM on H100), explicitly managed by

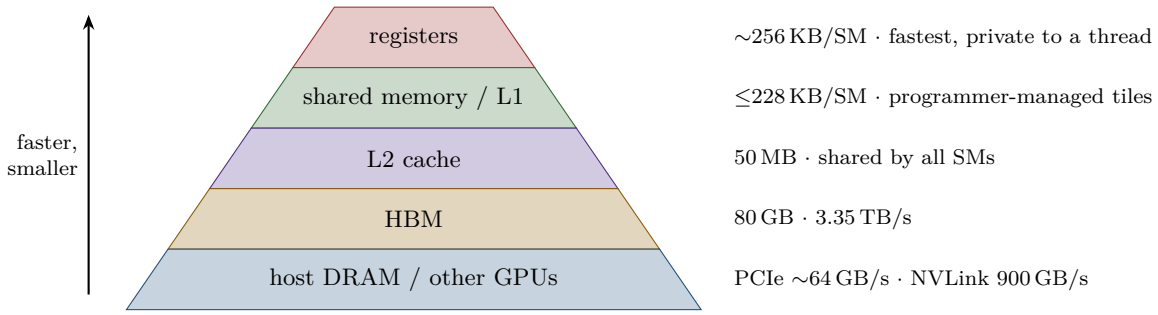


Figure 2.8: The memory hierarchy of an H100 SXM (approximate published figures). Every kernel's performance is determined by how often it must go down this pyramid. Fast kernels such as FlashAttention (Chapter 4) restructure computation so that data loaded from HBM are reused many times in shared memory and registers.

the kernel; a chip-wide *L2 cache* (50 MB); and off-chip *high-bandwidth memory* (HBM: 80 GB at 3.35 TB/s). Host memory sits beyond PCIe (tens of GB/s), and other GPUs are reachable over NVLink (900 GB/s aggregate per H100).

Kernels. A GPU kernel is a function executed by a grid of thread blocks. A typical PyTorch operation, say `torch.add(x, y)`, launches one kernel that reads x and y from HBM, adds them in registers, and writes the result back to HBM. A matrix multiplication launches a cuBLAS kernel that tiles the output, stages input tiles in shared memory, and feeds tensor cores from registers. The difference in how much work each loaded byte supports is the subject of the next section.

2.11 The roofline model

A kernel needs time for its arithmetic and time to move its data. If the two overlap perfectly (a good first approximation for well-written kernels), its runtime is bounded below by the larger:

$$t \;\geq\; \max\!\left(\frac{F}{\pi},\; \frac{Q}{\beta}\right), \tag{2.2}$$

where F is the number of FLOPs, Q the bytes moved to and from HBM, π the peak FLOP/s and β the peak bandwidth. Dividing F by t gives the attainable throughput as a function of the kernel's *arithmetic intensity* $I = F/Q$ (FLOPs per byte):

$$\text{attainable FLOP/s} \;=\; \min(\pi,\; I \cdot \beta). \tag{2.3}$$

Plotted on log–log axes, this is a sloped line (bandwidth-bound region) meeting a flat line (compute-bound region) at the *ridge point* $I^\star = \pi/\beta$: the *roofline* (Williams, Waterman and Patterson, 2009). Kernels left of the ridge are memory-bound; making their arithmetic faster does nothing, only moving fewer bytes helps. Kernels right of it are compute-bound.

2.11.1 Arithmetic intensity of the kernels that matter

Matrix multiplication. For $\mathbf{C} = \mathbf{AB}$ with $\mathbf{A} \in \mathbb{R}^{m\times k}$, $\mathbf{B} \in \mathbb{R}^{k\times n}$ stored with b bytes per element, $F = 2mnk$ and, at best, $Q = b(mk + kn + mn)$ (each operand read once, the result written once), so

$$I_{\text{GEMM}} = \frac{2mnk}{b(mk + kn + mn)} \xrightarrow{m=n=k} \frac{2n}{3b}. \tag{2.4}$$

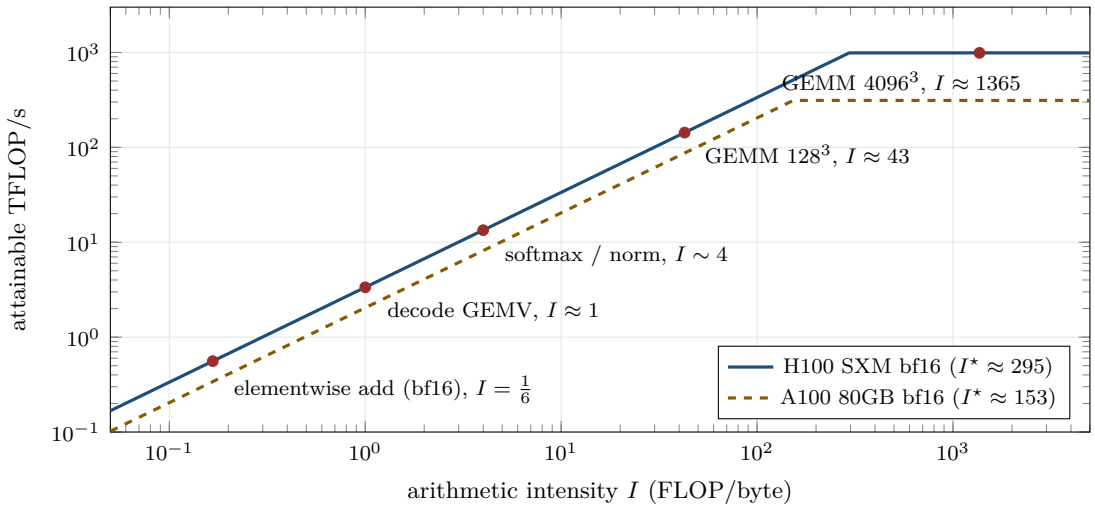


Figure 2.9: Rooflines for H100 and A100 in bf16 (published peak numbers) with typical LLM kernels placed at their arithmetic intensity (computed, not measured). Only large matrix multiplications reach the compute roof; elementwise operations, normalisations and batch-1 decoding live on the bandwidth slope.

In bf16 a square GEMM has $I = n/3$: it crosses the H100 ridge near $n \approx 885$. Large training GEMMs are compute-bound; small ones are not.

Matrix–vector products. With $m = 1$ (one token's activations times a weight matrix), $I = 2nk/(b(k + kn + n)) \approx 2/b$: about 1 FLOP/byte in bf16. This is the operation at the heart of batch-1 decoding, and it sits at about 1/300 of the H100's ridge point. With a batch of B tokens sharing the same weights, $m = B$ and $I \approx 2B/b$ for $B \ll k, n$; batching is the lever that slides decoding up the roofline.

Elementwise operations. `y = x + z` in bf16 does 1 FLOP per element and moves 6 bytes: $I = 1/6$. Activation functions, residual additions, dropout, and the scaling in attention are all like this. Normalisations and softmax perform a handful of FLOPs per element over two passes, I of order 1–10. All are deeply memory-bound, so the only way to speed them up is to avoid materialising their inputs and outputs in HBM, by *fusing* them into neighbouring kernels.

Napkin Math: Fusion pays for memory-bound chains

Take a bf16 activation tensor of $8 \times 4096 \times 4096$ elements (256 MiB) passing through bias-add, GELU and dropout as three separate kernels: each reads and writes the tensor, about $3 \times 2 \times 256$ MiB ≈ 1.6 GB of traffic, ≈0.48 ms at 3.35 TB/s. Fused into one kernel it reads once and writes once: 0.54 GB, ≈0.16 ms, a 3× speedup with identical FLOPs. This is exactly what `torch.compile`'s code generator does automatically (Section 2.12) and what we do by hand in Triton in Appendix A.

Systems Lens: Reading the rest of the book through the roofline

Many optimisations in later chapters are roofline arguments. FlashAttention (Chapter 4) raises attention's intensity by never writing the $T \times T$ score matrix to HBM. The KV cache (Chapter 11) trades recomputation (FLOPs) for memory traffic. Grouped-query attention

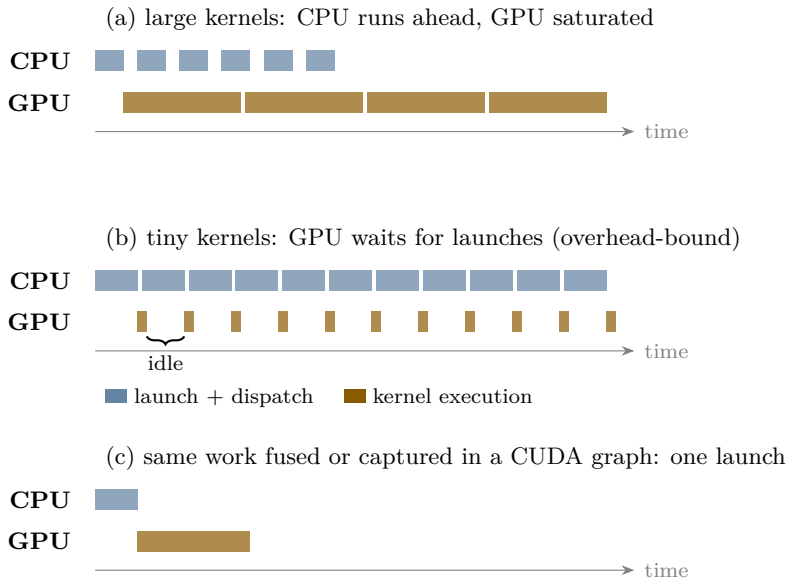


Figure 2.10: Asynchronous execution. (a) With large kernels, the CPU enqueues work faster than the GPU consumes it, so launch overhead is hidden. (b) With tiny kernels, each launch costs more CPU time than the GPU spends executing, leaving the GPU idle. (c) Fusion and CUDA graphs collapse many launches into one.

(Chapter 6) shrinks the bytes read per decoded token. Batching and continuous batching (Chapter 12) raise decoding's I by sharing weight reads. Weight quantisation (Chapter 13) cuts Q directly. Speculative decoding verifies several tokens per weight read. Each is a move to the right on Fig. 2.9, or a reduction of Q at fixed F.

2.12 The execution model: asynchrony, overhead and compilation

2.12.1 Kernels are launched asynchronously

When Python calls a CUDA operation, PyTorch does the bookkeeping on the CPU (argument checks, dispatch, allocation), enqueues the kernel on a *CUDA stream*, and returns immediately, before the GPU has done any work. The CPU races ahead, enqueuing more kernels, while the GPU drains the queue in order. When the CPU is ahead, launch overhead is hidden and the GPU is always busy. When individual kernels are tiny (small tensors, many elementwise ops), the GPU finishes each kernel faster than the CPU can enqueue the next, and the program becomes *overhead-bound*: the GPU idles between kernels (Fig. 2.10).

Napkin Math: When does launch overhead matter?

A Python-level PyTorch op costs on the order of 5–20 µs of CPU time; a kernel that moves 1 MB at 3.35 TB/s runs in 0.3 µs. Decoding one token of Qwen3-0.6B touches 28 layers with a dozen or more small kernels each: several hundred launches, i.e. milliseconds of CPU time, versus a bandwidth floor of about 0.36 ms (1.2 GB/3.35 TB/s). Eager-mode decoding of small models is therefore often CPU-bound by a large factor. This is why serving engines capture the decode step in a CUDA graph and why Chapter 11 shows large

speedups from `torch.compile` on small models.

2.12.2 Synchronisation points

Because work is asynchronous, anything that needs a *value* computed on the GPU forces the CPU to wait for the queue to drain: `.item()`, `.tolist()`, `print(tensor)`, `.cpu()`, and Python control flow that depends on tensor values (`if loss > 0:`). An innocent `loss.item()` for logging every step serialises CPU and GPU and destroys overlap. Log every k steps, or accumulate on the device. The same asynchrony makes naive timing wrong: `time.time()` around a CUDA call measures enqueue time, not execution time, unless the device is synchronised first (Section 2.13).

2.12.3 Streams and overlap

A stream is an in-order queue of kernels and copies. Work in different streams may overlap, which is how communication (NCCL kernels on their own stream), host-to-device copies (with pinned memory) and compute run concurrently. DDP's bucketed all-reduce and the loader's `non_blocking` copies exploit this; in Chapters 8 and 12 we schedule overlap explicitly.

2.12.4 The caching allocator

`cudaMalloc` and `cudaFree` are slow and synchronising, so PyTorch keeps freed blocks in a cache and reuses them. Consequently `nvidia-smi` reports the cache's size rather than live tensors, and `torch.cuda.memory_allocated()` versus `torch.cuda.memory_reserved()` tell the two apart. Allocation patterns with many differently sized tensors (variable sequence lengths are the classic case) fragment the cache, which can produce out-of-memory errors with plenty of nominally free memory. The KV-cache designs of Chapters 11 and 12 preallocate to avoid exactly this.

2.12.5 Graphs and compilation

Eager execution is flexible but pays per-op overhead and cannot optimise across operations. Two mechanisms recover performance. **CUDA graphs** record a fixed sequence of kernel launches once and replay it with a single launch; they require static shapes and static memory addresses, which is why they suit the fixed-shape decode step of a serving engine. `torch.compile` captures Python-level programs into a graph (TorchDynamo), derives the backward graph ahead of time (AOTAutograd), and generates fused kernels (TorchInductor, emitting Triton on GPUs and C++/OpenMP on CPUs), optionally using CUDA graphs (`mode="reduce-overhead"`). The first call is slow, since it compiles, and changing input shapes can trigger recompilation. We use compilation for inference in Chapter 11 and for training in Chapter 7.

Listing 2.14: Compiling a module. Same semantics, fewer and larger kernels.

```
model = MLP(4096, 4096, hidden=(16384,)).to(device)
fast = torch.compile(model)                    # default: fuse ops, Triton kernels
x = torch.randn(64, 4096, device=device)
fast(x)                                        # first call compiles (seconds)
fast(x)                                        # later calls are fast
fast_rt = torch.compile(model, mode="reduce-overhead")  # + CUDA graphs
```

2.13 Measuring and profiling

Performance claims without careful measurement are folklore. Three rules make timings trustworthy: **warm up** (the first calls pay for allocator growth, cuBLAS heuristic selection, or compilation); **synchronise** before reading the clock (asynchrony again); and **repeat and take the median** (robust to noise from other processes). `llmsys.profiling.benchmark` implements exactly this:

Listing 2.15: Honest timing (from `llmsys/profiling.py`).

```
def benchmark(fn, warmup=3, iters=10, min_time=0.0):
    for _ in range(warmup):
        fn()
    _sync()                                      # drain queued GPU work
    times, spent = [], 0.0
    while len(times) < iters or spent < min_time:
        t0 = time.perf_counter()
        fn()
        _sync()                                  # wait for this call's kernels
        dt = time.perf_counter() - t0
        times.append(dt); spent += dt
    return statistics.median(times)
```

From a time, derive rates and compare them with the roofline. For a GEMM, achieved FLOP/s $= 2mnk/t$ and achieved bandwidth $= b(mk + kn + mn)/t$:

Listing 2.16: Placing a measured kernel on the roofline.

```
from llmsys.profiling import benchmark, matmul_stats, Roofline
a = torch.randn(4096, 4096, device="cuda", dtype=torch.bfloat16)
b = torch.randn(4096, 4096, device="cuda", dtype=torch.bfloat16)
t = benchmark(lambda: a @ b)
s = matmul_stats(4096, 4096, 4096, t, bytes_per_el=2)
roof = Roofline(peak_flops=989e12, peak_bw=3.35e12)
print(f"{s['flops_per_s']/1e12:.0f} TFLOP/s of "
      f"{roof.attainable(s['intensity'])/1e12:.0f} attainable "
      f"({roof.bound(s['intensity'])}-bound)")
```

When a whole training step is slow, the question is *which* kernels dominate. `torch.profiler` records every operator and kernel with CPU and GPU times and can export a Chrome/Perfetto trace for timeline inspection; `llmsys.profiling.top_ops` returns the most expensive ops as a table:

Listing 2.17: Op-level profile of one training step.

```
from torch.profiler import profile, ProfilerActivity
acts = [ProfilerActivity.CPU, ProfilerActivity.CUDA]
with profile(activities=acts, record_shapes=True) as prof:
    loss = F.cross_entropy(model(x), y); loss.backward(); opt.step()
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=10))
prof.export_chrome_trace("step.json")          # open in ui.perfetto.dev
```

For kernel-level questions (achieved occupancy, memory throughput, tensor-core utilisation, stalls), NVIDIA's Nsight Systems (timelines across CPU, GPU, NCCL and NVTX ranges)

and Nsight Compute (per-kernel hardware counters, including a roofline view) are the tools of record; Appendix A uses both.

Pitfall: Common measurement mistakes

(1) Timing without synchronisation measures launch time. (2) Timing the first iteration measures compilation and allocator warm-up. (3) Benchmarking with inputs that fit in L2 cache (50 MB on H100) overstates bandwidth; use buffers several times larger than L2 or flush it. (4) Reporting a mean over runs that include a stray outlier: use medians and report spread. (5) Comparing dtypes without checking that the fast path (tensor cores, tf32) was actually enabled. (6) On shared machines, noisy neighbours make CPU numbers erratic; repeat until stable.

Systems Lens: Model FLOPs utilisation

For an entire training run the headline metric is *model FLOPs utilisation* (MFU): useful model FLOPs per second (for a dense transformer, $6N$ per training token, ignoring recomputation) divided by the hardware peak. If a 1.3B model trains at 150,000 tokens/s on 8 H100s, MFU $= 150{,}000 \times 6 \times 1.3 \times 10^9 / (8 \times 989 \times 10^{12}) \approx 15\%$, a sign of overheads (small kernels, communication, data loading) worth profiling. Well-tuned large-scale runs report 35–55%. MFU is the training analogue of the roofline fraction for a single kernel, and `llmsys.flops.mfu` computes it.

2.14 Summary

- PyTorch combines a tensor library, reverse-mode autograd, and deep-learning utilities. Models are `nn.Modules`; data flow through `Dataset`/`DataLoader`; training is the five-step loop of forward, loss, zero-grad, backward, step.
- Tensors are views onto storage with strides; transposes and slices are free until a kernel needs contiguous memory. bf16 is the default LLM format: fp32 range, low precision.
- Backward costs about twice the forward and requires saved activations, hence $6N$ FLOPs per training token and activation memory that scales with batch and sequence length.
- DDP replicates the model per GPU, shards the data, and overlaps bucketed gradient all-reduces with backward.
- GPUs deliver most FLOPs through tensor cores and feed them through a memory hierarchy. The roofline $\min(\pi, I\beta)$ says whether a kernel is limited by compute or bandwidth; most LLM operations other than large GEMMs are bandwidth-bound.
- Execution is asynchronous: hidden launch overhead becomes visible with small kernels, and synchronisation points serialise CPU and GPU. Fusion, CUDA graphs and `torch.compile` reduce launches and memory traffic.
- Measure with warm-up, synchronisation and medians; profile to find dominant kernels; summarise training efficiency with MFU.

2.15 Exercises

Exercise 2.1 (*Strides*). For a contiguous tensor of shape $(2, 3, 4)$, write down its strides. What are the shape and strides after `.transpose(0, 2)`? Explain why `.view(-1)` fails on the transposed tensor while `.reshape(-1)` succeeds, and what the latter costs.

Exercise 2.2 (*bf16 rounding*). Compute the smallest update δ such that `torch.tensor(1.0, dtype=torch.bfloat16) + delta` differs from 1.0. What does this imply for applying an update of size $\eta\, g = 10^{-4}$ to a weight of magnitude 1 stored in bf16? How do fp32 master weights fix this?

Exercise 2.3 (*Manual gradients*). Derive $\partial \mathcal{L}/\partial z = a - y$ for sigmoid followed by binary cross-entropy, and extend it to a batch with the mean loss. Verify your formula against autograd numerically (this is Lab 2, Task A1).

Exercise 2.4 (*Parameter counting*). Count the parameters of an MLP with layer sizes $784 \to 512 \to 256 \to 10$. How many bytes does it need for weights, gradients and Adam states in fp32? How many FLOPs does one forward pass on a batch of 128 cost?

Exercise 2.5 (*Ridge points*). Compute the ridge point of an A100 (312 TFLOP/s, 2.04 TB/s) and an RTX 4090 (165 TFLOP/s, 1.01 TB/s) in bf16. At what square GEMM size n does each become compute-bound? Which GPU needs a larger decode batch to stop being memory-bound?

Exercise 2.6 (*Intensity of a transformer MLP*). A GPT-style MLP computes $\mathrm{GELU}(\mathbf{X}\mathbf{W}_1)\mathbf{W}_2$ with $\mathbf{X} \in \mathbb{R}^{BT\times d}$, $\mathbf{W}_1 \in \mathbb{R}^{d\times 4d}$, $\mathbf{W}_2 \in \mathbb{R}^{4d\times d}$, $d = 4096$, bf16. Compute FLOPs and minimal bytes for the two GEMMs and the GELU as separate kernels for $BT = 1$ and $BT = 4096$. Which kernels are memory-bound in each case? What is the saving from fusing GELU into the first GEMM's epilogue?

Exercise 2.7 (*DDP communication*). A 7B-parameter model is trained with DDP on 8 GPUs connected by NVLink (assume 300 GB/s effective all-reduce bandwidth per GPU) and, alternatively, on 8 nodes connected by 100 Gb/s Ethernet. Estimate the all-reduce time per step for bf16 gradients with a ring algorithm. If a step's compute takes 1.5 s, which setting can hide communication behind backward? (Also: would the model even fit for DDP?)

Exercise 2.8 (*Launch overhead*). Write a function applying 100 elementwise operations to a 1024-element tensor and time it eagerly on a GPU (or CPU) with proper synchronisation. Repeat with a 10^8-element tensor. Then apply `torch.compile`. Explain the three timings using Fig. 2.10.

Exercise 2.9 (*Profiling*). Profile one training step of the MLP of Listing 2.14 with batch sizes 8 and 4096. Report the top five ops by device time in each case and classify each as compute- or memory-bound using the roofline. How does the fraction of time in `aten::mm`/`addmm` change?

Exercise 2.10 (*MFU sanity check*). A team reports training a 3B model at 2.1 million tokens/s on 64 H100s. Compute the implied MFU. Is the claim plausible? What would you ask them about how "tokens" and "FLOPs" were counted?

2.16 Lab 2: PyTorch essentials and an empirical roofline

Lab 2: PyTorch essentials and an empirical roofline

Code: `code/labs/lab02_pytorch_roofline`

Goals

Part A makes the PyTorch workflow second nature: gradients by hand versus autograd, an `nn.Module`, a `DataLoader`, a training loop, checkpoints, and DDP on CPU processes. Part B measures your own machine: achieved FLOP/s and bandwidth, an empirical roofline, number formats, views versus copies, and an op-level profile of a training step.

Setup

```
cd code
pytest -q labs/lab02_pytorch_roofline                       # grades starter.py
LAB_IMPL=solution pytest -q labs/lab02_pytorch_roofline     # reference
```

Tasks

Part A: PyTorch.

1. `logistic_regression_grads(x, y, w, b)`: hand-derived gradients of mean BCE; the test compares them with autograd to 10^{-6}.
2. `build_mlp(in_dim, hidden, out_dim)`: an `nn.Sequential` returning logits; the test checks shapes and the exact parameter count.
3. `train_classifier` and `accuracy`: the loop of Listing 2.9 with SGD; it must reach over 90% test accuracy on the two-blob `ToyDataset`.
4. `save_and_reload`: a `state_dict` round trip that reproduces the accuracy exactly.
5. Run `ddp_demo.py -world-size 2`. The provided test checks that the two replicas end with bit-identical weights. Explain why they do.

Part B: the machine.

6. `matmul_intensity`, `elementwise_intensity`, `roofline_attainable`: the formulas of Section 2.11.
7. `measure_matmul_sweep` and `measure_bandwidth_sweep`: timed sweeps using `llmsys.profiling.benchmark`; `empirical_roofline` picks the best observed FLOP/s and bytes/s.
8. `shares_storage`: detect whether a tensor is a view, using storage pointers.
9. `format_table`: max, epsilon and smallest normal of fp32/bf16/fp16 from `torch.finfo`.
10. `profile_train_step`: profile forward, backward and step and return the top ops.

Measurements

Run `python labs/lab02_pytorch_roofline/run.py` on your machine (`-device cuda -dtype bf16` on a GPU) to produce `roofline.png`. Expect GEMM points to climb the bandwidth slope and flatten at the compute roof for n in the low thousands, and elementwise points to sit near $I = 1/8$ (fp32) or $1/4$ (bf16) on the bandwidth slope. On a GPU, compare your empirical peaks with the datasheet. Reaching 60–80% of peak bandwidth and 60–80% of peak bf16 FLOP/s with plain PyTorch is typical. On a laptop CPU, expect noisy results; repeat runs and note the variance.

Deliverables

Passing tests; your roofline plot annotated with the ridge point; a short table of the top

five ops in a training step at batch sizes 8 and 4096 with a one-line roofline explanation of each.

Stretch goals

(1) Add a bf16 and tf32 sweep and quantify the tensor-core speedup. (2) Time 100 tiny elementwise ops eagerly, with `torch.compile`, and with `mode="reduce-overhead"`. (3) Use Nsight Compute's roofline section on one GEMM and one elementwise kernel and compare with your empirical roofline. (4) Measure DDP scaling efficiency for 1, 2 and 4 processes on a larger MLP and relate it to all-reduce volume.

Part II

Building a Large Language Model

Chapter 3

Text Data, Tokenization, and the Input Pipeline

A language model never sees text. It sees a stream of integers, and immediately turns each integer into a vector by indexing into a table. Everything upstream of that table lookup—how raw bytes are split into pieces, how the pieces are numbered, how the numbered stream is cut into training examples, and how those examples reach the accelerator—is the subject of this chapter.

It is tempting to treat this as plumbing. It is not. The tokenizer fixes the vocabulary size V, which sets the size of the largest matrices in a small model and the size of the largest *activation* in a large one (the logits). It fixes how many tokens a document becomes, which sets sequence length, which sets attention cost and KV-cache size at inference time. And the input pipeline decides whether a multi-million-dollar training cluster spends its time doing matrix multiplications or waiting on Python. We build every piece from scratch, then look at each one through a systems lens.

3.1 From symbols to vectors: embeddings

LLM 2.1

Neural networks compute with real-valued tensors; text is a sequence of discrete symbols. The bridge is an *embedding*: a map from a discrete object to a point in $\mathbb{R}^d$. The same idea is used for images, audio and video—each modality gets its own embedding model—but for a decoder-only LLM the object being embedded is a *token*, and the map is a learned lookup table $\mathbf{E} \in \mathbb{R}^{V \times d}$ whose row i is the vector for token i.

Before LLMs, embeddings were often trained as standalone artefacts. *Word2Vec* (Mikolov et al., 2013) learned one vector per word by training a shallow network to predict a word from its neighbours (or the neighbours from the word). The resulting space has a useful geometry: words that occur in similar contexts land near each other, so after projecting to two dimensions one sees clusters (Fig. 3.2). Larger d can encode more relations, at a cost proportional to d in every layer that touches the vector.

LLMs do *not* reuse such pretrained word vectors. The embedding table is just another weight matrix, initialised randomly and optimised jointly with the rest of the network for next-token prediction. Its width is the model width d_{model}: 768 for GPT-2 small, 1024 for Qwen3-0.6B,

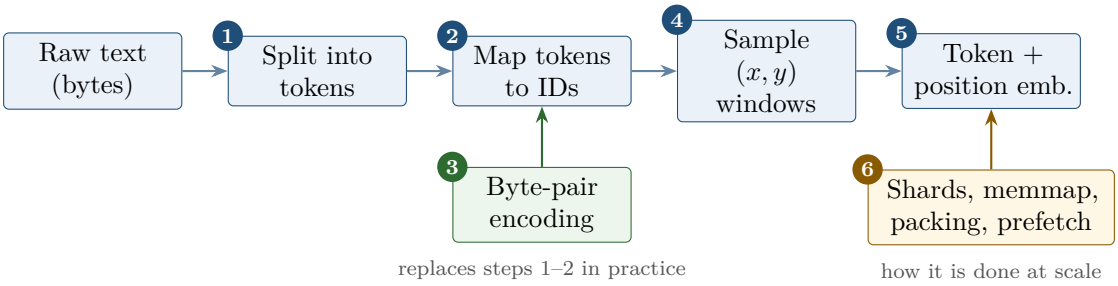


Figure 3.1: Roadmap of this chapter. Steps 1–2 build a word-level tokenizer to expose the ideas, step 3 replaces it with byte-level BPE as used by GPT-2 and Qwen3, step 4 turns a token stream into next-token-prediction examples, step 5 converts IDs to vectors, and step 6 asks how all of this is engineered when the corpus has 10^{13} tokens.

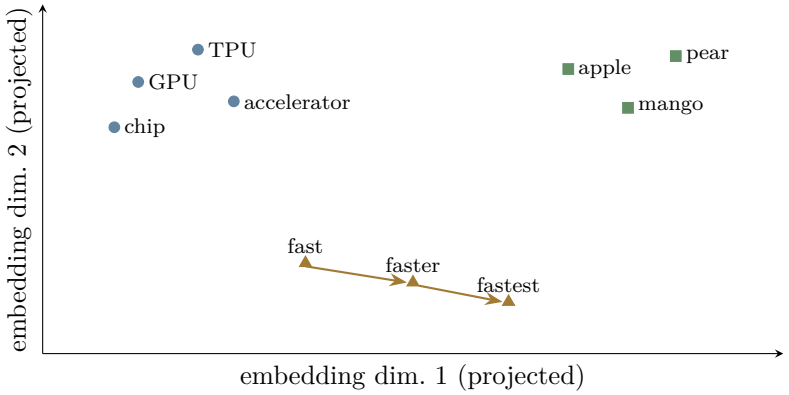


Figure 3.2: A schematic 2-D projection of a word-embedding space (illustrative, not trained). Semantically related words cluster, and some relations (comparative, superlative) appear as roughly consistent offsets.

and in the thousands for larger models. Because the table is trained end to end, its geometry is whatever the downstream layers find useful; the clustering of Fig. 3.2 tends to emerge anyway.

Systems Lens: Embeddings are a gather, not a matmul

Mathematically $\boldsymbol{e}_i = \mathbf{1}_i^\top \mathbf{E}$, a one-hot vector times the table. Operationally, nobody materialises one-hot vectors: the lookup is a *gather* of T rows, reading $T \cdot d$ elements from a $V \cdot d$ table—pure memory traffic, zero FLOPs. The backward pass is a *scatter-add* into those rows, so the gradient of $\mathbf{E}$ is sparse (only rows of tokens present in the batch are non-zero). Optimisers like AdamW still update every row's moments, which is why some large-vocabulary systems use sparse or row-wise optimisers for embeddings, and why the embedding table is a natural candidate for sharding by rows across devices (Chapter 8).

3.2 Tokenizing text

LLM 2.2

To build intuition we start with the simplest useful tokenizer: split on whitespace and punctuation. Our sample text is a short story—the original book uses Edith Wharton's public-domain story *The Verdict* (1908), about twenty thousand characters long, which is small enough to train on a laptop yet long enough to be interesting. Any plain-text file works.

Python's `re.split` with a capturing group returns both the pieces and the separators, which

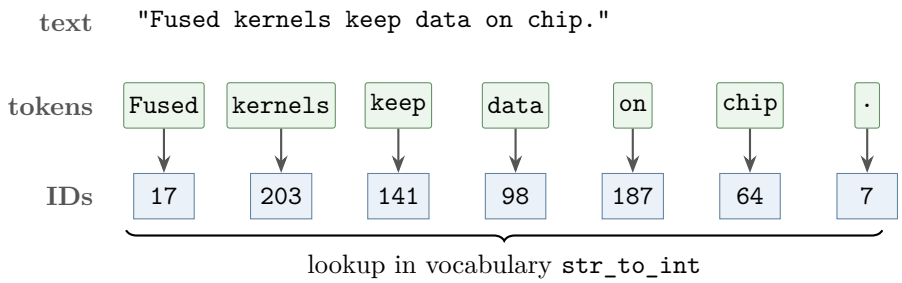


Figure 3.3: Tokenization then ID mapping. The ID values are arbitrary positions in a sorted vocabulary built from the training text.

lets us keep punctuation as tokens of its own:

Listing 3.1: A regular-expression word splitter (Lab 3, Part A).

```
import re

WORD_SPLIT = re.compile(r'([,.:;?_!"()\']|--|\s)')

def split_text(text):
    # capture group => separators are returned too; drop pure whitespace
    return [t for t in WORD_SPLIT.split(text) if t.strip()]

split_text('Fused kernels--fast, "lean" ones?')
# ['Fused', 'kernels', '--', 'fast', ',', '"', 'lean', '"', 'ones', '?']
```

Three design decisions are hiding in those two lines:

1. **Case is preserved.** `Fused` and `fused` become different tokens. This doubles some vocabulary entries but lets the model learn capitalisation conventions (sentence starts, proper nouns), which it needs in order to *generate* well-formed text.

2. **Whitespace is discarded.** Fine for a toy; fatal for code, where indentation is syntax. Byte-level BPE (Section 3.5) keeps whitespace by folding it into tokens.

3. **Punctuation is separated.** `fast,` becomes `fast` and `,`, so the vocabulary does not need a separate entry for every word–punctuation combination.

3.3 Converting tokens into token IDs

LLM 2.3

A *vocabulary* is a bijection between token strings and integers. For a word-level tokenizer we build it by collecting the unique tokens of the training corpus, sorting them (so the mapping is deterministic), and enumerating. Encoding maps strings to IDs; decoding maps IDs back and re-attaches punctuation to the preceding word.

Listing 3.2: Vocabulary construction and a reversible word-level tokenizer.

```
def build_vocab(tokens, specials=("<|endoftext|>", "<|unk|>")):
    words = sorted(set(tokens))
    words += [s for s in specials if s not in words]
    return {w: i for i, w in enumerate(words)}

```

```python
class WordTokenizer:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}
        self.unk = vocab.get("<|unk|>")

    def encode(self, text):
        return [self.str_to_int.get(t, self.unk) for t in split_text(text)]

    def decode(self, ids):
        text = " ".join(self.int_to_str[i] for i in ids)
        return re.sub(r'\s+([,.:;?!"()\'])', r"\1", text)  # re-attach punctuation
```

On a twenty-thousand-character story this yields a vocabulary of roughly a thousand entries. Encoding and then decoding a sentence drawn from the training text reproduces it (modulo the whitespace normalisation of the decoder). Encoding a sentence with a word that never appeared in training is where things go wrong—which motivates special tokens.

Pitfall: Decoding is not always the inverse of encoding

Any tokenizer that *discards* information during splitting (whitespace runs, tabs, Unicode normalisation, lower-casing) cannot be inverted exactly. For training data this is a silent data transformation; for a model that must emit code or JSON it is a correctness bug. A useful unit test for any tokenizer is `decode(encode(s)) == s` over a fuzzed corpus of strings, including emoji, combining characters, and invalid UTF-8 boundaries.

3.4 Adding special context tokens

LLM 2.4

Two problems remain: words outside the vocabulary, and the boundaries between unrelated documents. Both are solved by reserving vocabulary entries that never occur as ordinary text.

- `<|unk|>` stands in for any out-of-vocabulary word. It makes encoding total, but at a cost: every unknown word collapses to the same ID, so information is irreversibly lost.
- `<|endoftext|>` is inserted between independent documents when they are concatenated into one training stream. It tells the model "what follows is unrelated to what preceded" and, at inference time, serves as a stop signal.

Other conventions exist. Some models use `[BOS]` (beginning of sequence) and `[EOS]` (end of sequence) separately; many use a `[PAD]` token to right-pad batches to equal length. GPT-2 uses exactly one special token, `<|endoftext|>` (ID 50256), for document separation *and* as a padding filler when one is needed; padding positions are then excluded from attention and from the loss by masks, so what the filler token is does not matter (Chapters 9 and 10). Chat models add more structure: Qwen3 wraps each turn in `<|im_start|>` … `<|im_end|>` and marks reasoning with `<think>` tags (Chapters 11 and 18).

3.5 Byte pair encoding

LLM 2.5

Word-level vocabularies have two fatal flaws for open-domain text: they cannot represent unseen words without `<|unk|>`, and to cover a large corpus they need hundreds of thousands of entries.

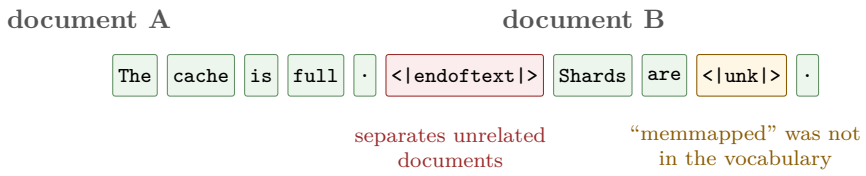


Figure 3.4: Two special tokens in a concatenated training stream. The out-of-vocabulary word is replaced by `<|unk|>`; the document boundary is made explicit with `<|endoftext|>`.

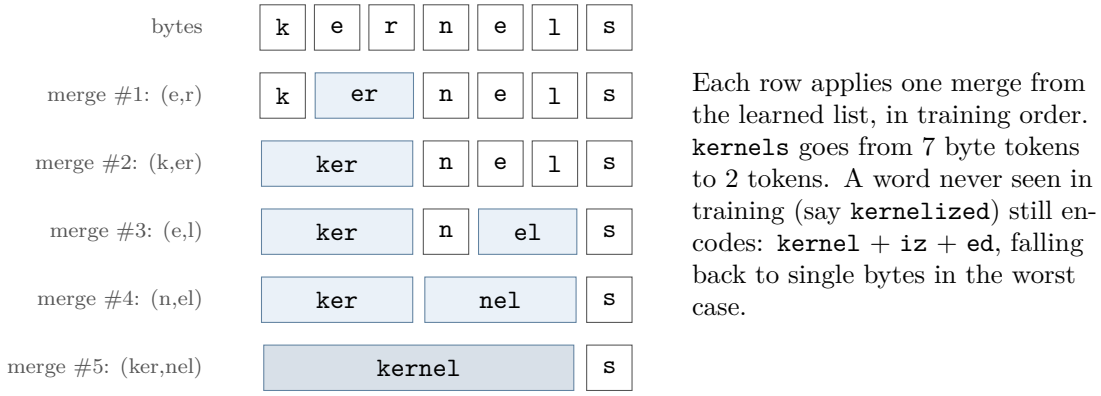


Figure 3.5: Encoding with a learned merge list. Encoding repeatedly applies the *earliest-learned* applicable merge until none applies.

Character-level vocabularies have the opposite problem: tiny vocabulary, but sequences four to five times longer, and attention cost grows quadratically with length. *Byte pair encoding* (BPE; Gage 1994, adapted to NLP by Sennrich et al., 2016) interpolates between the two, and the byte-level variant introduced with GPT-2 (Radford et al., 2019) removes the unknown-token problem entirely.

3.5.1 The algorithm

Byte-level BPE starts from the 256 possible byte values as the initial vocabulary. Any string, in any language, encodes to UTF-8 bytes, so *every* input is representable—there is no `<|unk|>`. Training then repeats:

1. Count every adjacent pair of symbols in the corpus (weighted by frequency).
2. Take the most frequent pair (a, b) and add a new symbol c whose byte string is the concatenation of those of a and b.
3. Replace every occurrence of (a, b) in the corpus by c.

until the vocabulary reaches the target size V. The ordered list of merges *is* the tokenizer. GPT-2 performed 50,000 merges on top of 256 bytes, plus one special token: $256 + 50{,}000 + 1 = 50{,}257$.

Two refinements make this work on real text. First, a *pre-tokenization* regex splits the text into chunks (words with an optional leading space, digit runs, punctuation runs, whitespace) and merges are never allowed to cross chunk boundaries; otherwise the most frequent “pair” would quickly become something like `"e " + "the"`. Second, frequent words therefore become single tokens *including their leading space* (`" the"`), which is how byte-level BPE preserves whitespace exactly.

3.5.2 Implementation

Listing 3.3 is a complete, if slow, byte-level BPE trainer. Words are stored as tuples of IDs with their corpus frequency, so a word that appears 10^4 times is processed once per merge, not 10^4 times.

Listing 3.3: Naive byte-level BPE training (Lab 3, Part B).

```
def get_pair_counts(words):          # words: {tuple_of_ids: frequency}
    counts = Counter()
    for ids, freq in words.items():
        for pair in zip(ids, ids[1:]):
            counts[pair] += freq
    return counts

def train_bpe(text, vocab_size):
    words = {tuple(w.encode("utf-8")): f for w, f in pretokenize(text).items()}
    merges = {}
    for new_id in range(256, vocab_size):
        counts = get_pair_counts(words)
        if not counts:
            break
        best = max(counts, key=lambda p: (counts[p], -p[0], -p[1]))  # det. ties
        merges[best] = new_id
        words = {tuple(merge_pair(w, best, new_id)): f for w, f in words.items()}
    return merges
```

Encoding a new string applies the learned merges *in training order*: inside each chunk, find the adjacent pair with the smallest merge ID, merge it, and repeat (Listing 3.4). Decoding is trivial: concatenate the byte strings of the IDs and decode UTF-8. Because merges can split a multi-byte UTF-8 character across tokens, a *single* token may not be valid UTF-8 on its own—streaming detokenizers must buffer bytes until a character completes (Chapter 11).

Listing 3.4: BPE encoding by merge rank; decoding by byte concatenation.

```
def bpe_encode(text, merges):
    out = []
    for chunk in PRETOKENIZE.findall(text):
        ids = list(chunk.encode("utf-8"))
        while len(ids) >= 2:
            pair = min(zip(ids, ids[1:]), key=lambda p: merges.get(p, float("inf")))
            if pair not in merges:
                break                       # no applicable merge left
            ids = merge_pair(ids, pair, merges[pair])
        out.extend(ids)
    return out

def bpe_decode(ids, merges):
    vocab = build_byte_vocab(merges)        # id -> bytes
    return b"".join(vocab[i] for i in ids).decode("utf-8", errors="replace")
```

In practice we use a production tokenizer rather than our own: OpenAI's `tiktoken` (written in Rust) ships GPT-2's exact merges, and Hugging Face `tokenizers` loads Qwen3's 151,936-entry vocabulary from a `tokenizer.json`. Our `llmsys.tokenizer.ByteBPETokenizer` implements the same algorithm as Listings 3.3 and 3.4 with save/load and special-token handling.

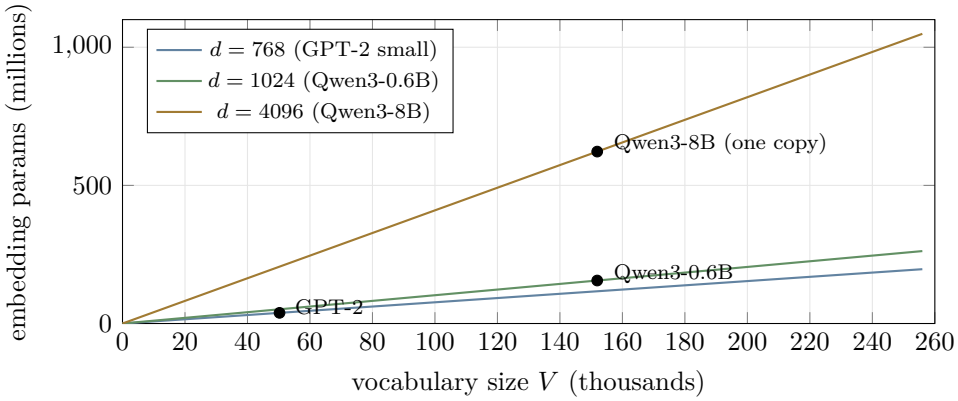


Figure 3.6: Parameters in one $V \times d$ embedding table (analytic). Untied models pay this twice. For small models the vocabulary dominates the parameter budget; for large models it is a few percent, but the per-token logits computation and the training-time logits tensor still scale with V.

Systems Lens: Making BPE training fast

The naive trainer recounts every pair in every word after every merge: cost $O(M \cdot W)$ for M merges over W unique-word symbols. But a merge of (a, b) only changes words that *contain* (a, b)—usually a tiny fraction. An incremental trainer keeps (i) global pair counts and (ii) an inverted index *pair* $\rightarrow$ *words containing it*; for each merge it visits only the affected words, subtracts their old pairs, rewrites them, and adds the new pairs (Lab 3, Part C). On a 485 kB synthetic corpus in this book's build environment, the incremental version learned the same 344 merges 6× faster than the naive one (2.6 s $\rightarrow$ 0.4 s, single CPU core, shared machine). Production trainers go further: a max-heap with lazy deletion makes "find the best pair" $O(\log P)$ instead of $O(P)$, pre-tokenization runs in parallel over shards, and everything is in Rust or C++.

Napkin Math: What the vocabulary costs

The vocabulary appears twice in the model: the input embedding $\mathbf{E} \in \mathbb{R}^{V \times d}$ and the output projection to logits (often tied to $\mathbf{E}$).

- GPT-2 small: $V \cdot d = 50{,}257 \times 768 = 38.6$M parameters—31% of the 124M total. Each token's logits cost $2Vd = 77$ MFLOP, a quarter of the $\approx 2N$ forward FLOPs.
- Qwen3-0.6B: $151{,}936 \times 1024 = 155.6$M parameters (tied), $\approx 26\%$ of the model.
- The logits *activation* during training is $B \cdot T \cdot V$ values. For $B = 8$, $T = 4096$, $V = 151{,}936$ in fp32 that is $8 \cdot 4096 \cdot 151{,}936 \cdot 4 \approx 19.9$ GB—more than the weights, optimizer state and all other activations of a 0.6B model combined. This is why fused "linear + cross-entropy" kernels that compute the loss chunk-by-chunk without ever storing the full logits matter for large-vocabulary models.

Conversely, a larger vocabulary compresses text into fewer tokens (more bytes per token), shortening sequences and so reducing attention FLOPs and KV-cache size at inference time. Vocabulary size is therefore a genuine systems trade-off, not a modelling detail.

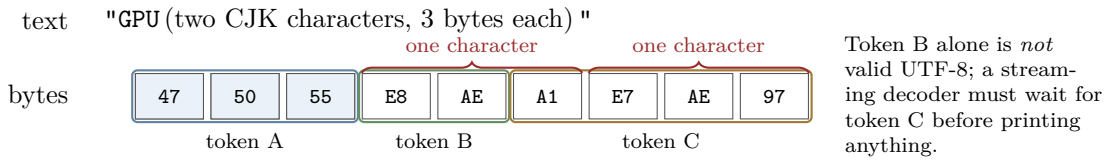


Figure 3.7: Byte-level tokens need not align with character boundaries. Hex bytes shown; the token split is illustrative.

3.6 Tokenizers in the wild

Byte-level BPE is conceptually simple, but production tokenizers are full of details that change model behaviour, cost, and even security. This section collects the ones a systems engineer is most likely to trip over.

3.6.1 UTF-8 and streaming detokenization

UTF-8 encodes a code point in one to four bytes: ASCII in one, most European and Middle-Eastern scripts in two, most CJK characters in three, emoji in four. Because BPE operates on bytes, a token boundary can fall *inside* a character (Fig. 3.7). This is invisible when a whole sequence is decoded at once, but an inference server streaming tokens to a client one at a time must not emit half a character. The fix is a small incremental decoder that holds back bytes until they form complete code points (Listing 3.5); Python's `codecs` module provides exactly this. Hugging Face's and vLLM's detokenizers implement the same idea with an offset-based "prefix/read" window.

Listing 3.5: Streaming detokenization that never emits partial characters.

```python
import codecs

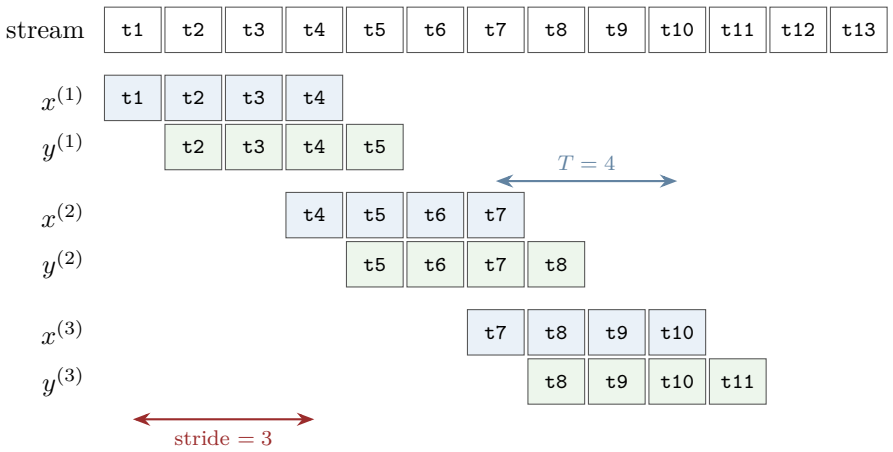
class StreamingDetokenizer:
    def __init__(self, vocab):                  # vocab: id -> bytes
        self.vocab = vocab
        self.dec = codecs.getincrementaldecoder("utf-8")(errors="replace")

    def push(self, token_id):                   # call once per generated token
        return self.dec.decode(self.vocab[token_id])   # "" until a char completes

    def flush(self):
        return self.dec.decode(b"", final=True)
```

3.6.2 Numbers, whitespace and other artefacts

Pre-tokenization rules matter for what a model finds easy. GPT-2's regex lets BPE merge arbitrary digit strings, so `"1234"` and `"12345"` may be split inconsistently, which makes digit-level arithmetic harder to learn. Many newer tokenizers split numbers into single digits or into groups of at most three digits so that place value is represented consistently. Similarly, `" the"` and `"the"` are different tokens; a prompt that ends with a trailing space forces the model to continue from an unusual token boundary and can visibly degrade completions—one reason serving APIs sometimes "heal" the last prompt token. Finally, tokens that were frequent in the tokenizer's training data but rare in the model's training data (the infamous "glitch tokens") have nearly untrained embeddings and can trigger erratic behaviour.



Inputs (blue) and targets (green) are offset by one token. Stride $< T$ overlaps windows; stride $= T$ tiles the stream disjointly.

Figure 3.8: Sliding-window sampling with context length $T = 4$ and stride 3.

3.6.3 Special tokens are a security boundary

Chat models delimit roles with special tokens (Chapter 11). If a tokenizer parsed the *string* `<|im_start|>` followed by `system` inside user-supplied text into the real special-token ID, a user could forge a system message. Production tokenizers therefore distinguish "special tokens allowed in this text" from ordinary text: `tiktoken`'s `allowed_special` and `disallowed_special` arguments exist for exactly this reason, and our `ByteBPETokenizer.encode` takes an `allowed_special` tuple. Serving stacks should tokenize templates and user content separately and concatenate IDs, never concatenate strings.

Napkin Math: The cost of a language

API pricing and context limits are in *tokens*, but users write in *characters*. English prose with a modern BPE tokenizer averages roughly 4 bytes (about 4 characters) per token. A script that needs 3 bytes per character and is underrepresented in the merge list may average closer to 1–1.5 bytes per token, i.e. more than one token per character. The same sentence then consumes several times more tokens: several times the price, several times the KV cache, and a context window that holds several times less text. Measuring bytes/token per language on a held-out multilingual set (Lab 3, `compression_ratio`) is the right first step before choosing or extending a tokenizer.

3.7 Data sampling with a sliding window

LLM 2.6

An LLM is trained to predict the next token. Given a tokenized corpus $t_1, t_2, \ldots, t_N$, a training example is a window of T consecutive tokens as input, $x = (t_{s+1}, \ldots, t_{s+T})$, and the same window shifted one position to the right as target, $y = (t_{s+2}, \ldots, t_{s+T+1})$. One window thus yields T prediction tasks at once: from t_{s+1} predict t_{s+2}; from $t_{s+1..s+2}$ predict t_{s+3}; …; from all T inputs predict t_{s+T+1}. Causal masking (Chapter 4) ensures position i cannot peek at targets to its right, so all T losses are computed in one forward pass.

The *stride* controls how far consecutive windows start apart. Stride $= T$ partitions the stream into disjoint windows—each token is an input exactly once per epoch. Stride $< T$ creates overlap: more examples, but they are highly correlated and the model sees the same text many times per epoch, which encourages memorisation of a small corpus. Stride $= 1$ creates the maximum $N - T$ examples at a T-fold redundancy. For pretraining at scale, disjoint windows

(stride $= T$) are standard.

PyTorch structures this as a `Dataset` (how to fetch example i) plus a `DataLoader` (how to batch, shuffle and parallelise fetching). Our dataset precomputes window views over a single token tensor (Listing 3.6); the loader stacks `batch_size` windows into (B, T) input and target tensors. `drop_last=True` discards a final short batch, which would otherwise produce a noisy, small-batch gradient step and a different tensor shape (bad for compiled graphs).

Listing 3.6: A sliding-window dataset and loader (core: `llmsys/data.py`).

```
class SlidingWindowDataset(Dataset):
    def __init__(self, token_ids, max_length, stride):
        ids = torch.as_tensor(token_ids, dtype=torch.long)
        n = 0
        if len(ids) > max_length:
            n = (len(ids) - max_length - 1) // stride + 1
        self.inputs = [ids[i*stride : i*stride + max_length] for i in range(n)]
        self.targets = [ids[i*stride + 1 : i*stride + max_length + 1]
                        for i in range(n)]
    def __len__(self):
        return len(self.inputs)
    def __getitem__(self, i):
        return self.inputs[i], self.targets[i]

loader = DataLoader(SlidingWindowDataset(ids, 256, 256),
                    batch_size=8, shuffle=True, drop_last=True, num_workers=0)
x, y = next(iter(loader))        # both (8, 256), torch.long
```

It is worth printing what the loader actually yields for tiny settings, because off-by-one errors in the target shift are the most common bug in hand-written pretraining code and they do not crash—the model simply learns to copy its input.

Listing 3.7: Loader output for a toy stream `0..11` with $T = 4$: stride 1 vs. stride 4.

```
stride=1, batch_size=2                stride=4, batch_size=2
x = [[0, 1, 2, 3],                    x = [[0, 1, 2, 3],
     [1, 2, 3, 4]]                         [4, 5, 6, 7]]
y = [[1, 2, 3, 4],                    y = [[1, 2, 3, 4],
     [2, 3, 4, 5]]                         [5, 6, 7, 8]]
-> 8 windows per epoch                -> 2 windows per epoch (tokens 8..11
   (each token reused up to 4x)          only appear as the last targets)
```

A simple invariant catches the shift bug: for every batch, `x[:, 1:]  == y[:, :-1]` must hold elementwise. Lab 3's tests check it, and it is cheap enough to leave as an assertion in a real training loop's first step.

Pitfall: Slices are views

`ids[a:b]` is a *view* sharing storage with `ids`, so the list of windows costs almost nothing beyond the original tensor. If a worker process pickles such a view to send it to the main process, however, PyTorch serialises the *whole* underlying storage. With `num_workers>0` and large corpora, either use memory-mapped files (Section 3.10) or `.clone()` each window before returning it.

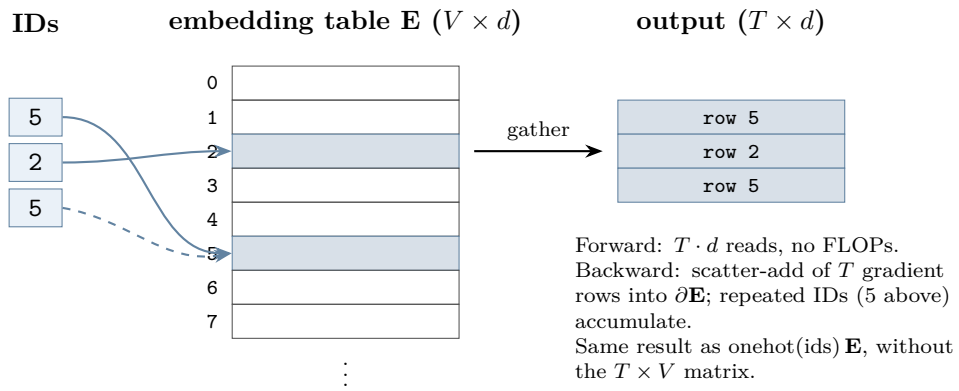


Figure 3.9: An embedding layer is a gather of rows; its gradient is a scatter-add.

3.8 Creating token embeddings

LLM 2.7

The last step converts a batch of IDs into vectors with `nn.Embedding(V, d)`—a trainable $V \times d$ matrix whose forward pass is a row gather. A (B, T) `long` tensor becomes a (B, T, d) float tensor. Figure 3.9 shows the equivalence with a one-hot matrix product, which is useful for reasoning about gradients: since $\boldsymbol{e} = \mathbf{1}_i^\top \mathbf{E}$, $\partial \mathcal{L}/\partial \mathbf{E} = \mathbf{1}_i \, (\partial \mathcal{L}/\partial \boldsymbol{e})$, i.e., the gradient flows only into row i.

Listing 3.8: Embedding lookup equals a one-hot product.

```
torch.manual_seed(0)
emb = torch.nn.Embedding(num_embeddings=6, embedding_dim=3)
ids = torch.tensor([5, 2, 5])
a = emb(ids)                                          # gather: (3, 3)
b = F.one_hot(ids, 6).float() @ emb.weight            # same numbers, (3,6)@(6,3)
assert torch.allclose(a, b)
```

For a GPT-2-sized model the table is 50,257 × 768; a batch of 8 × 1024 IDs produces an $(8, 1024, 768)$ tensor of 6.3M floats (25 MB in fp32). In a real model the table is initialised randomly and learned; our tests initialise it with a fixed seed so results are reproducible.

3.9 Encoding word positions

LLM 2.8

Self-attention, which we build in Chapter 4, is *permutation-equivariant*: shuffling the input vectors shuffles the outputs the same way. Without extra information the model cannot distinguish "dog bites man" from "man bites dog". Token embeddings alone do not help, since the same ID always maps to the same vector regardless of where it appears. Position information must be injected explicitly, and there are two families of approaches.

Absolute position embeddings. Each position $p \in \{0, \ldots, T_{\max} - 1\}$ gets its own vector $\boldsymbol{p}_p$, which is *added* to the token embedding: $\boldsymbol{h}_p = \mathbf{E}[t_p] + \boldsymbol{p}_p$. The original Transformer used fixed sinusoids, $\boldsymbol{p}_{p,2i} = \sin(p/10000^{2i/d})$, $\boldsymbol{p}_{p,2i+1} = \cos(p/10000^{2i/d})$; GPT-2 uses a *learned* table `nn.Embedding(context_length, d)`, optimised like any other weight. Learned absolute positions are simple and effective, but the model has no vectors for positions beyond $T_{\max}$ (1024 for GPT-2), so it cannot run on longer inputs at all.

The sinusoidal scheme has a property worth noticing: for any fixed offset k, the encoding of

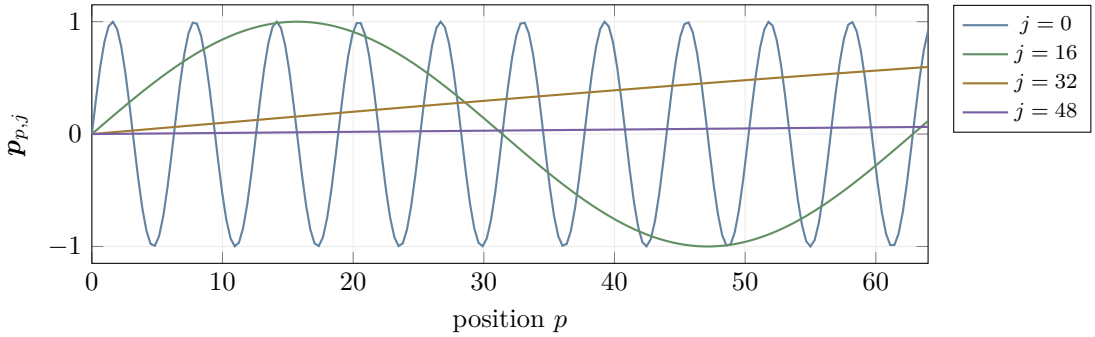


Figure 3.10: Sinusoidal position encodings for $d = 64$ (analytic): each pair of dimensions oscillates at a geometrically decreasing frequency, from period 2π (dimension 0) to very long periods. A position is identified by the joint phase of all frequencies, like the hands of a clock. RoPE (Chapter 6) uses the same frequency ladder but applies it as a rotation of queries and keys.

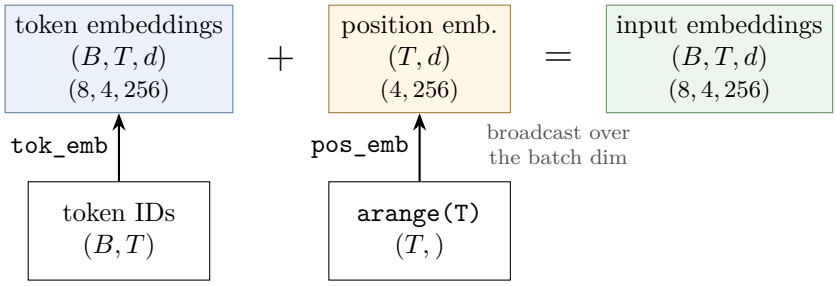


Figure 3.11: GPT-2's input layer: token embeddings plus learned absolute position embeddings, broadcast across the batch. The shapes in small print are the chapter's running example.

position $p + k$ is a fixed rotation (a linear function independent of p) of the encoding of p, because $\sin(\omega(p+k))$ and $\cos(\omega(p+k))$ are linear combinations of $\sin(\omega p)$ and $\cos(\omega p)$. Relative offsets are therefore easy for a linear layer to read out. RoPE takes this observation to its conclusion by applying the rotation to queries and keys directly, so that attention scores depend on $i - j$ *exactly*.

Relative position encodings. Here what matters is the *distance* between two tokens, not their absolute index. Variants add a learned bias depending on $i - j$ to attention scores (T5), add a linear distance penalty (ALiBi; Press et al., 2022), or—most popular today—rotate query and key vectors by position-dependent angles so their dot product depends only on $i - j$ (RoPE; Su et al., 2021), used by Llama, Qwen3 and most modern open models. Chapter 6 implements RoPE and its context-extension tricks.

Listing 3.9: The complete input layer of GPT-2 (Lab 3, Part E).

```
class InputEmbedding(nn.Module):
    def __init__(self, vocab_size, context_length, emb_dim):
        super().__init__()
        self.tok = nn.Embedding(vocab_size, emb_dim)
        self.pos = nn.Embedding(context_length, emb_dim)

    def forward(self, ids):                          # ids: (B, T)
        T = ids.shape[1]
        pos = self.pos(torch.arange(T, device=ids.device))   # (T, d)
        return self.tok(ids) + pos[None]             # (B, T, d)

```

```
emb = InputEmbedding(vocab_size=50257, context_length=4, emb_dim=256)
x, y = next(iter(loader))           # loader with max_length=4, batch_size=8
emb(x).shape                        # torch.Size([8, 4, 256])
```

Systems Lens: Position encoding and the serving stack

The choice of positional scheme leaks into systems in non-obvious ways. (i) With learned absolute positions, a *left-padded* batch shifts every row's positions, changing outputs; with RoPE only relative offsets matter, so a uniform shift is harmless (Chapter 12 and `llmsys.generate.generate_batch`). (ii) Prefix caching across requests (Chapter 12) requires that a cached prefix's keys do not depend on what follows—true for all causal schemes, but KV entries encoded with absolute or rotary positions are only reusable at the *same* positions. (iii) Context length extension (Chapter 6) is a pure positional-encoding problem: RoPE can be rescaled (position interpolation, NTK, YaRN) without retraining from scratch; a learned table cannot.

3.10 The input pipeline at scale

Everything so far fits in memory: tokenize a 20 kB story, keep the IDs in a tensor, slice windows. Pretraining corpora are measured in trillions of tokens. This section describes what changes.

Tokenize once, offline. Tokenization is CPU work that does not depend on model weights, so it is done once in a preprocessing job and the result stored as flat binary arrays of token IDs, a format popularised by projects like nanoGPT and Megatron-LM. If $V \leq 2^{16}$ (GPT-2), each token fits in `uint16`; Qwen3's 151,936-entry vocabulary needs `uint32`.

Memory-map the shards. `np.memmap` maps a file into the address space without reading it; the OS pages in only the bytes a worker touches. A dataset item is then an offset computation plus a small copy (Listing 3.10). This scales to files far larger than RAM, and many worker processes can share the page cache.

Listing 3.10: Writing and reading a token shard (Lab 3, Part D).

```
def write_shard(ids, path, vocab_size):
    dtype = np.uint16 if vocab_size <= 2**16 else np.uint32
    arr = np.asarray(ids, dtype=dtype)
    arr.tofile(path)                    # raw little-endian array, no header
    return arr.nbytes

class ShardDataset(torch.utils.data.Dataset):
    def __init__(self, path, block_size, vocab_size):
        dtype = np.uint16 if vocab_size <= 2**16 else np.uint32
        self.data = np.memmap(path, dtype=dtype, mode="r")
        self.block_size = block_size
    def __len__(self):
        return (len(self.data) - 1) // self.block_size
    def __getitem__(self, i):
        s = i * self.block_size
        raw = self.data[s:s + self.block_size + 1]      # touches only these pages
        chunk = torch.from_numpy(raw.astype(np.int64))
        return chunk[:-1], chunk[1:]
```

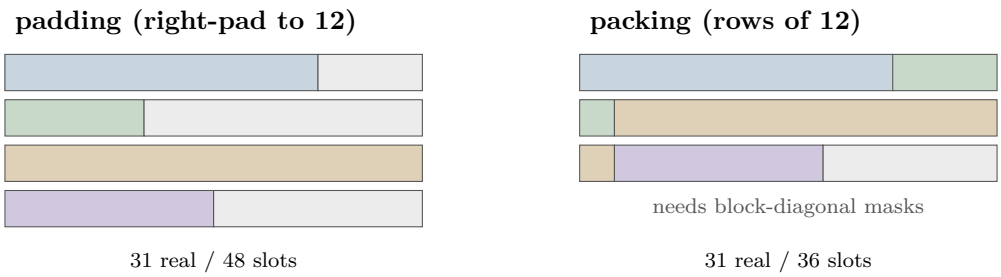


Figure 3.12: The same four examples (lengths 9, 4, 12, 6) batched by padding and by packing. Grey cells are wasted compute.

Pack documents. Real corpora are collections of documents of wildly different lengths. Padding each to T wastes compute on padding tokens. Instead, documents are concatenated with `<|endoftext|>` separators into one stream and cut into fixed windows—*packing*. A window may then contain the end of one document and the start of another. Standard GPT-style pretraining lets tokens attend across the boundary (the separator teaches the model to ignore what came before); more careful setups use block-diagonal "document masks" so attention never crosses a boundary, which FlashAttention's variable-length interface supports at no extra cost (Chapters 4 and 10).

Shuffle at several levels. A global random permutation of 10^9 windows is expensive to store and destroys locality. Practical pipelines shuffle the *order of shards*, then the order of windows within a shard, and sometimes add a streaming *shuffle buffer*. Determinism matters: for checkpoint/restart (Chapter 7) the loader must be able to resume at exactly the sample where it stopped, which is easy with index arithmetic over memmapped shards and hard with stateful streaming iterators.

Overlap loading with compute. A loader created with `num_workers=k`, `pin_memory=True` and `prefetch_factor=2` runs k worker processes that fetch batches ahead of time into page-locked host memory, from which a non-blocking `.to("cuda", non_blocking=True)` copy can overlap the previous step's compute.

3.10.1 Variable-length batches: padding versus packing

Fine-tuning and serving work with *individual* examples of different lengths (an SMS, an instruction–response pair, a chat request) rather than an endless stream. There are two ways to batch them (Fig. 3.12). *Padding* right-pads every sequence to the longest in the batch with a filler ID and masks the padding out of attention and loss. It is simple and keeps examples independent, but the padded positions still flow through every matmul. *Packing* concatenates several examples into each row, separated by end-of-text tokens, and uses document-aware masks (or position resets) so examples do not attend to each other. Packing wastes almost nothing but complicates masking, position IDs, and per-example loss normalisation.

Napkin Math: Expected padding waste

If lengths are i.i.d. uniform on $\{1, \dots, T\}$ and every batch is padded to T, the expected fraction of real tokens is $\mathbb{E}[\ell]/T = (T+1)/(2T) \approx 50\%$: half the FLOPs of every linear layer are spent on padding (attention is worse, since its cost is quadratic in the padded

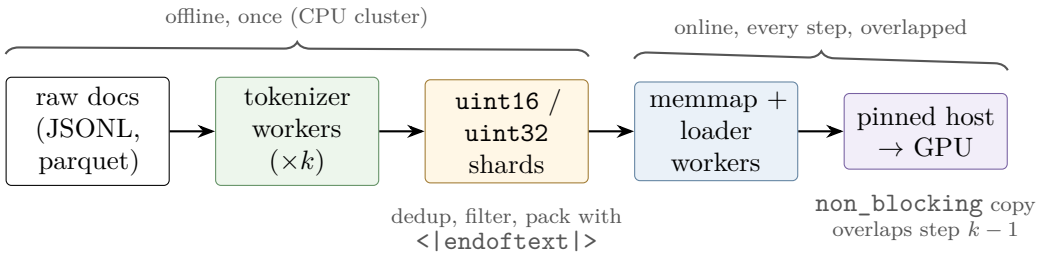


Figure 3.13: A production pretraining input pipeline. Tokenization happens once, offline; the per-step path is reduced to offset arithmetic, page-cache reads and an asynchronous host-to-device copy.

length). Padding only to the longest sequence *in the batch* helps less than one might hope for large batches: with $B = 32$, the batch maximum of uniform lengths is close to T already ($\mathbb{E}[\max] \approx T \cdot B/(B+1)$). Sorting or *bucketing* examples by length before batching (Chapter 9) recovers most of the waste without changing the model; packing recovers nearly all of it.

Napkin Math: Can the data pipeline keep up?

Suppose a cluster trains a 7B model at 40% MFU on 256 H100s. Using $6N$ FLOPs per token, the token throughput is $0.4 \cdot 256 \cdot 989 \times 10^{12}/(6 \cdot 7 \times 10^{9}) \approx 2.4$M tokens/s.

- *Reading pre-tokenized shards*: 2.4M tokens/s × 4 bytes (`uint32`) ≈ 9.6 MB/s for the whole cluster—negligible against any file system or NIC.
- *Tokenizing on the fly*: at ∼4 bytes of text per token, the cluster needs ∼9.6 MB/s of text tokenized. Our pure-Python encoder ran at ≈ 0.3 MB/s per core in this environment, so it would need about 30 dedicated cores; a Rust tokenizer at tens of MB/s per core needs one or two. Either is feasible, but CPU spent tokenizing competes with loader workers, and offline tokenization also makes runs reproducible.

The lesson: once data is pre-tokenized, I/O bandwidth is almost never the bottleneck for text LLMs; Python-level per-sample overhead (and, for multimodal models, image decoding) is.

Deeper Dive: Data quality is a systems problem too

Before tokenization, web-scale corpora go through language identification, heuristic quality filters, model-based classifiers, PII scrubbing, and deduplication. Exact dedup hashes documents; near-duplicate detection uses MinHash signatures with locality-sensitive hashing over n-gram shingles, which turns an $O(n^2)$ comparison into a bucketed join. These jobs run on CPU clusters with Spark/Ray-style frameworks and dominate the engineering effort of datasets such as FineWeb (Penedo et al., 2024). Benchmark *contamination*—test questions leaking into pretraining data—is detected with the same n-gram machinery (Chapter 14).

3.11 Putting it together: from a file to a (B, T, d) tensor

Listing 3.11 strings the chapter's components together using the shared `llmsys` package. It trains a small BPE tokenizer on a text file, tokenizes the file, builds a loader of disjoint windows,

stage	object	shape	dtype
raw text	`str`	$\approx 2 \times 10^4$ characters	—
token IDs	`list[int]`	$N \approx 5 \times 10^3$ (depends on V)	int
one batch	`x, y`	$(B, T) = (8, 4)$	`int64`
token embeddings	`tok_emb(x)`	$(8, 4, 256)$	`float32`
position embeddings	`pos_emb(arange(T))`	$(4, 256)$, broadcast	`float32`
input to block 1	`h`	$(8, 4, 256)$	`float32`

Table 3.1: Shapes through the input pipeline for the running example ($B = 8$, $T = 4$, $d = 256$).

and produces input embeddings ready for the attention layers of Chapter 4. Table 3.1 traces the shapes.

Listing 3.11: End-to-end input pipeline with the `llmsys` core modules.

```python
import torch
from llmsys.tokenizer import ByteBPETokenizer
from llmsys.data import create_dataloader

text = open("the-verdict.txt", encoding="utf-8").read()
tok = ByteBPETokenizer.train(text, vocab_size=1024)          # 256 bytes + merges
loader = create_dataloader(text, tok, batch_size=8, max_length=4, stride=4,
                           shuffle=False)
x, y = next(iter(loader))                       # (8, 4) inputs, (8, 4) targets

d, T = 256, 4
tok_emb = torch.nn.Embedding(tok.vocab_size, d)
pos_emb = torch.nn.Embedding(T, d)
h = tok_emb(x) + pos_emb(torch.arange(T))      # (8, 4, 256)
```

Deeper Dive: Do we need tokenizers at all?

Tokenization is a fixed, non-learned compression stage bolted onto an otherwise end-to-end model, and many artefacts above stem from it. Byte-level models avoid it by operating directly on bytes, at the cost of sequences about four times longer. Hierarchical architectures recover efficiency by grouping bytes into patches processed by a large "global" model and small "local" models, e.g. MegaByte (Yu et al., 2023), or by forming patches dynamically from next-byte entropy, as in the Byte Latent Transformer (Pagnoni et al., 2024). For now, every widely deployed LLM—including everything in this book—uses a subword tokenizer.

3.12 Summary

- LLMs consume integer token IDs, which an embedding table turns into d-dimensional vectors. The table is learned jointly with the model; its forward pass is a gather and its gradient a sparse scatter-add.
- A tokenizer splits text and maps pieces to IDs through a vocabulary. Word-level tokenizers need `<|unk|>` for unseen words and a huge vocabulary for coverage.
- Byte-level BPE starts from 256 bytes and greedily merges frequent pairs; it can encode any input, keeps whitespace, and is what GPT-2 (50,257 tokens) and Qwen3 (151,936) use. Encoding applies merges in training order.

- Vocabulary size trades parameters and logit cost against sequence length; for large vocabularies the training-time logits tensor can exceed all other memory.
- Training examples are windows of T tokens with targets shifted by one; stride controls overlap. Datasets/DataLoaders batch them into (B, T) tensors.
- Position must be injected explicitly. GPT-2 adds learned absolute position embeddings; modern models use rotary embeddings (Chapter 6).
- At scale: tokenize offline into binary shards, memory-map them, pack documents with separators, shuffle hierarchically, and overlap loading with compute.

3.13 Exercises

Exercise 3.1 (*stride arithmetic*). A corpus has N tokens. Give a closed form for the number of (x, y) windows produced by `SlidingWindowDataset` with context length T and stride s, and for the total number of *input token positions* processed per epoch. How many times is a typical token seen as an input per epoch when $s = 1$? when $s = T/4$?

Exercise 3.2 (*tokenizer round-trip*). Construct three strings for which the word-level `WordTokenizer` of Listing 3.2 fails `decode(encode(s)) == s` even when every word is in the vocabulary. Explain why byte-level BPE does not have this problem.

Exercise 3.3 (*unknown words in BPE*). Using a BPE tokenizer trained on English prose, encode a string of random lowercase letters of length 20 and a 20-character word in a non-Latin script. Report bytes per token for each and explain the difference in terms of the merge list.

Exercise 3.4 (*merge rank, not frequency*). Explain why `bpe_encode` must apply merges in the order they were learned rather than, say, greedily merging the most frequent pair *in the input string*. Give a concrete merge list and input where the two strategies produce different token sequences.

Exercise 3.5 (*complexity of incremental BPE*). Let W be the total number of symbols across unique pre-token chunks, M the number of merges, and a_m the number of chunks affected by merge m. Give the time complexity of the naive and incremental trainers. What additional data structure reduces the "find the best pair" step from $O(P)$ to $O(\log P)$, and what complication do decreasing counts introduce?

Exercise 3.6 (*vocabulary trade-off*). For a model with $d = 2048$, $L = 24$ layers, and a fixed budget of 10^{11} training tokens of English text, compare $V = 32$k and $V = 256$k, assuming the larger vocabulary compresses text 15% better. Estimate (a) embedding parameters (tied), (b) logits FLOPs per token, (c) the number of training *tokens* needed to cover the same text, and (d) the size of the fp32 logits tensor for $B = 4$, $T = 8192$. Which choice would you make for a 0.5B model? For a 70B model?

Exercise 3.7 (*embedding gradients*). Write a test that shows `emb.weight.grad` is non-zero only in rows whose IDs occur in the batch, and that a row whose ID occurs k times receives the

sum of k gradient vectors. How does `nn.Embedding(..., sparse=True)` change the gradient's representation, and which optimisers accept it?

Exercise 3.8 (*positional permutation test*). Build a single self-attention layer without positional information and verify numerically that permuting the input rows permutes the output rows identically. Then add learned position embeddings and show the property disappears.

Exercise 3.9 (*sinusoids*). Show that for the sinusoidal encoding there is a linear map $\mathbf{M}_k$, independent of p, with $\boldsymbol{p}_{p+k} = \mathbf{M}_k \boldsymbol{p}_p$. Why does this make it plausible for a network to learn relative-position behaviour from absolute sinusoidal encodings?

Exercise 3.10 (*document masks*). Packing places several documents into one window. Write a function that, given a window's token IDs and the end-of-text ID, returns the (T, T) boolean mask that is causal *and* block-diagonal by document. How many attention scores does it save, relative to the plain causal mask, for a window with four documents of equal length?

Exercise 3.11 (*pipeline measurement*). Using `labs/lab03_tokenizer/bench.py`, measure random-access window reads per second from a memmapped shard with (a) a warm page cache and (b) after dropping caches (Linux: `echo 3 > /proc/sys/vm/drop_caches`, as root). Relate the cold-cache number to your disk's random-read IOPS.

3.14 Lab 3: Tokenizers and the input pipeline

Lab 3: Tokenizers and the input pipeline

Code: `code/labs/lab03_tokenizer`

Goals

Build every stage between a text file and a (B, T, d) tensor yourself, and measure the costs that matter at scale: tokenizer training time, encode throughput, compression, and shard access rate.

Setup

```
cd code
pytest -q labs/lab03_tokenizer                    # runs against starter.py
LAB_IMPL=solution pytest -q labs/lab03_tokenizer  # reference
```

Each task corresponds to one stubbed function in `starter.py`; docstrings give the contract and a hint.

Tasks

1. **Word-level tokenizer.** Implement `split_text`, `build_vocab`, `WordTokenizer.encode/decode` with `<|unk|>` fallback. The tests check punctuation handling and round-trips.
2. **BPE primitives.** Implement `get_pair_counts` (frequency-weighted) and `merge_pair` (left-to-right, non-overlapping: `[1,1,1]` with pair `(1,1)` becomes `[7,1]`).
3. **BPE training and coding.** Implement `train_bpe`, `bpe_encode` (apply merges by rank), `bpe_decode`, `compression_ratio`. Round-trip must be exact for arbitrary Unicode.

4. **Incremental trainer.** Implement `train_bpe_fast` with pair counts plus an inverted index. It must return *exactly* the same merges as `train_bpe` (deterministic tie-breaking is provided by `_best_pair`).
5. **Sampling and storage.** Implement `sliding_windows`, `pack_documents`, `write_shard` (narrowest dtype) and `ShardDataset.__getitem__`.
6. **Input layer.** Implement `InputEmbedding.forward` and `embedding_as_matmul`.

Measurements

Run `bench.py` (optionally `-text` on a real corpus such as a public-domain book):

- training time of naive vs. incremental BPE as a function of `-vocab`; plot both. Expect the gap to grow with vocabulary size and corpus diversity;
- encode throughput (MB/s) and bytes/token of your tokenizer, and of `tiktoken`'s GPT-2 encoding if available. Expect the Rust tokenizer to be one to two orders of magnitude faster;
- random window reads per second from the memmapped shard.

Deliverables

All tests passing; a short table of the measurements; a two-sentence answer to: *at what cluster token throughput would on-the-fly tokenization with your implementation need more CPU cores than a typical 8-GPU node has?*

Stretch goals

- Replace `_best_pair`'s linear scan with a max-heap using lazy invalidation; verify identical merges.
- Add document-boundary-aware window sampling that returns, with each window, the block-diagonal mask from Section 3.13.
- Load GPT-2's real merges (from `tiktoken`'s files) into `ByteBPETokenizer` and check token-for-token agreement on a test corpus (requires network access on your machine).

Chapter 4

Attention Mechanisms, from Dot Products to FlashAttention

Attention is the one operation in a transformer that mixes information *across* positions. Everything else—embeddings, layer norms, MLPs—acts on each token independently. It is also the one operation whose cost grows quadratically with sequence length, and so the one that dominates long-context training, shapes the KV cache that dominates serving memory, and motivated the most influential systems paper of the transformer era, FlashAttention.

This chapter builds attention up in small steps: a weight-free version that exposes the core "weighted average of the inputs" idea, then trainable queries, keys and values, then causal masking and dropout, then multi-head attention in two implementations. With the algorithm in hand, we analyse its FLOPs and memory traffic, derive the online softmax, and implement an IO-aware tiled attention whose forward and backward passes match the naive version to machine precision without ever forming the $T \times T$ attention matrix.

4.1 The problem with modeling long sequences

LLM 3.1

Consider translating a sentence. Word-by-word substitution fails immediately: languages reorder words, split one word into several, and resolve meanings from context far away in the sentence. A translation model must read the whole source, keep what matters, and consult the relevant parts as it writes each output word.

Before transformers, the standard architecture was the *encoder–decoder recurrent neural network* (Sutskever et al., 2014; Cho et al., 2014). The encoder RNN reads the source tokens one at a time, updating a hidden state $\boldsymbol{h}_t = f(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$. Its final hidden state—a single fixed-size vector—is handed to the decoder, which generates the output one token at a time from that vector and its own running state (Fig. 4.2, left).

The weakness is the bottleneck: the entire meaning of an arbitrarily long input must be squeezed through one vector of a few hundred numbers, and the decoder has no way to look back at individual input positions. Empirically, translation quality fell off sharply for long sentences. A second, systems-flavoured weakness is that the recurrence is *sequential*: $\boldsymbol{h}_t$ cannot be computed before $\boldsymbol{h}_{t-1}$, so training cannot parallelise across the time dimension of a sequence, leaving most of a GPU idle.

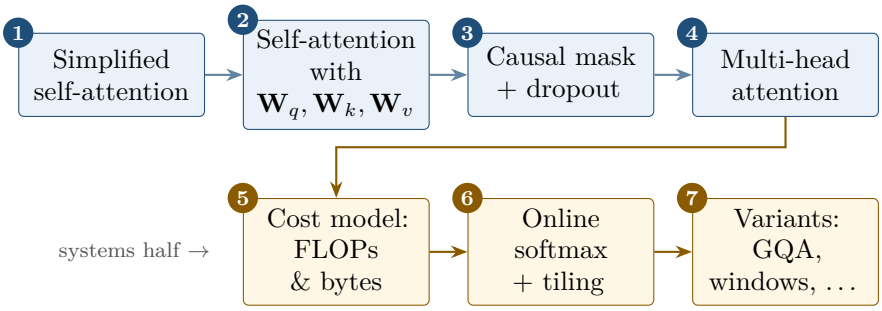


Figure 4.1: Roadmap: steps 1–4 build the attention used in GPT-2 (blue); steps 5–7 analyse and optimise it (amber).

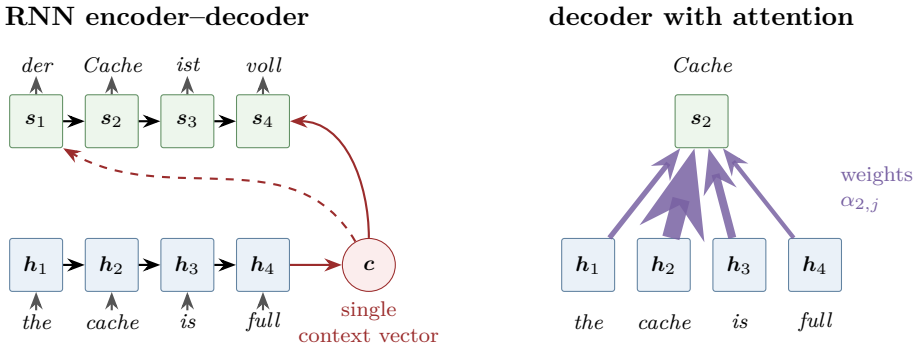


Figure 4.2: Left: the encoder–decoder RNN must compress the whole input into one vector $\boldsymbol{c}$. Right: with attention, each decoder step forms its own weighted combination of *all* encoder states; line width shows how strongly each input is used to produce "Cache".

4.2 Capturing data dependencies with attention

LLM 3.2

Bahdanau et al. (2015) removed the bottleneck by letting the decoder, at every output step, compute a fresh set of weights over *all* encoder hidden states and use their weighted sum as context (Fig. 4.2, right). The weights are produced by a small learned scoring function and normalised with a softmax, so the decoder can "attend" to whichever input positions are relevant to the word it is producing.

Three years later, Vaswani et al. (2017) asked what happens if one drops the recurrence entirely and builds the whole network out of attention. The resulting *transformer* uses *self-attention*: a sequence attends to *itself*, so every position's new representation is a learned, data-dependent mixture of every (allowed) position's old representation. Without recurrence, all positions of a sequence are processed in parallel by large matrix multiplications, which is precisely what GPUs are built for. That property, more than any modelling advantage, is why transformers displaced RNNs.

Key Idea: Self-attention in one sentence

For each position i, compute a probability distribution $\alpha_{i,\cdot}$ over positions, based on how well i's "question" matches each position's "label", and output the α-weighted average of the positions' "contents". Queries, keys and values are those three roles.

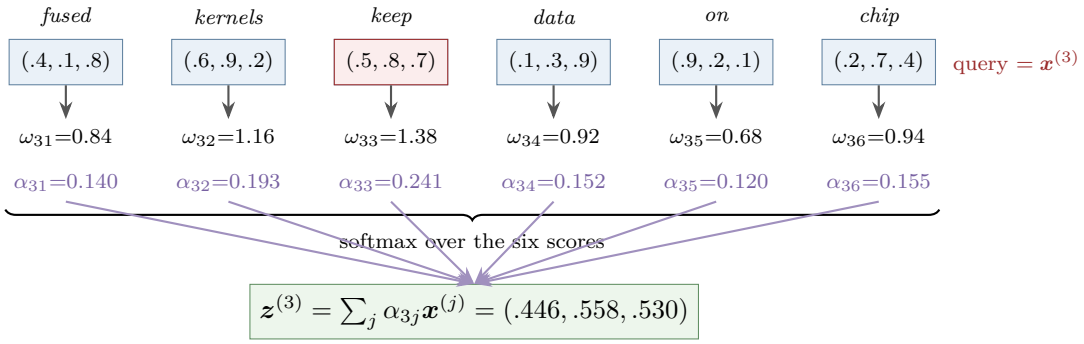


Figure 4.3: Weight-free self-attention for the query "keep": dot-product scores against every input, softmax to weights, weighted sum to the context vector.

4.3 Self-attention without trainable weights

LLM 3.3

To isolate the mechanism, we first drop all learned parameters. Take a six-token input whose 3-dimensional embeddings we simply write down (real models use d in the hundreds or thousands and learn the embeddings, Chapter 3):

Listing 4.1: The running example: six tokens with toy 3-d embeddings.

```
inputs = torch.tensor(
  [[0.40, 0.10, 0.80],   # fused     (x1)
   [0.60, 0.90, 0.20],   # kernels   (x2)
   [0.50, 0.80, 0.70],   # keep      (x3)
   [0.10, 0.30, 0.90],   # data      (x4)
   [0.90, 0.20, 0.10],   # on        (x5)
   [0.20, 0.70, 0.40]])  # chip      (x6)
```

4.3.1 One query at a time

LLM 3.3.1

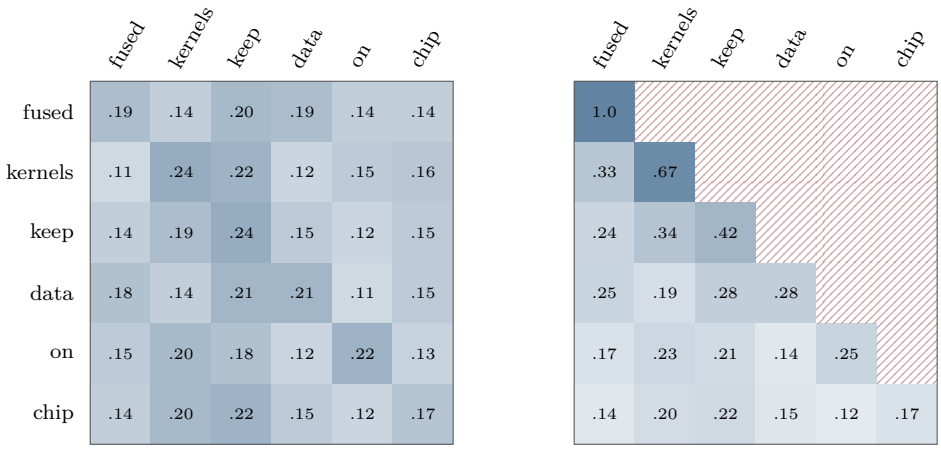
Pick the third token, "keep", as the *query*, $\boldsymbol{q} = \boldsymbol{x}^{(3)}$. Self-attention computes its *context vector* $\boldsymbol{z}^{(3)}$ in three steps.

1. **Scores.** Measure the similarity of the query to every input with a dot product: $\omega_{3j} = \boldsymbol{x}^{(3)} \cdot \boldsymbol{x}^{(j)}$. A dot product is large when two vectors point the same way and have large norms, so it serves as an un-normalised relevance score. For our numbers: $\boldsymbol{\omega}_3 = (0.84,\ 1.16,\ 1.38,\ 0.92,\ 0.68,\ 0.94)$.

2. **Weights.** Normalise the scores into a probability distribution with a softmax, $\alpha_{3j} = e^{\omega_{3j}} / \sum_k e^{\omega_{3k}}$, giving $\boldsymbol{\alpha}_3 = (0.140, 0.193, 0.241, 0.152, 0.120, 0.155)$. Softmax keeps every weight positive, makes them sum to one, and exaggerates differences (it is a smooth arg-max).

3. **Context.** Average the inputs with those weights: $\boldsymbol{z}^{(3)} = \sum_j \alpha_{3j} \boldsymbol{x}^{(j)} = (0.446,\ 0.558,\ 0.530)$.

Figure 4.3 draws the computation. Note that the query attends most strongly to itself (its dot product with itself is its squared norm), a pattern that trainable projections will be free to change.



(a) weight-free, all keys (b) causal (Section 4.5)

Figure 4.4: Attention weight matrices $\mathbf{A}$ for the running example (rows: queries, columns: keys; numbers computed with Listing 4.2). Each row sums to 1. In (b) the hatched future positions are masked and each row is renormalised over the visible prefix.

Pitfall: Never write softmax as `exp(x)/exp(x).sum()`

A score of 100 overflows `exp` in fp16 (max ≈ 65504) and one of 1000 overflows fp32. Since $\mathrm{softmax}(\boldsymbol{x}) = \mathrm{softmax}(\boldsymbol{x} - c)$ for any constant c, numerically stable implementations subtract the row maximum first. `torch.softmax` does this; so must every hand-written kernel. The running maximum is also the key to the online softmax of Section 4.8.

4.3.2 All queries at once

LLM 3.3.2

Repeating the procedure for every query is just three matrix operations on the input matrix $\mathbf{X} \in \mathbb{R}^{T \times d}$:

$$\boldsymbol{\Omega} = \mathbf{X}\mathbf{X}^{\top} \in \mathbb{R}^{T \times T}, \qquad \mathbf{A} = \mathrm{softmax}_{\mathrm{row}}(\boldsymbol{\Omega}), \qquad \mathbf{Z} = \mathbf{A}\mathbf{X} \in \mathbb{R}^{T \times d}. \tag{4.1}$$

Row i of $\mathbf{A}$ is the distribution of query i over all keys (Fig. 4.4). In PyTorch it is three lines, and replacing Python loops with these matrix operations is the first and most important performance optimisation in this chapter: one $T \times d$ by $d \times T$ GEMM instead of T^2 tiny dot products.

Listing 4.2: Weight-free self-attention for all tokens (Lab 4, Part A).

```
def simple_attention(x):                # x: (T, d)
    scores = x @ x.T                    # (T, T) all pairwise dot products
    weights = torch.softmax(scores, dim=-1)   # each row sums to 1
    return weights @ x, weights         # (T, d) context vectors
```

4.4 Self-attention with trainable weights

LLM 3.4

The weight-free version has no parameters to learn: what a token attends to is fixed by raw embedding similarity, and what it retrieves is the raw embedding itself. *Scaled dot-product attention* (Vaswani et al., 2017) introduces three learned projection matrices $\mathbf{W}_q, \mathbf{W}_k \in \mathbb{R}^{d_{\mathrm{in}} \times d_k}$ and $\mathbf{W}_v \in \mathbb{R}^{d_{\mathrm{in}} \times d_v}$ that give each token three separate roles:

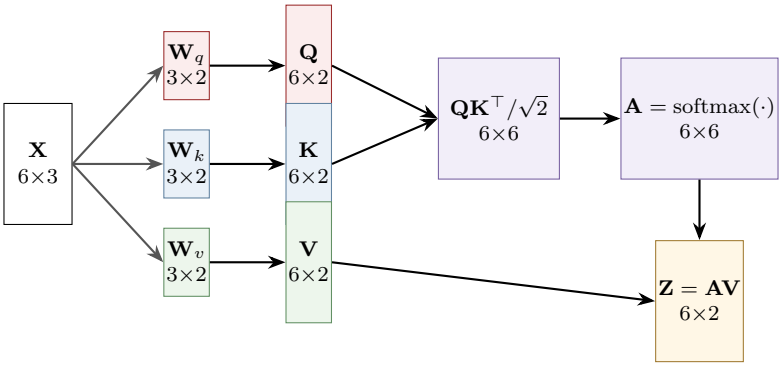


Figure 4.5: Shapes in single-head scaled dot-product attention for the running example ($T = 6$, $d_{\text{in}} = 3$, $d_k = d_v = 2$).

- **query** $\boldsymbol{q}_i = \boldsymbol{x}_i \mathbf{W}_q$ — what token i is looking for;
- **key** $\boldsymbol{k}_j = \boldsymbol{x}_j \mathbf{W}_k$ — what token j advertises about itself;
- **value** $\boldsymbol{v}_j = \boldsymbol{x}_j \mathbf{W}_v$ — what token j contributes if attended to.

The terminology comes from databases and key–value stores: a query is matched against keys, and the matching entries' values are returned—except that here matching is soft and every value is returned with some weight.

4.4.1 Computing the attention weights step by step

LLM 3.4.1

For the full input,

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_k, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_v, \qquad \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\Big)\mathbf{V}. \quad (4.2)$$

Figure 4.5 traces the shapes for our example with $d_{\text{in}} = 3$ and $d_k = d_v = 2$ (deliberately different so the shapes cannot be confused; in GPT models $d_{\text{in}} = d_{\text{out}}$).

Why divide by $\sqrt{d_k}$? Suppose the entries of $\boldsymbol{q}$ and $\boldsymbol{k}$ are independent with mean 0 and variance 1. Then $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{m=1}^{d_k} q_m k_m$ has mean 0 and variance d_k. With $d_k = 128$, raw scores have a standard deviation of about 11; a softmax over such scores is nearly one-hot, and its gradient (which is proportional to $\alpha_i(1-\alpha_i)$ on the diagonal) nearly vanishes. Dividing by $\sqrt{d_k}$ restores unit variance at initialisation and keeps the softmax in a regime where gradients flow. (Some newer models instead normalise $\boldsymbol{q}$ and $\boldsymbol{k}$ directly—the QK-norm of Qwen3, Chapter 6.)

4.4.2 A compact self-attention class

LLM 3.4.2

Wrapping the computation in an `nn.Module` gives us trainable parameters, device movement and state-dict serialisation for free:

Listing 4.3: Single-head self-attention with trainable projections (Lab 4, Part B).

```
class SelfAttention(nn.Module):
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key   = nn.Linear(d_in, d_out, bias=qkv_bias)
```

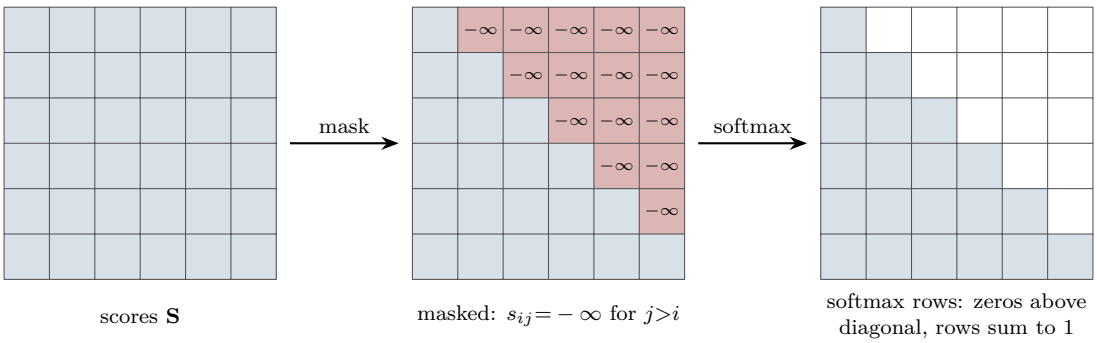


Figure 4.6: Causal masking applied before the softmax. Only the lower triangle (including the diagonal) receives probability mass.

```
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):                          # (..., T, d_in)
        q, k, v = self.W_query(x), self.W_key(x), self.W_value(x)
        scores = q @ k.transpose(-2, -1) / math.sqrt(k.shape[-1])
        return torch.softmax(scores, dim=-1) @ v    # (..., T, d_out)
```

A first version might store the projections as raw `nn.Parameter(torch.rand(d_in, d_out))` and compute `x @ W`. Using `nn.Linear` instead changes two things. First, `nn.Linear` stores its weight as $(d_{\text{out}}, d_{\text{in}})$ and computes $\boldsymbol{x}\mathbf{W}^\top$—the two versions compute the same function if one copies `W.T` into the linear layer, which is a useful test. Second, `nn.Linear` uses a scaled (Kaiming-uniform) initialisation, whereas `torch.rand` draws from $[0, 1)$; all-positive, unscaled weights make scores grow with d and training less stable.

4.5 Hiding future words with causal attention

LLM 3.5

A language model is trained to predict token $t+1$ from tokens $1..t$. If position t could attend to position $t+1$ during training, the task would be trivial (copy the answer) and the model would learn nothing useful for generation, where the future does not yet exist. *Causal* (or *masked*) attention restricts query i to keys $j \le i$.

4.5.1 Applying a causal attention mask

LLM 3.5.1

There are two equivalent ways to apply the mask (Fig. 4.6).

1. *Mask after softmax, then renormalise*: compute $\mathbf{A} = \text{softmax}(\mathbf{S})$, zero the entries above the diagonal, and divide each row by its new sum.

2. *Mask before softmax*: set $s_{ij} = -\infty$ for $j > i$, then apply softmax. Since $e^{-\infty} = 0$ the masked entries get exactly zero weight and the remaining entries are already normalised.

They agree exactly: in route 1, row i's surviving entries are $e^{s_{ij}}/Z$ with $Z = \sum_k e^{s_{ik}}$ over *all* k; renormalising divides by $\sum_{k \le i} e^{s_{ik}}/Z$, and the full-row Z cancels. In particular, the future *does not leak*: although route 1's first softmax looks at future scores, their influence is removed entirely by the renormalisation. Route 2 is what every implementation uses—it is one fused operation, does less work, and is what FlashAttention exploits to skip masked tiles entirely.

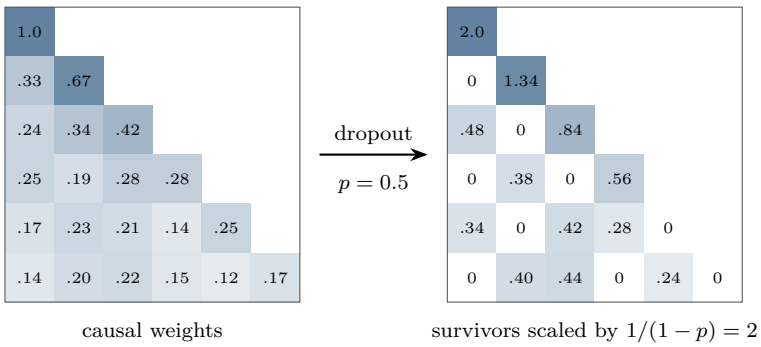


Figure 4.7: Dropout on the attention weights (one random draw, illustrative). Zeros are dropped connections; surviving weights double so each row's *expected* sum is still 1.

Listing 4.4: Two routes to causal weights; they are equal to float precision.

```
def causal_softmax(scores):                          # route 2: -inf before softmax
    T = scores.shape[-1]
    mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)
    return torch.softmax(scores.masked_fill(mask, float("-inf")), dim=-1)

def mask_after_softmax(scores):                      # route 1: zero, renormalise
    w = torch.tril(torch.softmax(scores, dim=-1))
    return w / w.sum(dim=-1, keepdim=True)
```

4.5.2 Masking additional attention weights with dropout

LLM 3.5.2

Dropout zeroes a random subset of activations during training (each with probability p) and scales the survivors by $1/(1-p)$ so expected values are unchanged; at evaluation time it is the identity. In GPT-2, dropout is applied in three places, one of them to the attention weight matrix after the softmax, so that a token cannot rely too heavily on any single other position (Fig. 4.7). With $p = 0.5$ a weight of 0.28 becomes either 0 or 0.56, and a row no longer sums exactly to 1—only in expectation.

Systems Lens: Dropout is not free in fused kernels

In eager PyTorch, attention dropout materialises a $B \times H \times T \times T$ random mask and stores it for the backward pass—as large as the attention matrix itself. FlashAttention instead generates dropout bits on the fly from a counter-based RNG (Philox) keyed by a seed and an offset, and *regenerates* identical bits in the backward pass, storing nothing. This is why fused kernels take a seed/offset rather than a mask, and why bitwise reproducibility of dropout across different kernels or tile sizes is not guaranteed. Most modern LLMs pretrain with attention dropout set to 0 anyway; with trillions of tokens and a single epoch, overfitting is not the concern and the extra RNG work is wasted.

4.5.3 Implementing a compact causal attention class

LLM 3.5.3

Two practical changes turn the above into a reusable layer: it must accept batches of shape (B, T, d_{in}), and the mask should be created once and moved with the module. Registering the mask with `register_buffer` does the latter: buffers are moved by `.to(device)` with the parameters but are not trained. We register it with `persistent=False` so it is not written to checkpoints (it can always be recomputed).

Listing 4.5: Batched causal attention with dropout (Lab 4, Part C).

```python
class CausalAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout=0.0,
                 qkv_bias=False):
        super().__init__()
        self.d_out = d_out
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key   = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)
        mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
        self.register_buffer("mask", mask.bool(), persistent=False)

    def forward(self, x):                                   # (B, T, d_in)
        B, T, _ = x.shape
        q, k, v = self.W_query(x), self.W_key(x), self.W_value(x)
        scores = q @ k.transpose(1, 2) / math.sqrt(self.d_out)  # (B, T, T)
        scores = scores.masked_fill(self.mask[:T, :T], float("-inf"))
        w = self.dropout(torch.softmax(scores, dim=-1))
        return w @ v                                         # (B, T, d_out)
```

The slice `self.mask[:T, :T]` lets the layer handle any length up to `context_length`. A good test of causality (Lab 4) perturbs the inputs at positions $\geq t$ and checks that outputs at positions $< t$ are bit-for-bit unchanged.

4.6 Extending single-head attention to multi-head attention

LLM 3.6

A single head computes one attention pattern per query. But a token typically needs several kinds of information at once—the subject it agrees with, the noun a pronoun refers to, the previous token, the matching bracket. *Multi-head attention* runs H attention heads in parallel, each with its own projections into a smaller $d_h = d_{\text{out}}/H$-dimensional subspace, concatenates their outputs, and mixes them with an output projection $\mathbf{W}_o$:

$$\text{MHA}(\mathbf{X}) = \big[\ \text{head}_1 \ \|\ \cdots \ \|\ \text{head}_H\ \big]\mathbf{W}_o, \qquad \text{head}_h = \text{Attention}(\mathbf{X}\mathbf{W}_q^{(h)}, \mathbf{X}\mathbf{W}_k^{(h)}, \mathbf{X}\mathbf{W}_v^{(h)}). \tag{4.3}$$

The total parameter count and FLOPs match one big head of width d_{out}; what changes is that the model gets H independent attention distributions per token.

4.6.1 Stacking multiple single-head attention layers

LLM 3.6.1

The literal implementation keeps a list of `CausalAttention` modules and concatenates their outputs along the feature dimension. With $d_{\text{out}} = 2$ per head and $H = 2$ heads, the output is $(B, T, 4)$.

Listing 4.6: Multi-head attention as a list of heads.

```python
class MultiHeadAttentionWrapper(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads,
                 qkv_bias=False):
        super().__init__()
        self.heads = nn.ModuleList(
            [CausalAttention(d_in, d_out, context_length, dropout, qkv_bias)
```

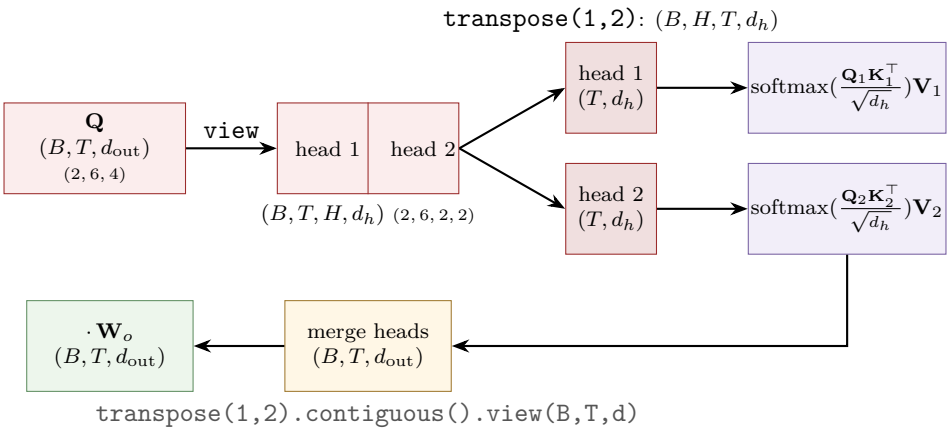


Figure 4.8: Weight-split multi-head attention: one projection, then reshape into heads, batched attention over (B, H), merge, output projection. Same shapes for $\mathbf{K}$ and $\mathbf{V}$.

```
            for _ in range(num_heads)])

    def forward(self, x):
        return torch.cat([h(x) for h in self.heads], dim=-1)   # (B, T, H*d_out)
```

4.6.2 Implementing multi-head attention with weight splits

LLM 3.6.2

The wrapper runs the heads one after another: $3H$ small projection GEMMs, H score GEMMs, H softmaxes and H value GEMMs, each a separate kernel launch on a GPU. The standard implementation instead uses *one* $d_{\text{in}} \times d_{\text{out}}$ projection each for $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and splits the result into heads by reshaping (Fig. 4.8): $(B, T, d_{\text{out}}) \xrightarrow{\text{view}} (B, T, H, d_h) \xrightarrow{\text{transpose}} (B, H, T, d_h)$. Batched matrix multiplication then treats (B, H) as batch dimensions, so all heads are computed by single kernels.

Listing 4.7: Multi-head attention with weight splits (Lab 4, Part D; same parameter names as `llmsys.layers.MultiHeadAttention`).

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, num_heads,
                 qkv_bias=False):
        super().__init__()
        assert d_out % num_heads == 0
        self.d_out, self.num_heads = d_out, num_heads
        self.head_dim = d_out // num_heads
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key   = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out)      # mixes the heads
        self.dropout = nn.Dropout(dropout)
        mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
        self.register_buffer("mask", mask.bool(), persistent=False)

    def forward(self, x):
        B, T, _ = x.shape
        H, Dh = self.num_heads, self.head_dim
        split = lambda t: t.view(B, T, H, Dh).transpose(1, 2)    # (B, H, T, Dh)
        q, k, v = (split(f(x)) for f in (self.W_query, self.W_key, self.W_value))
```

```
        scores = q @ k.transpose(2, 3) / math.sqrt(Dh)          # (B, H, T, T)
        scores = scores.masked_fill(self.mask[:T, :T], float("-inf"))
        w = self.dropout(torch.softmax(scores, dim=-1))
        ctx = (w @ v).transpose(1, 2).contiguous().view(B, T, self.d_out)
        return self.out_proj(ctx)
```

Note the `.contiguous()` before the final `view`: after `transpose` the tensor's memory layout no longer matches its logical shape, and `view` requires compatible strides. `.contiguous()` performs a physical copy into the merged-head layout (alternatively, `.reshape` copies only when needed). For GPT-2 small, $H = 12$ heads of $d_h = 64$ give $d_{\text{out}} = 768$; GPT-2 XL uses 25 heads of 64 for $d = 1600$.

Systems Lens: Why weight splitting is faster: kernel granularity

For GPT-2 small ($d = 768$, $H = 12$) the wrapper performs 36 projections of size $(BT \times 768) \cdot (768 \times 64)$; the weight-split version performs 3 of size $(BT \times 768) \cdot (768 \times 768)$—the same FLOPs. But a 768×64 weight gives each GEMM far fewer output tiles to spread over a GPU's 100+ streaming multiprocessors, so many SMs idle, and each call pays a few microseconds of launch overhead. GPT-2's original code goes one step further and fuses Q, K and V into a single 768×2304 projection (`c_attn`), which is why the weight loader in `llmsys/gpt.py` splits it in three. The general lesson—*batch small operations into large ones*—reappears at every level of this book, from heads to requests (Chapter 12).

4.7 The cost of attention

We now treat attention as a workload. Let B be the batch size, T the sequence length, H the number of heads, d_h the head dimension and $d = Hd_h$ the model width.

FLOPs. Counting a multiply–add as 2 FLOPs, one forward pass of one attention layer costs

$$\text{projections } (\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{W}_o): \; 4 \cdot 2BTd^2 = 8BTd^2, \tag{4.4}$$

$$\mathbf{QK}^\top \text{ and } \mathbf{AV}: \; 2 \cdot 2BHT^2 d_h = 4BT^2 d. \tag{4.5}$$

The quadratic term overtakes the projections when $4BT^2 d > 8BTd^2$, i.e. when $T > 2d$: around 1.5k tokens for GPT-2 small, 8k for a $d = 4096$ model. Causal masking halves the useful work of the quadratic term (only the lower triangle matters), which a good kernel exploits.

Table 4.1 instantiates these formulas for one GPT-2-small attention layer. Even at GPT-2's modest context of 1024, the score and value products are a third of the layer's FLOPs, and the materialised attention matrix is the largest activation in the layer by a wide margin.

Memory. The naive implementation materialises the scores $\mathbf{S}$ and probabilities $\mathbf{A}$, each $B \cdot H \cdot T^2$ elements, and keeps $\mathbf{A}$ (and a dropout mask) for the backward pass. Everything else is $O(BTd)$.

Napkin Math: The T^2 wall

Take one layer of a model with $H = 32$ heads at $T = 8192$, $B = 1$, bf16 (2 bytes): $32 \cdot 8192^2 \cdot 2 = 4.3\,\text{GB}$ for one copy of the attention matrix. Training must keep it for

quantity (one layer, $B = 8$, $T = 1024$, $d = 768$, $H = 12$)	formula	value	share
Q, K, V, O projections (fwd FLOPs)	$8BTd^2$	38.7 GFLOP	67%
$\mathbf{QK}^\top$ and $\mathbf{AV}$ (fwd FLOPs, dense)	$4BT^2d$	25.8 GFLOP	33%
input / output activations (fp32)	$BTd \cdot 4\,\mathrm{B}$	25.2 MB each	
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$ (fp32)	$3BTd \cdot 4\,\mathrm{B}$	75.5 MB	
attention matrix $\mathbf{A}$ (fp32), kept for backward	$BHT^2 \cdot 4\,\mathrm{B}$	403 MB	

Table 4.1: FLOPs and activation memory of one GPT-2-small attention layer (analytic). The T^2 terms are already significant at $T = 1024$; they dominate beyond $T \approx 2d$.

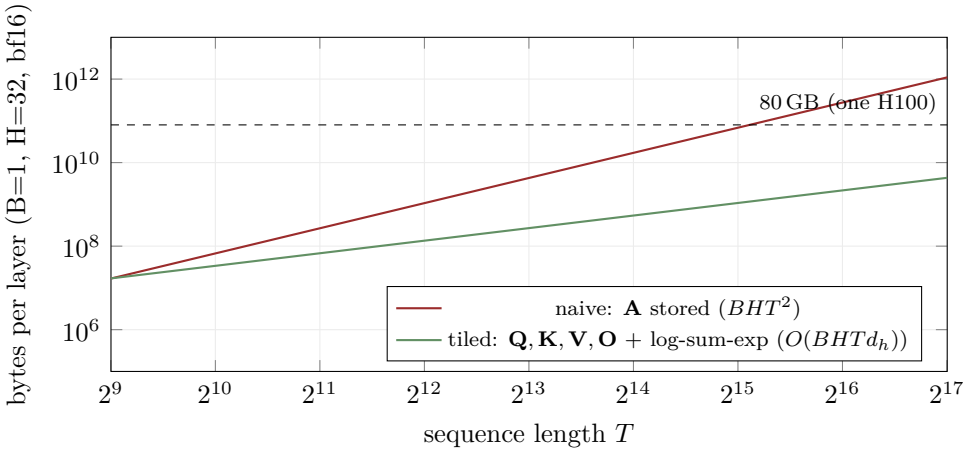


Figure 4.9: Memory needed per attention layer for the backward pass (analytic). The naive implementation stores the $T \times T$ probabilities; IO-aware attention stores only inputs, output and one scalar per row.

backward in *every* layer: 32 layers would need ≈ 137 GB—more than an 80 GB H100 holds, for a single sequence. At $T = 32{,}768$ one layer alone needs ≈ 69 GB. Long-context training with materialised attention is simply impossible.

Arithmetic intensity. Is naive attention compute-bound or memory-bound? Per head, $\mathbf{QK}^\top$ performs $2T^2d_h$ FLOPs and must at minimum write T^2 scores. The mask, scaling, softmax and dropout are elementwise passes over T^2 values with only a handful of FLOPs each. With $d_h = 128$ the score GEMM itself is fine, but the elementwise passes have an arithmetic intensity of order 1 FLOP/byte, far below the hundreds of FLOPs/byte an H100 needs to be compute-bound (Chapter 2).

Napkin Math: Naive attention on an H100: where the time goes

One head, $T = 8192$, $d_h = 128$, bf16. Matmul FLOPs: $4T^2d_h = 34.4\,\mathrm{GFLOP}$, i.e. $\approx 35\,\mu\mathrm{s}$ at 989 TFLOP/s. If the unfused implementation makes about four round trips of the $T \times T$ matrix through HBM (write $\mathbf{S}$; read $\mathbf{S}$, write $\mathbf{A}$; read $\mathbf{A}$ for $\mathbf{AV}$), traffic is $4 \cdot T^2 \cdot 2\,\mathrm{B} \approx 537\,\mathrm{MB}$, i.e. $\approx 160\,\mu\mathrm{s}$ at 3.35 TB/s. The kernel spends about 4.6× longer moving the attention matrix than computing with it. Avoiding those round trips—not reducing FLOPs—is the goal.

4.8 IO-aware attention: the online softmax and FlashAttention

FlashAttention (Dao et al., 2022) computes *exact* attention—the same numbers as Eq. (4.2) up to floating-point reassociation—without writing $\mathbf{S}$ or $\mathbf{A}$ to HBM. It rests on two ideas: *tiling*, so that blocks of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ fit in on-chip SRAM (shared memory and registers), and the *online softmax*, which makes it possible to normalise rows whose entries are seen one block at a time.

4.8.1 The online softmax

The softmax of a row $\boldsymbol{x}$ needs its maximum $m = \max_j x_j$ (for stability) and normaliser $\ell = \sum_j e^{x_j - m}$—global quantities over the whole row. The trick, due to Milakov and Gimelshein (2018), is that both can be maintained incrementally. Suppose we have processed a prefix of the row and hold (m, ℓ) for it. A new block $\boldsymbol{x}_B$ arrives:

$$m' = \max\left(m,\ \max_j x_{B,j}\right), \qquad \ell' = \ell\, e^{m - m'} + \sum_j e^{x_{B,j} - m'}. \tag{4.6}$$

The factor $e^{m-m'} \le 1$ *rescales* the old normaliser from the old reference maximum to the new one. After the last block, (m, ℓ) are exactly the global values.

Listing 4.8: Online softmax over blocks (Lab 4, Part E).

```
def online_softmax(x, block):
    m = torch.full(x.shape[:-1], float("-inf"))     # running max
    l = torch.zeros(x.shape[:-1])                   # running normaliser
    for s in range(0, x.shape[-1], block):
        xb = x[..., s:s + block]
        m_new = torch.maximum(m, xb.max(dim=-1).values)
        l = l * torch.exp(m - m_new) \
            + torch.exp(xb - m_new[..., None]).sum(-1)
        m = m_new
    return torch.exp(x - m[..., None]) / l[..., None]   # second pass
```

Listing 4.8 still makes a second pass to emit probabilities. Attention does not need the probabilities themselves, only $\mathbf{AV}$, and the same rescaling applies to a running *output accumulator*: if $\boldsymbol{o}$ holds $\sum_j e^{s_j - m}\boldsymbol{v}_j$ for the prefix, then on a new block $\boldsymbol{o}' = e^{m-m'}\boldsymbol{o} + \sum_{j\in B} e^{s_j - m'}\boldsymbol{v}_j$, and the final output is $\boldsymbol{o}/\ell$. A single pass over $\mathbf{K}$ and $\mathbf{V}$ suffices.

4.8.2 The tiled forward pass

Putting the pieces together gives Algorithm 4.1 (boxed below), implemented in plain PyTorch in `llmsys/flash.py` (Listing 4.9). On a GPU, the outer loop over query tiles is the grid of thread blocks (one block per (b, h, i)), and the inner loop runs inside each block, keeping $\mathbf{Q}_i$, the current $\mathbf{K}_j, \mathbf{V}_j$, the score tile and the accumulators in shared memory and registers. Appendix A writes this kernel in Triton.

Listing 4.9: Tiled attention forward with online softmax (`llmsys/flash.py`, simplified).

```
for qs in range(0, Tq, block_q):                        # parallel on a GPU
    qb = q[..., qs:qs + block_q, :]
    m = full(-inf); l = zeros(); acc = zeros(block_q, D)
    k_end = min(Tk, qs + block_q + offset) if causal else Tk  # skip future
    for ks in range(0, k_end, block_k):                 # sequential per block
        kb, vb = k[..., ks:ks + block_k, :], v[..., ks:ks + block_k, :]
```

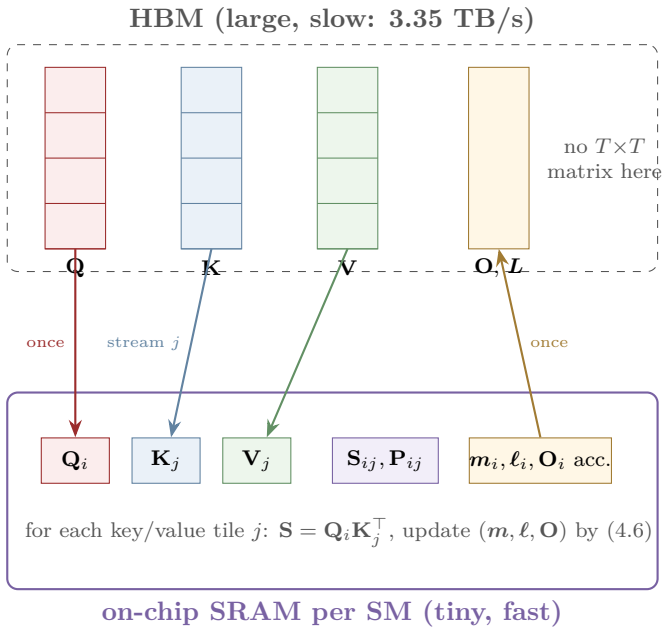


Figure 4.10: FlashAttention's data movement for one query tile $\mathbf{Q}_i$ (one thread block). $\mathbf{Q}_i$ is loaded once, key/value tiles stream through SRAM, scores and probabilities live only on chip, and the output tile plus one log-sum-exp per row are written once.

Algorithm 4.1 (tiled attention forward; one (b, h) slice)
Input: $\mathbf{Q} \in \mathbb{R}^{T_q \times D}$, $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T_k \times D}$, tile sizes B_r, B_c.
for each query tile $\mathbf{Q}_i$ (B_r rows) **in parallel**:
  $\boldsymbol{m} \leftarrow -\infty$, $\boldsymbol{\ell} \leftarrow 0$, $\mathbf{O} \leftarrow 0$ (registers/SRAM)
  for each key/value tile j not entirely in the causal future:
    load $\mathbf{K}_j, \mathbf{V}_j$; $\mathbf{S} \leftarrow \mathbf{Q}_i\mathbf{K}_j^\top/\sqrt{D}$ (+ mask)
    $\boldsymbol{m}' \leftarrow \max(\boldsymbol{m}, \operatorname{rowmax}\mathbf{S})$; $\mathbf{P} \leftarrow e^{\mathbf{S}-\boldsymbol{m}'}$; $\boldsymbol{\alpha} \leftarrow e^{\boldsymbol{m}-\boldsymbol{m}'}$
    $\boldsymbol{\ell} \leftarrow \boldsymbol{\alpha} \odot \boldsymbol{\ell} + \operatorname{rowsum}\mathbf{P}$; $\mathbf{O} \leftarrow \operatorname{diag}(\boldsymbol{\alpha})\mathbf{O} + \mathbf{P}\mathbf{V}_j$; $\boldsymbol{m} \leftarrow \boldsymbol{m}'$
  write $\mathbf{O}_i \leftarrow \operatorname{diag}(\boldsymbol{\ell})^{-1}\mathbf{O}$ and $\boldsymbol{L}_i \leftarrow \boldsymbol{m} + \log\boldsymbol{\ell}$ to HBM.

```
        s = (qb @ kb.transpose(-2, -1)) * scale        # lives in SRAM only
        if causal:
            s = s.masked_fill(k_pos[None, :] > q_pos[:, None], -inf)
        m_new = torch.maximum(m, s.max(-1).values)
        p = torch.exp(s - m_new[..., None])
        alpha = torch.exp(m - m_new)                   # rescale old state
        l = alpha * l + p.sum(-1)
        acc = alpha[..., None] * acc + p @ vb
        m = m_new
    out[..., qs:qs + block_q, :] = acc / l[..., None]
    lse[..., qs:qs + block_q] = m + torch.log(l)       # saved for backward
```

IO complexity. Let M be the SRAM capacity in elements. A query tile of B_r rows needs room for itself, a key and a value tile, and the accumulator, so $B_r, B_c = \Theta(M/D)$. Each of the T/B_r query tiles streams all of $\mathbf{K}$ and $\mathbf{V}$ once, for $\Theta(T^2D/B_r) = \Theta(T^2D^2/M)$ element reads, versus $\Theta(TD + T^2)$ for the naive algorithm. Dao et al. prove this is optimal up to constants for exact attention. Because D^2/M is a small constant for typical head dimensions (about $1/6$ for $D = 128$ in bf16 with H100-sized shared memory), the asymptotic win is a constant

factor of traffic *plus* the elimination of all T^2-sized intermediate tensors and the separate mask/softmax/dropout kernels; in practice the latter dominate.

4.8.3 The backward pass: recompute instead of store

Backpropagation through $\mathbf{O} = \mathbf{PV}$ with $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$ normally needs $\mathbf{P}$. FlashAttention stores only $\mathbf{O}$ and the per-row log-sum-exp $\boldsymbol{L}$, and *recomputes* each tile $\mathbf{P}_{ij} = \exp(\mathbf{S}_{ij} - \boldsymbol{L}_i)$ during the backward pass from $\mathbf{Q}_i$ and $\mathbf{K}_j$. With $\mathbf{dO}$ the incoming gradient, the tile-local gradients are

$$\mathbf{dV}_j \mathrel{+}= \mathbf{P}_{ij}^\top \mathbf{dO}_i, \qquad \mathbf{dP}_{ij} = \mathbf{dO}_i \mathbf{V}_j^\top, \qquad \mathbf{dS}_{ij} = \mathbf{P}_{ij} \odot (\mathbf{dP}_{ij} - \boldsymbol{\delta}_i), \tag{4.7}$$

$$\mathbf{dQ}_i \mathrel{+}= \mathbf{dS}_{ij} \mathbf{K}_j, \qquad \mathbf{dK}_j \mathrel{+}= \mathbf{dS}_{ij}^\top \mathbf{Q}_i, \tag{4.8}$$

(each $\mathbf{dS}$ also carries the $1/\sqrt{D}$ scale), where the softmax Jacobian's row term $\boldsymbol{\delta}_i = \mathrm{rowsum}(\mathbf{P}_i \odot \mathbf{dP}_i)$ would seem to need the whole row. It does not: $\mathrm{rowsum}(\mathbf{P} \odot \mathbf{dOV}^\top) = \mathrm{rowsum}(\mathbf{dO} \odot \mathbf{PV}) = \mathrm{rowsum}(\mathbf{dO} \odot \mathbf{O})$, which is computed once up front from $\mathbf{O}$ and $\mathbf{dO}$. Recomputation costs roughly one extra $\mathbf{QK}^\top$ GEMM, but saves storing and re-reading $O(T^2)$ bytes; on memory-bound hardware this makes the backward pass *faster*, not just leaner. This is activation checkpointing (Chapter 7) applied at the granularity of a single kernel.

Systems Lens: What FlashAttention-2 and -3 changed

FlashAttention-2 (Dao, 2023) reorganised the work: it parallelises over the sequence dimension as well as batch and heads (important when $B \cdot H$ is smaller than the number of SMs, as in long-context training), swaps the loop order in the backward pass so that $\mathbf{dK}_j, \mathbf{dV}_j$ are accumulated in registers and written once, partitions work across warps to avoid shared-memory communication, and minimises non-matmul FLOPs (e.g. deferring the $1/\ell$ rescale to the end), because on tensor-core GPUs a non-matmul FLOP is an order of magnitude more expensive than a matmul FLOP. FlashAttention-3 (Shah et al., 2024) targets Hopper: asynchronous TMA loads and warp-specialised producer/consumer pipelines overlap data movement with WGMMA tensor-core instructions, softmax of one tile is overlapped with the GEMMs of the next, and FP8 support uses incoherent processing to limit quantisation error. For decoding, where $T_q = 1$, parallelising over queries is useless; *split-KV* kernels (FlashDecoding) parallelise over chunks of the KV cache and combine partial results with the same log-sum-exp algebra (Chapter 11).

Pitfall: Exact, but not bit-identical

Tiling changes the order of floating-point additions, so FlashAttention's output differs from the naive result in the last bits, and different tile sizes or GPUs give different bits. Our float64 tests in Lab 4 agree to $\sim 10^{-15}$; in bf16 expect relative differences around 10^{-3}. This matters for RL (Chapter 19), where the rollout engine and the trainer may use different attention kernels and so assign slightly different probabilities to the same tokens.

4.8.4 Using it from PyTorch

In practice one calls PyTorch's fused entry point,

```
F.scaled_dot_product_attention(q, k, v, is_causal=True)    # "SDPA"
```

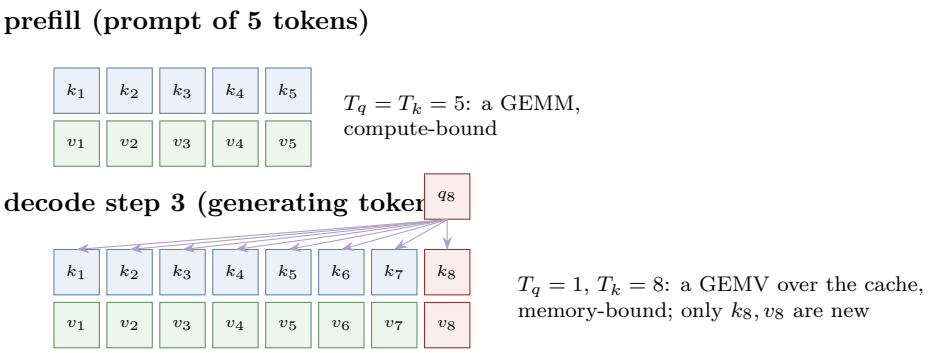


Figure 4.11: Prefill processes the prompt in parallel and fills the cache; each decode step adds one key/value pair (red) and attends with a single query over everything cached.

and lets PyTorch dispatch to a FlashAttention, memory-efficient (Rabe and Staats, 2021, style), cuDNN, or plain math backend depending on hardware, dtype, head dimension and masking. The core `llmsys` attention layers accept `use_sdpa=True` to switch from the explicit implementation to SDPA, and the core test suite checks that both produce the same logits. One caveat: an arbitrary boolean `attn_mask` can force a slower backend than `is_causal=True`, because fused kernels generate structured masks on the fly instead of reading a $T \times T$ mask from memory.

4.9 Attention at inference time: a preview of the KV cache

During training, all T queries are processed at once. During generation (Chapter 11), the model produces one token per step, and causal masking guarantees that the keys and values of earlier positions never change once computed. They can therefore be stored—the *KV cache*—so that each new step projects only the newest token into $\boldsymbol{q}, \boldsymbol{k}, \boldsymbol{v}$, appends $\boldsymbol{k}, \boldsymbol{v}$ to the cache, and attends with a single query over all cached keys (Fig. 4.11). The core `llmsys` attention modules accept a cache object for exactly this purpose, and the general causal mask `llmsys.layers.causal_mask(q_len, k_len)` handles $T_q < T_k$ by placing query i at absolute position $T_k - T_q + i$.

Napkin Math: Decode attention is a memory stream

For one decode step with context T_k, one layer reads the whole cache: $2 \cdot H_{kv} \cdot T_k \cdot d_h$ values, and does $\approx 4 \cdot H \cdot T_k \cdot d_h$ FLOPs with them. With $H = H_{kv}$ (plain MHA) and bf16 that is about $4\,\text{FLOPs}/(4\,\text{bytes}) = 1$ FLOP/byte, two to three orders of magnitude below the H100's balance point of $989/3.35 \approx 295$ FLOP/byte. Decode attention runs at memory bandwidth, so its time is (cache bytes)/(3.35 TB/s). Two levers follow: read fewer bytes (GQA/MQA, cache quantisation; Chapters 6 and 13), or amortise each byte over more queries—which is what batching many requests does *not* achieve for attention, since each request has its own cache (Chapter 12).

Deeper Dive: What do heads learn?

Interpretability work has found recurring head types in trained transformers: *previous-token heads* that attend to position $i - 1$; *induction heads* (Olsson et al., 2022) that look for an earlier occurrence of the current token and attend to the token that followed it,

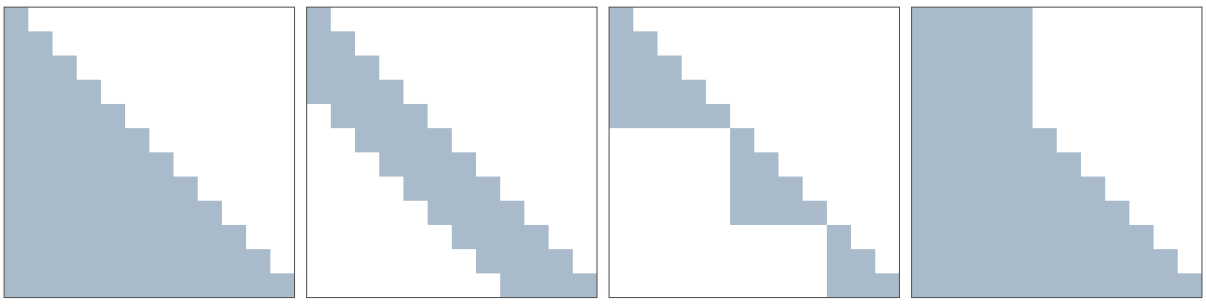


Figure 4.12: Attention masks as query (row) × key (column) patterns; shaded cells are allowed. Tiles that are entirely unshaded can be skipped by a tiled kernel.

implementing in-context copying of patterns ("... A B ... A → B"); and heads whose attention collapses onto the first token or onto delimiters, acting as a no-op "sink". Such *attention sinks* matter for systems: StreamingLLM (Xiao et al., 2024) shows that a windowed KV cache must keep the first few tokens to remain stable, a constraint any cache-eviction policy must respect.

4.10 Beyond dense multi-head attention

The attention of this chapter is *dense*: every query sees every allowed key, and every head has its own keys and values. Most of the attention variants in modern models change one of those two properties, for systems reasons more than modelling reasons.

Sharing keys and values: MQA and GQA. At inference time the keys and values of all past tokens are cached (Chapter 11), and reading that cache is what makes decoding slow. Multi-query attention (Shazeer, 2019) keeps H query heads but a single key/value head; grouped-query attention (Ainslie et al., 2023) uses G key/value heads, each shared by H/G query heads. The KV cache shrinks by H/G. Qwen3-0.6B has 16 query heads and 8 KV heads; Chapter 6 implements GQA and quantifies the savings. DeepSeek's multi-head latent attention (MLA) instead caches a low-rank compressed latent from which keys and values are reconstructed.

Restricting the pattern. Sliding-window attention lets each token attend only to the previous w tokens, making cost $O(Tw)$ and the KV cache bounded; models often interleave windowed and global layers. Block-sparse and dilated patterns (Child et al., 2019; Beltagy et al., 2020) are other points in the design space. Document masks for packed training sequences (Chapter 3) are a block-diagonal restriction whose purpose is correctness rather than speed. Prefix-LM masks let a prompt attend bidirectionally while the continuation remains causal. Figure 4.12 shows these masks side by side; all are supported efficiently by FlashAttention-style kernels because whole tiles can be skipped.

Changing the algorithm. Linear attention (Katharopoulos et al., 2020) replaces $\exp(\boldsymbol{q} \cdot \boldsymbol{k})$ by a kernel feature map so that $\mathbf{AV}$ can be computed as $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top \mathbf{V})$ in $O(Td^2)$, turning attention into a recurrence with a fixed-size state; state-space models and their hybrids pursue the same goal. These trade exact retrieval over the whole context for constant per-token cost.

Distributing the sequence. When even $O(T)$ memory per device is too much, the sequence itself is split across GPUs. Ring attention (Liu et al., 2023) passes key/value blocks around a ring of devices while each device keeps its query block, using exactly the online-softmax merge of Eq. (4.6) to combine partial results. Chapter 8 covers this context parallelism, and Chapter 12 covers PagedAttention, which changes *where* keys and values live in memory rather than which ones are attended to.

Napkin Math: Choosing a tile size

An H100 SM has up to 228 KB of shared memory. With $D = 128$ in bf16, one 64×128 tile is 16 KB. A forward block holding $\mathbf{Q}_i$ ($B_r = 128$ rows: 32 KB), double-buffered $\mathbf{K}_j$ and $\mathbf{V}_j$ ($B_c = 128$: $2 \times 2 \times 32\,\text{KB} = 128\,\text{KB}$) uses 160 KB, leaving headroom; the 128×128 fp32 score tile lives in registers spread across the block's warps. Doubling D to 256 halves the feasible tile sizes, which is one reason head dimensions above 128 are less efficient and kernels specialise by head dimension.

4.11 Summary

- Attention replaces the RNN's single context vector with a per-output, data-dependent weighted average over all inputs; self-attention applies this within one sequence and parallelises across positions.
- Scaled dot-product attention projects inputs to queries, keys and values, computes $\text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_k})\mathbf{V}$, and scales by $\sqrt{d_k}$ to keep the softmax out of saturation.
- Causal masking (scores set to $-\infty$ above the diagonal) prevents a position from seeing its future; masking after the softmax with renormalisation is equivalent. Dropout on attention weights is a training-only regulariser.
- Multi-head attention runs H heads of width d/H; the weight-split implementation computes all heads with a few large batched GEMMs and an output projection.
- Attention costs $O(T^2 d)$ FLOPs and, naively, $O(BHT^2)$ memory; at long context the memory and the HBM traffic of the $T \times T$ matrix, not the FLOPs, are the binding constraints.
- FlashAttention tiles $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, uses the online softmax to normalise incrementally, stores one log-sum-exp per row, and recomputes probabilities in the backward pass: exact attention with $O(T)$ memory and far less HBM traffic.
- GQA/MQA, windowed and sparse masks, linear attention and ring attention modify dense attention to cut KV-cache size, compute, or per-device memory.

4.12 Exercises

Exercise 4.1 (*hand computation*). Using the running example, compute by hand (or in five lines of PyTorch) the context vector for the query "chip" with weight-free attention, and verify it against the last row of Fig. 4.4(a). Then compute its causal context vector.

Exercise 4.2 (*scaling*). Draw $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_k}$ with i.i.d. standard normal entries for $d_k \in$

$\{16, 64, 256, 1024\}$ and $T = 512$ keys. Plot the average maximum softmax probability with and without the $1/\sqrt{d_k}$ scale. Relate what you see to the softmax gradient $\partial\alpha_i/\partial s_j = \alpha_i(\delta_{ij} - \alpha_j)$.

Exercise 4.3 (*Parameter vs. Linear*). Implement self-attention with raw `nn.Parameter` matrices used as `x @ W`. Show how to copy weights between it and Listing 4.3 so that both produce identical outputs.

Exercise 4.4 (*no leakage*). Prove algebraically that masking after the softmax followed by renormalisation equals masking with $-\infty$ before the softmax. Then show numerically that, for route 1, the gradient of position 2's output with respect to position 5's input is exactly zero.

Exercise 4.5 (*head bookkeeping*). GPT-2 medium has $d = 1024$ and 16 heads. Give the shapes of every intermediate tensor in Listing 4.7 for $B = 4$, $T = 1024$, and the number of parameters in the attention block (with Q/K/V biases, as in the released weights).

Exercise 4.6 (*FLOP crossover*). For a 32-layer model with $d = 4096$, $H = 32$ and SwiGLU MLP width 11008, find the sequence length at which attention-score FLOPs ($\mathbf{QK}^\top$ and $\mathbf{AV}$, with causal masking exploited) equal all other per-token forward FLOPs.

Exercise 4.7 (*online softmax algebra*). Prove by induction that Eq. (4.6) yields the global (m, ℓ). Then show that two partial states $(m_1, \ell_1, \boldsymbol{o}_1)$ and $(m_2, \ell_2, \boldsymbol{o}_2)$ computed over *disjoint* key sets can be merged into the state for their union, and write the merge function. Where does this merge appear in split-KV decoding and in ring attention?

Exercise 4.8 (*the δ identity*). Verify numerically that $\mathrm{rowsum}(\mathbf{P} \odot \mathbf{dP}) = \mathrm{rowsum}(\mathbf{dO} \odot \mathbf{O})$, then derive the gradient of a row softmax from scratch to see why $\boldsymbol{\delta}$ appears.

Exercise 4.9 (*IO model*). Using `llmsys.flash.attention_hbm_bytes`, tabulate naive vs. tiled HBM traffic for $T \in \{1\text{k}, 4\text{k}, 16\text{k}, 64\text{k}\}$ and $D \in \{64, 128, 256\}$ with 228 KB of SRAM. At what D does tiling stop helping in this model? Discuss what the model leaves out.

Exercise 4.10 (*skipping masked tiles*). For causal attention with $T = 8192$ and square tiles of 128, what fraction of tiles is fully masked, fully visible, and on the diagonal? How much of the naive $\mathbf{QK}^\top$ work does a tile-skipping kernel avoid?

Exercise 4.11 (*sliding window cache*). A model uses a sliding window of $w = 4096$ in every layer. Derive the KV-cache size as a function of generated length and compare to full attention at $T = 32\text{k}$. What breaks if you want to add prefix caching across requests?

Exercise 4.12 (*measurement*). Run `labs/lab04_attention/bench.py` on a GPU for T from 512 to 16k. Plot time and peak memory of the explicit implementation and SDPA. Identify the length at which the explicit version runs out of memory and compare with the napkin estimate of Section 4.7.

4.13 Lab 4: Attention, from dot products to IO-aware tiling

Lab 4: Attention, from dot products to IO-aware tiling

Code: `code/labs/lab04_attention`

Goals

Implement every attention variant of this chapter, prove to yourself (with tests) that the fast versions compute the same function as the simple ones, and measure where the time and memory go as the sequence grows.

Setup

```
cd code
pytest -q labs/lab04_attention                      # grades starter.py
LAB_IMPL=solution pytest -q labs/lab04_attention    # reference
python labs/lab04_attention/bench.py [--device cuda]
```

Tasks

1. **Weight-free attention.** `simple_attention`: scores, softmax, weighted sum. The test checks one row against a hand-written loop.
2. **Trainable self-attention.** `SelfAttention.forward` with the $1/\sqrt{d_k}$ scale.
3. **Causal masking.** `causal_softmax` and `mask_after_softmax`; they must agree. `CausalAttention.forward` with dropout: the test perturbs future inputs and checks past outputs are unchanged, and that dropout is active only in `train()` mode.
4. **Multi-head.** `MultiHeadAttentionWrapper.forward` and `MultiHeadAttention.forward`. Your weight-split module must produce the same output as the core `llmsys.layers.MultiHeadAttention` after `load_state_dict`, and with one head it must reduce to `out_proj(CausalAttention(x))`.
5. **Online softmax.** `online_softmax` for arbitrary block sizes.
6. **Tiled attention.** `tiled_attention`: never allocate a $T \times T$ tensor; skip key tiles that are entirely in the causal future. Must match naive attention to 10^{-10} in float64 for non-divisible tile sizes.
7. **Napkin.** `score_matrix_bytes`.

Measurements

- Wrapper vs. weight-split MHA latency for GPT-2-small shapes. On a GPU expect the weight-split version to win clearly, by more at small batch; on a CPU the gap is smaller.
- Time and peak memory of explicit attention vs. SDPA for $T = 256 \ldots 16$k. Expect explicit peak memory to grow as T^2 and SDPA's as T, and SDPA to be faster beyond a few hundred tokens.
- Your tiled reference is slow (Python loops); measure it only to see that its memory stays flat.

Deliverables

All tests passing; plots of time and memory vs. T; a paragraph comparing the measured out-of-memory point to the napkin estimate.

Stretch goals

- Implement the tiled *backward* pass yourself and check it with `torch.autograd.gradcheck`; compare with `llmsys.flash.FlashAttentionFunction`.
- Add document masks (block-diagonal causal) to `tiled_attention`, skipping tiles that cross document boundaries entirely.
- Port `tiled_attention` to Triton following Appendix A.

Chapter 5

Implementing a GPT Model

In Chapter 3 we turned raw text into tensors of token IDs and embeddings, and in Chapter 4 we built the causal multi-head attention module that lets every token look back at its predecessors. Those are the two hardest ingredients. This chapter assembles them, together with a handful of small but essential components (layer normalization, a position-wise feed-forward network, residual connections, dropout), into a complete GPT-2-style language model that maps a batch of token IDs to next-token logits, and then uses that model to generate text.

For a systems person this chapter has a second, equally important goal: *accounting*. Once the model exists as code we can say *exactly* how many parameters it has, how many floating-point operations a forward pass costs, how many bytes of activations the backward pass will need, and which kernels dominate the runtime. Every later systems decision in this book—how big a batch fits on a GPU, when decoding becomes memory bound, how to shard a model across devices, how to size a KV cache—rests on the arithmetic we derive here. We will check that arithmetic against PyTorch itself, using `FlopCounterMode` and autograd's saved-tensor hooks, so that nothing is taken on faith.

5.1 Coding an LLM architecture

LLM 4.1

A GPT (*generative pretrained transformer*) is a decoder-only transformer: a stack of identical blocks, each combining causal self-attention with a small per-token neural network, sandwiched between an embedding layer at the bottom and a linear "unembedding" layer at the top. The original GPT-2 release (Radford et al., 2019) came in four sizes. We will build the smallest, usually called the 124M model, but every line of code is size-agnostic: the four variants differ only in three numbers.

Variant	d_{model}	Layers L	Heads H	Params (tied)	Params (untied, no QKV bias)
GPT-2 small	768	12	12	124,439,808	163,009,536
GPT-2 medium	1024	24	16	354,823,168	406,212,608
GPT-2 large	1280	36	20	774,030,080	838,220,800
GPT-2 XL	1600	48	25	1,557,611,200	1,637,792,000

Table 5.1: The GPT-2 family. All share a vocabulary of 50,257 tokens and a context of 1024 positions. "Tied" counts the released checkpoints, which share the token-embedding and output matrices and use biases on the query/key/value projections; the last column is our from-scratch configuration with a separate output head. Counts computed by the Lab 5 code.

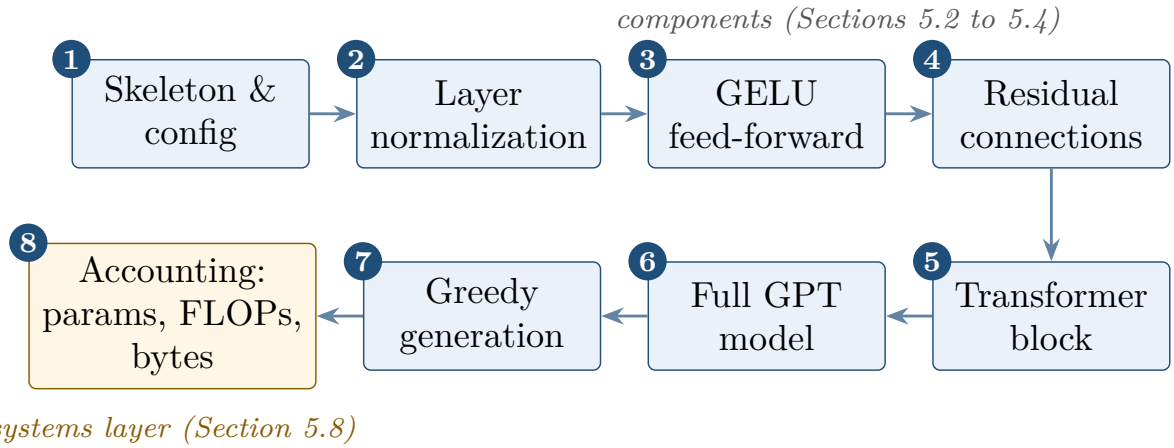


Figure 5.1: Roadmap of this chapter. We first fix the overall skeleton and configuration, build the three missing components, compose them into a transformer block and then a full model, generate text with it, and finally account for every parameter, FLOP and byte.

We describe a model by a plain dictionary. Keeping the configuration as data (rather than hard-coding numbers) is what lets the same class instantiate a 64-dimensional toy model for unit tests and the real 1.5B-parameter model for inference.

Listing 5.1: The configuration that defines GPT-2 small in our code.

```
GPT_CONFIG_124M = dict(
    vocab_size=50257,      # BPE vocabulary of the GPT-2 tokenizer (Ch. 3)
    context_length=1024,   # number of learned position embeddings
    emb_dim=768,           # d_model: width of the residual stream
    n_heads=12,            # attention heads per layer, head_dim = 768/12 = 64
    n_layers=12,           # number of transformer blocks
    drop_rate=0.1,         # dropout probability (0.0 is common for fine-tuning)
    qkv_bias=False,        # Q/K/V biases (True for the OpenAI weights)
)
```

Shapes are the contract. Before writing any layer it pays to fix the shape of the data at every interface, because the whole model is just a sequence of shape-preserving transformations on a tensor of width d_{model}. Figure 5.2 shows the top-down view. A batch of B sequences of T token IDs, shape (B, T), is embedded into (B, T, d_{model}) by adding a token embedding and a learned position embedding. That tensor—the *residual stream*—flows through L transformer blocks, each of which reads it and writes an update of the same shape. A final normalization and a linear map to the vocabulary produce logits of shape (B, T, V): one score per vocabulary entry, for every position.

Stub-first development. A good way to build any multi-component system is to write the outermost structure first with placeholder components that already honor the interfaces, run it end to end, and then replace the placeholders one by one. In PyTorch `nn.Identity` is a perfect placeholder for a block or a normalization layer: it accepts a tensor of any shape and returns it unchanged. The skeleton in Listing 5.2 runs, produces correctly shaped logits, and gives us a harness to test each real component as it arrives.

Listing 5.2: A runnable skeleton: real embeddings and head, placeholder blocks and norm.

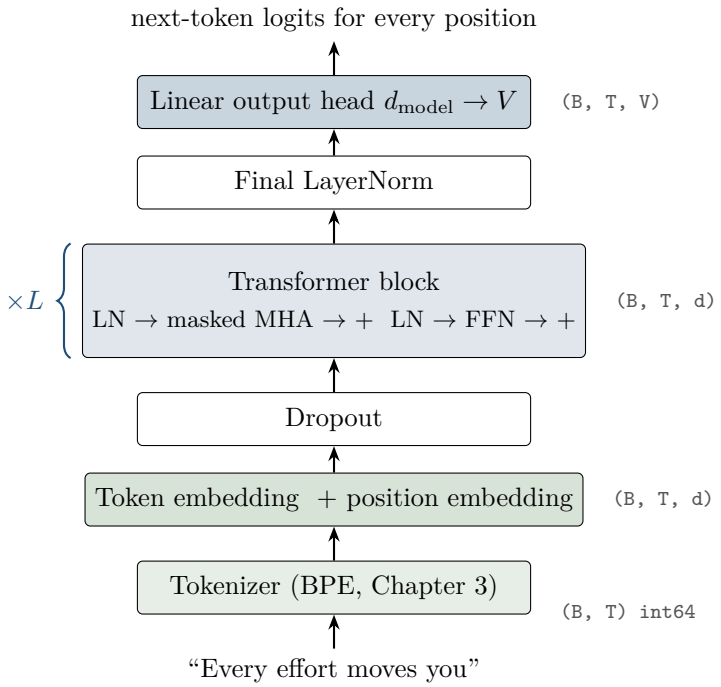


Figure 5.2: Top-down view of a GPT model with the tensor shape at each interface. Everything between the embedding and the output head preserves the shape (B, T, d_{model}).

```python
import torch, torch.nn as nn

class SkeletonGPT(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        # placeholders that respect the (B, T, d) -> (B, T, d) contract
        stubs = [nn.Identity() for _ in range(cfg["n_layers"])]
        self.trf_blocks = nn.Sequential(*stubs)
        self.final_norm = nn.Identity()
        self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

    def forward(self, idx):                          # idx: (B, T)
        T = idx.shape[1]
        x = self.tok_emb(idx) + self.pos_emb(torch.arange(T, device=idx.device))
        x = self.trf_blocks(self.drop_emb(x))        # (B, T, d)
        return self.out_head(self.final_norm(x))     # (B, T, V)

logits = SkeletonGPT(GPT_CONFIG_124M)(torch.randint(0, 50257, (2, 4)))
print(logits.shape)   # torch.Size([2, 4, 50257])
```

Each of the 2×4 output rows is a 50,257-dimensional vector. Position t of row b holds the model's (currently random) scores for the token that should follow the first $t + 1$ tokens of sequence b. During training *every* position contributes a prediction, which is why a single sequence of length T yields T training examples.

Systems Lens: The vocabulary is a systems parameter

Two things about $V = 50{,}257$ matter for performance. First, the output head is the single largest matrix in GPT-2 small: $Vd_{\text{model}} = 38{,}597{,}376$ parameters, 24% of the untied model. Second, 50,257 is an awkward GEMM dimension: it is odd, so tiles of 64 or 128 columns on a tensor-core GPU always leave a ragged remainder. Padding the vocabulary to 50,304 ($= 393 \times 128$) adds only 36,096 never-used parameters but lets every tile be full; nanoGPT-style training scripts do exactly this and observe a measurable speedup. The general rule: choose d_{model}, d_{ff}, head dimension and vocabulary as multiples of 64 (better 128) so the matmul kernels never run partially empty tiles.

Napkin Math: The logits tensor is bigger than the model

For a training batch of $B = 8$ sequences of $T = 1024$ tokens, the logits have $8 \times 1024 \times 50{,}257 \approx 4.1 \times 10^8$ entries. In fp32 that is 1.65 GB—more than twice the 0.62 GB of fp32 weights of the untied 163M model—and the softmax inside the cross-entropy loss materializes another tensor of the same size for the backward pass. This is why large-vocabulary models (Qwen3 has $V \approx 152{,}000$) compute the loss in *chunks* of the sequence, or with fused "linear + cross-entropy" kernels that never store the full logits (Chapter 7).

5.2 Normalizing activations with layer normalization

LLM 4.2

Deep networks are hard to train when the scale of the activations drifts from layer to layer: gradients either explode or shrink exponentially with depth, and the optimizer spends its effort chasing scale rather than learning features. *Layer normalization* (Ba et al., 2016) fixes the scale locally. For each token vector $\boldsymbol{x} \in \mathbb{R}^{d_{\text{model}}}$ independently, it subtracts the mean of the features and divides by their standard deviation, then applies a learned per-feature affine map:

$$\text{LN}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}, \qquad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2. \tag{5.1}$$

The trainable *scale* $\boldsymbol{\gamma}$ (initialized to ones) and *shift* $\boldsymbol{\beta}$ (initialized to zeros) let the network undo the normalization where that helps; the small $\epsilon = 10^{-5}$ prevents division by zero for a constant input.

Along which axis? The statistics are computed over the *feature* dimension, which in a (B, T, d_{model}) tensor is the last one, so the code uses `dim=-1` with `keepdim=True` to keep the result broadcastable. Figure 5.3 contrasts this with batch normalization, which reduces over the batch (and sequence) axes instead. The difference is decisive for language models: layer norm treats each token in isolation, so its output does not depend on what else happens to be in the batch, on the padding, or on how many GPUs the batch is split across.

Listing 5.3: Layer normalization written out explicitly (as in `llmsys/layers.py`).

```
class LayerNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.scale = nn.Parameter(torch.ones(emb_dim))    # gamma
        self.shift = nn.Parameter(torch.zeros(emb_dim))   # beta
```

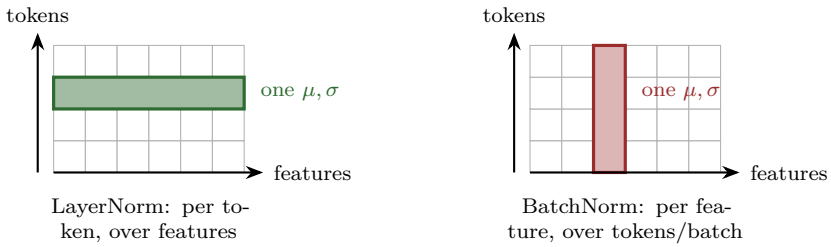


Figure 5.3: Which entries share normalization statistics. Layer norm (left) computes one mean and variance per token across its d_{model} features; batch norm (right) computes one per feature across all tokens in the batch, which couples examples together.

```
    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)  # divide by d, not d-1
        return self.scale * (x - mean) / torch.sqrt(var + self.eps) + self.shift
```

Two details deserve comment. We use the *biased* variance estimator (dividing by d rather than $d-1$). Bessel's correction is meaningless here because we are not estimating a population variance from a sample, and for $d = 768$ the difference is 0.13% anyway; the real reason is compatibility, since GPT-2 was trained in TensorFlow, whose default is the biased estimator, and we will load its weights in Chapter 7. Second, a quick test confirms the contract: after normalization (with $\gamma = 1, \beta = 0$) every token has mean ≈ 0 (on the order of 10^{-8} in fp32, i.e., rounding noise) and variance ≈ 1.

Listing 5.4: Checking the normalization contract.

```
torch.manual_seed(0)
x = torch.randn(2, 5, 16) * 7 + 3          # badly scaled activations
y = LayerNorm(16)(x)
print(y.mean(-1).abs().max())              # ~1e-7
print(y.var(-1, unbiased=False).mean())    # ~1.0
```

Systems Lens: LayerNorm is a bandwidth problem, not a FLOP problem

Equation (5.1) costs roughly 8 FLOPs per element but must read every element of $\boldsymbol{x}$ and write every element of the output. Its arithmetic intensity is therefore on the order of 1–2 FLOP/byte, two orders of magnitude below the ridge point of any modern GPU (Chapter 2): the kernel runs at memory bandwidth, and the only way to make it faster is to move fewer bytes. Our explicit implementation is about as bad as it can be—`mean`, `var`, the subtraction, the division, the scale and the shift each launch a separate kernel that makes a full round trip through HBM. A fused LayerNorm kernel reads $\boldsymbol{x}$ once (computing mean and variance in a single pass with Welford's algorithm or the $E[x^2] - E[x]^2$ identity in fp32), keeps the row in registers or shared memory, and writes once. Even better is to fuse the norm into the *neighbouring* matmul or residual add so the normalized tensor never touches HBM at all; this is one of the main wins of `torch.compile` (Chapter 7) and of the Triton kernels in Appendix A.

Napkin Math: How long should one LayerNorm take on an H100?

For a $B \times T \times d_{\text{model}} = 8 \times 1024 \times 768$ bf16 activation, a fused kernel reads 12.6 MB and writes 12.6 MB. At 3.35 TB s^{-1} that is 25.2 MB/3.35 TB s^{-1} $\approx$ 7.5 µs. The unfused version with six kernels moves roughly six times as many bytes and pays six kernel launches of a few microseconds each: easily 40 µs to 60 µs. Multiply by $2L + 1 = 25$ norms per forward pass and the difference is a millisecond per step—comparable to the time of the attention matmuls at this size.

Deeper Dive: Why not batch normalization?

Batch norm normalizes each feature using statistics pooled over the batch. For language models this is a poor fit for three systems reasons: (i) at inference time the batch is arbitrary (often a single request), so batch norm must fall back to running averages collected during training, and train/test behaviour diverges; (ii) sequences have variable length, and padded positions would pollute the statistics; (iii) under data parallelism, the statistics either differ per GPU or require an extra all-reduce in every layer ("SyncBatchNorm"). Layer norm has none of these problems because it never looks across tokens. Its cousin RMSNorm, which drops the mean subtraction and the shift, is the default in modern LLMs (Section 6.2).

5.3 A feed-forward network with GELU activations

LLM 4.3

Attention mixes information *across* positions; the feed-forward network (FFN, also called the MLP) transforms each position *independently*. It is a two-layer perceptron that first expands the d_{model}-dimensional token vector to $d_{\text{ff}} = 4d_{\text{model}}$ dimensions, applies a nonlinearity, and projects back:

$$\text{FFN}(\boldsymbol{x}) = \mathbf{W}_2\ \text{GELU}(\mathbf{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2, \qquad \mathbf{W}_1 \in \mathbb{R}^{4d\times d},\ \mathbf{W}_2 \in \mathbb{R}^{d\times 4d}. \tag{5.2}$$

The expansion gives the network a wide hidden space in which features can be computed and gated; the contraction brings the result back to the width of the residual stream so blocks can be stacked. Mechanistic-interpretability work often describes the FFN as a key–value memory: rows of $\mathbf{W}_1$ act as pattern detectors ("keys") and the corresponding columns of $\mathbf{W}_2$ as the vectors written into the residual stream when the pattern fires ("values").

GELU. GPT-2 uses the Gaussian error linear unit (Hendrycks and Gimpel, 2016) instead of ReLU. $\text{GELU}(x) = x\,\Phi(x)$, where Φ is the standard normal CDF: the input is weighted by the probability that a standard Gaussian falls below it. GPT-2 was trained with the cheaper tanh approximation

$$\text{GELU}(x) \approx \tfrac{1}{2}x\left(1 + \tanh\left[\sqrt{2/\pi}\,(x + 0.044715\,x^3)\right]\right), \tag{5.3}$$

and since we will load its weights we must use the same formula. Figure 5.4 compares the shapes. Unlike ReLU, GELU is smooth at the origin and lets small negative inputs pass a small negative output (its minimum is about -0.17 near $x \approx -0.75$), so neurons with negative pre-activations still receive gradient. Those properties tend to make optimization of deep transformers slightly easier; the SiLU/Swish curve, used in the gated FFNs of modern models (Section 6.3), is almost indistinguishable.

Listing 5.5: GELU and the expand–contract feed-forward network.

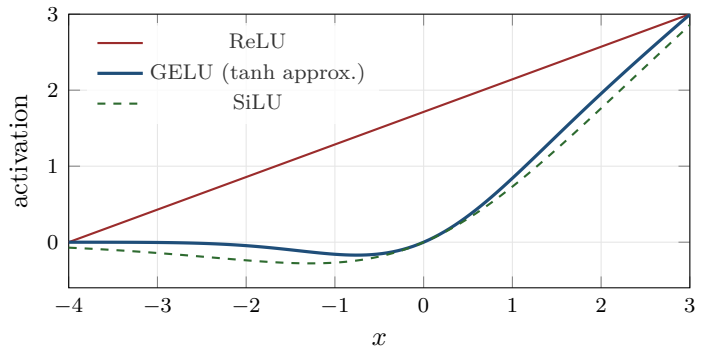


Figure 5.4: ReLU, GELU and SiLU. GELU and SiLU are smooth, non-monotonic, and give small negative outputs for moderately negative inputs. (Plotted analytically.)

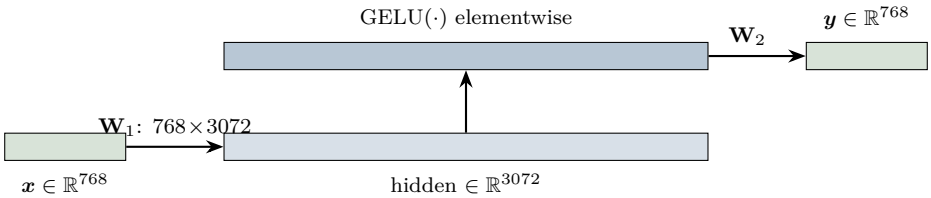


Figure 5.5: The position-wise FFN expands each token vector fourfold, applies GELU, and contracts it back, so it can be stacked like any other shape-preserving layer.

```python
import math

class GELU(nn.Module):
    def forward(self, x):
        c = math.sqrt(2.0 / math.pi)
        return 0.5 * x * (1.0 + torch.tanh(c * (x + 0.044715 * x.pow(3))))

class FeedForward(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(emb_dim, 4 * emb_dim),    # expand:   (B,T,d)  -> (B,T,4d)
            GELU(),
            nn.Linear(4 * emb_dim, emb_dim),    # contract: (B,T,4d) -> (B,T,d)
        )

    def forward(self, x):
        return self.layers(x)
```

Systems Lens: Where the FLOPs live: the FFN's GEMMs

Per block, the two FFN matrices hold $8d_{\text{model}}^2$ weights versus $4d_{\text{model}}^2$ in attention's four projections, so the FFN owns two thirds of every block's matmul FLOPs. Applied to $N = BT$ tokens at once, the first layer is a single GEMM $(N \times d) \cdot (d \times 4d)$ costing $8Nd^2$ FLOPs and moving (in bf16) $2Nd + 8d^2 + 8Nd$ bytes (read activations, read weights, write

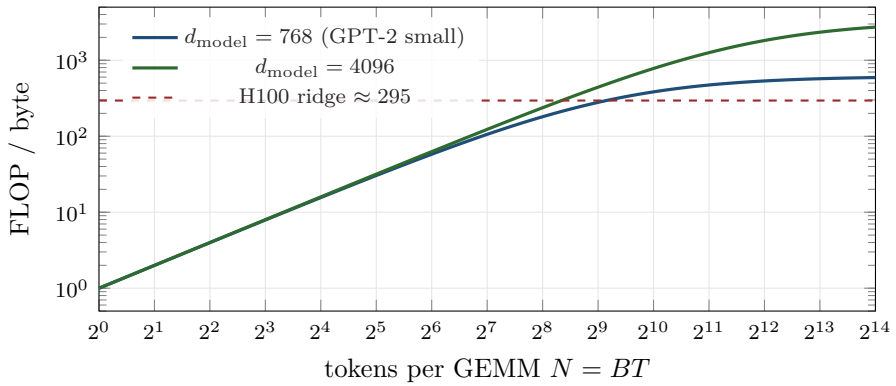


Figure 5.6: Arithmetic intensity of the first FFN GEMM (bf16) as a function of how many tokens it processes at once (analytic). Below the dashed ridge line the kernel is memory bound. The curve for $d_{\text{model}} = 768$ crosses the ridge at about 570 tokens and for $d_{\text{model}} = 4096$ at about 320.

output). Its arithmetic intensity

$$I(N) = \frac{8Nd^2}{10Nd + 8d^2} \xrightarrow{N\to\infty} 0.8\,d$$

grows with the number of tokens in the GEMM and saturates at $0.8d$ FLOP/byte. Whether this kernel is compute- or memory-bound is therefore decided by *how many tokens you batch together*, a fact that will reappear as the central tension of LLM inference (Chapter 11): during training or prefill N is in the thousands and the FFN is compute bound; during single-sequence decoding $N = 1$, the intensity collapses to about 1 FLOP/byte, and the GEMM becomes a pure weight-streaming operation. Figure 5.6 plots $I(N)$ against the H100 ridge point.

Napkin Math: When does the FFN become compute bound?

Setting $I(N)$ equal to the H100's ridge point $989/3.35 \approx 295$ FLOP/byte and solving $8Nd^2 = 295\,(10Nd + 8d^2)$ for N gives $N = 2360d/(8d - 2950)$: about **567 tokens** for $d = 768$ and **324 tokens** for $d = 4096$. A training micro-batch of 8×1024 tokens is far beyond both; a decode step with batch 32 is far below. Wider models reach the ridge with fewer tokens because the weights are re-used more per byte loaded. Note also that GPT-2 small can *never* exceed $0.8 \times 768 \approx 614$ FLOP/byte in this GEMM, only twice the ridge point: small models are never very far from the memory wall.

5.4 Adding shortcut connections

LLM 4.4

A deep stack of layers computes a composition $f_L \circ \cdots \circ f_1$, and by the chain rule the gradient reaching layer 1 is a product of L Jacobians. If those Jacobians typically shrink vectors (saturating sigmoids are the textbook case), the product decays exponentially and the early layers stop learning: the *vanishing gradient* problem. Shortcut (residual, skip) connections, introduced for very deep image networks by He et al. (2016), replace each layer $x \mapsto f(x)$ by $x \mapsto x + f(x)$. The Jacobian of a residual layer is $\mathbf{I} + \partial f/\partial x$, so the backward pass always contains an identity path from the loss straight to every layer.

We can watch this happen. Listing 5.6 builds a 16-layer MLP of width 32 with sigmoid

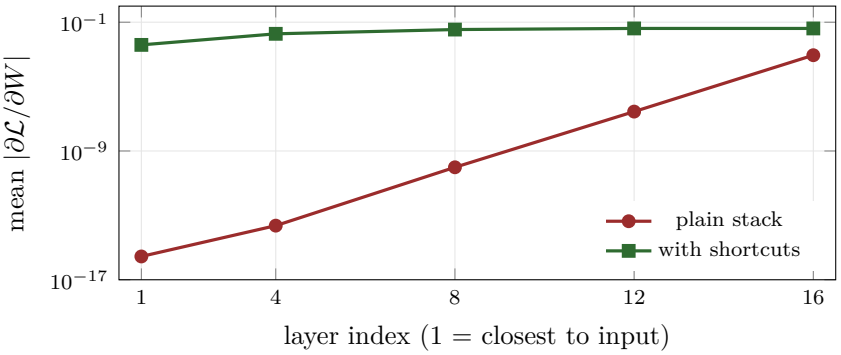


Figure 5.7: Gradient magnitude per layer in a 16-layer sigmoid MLP, measured on CPU in this environment with Listing 5.6. Without shortcuts the gradient shrinks by about 12 orders of magnitude between the last and first layer; with them it stays within one order of magnitude.

activations, once plain and once with residual connections, runs one backward pass on random data, and prints the mean absolute gradient of each layer's weights.

Listing 5.6: A vanishing-gradient experiment (our own setup).

```python
class DeepMLP(nn.Module):
    def __init__(self, depth=16, width=32, residual=False):
        super().__init__()
        self.residual = residual
        self.layers = nn.ModuleList(
            nn.Sequential(nn.Linear(width, width), nn.Sigmoid())
            for _ in range(depth))
        self.head = nn.Linear(width, 1)

    def forward(self, x):
        for layer in self.layers:
            x = x + layer(x) if self.residual else layer(x)
        return self.head(x)

x, y = torch.randn(64, 32), torch.randn(64, 1)
for residual in (False, True):
    torch.manual_seed(1)
    net = DeepMLP(residual=residual)
    nn.functional.mse_loss(net(x), y).backward()
    grads = [l[0].weight.grad.abs().mean().item() for l in net.layers]
    print(residual, [f"{g:.1e}" for g in grads[::4]])
```

Figure 5.7 shows the result: in the plain network the first layer receives gradients around 10^{-16}, which in fp32 arithmetic is effectively zero, while in the residual network every layer gets a usable signal.

The residual stream. In a transformer, residual connections do more than fix gradients; they define the architecture's central data structure. It is helpful to picture a single vector per token—the residual stream—flowing from the embedding to the output head, with every attention and FFN sublayer *reading* from it (through a LayerNorm) and *adding* a write back into it (Fig. 5.8). Because sublayers only add, information placed in the stream early (e.g., the token identity) remains available to all later layers unless something actively cancels it.

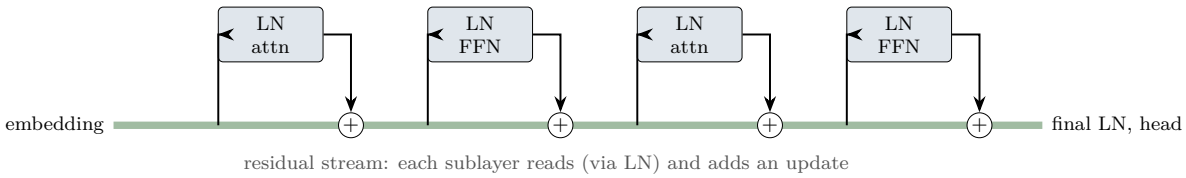


Figure 5.8: The residual-stream view of a transformer: two blocks shown, each with an attention and an FFN sublayer reading the stream and adding their output back.

Deeper Dive: Pre-LN, post-LN, and residual scaling

The original transformer (Vaswani et al., 2017) placed the normalization *after* the addition, $x \leftarrow \mathrm{LN}(x + f(x))$ ("post-LN"). GPT-2 moved it *before* the sublayer, $x \leftarrow x + f(\mathrm{LN}(x))$ ("pre-LN"). Xiong et al. (2020) showed that post-LN produces large gradients near the output at initialization and needs a careful learning-rate warmup, whereas pre-LN keeps the identity path clean (nothing ever normalizes the stream itself) and trains stably at depth. The price is that the stream's magnitude grows with depth: each of the $2L$ sublayers adds a roughly independent update, so the variance of the stream grows linearly in L. GPT-2 therefore scales the initial weights of each sublayer's *output* projection by $1/\sqrt{2L}$, and adds a final LayerNorm before the head so the logits see a normalized vector. Both conventions survive in today's models.

5.5 Connecting attention and linear layers in a transformer block

LLM 4.5

We now have every ingredient of the GPT-2 block (Fig. 5.9): masked multi-head attention from Chapter 4, the FFN, two LayerNorms, two residual connections, and dropout applied to each sublayer's output before it is added back (sometimes called *residual dropout*). The block is shape-preserving: (B, T, d_{model}) in, (B, T, d_{model}) out. That property is not incidental—it is what allows the block to be stacked any number of times and what makes the model's depth a single configuration number.

Listing 5.7: The transformer block (cf. `llmsys/gpt.py`).

```python
from llmsys.layers import MultiHeadAttention

class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = MultiHeadAttention(
            d_in=cfg["emb_dim"], d_out=cfg["emb_dim"],
            context_length=cfg["context_length"], num_heads=cfg["n_heads"],
            dropout=cfg["drop_rate"], qkv_bias=cfg["qkv_bias"])
        self.ff = FeedForward(cfg["emb_dim"])
        self.norm1 = LayerNorm(cfg["emb_dim"])
        self.norm2 = LayerNorm(cfg["emb_dim"])
        self.drop_shortcut = nn.Dropout(cfg["drop_rate"])

    def forward(self, x):
        x = x + self.drop_shortcut(self.att(self.norm1(x)))   # attention
        x = x + self.drop_shortcut(self.ff(self.norm2(x)))    # FFN sublayer
```

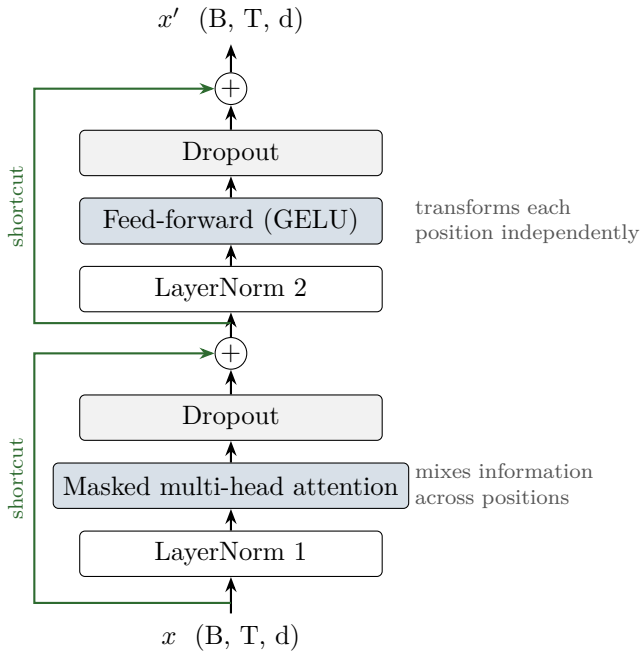


Figure 5.9: The pre-LN GPT-2 transformer block. Green arrows are the two shortcut connections.

```
        return x
```

The code mirrors the figure line by line. Note that we never write `shortcut = x` and add it later: `x = x + f(x)` creates a *new* tensor, so the old `x` is still referenced by the autograd graph and the in-place semantics are safe.

Systems Lens: Anatomy of one block: what is compute bound and what is not

Table 5.2 lists the operations in one GPT-2 small block for a training micro-batch of $N = BT = 8192$ tokens in bf16, with their FLOPs and the minimum bytes they must move. Four GEMM groups (QKV, attention scores and weighted sum, output projection, two FFN layers) are comfortably compute bound; everything else—two LayerNorms, softmax, GELU, two dropouts, two residual adds—is elementwise or a row reduction with intensity of a few FLOP/byte. In an eager PyTorch run, those cheap operations are a long tail of small memory-bound kernels, and on a GPU they can take 30–50% of the step time despite contributing under 1% of the FLOPs. This is the motivation for fusion: FlashAttention fuses mask+softmax+dropout into the attention matmuls (Chapter 4), GEMM epilogues fuse bias+GELU, and compilers fuse residual-add+LayerNorm.

5.6 Coding the GPT model

LLM 4.6

With the block in hand, the full model is short. Listing 5.8 replaces the placeholders of the skeleton with real blocks and a real final LayerNorm. (The library version in `llmsys/gpt.py` additionally threads an optional KV cache and padding mask through the blocks; we will need those from Chapter 11 onwards.)

Listing 5.8: The complete GPT model.

```
class GPTModel(nn.Module):
    def __init__(self, cfg):
```

Operation (one block)	FLOPs	Min. bytes moved	Bound
Q, K, V projections ($3 \times d \times d$)	$6Nd^2$	$6Nd + 6d^2 + \dots$	compute
$QK^\top$ and AV (per sequence $T \times T$)	$4NTd$	$\propto BHT^2$ if unfused	compute
mask + softmax + dropout	$\approx 5BHT^2$	$\approx 4BHT^2 \cdot 2$	memory
output projection	$2Nd^2$	$4Nd + 2d^2$	compute
FFN up + down	$16Nd^2$	$20Nd + 16d^2$	compute
GELU (tanh)	$\approx 15 \cdot 4Nd$	$16Nd$	memory
2× LayerNorm, 2× residual add, 2× dropout	$\approx 30Nd$	$\approx 24Nd$	memory

Table 5.2: Operation inventory for one pre-LN block on $N = BT$ tokens ($d = d_{\text{model}}$, bf16, 2 bytes per element). Counts are approximate for the elementwise rows.

```
        super().__init__()
        self.cfg = cfg
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.ModuleList(
            [TransformerBlock(cfg) for _ in range(cfg["n_layers"])])
        self.final_norm = LayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

    def forward(self, idx):                          # (B, T) token ids
        B, T = idx.shape
        pos = torch.arange(T, device=idx.device)
        x = self.drop_emb(self.tok_emb(idx) + self.pos_emb(pos))
        for blk in self.trf_blocks:
            x = blk(x)
        return self.out_head(self.final_norm(x))    # (B, T, V) logits
```

Counting parameters. Instantiating the model with the 124M configuration and summing `p.numel()` over `model.parameters()` gives 163,009,536, not 124 million. The difference is *weight tying*: the released GPT-2 reuses the token-embedding matrix ($V \times d_{\text{model}}$) as the output projection, while our model has a separate `out_head` of the same shape. Removing it subtracts $50{,}257 \times 768 = 38{,}597{,}376$, leaving 124,412,160; the remaining 27,648 difference to the official 124,439,808 comes from the QKV biases the released checkpoints use ($3 \times 768 \times 12$). Every parameter can be accounted for in closed form. Per block,

$$P_{\text{attn}} = \underbrace{3d^2}_{W_Q, W_K, W_V} + \underbrace{3d}_{\text{QKV bias, optional}} + \underbrace{d^2 + d}_{\text{out proj}}, \tag{5.4}$$

$$P_{\text{FFN}} = \underbrace{4d^2 + 4d}_{W_1, b_1} + \underbrace{4d^2 + d}_{W_2, b_2}, \qquad P_{\text{LN}} = 2 \times 2d, \tag{5.5}$$

and for the whole model

$$P = \underbrace{Vd}_{\text{tok emb}} + \underbrace{T_{\max} d}_{\text{pos emb}} + L\left(P_{\text{attn}} + P_{\text{FFN}} + P_{\text{LN}}\right) + \underbrace{2d}_{\text{final LN}} + \underbrace{Vd}_{\text{head, if untied}} \approx 12Ld^2 + 2Vd. \tag{5.6}$$

Lab 5 asks you to implement this formula and asserts that it matches the module exactly for several configurations. Figure 5.10 shows how the budget splits across the four GPT-2 sizes: the embedding and head dominate the small model, while for GPT-2 XL the transformer blocks

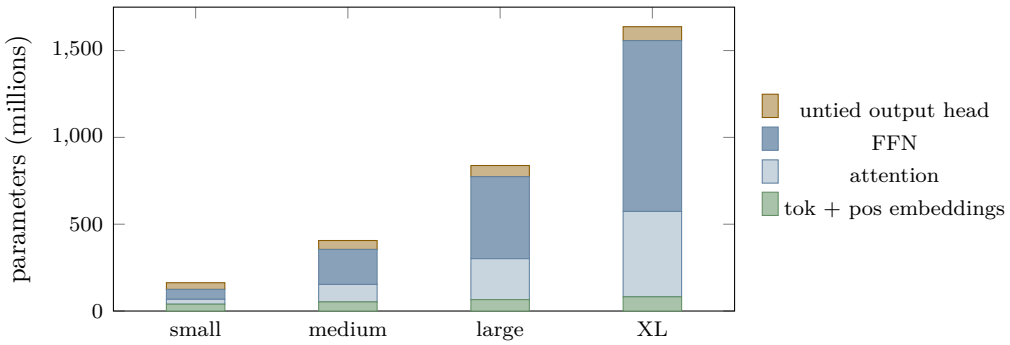


Figure 5.10: Parameter breakdown of the four GPT-2 sizes in our untied configuration (LayerNorm parameters are too small to see). Computed with `count_params_analytic` from Lab 5.

account for 90% of the parameters and the FFN alone for 60%.

Napkin Math: Memory footprint of the weights

At 4 bytes per fp32 parameter, the untied 163M model needs $163{,}009{,}536 \times 4\,\text{B} \approx 622\,\text{MiB}$; the tied 124M checkpoint in bf16 needs 249 MB. GPT-2 XL in fp32 is 6.1 GiB. These numbers are only the *weights*: training with AdamW in mixed precision needs about 16 bytes per parameter for weights, gradients and optimizer state (Chapter 7), so GPT-2 XL needs ≈ 26 GB before a single activation is stored—already more than a 24 GB consumer GPU.

Systems Lens: Weight tying as a systems trade-off

Tying saves $V d_{\text{model}}$ parameters (24% of GPT-2 small, 5% of GPT-2 XL) and the corresponding gradient and optimizer memory. It does *not* save FLOPs: the output projection is still a $d \times V$ GEMM on every position. It also couples two very different gradient signals (sparse row updates from the embedding lookup, dense updates from the head) into one tensor, and in pipeline-parallel training the first and last stages must synchronize that shared tensor's gradient with an extra all-reduce. Small models (GPT-2, Qwen3-0.6B to 4B) tend to tie; large ones (Qwen3-8B and up, Llama 3) usually do not.

5.6.1 Initialization

A freshly constructed `nn.Linear` uses PyTorch's default Kaiming-uniform initialization, which was designed for ReLU convolutional networks. GPT-2 instead draws every weight matrix from $\mathcal{N}(0, 0.02^2)$, sets biases to zero, and (as discussed in Section 5.4) shrinks the standard deviation of the two *output* projections in each block (attention's `out_proj` and the FFN's second linear) by $1/\sqrt{2L}$, so that the $2L$ residual updates sum to a stream of roughly constant scale at initialization. Listing 5.9 applies this recipe. It matters only when training from scratch—loaded checkpoints overwrite every value—but a bad initialization is one of the most common reasons a from-scratch pretraining run diverges in its first few hundred steps.

Listing 5.9: GPT-2 style initialization.

```
import math

def init_gpt2_weights(model, std=0.02):
    L = model.cfg["n_layers"]
```

```
    for name, p in model.named_parameters():
        if p.dim() == 2:                              # weight matrices and embeddings
            s = std / math.sqrt(2 * L) if name.endswith(
                ("out_proj.weight", "ff.layers.2.weight")) else std
            torch.nn.init.normal_(p, mean=0.0, std=s)
        elif name.endswith(("bias", "shift")):
            torch.nn.init.zeros_(p)
        # LayerNorm scales keep their default value of 1
```

Systems Lens: Initialization at scale

For a 124M-parameter model initialization takes a fraction of a second; for a 70B-parameter model, drawing 70 billion Gaussian samples on one CPU core takes minutes and needs 280 GB of host memory in fp32. Large-scale training frameworks therefore construct the model on PyTorch's `meta` device (shapes only, no storage), shard it across GPUs first (FSDP, Chapter 8), and only then run the initializer on each GPU's shard. Two subtleties follow. First, every data-parallel replica must start from *identical* weights, so either one rank initializes and broadcasts, or all ranks seed their random generators identically. Second, the initializer must be written per-parameter-name (as in Listing 5.9) rather than per-module-constructor, because under sharding the module that "owns" a weight may never see its full shape.

Deeper Dive: What dropout costs

Dropout looks free, but in an eager implementation each dropout call generates a random tensor of the same shape as its input, stores a boolean mask for the backward pass (1 byte per element, or one bit if packed), and launches two extra kernels. GPT-2 has $2L + 1$ residual/embedding dropouts plus L dropouts on the BHT^2 attention probabilities, the largest activation tensor in the model. Fused kernels regenerate the mask in the backward pass from the random generator's seed and offset (Philox counters) instead of storing it, which is how FlashAttention supports attention dropout without materializing anything. For pretraining modern LLMs on trillions of tokens, the usual answer is simpler: set the dropout rate to zero (Chapter 6).

5.7 Generating text

LLM 4.7

The model outputs logits for every position, but to generate text we only need the last one: it scores every candidate for the token that follows the whole input. Generation is a loop (Fig. 5.11): run the model, take the logits at position -1, pick a token, append it to the input, repeat. Picking the token with the highest score is *greedy decoding*. Because softmax is monotonic, the argmax of the logits equals the argmax of the probabilities, so the softmax can be skipped; we will need the probabilities once we sample (Chapters 7 and 15).

Listing 5.10: Greedy generation without a cache.

```
@torch.no_grad()
def generate_greedy(model, idx, max_new_tokens, context_size):
    model.eval()                                  # disable dropout
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]         # never exceed the position table
        logits = model(idx_cond)[:, -1, :]        # (B, V): only the last position
```

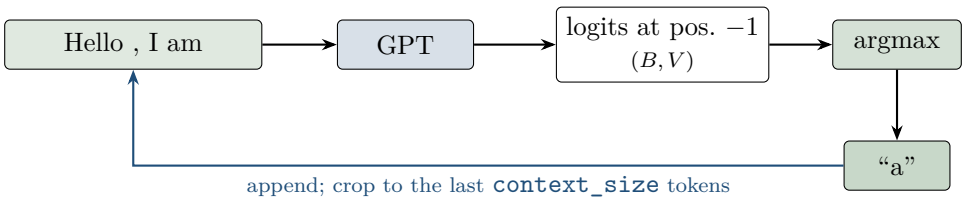


Figure 5.11: The autoregressive generation loop. Each iteration re-runs the model on the (cropped) sequence, keeps only the last position's logits, and appends the chosen token.

```
        next_id = torch.argmax(logits, dim=-1, keepdim=True)
        idx = torch.cat([idx, next_id], dim=1)
    return idx
```

Three details are easy to get wrong. (i) `model.eval()` switches dropout off; leaving it on makes generation nondeterministic and slightly worse. (ii) The crop `idx[:, -context_size:]` is mandatory for a model with learned absolute positions: GPT-2 has exactly 1024 position embeddings and indexing the 1025th raises an error. (iii) Since the weights are random, the untrained model produces fluent-looking nonsense: a valid sequence of token IDs that decodes to unrelated word pieces. Giving the model useful weights is the subject of Chapter 7, where we both train it and load OpenAI's checkpoint.

Napkin Math: The quadratic cost of cache-less generation

Each step of Listing 5.10 re-processes the whole prefix. A forward pass over a sequence of length t costs about $t \cdot (2P_{\text{matmul}} + 4Ldt)$ FLOPs (Section 5.8). Generating 1000 tokens from a 10-token prompt with GPT-2 small therefore costs $\sum_{t=10}^{1009} F(t) \approx 139$ TFLOP (computed with the Lab 5 formula), whereas processing each new token only once—which is what the KV cache of Chapter 11 enables—costs about 0.27 TFLOP, roughly **500**× less. The cache-less loop is $O(n^2)$ in the matmuls and $O(n^3)$ in attention; it is fine for a teaching example and useless in production.

5.8 Accounting for every FLOP and byte

We close the chapter with the systems analysis that the rest of the book builds on. The goal is three formulas—FLOPs, weight memory, activation memory—each validated against PyTorch.

5.8.1 FLOPs of a forward pass

A matrix product $(m \times k) \cdot (k \times n)$ performs mkn multiply–accumulates, conventionally counted as $2mkn$ FLOPs. Every `nn.Linear` applied to N tokens is such a product with $m = N$, so it costs $2N$ times its number of weights. Summing over the block gives $2N \cdot 12d^2$; the head adds $2NVd$. The two attention matmuls are different because their "weights" are other activations: $QK^\top$ for one head of one sequence is $(T \times d_h) \cdot (d_h \times T)$, i.e. $2T^2 d_h$ FLOPs, and AV costs the same, so per layer and sequence the pair costs $4T^2 d_h H = 4T^2 d$. Putting it together for B sequences of length T:

$$F_{\text{fwd}}(B, T) = BT\Big[\, L\,(24d^2 + 4Td) + 2Vd \,\Big]. \tag{5.7}$$

Per token, the matmul part is twice the number of non-embedding parameters (plus the head), the classic "$2N$ FLOPs per token" rule; the attention part grows linearly with context and overtakes the $24d^2$ term once $T > 6d$ (4608 tokens for GPT-2 small).

Listing 5.11: Verifying Eq. (5.7) against PyTorch's own FLOP counter.

```python
from torch.utils.flop_counter import FlopCounterMode

def forward_flops(cfg, batch, seq_len):
    d, V, L = cfg["emb_dim"], cfg["vocab_size"], cfg["n_layers"]
    per_token = L * (24 * d * d + 4 * seq_len * d) + 2 * d * V
    return batch * seq_len * per_token

model = GPTModel(cfg).eval()
idx = torch.randint(0, cfg["vocab_size"], (2, 16))
with FlopCounterMode(display=False) as fc:
    model(idx)
assert fc.get_total_flops() == forward_flops(cfg, 2, 16)   # exact match
```

The match in Listing 5.11 is *exact* because `FlopCounterMode` counts only matmul-like operators with the same $2mkn$ convention, and our explicit attention computes the full $T \times T$ score matrix even though half of it is masked. (A causal FlashAttention kernel skips the masked tiles and does half the attention FLOPs; FLOP counters then disagree with "useful" FLOPs, a recurring source of confusion when people report utilization.) For GPT-2 small at $T = 1024$ the formula gives 0.285 GFLOP per token.

Key Idea: The 6N rule

The backward pass of a matmul computes two products of the same size as the forward one (gradient with respect to the input and with respect to the weight), so it costs about twice the forward. Training therefore costs $\approx 3 \times F_{\text{fwd}}$, i.e. about $6P$ FLOPs per token for a model with P (non-embedding) parameters when attention is a minor term. Training GPT-2 small on 10 billion tokens costs $6 \times 1.24 \times 10^8 \times 10^{10} \approx 7.4 \times 10^{18}$ FLOPs: about 2.1 hours on one H100 at 100% utilization, or roughly 5 hours at a realistic 40% MFU (Chapter 7).

5.8.2 Activation memory

For the backward pass, autograd keeps every tensor that some operation's derivative needs. Walking through our block on s tokens per sequence, batch b, width d and h heads, the saved tensors are: the input of LN1 (sbd elements); its output, shared by the Q/K/V projections (sbd); Q and K for the score matmul ($2sbd$); the softmax probabilities (bhs^2); V (sbd); the context fed to the output projection (sbd); the input of LN2 (sbd); the FFN input (sbd); the pre-GELU hidden ($4sbd$); and the post-GELU hidden fed to the second linear ($4sbd$). In total

$$A_{\text{layer}} \approx (16\, sbd + bhs^2) \text{ elements}, \tag{5.8}$$

which with 2-byte activations and 1-byte dropout masks becomes the widely quoted $sbd\,(34 + 5hs/d)$ bytes of Korthikanti et al. (2022). Two consequences: activation memory is linear in batch size and in depth, and the attention term grows *quadratically* in sequence length, which is precisely what FlashAttention removes by never storing the probabilities.

Listing 5.12: Measuring saved activations with autograd hooks.

```
def measure_saved_activation_bytes(model, idx):
    params = {p.untyped_storage().data_ptr() for p in model.parameters()}
    seen = {}
    def pack(t):                          # called for every tensor saved for backward
        ptr = t.untyped_storage().data_ptr()
        if ptr not in params and ptr not in seen:
            seen[ptr] = t.untyped_storage().nbytes()
        return t
    with torch.autograd.graph.saved_tensors_hooks(pack, lambda t: t):
        model(idx)
    return sum(seen.values())
```

Running Listing 5.12 on the tiny test configuration ($d = 64$, 2 layers, 4 heads, one sequence of 64 tokens, fp32) gives the numbers in Table 5.3. The measured value is about 2.1× the estimate of Eq. (5.8). The explanation is instructive: our explicit tanh-GELU is built from half a dozen primitive operations, and autograd saves several of their intermediates, each $4d$ wide. Replacing it with PyTorch's fused `nn.GELU(approximate="tanh")`, which saves only its input, removes 28% of all activation memory; the explicit LayerNorm similarly saves extra intermediates that a fused kernel would not. Fusion saves memory, not just time.

Configuration (tiny GPT, $b = 1$, $s = 64$, fp32)	Saved activations
Estimate, Eq. (5.8)	655 kB
Measured, explicit GELU and LayerNorm	1,387 kB
Measured, fused `nn.GELU`, explicit LayerNorm	994 kB

Table 5.3: Activation memory: estimate vs. measurement with saved-tensor hooks (measured on CPU in this environment). The measured value doubles exactly when the batch doubles.

Napkin Math: Activations of GPT-2 small at full context

For $b = 8$, $s = 1024$, $d = 768$, $h = 12$, $L = 12$ in bf16, Eq. (5.8) gives per layer $16 \cdot 1024 \cdot 8 \cdot 768 \cdot 2\,\text{B} \approx 201\,\text{MB}$ for the linear terms and $8 \cdot 12 \cdot 1024^2 \cdot 2\,\text{B} \approx 201\,\text{MB}$ for the attention probabilities: 0.4 GB per layer, 4.8 GB for the whole model, plus the 1.65 GB fp32 logits. That is roughly ten times the weights. At $s = 8192$ the attention term alone would be 12.9 GB *per layer*. This is why long-context training requires both FlashAttention (no stored probabilities) and activation checkpointing (Chapter 7).

5.8.3 Where does the time go?

FLOPs are not time. The Lab 5 `bench.py` script profiles a forward pass with `torch.profiler` and groups the time by operator. On a GPU you will typically find the large GEMMs (`aten::addmm`, `aten::mm`) at the top of the table but far from 100% of the time, followed by a long tail of elementwise kernels (`aten::mul`, `aten::add`, `aten::tanh`, `aten::pow`) from our explicit GELU and LayerNorm, and copies from `transpose().contiguous()` in attention. Dividing Eq. (5.7) by the measured time gives the achieved FLOP/s; dividing that by the device peak gives the *model FLOPs utilization* (MFU), the single most useful efficiency number in LLM systems and the subject of Chapter 7.

Systems Lens: A first look at kernel fusion and compilation

The gap between "FLOPs" and "time" in an eager model has three sources: memory-bound elementwise kernels that each round-trip through HBM, per-kernel launch overhead (a few microseconds each, which dominates small models), and extra copies from layout changes. All three are attacked by the same technique—*fusion*—either by hand (fused LayerNorm, fused bias-GELU, FlashAttention), by vendor libraries (cuBLASLt epilogues), or automatically by `torch.compile`, whose TorchInductor backend traces the model, groups pointwise and reduction operations into generated Triton kernels, and can capture the whole forward pass in a CUDA graph to eliminate launch overhead. We will measure the effect in Chapters 7 and 11 and write such kernels ourselves in Appendix A.

5.9 Summary

- A GPT model is an embedding layer, a stack of L identical shape-preserving transformer blocks, a final LayerNorm, and a linear head producing (B, T, V) logits.
- LayerNorm normalizes each token across its features; it is independent of the batch and therefore well suited to variable-length, distributed training. It is memory bound and an important fusion target.
- The FFN expands to $4d_{\text{model}}$, applies GELU, and contracts back. It holds two thirds of each block's weights and FLOPs; its arithmetic intensity depends on how many tokens are processed at once.
- Residual connections keep gradients alive in deep networks and define the residual stream; pre-LN placement and $1/\sqrt{2L}$ output scaling keep training stable.
- GPT-2 small has 163,009,536 parameters with a separate head and 124,439,808 in the released tied checkpoint; parameters, FLOPs ($\approx 2P$ per token forward, $6P$ training) and activation memory ($\approx 16sbd + bhs^2$ elements per layer) can all be computed exactly and checked against PyTorch.
- Greedy generation repeatedly runs the model and appends the argmax token; without a cache it re-processes the whole prefix each step, which is quadratically wasteful.

5.10 Exercises

Exercise 5.1 (*GPT-2 sizes*). Using Eq. (5.6), compute the parameter counts of GPT-2 medium, large and XL in the untied configuration, and separately the share held by attention and by the FFN. Check your numbers against Table 5.1 and Fig. 5.10. At what d_{model} does the FFN first hold more parameters than the token embedding plus the head, for $L = 12$ and $V = 50{,}257$?

Exercise 5.2 (*Separate dropout rates*). Our configuration uses one `drop_rate` for the embedding dropout, the residual dropout, and the attention-weight dropout. Change the configuration and code so the three rates can be set independently, and argue which of the three is cheapest to remove from a systems perspective.

Exercise 5.3 (*Vocabulary padding*). Pad the vocabulary of GPT-2 small to the next multiple of 128. How many parameters and how many bytes of fp32 optimizer state (two Adam moments) does that add? What must the generation code do to make sure the padded rows are never sampled?

Exercise 5.4 (*Ridge point*). Derive the crossover $N^\star$ at which the first FFN GEMM becomes compute bound on an A100 (312 TFLOP/s bf16, 2.04 TB/s) for $d = 768$ and $d = 4096$. Repeat for the second FFN GEMM, whose input is $4d$ wide. Which of the two GEMMs reaches the ridge first, and why?

Exercise 5.5 (*Residual growth*). Initialize GPT-2 small with all weights drawn from $\mathcal{N}(0, 0.02^2)$ and measure the RMS of the residual stream after each block on random tokens. Then scale the weights of every `out_proj` and second FFN linear by $1/\sqrt{2L}$ and measure again. Plot both curves and explain the shape of the first.

Exercise 5.6 (*Post-LN*). Implement a post-LN variant of `TransformerBlock`. Train a 24-layer tiny model with both variants for a few hundred steps on the same data without learning-rate warmup. Which one diverges first as you raise the learning rate?

Exercise 5.7 (*Fused activations save memory*). Using Listing 5.12, measure saved-activation bytes for (a) our explicit model, (b) the model with `nn.GELU(approximate="tanh")`, (c) additionally with `nn.LayerNorm`. Explain each reduction by listing the tensors autograd stops saving.

Exercise 5.8 (*Causal FLOPs*). Equation (5.7) counts the full $T \times T$ attention. Derive the "useful" FLOPs when only the causal half is computed, and compute the relative difference for GPT-2 small at $T = 1024$ and $T = 8192$. Why does it matter when comparing MFU numbers across papers?

Exercise 5.9 (*Cost of generation*). Write a function that returns the total FLOPs of generating n tokens from a p-token prompt (a) without a cache, (b) with a cache. Plot the ratio as a function of n for $p = 100$. At which n does the cached version save $100\times$?

Exercise 5.10 (*Chunked loss*). Design (and implement) a cross-entropy computation that processes the sequence in chunks of 128 positions so that the full (B, T, V) logits tensor is never materialized. What is the peak memory saving for $B = 8$, $T = 1024$, $V = 50{,}257$ in fp32? What extra cost do you pay in the backward pass?

5.11 Lab 5: Assemble GPT-2 and account for every parameter, FLOP and byte

Lab 5: Assemble GPT-2 and account for every parameter, FLOP and byte

Code: `code/labs/lab05_gpt`

Goals

Build the GPT-2 architecture from its components so that it is bit-compatible with `llmsys.gpt.GPTModel`, then turn the chapter's accounting into tested code: an exact parameter formula, an exact forward-FLOP formula, and an activation-memory measurement.

Setup

```
cd code && pip install -r requirements.txt
pytest -q labs/lab05_gpt            # runs the tests against your starter.py
```

Tasks

1. **Components.** Implement `LayerNorm.forward`, `gelu_tanh` and `FeedForward.__init__`. Keep the attribute names (`scale`, `shift`, `layers`) so state dicts are interchangeable with the library.
2. **Block and model.** Implement `TransformerBlock.forward` (pre-LN, residual dropout) and `GPTModel.forward`. The test loads the library model's weights into yours and requires identical logits.
3. **Generation.** Implement `generate_greedy` with context cropping; the test generates past `context_length` to make sure you crop.
4. **Accounting.** Implement `count_params_analytic` (with and without tying) and `forward_flops`. The tests compare against `sum(p.numel())` and against `FlopCounterMode` for several shapes, and pin the GPT-2 small numbers 163,009,536 and 124,439,808.
5. **Activation memory.** Implement `activation_bytes_estimate` (Eq. (5.8)) and `measure_saved_activation_bytes` with `saved_tensors_hooks`.

Measurements

- Run `python labs/lab05_gpt/bench.py -impl starter -seq 256 -batch 4` on your machine. Record forward time, achieved TFLOP/s, and the top 10 operators. Then swap in the fused GELU and `nn.LayerNorm` and re-run. Expect the elementwise tail to shrink noticeably on a GPU.
- Plot measured activation bytes against batch size (should be a straight line through the origin) and against sequence length (should curve upwards: the bhs^2 term).
- If you have the HF `gpt2` `model.safetensors`, pass `-weights` and confirm your model generates coherent English with greedy decoding.

Deliverables

Passing tests; a short table of operator times before and after fusion; the two activation plots with one paragraph explaining their shapes.

Stretch goals

- Wrap the model in `torch.compile` and compare time and saved-activation bytes.

- Implement the chunked cross-entropy of the last exercise and measure peak memory with `torch.cuda.max_memory_allocated`.

Chapter 6

Modern Architectures: Qwen3 from Scratch

GPT-2 was released in 2019. Six years later the dominant open-weight models—Llama, Mistral, Gemma, DeepSeek, Qwen—are still decoder-only transformers with the same overall shape as the model of Chapter 5: embed, stack L residual blocks of attention plus an MLP, normalize, project to the vocabulary. What changed are the parts inside the block, and a striking number of those changes were driven not by modeling accuracy but by *systems* concerns: cheaper normalization, attention whose KV cache is several times smaller, position encodings that need no learned table and extrapolate to longer contexts, and sparse MLPs that grow the parameter count without growing the FLOPs per token.

This chapter rebuilds one representative modern model, Qwen3 (Qwen Team, 2025), component by component, and explains each design change from both angles: what it does to the model and what it does to the machine. Qwen3 is the base model for the whole second half of this book—inference (Chapter 11), serving (Chapter 12), evaluation (Chapter 14), reinforcement learning (Chapter 17) and distillation (Chapter 20)—so the code we write here is the code we will run for the next fourteen chapters. We finish with the larger members of the dense Qwen3 family, how to load and plan memory for them, and with mixture-of-experts layers, the main architectural lever for scaling past what a dense model can afford.

6.1 From GPT-2 to Qwen3

RM C

Figure 6.2 places the two architectures next to each other. The skeleton is unchanged: token embedding, L pre-norm residual blocks, final norm, linear head. The differences are all local and are summarized in Table 6.1.

Notice the absence of dropout. GPT-2 was trained for a few epochs over a 40 GB corpus; modern models train for a single pass over tens of trillions of tokens, so they barely overfit and dropout would only add noise and a random-mask kernel per sublayer. Removing biases simplifies tensor parallelism (a bias on a row-parallel layer must be added exactly once after the all-reduce, Chapter 8) and has no measurable cost in quality.

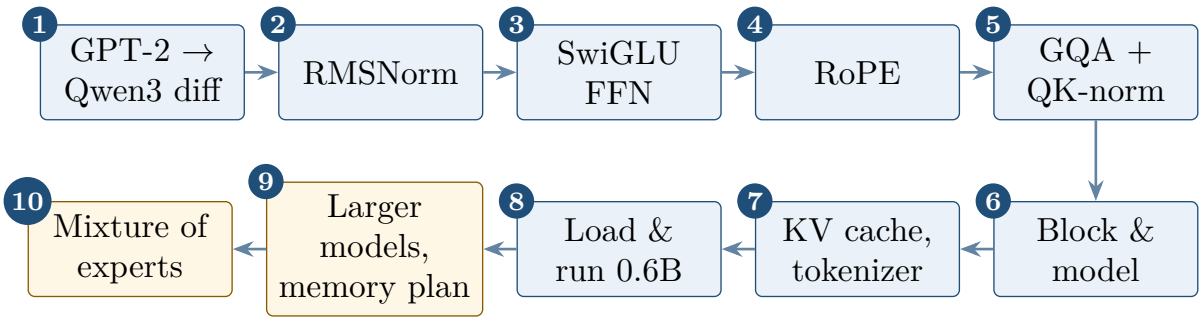


Figure 6.1: Roadmap of this chapter: we replace GPT-2's components one at a time (1–5), assemble and load the real Qwen3-0.6B (6–8), then scale up to larger dense models and to sparse mixture-of-experts layers (9–10).

Component	GPT-2	Qwen3 (dense)
Normalization	LayerNorm (mean, variance, scale, shift)	RMSNorm (RMS, scale only)
MLP	W_2 GELU$(W_1 x)$, biases, $d_{\text{ff}} = 4d$	$W_3(\text{SiLU}(W_1 x) \odot W_2 x)$, no biases
Positions	learned absolute table added to input	rotary, applied to q, k in every layer
Attention	MHA, H K/V heads	GQA, 8 K/V heads shared by H query heads
Extra norms	–	RMSNorm on each query and key head
Biases	everywhere	none
Dropout	0.1	none (pretraining on enough data to not overfit)
Vocabulary	50,257	151,936 (padded)
Context	1024	32,768 native, 40,960 positions configured
Weights dtype	fp32 checkpoint	bf16 checkpoint

Table 6.1: What changed between GPT-2 and Qwen3.

6.2 RMSNorm

RM C.1

Root-mean-square normalization (Zhang and Sennrich, 2019) keeps only the rescaling half of layer norm. It divides each token vector by its root mean square and applies a learned per-feature gain, without subtracting the mean and without a shift:

$$\text{RMSNorm}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x}}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}}. \tag{6.1}$$

The output is not centered: its mean is whatever the input's mean was, scaled. Empirically this does not hurt—re-centering turns out to be unnecessary for the optimization benefits of normalization, which come mainly from controlling the scale.

Listing 6.1: RMSNorm with an fp32 accumulation (cf. `llmsys/layers.py`).

```python
class RMSNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.scale = nn.Parameter(torch.ones(emb_dim))    # gamma; no shift

    def forward(self, x):
        in_dtype = x.dtype
        x = x.to(torch.float32)                    # square and mean in fp32
```

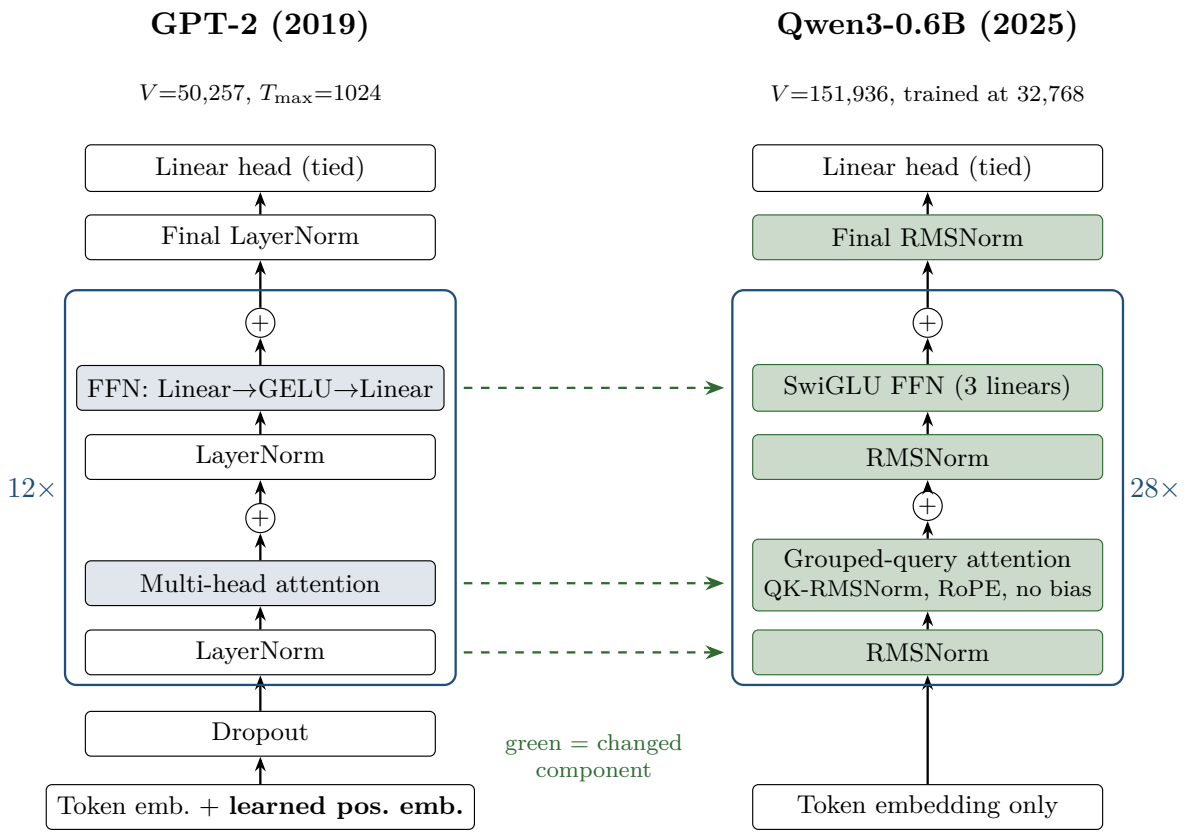


Figure 6.2: GPT-2 small and Qwen3-0.6B side by side. Both are pre-norm decoder-only transformers; Qwen3 swaps every normalization for RMSNorm, the GELU MLP for a gated SwiGLU MLP, learned absolute positions for rotary embeddings applied inside attention, and multi-head for grouped-query attention with per-head query/key normalization. Dropout is gone.

```
        x = x * torch.rsqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return (x * self.scale.to(torch.float32)).to(in_dtype)
```

The explicit up-cast matters. The model runs in bf16, whose 8-bit mantissa gives only about three significant decimal digits; summing $d_{\text{model}} = 1024$ squares in bf16 accumulates rounding error, and squaring large activations can overflow fp16. Doing the reduction in fp32 and casting back is what every production kernel does, and it is required to reproduce the official logits.

Systems Lens: Why RMSNorm is faster

LayerNorm needs two row statistics, the mean and the variance; RMSNorm needs one, $\sum_i x_i^2$. In a fused kernel both read the row once, so the bandwidth cost is identical; the win is in the reduction itself (one warp-level tree reduction instead of two, fewer registers, no dependency between mean and variance) and in having one parameter vector instead of two. Reported end-to-end speedups are modest (Zhang and Sennrich report 7–64% for the norm itself depending on the model), but at the scale of thousands of GPUs every elementwise microsecond counts. More importantly for fusion, RMSNorm's output depends on $\boldsymbol{x}$ only through one scalar per row, so it fuses especially cleanly into the preceding residual add ("add + RMSNorm" kernels are standard in vLLM and TensorRT-LLM) and even into the following GEMM's prologue.

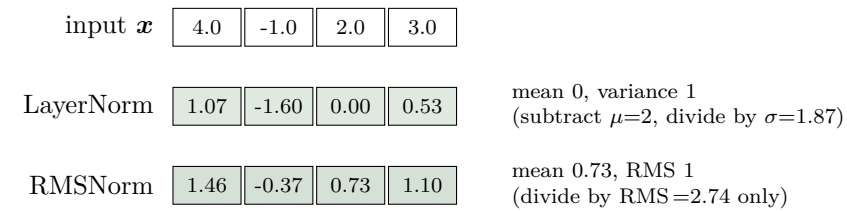


Figure 6.3: The same four-feature token normalized by LayerNorm and by RMSNorm ($\gamma = 1$). Both put the vector on a fixed scale; only LayerNorm also removes the mean.

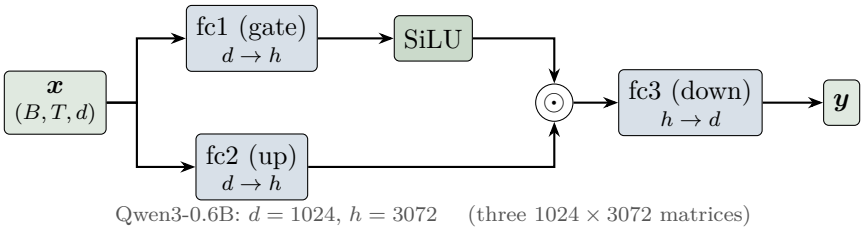


Figure 6.4: The SwiGLU feed-forward block: two parallel up-projections, one gated by SiLU, multiplied elementwise and projected back down.

6.3 Gated feed-forward networks: SwiGLU

RM C.2

Qwen3's MLP is a *gated linear unit* (Dauphin et al., 2017; Shazeer, 2020). Instead of one up-projection followed by a nonlinearity, it computes two up-projections of the same input; one passes through a nonlinearity and acts as a soft, input-dependent *gate* on the other, and the elementwise product is projected back down:

$$\mathrm{SwiGLU}(\boldsymbol{x}) = \mathbf{W}_3\big(\,\mathrm{SiLU}(\mathbf{W}_1\boldsymbol{x}) \odot \mathbf{W}_2\boldsymbol{x}\big), \qquad \mathrm{SiLU}(z) = z\,\sigma(z). \tag{6.2}$$

The name combines Swish (= SiLU with unit temperature) and GLU. Figure 6.4 shows the data flow; in the code we keep the names `fc1` (gate), `fc2` (up) and `fc3` (down) for the three bias-free linear layers.

Listing 6.2: SwiGLU feed-forward network.

```
class SwiGLUFeedForward(nn.Module):
    def __init__(self, emb_dim, hidden_dim, dtype=None):
        super().__init__()
        self.fc1 = nn.Linear(emb_dim, hidden_dim, bias=False, dtype=dtype)  # gate
        self.fc2 = nn.Linear(emb_dim, hidden_dim, bias=False, dtype=dtype)  # up
        self.fc3 = nn.Linear(hidden_dim, emb_dim, bias=False, dtype=dtype)  # down

    def forward(self, x):
        return self.fc3(nn.functional.silu(self.fc1(x)) * self.fc2(x))
```

Same budget, different shape. A GELU MLP with $d_{\mathrm{ff}} = 4d$ has $8d^2$ weights; a SwiGLU MLP with hidden width h has $3dh$. Matching the parameter (and FLOP) budget therefore means $h = \frac{8}{3}d$, which is why Llama-style models often have hidden sizes that look odd (e.g. $\frac{8}{3} \cdot 4096 \approx 11008$ in Llama 2-7B). It is a common misconception that the gated MLP is "more expensive": at equal parameters it costs the same, and Shazeer (2020) found it trains to lower perplexity. Qwen3 spends a bit more than parity: $h = 3d$ for 0.6B, 1.7B and 8B, and $h = 5d$ for 32B (Table 6.3).

Systems Lens: Three GEMMs or two?

Written naively, SwiGLU launches three GEMMs, a SiLU kernel and a multiply kernel, and stores two h-wide intermediates for the backward pass. Production implementations concatenate $\mathbf{W}_1$ and $\mathbf{W}_2$ into one $d \times 2h$ matrix, so a single GEMM with twice the output width produces both branches (better tile utilization, one weight read), then apply a fused "SiLU-and-multiply" kernel that reads $2h$ and writes h values per token. Under tensor parallelism (Chapter 8) the gate and up matrices are split column-wise across GPUs with the *same* partition, so each GPU can apply the gate locally and only the down projection needs an all-reduce—one reason gated MLPs fit Megatron-style parallelism so naturally.

Deeper Dive: GELU, SiLU, and friends

GELU weights its input by a Gaussian CDF; SiLU weights it by a logistic sigmoid; their curves differ by a few percent (Fig. 5.4). SiLU is cheaper to compute exactly (one exponential), which is one reason it became the default in gated MLPs. Variants with GELU as the gate (GeGLU, used in Gemma) behave almost identically. What matters far more than the choice of smooth activation is the multiplicative gate, which lets the network represent products of features directly.

6.4 Rotary position embeddings

RM C.3

GPT-2 adds a learned vector $\boldsymbol{p}_t$ to each token embedding. That has three drawbacks: the table has a fixed size (GPT-2 cannot see position 1024 at all), the position signal must survive through every layer's residual stream, and attention scores depend on absolute positions even though language is mostly translation invariant ("the cat" means the same at position 5 and 5000). Rotary position embeddings (RoPE; Su et al., 2021) inject position *inside* attention, by rotating queries and keys, in such a way that the score between a query at position m and a key at position n depends only on their offset $m - n$.

The idea in two dimensions. Take a 2-D query $\boldsymbol{q}$ and key $\boldsymbol{k}$. Rotate the query by angle $m\theta$ and the key by $n\theta$, where $R(\alpha)$ is the 2×2 rotation matrix:

$$\langle R(m\theta)\boldsymbol{q},\ R(n\theta)\boldsymbol{k}\rangle = \boldsymbol{q}^\top R(m\theta)^\top R(n\theta)\boldsymbol{k} = \boldsymbol{q}^\top R((n-m)\theta)\boldsymbol{k}. \tag{6.3}$$

Rotations compose by adding angles and $R^\top = R^{-1}$, so the dot product sees only the relative rotation. In d_h dimensions, RoPE splits the head vector into $d_h/2$ pairs and rotates pair i at its own frequency

$$\theta_i = \beta^{-2i/d_h}, \qquad i = 0, \ldots, d_h/2 - 1, \tag{6.4}$$

where β is the *base* (10,000 in the original paper, 1,000,000 in Qwen3). The first pairs rotate quickly (one radian per token for $i = 0$), the last ones very slowly; together they act like the hands of a clock with many hands, giving every position a unique, smoothly varying signature (Fig. 6.5).

Two equivalent layouts. The original formulation pairs *adjacent* dimensions $(2i, 2i+1)$; the GPT-NeoX/Llama/Qwen code pairs dimension i with $i + d_h/2$ ("rotate half"), which is cheaper to implement with slicing. The two are the same function up to a fixed permutation of the head dimensions: interleaved slot $2i$ holds halves-dimension i and slot $2i+1$ holds

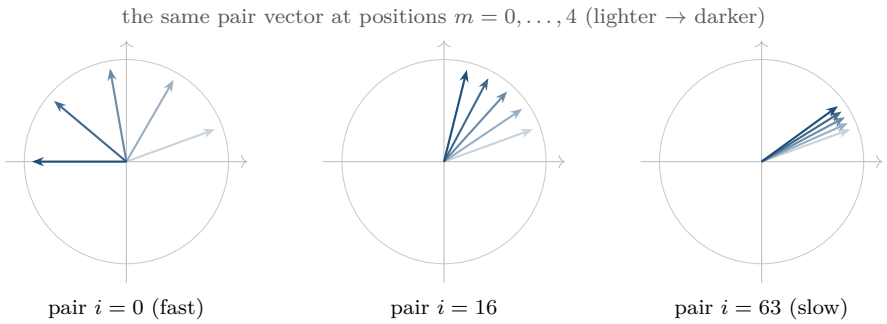


Figure 6.5: RoPE rotates each 2-D pair of a query/key head by an angle proportional to the token position, with a different speed per pair. (Angles exaggerated for visibility.)

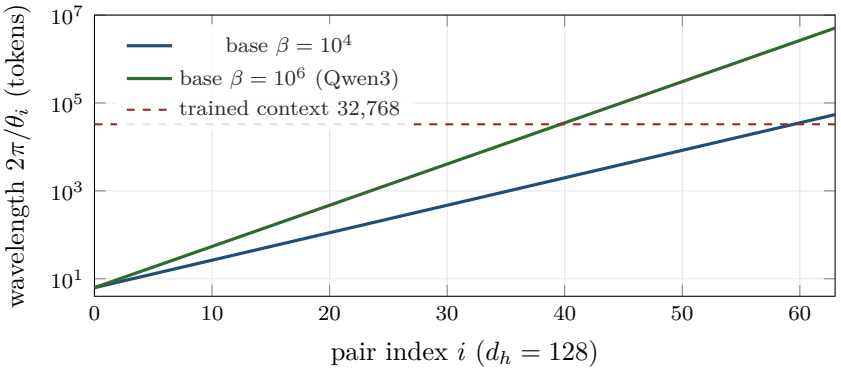


Figure 6.6: Wavelength of each RoPE frequency (analytic). With $\beta = 10^4$ the slowest pair completes a full turn every $\approx 54{,}000$ tokens; with $\beta = 10^6$ every $\approx 5.1 \times 10^6$ tokens, so many more pairs rotate by less than one turn across a long context and can encode coarse position.

$i + d_h/2$. Since the query and key projections are learned, permuting the rows of $\mathbf{W}_Q$ and $\mathbf{W}_K$ converts a checkpoint from one layout to the other, which is exactly what weight converters between frameworks do. Lab 6 verifies the equivalence numerically. We use the rotate-half layout throughout:

Listing 6.3: RoPE tables and application (rotate-half layout).

```
def compute_rope_params(head_dim, theta_base=10_000.0, context_length=4096):
    i2 = torch.arange(0, head_dim, 2).float()
    inv_freq = 1.0 / theta_base ** (i2 / head_dim)      # theta_i
    pos = torch.arange(context_length).float()
    angles = pos[:, None] * inv_freq[None, :]           # (T, D/2)
    angles = torch.cat([angles, angles], dim=1)         # freq i at i and i+D/2
    return torch.cos(angles), torch.sin(angles)

def apply_rope(x, cos, sin, offset=0):                  # x: (B, H, T, D)
    T, D = x.shape[-2], x.shape[-1]
    x1, x2 = x[..., : D // 2], x[..., D // 2:]
    rotated = torch.cat((-x2, x1), dim=-1)              # (x1, x2) -> (-x2, x1)
    c, s = cos[offset:offset + T], sin[offset:offset + T]
    return (x.float() * c + rotated.float() * s).to(x.dtype)
```

For each pair (x_1, x_2) the last line computes $(x_1 \cos\phi - x_2 \sin\phi,\ x_2 \cos\phi + x_1 \sin\phi)$, a rotation by ϕ. The `offset` argument is the absolute position of the first token in `x`; during cached decoding (Section 6.8) we process one token at a time and must rotate it by its true position,

not by zero.

Systems Lens: RoPE in a serving engine

RoPE has three systems consequences. (i) *No position table in the weights*: the cos/sin tables are recomputed at load time (we store them as non-persistent buffers so they never appear in the checkpoint), and the context limit becomes a property of training data rather than of the parameter file. (ii) *The KV cache stores rotated keys*: keys are rotated before they are written to the cache, so a cached key never needs to be touched again, and decoding applies RoPE to just one new query and key per layer. (iii) *Relative positions make left padding free*: in a batch whose rows are left-padded by different amounts (Chapter 12), every row is shifted by a constant, which cancels in Eq. (6.3); with GPT-2's absolute embeddings the same padding changes the outputs. Production kernels fuse RoPE into the QKV projection's epilogue or into the attention kernel's load of q and k, so the rotation costs no extra memory traffic.

Deeper Dive: Stretching the context: PI, NTK-aware scaling, YaRN

A RoPE model sees angles $m\theta_i$ only for m up to its training length. Beyond it, the fast pairs have wrapped around many times (fine) but the slow pairs reach angles never seen in training (not fine). *Position interpolation* (Chen et al., 2023) divides all positions by a factor s, squeezing a context of sT into the trained range at the cost of resolution between neighbouring tokens; it works with a short fine-tune. *NTK-aware* scaling instead enlarges the base to $\beta' = \beta\, s^{d_h/(d_h-2)}$, which leaves the fastest frequency unchanged and slows the slowest by exactly s—high-frequency (local) information is preserved, low-frequency (global) information is interpolated. *YaRN* (Peng et al., 2023) interpolates per frequency band and rescales attention temperature; it is what Qwen3 recommends for running beyond its native 32,768 tokens (up to 4×). Static YaRN scaling applies to every input regardless of length and can slightly hurt short prompts, so it should be enabled only when long inputs are actually expected. Lab 6 implements PI and the NTK base formula.

6.5 Grouped-query attention

RM C.4

In multi-head attention every query head has its own key and value head. During autoregressive decoding, keys and values of all past tokens are cached (Section 6.8), and reading that cache is the dominant memory traffic of long-context inference. *Multi-query attention* (MQA; Shazeer, 2019) shares a single key/value head among all query heads, shrinking the cache by $H\times$ but costing some quality. *Grouped-query attention* (GQA; Ainslie et al., 2023) interpolates: the H query heads are divided into G groups and each group shares one key/value head (Fig. 6.7). With $G = H$ it is MHA, with $G = 1$ it is MQA; Qwen3 uses $G = 8$ for every model size.

Listing 6.4: Grouped-query attention with QK-norm, RoPE and an optional KV cache.

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_in, num_heads, num_kv_groups, head_dim,
                 qk_norm=True, dtype=None):
        super().__init__()
        self.H, self.G, self.D = num_heads, num_kv_groups, head_dim
        self.group_size = num_heads // num_kv_groups
        kw = dict(bias=False, dtype=dtype)
        self.W_query = nn.Linear(d_in, self.H * self.D, **kw)
```

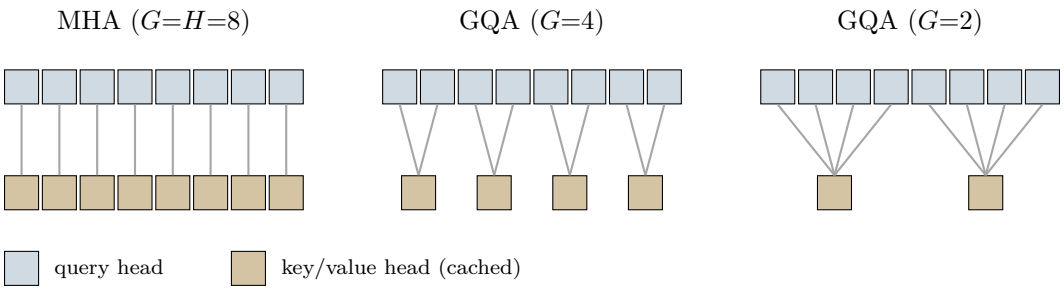


Figure 6.7: Multi-head vs. grouped-query attention with 8 query heads. Each orange square is a K/V head that must be stored in the KV cache; GQA keeps all query heads but cuts the cache by H/G.

```
        self.W_key = nn.Linear(d_in, self.G * self.D, **kw)     # G, not H, heads
        self.W_value = nn.Linear(d_in, self.G * self.D, **kw)
        self.out_proj = nn.Linear(self.H * self.D, d_in, **kw)
        self.q_norm = RMSNorm(self.D) if qk_norm else None
        self.k_norm = RMSNorm(self.D) if qk_norm else None

    def forward(self, x, cos, sin, cache=None):
        B, T, _ = x.shape
        q = self.W_query(x).view(B, T, self.H, self.D).transpose(1, 2)
        k = self.W_key(x).view(B, T, self.G, self.D).transpose(1, 2)
        v = self.W_value(x).view(B, T, self.G, self.D).transpose(1, 2)
        if self.q_norm is not None:                     # QK-norm, per head
            q, k = self.q_norm(q), self.k_norm(k)
        off = cache.seq_len(self.layer_idx) if cache is not None else 0
        q, k = apply_rope(q, cos, sin, off), apply_rope(k, cos, sin, off)
        if cache is not None:                           # cache holds G heads only
            k, v = cache.update(self.layer_idx, k, v)
        k = k.repeat_interleave(self.group_size, dim=1)   # expand to H heads
        v = v.repeat_interleave(self.group_size, dim=1)
        mask = causal_mask(T, k.shape[2], x.device)       # handles the offset
        scores = (q @ k.transpose(-2, -1)) / self.D ** 0.5
        scores = scores.masked_fill(mask, float("-inf"))
        w = torch.softmax(scores.float(), dim=-1).to(q.dtype)
        ctx = (w @ v).transpose(1, 2).reshape(B, T, self.H * self.D)
        return self.out_proj(ctx)
```

Compared to the MHA of Chapter 4, four lines changed: the key and value projections are G/H as wide; QK-norm normalizes each query and key head; RoPE rotates q and k with the cache offset; and `repeat_interleave` copies each K/V head to the H/G query heads of its group *after* the cache update, so the cache itself stays small. Note also that Qwen3-0.6B's query projection maps $d_{\text{model}} = 1024$ to $Hd_h = 16 \times 128 = 2048$: the attention width is decoupled from the residual width.

6.5.1 QK-norm

RM C.4

Qwen3 additionally applies an RMSNorm (with its own learned gain, shared across heads) to every query and key head vector before RoPE. Attention logits are dot products $\boldsymbol{q} \cdot \boldsymbol{k}$; if the norms of $\boldsymbol{q}$ and $\boldsymbol{k}$ drift upwards during training, the softmax saturates into near one-hot distributions, gradients vanish and the loss can spike. Normalizing q and k bounds the logits (Dehghani et al., 2023 used it to stabilize 22B-parameter vision transformers) and lets large

models train at higher learning rates. The cost is two tiny RMSNorms over $d_h = 128$ elements per head.

Napkin Math: KV-cache bytes per token

Each cached token stores one key and one value vector per KV head per layer:

$$\text{KV bytes/token} = 2 \times L \times G \times d_h \times \text{bytes/element}.$$

For Qwen3-0.6B in bf16: $2 \times 28 \times 8 \times 128 \times 2 = 114{,}688\,\text{B} = 112\,\text{KiB}$. With full MHA (16 K/V heads) it would be 224 KiB. For Qwen3-32B the saving is $64/8 = 8\times$: 256 KiB instead of 2 MiB per token. A single 32,768-token conversation with the 32B model therefore needs 8.6 GB of cache with GQA and would need 69 GB with MHA—more than the model's own weights. (For comparison, GPT-2 small needs only 36 KiB per token; Qwen3-0.6B's cache is three times larger because it is deeper and its attention is twice as wide as its residual stream.)

Systems Lens: GQA raises the arithmetic intensity of decoding

During a decode step each sequence contributes one query per head. Attention over a context of T tokens then costs $4THd_h$ FLOPs and must read $2 \cdot 2TGd_h$ bytes of bf16 cache, an arithmetic intensity of H/G FLOP/byte: 1 for MHA, the group size for GQA (2 for Qwen3-0.6B, 4 for 8B, 8 for 32B). That is still far below any GPU's ridge point—decode attention is memory bound no matter what—but it means the time spent reading the cache, which dominates long-context decoding, drops by the group size. A well-written kernel therefore does *not* materialize the `repeat_interleave`'d tensors as our reference code does; it loads each K/V tile once into shared memory and lets all H/G query heads of the group consume it (FlashAttention and FlashInfer do this; PyTorch's SDPA exposes it via `enable_gqa=True`). Our explicit expansion multiplies the bytes read back up by H/G and throws the benefit away—a good example of an optimization that exists only if the kernel knows about it.

6.6 The Qwen3 transformer block

RM C.5

The block is the GPT-2 block with the new parts slotted in (Fig. 6.8): RMSNorm before attention, GQA with QK-norm and RoPE, residual add, RMSNorm before the SwiGLU MLP, residual add. There is no dropout and no bias anywhere. The RoPE tables are owned by the model and passed down, so all 28 layers share one copy.

Listing 6.5: The Qwen3 transformer block.

```
class Qwen3Block(nn.Module):
    def __init__(self, cfg, dtype=None):
        super().__init__()
        self.att = GroupedQueryAttention(
            d_in=cfg["emb_dim"], num_heads=cfg["n_heads"],
            num_kv_groups=cfg["n_kv_groups"], head_dim=cfg["head_dim"],
            qk_norm=cfg["qk_norm"], dtype=dtype)
        self.ff = SwiGLUFeedForward(cfg["emb_dim"], cfg["hidden_dim"], dtype=dtype)
        self.norm1 = RMSNorm(cfg["emb_dim"])
        self.norm2 = RMSNorm(cfg["emb_dim"])
```

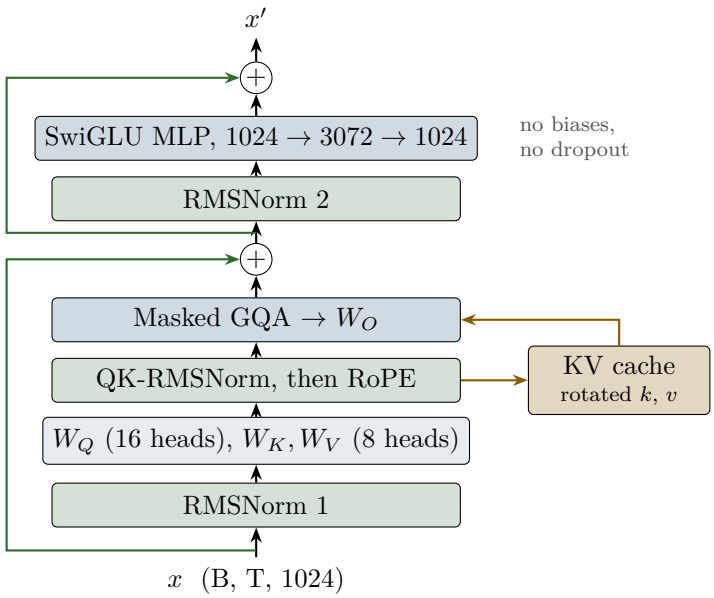


Figure 6.8: One Qwen3-0.6B transformer block (repeated 28 times), showing where the KV cache plugs in.

```

    def forward(self, x, cos, sin, cache=None, pad_mask=None):
        x = x + self.att(self.norm1(x), cos, sin, cache, pad_mask)
        x = x + self.ff(self.norm2(x))
        return x
```

6.7 The main model

RM C.6

Listing 6.6: Qwen3 configuration and model (cf. `llmsys/qwen3.py`).

```
QWEN3_CONFIG_06B = dict(
    vocab_size=151_936, context_length=40_960, emb_dim=1024,
    n_heads=16, n_kv_groups=8, head_dim=128, qk_norm=True,
    n_layers=28, hidden_dim=3072, rope_base=1_000_000.0, tie_embeddings=True)

class Qwen3Model(nn.Module):
    def __init__(self, cfg, dtype=None):
        super().__init__()
        self.cfg = cfg
        V, d = cfg["vocab_size"], cfg["emb_dim"]
        self.tok_emb = nn.Embedding(V, d, dtype=dtype)
        self.trf_blocks = nn.ModuleList(
            Qwen3Block(cfg, dtype) for _ in range(cfg["n_layers"]))
        for i, blk in enumerate(self.trf_blocks):
            blk.att.layer_idx = i                  # each layer knows its cache slot
        self.final_norm = RMSNorm(d)
        self.out_head = nn.Linear(d, V, bias=False, dtype=dtype)
        if cfg["tie_embeddings"]:
            self.out_head.weight = self.tok_emb.weight
        cos, sin = compute_rope_params(cfg["head_dim"], cfg["rope_base"],
                                       cfg["context_length"])
        self.register_buffer("cos", cos, persistent=False)  # not in checkpoint
```

```
        self.register_buffer("sin", sin, persistent=False)

    def forward(self, in_idx, cache=None, pad_mask=None):
        x = self.tok_emb(in_idx)                    # no position embedding!
        for blk in self.trf_blocks:
            x = blk(x, self.cos, self.sin, cache=cache, pad_mask=pad_mask)
        return self.out_head(self.final_norm(x))
```

A few configuration values deserve comment. The vocabulary of 151,936 rows is padded beyond the roughly 151.7k tokens the tokenizer actually defines—a multiple of 128 for the GEMM reasons of Section 5.1. `context_length` is 40,960 because the RoPE tables must cover the longest sequence we ever run, and Qwen configures that much headroom, although the model was trained on sequences of up to 32,768 tokens. Finally, the checkpoint is stored in bfloat16; we instantiate in bf16 directly (`dtype=torch.bfloat16`) so the 0.6B model needs about 1.2 GB rather than 2.4 GB, while the RoPE tables stay in fp32 for accuracy at large positions.

Pitfall: Casting the whole model to bf16

`model.to(torch.bfloat16)` converts *all* floating-point buffers, including our cos/sin tables. In bf16, positions above a few hundred are already represented with errors of several units in the last place of the angle, and the rotation becomes visibly wrong at long range. Construct the parameters in bf16 (or cast only `model.parameters()`), and keep the RoPE tables in fp32. Lab 6's `run.py` does the latter.

Napkin Math: Where Qwen3-0.6B's parameters are

Using the Lab 6 formula: the tied embedding is $151{,}936 \times 1024 = 155.6\,\text{M}$ parameters (26% of the model); each block has $1024 \cdot 2048 \cdot 2 + 1024 \cdot 1024 \cdot 2 + 3 \cdot 1024 \cdot 3072 \approx 15.7\,\text{M}$ (attention 6.3M, MLP 9.4M); 28 blocks give 440 M; total 596 M. In bf16 the weights take 1.19 GB. A decode step at batch 1 reads all of them: on an H100 at $3.35\,\text{TB}\,\text{s}^{-1}$ the lower bound is $1.19\,\text{GB}/3.35\,\text{TB}\,\text{s}^{-1} \approx 0.36\,\text{ms}$, i.e. at most $\approx 2{,}800$ tokens/s for a single stream—before any kernel launch overhead, which in practice dominates a model this small (Chapter 11).

6.8 KV cache

RM C.7

Autoregressive generation re-reads the whole prefix at every step (Section 5.7). Since a past token's keys and values in every layer depend only on the tokens before it, they never change and can be stored. With a KV cache, the first forward pass processes the prompt (*prefill*) and stores its keys and values; each later step processes *only* the newest token (*decode*), appends its key and value to the cache, and attends over the cached history. Chapter 11 studies the cache in depth; here we need only its interface, which our attention module already uses:

Listing 6.7: The two cache flavours (from `llmsys/kvcache.py`, abridged).

```
class KVCache:                          # grows by concatenation
    def __init__(self, n_layers):
        self.k, self.v = [None] * n_layers, [None] * n_layers
    def seq_len(self, layer_idx=0):
        return 0 if self.k[layer_idx] is None else self.k[layer_idx].shape[2]
    def update(self, layer_idx, k_new, v_new):
```

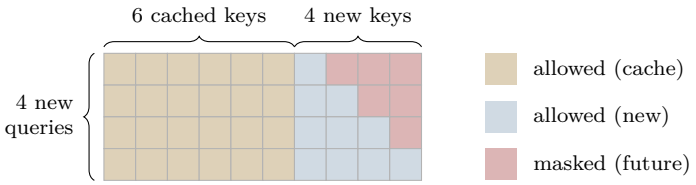


Figure 6.9: Causal mask for a chunk of 4 new tokens attending over a cache of 6 earlier tokens. The mask is `triu(ones(4,10), diagonal=6+1)`; for single-token decode it disappears entirely.

```
        if self.k[layer_idx] is None:
            self.k[layer_idx], self.v[layer_idx] = k_new, v_new
        else:
            self.k[layer_idx] = torch.cat([self.k[layer_idx], k_new], dim=2)
            self.v[layer_idx] = torch.cat([self.v[layer_idx], v_new], dim=2)
        return self.k[layer_idx], self.v[layer_idx]

class StaticKVCache:                # preallocated (B, G, max_len, D) buffers
    def update(self, layer_idx, k_new, v_new):
        s = self.lengths[layer_idx]; e = s + k_new.shape[2]
        self.k[layer_idx][:, :, s:e] = k_new
        self.v[layer_idx][:, :, s:e] = v_new
        self.lengths[layer_idx] = e
        return self.k[layer_idx][:, :, :e], self.v[layer_idx][:, :, :e]
```

The only subtle point is the causal mask. With T_q new queries and $T_k = T_{\text{past}} + T_q$ keys, query i sits at absolute position $T_{\text{past}} + i$ and may attend to keys $0, \ldots, T_{\text{past}} + i$. The mask is therefore a rectangular $(T_q \times T_k)$ matrix whose forbidden region starts at diagonal $T_{\text{past}} + 1$ (Fig. 6.9); in the decode case $T_q = 1$ and the mask is empty.

Systems Lens: Concatenation vs. preallocation

`torch.cat` allocates a new tensor and copies the whole history at every step, so generating n tokens moves $O(n^2)$ bytes per layer and fragments the allocator. The static cache allocates $(B, G, T_{\max}, d_h)$ once and writes one slice per step: $O(n)$ bytes, fixed addresses, and fixed shapes—which is what CUDA graphs and `torch.compile(mode="reduce-overhead")` need (Chapter 11). Its weakness is the opposite: it reserves memory for $T_{\max}$ tokens per sequence even if most sequences are short, which is exactly the waste PagedAttention eliminates (Chapter 12).

6.9 The tokenizer

RM C.8

Qwen3 uses a byte-level BPE tokenizer of the kind we built in Chapter 3, with roughly 151.6k learned tokens covering many languages and code, plus a few dozen *special tokens* that structure conversations and reasoning. The ones we need are:

Token	Role
`<|endoftext|>`	document separator in pretraining; used as padding
`<|im_start|>`, `<|im_end|>`	begin/end of a chat turn (ChatML); `<|im_end|>` stops generation
`<think>`, `</think>`	delimit the model's reasoning trace

A chat-tuned model expects its input in the format it was trained on. For Qwen3 that is

ChatML:

Listing 6.8: The Qwen3 chat format for one user turn, thinking disabled.

```
<|im_start|>user
What is 7 * 8?<|im_end|>
<|im_start|>assistant
<think>

</think>
```

Qwen3's post-trained checkpoints are *hybrid* reasoning models: by default the assistant first writes a reasoning trace inside `<think>...</think>` and then the answer. Prefilling an *empty* think block, as above, switches thinking off; leaving it out lets the model think. Our tokenizer wrapper (`llmsys.qwen3.Qwen3Tokenizer`) delegates the BPE itself to the fast Rust implementation in the `tokenizers` library and adds the chat template:

Listing 6.9: Applying the chat template (from `llmsys/qwen3.py`).

```
def apply_chat_template(self, prompt_or_messages, enable_thinking=False):
    msgs = prompt_or_messages
    if isinstance(msgs, str):
        msgs = [{"role": "user", "content": msgs}]
    s = "".join(f"<|im_start|>{m['role']}\n{m['content']}<|im_end|>\n"
                for m in msgs)
    s += "<|im_start|>assistant\n"
    if not enable_thinking:
        s += "<think>\n\n</think>\n\n"       # empty trace = answer directly
    return s
```

Pitfall: Base vs. chat formatting

A *base* checkpoint (e.g. `Qwen3-0.6B-Base`) has never been trained on ChatML; feed it plain text, and stop on `<|endoftext|>`. A post-trained checkpoint should always receive the template, and generation should stop on `<|im_end|>`. Mixing these up is the most common reason a from-scratch implementation "works" but produces rambling output. Evaluation numbers are also sensitive to the exact template; Chapter 14 returns to this.

6.10 Using the model

RM C.9

Loading the official weights is a matter of mapping names. The Hugging Face checkpoint is a set of safetensors files whose tensor names follow the `transformers` implementation; Table 6.2 gives the mapping to our modules, and `load_qwen3_hf_state_dict` applies it with a shape check on every tensor.

Listing 6.10: Loading Qwen3-0.6B and streaming a reply.

```
# one-time, on a machine with internet:
#   huggingface-cli download Qwen/Qwen3-0.6B --local-dir qwen3-0.6b
from llmsys.qwen3 import load_qwen3_from_dir, Qwen3Tokenizer
from llmsys.generate import generate_stream

device = "cuda" if torch.cuda.is_available() else "cpu"
model = load_qwen3_from_dir("qwen3-0.6b", size="0.6B",
```

Hugging Face name	Our module
`model.embed_tokens.weight`	`tok_emb.weight` (also the head if tied)
`model.layers.`i`.input_layernorm.weight`	`trf_blocks[`i`].norm1.scale`
`model.layers.`i`.self_attn.{q,k,v,o}_proj.weight`	`att.W_query/W_key/W_value/out_proj`
`model.layers.`i`.self_attn.{q,k}_norm.weight`	`att.q_norm.scale`, `att.k_norm.scale`
`model.layers.`i`.post_attention_layernorm.weight`	`trf_blocks[`i`].norm2.scale`
`model.layers.`i`.mlp.{gate,up,down}_proj.weight`	`ff.fc1`, `ff.fc2`, `ff.fc3`
`model.norm.weight`	`final_norm.scale`
`lm_head.weight` (untied models only)	`out_head.weight`

Table 6.2: Weight-name mapping between the Hugging Face Qwen3 checkpoint and our implementation.

```
                              dtype=torch.bfloat16, device=device)
tok = Qwen3Tokenizer("qwen3-0.6b/tokenizer.json")

prompt = "Explain RoPE in two sentences."
ids = torch.tensor([tok.encode(prompt, chat=True)], device=device)
for t in generate_stream(model, ids, 200, eos_id=tok.eos_token_id):
    print(tok.decode(t[0].tolist()), end="", flush=True)
```

Before trusting a from-scratch port, compare its logits with a reference implementation on a few prompts. Expect agreement to within bf16 noise (maximum absolute logit differences of order 10^{-2}), not bit-exactness: two correct implementations can sum in different orders, use different attention kernels, or upcast at different points. Large discrepancies almost always come from one of four bugs: a wrong RoPE layout, RoPE tables in bf16, forgetting QK-norm, or a missing chat template.

6.11 Scaling up: larger Qwen3 models

The 0.6B model is ideal for learning: it runs on a laptop and every experiment is fast. But the code in this chapter is size-agnostic, and running a larger model is mostly a matter of a different configuration and a different checkpoint.

6.11.1 Larger dense configurations

RM D.1

Table 6.3 lists the dense Qwen3 family. All members share the vocabulary, head dimension 128, eight KV heads, QK-norm and the RoPE base; they differ in width, depth, number of query heads and MLP width. The smaller three tie their embeddings; from 8B on, the output head is a separate matrix.

Model	d_{model}	L	H	G	h (MLP)	Tied	Params	KV/token (bf16)
0.6B	1024	28	16	8	3072	yes	0.60B	112 KiB
1.7B	2048	28	16	8	6144	yes	1.72B	112 KiB
4B	2560	36	32	8	9728	yes	4.02B	144 KiB
8B	4096	36	32	8	12288	no	8.19B	144 KiB
14B	5120	40	40	8	17408	no	14.77B	160 KiB
32B	5120	64	64	8	25600	no	32.76B	256 KiB

Table 6.3: The dense Qwen3 family. Parameter and KV-cache sizes computed from the configurations with the Lab 6 code. Note how the KV cache per token grows much more slowly than the parameter count.

6.11.2 Downloading larger checkpoints

RM D.2

Checkpoints above a few gigabytes are split into *shards* (`model-00001-of-00005.safetensors`, ...) plus a JSON index, `model.safetensors.index.json`, that maps every tensor name to the shard containing it. The simplest robust approach is to download the whole repository snapshot (`huggingface-cli download Qwen/Qwen3-8B -local-dir qwen3-8b`) and let the loader read every `*.safetensors` file in the directory, as `load_qwen3_from_dir` does. Keep each model in its own directory; mixing shards of different models in one folder is a classic way to load garbage silently, which is why our loader checks every tensor's shape.

6.11.3 Loading a larger base model

RM D.3

Naive loading of an 8B model needs memory for *two* copies of the weights: the randomly initialized model and the state dict read from disk. Three standard tricks avoid that (Lab 6's `run.py` uses the first and third):

Listing 6.11: Memory-frugal loading: meta-device construction and per-shard copy.

```
from safetensors import safe_open

with torch.device("meta"):              # 1) allocate no storage at all
    model = Qwen3Model(QWEN3_CONFIGS["8B"], dtype=torch.bfloat16)
model = model.to_empty(device="cuda")   # 2) uninitialized storage on the GPU
for shard in sorted(glob.glob("qwen3-8b/*.safetensors")):
    # 3) memory-map the shard and copy one tensor at a time
    with safe_open(shard, framework="pt", device="cuda") as f:
        for name in f.keys():
            copy_into(model, name, f.get_tensor(name))  # name map: Table 6.2
# buffers created on the meta device (RoPE tables) must be recomputed
```

Meta-device construction skips the random initialization (which for 8B parameters is also tens of seconds of CPU time), `to_empty` materializes uninitialized storage directly on the GPU, and `safe_open` memory-maps each shard so only one tensor at a time is resident in host memory. Peak memory is then the model plus one tensor, rather than twice the model.

6.11.4 Loading a larger reasoning variant

RM D.4

Each Qwen3 size is published as a `-Base` checkpoint (pretrained only) and as a post-trained hybrid-thinking checkpoint without suffix. Switching between them changes only the weights directory and the tokenizer settings: the reasoning variant needs the chat template and stops on `<|im_end|>`, and thinking is controlled by the empty-think-block trick of Section 6.9. Everything else—model class, generation loop, KV cache—is identical. In Chapters 14 and 15 we will use the base 0.6B model as the student we try to improve, and the reasoning variants as reference points and as distillation teachers (Chapter 20).

6.11.5 Practical recommendations and memory planning

RM D.5

Which size should you use? The honest systems answer is: the largest one whose weights *plus the KV cache you need* fit in memory with headroom. Table 6.4 applies the formulas of this chapter to two common devices, assuming bf16 weights and cache and 10% of memory reserved for activations, workspace and fragmentation.

	24 GB GPU (e.g. RTX 4090, L4)		80 GB GPU (A100/H100)	
Model	weights	max. 8k-token sequences	weights	max. 8k-token sequences
0.6B	1.2 GB	21	1.2 GB	75
1.7B	3.4 GB	19	3.4 GB	72
4B	8.0 GB	11	8.0 GB	52
8B	16.4 GB	4	16.4 GB	46
14B	29.5 GB	–	29.5 GB	31
32B	65.5 GB	–	65.5 GB	3

Table 6.4: How many concurrent 8192-token sequences fit next to the weights (bf16, 10% reserve), from `max_concurrent_sequences` in Lab 6. This number bounds the batch size, and hence the throughput, of a serving engine.

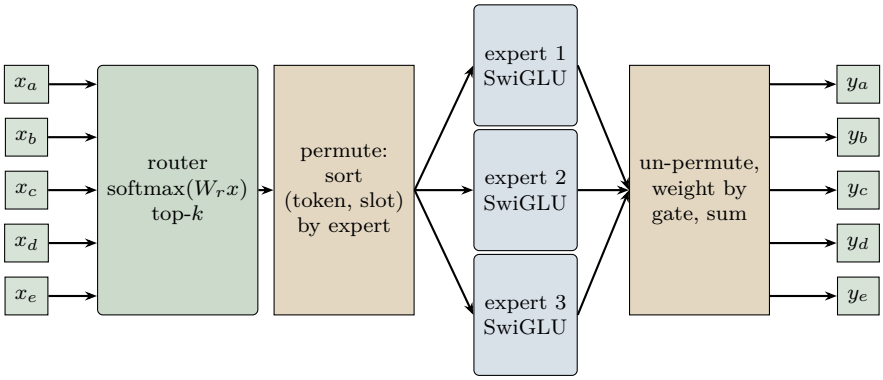


Figure 6.10: The MoE data path as implemented in `llmsys/moe.py`: route, permute token copies into per-expert contiguous groups, run each expert as one batched GEMM, then un-permute and combine with the gate weights. In expert-parallel training the permute and un-permute steps become all-to-all exchanges between GPUs.

A few rules of thumb follow. For the experiments in this book, 0.6B on any GPU (or even CPU) is enough. If you have a 24 GB card, the 4B model is the sweet spot for RL and evaluation: it fits with room for a useful batch. The 8B model fits on 24 GB but leaves room for only a handful of long sequences, which makes batched sampling (Chapters 15 and 17) slow; quantization (Chapter 13) or a larger GPU helps. The 32B model needs either an 80 GB GPU with 8-bit weights, or tensor parallelism across two or more GPUs (Chapters 8 and 13).

6.12 Beyond dense models: mixture of experts

Every token in a dense model uses every parameter. A *mixture-of-experts* (MoE) layer replaces the single MLP with E smaller MLPs ("experts") and a learned *router* that sends each token to only $k \ll E$ of them (Shazeer et al., 2017; Fedus et al., 2022). The parameter count grows with E, but the FLOPs per token grow only with k. Qwen3 itself ships two MoE models next to the dense ones; Qwen3-30B-A3B, for example, has 48 layers each with $E = 128$ experts of hidden width 768, of which $k = 8$ are active per token. Its total parameter count is about 30.5B, but only about 3.3B are used for any given token (both numbers from our `moe_param_counts` applied to its configuration), so its per-token compute is close to a 3B dense model while its capacity is closer to a 30B one.

Routing. For a token $\boldsymbol{x}$ the router computes probabilities $\boldsymbol{p} = \mathrm{softmax}(\mathbf{W}_r \boldsymbol{x})$ over the E experts, keeps the k largest, renormalizes them to sum to one (the Qwen-MoE and Mixtral convention), and outputs $\boldsymbol{y} = \sum_{e \in \text{top-}k} p_e \, \mathrm{Expert}_e(\boldsymbol{x})$. The reference implementation in Listing 6.12 makes the systems structure explicit: rather than looping over tokens, it sorts the (token, slot) assignments by expert so that each expert processes one contiguous slice—the single-GPU analogue of the permutation that expert-parallel systems perform with an all-to-all.

Listing 6.12: Sort-based MoE dispatch (from `llmsys/moe.py`, abridged).

```
def forward(self, x):
    x2d = x.reshape(-1, x.shape[-1]); N = x2d.shape[0]
    probs = torch.softmax(self.gate(x2d).float(), dim=-1)       # (N, E)
    w, idx = torch.topk(probs, self.top_k, dim=-1)              # (N, k)
    w = (w / w.sum(-1, keepdim=True)).to(x.dtype)
    expert = idx.flatten()                                      # (N*k,)
    token = torch.arange(N).repeat_interleave(self.top_k)
    order = torch.argsort(expert, stable=True)                  # permute
    expert, weight, token = expert[order], w.flatten()[order], token[order]
    counts = torch.bincount(expert, minlength=self.n_experts).tolist()
    out, start = torch.zeros_like(x2d), 0
    for e, n in enumerate(counts):
        if n:
            t, g = token[start:start + n], weight[start:start + n, None]
            out.index_add_(0, t, g * self.experts[e](x2d[t]))   # combine
            start += n
    return out.reshape(x.shape)
```

Load balancing. Left alone, the router tends to collapse: a few experts win early, receive more gradient, get better, and win more. The standard remedy is an auxiliary loss that is minimized by uniform routing. With f_e the fraction of token slots routed to expert e (a count, not differentiable) and P_e the mean router probability of expert e (differentiable),

$$\mathcal{L}_{\text{aux}} = E \sum_{e=1}^{E} f_e \, P_e, \tag{6.5}$$

which equals 1 under perfect balance and grows towards E under collapse (Fedus et al., 2022). It is added to the language-modeling loss with a small coefficient. Some recent models replace it with bias-based "auxiliary-loss-free" balancing (DeepSeek-V3), or compute balance statistics over a global batch rather than per micro-batch (as the Qwen team reports doing).

Systems Lens: Why MoE is a systems problem

MoE trades FLOPs for memory and communication. (i) *Memory*: all E experts must be resident somewhere, so a 30B-total MoE needs the memory of a 30B dense model even though it computes like a 3B one; it is cheap to *run* only once you can afford to *hold* it. (ii) *Expert parallelism*: experts are spread across GPUs, and every MoE layer performs two all-to-all exchanges (dispatch tokens to their experts, return results). Per token and layer that is $2kd_{\text{model}}$ bf16 values on the wire—for Qwen3-30B-A3B, $2 \cdot 8 \cdot 2048 \cdot 2 = 64\,\text{KiB}$ per token per layer, which at 48 layers and thousands of tokens per batch quickly approaches NVLink bandwidth (Chapter 8). (iii) *Load imbalance*: the slowest expert determines the layer's latency, so systems either impose a *capacity* (each expert processes at most

$c \cdot Nk/E$ tokens; overflow tokens are dropped and pass through the residual connection) or use dropless grouped-GEMM kernels (MegaBlocks; Gale et al., 2023) that handle ragged per-expert batches efficiently. (iv) *Small GEMMs*: each expert sees only about Nk/E tokens, so expert GEMMs are much narrower than a dense MLP's and are often memory bound—during decode especially, where an MoE model reads far more distinct weights per step than a dense model of equal active size.

Napkin Math: Total vs. active parameters

Per MoE layer with SwiGLU experts of hidden width h: each expert has $3dh$ weights; the router has dE. For Qwen3-30B-A3B ($d = 2048$, $h = 768$, $E = 128$, $k = 8$) one expert is 4.72 M parameters, so a layer holds 604 M expert parameters but activates 37.7 M. Over 48 layers plus attention (18.9 M per layer with 32 query and 4 KV heads) and untied embeddings (622 M), the total is ≈ 30.5 B and the active count ≈ 3.35 B. Training FLOPs scale with the active count (6×3.35B per token), but serving memory scales with the total. At batch 1 a decode step reads ≈ 6.7 GB of active bf16 weights; at large batch, nearly all experts are hit and the step reads close to all 61 GB.

Deeper Dive: Other directions in attention design

GQA is not the only way to shrink the KV cache. *Multi-head latent attention* (MLA, DeepSeek-V2, 2024) caches a single low-rank latent vector per token from which per-head keys and values are reconstructed, compressing the cache further than GQA while keeping per-head expressivity. *Sliding-window* attention (Mistral, Gemma) limits some or all layers to the last w tokens, bounding their cache at w entries. *Hybrid* architectures interleave a minority of full-attention layers with linear-attention or state-space layers whose state is constant-size, so most layers need no growing cache at all (Qwen3-Next is one such design). All of these are, at heart, answers to the same napkin-math question as GQA: how many bytes per token does the model need to remember, and how fast can the hardware read them?

6.13 Summary

- Qwen3 keeps GPT-2's pre-norm decoder skeleton but replaces LayerNorm with RMSNorm, the GELU MLP with SwiGLU, learned positions with RoPE, and MHA with GQA plus QK-norm; it drops biases and dropout.
- RMSNorm needs one row statistic instead of two and fuses cleanly with residual adds; compute it in fp32.
- SwiGLU at hidden width $\frac{8}{3}d$ matches a GELU MLP's budget; production code fuses gate and up projections into one GEMM.
- RoPE rotates query/key pairs at geometrically spaced frequencies so that scores depend only on relative position; it needs no weights, makes the cache store rotated keys, and makes left padding harmless. Keep its tables in fp32.
- GQA shares each K/V head among H/G query heads, cutting the KV cache (and decode attention traffic) by H/G; kernels must avoid materializing the expanded heads to realize

the gain.

- KV bytes per token $= 2LGd_h \cdot$bytes; Qwen3-0.6B needs 112 KiB, Qwen3-32B 256 KiB. Together with the weights, this determines how many sequences fit on a device.
- Larger dense Qwen3 models use the same code; load them with meta-device construction and per-shard copying to avoid holding two copies.
- Mixture-of-experts layers decouple parameters from per-token FLOPs at the cost of memory, all-to-all communication and load-balancing complexity.

6.14 Exercises

Exercise 6.1 (*RMSNorm vs. LayerNorm*). Show that if every token vector has zero mean, RMSNorm and LayerNorm (with $\beta = 0$) produce identical outputs. Then measure, with `saved_tensors_hooks` from Chapter 5, how many bytes each of the two explicit implementations saves for the backward pass on a $(4, 256, 1024)$ input.

Exercise 6.2 (*SwiGLU parity*). A GELU MLP with $d_{\text{ff}} = 4d$ and a SwiGLU MLP with hidden width h are to have equal parameter counts. Derive h. Llama 2-7B uses $d = 4096$ and $h = 11008$; by what percentage does it deviate from parity, and why might a model designer round h to a multiple of 256?

Exercise 6.3 (*RoPE relativity*). Prove Eq. (6.3) for the d_h-dimensional case in the rotate-half layout by writing the rotation as a block-diagonal matrix in a permuted basis. Then verify numerically that $\langle \text{RoPE}_m(q), \text{RoPE}_n(k) \rangle$ is unchanged when both m and n are shifted by the same amount, and explain why that makes left padding harmless.

Exercise 6.4 (*Layout conversion*). Write a function that converts the $\mathbf{W}_Q$ and $\mathbf{W}_K$ weights of a checkpoint trained with interleaved RoPE into the rotate-half layout, and test it end to end with a tiny GQA module. Does $\mathbf{W}_V$ need to be permuted? Why or why not?

Exercise 6.5 (*KV budget*). For Qwen3-8B, compute the KV-cache size of one 131,072-token context in bf16 and in fp8. Could one such context fit on an 80 GB GPU next to the bf16 weights? What if you also quantize the weights to 8 bits?

Exercise 6.6 (*GQA intensity*). Derive the arithmetic intensity of single-token decode attention as a function of H, G and the bytes per cache element, and evaluate it for Qwen3-0.6B, 8B and 32B in bf16 and fp8. How does the materialized `repeat_interleave` in our reference code change the bytes actually read?

Exercise 6.7 (*RoPE precision*). Compute the RoPE tables for Qwen3 in fp32 and in bf16 and plot the absolute angle error $|\phi_{\text{bf16}} - \phi_{\text{fp32}}|$ for the fastest frequency at positions 0 to 32,768. At roughly what position does the error exceed 0.1 radians?

Exercise 6.8 (*Context extension*). Using Lab 6's `ntk_aware_base`, compute the base Qwen3 would need to stretch its trained context by 4× with NTK-aware scaling. Which pair index i has its wavelength changed by the largest factor, and which by the smallest?

Exercise 6.9 (*MoE capacity*). With $N = 4096$ tokens, $E = 64$, $k = 2$ and capacity factor $c = 1.25$, how many token slots can each expert accept? If routing were uniformly random, use a normal approximation to estimate the expected fraction of dropped token slots. How does the answer change with $c = 1.0$ and $c = 2.0$?

Exercise 6.10 (*Expert-parallel communication*). Qwen3-30B-A3B runs with its 128 experts split evenly over 8 GPUs. For a batch of 16,384 tokens, estimate the all-to-all bytes per MoE layer per GPU (dispatch plus combine, bf16), and the time on NVLink at $450\,\mathrm{GB\,s^{-1}}$ per direction. Compare with the time of the experts' GEMMs on one H100.

Exercise 6.11 (*Memory-frugal loading*). Instrument Listing 6.11 to record peak memory (CUDA's `max_memory_allocated`, or the resident set size on CPU) and compare it with naive loading via `load_state_dict` for the 1.7B model.

6.15 Lab 6: Qwen3 from scratch, and planning its memory

Lab 6: Qwen3 from scratch, and planning its memory

Code: `code/labs/lab06_qwen3`

Goals

Re-implement every Qwen3 component so that your model is logit-identical to `llmsys.qwen3` (and therefore loads the official weights), prove RoPE's properties numerically, implement MoE dispatch the way systems do, and turn the chapter's memory formulas into a planning tool.

Setup

```
cd code
pytest -q labs/lab06_qwen3                          # tests your starter.py
python labs/lab06_qwen3/run.py --impl starter --plan --gpu-gb 24 --seq 8192
```

Tasks

1. **RMSNorm.** Implement `rms_norm` with fp32 accumulation.
2. **RoPE.** Implement `compute_rope`, `apply_rope` (rotate-half), `apply_rope_interleaved` (complex arithmetic), and `halves_to_interleaved_perm`. The tests check the relative-position property and the layout equivalence. Implement `position_interpolation_rope` and `ntk_aware_base`.
3. **SwiGLU.** Implement `SwiGLU.forward`.
4. **GQA.** Implement `GroupedQueryAttention.forward` with QK-norm, RoPE with cache offset, cache update storing only G heads, and masked attention. The test checks prefill-then-decode against the library.
5. **Model.** Implement `Qwen3Block.forward` and `Qwen3Model.forward`.
6. **Memory planning.** Implement `kv_cache_bytes`, `count_params` (exact, tied embed-

dings counted once) and `max_concurrent_sequences`.

7. **MoE.** Implement `route_topk`, `load_balancing_loss` and the sort-based `moe_dispatch_forward`, checked against `llmsys.moe.MoEFeedForward.forward_reference`.

Measurements

- With internet access, download `Qwen/Qwen3-0.6B` and run `run.py -impl starter -weights-dir qwen3-0.6b`, once with and once without `-thinking`. Record tokens/s on your device and compare with the napkin-math bound of Section 6.7.
- Plot decode time per token versus context length (512 to 16k) with the materialized `repeat_interleave` and with `F.scaled_dot_product_attention(..., enable_gqa=True)` on a GPU. Expect the gap to widen with context.
- Time `MoEFeedForward.forward` vs. `forward_reference` for $E \in \{8, 32, 128\}$; record the tokens-per-expert histogram and the auxiliary loss for a random router.

Deliverables

Passing tests; your memory-planning table for the GPU you actually have; the decode-time plot with a paragraph relating it to Section 6.5's intensity argument.

Stretch goals

- Load `Qwen/Qwen3-4B` or `-8B` with the meta-device loader and report peak memory.
- Add a capacity factor to your dispatch and measure the fraction of dropped tokens for a trained vs. a random router.
- Fuse gate and up projections into a single $d \times 2h$ `nn.Linear`, convert the checkpoint weights accordingly, and measure the speedup.

Part III

Training

Chapter 7

Pretraining on Unlabeled Data

In Chapter 5 we assembled a GPT model whose weights are random numbers. It produces fluent-looking *shapes* of text—token IDs in the right range, at the right rate—but no meaning. This chapter turns that untrained network into a language model. We will define what "good" means for a next-token predictor (cross-entropy and perplexity), write the training loop, train a small model on a short story, look at how sampling choices change what the model says, save and restore every bit of training state, and finally load the weights OpenAI released for GPT-2 so that the remaining chapters can start from a competent model.

For a systems reader the more interesting half of the chapter comes after the loop works. Pretraining is the single most expensive thing anyone does with a language model: frontier runs burn 10^{24}–10^{26} floating-point operations on tens of thousands of accelerators for months. We therefore spend the second half on the *economics* of a training step: where the bytes live (weights, gradients, optimizer moments, activations), how mixed precision and activation checkpointing change that budget, how to measure efficiency with model FLOPs utilization (MFU), how to checkpoint without stalling the cluster, and how scaling laws tell you how many parameters and tokens to buy with a given compute budget. Chapter 8 then spreads this single-device loop across many devices.

7.1 Evaluating generative text models

LLM 5.1

Before we can improve a model we need a number that says how bad it currently is. For classification, accuracy is the obvious choice; for a generative model it is not, because the model's job is to put probability on *all* plausible continuations, not to hit a single label. The standard metric is the average negative log-likelihood of held-out text, which is also exactly the quantity we minimize during training.

7.1.1 Generating text with an untrained model

LLM 5.1.1

We reuse `GPTModel` from Chapter 5 with one change: the context window is reduced from 1,024 to 256 tokens. Attention cost grows quadratically with the window (Chapter 4), and our first training corpus is a short story of about five thousand tokens, so long windows would only shrink the number of training examples. Dropout can be set to 0.1 or 0; modern LLM pretraining usually uses 0 because a single pass over a huge corpus rarely overfits.

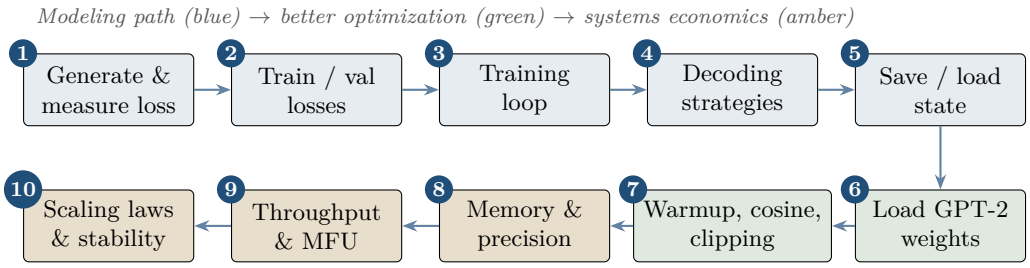


Figure 7.1: Roadmap of this chapter. Steps 1–6 take an untrained GPT to a pretrained one; step 7 makes optimization robust; steps 8–10 are the systems view of what a training step costs and how to spend a compute budget.

Two tiny helpers convert between text and the `(batch, seq)` tensors the model expects. They exist mostly so that every later listing reads naturally.

Listing 7.1: Text ↔ token-ID helpers and a first, meaningless generation.

```python
import torch
from llmsys.gpt import GPTModel, GPT_CONFIG_124M
from llmsys.generate import generate_simple

def text_to_ids(text, tok):
    return torch.tensor(tok.encode(text)).unsqueeze(0)      # add batch dim

def ids_to_text(ids, tok):
    return tok.decode(ids.squeeze(0).tolist())               # drop batch dim

cfg = dict(GPT_CONFIG_124M, context_length=256)
torch.manual_seed(123)
model = GPTModel(cfg).eval()
out = generate_simple(model, text_to_ids("Every effort moves you", tok),
                      max_new_tokens=10, context_size=cfg["context_length"])
print(ids_to_text(out, tok))   # -> prompt followed by 10 random-looking word pieces
```

With random weights the output distribution over the 50,257-token vocabulary is close to uniform, so the generated continuation is a sequence of unrelated word pieces. Our goal is to make the probability of the *actual* next token large.

7.1.2 The text generation loss: cross-entropy and perplexity

LLM 5.1.2

Take a batch of inputs $\mathbf{X} \in \{0, \dots, V-1\}^{B \times T}$ and the targets $\mathbf{Y}$, which are the same token sequences shifted left by one position (Chapter 3). The model maps $\mathbf{X}$ to logits $\mathbf{Z} \in \mathbb{R}^{B \times T \times V}$. For each position (b, t) a softmax turns the logit row into a distribution,

$$p_\theta(v \mid x_{b,\le t}) \;=\; \frac{\exp(z_{b,t,v})}{\sum_{u=1}^{V} \exp(z_{b,t,u})}, \tag{7.1}$$

and training wants the probability assigned to the true next token $y_{b,t}$ to be as close to one as possible. Multiplying probabilities over positions underflows immediately, so we work with logarithms and average over the BT positions:

$$\mathcal{L}(\theta) \;=\; -\frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \log p_\theta(y_{b,t} \mid x_{b,\le t}) \;=\; -\frac{1}{BT} \sum_{b,t} \Big(z_{b,t,y_{b,t}} - \operatorname{logsumexp}_u z_{b,t,u} \Big). \tag{7.2}$$

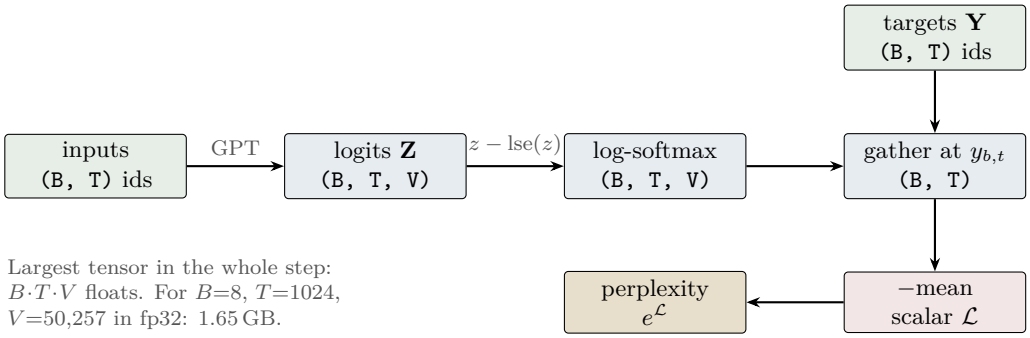


Figure 7.2: From token IDs to a scalar loss. The target IDs only select one entry per row of the log-probability tensor; everything else in that V-wide row only enters through the normalizer.

This is the *cross-entropy* between the one-hot data distribution and the model's distribution, and the right-hand form is how it should be computed: subtracting the log-sum-exp directly never forms tiny probabilities that would round to zero. Figure 7.2 traces the computation for one sequence.

Listing 7.2: Cross-entropy from scratch (Lab 7, Task 1). It matches `F.cross_entropy` to floating-point precision.

```python
def cross_entropy_from_scratch(logits, targets):
    logits = logits.float()                                  # (B, T, V)
    lse = torch.logsumexp(logits, dim=-1, keepdim=True)      # (B, T, 1)
    log_probs = logits - lse                                 # log softmax
    target_lp = log_probs.gather(-1, targets.unsqueeze(-1)).squeeze(-1)
    return -target_lp.mean()

# PyTorch's fused version expects (N, V) logits and (N,) targets:
loss = torch.nn.functional.cross_entropy(logits.flatten(0, 1), targets.flatten())
```

Perplexity. The exponential of the loss, $\text{PPL} = e^{\mathcal{L}}$, has a pleasant reading: it is the size of a uniform distribution that would be equally surprised by the data. A freshly initialized GPT-2 assigns roughly uniform probability to all $V = 50{,}257$ tokens, so its loss is close to $\ln 50{,}257 \approx 10.82$ and its perplexity close to 50,257. A model with perplexity 20 is, on average, "choosing among 20 equally likely tokens".

Key Idea: The initial-loss sanity check

At initialization the logits are small and nearly equal, so the loss should be within a few tenths of $\ln V$. If your first loss is 30 or 3, something is broken (logit scaling, targets not shifted, padding counted as targets, wrong vocabulary size). This one-line check catches a surprising fraction of pipeline bugs before any GPU-hours are spent.

Pitfall: Perplexities are tokenizer-dependent

Loss is measured *per token*. Two models with different tokenizers see different numbers of tokens for the same text, so their perplexities are not comparable. When comparing across tokenizers, report bits per byte (or per character): multiply the per-token loss by tokens/bytes and divide by $\ln 2$.

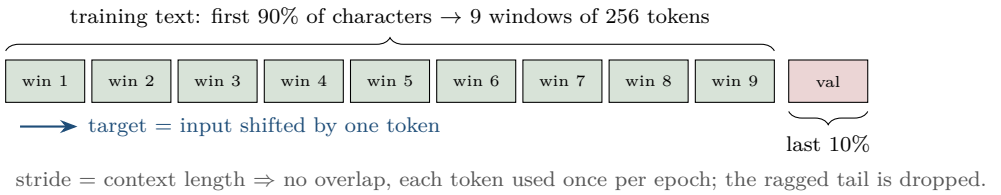


Figure 7.3: Splitting a short corpus into training and validation windows.

Systems Lens: The logits tensor is the memory hog of the loss

Everything before the output head scales with d_{model}; the logits scale with the vocabulary. For GPT-2 small at $B = 8$, $T = 1024$ the logits hold $8 \cdot 1024 \cdot 50{,}257 \approx 4.1 \times 10^8$ values: 1.65 GB in fp32, plus the same again for their gradient, plus the softmax output if the implementation materializes it. With Qwen-style vocabularies of 152k tokens it is three times worse. Production stacks therefore never materialize the full logits during training:

- **Chunked / fused cross-entropy** computes the head matmul, log-sum-exp and gradient chunk by chunk over the sequence (e.g. the Liger kernels, or `torch.compile` fusing the head and loss), so only one chunk of logits is alive at a time.
- **Vocabulary-parallel cross-entropy** (Megatron-LM) shards the output head across tensor-parallel ranks; each rank computes a partial max and partial sum-of-exponentials over its vocabulary slice and two tiny all-reduces combine them (Chapter 8).

Always cast logits to fp32 before the log-sum-exp; bf16 has only 8 bits of mantissa and the normalizer of a 50k-way softmax loses most of its precision otherwise.

LLM 5.1.3

7.1.3 Training and validation losses

Our first corpus is deliberately tiny: Edith Wharton's 1908 short story "The Verdict" (public domain), the same text used to build the tokenizer pipeline in Chapter 3. It is about 20,000 characters, which the GPT-2 tokenizer turns into roughly 5,100 tokens. That is small enough to train on a laptop in minutes and large enough to watch every phenomenon of pretraining appear—including overfitting.

We split the *characters* 90/10 into training and validation text, tokenize each part, and cut both into non-overlapping windows of the context length using the sliding-window dataset of Chapter 3 with `stride = context_length` (Fig. 7.3). The validation loader keeps `drop_last=False` and `shuffle=False` so that every validation token is seen and results are deterministic. A useful sanity check is to count the tokens in each loader: with a context of 256 and batch size 2, the 90% split yields only nine training batches and the 10% split a single validation batch. A corpus this small is a toy, and we will treat the results as such.

The function that evaluates a loader simply averages the per-batch loss over at most `num_batches` batches (`llmsys.train.loss_loader`); limiting the number of batches keeps periodic evaluation cheap on large datasets. Evaluated before any training, both losses come out near $\ln V \approx 11$, as expected.

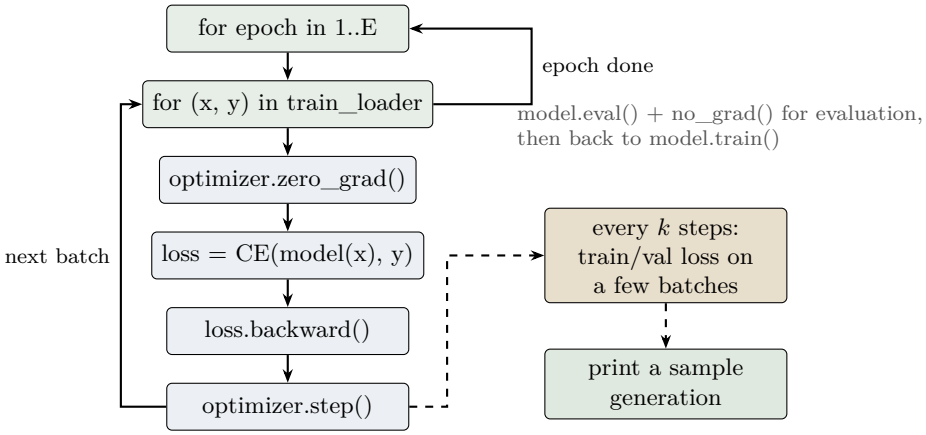


Figure 7.4: The pretraining loop. Solid arrows are executed every step; dashed arrows only periodically, because evaluation and generation are pure overhead for the optimizer.

Napkin Math: How big is "small"?

5,100 tokens is nothing. GPT-2 was trained on WebText, about 40 GB of text (on the order of 10^{10} tokens). Llama 3 used about 1.5×10^{13} tokens. At a chat-like 3,000 tokens per day, a person would need roughly 14 million years to read Llama 3's training set. Our corpus is one afternoon of reading—so the model will memorize it.

7.2 Training an LLM

LLM 5.2

The pretraining loop is the ordinary deep-learning loop (Chapter 2) applied to the next-token loss. Figure 7.4 shows its structure: an outer loop over epochs, an inner loop over batches, and the five-beat rhythm of zero-grad, forward, loss, backward, step. Periodically we evaluate both losses and print a sample generation so a human can see whether the model is learning language or noise.

Listing 7.3: A minimal pretraining loop with periodic evaluation.

```
from llmsys.train import loss_batch, loss_loader

def train_simple(model, train_loader, val_loader, optimizer, device, num_epochs,
                 eval_freq, eval_iter, start_context, tok):
    history, tokens_seen, step = [], 0, 0
    for epoch in range(num_epochs):
        model.train()
        for x, y in train_loader:
            optimizer.zero_grad()
            loss = loss_batch(model, x, y, device)
            loss.backward()
            optimizer.step()
            tokens_seen += x.numel()
            step += 1
            if step % eval_freq == 0:
                model.eval()
                with torch.no_grad():
                    tr = loss_loader(model, train_loader, device, eval_iter)
                    va = loss_loader(model, val_loader, device, eval_iter)
                model.train()
```

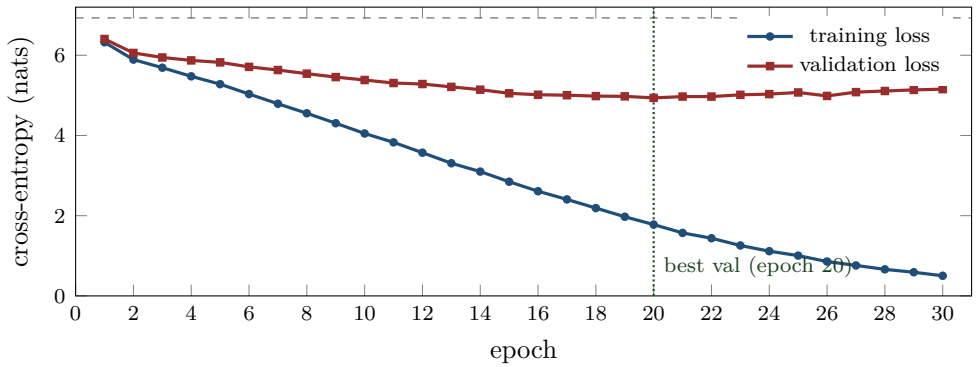


Figure 7.5: Overfitting on a tiny corpus, measured on CPU in this book's build environment: a 3-layer, 96-wide GPT (vocabulary 1,024, context 64) trained for 30 epochs on 18k characters (12 batches of 8×64 tokens per epoch) with AdamW, LR 10^{-3}. Training loss falls steadily toward zero while validation loss bottoms out at epoch 20 and then rises: the model is memorizing.

```
                history.append((step, tokens_seen, tr, va))
                print(f"ep {epoch+1} step {step}: train {tr:.3f} val {va:.3f}")
        print_sample(model, tok, start_context, device)   # eyeball progress
    return history
```

We use AdamW with learning rate 4×10^{-4} and weight decay 0.1. AdamW (Loshchilov & Hutter, 2019) is Adam with *decoupled* weight decay: instead of adding $\lambda\theta$ to the gradient, where Adam's per-coordinate scaling would distort it, the parameter is shrunk directly by $\eta\lambda\theta$ after the adaptive update. For LLMs this matters because the adaptive denominators differ by orders of magnitude between embedding rows of rare and frequent tokens.

What happens on a tiny corpus. Training GPT-2-small-shaped models for ten epochs on the nine training batches of "The Verdict" produces the textbook picture of overfitting: the training loss falls from about 10 to well below 1, while the validation loss drops at first and then flattens out around 6 and starts creeping up. Generated samples turn from noise into grammatical English—and then into verbatim passages of the story. Figure 7.5 shows the same phenomenon reproduced on the CPU of the machine that built this book, with a smaller model and a different small corpus.

Key Idea: Overfitting here is a dataset-size artifact

A 124M-parameter model has enormous capacity relative to 5k tokens: it can store the text. Real pretraining runs see each token roughly once (one epoch or less), so the validation and training losses track each other closely and overfitting in the classical sense is rare. When data is scarce, multiple epochs still help up to about four repetitions, after which returns diminish quickly (Muennighoff et al., 2023).

7.3 Decoding strategies to control randomness

LLM 5.3

The trained model outputs a distribution over the next token; *decoding* decides which token to emit. So far we used greedy decoding (`argmax`): deterministic and, for a model that has memorized its training text, a perfect recipe for regurgitation. Sampling from the distribution instead adds diversity, and two simple knobs control how adventurous the samples are. Chapter 15

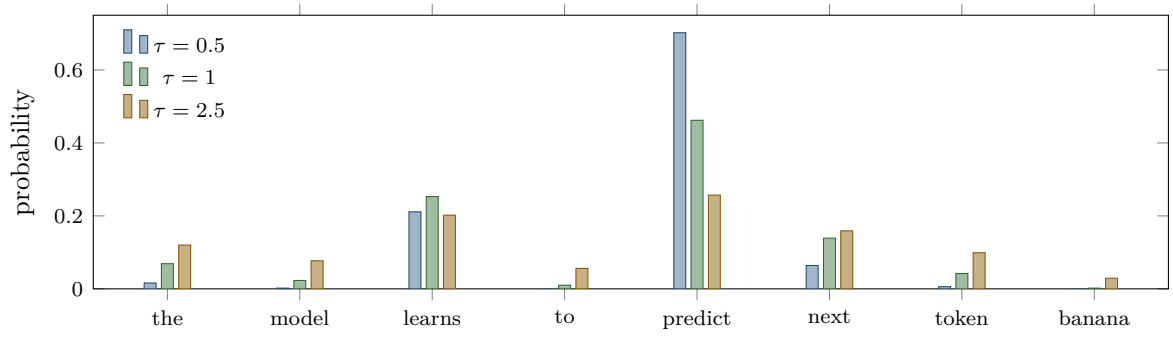
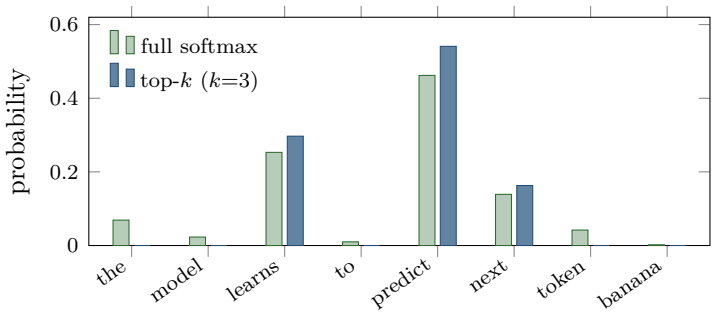


Figure 7.6: Temperature reshapes the next-token distribution without changing the ranking of tokens. Logits used: $(2.1, 1.0, 3.4, 0.2, 4.0, 2.8, 1.6, -1.5)$. At $\tau = 2.5$ even "banana" gets sampled about 3% of the time.



Step 1. Find the k-th largest logit $z_{(k)}$.
Step 2. Set $z_v \leftarrow -\infty$ for every $z_v < z_{(k)}$.
Step 3. Softmax (with temperature) over what is left; the five masked words get exactly zero probability.
Step 4. Sample with `torch.multinomial`.

Figure 7.7: Top-k filtering keeps the k most likely tokens and renormalizes.

returns to these knobs (and adds top-p) when we use sampling deliberately to make reasoning models more accurate.

7.3.1 Temperature scaling

LLM 5.3.1

Replacing `argmax` with `torch.multinomial(softmax(z))` samples tokens in proportion to their probabilities. *Temperature* $\tau > 0$ divides the logits before the softmax,

$$p_\tau(v) = \frac{\exp(z_v/\tau)}{\sum_u \exp(z_u/\tau)}. \tag{7.3}$$

$\tau < 1$ sharpens the distribution toward the argmax (and $\tau \to 0$ recovers greedy decoding); $\tau > 1$ flattens it toward uniform. Figure 7.6 applies three temperatures to an eight-word toy vocabulary.

7.3.2 Top-k sampling

LLM 5.3.2

High temperatures give diversity but also give the long tail of nonsense tokens a real chance. *Top-k* sampling restricts the choice to the k highest-scoring tokens by setting all other logits to $-\infty$ before the softmax, so their probability becomes exactly zero and the remaining mass is renormalized among the survivors (Fig. 7.7). Combining a moderate top-k (say 25–50) with a temperature above one is a classic recipe for varied but mostly coherent text.

7.3.3 A generation function with temperature, top-k and EOS

LLM 5.3.3

Listing 7.4 folds both knobs into the generation loop and adds an optional end-of-sequence ID so generation can stop early. `temperature=0` keeps the old greedy behaviour.

Listing 7.4: Generation with temperature, top-k, and early stopping (Lab 7, Task 4).

```
@torch.no_grad()
def generate_text(model, idx, max_new_tokens, context_size, temperature=0.0,
                  top_k=None, eos_id=None, generator=None):
    model.eval()
    for _ in range(max_new_tokens):
        logits = model(idx[:, -context_size:])[:, -1, :].float()   # last position
        if top_k is not None:
            kth = torch.topk(logits, top_k, dim=-1).values[:, -1:]
            logits = torch.where(logits < kth, float("-inf"), logits)
        if temperature > 0.0:
            probs = torch.softmax(logits / temperature, dim=-1)
            nxt = torch.multinomial(probs, 1, generator=generator)
        else:
            nxt = torch.argmax(logits, dim=-1, keepdim=True)
        if eos_id is not None and bool((nxt == eos_id).all()):
            break
        idx = torch.cat([idx, nxt], dim=1)
    return idx
```

Systems Lens: Sampling is cheap, but not free, and must be reproducible

On a GPU the sampling step runs once per generated token on a (B, V) tensor. Top-k is a partial selection ($O(V)$ with a radix-select kernel) while top-p needs a sort ($O(V \log V)$); for large batches and vocabularies of 150k tokens inference engines fuse temperature, top-k/top-p and the multinomial draw into one kernel and avoid sorting by rejection-style sampling over a pivot. Two other systems properties matter more than speed:

- **Reproducibility:** pass an explicit `torch.Generator` per request (seeded from the request ID) rather than relying on the global RNG, which other requests in the same batch would advance in a data-dependent order.
- **Batch invariance:** with dynamic batching (Chapter 12) the same request may be computed in different batch shapes, which changes matmul reduction orders and therefore low-order bits of the logits. Greedy decoding can then flip at near-ties. Bitwise-reproducible serving requires batch-invariant kernels.

7.4 Saving and loading model weights

LLM 5.4

Training runs end, crash, and get pre-empted, so persisting state is part of the loop, not an afterthought. PyTorch's recommended unit of persistence is the `state_dict`: an ordered dictionary from parameter names to tensors, which decouples the saved data from the Python class that produced it.

Listing 7.5: Saving and restoring *all* training state (Lab 7, Task 5).

```
def save_checkpoint(path, model, optimizer, step):
    torch.save({"model": model.state_dict(),
```

```
                "optimizer": optimizer.state_dict(),    # Adam moments m, v + step
    counts
                "step": step,
                "rng_cpu": torch.get_rng_state(),       # dropout masks, data order
    ...
                "rng_cuda": torch.cuda.get_rng_state_all()
                            if torch.cuda.is_available() else None}, path)

def load_checkpoint(path, model, optimizer=None):
    ck = torch.load(path, map_location="cpu", weights_only=False)
    model.load_state_dict(ck["model"])
    if optimizer is not None:
        optimizer.load_state_dict(ck["optimizer"])
    torch.set_rng_state(ck["rng_cpu"])
    return ck["step"]
```

For inference only the weights are needed (and `model.eval()` afterwards, to turn off dropout). To *continue training*, the optimizer state is just as important: AdamW keeps a first-moment and second-moment estimate per parameter, and restarting them from zero produces a burst of overly large updates (the bias correction treats the run as just started). Lab 7 verifies the strongest form of correctness: training 4 steps, checkpointing, restoring into a freshly constructed model, and training 4 more must give bit-identical weights to training 8 steps straight.

Napkin Math: Checkpoint sizes

GPT-2 small has 124M unique parameters. In fp32 the weights take $124\text{M} \times 4 = 0.50\,\text{GB}$; adding AdamW's two fp32 moments triples that to about 1.5 GB. For a 70B model trained in mixed precision the full training state is about 16 bytes per parameter $\approx 1.1\,\text{TB}$—written every few hundred steps by thousands of ranks.

Systems Lens: Checkpointing at scale

- **Format.** `torch.save` pickles Python objects; loading an untrusted pickle can execute code. The *safetensors* format stores a JSON header plus raw little-endian tensor bytes, so it is safe, can be `mmap`-ed for zero-copy lazy loading, and supports reading single tensors. It is the de facto format for released weights (the GPT-2 and Qwen3 loaders in `llmsys` read it).
- **Sharded checkpoints.** With ZeRO/FSDP (Chapter 8) no rank owns the full state. Each rank writes its own shard plus metadata describing the global layout, so a job can be resumed on a different number of GPUs by resharding at load time (`torch.distributed.checkpoint` does this).
- **Asynchronous checkpointing.** Copy state to pinned host memory (fast, over PCIe), resume training, and let a background thread persist to storage. The stall shrinks from "write 1 TB to a parallel file system" to "copy a few GB per GPU to host".
- **How often?** Young's approximation gives the checkpoint interval that minimizes expected lost work: $T_{\text{opt}} \approx \sqrt{2\,\delta\,M}$, with δ the time to write a checkpoint and M the mean time between failures of the *whole job*. A 60 s checkpoint and a 5-hour job-level MTBF give $T_{\text{opt}} \approx \sqrt{2 \cdot 60 \cdot 18{,}000}\,\text{s} \approx 25$ minutes. Asynchronous checkpointing

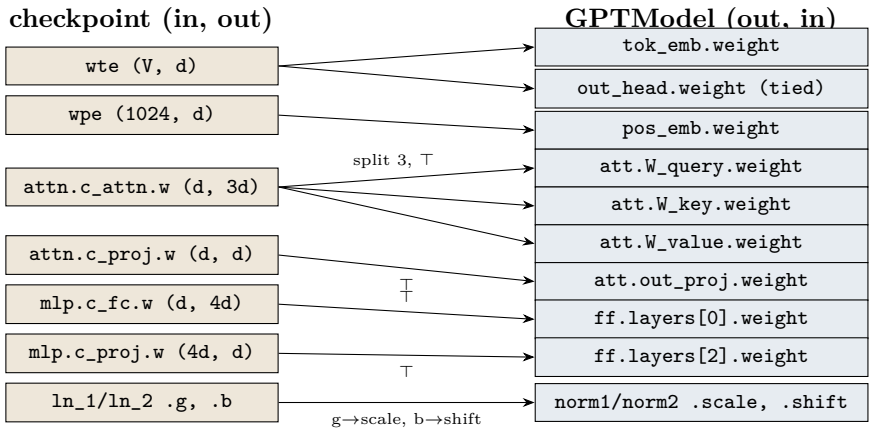


Figure 7.8: Mapping GPT-2 checkpoint tensors (per block, left) onto `GPTModel` parameters (right). ⊤ marks a transpose; biases follow their weights (the QKV bias is split into three, without transposes).

shrinks δ, so it also lets you checkpoint more often.

7.5 Loading pretrained weights from OpenAI

LLM 5.5

Training a GPT-2-class model on a web-scale corpus costs far more than a laptop can deliver, but OpenAI released the weights of all four GPT-2 sizes. Loading them into our own `GPTModel` is a good exercise in how architectures and checkpoint layouts relate, and the resulting model is the starting point of Chapters 9 and 10.

The original release is a TensorFlow checkpoint plus `hparams.json` (which holds `n_vocab=50257`, `n_ctx=1024`, `n_embd`, `n_head`, `n_layer`). After reading it into NumPy, the parameters form a nested dictionary with the token and position embeddings (`wte`, `wpe`), the final LayerNorm (`g`, `b`) and a list of per-block dictionaries. The same weights are also distributed in Hugging Face's safetensors layout (`h.0.attn.c_attn.weight`, ...), which is what `llmsys.gpt.load_gpt2_hf_state_dict` reads. Either way, three details have to be handled; Fig. 7.8 summarizes the mapping.

1. **Transposes.** GPT-2's projections are stored as "Conv1D" matrices of shape `(in, out)`, whereas `nn.Linear` stores `(out, in)`. Every projection is transposed on load.

2. **Fused QKV.** The query, key and value projections are one matrix `c_attn` of shape `(d, 3d)` (and a bias of length $3d$). We split it into three equal column blocks. This also means the released model *has* QKV biases, so the config must set `qkv_bias=True`—unlike the bias-free attention we trained from scratch.

3. **Weight tying.** GPT-2 reuses the token-embedding matrix as the output head. Our `GPTModel` keeps a separate `out_head`, which is why a naive parameter count of it gives 163M rather than 124M; we copy `wte` into both.

Listing 7.6: Core of the OpenAI-layout loader (Lab 7, Task 9). The Hugging Face variant in `llmsys/gpt.py` is identical up to key names.

```
def load_openai_gpt2_params(model, params):
    put = lambda param, arr: param.data.copy_(torch.as_tensor(arr))   # + shape
    check
    put(model.tok_emb.weight, params["wte"])
```

model	params	layers L	d_{model}	heads H	weights (fp32)
gpt2-small	124M	12	768	12	0.50 GB
gpt2-medium	355M	24	1024	16	1.42 GB
gpt2-large	774M	36	1280	20	3.10 GB
gpt2-xl	1558M	48	1600	25	6.23 GB

Table 7.1: The four released GPT-2 sizes share one architecture and differ only in depth, width and head count (all use $V = 50{,}257$, context 1,024, $d_h = 64$).

```
    put(model.pos_emb.weight, params["wpe"])
    for blk, p in zip(model.trf_blocks, params["blocks"]):
        qw, kw, vw = np.split(p["attn"]["c_attn"]["w"], 3, axis=-1)   # (d, d) each
        qb, kb, vb = np.split(p["attn"]["c_attn"]["b"], 3, axis=-1)
        put(blk.att.W_query.weight, qw.T); put(blk.att.W_query.bias, qb)
        put(blk.att.W_key.weight,   kw.T); put(blk.att.W_key.bias,   kb)
        put(blk.att.W_value.weight, vw.T); put(blk.att.W_value.bias, vb)
        put(blk.att.out_proj.weight, p["attn"]["c_proj"]["w"].T)
        put(blk.ff.layers[0].weight, p["mlp"]["c_fc"]["w"].T)
        put(blk.ff.layers[2].weight, p["mlp"]["c_proj"]["w"].T)
        # ... biases and the two LayerNorms (g -> scale, b -> shift)
    put(model.final_norm.scale, params["g"]); put(model.final_norm.shift, params["b"
    ])
    put(model.out_head.weight, params["wte"])                          # weight tying
```

Once loaded, the model writes coherent English from a prompt such as "Every effort moves you", and its loss on "The Verdict" is far below that of anything we can train on the story itself. A strong check that the mapping is right: the pretrained model's validation loss on a held-out paragraph should be around 3–4 nats, not ∼10. A wrong transpose or a swapped Q/K block yields a loss near the uniform baseline.

Pitfall: Silent shape coincidences

Square matrices ($d \times d$) can be loaded with the wrong orientation without any shape error. Always assert shapes, and always validate a loaded model with a loss on real text or a reference output, not just "it runs".

7.6 Bells and whistles for the training loop

LLM App. D

The simple loop of Listing 7.3 works for a toy, but real pretraining is fragile: early updates can blow up the network before the adaptive optimizer has calibrated its statistics, late updates with a large step size keep the model bouncing around a minimum, and a single unlucky batch can produce a gradient large enough to undo hours of progress. Three techniques address these, and every serious LLM run uses all three.

7.6.1 Learning-rate warmup

LLM D.1

At initialization, Adam's second-moment estimate v is built from only a few gradients and is a noisy, typically too-small denominator; at the same time the gradients of a randomly initialized transformer can be large. Starting at full learning rate therefore produces destabilizing updates. *Warmup* ramps the learning rate linearly from a small `initial_lr` to `peak_lr` over the first W

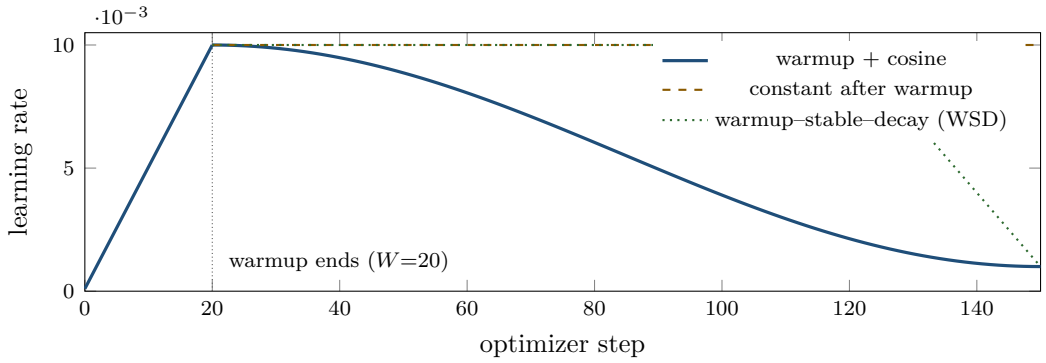


Figure 7.9: Learning-rate schedules for a 150-step run: linear warmup from 10^{-4} to 10^{-2}, then cosine decay to $\eta_{\min} = 10^{-3}$. The dotted curve is the warmup–stable–decay schedule popular in recent pretraining because a run can be extended from any point of its stable phase.

optimizer steps,

$$\eta_t = \eta_{\text{init}} + (\eta_{\text{peak}} - \eta_{\text{init}})\,\frac{t}{W}, \qquad t < W. \tag{7.4}$$

Typical warmups are 0.1%–20% of total steps; in long pretraining runs a few thousand steps is common regardless of total length. Implementation-wise the schedule lives *outside* the optimizer: before each step, write the desired value into every `optimizer.param_groups[i]["lr"]`.

LLM D.2

7.6.2 Cosine decay

After warmup, the learning rate should shrink so that late updates refine rather than overshoot. Cosine decay follows half a cosine period from the peak to a floor $\eta_{\min}$ (often 10% of the peak):

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\text{peak}} - \eta_{\min})\Big(1 + \cos\big(\pi\,\tfrac{t-W}{S-W}\big)\Big), \qquad W \le t \le S, \tag{7.5}$$

with S the total number of steps. Figure 7.9 shows the combined schedule.

Deeper Dive: Why cosine, and what replaced it

Cosine has no deep optimality property; it is a smooth way to spend most of the budget at a high learning rate and anneal at the end. Its practical weakness is that the total number of steps S must be fixed in advance—you cannot cheaply continue a run, because the learning rate has already decayed. *Warmup–stable–decay* (WSD) schedules hold the peak learning rate for most of training and decay linearly or with a $1 - \sqrt{\cdot}$ shape over the last 10–20% of steps; checkpoints from the stable phase can be branched and annealed at any time, which makes scaling-law studies far cheaper (Hu et al., 2024, MiniCPM; Hägele et al., 2024).

LLM D.3

7.6.3 Gradient clipping

Rare batches produce gradients far larger than typical (an unusual document, a numerical outlier in attention logits). *Global-norm clipping* bounds the size of every update by rescaling the whole gradient vector $g = (g_1, \dots, g_P)$ when its ℓ_2 norm exceeds a threshold c (commonly $c = 1$):

$$\|g\|_2 = \sqrt{\textstyle\sum_i g_i^2}, \qquad g \leftarrow g \cdot \min\Big(1, \frac{c}{\|g\|_2}\Big). \tag{7.6}$$

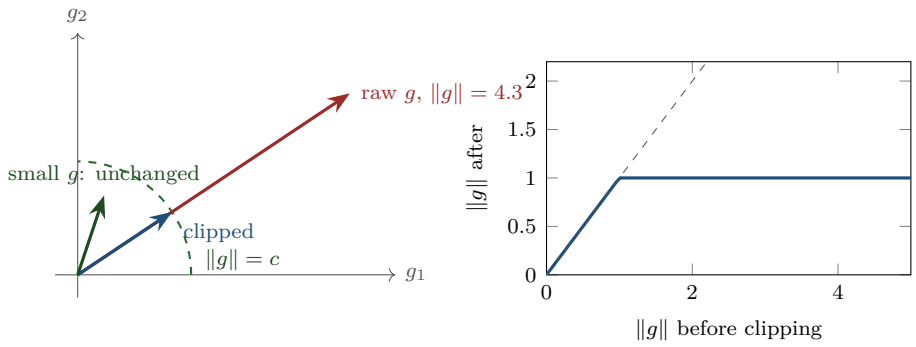


Figure 7.10: Global-norm clipping with $c = 1$: gradients inside the ball are untouched; larger ones are scaled back onto it, keeping their direction.

Because every coordinate is multiplied by the same factor, the *direction* of the update is preserved and only its length changes (Fig. 7.10). PyTorch provides `torch.nn.utils.clip_grad_norm_`; Lab 7 implements it by hand. Call it after `backward()` and before `optimizer.step()`—and, when using fp16 loss scaling, after unscaling the gradients.

Pitfall: Clipping hides problems as well as fixing them

Log the pre-clip norm every step (Lab 7's loop records it). A healthy run clips occasionally; a run that clips on every step is effectively running at a lower, norm-dependent learning rate, and a steadily rising gradient norm is often the first sign of an impending loss spike.

7.6.4 The modified training function

LLM D.4

Listing 7.7 combines everything: warmup + cosine, gradient clipping, gradient accumulation, optional bf16 autocast (Section 7.8), periodic evaluation and checkpoints. Note the ordering inside a step: compute the scheduled learning rate, clip, step, zero the gradients.

Listing 7.7: The full training function from Lab 7 (abridged).

```
def train_model(model, train_loader, val_loader, optimizer, cfg, start_step=0):
    total = cfg.max_steps or cfg.epochs * (len(train_loader) // cfg.grad_accum)
    warmup = int(cfg.warmup_frac * total)
    step, micro = start_step, 0
    for _ in range(cfg.epochs):
        for x, y in train_loader:
            with torch.autocast("cuda", dtype=cfg.amp_dtype,
                                enabled=cfg.amp_dtype is not None):
                logits = model(x.to(cfg.device))
            loss = F.cross_entropy(logits.float().flatten(0, 1),
                                   y.to(cfg.device).flatten()) / cfg.grad_accum
            loss.backward()                          # grads accumulate in .grad
            micro += 1
            if micro % cfg.grad_accum:
                continue                             # not yet a full (virtual) batch
            lr = warmup_cosine_lr(step, total, cfg.peak_lr, warmup,
                                  cfg.initial_lr, cfg.min_lr)
            for g in optimizer.param_groups:
                g["lr"] = lr
            grad_norm = clip_grad_norm_(model.parameters(), cfg.grad_clip)
            optimizer.step()
```

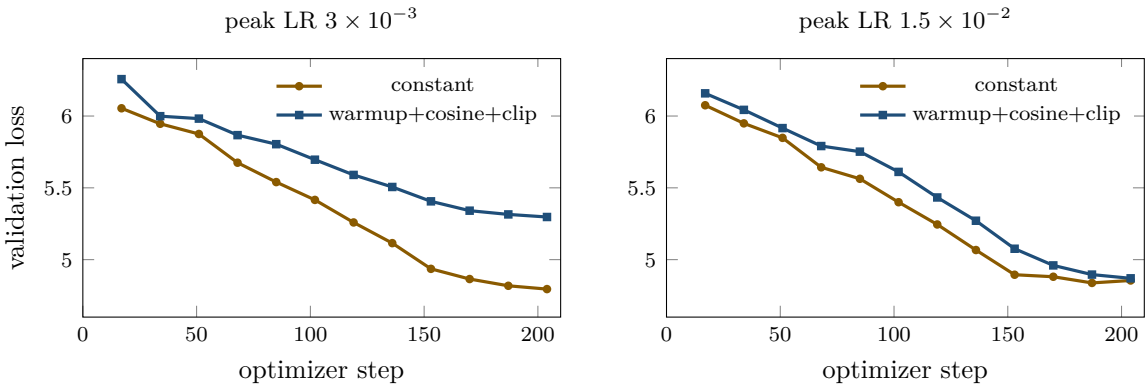


Figure 7.11: Constant learning rate versus warmup (25 steps) + cosine decay (to 10% of peak) + clipping at 1.0, measured on CPU in this book's build environment (3-layer GPT, 204 optimizer steps on 135k characters). At this tiny scale the plain schedule is as good or better: with a budget of 200 steps, cosine decay lowers the learning rate before the model has finished learning, and nothing is large enough to blow up. The value of warmup and clipping appears with large models, high learning rates and long runs, where a single divergence costs days.

```
            optimizer.zero_grad(set_to_none=True)
            step += 1
            # ... periodic evaluation, logging of lr/grad_norm, checkpointing
```

Figure 7.11 compares a constant learning rate with warmup + cosine + clipping on the same model and data (measured on this book's build machine). The honest result is that on a small, short run the bells and whistles do not help—and cosine decay can even hurt, because it anneals before the model has converged. Their value grows with model size, learning rate and run length, which is precisely where an unstable run is most expensive. Do not judge these techniques by toy runs; judge them by the failures they prevent at scale.

7.7 Where the memory goes

We now switch to the systems view. A training step must keep four kinds of state in device memory, and only one of them depends on the batch.

Model states. With N parameters and AdamW in standard mixed precision (bf16 compute, fp32 master copy), the per-parameter bill is

bf16 weights used in forward/backward	2 bytes
bf16 gradients	2 bytes
fp32 master weights	4 bytes
fp32 Adam first moment m	4 bytes
fp32 Adam second moment v	4 bytes
total	**16 bytes**

Pure fp32 training without a master copy also costs $4 + 4 + 4 + 4 = 16$ bytes per parameter. A 7B model therefore needs ≈ 112 GB of model states before a single activation is stored—more than one 80 GB GPU. This number motivates ZeRO and FSDP in Chapter 8, which shard these 16 bytes across data-parallel ranks.

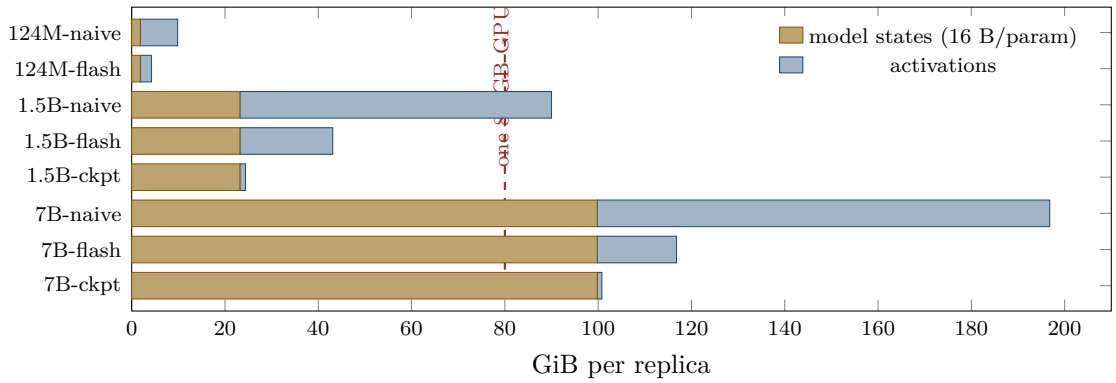


Figure 7.12: Memory per data-parallel replica from Eq. (7.7) and the 16-byte rule. 124M and 1.5B: $s = 1024$, $b = 8$; 7B (L=32, h=4096, a=32): $s = 4096$, $b = 1$. "naive" materializes attention scores, "flash" does not, "ckpt" recomputes each block and stores only block inputs. Even with checkpointing, the 7B model's states alone exceed one 80 GB GPU.

Activations. Backpropagation needs intermediate values from the forward pass. Korthikanti et al. (2022) count, for a GPT block with sequence length s, micro-batch b, hidden size h and a heads, about

$$M_{\text{act}}^{\text{layer}} \approx s\,b\,h\left(34 + \frac{5\,a\,s}{h}\right) \text{ bytes (16-bit activations).} \tag{7.7}$$

The 34 collects the inputs of the QKV, output and MLP projections, the LayerNorm inputs, the GELU input, the dropout masks, and so on; the $5as/h$ term is the $a \times s \times s$ attention score and probability matrices (plus a dropout mask). FlashAttention (Chapter 4) never stores those matrices, removing the quadratic term entirely.

Temporary buffers and fragmentation. Workspace for matmuls and communication buffers, plus fragmentation in the caching allocator (Chapter 2), typically add another 5–15%.

Napkin Math: GPT-2 small vs. a 7B model

GPT-2 small ($N = 124$M, $L = 12$, $h = 768$, $a = 12$, $s = 1024$, $b = 8$): model states $16 \times 124\text{M} = 1.85$ GiB; activations without FlashAttention $12 \cdot 1024 \cdot 8 \cdot 768\,(34 + 5 \cdot 12 \cdot 1024/768) \approx 8.0$ GiB, with FlashAttention 2.4 GiB. At this scale *activations* dominate.
A 7B model ($L = 32$, $h = 4096$, $a = 32$) at $s = 4096$, $b = 1$: states 99.8 GiB, activations 97 GiB (naive) or 17 GiB (Flash). At this scale *model states* dominate and no single GPU suffices even at micro-batch 1.

7.8 Mixed precision

Tensor cores deliver their advertised throughput only for 16-bit (or 8-bit) inputs: an H100 SXM does about 989 dense bf16 TFLOP/s versus roughly 67 TFLOP/s for plain fp32 on the CUDA cores. 16-bit activations also halve activation memory and memory traffic. *Mixed precision* runs the matmuls in 16 bits while keeping numerically sensitive pieces in fp32.

The recipe (Micikevicius et al., 2018) has three ingredients:

1. **Autocast.** `torch.autocast(device, dtype=torch.bfloat16)` runs matmuls and convolutions in bf16 but keeps reductions such as softmax, LayerNorm statistics and the loss in fp32. Parameters stay fp32; autocast casts them on the fly (and caches the casts within

format	sign/exp/mantissa	max	smallest normal	typical use
fp32	1/8/23	3.4×10^{38}	1.2×10^{-38}	master weights, Adam states
bf16	1/8/7	3.4×10^{38}	1.2×10^{-38}	activations, weights, grads
fp16	1/5/10	65,504	6.1×10^{-5}	older GPUs; needs loss scaling
fp8 E4M3	1/4/3	448	1.6×10^{-2}	forward GEMM inputs (H100+)
fp8 E5M2	1/5/2	57,344	6.1×10^{-5}	gradients (H100+)

Table 7.2: Floating-point formats in LLM training. bf16 trades mantissa bits for fp32's exponent range, which is why it rarely needs loss scaling.

the forward pass).

2. **fp32 master weights.** The update $\theta \leftarrow \theta - \eta\,\Delta$ is applied in fp32. In bf16, with 8 bits of precision, an update smaller than $2^{-8} \approx 0.4\%$ of a weight rounds away entirely; late in training most updates are that small.

3. **Loss scaling (fp16 only).** fp16's smallest normal number is 6×10^{-5}, and many gradient values are smaller. Multiplying the loss by a scale S (e.g. 2^{16}) shifts gradients into range; they are divided by S before clipping and the optimizer step. `torch.amp.GradScaler` adjusts S dynamically, halving it when an overflow (inf/NaN) is detected and skipping that step. bf16 has fp32's exponent range and normally needs no scaling, which is why it became the default on A100-class and newer hardware.

Systems Lens: FP8 training

Hopper and later GPUs double tensor-core throughput again with 8-bit floats. FP8 training (e.g. NVIDIA Transformer Engine, DeepSeek-V3's recipe) keeps master weights and optimizer states in higher precision, casts GEMM inputs to FP8 with *per-tensor or per-block scaling factors* chosen from recent maxima ("delayed scaling" or fine-grained tile-wise scales), and accumulates in higher precision. The engineering problem is outliers: a single huge activation forces a large scale and flushes everything else in that tensor to zero, which is why fine-grained (e.g. 1×128 or 128×128 block) scaling is used for the most sensitive tensors.

7.9 Trading compute for memory, and measuring efficiency

Gradient accumulation. If the batch that optimization wants (say 10^6 tokens) does not fit, split it into k micro-batches, call `backward()` on each loss divided by k, and step once. Gradients add in `.grad`, so the result equals the large-batch gradient exactly (when the loss is a mean over equal-sized micro-batches and there is no batch-dependent layer such as BatchNorm). Lab 7 tests this equivalence. Accumulation costs no extra memory beyond one micro-batch of activations; its cost is that k forward/backward passes run sequentially.

Activation checkpointing. Rather than storing every intermediate, store only the input of each transformer block and recompute the block's internals during the backward pass. With L layers and per-layer activation size A, memory falls from LA to roughly $L \cdot a_{\text{in}} + A$ (block inputs plus one block's worth of live intermediates) at the cost of one extra forward pass—about 33% more FLOPs, since a backward pass costs about twice a forward. More generally, checkpointing every k-th layer stores L/k checkpoints and recomputes segments of k layers,

so memory is $O(L/k + k)$, minimized at $k = \sqrt{L}$ for $O(\sqrt{L})$ memory (Chen et al., 2016). *Selective* recomputation (Korthikanti et al., 2022) recomputes only the cheap-to-recompute but memory-hungry parts (attention softmax and dropout), recovering most of the memory savings for a few percent of extra compute.

Listing 7.8: Activation checkpointing by wrapping each block (Lab 7, Task 6).

```python
from torch.utils.checkpoint import checkpoint

def enable_activation_checkpointing(model):
    for blk in model.trf_blocks:
        inner = blk.forward
        def wrapped(x, cache=None, pad_mask=None, _inner=inner, _blk=blk):
            if _blk.training and cache is None and torch.is_grad_enabled():
                # store only x; recompute attention + MLP internals in backward
                return checkpoint(_inner, x, None, pad_mask, use_reentrant=False)
            return _inner(x, cache=cache, pad_mask=pad_mask)
        blk.forward = wrapped
```

Throughput and MFU. The forward pass of a dense transformer costs about $2N$ FLOPs per token (one multiply and one add per parameter), and the backward pass twice that, so training costs about

$$C \approx 6\,N\,D \quad \text{FLOPs for } D \text{ training tokens} \tag{7.8}$$

(plus an attention term $\approx 12\,L\,h\,s$ per token, which matters only for long contexts). *Model FLOPs utilization* compares achieved model FLOPs with the hardware peak:

$$\text{MFU} = \frac{6\,N \cdot \text{tokens/s}}{\text{peak FLOP/s}}. \tag{7.9}$$

Recomputation from activation checkpointing does *not* count as model FLOPs (that would be *hardware* FLOPs utilization, HFU), so MFU honestly penalizes it. Well-tuned large dense runs report MFUs of roughly 35–55%; small models on single GPUs are often far lower because kernels are short and launch overheads and memory-bound elementwise ops dominate.

Napkin Math: What does 30% MFU mean for GPT-2 small?

Suppose GPT-2 small trains at a hypothetical 400,000 tokens/s on one H100. Model FLOP/s $= 6 \times 124\text{M} \times 4 \times 10^5 \approx 2.98 \times 10^{14}$, i.e. 30% of the 989 TFLOP/s bf16 peak. A compute-optimal run for this model (≈ 20 tokens per parameter, see below) is 2.5×10^9 tokens $\Rightarrow 6 \cdot 124\text{M} \cdot 2.5\text{B} \approx 1.8 \times 10^{18}$ FLOPs, about 1.3 H100-hours at 40% MFU. GPT-2 small is cheap; data loading and evaluation overheads, not FLOPs, are what make naive reproductions slow.

Systems Lens: The usual suspects when MFU is low

1. **Input pipeline stalls.** Tokenize offline into binary shards and memory-map them (`TokenMemmapDataset` in `llmsys.data`); use pinned memory and `non_blocking=True` copies; keep `num_workers`> 0 if any CPU work remains.
2. **Small, unfused kernels.** LayerNorm, GELU, residual adds, dropout and the optimizer are memory-bound. `torch.compile` fuses chains of them; `AdamW(fused=True)`

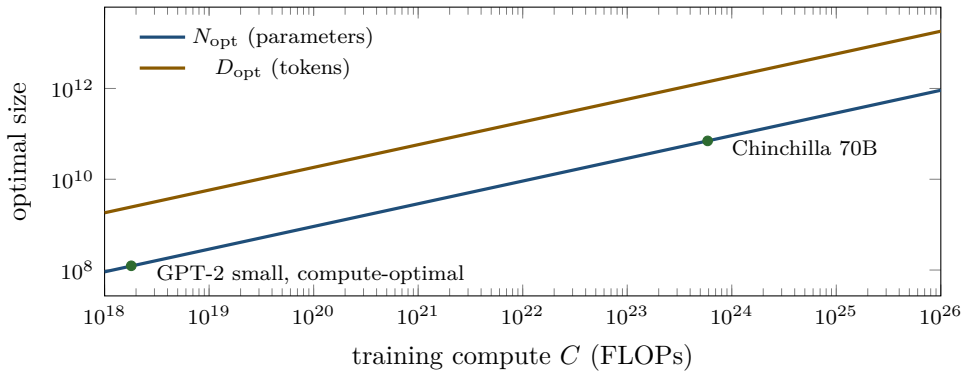


Figure 7.13: Chinchilla-style planning curves from $C = 6ND$ and $D = 20N$. Both grow as $\sqrt{C}$.

updates all parameters in a handful of kernels instead of several per tensor.

3. **Shapes that miss the tensor cores.** Keep d_{model}, d_{ff}, head dimension and vocabulary size multiples of 64 or 128. Padding GPT-2's vocabulary from 50,257 to 50,304 is a well-known free speed-up.
4. **Host-device synchronization.** `loss.item()` every step, Python-side `if` on tensor values, or printing forces the CPU to wait for the GPU and drains its queue of launched kernels. Log asynchronously or every k steps.
5. **Attention.** Use `F.scaled_dot_product_attention` (FlashAttention backend); the core models accept `use_sdpa=True`.

7.10 Scaling laws and compute budgeting

How should a fixed compute budget C be split between model size N and data D? Kaplan et al. (2020) fit the loss as power laws in N, D and C and concluded that size should grow faster than data. Hoffmann et al. (2022, "Chinchilla") revisited this with learning-rate schedules matched to each run's length and found that N and D should grow *equally*: $N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$, with roughly 20 training tokens per parameter. They fit

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}, \tag{7.10}$$

an irreducible entropy E plus terms for finite model capacity and finite data. Combining $D = 20N$ with Eq. (7.8) gives the planning rule

$$N_{\text{opt}} \approx \sqrt{\frac{C}{120}}, \qquad D_{\text{opt}} \approx 20\, N_{\text{opt}}. \tag{7.11}$$

For $C = 5.9 \times 10^{23}$ FLOPs this gives $N \approx 70$B and $D \approx 1.4$T tokens, which is Chinchilla itself.

Key Idea: Compute-optimal is not deployment-optimal

Chinchilla minimizes *training* compute for a target loss. A model that will serve billions of tokens should be smaller than compute-optimal and trained on far more tokens, because inference cost scales with N. This is why recent open models are "over-trained": Llama 3 8B saw about 15T tokens, nearly 2,000 tokens per parameter. Systems teams care about

this trade-off directly, since the serving fleet (Chapters 11 and 12) usually costs more over a model's lifetime than its training run.

Napkin Math: Planning a 7B run

How long does a 7B model take on 2T tokens with 512 H100s at 40% MFU? Per GPU, $0.4 \cdot 989 \times 10^{12} / (6 \cdot 7 \times 10^{9}) \approx 9{,}400$ tokens/s. Cluster: 4.8M tokens/s. $2 \times 10^{12} / 4.8 \times 10^{6} \approx 4.2 \times 10^{5}$ s ≈ 4.8 days, if nothing fails. With a cluster-wide MTBF of a few hours, checkpointing policy (Section 7.4) and fast restarts are what decide whether it takes 5 days or 8.

7.11 Training stability at scale

Large runs fail in ways small runs do not. The most common is the *loss spike*: a sudden jump of the loss, sometimes recovering, sometimes diverging. Practical defenses, roughly in order of how often they are used:

- warmup, clipping and a not-too-aggressive peak learning rate (Section 7.6);
- lower Adam β_2 (0.95 instead of 0.999) so the second-moment estimate adapts quickly when gradient statistics shift;
- pre-LayerNorm blocks (Chapter 5), QK-normalization of queries and keys before the dot product (used by Qwen3, Chapter 6), and a small *z-loss* $10^{-4} \cdot (\log Z)^2$ on the softmax normalizer to keep output logits from drifting (PaLM);
- restarting from the last good checkpoint and skipping the data batches around the spike;
- removing biases and using careful initialization (scaled residual initialization).

Systems Lens: Determinism and debuggability

Reproducing a spike requires bitwise-identical replay: the same data order (save the sampler state and the data-loader position, not just the epoch), the same RNG states (Lab 7 stores them), and deterministic kernels (`torch.use_deterministic_algorithms(True)`, which may cost speed). At cluster scale, add per-step logging of the loss, the gradient norm per layer group, the update-to-weight ratio $\|\Delta\theta\|/\|\theta\|$ and the largest attention logit. These cheap scalars are the "flight recorder" that turns a mysterious divergence into a diagnosable one.

7.12 Summary

- Pretraining minimizes the average negative log-likelihood of the next token (cross-entropy); its exponential, perplexity, starts near the vocabulary size and is only comparable between models with the same tokenizer.
- The training loop is the standard PyTorch loop; on a tiny corpus a GPT-2-sized model quickly memorizes the training text while the validation loss stalls.
- Temperature and top-k shape the sampling distribution; `temperature=0` is greedy.

- Persist model *and* optimizer *and* RNG state; use safetensors for released weights, sharded and asynchronous checkpoints at scale, and Young's rule for the interval.
- Loading GPT-2 requires transposing Conv1D weights, splitting the fused QKV projection, enabling QKV biases, and tying the output head to the token embedding.
- Warmup, cosine (or WSD) decay and global-norm clipping make optimization robust.
- Mixed-precision AdamW costs 16 bytes per parameter of model state; activations scale with $s\,b\,h\,L$ and, without FlashAttention, with s^2. Accumulation and checkpointing trade time for memory.
- Training costs $\approx 6ND$ FLOPs; MFU measures how well the hardware is used; Chinchilla says $D \approx 20N$ for compute-optimal training, while deployment favors smaller, longer-trained models.

7.13 Exercises

Exercise 7.1 (*Initial loss*). For a vocabulary of V tokens and logits drawn i.i.d. from $\mathcal{N}(0, \sigma^2)$, show that the expected initial loss is approximately $\ln V + \sigma^2/2$ for small σ. What does this predict for GPT-2 small if the output head is initialized with standard deviation 0.02 and the final hidden states have unit variance?

Exercise 7.2 (*Perplexity across tokenizers*). Model A (GPT-2 tokenizer) reaches loss 3.10 on a corpus that it tokenizes into 1.00M tokens; model B (a byte-level tokenizer) reaches loss 0.95 on the same 4.3MB of text. Convert both to bits per byte and decide which model is better.

Exercise 7.3 (*Training vs. validation*). Run Lab 7's `run.py train` on a short text for 10 epochs with dropout 0.0 and with 0.1. Plot both loss curves. At which epoch does the validation loss reach its minimum in each case, and how do generated samples differ between those two checkpoints?

Exercise 7.4 (*Temperature limits*). Prove that as $\tau \to 0^+$ the temperature-scaled softmax converges to a one-hot vector on the argmax (assume a unique maximum), and that as $\tau \to \infty$ it converges to uniform. What happens with ties at $\tau \to 0$?

Exercise 7.5 (*Top-k and temperature commute?*). Does applying temperature before or after the top-k mask change the sampled distribution? Does the same hold for top-p (Chapter 15)? Justify.

Exercise 7.6 (*Resume correctness*). List every piece of state required for a bit-exact resume of a single-GPU run that uses dropout, a shuffling `DataLoader` with two worker processes, bf16 autocast and a `GradScaler`. Which of them does Listing 7.5 miss?

Exercise 7.7 (*Checkpoint interval*). A job on 2,048 GPUs has a per-GPU MTBF of 50,000 hours (assume independent failures). A synchronous checkpoint takes 90 s; an asynchronous

one stalls training for 6 s. Compute Young's optimal interval for both and the expected fraction of time lost to checkpointing plus rework in each case.

Exercise 7.8 (*Gradient accumulation is not always exact*). Give two concrete situations in which k micro-batches with accumulation do *not* reproduce the gradient of one k-times larger batch. (Hint: think about how the loss is normalized when sequences contain padding, and about layers with batch statistics.)

Exercise 7.9 (*Activation memory*). Using Eq. (7.7), find the largest micro-batch size that fits GPT-2 xl ($L = 48$, $h = 1600$, $a = 25$, $s = 1024$) on an 80 GB GPU in mixed precision, (a) without FlashAttention, (b) with FlashAttention, (c) with full activation checkpointing. Leave 10% headroom.

Exercise 7.10 (*Measure MFU*). With `run.py bench` on any GPU, measure tokens/s for GPT-2 small at sequence length 1024 under: fp32; bf16 autocast; bf16 + SDPA; bf16 + SDPA + `torch.compile`; and the last one with activation checkpointing. Report MFU for each and explain the ranking.

Exercise 7.11 (*Chinchilla planning*). You have 1,000 H100-days at 40% MFU. What model size and token count are compute-optimal? If the model must serve 10^{12} tokens per month for a year at $2N$ FLOPs per token, would you rather train a model half that size on four times the data? Estimate total (training + inference) FLOPs for both options.

Exercise 7.12 (*Vocabulary-chunked loss*). Implement a cross-entropy that processes the sequence dimension in chunks of 128 positions, computing logits, loss and the gradient of the hidden states for each chunk inside a custom `autograd.Function`, so that the full (B, T, V) logits never exist. Verify gradients against `F.cross_entropy` and measure peak memory on a GPU.

7.14 Lab 7: An instrumented pretraining loop

Lab 7: An instrumented pretraining loop

Code: `code/labs/lab07_pretrain`

Goals

Build, from the loss up, a pretraining loop you would trust with real compute: correct objective, robust schedule, exact resume, memory-saving switches, and the napkin-math instruments to judge its efficiency.

Setup

`cd code && pip install -r requirements.txt`. All tests run on CPU in under a minute with a tiny GPT and a synthetic, noisy periodic token stream: `LAB_IMPL=starter python -m pytest -q labs/lab07_pretrain`.

Tasks

1. `cross_entropy_from_scratch` and `perplexity`: log-sum-exp formulation; must match `F.cross_entropy` and stay finite for logits of ±1000.
2. `warmup_cosine_lr`: linear warmup from `initial_lr` to `peak_lr`, then cosine decay to

`min_lr` (Eq. (7.5)).

3. `global_grad_norm` and `clip_grad_norm_`: must agree with `torch.nn.utils.clip_grad_norm_`.
4. `generate_text`: greedy, temperature, top-k, EOS, seeded generator.
5. `save_checkpoint` / `load_checkpoint`: 4+4 steps with a restore in between must equal 8 straight steps bit-for-bit.
6. `enable_activation_checkpointing`: identical gradients to the plain model.
7. `train_model`: warmup+cosine, clipping, accumulation, bf16 autocast, periodic evaluation, logging of LR and pre-clip gradient norm, checkpoints.
8. `training_memory_estimate`, `chinchilla_optimal`, `mfu`.
9. `load_openai_gpt2_params`: OpenAI's original nested layout (transposes, QKV split, tying).

Measurements

- `python run.py train -text the-verdict.txt -epochs 10`: plot train/val loss; you should reproduce the overfitting picture of Section 7.2 (val loss bottoms out around epoch 2–4 while train loss keeps falling).
- Repeat with a 10× larger text (any public-domain novel) and compare the gap.
- On a GPU: `python run.py bench -preset 124M -seq 1024` with and without `-amp bf16`, `-sdpa`, `-compile`, `-ckpt-act`. Record ms/step, peak memory and MFU; compare peak memory with `training_memory_estimate`. Expect bf16+SDPA to give the largest jump, compile a further gain, and checkpointing to cut activation memory by an order of magnitude at roughly 25–35% lower throughput.
- `python run.py sample -hf-dir ./gpt2`: sample from the real GPT-2 at temperatures 0, 0.7 and 1.5 with top-k 25.

Deliverables

Passing tests; a one-page report with the loss curves, the benchmark table (with MFU) and a comparison of measured vs. estimated memory.

Stretch goals

Add a `GradScaler` path for fp16 and show a run that diverges without it; implement the chunked cross-entropy of Exercise 7.12; add a WSD schedule and branch two annealing runs from the same stable-phase checkpoint.

Chapter 8

Distributed Training Systems

Chapter 7 ended with two uncomfortable numbers. A 7B-parameter model needs about 112 GB just for its weights, gradients and Adam moments—more than any single GPU holds—and a frontier pretraining run needs 10^{24}–10^{26} FLOPs, which at a realistic 4×10^{14} FLOP/s per GPU is thousands of GPU-years. Both numbers force the same conclusion: training must be spread across many devices, and the way it is spread decides whether the cluster computes or waits.

Neither source book trains at this scale, so this chapter is new material. It is written for someone who already knows what an all-reduce is and wants the *cost model*: how each parallelism strategy partitions memory, how many bytes it moves per step and over which links, how those transfers overlap with compute, and how the strategies compose into the 3D/4D layouts that real LLM training uses. Every algorithm is implemented from scratch in `llmsys.dist` and tested in Lab 8 on CPU processes with the gloo backend, which exposes the same collective API as NCCL.

8.1 The problem in numbers

Consider training a 70B dense model on 15T tokens, roughly the Llama 3 70B recipe. Training compute is $6ND \approx 6.3 \times 10^{24}$ FLOPs (Eq. (7.8)). On H100s at 40% MFU (≈ 396 TFLOP/s each) that is 1.6×10^{10} GPU-seconds, about 500 GPU-years: a 1-year run on 500 GPUs or a 2-month run on 3,000. Memory is just as binding: model states alone are $16 \times 70\text{B} = 1.12$ TB, fourteen 80 GB GPUs before counting activations.

There are only a few things one can cut into pieces (Fig. 8.2):

- the **batch**: every device holds the full model and processes different examples (data parallelism, DP); ZeRO/FSDP additionally shard the model states over the DP group;
- the **weight matrices within a layer**: each device holds a slice of every matrix and computes a slice of every activation (tensor parallelism, TP);
- the **stack of layers**: each device holds a contiguous group of layers and activations flow between devices (pipeline parallelism, PP);
- the **sequence**: each device holds part of every sequence; attention needs the rest (sequence/context parallelism, SP/CP);
- the **experts** of a mixture-of-experts layer (expert parallelism, EP).

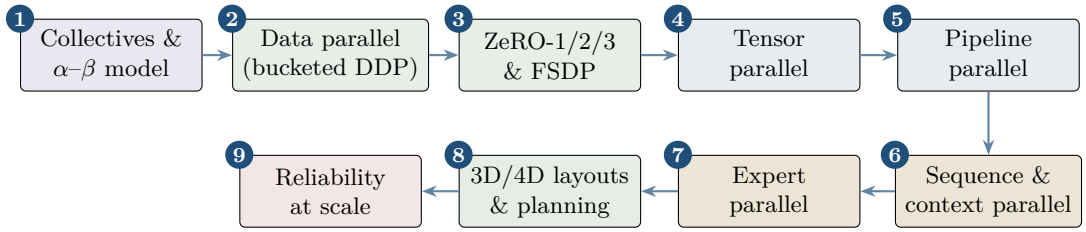


Figure 8.1: Roadmap. We first build the communication vocabulary (1), then partition the *batch* (2–3), the *weights inside a layer* (4), the *layers* (5), the *sequence* (6) and the *experts* (7), and finally compose them (8) and keep the resulting job alive (9).

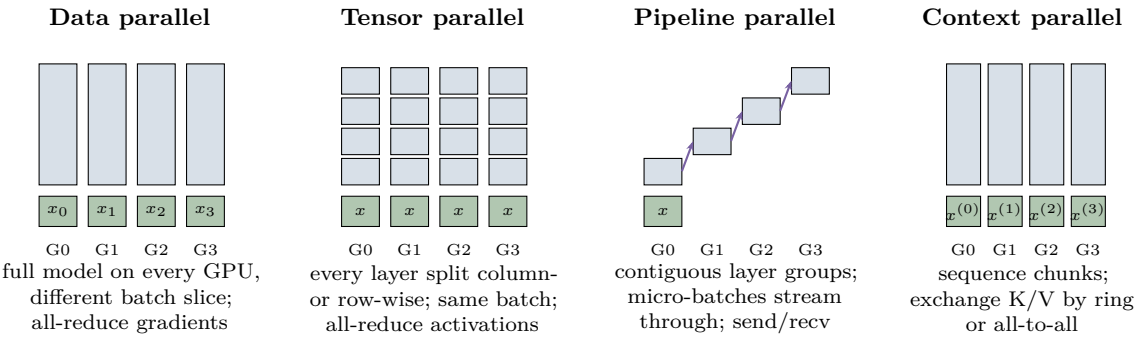


Figure 8.2: What each parallelism cuts. Blue blocks are layers (a tall block is the full stack), green boxes are the inputs each device sees. Expert parallelism (not shown) places different experts of an MoE layer on different devices and routes tokens to them with all-to-all.

They differ in what they save, what they cost in communication, and which network links that communication can tolerate. To compare them we need a communication cost model.

8.2 Communication fundamentals

8.2.1 Collectives

All distributed training is built from a handful of collective operations over a group of p ranks (Fig. 8.3). Each rank starts with a buffer of n bytes.

Broadcast one rank's buffer is copied to all others.

Reduce the element-wise sum of all buffers ends up on one rank.

All-reduce every rank ends up with the element-wise sum. (Gradient averaging.)

Reduce-scatter the sum is computed, but rank i only receives the i-th $1/p$ slice of it.

All-gather rank i contributes slice i; every rank ends up with the concatenation.

All-to-all rank i sends a distinct slice to every rank j (a transpose of the data across ranks). Used by MoE dispatch and Ulysses sequence parallelism.

The identity *all-reduce* = *reduce-scatter* + *all-gather* underlies both the ring algorithm and ZeRO.

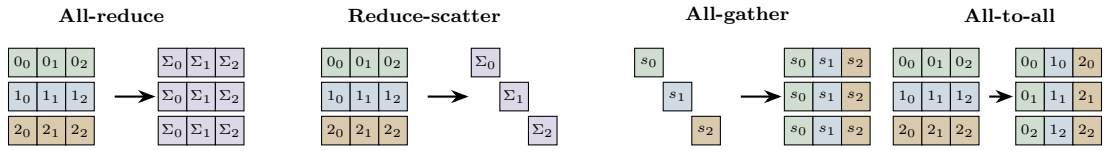


Figure 8.3: Collective semantics for three ranks (rows). r_k is chunk k of rank r's buffer, $\Sigma_k = \sum_r r_k$. Reduce-scatter leaves rank r with only Σ_r; all-to-all transposes chunk ownership.

8.2.2 The α–β cost model

A point-to-point message of n bytes costs

$$t(n) = \alpha + n\,\beta, \tag{8.1}$$

where α is a fixed per-message latency (software, NIC, switch; a few to tens of microseconds) and $\beta = 1/B$ is the inverse bandwidth of the link. Small messages are latency-bound, large ones bandwidth-bound; the crossover is at $n^\star = \alpha/\beta$, e.g. $10\,\mu\text{s} \times 50\,\text{GB/s} = 500\,\text{KB}$ on an InfiniBand NDR link. Gradient buckets of tens of MB are therefore firmly bandwidth-bound, which is why DDP bucketing (Section 8.3) is about *overlap*, not latency.

Proposition 8.1 (Bandwidth lower bound for all-reduce)**.** In an all-reduce over p ranks, each rank must send at least $\frac{p-1}{p}n$ bytes and receive at least $\frac{p-1}{p}n$ bytes for the reduce-scatter half, and the same again for the all-gather half—$2\frac{p-1}{p}n$ in total per rank.

Sketch. Each output element depends on p inputs held on p different ranks; for rank i to own its reduced $1/p$ slice, the other $p-1$ contributions to that slice ($\frac{p-1}{p}n$ bytes in total, after any partial pre-reduction) must reach it. Symmetric arguments apply to distributing the result.

8.2.3 Ring all-reduce

The ring algorithm meets this bound. Arrange the ranks in a ring and split the buffer into p chunks. In the *reduce-scatter* phase, at step $s = 0, \ldots, p-2$ rank r sends chunk $(r-s) \bmod p$ to its right neighbor and receives chunk $(r-s-1) \bmod p$ from its left neighbor, adding it to its own copy. After $p-1$ steps, rank r holds the fully reduced chunk $(r+1) \bmod p$. The *all-gather* phase circulates the reduced chunks in the same pattern without adding (Fig. 8.4). Each of the $2(p-1)$ steps moves n/p bytes on every link simultaneously:

$$t_{\text{ring}} = 2(p-1)\,\alpha + 2\,\frac{p-1}{p}\,n\,\beta. \tag{8.2}$$

The bandwidth term is independent of p for large p—the property that made ring all-reduce (Patarasuk & Yuan, 2009; popularized for deep learning by Baidu and Horovod) the default for data parallelism. Its weakness is the latency term, linear in p; for small messages or very large rings, tree or recursive halving/doubling algorithms with $O(\log p)$ latency win. NCCL chooses between ring, tree and (on NVSwitch systems) in-switch reduction (NVLS) based on message size and topology.

Listing 8.1: Ring all-reduce from point-to-point messages (`llmsys/dist/collectives.py`, Lab 8 Task 1).

```
def _sendrecv(send_buf, recv_buf, dst, src):
    reqs = [dist.isend(send_buf.contiguous(), dst), dist.irecv(recv_buf, src)]
```

	start				after step 1				after step 2				after step 3 (done)			
r0	0	0	0	0	0	0	0	30	0	0	230	30	0	Σ	230	30
r1	1	1	1	1	01	1	1	1	01	1	1	301	01	1	Σ	301
r2	2	2	2	2	2	12	2	2	012	12	2	2	012	12	2	Σ
r3	3	3	3	3	3	3	23	3	3	123	23	3	Σ	123	23	3

Figure 8.4: Ring reduce-scatter on four ranks (rows) and four chunks (columns). Entries list which ranks' contributions a chunk already contains; Σ means fully reduced. After $p-1=3$ steps rank r owns the reduced chunk $(r+1) \bmod 4$. Three more steps of all-gather copy each Σ around the ring.

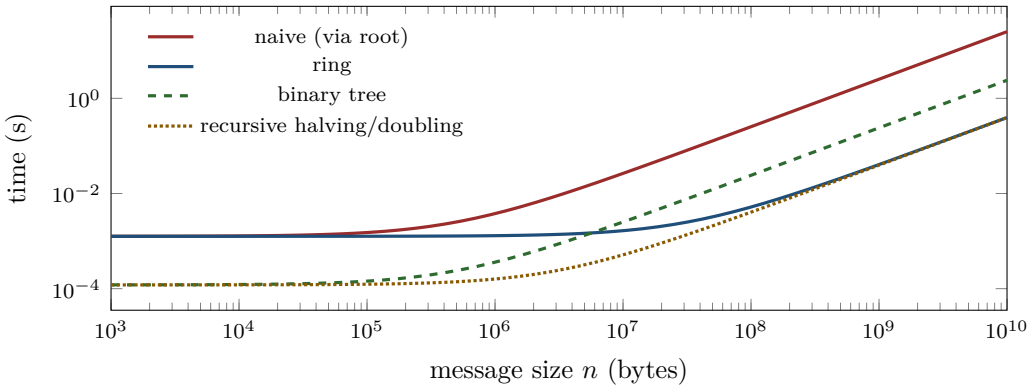


Figure 8.5: α–β all-reduce cost for $p = 64$, $\alpha = 10\,\mu s$, $B = 50\,\text{GB/s}$ (analytic, `llmsys.dist.costs.allreduce_time`). Tree algorithms win for small messages (latency), ring and halving/doubling for large ones (bandwidth); the naive root-based scheme is $\approx p/2$ times slower for large messages.

```
    for r in reqs:                          # post both before waiting: no ring
    deadlock
        r.wait()

def ring_allreduce(t):
    rank, p = dist.get_rank(), dist.get_world_size()
    right, left = (rank + 1) % p, (rank - 1) % p
    chunks = list(torch.tensor_split(t.view(-1), p))      # views into t
    for s in range(p - 1):                                # reduce-scatter
        send_i, recv_i = (rank - s) % p, (rank - s - 1) % p
        buf = torch.empty_like(chunks[recv_i])
        _sendrecv(chunks[send_i], buf, right, left)
        chunks[recv_i] += buf
    owned = (rank + 1) % p
    for s in range(p - 1):                                # all-gather
        send_i, recv_i = (owned - s) % p, (owned - s - 1) % p
        buf = torch.empty_like(chunks[recv_i])
        _sendrecv(chunks[send_i], buf, right, left)
        chunks[recv_i].copy_(buf)
    return t
```

8.2.4 The hardware the bytes travel on

Inside an HGX H100 node, eight GPUs are fully connected through NVSwitch with NVLink 4: about 450 GB/s per GPU in each direction. Between nodes, each GPU typically has its own 400 Gb/s InfiniBand or RoCE NIC, i.e. 50 GB/s per direction—an order of magnitude less.

PCIe Gen5 x16 sits at about 64 GB/s per direction. The large intra-/inter-node bandwidth gap is the single most important fact for parallelism layout: communication-heavy strategies (TP, SP) stay inside the NVLink domain; communication-light ones (DP, PP) span nodes. NCCL's reported *bus bandwidth* for all-reduce rescales the measured algorithm bandwidth by $2(p-1)/p$ (Eq. (8.2)) so that it can be compared directly with link speed.

Deeper Dive: Inside NCCL

NCCL splits a collective across several *channels* (each a ring or tree mapped onto a subset of links and driven by its own CUDA thread block) and pipelines each channel's chunk in small slices so that sending, receiving and reducing overlap. It picks among three wire *protocols*: *LL* (8-byte stores with an embedded flag: lowest latency, about half the bandwidth), *LL128* (128-byte lines with flags, near-full bandwidth on NVLink) and *Simple* (large chunks with explicit fences: full bandwidth, higher latency). On NVSwitch systems, *NVLS* performs the reduction inside the switch, and InfiniBand *SHARP* does the same in the network fabric, cutting the data each GPU must send for an all-reduce roughly in half. Because NCCL kernels occupy SMs, heavy communication also steals a few percent of compute from overlapping GEMMs—one reason comm/compute overlap is never perfectly free.

Napkin Math: Gradient all-reduce for a 7B model

bf16 gradients of 7B parameters are 14 GB. Ring all-reduce among the 8 GPUs of one node: $2 \cdot \frac{7}{8} \cdot 14\,\text{GB}/450\,\text{GB/s} \approx 54\,\text{ms}$. Across 64 GPUs over IB at 50 GB/s per GPU: $\approx 0.55\,\text{s}$. Compare with the compute of one step: at 32k tokens per GPU and 40% MFU, $6 \cdot 7\text{B} \cdot 32{,}768/396\,\text{TFLOP/s} \approx 3.5\,\text{s}$. Even across nodes, the all-reduce can be hidden behind the backward pass—if it is overlapped.

8.3 Data parallelism

Data parallelism replicates the model on p ranks, gives each rank a different $1/p$ of the global batch, and averages gradients before the optimizer step. If every rank starts from the same weights and applies the same averaged gradient with the same optimizer, the replicas stay identical forever, and the result is mathematically the same as single-device training on the whole batch (for mean-reduced losses over equal-sized shards). Lab 8 checks this equivalence to 10^{-6}.

The simplest implementation all-reduces each `.grad` after `backward()`. It has two problems. First, a model has hundreds of parameter tensors, many tiny (biases, norms), so the step pays the latency α hundreds of times. Second, communication starts only after the *whole* backward pass, so the network idles during compute and the GPUs idle during communication.

Bucketing and overlap. Gradients become available in roughly reverse order of the forward pass: the last layer's gradients are ready first. PyTorch DDP (Li et al., 2020) groups parameters, in reverse order, into buckets of about 25 MB, registers a hook on every parameter, and launches an asynchronous all-reduce for a bucket as soon as all its gradients exist. While NCCL reduces bucket k on a separate CUDA stream, the backward pass keeps computing the gradients of bucket $k+1$ (Fig. 8.6). Before the optimizer step, the training loop waits for all outstanding reductions.

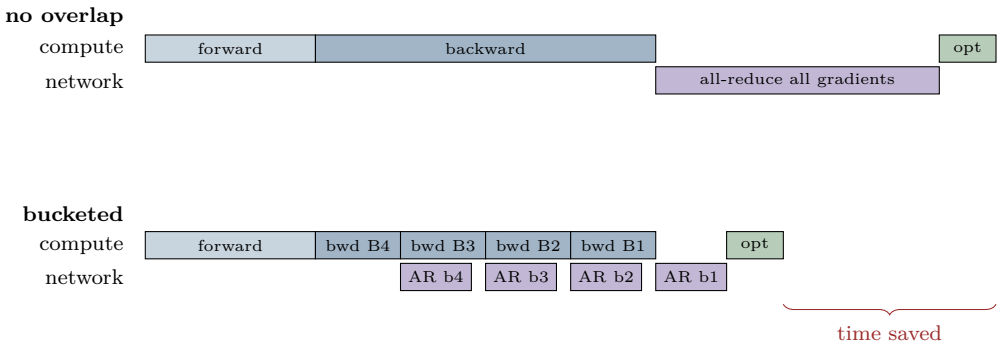


Figure 8.6: Bucketing overlaps gradient communication with the remaining backward computation. Only the reduction of the last bucket (the first layers) stays exposed. Buckets are formed in reverse layer order because that is the order in which gradients appear.

Listing 8.2: Bucketed DDP from scratch (`llmsys/dist/ddp.py`, abridged).

```python
class BucketedDDP(nn.Module):
    def __init__(self, module, bucket_cap_mb=25.0):
        super().__init__()
        self.module, self.world = module, dist.get_world_size()
        broadcast_parameters(module)                    # identical start everywhere
        self.buckets, cur, size = [], [], 0
        for p in reversed([p for p in module.parameters() if p.requires_grad]):
            cur.append(p); size += p.numel() * p.element_size()
            if size >= bucket_cap_mb * 2**20:
                self.buckets.append(_Bucket(cur)); cur, size = [], 0
        if cur: self.buckets.append(_Bucket(cur))
        for b in self.buckets:
            for p in b.params:
                self._bucket_of[p] = b
                p.register_post_accumulate_grad_hook(self._on_grad_ready)

    def _on_grad_ready(self, p):                        # runs inside backward()
        b = self._bucket_of[p]; b.ready += 1
        if b.ready == len(b.params):                    # bucket complete
            b.buffer = torch.cat([q.grad.reshape(-1) for q in b.params])
            b.work = dist.all_reduce(b.buffer, async_op=True)

    def finish_gradient_synchronization(self):
        for b in self.buckets:
            b.work.wait(); b.buffer /= self.world
            # ... copy slices of b.buffer back into each q.grad
```

Systems Lens: What production DDP adds

- **Gradient-as-bucket-view:** `.grad` tensors are views into the flat bucket buffer, so no `torch.cat`/copy-back is needed (our version copies twice).
- `no_sync()` for gradient accumulation: skip the all-reduce for all but the last micro-batch, otherwise accumulation multiplies communication by k.
- **Unused parameters** (e.g. inactive MoE experts) never fire their hook; DDP must detect them (`find_unused_parameters`) or the bucket never completes—exactly the

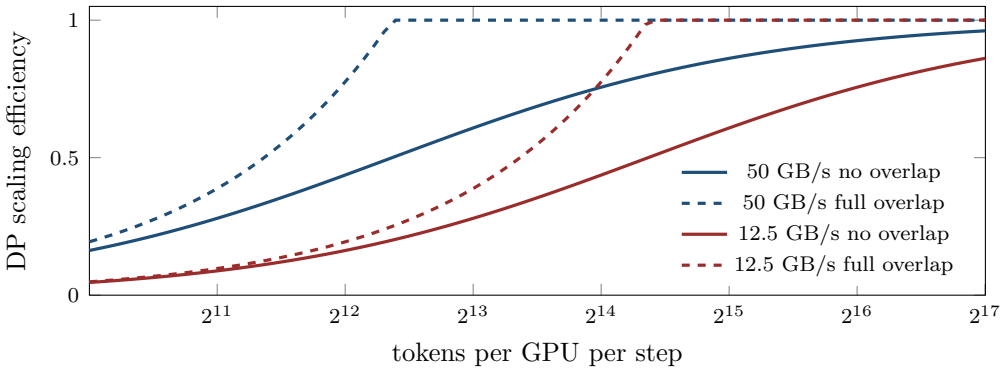


Figure 8.7: Analytic DP efficiency for a 7B model (bf16 gradients, 14 GB; ring all-reduce with large p; compute at 40% MFU of an H100, so $t_c \approx 0.106$ ms per token). With overlap, efficiency stays at 1 until the local batch drops below t_m/t_c tokens (about 5k tokens at 50 GB/s); without overlap it degrades smoothly. A slower network (e.g. 100 Gb Ethernet, 12.5 GB/s) moves the knee 4× to the right.

error our implementation raises.

- **Bucket order** is re-learned after the first iteration from the actual order in which hooks fired.
- **Compression hooks** (fp16/bf16 casting, PowerSGD) reduce bytes at some accuracy risk.

How far can DP scale? DP alone scales the batch: with p replicas and a per-GPU micro-batch of b sequences, the global batch is pb sequences. Beyond a *critical batch size* (McCandlish et al., 2018), larger batches stop reducing the number of steps needed, so extra replicas waste compute. LLM pretraining typically uses global batches of 1–16M tokens. And DP does nothing for memory: every replica still holds all 16 bytes per parameter. The next section removes that redundancy.

8.3.1 A step-time model for data parallelism

Let t_c be the compute time of one step on one GPU and t_m the time of the gradient all-reduce. Without overlap, the step takes $t_c + t_m$; with perfect overlap, $\max(t_c, t_m)$ (in practice the last bucket is always exposed). The *scaling efficiency* relative to a single GPU is therefore between

$$\eta_{\text{no overlap}} = \frac{t_c}{t_c + t_m} \qquad \text{and} \qquad \eta_{\text{overlap}} = \frac{t_c}{\max(t_c, t_m)}. \tag{8.3}$$

For ring all-reduce $t_m \approx 2\Psi/B$ is almost independent of p, while t_c is proportional to the *local* batch. Weak scaling (fixed local batch, growing global batch) therefore scales almost perfectly; strong scaling (fixed global batch split over more GPUs) shrinks t_c until communication dominates. Figure 8.7 plots both bounds for a 7B model as the local batch shrinks.

Feeding data to many ranks. Every DP rank must see a *different* slice of each global batch, and the union must be the same as in single-device training. PyTorch's `DistributedSampler` does this by permuting indices with a shared seed per epoch (`sampler.set_epoch(e)`) and giving rank r every p-th index starting at r. At pretraining scale, data is pre-tokenized into many binary shards; each rank streams its own subset of shards through a memory-mapped reader (`llmsys.data.TokenMemmapDataset`) and records its position so that a resumed job continues

stage	bytes/param	7.5B on 64 GPUs	communication per step
DP (stage 0)	16	120.0 GB	all-reduce grads: 2Ψ
ZeRO-1	$4 + 12/N_d$	31.4 GB	reduce-scatter grads + all-gather params: 2Ψ
ZeRO-2	$2 + 14/N_d$	16.6 GB	same: 2Ψ
ZeRO-3 / FSDP	$16/N_d$	1.9 GB	all-gather $\times 2$ + reduce-scatter: 3Ψ

Table 8.1: Model-state memory per GPU (computed with `llmsys.dist.costs.zero_memory_per_gpu`) and communication volume per rank (in units of the gradient size Ψ bytes) for the ZeRO stages.

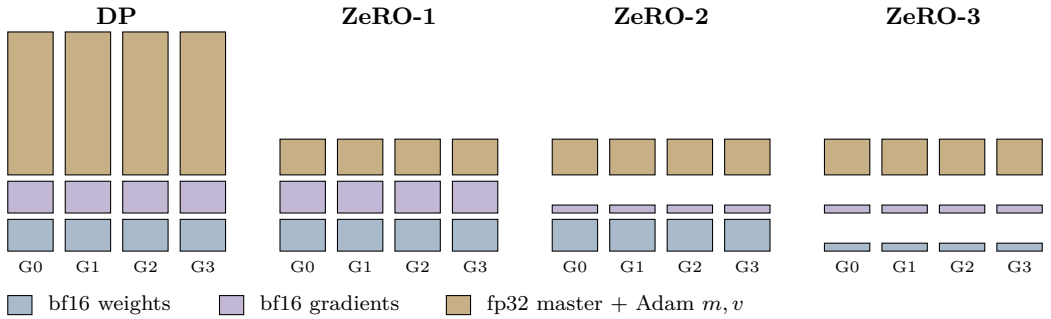


Figure 8.8: ZeRO stages on four GPUs. Bar heights are proportional to bytes per parameter (2, 2 and 12). Each stage shards one more kind of state so that every GPU keeps only its $1/N_d$ slice; ZeRO-3 shards everything.

exactly where it stopped. Forgetting `set_epoch` silently repeats the same order every epoch; giving two ranks the same seed for shard selection silently duplicates data.

Global batch bookkeeping. With micro-batch b sequences of length s, gradient accumulation k and DP degree d, one optimizer step consumes $b \cdot k \cdot d \cdot s$ tokens. Changing the number of GPUs while keeping the optimization identical means adjusting k so the product stays fixed; the learning rate schedule is defined over optimizer steps, not GPU-hours.

8.4 ZeRO and fully sharded data parallelism

In DP every rank stores identical copies of weights, gradients and optimizer moments. ZeRO (Rajbhandari et al., 2020) observes that each rank only *needs* the full set of some of these, some of the time, and shards the rest over the N_d data-parallel ranks in three stages. With Ψ parameters, 2-byte weights and gradients and $K = 12$ bytes of fp32 optimizer state per parameter:

$$\text{DP:} \quad (2 + 2 + K)\,\Psi \tag{8.4}$$

$$\text{ZeRO-1 (optimizer states):} \quad (2 + 2 + K/N_d)\,\Psi \tag{8.5}$$

$$\text{ZeRO-2 (+ gradients):} \quad (2 + (2 + K)/N_d)\,\Psi \tag{8.6}$$

$$\text{ZeRO-3 / FSDP (+ parameters):} \quad (2 + 2 + K)\,\Psi/N_d \tag{8.7}$$

ZeRO-1 and ZeRO-2. Each rank *owns* a $1/N_d$ shard of the parameters and keeps fp32 master weights and Adam moments only for that shard. After backward, gradients are *reduce-scattered* (each rank receives the summed gradient of its shard), the owner runs Adam on its shard, and the updated parameters are *all-gathered*. Since reduce-scatter + all-gather moves exactly the

bytes of one all-reduce, stages 1 and 2 cost the same communication as plain DP while cutting memory by up to 4× and 8×. Stage 2 also frees the non-owned gradient slices immediately after their reduction.

Our teaching implementation (Listing 8.3) assigns whole parameter tensors to owners and uses reduce/broadcast, because the gloo backend lacks reduce-scatter; production versions flatten parameters into one buffer and shard it element-wise for perfect balance.

Listing 8.3: ZeRO-1/2 with parameter-level ownership (`llmsys/dist/zero.py`, abridged).

```python
class ZeroAdamW:
    def __init__(self, params, lr, stage=1, **kw):
        self.params = [p for p in params if p.requires_grad]
        self.owner = partition_parameters(self.params, dist.get_world_size())
        mine = [p for p, o in zip(self.params, self.owner) if o == dist.get_rank()]
        self.local = torch.optim.AdamW(mine, lr=lr, **kw)   # states only for my shard

    @torch.no_grad()
    def step(self):
        for p, o in zip(self.params, self.owner):
            if self.stage == 1:
                dist.all_reduce(p.grad); p.grad /= self.world
            else:                                   # stage 2: only the owner keeps it
                dist.reduce(p.grad, dst=o)
                if dist.get_rank() == o: p.grad /= self.world
                else: p.grad = None
        self.local.step()                           # Adam on owned parameters only
        for p, o in zip(self.params, self.owner):
            dist.broadcast(p.data, src=o)           # everyone gets updated weights
```

ZeRO-3 and FSDP. Stage 3 also shards the bf16 parameters. The model is wrapped in *units* (typically one transformer block each). Just before a unit's forward, its parameters are all-gathered from all ranks; after the forward they are freed. The backward pass all-gathers them again, computes gradients, and reduce-scatters the gradients so each rank keeps only its shard. Communication is 1.5× that of DP (3Ψ vs. 2Ψ bytes), but only one or two units are ever materialized, so per-GPU memory falls to $16\Psi/N_d$ plus a unit or two of full parameters. PyTorch's FSDP (Zhao et al., 2023) implements this design; FSDP2 shards each parameter individually as a `DTensor`, which composes cleanly with tensor parallelism.

Systems Lens: Making FSDP fast

- **Prefetching.** Issue the all-gather for unit $i+1$ while unit i computes (forward), and for unit $i-1$ during unit i's backward. With enough compute per unit, the all-gathers are completely hidden.
- **Hybrid sharding (HSDP).** Shard within a node (fast NVLink) and replicate across nodes, combining FSDP's memory savings with DP's lighter cross-node traffic (a plain all-reduce of gradient shards across replicas).
- **Mixed-precision communication.** All-gather bf16 parameters and reduce-scatter bf16 (or fp32 for stability) gradients; keep fp32 master shards locally.

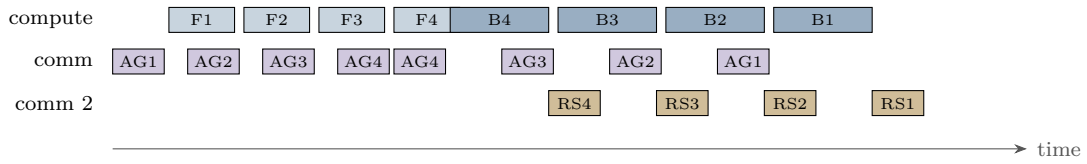


Figure 8.9: FSDP with prefetching for a 4-unit model. The all-gather (AG) of unit $i+1$ runs while unit i computes forward (F); in backward (B) units are re-gathered in reverse order and each unit's gradients are reduce-scattered (RS) as soon as they exist. Only one or two units' full parameters are ever resident.

- **Offload.** ZeRO-Offload/Infinity move optimizer states and even parameters to CPU memory or NVMe, trading PCIe bandwidth for capacity—useful for fine-tuning big models on few GPUs, rarely for pretraining throughput.

8.5 Tensor parallelism

ZeRO shards *storage*, but every layer still runs on one device with full-size activations. Tensor parallelism (Megatron-LM; Shoeybi et al., 2019) splits the *computation* of each layer. Take a linear layer $\mathbf{Y} = \mathbf{X}\mathbf{W}^\top$ with $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and t ranks.

Column-parallel Split $\mathbf{W}$ by output rows, $\mathbf{W} = [\mathbf{W}_1; \dots; \mathbf{W}_t]$. Rank r computes $\mathbf{Y}_r = \mathbf{X}\mathbf{W}_r^\top$, a slice of the output features. Input $\mathbf{X}$ must be replicated. In backward, each rank's $\partial\mathcal{L}/\partial\mathbf{X}$ is only the contribution of its slice, so the input gradient needs an **all-reduce**.

Row-parallel Split $\mathbf{W}$ by input columns. Rank r consumes the matching slice $\mathbf{X}_r$ of the input and produces a partial result $\mathbf{X}_r\mathbf{W}_r^\top$; the true output is the sum over ranks, an **all-reduce** in the forward pass. Its backward needs no communication: the output gradient is already replicated.

The trick is to chain them. In the MLP $\mathbf{Y} = \text{GELU}(\mathbf{X}\mathbf{A}^\top)\mathbf{B}^\top$, make $\mathbf{A}$ column-parallel and $\mathbf{B}$ row-parallel. Each rank then holds a slice of the hidden features, applies GELU locally (element-wise, so no communication), and feeds that slice straight into its row-parallel slice of $\mathbf{B}$. One all-reduce at the end of the forward pass and one at the start of the backward pass suffice (Fig. 8.10). Megatron calls the two conjugate operators f (identity forward, all-reduce backward) and g (all-reduce forward, identity backward). Self-attention splits the same way: heads are distributed column-parallel through the Q/K/V projections (each rank runs complete heads, so softmax stays local) and the output projection is row-parallel. A transformer layer thus needs **two all-reduces in forward and two in backward**.

Listing 8.4: The f/g operators and parallel linear layers (`llmsys/dist/tensor_parallel.py`, abridged).

```python
class CopyToTP(torch.autograd.Function):          # f
    @staticmethod
    def forward(ctx, x): return x
    @staticmethod
    def backward(ctx, g):
        g = g.clone(); dist.all_reduce(g); return g

class ReduceFromTP(torch.autograd.Function):      # g
    @staticmethod
```

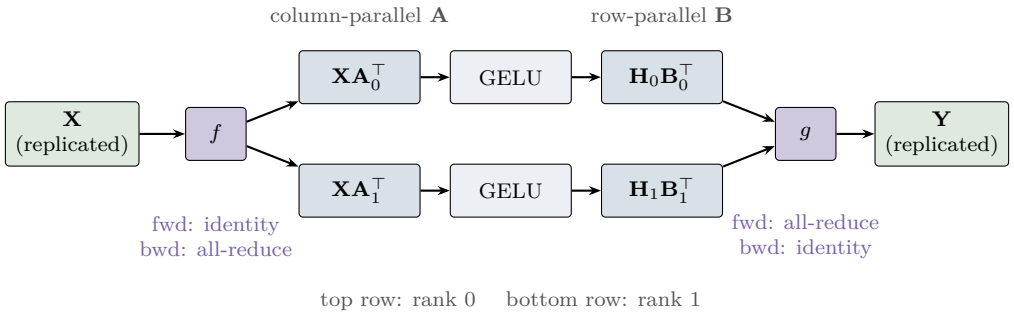


Figure 8.10: Megatron-style tensor-parallel MLP on two ranks. The hidden activation **H** never leaves its rank; only the f and g operators communicate.

```
    def forward(ctx, x):
        x = x.clone(); dist.all_reduce(x); return x
    @staticmethod
    def backward(ctx, g): return g

class ColumnParallelLinear(nn.Module):            # holds out_features / t rows
    def forward(self, x):
        return F.linear(CopyToTP.apply(x), self.weight, self.bias)

class RowParallelLinear(nn.Module):               # holds in_features / t columns
    def forward(self, x_local):
        y = ReduceFromTP.apply(F.linear(x_local, self.weight))
        return y + self.bias                      # bias added once, after the sum
```

Napkin Math: Why TP stays inside a node

A 70B layer ($h = 8192$) with $s = 4096$, $b = 1$ and $t = 8$: each all-reduce carries the $s \cdot b \cdot h$ bf16 activation, 67 MB; four per layer per step at $2 \cdot \frac{7}{8} \cdot 67\,\text{MB}$ each ≈ 0.47 GB per rank. At NVLink's 450 GB/s: 1.0 ms per layer, versus ≈ 5.4 ms of compute per layer per GPU at 50% MFU. Over 50 GB/s IB the same traffic takes 9.4 ms—longer than the compute. TP is therefore almost always confined to the 8 GPUs of an NVLink domain (or the 72 of an NVL72 rack).

Hiding TP communication. The all-reduces in Fig. 8.10 sit on the critical path: the next layer cannot start until g completes. Two families of techniques overlap them anyway. *Decomposition* (Wang et al., 2023; "async TP" in PyTorch, Flux, and NVIDIA's userbuffers) splits the GEMM adjacent to a collective into chunks along the sequence dimension, so that chunk i's reduce-scatter or all-gather proceeds while chunk $i + 1$ is multiplied. *Micro-batch interleaving* runs two independent micro-batches through the layer out of phase, so one computes while the other communicates (the approach behind DeepSeek-V3's dual micro-batch overlap). Both turn the 1.0 ms of the napkin math above into almost-free communication when the chunked GEMMs remain large enough to run efficiently.

Key Idea: TP degree is a trade-off, not a free lunch

Each doubling of t halves every GEMM's size along one dimension. A 4096×8192 GEMM split 16 ways becomes 4096×512, which may no longer reach peak tensor-core efficiency,

while the activation all-reduce stays the same size. Choose the smallest t that makes memory fit.

Sequence parallelism. TP leaves LayerNorm, dropout and the residual stream replicated: every rank stores the full $s \times b \times h$ activations there. Korthikanti et al. (2022) split these regions along the *sequence* dimension instead and replace each all-reduce with a reduce-scatter (entering the sequence-parallel region) plus an all-gather (leaving it). Communication volume is unchanged, since all-reduce = reduce-scatter + all-gather, while the replicated activations shrink by t.

Vocabulary parallelism. The embedding table and output head ($V \times h$, up to 1.2B parameters for $V = 152$k, $h = 8192$) are split along the vocabulary. The cross-entropy is then computed without gathering logits: each rank computes its local max and local $\sum \exp$ over its vocabulary slice, two small all-reduces produce the global normalizer, and the target logit comes from whichever rank owns the target ID.

8.6 Pipeline parallelism

Pipeline parallelism assigns contiguous groups of layers (stages) to different devices. Only the activations at stage boundaries cross devices—$s \cdot b \cdot h$ values per micro-batch, point-to-point—so PP tolerates slow inter-node links well. The catch is idleness: stage 2 cannot start until stage 1 finishes. GPipe (Huang et al., 2019) splits each batch into m micro-batches and streams them through the stages: all forwards, then all backwards, then one synchronous optimizer step. With p stages, per-micro-batch forward time t_f and backward time t_b, each stage is idle for $(p-1)(t_f + t_b)$ out of $(m+p-1)(t_f + t_b)$:

$$\text{bubble fraction} = \frac{p-1}{m+p-1}. \tag{8.8}$$

To keep the bubble small one needs $m \gg p$ (e.g. $m \geq 4p$ gives $< 20\%$). But GPipe holds the activations of *all* m micro-batches at once before the first backward.

1F1B. PipeDream-Flush / Megatron's *one-forward-one-backward* schedule (Narayanan et al., 2021) has the same bubble but starts each micro-batch's backward as early as possible: stage s runs $p-s-1$ warm-up forwards, then alternates one forward and one backward, then drains. A stage never holds more than $p-s$ micro-batches of activations, independent of m. Figure 8.11 shows both schedules as produced by `llmsys.dist.pipeline.simulate`.

Shrinking the bubble. *Interleaved* 1F1B gives each device v non-contiguous "virtual" stages (e.g. layers 1–4 and 17–20), shrinking the bubble by v at the cost of v times more point-to-point messages. *Zero-bubble* schedules (Qi et al., 2024) split the backward pass into the input-gradient part B (on the critical path) and the weight-gradient part W (which can be deferred), and fill the bubbles with W work. DeepSeek-V3's *DualPipe* feeds micro-batches from both ends of the pipeline and overlaps computation with the all-to-all communication of its MoE layers.

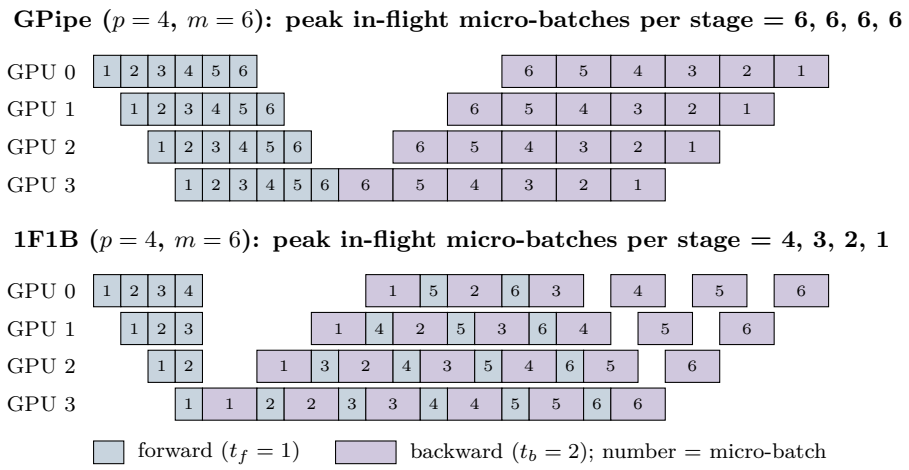


Figure 8.11: Pipeline schedules simulated with `llmsys.dist.pipeline`. Both take 27 time units and idle for $(p-1)/(m+p-1) = 1/3$ of the time (white gaps), but 1F1B frees each micro-batch's activations as soon as its backward completes.

Systems Lens: Practical pipeline concerns

- **Balance.** The slowest stage sets the pace. The embedding layer and the output head with its large softmax make the first and last stages heavier; Megatron lets you assign fewer transformer layers to them.
- **Micro-batch count vs. global batch.** m is bounded by global batch / (DP degree $\times$ micro-batch size). With a fixed global batch, more DP means fewer micro-batches per pipeline and a bigger bubble—a central tension of 3D parallelism.
- **Activation memory is uneven**: stage 0 holds p micro-batches under 1F1B, the last stage only one.
- **Weight updates are synchronous** (flush at the end of each batch), which keeps the math identical to non-pipelined training; asynchronous PipeDream variants with weight stashing are rarely used for LLMs.

8.7 Sequence and context parallelism for long contexts

At 128k-token contexts the activations of even one sequence overwhelm a GPU, and attention's $O(s^2)$ compute dominates. *Context parallelism* (CP) splits the sequence of every layer—not just the LayerNorm regions—across c devices. The MLP and projections are per-token and need no communication; attention needs every query to see every (earlier) key.

Ring attention (Liu et al., 2023). Each device keeps its query chunk and passes its key/value chunk around a ring. At each of c steps it computes attention of its queries against the visiting K/V chunk and folds the result into its output with the online-softmax rescaling of FlashAttention (Chapter 4): keep a running max m and normalizer ℓ per query, and rescale the partial output when a new block raises the max. Sending the next K/V chunk overlaps with computing on the current one. With causal masking, naive contiguous chunking makes the last device do c times more work than the first; *zig-zag* assignment (device i gets chunks i and $2c-1-i$) balances it.

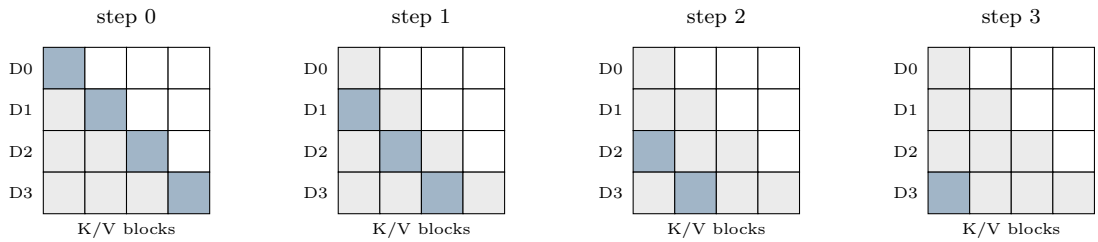


Figure 8.12: Ring attention on four devices with contiguous chunks. Row q is device q's query block; at step t it holds K/V block $(q-t) \bmod 4$ (dark). White cells are causally masked: device 0 has useful work only at step 0 while device 3 works at every step—the imbalance that zig-zag chunk assignment removes.

Ulysses (Jacobs et al., 2023). Alternatively, an all-to-all before attention re-partitions from "all heads, $1/c$ of the sequence" to "$1/c$ of the heads, the full sequence", runs ordinary (Flash)attention locally, and a second all-to-all switches back. Communication per device is $O(s\,h/c)$ and independent of the number of steps, but the degree is limited by the number of (KV) heads.

8.8 Expert parallelism

Mixture-of-experts layers (Chapter 6) replace the MLP with E expert MLPs and a router that sends each token to its top-k experts. With expert parallelism, experts are spread over e devices. Each MoE layer then needs two all-to-alls: *dispatch* (send each token's hidden state to the devices hosting its chosen experts) and *combine* (send results back). Per token and layer, about $2k \cdot h$ values cross the network in each direction. Systems issues include load imbalance (a popular expert becomes a straggler; auxiliary balancing losses or bias-based balancing mitigate it), token dropping when an expert exceeds its *capacity* (capacity factor$\times$ tokens$/E$), and topology-aware routing that limits each token to experts on a few nodes so that most all-to-all traffic stays on NVLink (as in DeepSeek-V3).

Napkin Math: All-to-all volume of a large MoE

Take a DeepSeek-V3-like layer ($h = 7168$, top-$k = 8$ routed experts) and 8,192 tokens per GPU. Dispatch sends $8192 \cdot 8 \cdot 7168$ bf16 values ≈ 0.94 GB per GPU per MoE layer, and combine sends the same back—every layer, forward and backward. At 50 GB/s that is ≈ 19 ms per direction per layer, comparable to the expert compute itself. Hence the emphasis on node-limited routing (most traffic on NVLink), FP8 dispatch, and schedules that overlap one micro-batch's all-to-all with another's computation.

8.9 Putting it together: 3D and 4D parallelism

Large runs compose these strategies on a *device mesh*. The ordering follows communication intensity: TP (and SP) innermost on NVLink; CP next; PP across nodes; DP/FSDP outermost (Fig. 8.13). In PyTorch, `init_device_mesh("cuda", (dp, pp, tp))` creates the sub-groups and FSDP2/TP/PP APIs each take one mesh dimension.

A planning recipe.

1. Pick the smallest TP degree (within a node) that, together with PP and ZeRO, makes

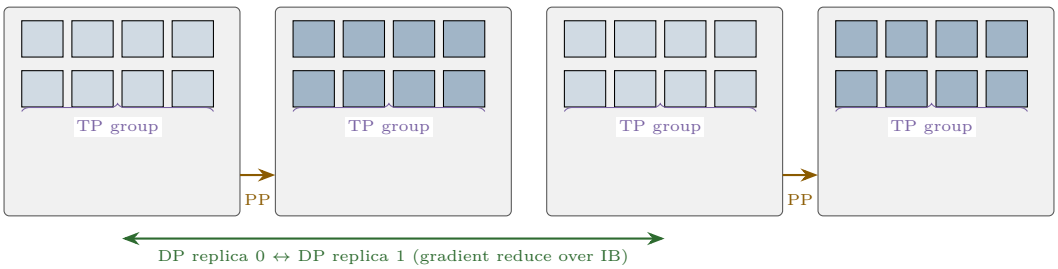


Figure 8.13: A 32-GPU mesh with TP = 8 (one node), PP = 2 (light/dark nodes are stages 0 and 1) and DP = 2. Rank = dp·16 + pp·8 + tp: TP neighbors are adjacent ranks on the same NVSwitch; PP sends activations to the next node; DP all-reduces span the whole cluster.

strategy	splits	saves memory of	communication	typical link
DP	batch	—	all-reduce grads, 2Ψ/step	IB
ZeRO-1/2	batch	optimizer (+grads)	same as DP	IB
FSDP/ZeRO-3	batch	everything	3Ψ/step, per-unit gathers	IB / NVLink
TP (+SP)	matrices	weights, activations	4 all-reduces of sbh per layer	NVLink
PP	layers	weights, activations	p2p sbh per micro-batch	IB
CP	sequence	activations	K/V ring or all-to-all	NVLink / IB
EP	experts	expert weights	2 all-to-alls per MoE layer	NVLink / IB

Table 8.2: Parallelism strategies at a glance.

model states and activations fit; larger TP shrinks per-GPU matmuls and lowers efficiency.

2. Add PP when the model does not fit within one NVLink domain; choose p so that layers divide evenly and $m \geq 4p$ micro-batches are available.

3. Add CP only when sequence length requires it.

4. Spend all remaining GPUs on DP (with ZeRO-1 or FSDP), and check that the global batch stays below the critical batch size.

5. Verify with napkin math: memory per GPU, bubble fraction, exposed communication.

Napkin Math: A 70B run on 1,024 H100s

Choose TP = 8, PP = 4, DP = 32 with ZeRO-1, $s = 4096$, global batch 4M tokens.

- Parameters per GPU: $70\text{B}/(8 \cdot 4) = 2.19\text{B}$. Model states with ZeRO-1: $(2 + 2 + 12/32) \times 2.19\text{B} \approx 9.6\,\text{GB}$.
- Activations: Eq. (7.7) with FlashAttention gives 1.14 GB per layer per sequence; TP+SP divides by 8; 20 layers per stage; stage 0 holds up to $p = 4$ micro-batches under 1F1B: $1.14/8 \times 20 \times 4 \approx 11.4\,\text{GB}$. Total $\approx 21\,\text{GB}$—comfortable on 80 GB, leaving room for larger micro-batches or less recomputation.
- Bubble: 1,024 sequences per step / DP 32 = 32 micro-batches per pipeline; $(4 - 1)/(32 + 3) = 8.6\%$. With PP = 8 it would be 17.9% (or 9.9% interleaved with $v = 2$).
- DP gradient traffic: each rank reduces its 4.4 GB bf16 shard over 32 replicas: $\approx 0.17\,\text{s}$

at 50 GB/s, hidden behind ≈ 4.1 s of compute per step at 40% MFU.

8.10 From scratch to production APIs

Everything above has a production counterpart in PyTorch. A training script is launched with one process per GPU by `torchrun`, which sets `RANK`, `LOCAL_RANK`, `WORLD_SIZE` and the rendezvous address:

```
torchrun --nnodes 4 --nproc-per-node 8 --rdzv-backend c10d \
         --rdzv-endpoint $HEAD_NODE:29500 train.py --tp 8 --dp 4
```

Listing 8.5: Composing TP and FSDP2 on a 2-D device mesh (sketch; PyTorch 2.4+ APIs).

```
import os, torch, torch.distributed as dist
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.fsdp import fully_shard, MixedPrecisionPolicy
from torch.distributed.tensor.parallel import (parallelize_module,
    ColwiseParallel, RowwiseParallel)

torch.cuda.set_device(int(os.environ["LOCAL_RANK"]))
dist.init_process_group("nccl")
mesh = init_device_mesh("cuda", (4, 8), mesh_dim_names=("dp", "tp"))

model = build_model()                                  # e.g. a Qwen3Model on "meta"
for blk in model.trf_blocks:                           # Megatron-style TP inside a
    node
    parallelize_module(blk, mesh["tp"], {
        "att.W_query": ColwiseParallel(), "att.W_key": ColwiseParallel(),
        "att.W_value": ColwiseParallel(), "att.out_proj": RowwiseParallel(),
        "ff.fc1": ColwiseParallel(), "ff.fc2": ColwiseParallel(),
        "ff.fc3": RowwiseParallel()})
mp = MixedPrecisionPolicy(param_dtype=torch.bfloat16, reduce_dtype=torch.float32)
for blk in model.trf_blocks:                           # FSDP units = blocks, over dp
    fully_shard(blk, mesh=mesh["dp"], mp_policy=mp)
fully_shard(model, mesh=mesh["dp"], mp_policy=mp)
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, fused=True)
```

The mapping to this chapter is direct: `ColwiseParallel`/`RowwiseParallel` are our column/row layers (the attention module must also be told that each rank now holds H/t heads); `fully_shard` is ZeRO-3 with per-block units, prefetching and reduce-scatter; the mesh encodes the rank layout of Fig. 8.13. Pipeline parallelism is available through `torch.distributed.pipelining` (schedules `ScheduleGPipe`, `Schedule1F1B`, `ScheduleInterleaved1F1B`), and `torch.distributed.checkpoint` saves and reshards DTensor-based state. Megatron-LM/Megatron-Core, DeepSpeed and TorchTitan are the reference systems that package these pieces for LLM pretraining.

Pitfall: Test the small configuration first

Every parallel layout must reproduce the single-device loss curve. Before a large launch, run the same seed on 1 GPU and on a tiny mesh (e.g. TP = 2, DP = 2) for a few hundred steps and compare losses to several decimal places. This is exactly what Lab 8's tests do, in miniature.

8.11 Reliability at scale

At thousands of GPUs, failure is the steady state. Meta reported 466 job interruptions during a 54-day period of Llama 3 405B pretraining on 16k H100s, 419 of them unexpected, most attributed to GPU or HBM faults and other hardware issues (Llama Team, 2024)—roughly one every three hours. Training infrastructure therefore treats recovery as a first-class path:

- **Fast, frequent checkpoints** (asynchronous, sharded, Section 7.4) and fast restarts (pre-warmed spare nodes, cached container images, quick NCCL re-initialization).
- **Elastic launch.** `torchrun` restarts the worker group on failure; with resharding-capable checkpoints the job can resume on fewer or more nodes.
- **Hang and straggler detection.** A single slow GPU (thermal throttling, a flaky link) slows every synchronous collective. Collective timeouts, per-rank step-time telemetry and NCCL flight recorders identify the culprit.
- **Silent data corruption.** Rare hardware faults flip bits without crashing. Symptoms are loss spikes or NaNs on one replica; defenses include comparing gradient norms across DP replicas (they should match exactly after the all-reduce) and periodic known-answer tests.

Pitfall: Determinism across ranks

DP correctness assumes every replica applies *bitwise identical* updates. Non-deterministic kernels or different reduction orders across ranks make replicas drift apart silently. Broadcast parameters at start, use the same seeds for anything that must agree (dropout masks are fine to differ; parameter initialization is not), and periodically check a weight checksum across ranks.

8.12 Summary

- Collectives are the vocabulary of distributed training; the α–β model with the ring identity all-reduce = reduce-scatter + all-gather explains most of their costs.
- DP scales throughput but not memory; bucketing and overlap hide gradient communication.
- ZeRO-1/2 shard optimizer states and gradients at no extra communication; ZeRO-3/FSDP shard everything for 1.5× the traffic.
- TP splits matrices with paired column/row layers (two all-reduces per pass per sub-block) and needs NVLink-class bandwidth; SP removes the remaining replicated activations.
- PP splits layers; its bubble is $(p-1)/(m+p-1)$; 1F1B bounds activation memory, interleaving and zero-bubble schedules shrink idle time.
- CP (ring attention, Ulysses) handles long sequences; EP distributes MoE experts with all-to-all.
- Real jobs compose these on a device mesh ordered by bandwidth needs and must be engineered for frequent failure.

8.13 Exercises

Exercise 8.1 (*Ring all-reduce cost*). Derive Eq. (8.2) by counting the bytes each rank sends in each of the $2(p-1)$ steps. Then show that the recursive halving/doubling algorithm achieves the same bandwidth term with only $2\log_2 p$ latency terms when p is a power of two.

Exercise 8.2 (*Two-level all-reduce*). A cluster has N nodes of 8 GPUs, with intra-node bandwidth B_1 and per-GPU inter-node bandwidth $B_2 \ll B_1$. Design a hierarchical all-reduce (reduce-scatter in the node, all-reduce across nodes on 1/8 shards, all-gather in the node) and give its cost. Compare with a flat ring over all $8N$ GPUs for $B_1 = 450$, $B_2 = 50\,\text{GB/s}$.

Exercise 8.3 (*Bucket size*). With $\alpha = 20\,\mu\text{s}$ and 50 GB/s, what bucket size keeps the latency overhead below 5% of the transfer time? Why might a very large bucket still be bad for overlap?

Exercise 8.4 (*No-sync accumulation*). Modify `BucketedDDP` to support a `no_sync()` context manager. Measure, with the gloo backend, the time for 4-step gradient accumulation with and without it.

Exercise 8.5 (*ZeRO arithmetic*). Reproduce Table 8.1. For a 13B model on 16 GPUs with 80 GB each, which is the lowest ZeRO stage that leaves at least 30 GB per GPU for activations?

Exercise 8.6 (*TP backward*). Show that for a column-parallel linear layer the gradient with respect to its input is $\sum_r \frac{\partial \mathcal{L}}{\partial \mathbf{Y}_r}\mathbf{W}_r$, which justifies the all-reduce in the backward of f. Why does the row-parallel layer need no communication in backward?

Exercise 8.7 (*Attention under TP*). Extend `TensorParallelMLP` to a `TensorParallelAttention` module (heads split across ranks, column-parallel QKV, row-parallel output). Verify against `MultiHeadAttention` from `llmsys.layers` with gloo on two processes. What constraint on the number of heads arises? How does GQA (Chapter 6) complicate it when t exceeds the number of KV heads?

Exercise 8.8 (*Bubbles*). Using `llmsys.dist.pipeline`, plot bubble fraction and peak in-flight micro-batches for GPipe and 1F1B with $p = 8$ and $m \in \{8, 16, 32, 64\}$. Then implement an interleaved schedule with $v = 2$ and confirm that the bubble roughly halves.

Exercise 8.9 (*Causal load balance in ring attention*). With c devices and contiguous chunks, count the number of (query block, key block) pairs each device computes under a causal mask. Show that the zig-zag assignment equalizes the work.

Exercise 8.10 (*Plan a run*). Plan the parallel layout for a 405B dense model ($L = 126$, $h = 16{,}384$, $s = 8192$) on 16,384 H100s with a 16M-token global batch. Report TP, PP, CP, DP, per-GPU memory, bubble and the exposed DP communication, stating every assumption.

8.14 Lab 8: Distributed training from scratch

Lab 8: Distributed training from scratch

Code: `code/labs/lab08_distributed`

Goals

Implement the core distributed-training primitives yourself, verify that each is *mathematically equivalent* to single-process training, and connect measured behavior to the cost models of this chapter.

Setup

No GPUs needed. `llmsys.dist.launch.run_distributed(fn, world_size)` spawns CPU processes with the gloo backend; every collective you use also exists in NCCL. Run the tests with `LAB_IMPL=starter python -m pytest -q labs/lab08_distributed` (one process group of 3 ranks runs all distributed checks; a non-power-of-two world catches indexing bugs).

Tasks

1. `naive_allreduce`, `ring_reduce_scatter`, `ring_all_gather`, `ring_allreduce` using only `isend`/`irecv`; must handle any tensor shape and sizes not divisible by p.
2. `allreduce_time` (α–β, ring and naive), `zero_memory_per_gpu` (stages 0–3), `pipeline_bubble_fraction`.
3. `BucketedDDP`: reverse-order buckets, post-accumulate-grad hooks, async all-reduce, `finish_gradient_synchronization`. Weights after 3 steps must equal full-batch single-process AdamW, even when ranks start from different initializations.
4. `ZeroAdamW` stage 1 and 2 with balanced `partition_parameters`: same equivalence; each rank stores Adam state only for the parameters it owns.
5. `CopyToTP`, `ReduceFromTP`, `ColumnParallelLinear`, `RowParallelLinear`: the tensor-parallel MLP must match the full MLP in outputs, input gradients and the corresponding weight-gradient slices.
6. `gpipe_order` and `one_f_one_b_order`: the provided simulator must report bubble $(p-1)/(m+p-1)$ and peak in-flight m (GPipe) and $p-s$ (1F1B).

Measurements

- `python bench.py allreduce -world 4`: time naive, ring and library all-reduce for 1 KB–16 MB and fit α, β for each. Expect the library to win at small sizes (better latency) and your ring to beat the naive scheme at large sizes. On a multi-GPU node switch to `backend="nccl"` and compare the fitted bandwidth with NVLink specs. (Timings on a shared or oversubscribed CPU are noisy; take the minimum of several repetitions.)
- `python bench.py ddp -world 2`: per-parameter all-reduce vs. bucketed DDP.
- `python bench.py pipeline -stages 8 -mb 32`: compare with Eq. (8.8).

Deliverables

Passing tests; a plot of the fitted α–β curves against your measurements; a short table comparing DDP variants; and a written 3D-parallel plan for a 70B model on 256 GPUs with the napkin math of Section 8.9.

Stretch goals

Implement ZeRO-3 for an `nn.Sequential` (gather each layer's parameters in a forward pre-hook, free them afterwards, re-gather in backward); a hierarchical all-reduce; a real two-stage pipeline with `dist.send`/`dist.recv` executing your 1F1B order; ring attention on top of `llmsys.flash` from Chapter 4.

Chapter 9

Fine-Tuning for Classification

A pretrained language model is a general-purpose engine for predicting the next token. Most deployed models, though, have a narrower job: route a support ticket, flag a message as spam, decide whether a code change touches a security-sensitive file. *Classification fine-tuning* is the most direct way to turn a pretrained model into such a component. We keep the transformer trunk, throw away the 50,257-way vocabulary projection, put a tiny C-way projection in its place, and train on labelled examples.

The modelling change is small, but it raises several systems questions that recur for the rest of the book. Which parameters do we update, and what does freezing the rest save in memory and in FLOPs? How much of each batch is padding, and how do we stop paying for it? Which token's hidden state should the head read, and does that answer change with padding? A classifier is also the simplest example of a *prefill-only* inference workload: one forward pass, no decoding loop, no KV cache that outlives the request. That makes it a useful contrast to the generative serving systems of Chapters 11 and 12.

9.1 The fine-tuning landscape

LLM 6.1

Once pretraining (Chapter 7) is done, there are two classic ways to adapt a language model to a task.

Instruction fine-tuning trains the model to read a natural-language request and answer in natural language. The output space stays the full vocabulary, so a single model can handle a wide range of tasks: summarise, translate, fix the grammar, explain a stack trace. The cost is that it needs diverse, carefully formatted data, and the outputs are free text that must be parsed. This is Chapter 10.

Classification fine-tuning trains the model to emit one label from a fixed set. The output space shrinks to C classes (spam/not-spam, sentiment, intent, toxicity category). The model is useless for anything else, but it is cheap to train, its output needs no parsing, and its accuracy is easy to measure.

In a production pipeline you often need both. An instruction-tuned assistant may sit behind a battery of small classifiers that screen inputs, route requests to specialised models, or detect policy violations. These classifiers run on *every* request, so their cost per token matters more

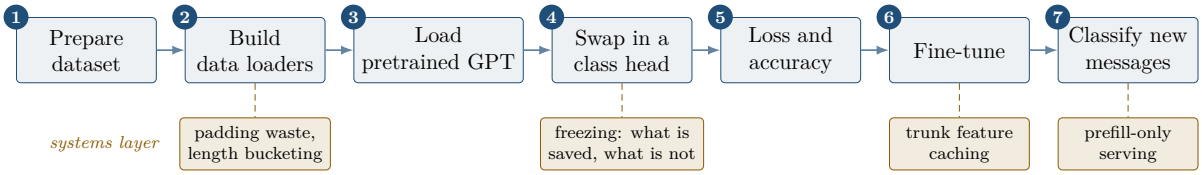


Figure 9.1: Chapter roadmap. The top row is the modelling pipeline for turning a pretrained GPT into a spam classifier; the dashed boxes are the systems questions we attach to each stage.

than their absolute size suggests.

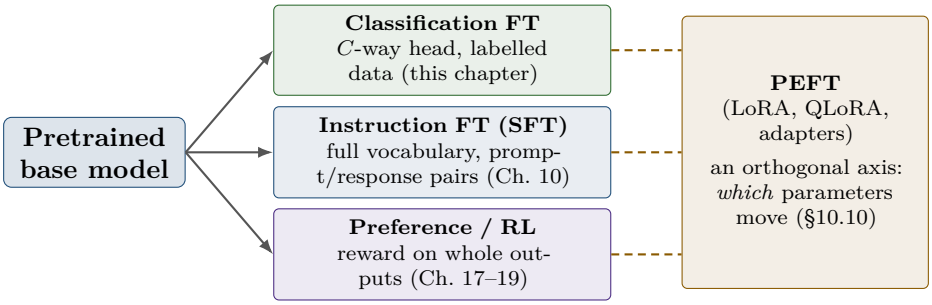


Figure 9.2: Ways to adapt a pretrained model. *What* we optimise (labels, responses, rewards) is one axis; *which parameters* we update (all, the last few layers, or low-rank adapters) is a separate axis that applies to every method.

A second axis, separate from the first, is *how many parameters move*. You can update all of them (full fine-tuning), only the top few layers plus the new head (the default in this chapter), or small low-rank adapters (LoRA, Section 10.10). The choice sets the memory footprint, the per-step compute, and how much capacity the model has to adapt.

Systems Lens: What each adaptation regime keeps in memory

For a model with N parameters trained with AdamW in mixed precision, the persistent state is about $16N$ bytes (Chapter 7): bf16 weights (2), bf16 gradients (2), fp32 master weights (4), and two fp32 Adam moments (8). Freezing a parameter removes everything except the 2-byte weight. If only a fraction f of the parameters is trainable, the persistent state is roughly $2N + 14fN$ bytes. For GPT-2 small with only the last block, the final LayerNorm and the head trainable, $f \approx 0.057$ (Section 9.5). The optimizer state then becomes a rounding error, and the memory budget is set by weights and activations.

Napkin Math: Full vs. partial fine-tuning of GPT-2 small

GPT-2 small has $N \approx 124$M unique parameters. Full fine-tuning with AdamW: $16 \times 124\text{M} \approx 2.0$ GB of persistent state. Training only the last block, the final norm and a 2-way head (≈ 7.1M parameters): $2 \times 124\text{M} + 14 \times 7.1\text{M} \approx 248 + 99 = 347$ MB. The six-fold reduction matters little on a datacenter GPU. On a laptop, or when one GPU hosts dozens of task-specific classifiers, it decides whether the job fits.

9.2 Preparing the dataset

LLM 6.2

Our running example is the *SMS Spam Collection*, a public corpus of 5,574 short text messages. Each is labelled `ham` (legitimate, 4,827 messages) or `spam` (747 messages). It is distributed as

a tab-separated file, one `label<TAB>text` pair per line. The data is small enough to fine-tune GPT-2 small on a laptop in minutes, and realistic enough to show the usual problems: class imbalance, long-tailed message lengths, and near-duplicate messages.

Class imbalance. Only $747/5574 \approx 13.4\%$ of messages are spam. A model that always predicts `ham` already scores 86.6% accuracy, which makes accuracy a poor signal early in training. The simplest fix is *undersampling*: keep every spam message and draw an equal-sized random subset of 747 ham messages, giving a balanced dataset of 1,494 examples. Undersampling throws data away, which is acceptable here because we want a fast, interpretable experiment. In production you would more likely keep all the data and use one of the following:

- a class-weighted loss, where the weight of class c is proportional to $1/\text{freq}(c)$;
- oversampling the minority class, either by repetition or by generating paraphrases with an LLM;
- reporting precision, recall and F1 (or the area under the precision–recall curve) instead of accuracy, and choosing a decision threshold for the cost you care about. Here that cost is usually a false positive: a real message hidden as spam.

Listing 9.1: Undersampling to balance classes, then a seeded three-way split (from `llmsys/finetune.py`).

```python
def balance_by_undersampling(labels, seed=123):
    by_class = {}
    for i, y in enumerate(labels):
        by_class.setdefault(int(y), []).append(i)
    n_min = min(len(v) for v in by_class.values())
    rng = random.Random(seed)
    keep = []
    for y in sorted(by_class):
        idx = by_class[y]
        keep.extend(idx if len(idx) == n_min else rng.sample(idx, n_min))
    rng.shuffle(keep)
    return keep

def split_indices(n, train_frac=0.7, val_frac=0.1, seed=123):
    idx = list(range(n))
    random.Random(seed).shuffle(idx)
    a = int(n * train_frac)
    b = a + int(n * val_frac)
    return idx[:a], idx[a:b], idx[b:]       # test = the remainder
```

Splitting. We shuffle once with a fixed seed and cut the balanced set 70/10/20 into training (1,045), validation (149) and test (300) examples. Labels are mapped to integers, `ham`$\to 0$ and `spam`$\to 1$, because `cross_entropy` expects class indices.

Pitfall: Leakage through duplicates

SMS corpora contain many exact and near-exact duplicates, especially among spam campaigns that send the same template to thousands of numbers. If a message appears in both training and test, test accuracy measures memorisation rather than generalisation.

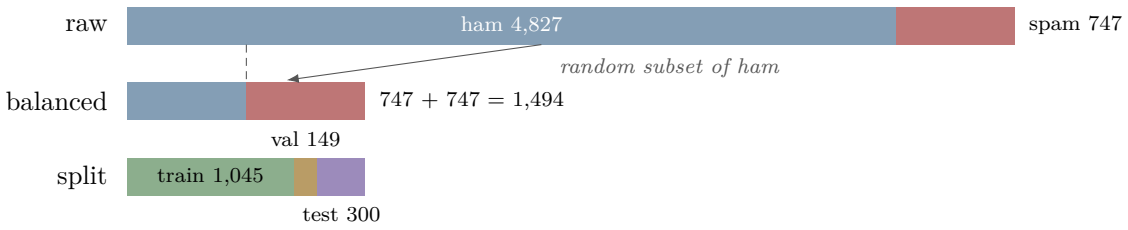


Figure 9.3: From the raw SMS collection to a balanced, three-way split, drawn to scale. Undersampling discards about 85% of the ham messages.

Deduplicate *before* splitting: hash a normalised form (lowercased, whitespace collapsed, digits masked) and keep one copy per hash. The same discipline applies to the much larger contamination problem in LLM evaluation (Chapter 14).

9.3 Creating data loaders

LLM 6.3

Text messages vary in length, but a tensor is a rectangle. The SMS messages, tokenised with the GPT-2 BPE tokenizer (Chapter 3), run from a handful of tokens to well over a hundred. There are two simple ways to make them rectangular:

1. **Truncate** every message to the length of the shortest one. This is cheap but destroys information, because most spam cues sit beyond the first few tokens.
2. **Pad** every message with a filler token up to some length $T_{\max}$. Since GPT-2 has no dedicated padding token, we reuse `<|endoftext|>` (id 50256). That is safe as long as the model never reads the hidden states at padded positions; see Section 9.5.

The simplest padding policy, and the one used in the LLM book, fixes $T_{\max}$ to the longest training message (120 tokens for this dataset) and pads the validation and test sets to the same length, truncating anything longer. Every batch then has the same shape, which is convenient for debugging and for compilers that specialise on shapes.

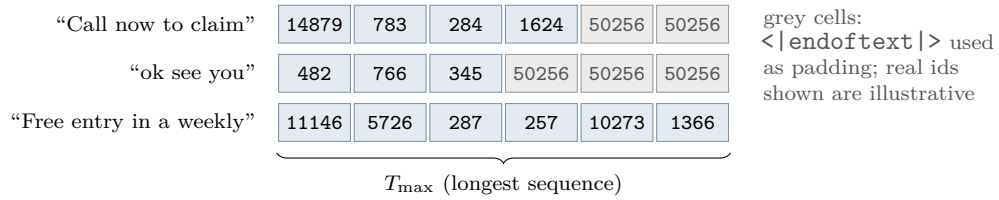


Figure 9.4: Tokenise, then pad to a common length. Every grey cell costs as much compute as a real token and, in attention, adds a row and a column to the score matrix.

Dynamic padding. A better default is to pad each *batch* to its own longest member. The dataset stores variable-length token lists, and the collate function pads them when the batch is assembled. It also returns the true lengths, which we will need to find the last real token.

Listing 9.2: A dataset that stores token lists, and a collate function that pads per batch and returns lengths.

```
class ClassificationDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=None):
```

```
        self.encoded = [tokenizer.encode(t)[:max_length] if max_length
                        else tokenizer.encode(t) for t in texts]
        self.labels = [int(y) for y in labels]
    def __len__(self):
        return len(self.labels)
    def __getitem__(self, i):
        return self.encoded[i], self.labels[i]

def pad_batch(seqs, pad_id, side="right", max_length=None):
    if max_length is not None:
        seqs = [list(s)[:max_length] for s in seqs]
    T = max(len(s) for s in seqs)
    ids = torch.full((len(seqs), T), pad_id, dtype=torch.long)
    for i, s in enumerate(seqs):
        t = torch.as_tensor(list(s), dtype=torch.long)
        if side == "right": ids[i, :len(s)] = t
        else:               ids[i, T - len(s):] = t
    return ids, torch.tensor([len(s) for s in seqs])

def classification_collate(batch, pad_id, side="right"):
    seqs, labels = zip(*batch)
    ids, lengths = pad_batch(seqs, pad_id, side)
    return ids, lengths, torch.tensor(labels)
```

We wrap the three splits in `DataLoaders` with batch size 8, shuffling and `drop_last=True` for training only. Dropping the last incomplete batch keeps every optimisation step at the same batch size. Validation and test keep all their examples, so every example is scored. A quick sanity check is to iterate once and print the shapes: with batch size 8 and global padding, input batches are 8×120 and label batches are 8. With dynamic padding the second dimension varies from batch to batch.

Systems Lens: The real price of padding

Padding costs more than the extra tokens suggest. For a batch of B sequences padded to T, the MLP and projection FLOPs scale with BT, while attention scores scale with BT^2. A single 120-token outlier in a batch of 10-token messages makes that batch about 12× more expensive in the linear layers and about 144× more expensive in the attention score matrix. Two levers fix this:

- **Length bucketing.** Group sequences of similar length into the same batch. Sorting the whole dataset by length destroys the randomness SGD relies on. A standard compromise is to shuffle, cut the data into *mega-batches* of, say, 50× the batch size, sort within each mega-batch, slice into batches, and shuffle the batch order (Listing 9.3).
- **Token-budget batching.** Fix the number of *tokens* per batch instead of the number of sequences, so batches of short messages hold more of them. This keeps GPU utilisation and memory use flat. Large-scale training and serving systems almost always batch by tokens.

The next step after bucketing is *packing* (Section 10.3.2), which removes padding completely by concatenating sequences and masking attention between them.

Listing 9.3: Sort-within-mega-batch bucketing and a padding-waste meter.

```
def length_bucketed_batches(lengths, batch_size, seed=0, bucket_mult=50):
    rng = random.Random(seed)
    idx = list(range(len(lengths)))
    rng.shuffle(idx)
    mega = batch_size * bucket_mult
    batches = []
    for s in range(0, len(idx), mega):
        chunk = sorted(idx[s:s + mega], key=lambda i: lengths[i])
        batches.extend(chunk[j:j + batch_size] for j in range(0, len(chunk),
    batch_size))
    rng.shuffle(batches)                    # randomise batch order
    return batches

def padding_stats(lengths, batches):
    real = slots = 0
    for b in batches:
        L = [lengths[i] for i in b]
        real += sum(L)
        slots += max(L) * len(L)
    return dict(real_tokens=real, padded_slots=slots, waste=1 - real / max(slots, 1)
    )
```

Figure 9.5 shows what bucketing buys. We generated 4,000 synthetic message lengths with a log-normal distribution (median 13 tokens, maximum 108), similar in shape to real SMS data, and measured the fraction of computed token slots that are padding. Padding everything to the global maximum wastes 86% of all slots. Dynamic padding with random batches wastes 40–76%, rising with batch size because a larger batch is more likely to contain an outlier. Bucketing keeps waste below 10% at every batch size.

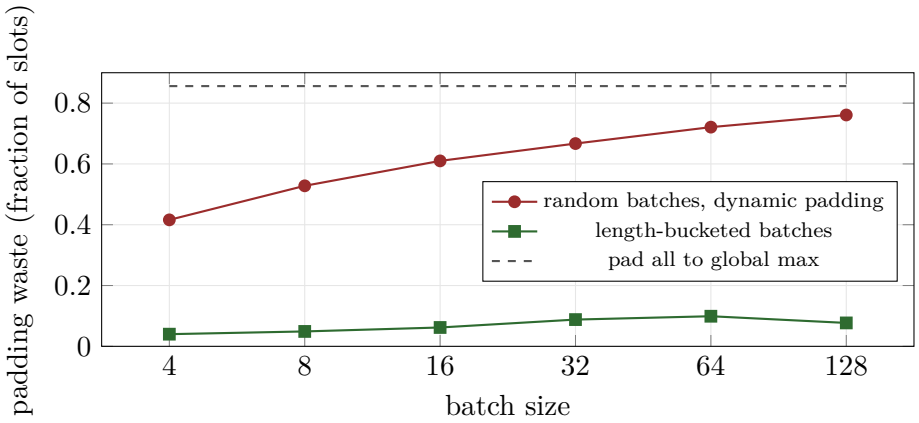


Figure 9.5: Padding waste versus batch size, measured in this environment on 4,000 synthetic log-normal message lengths (`labs/lab09_classify`). Bucketing with mega-batches of 50× the batch size removes nearly all of the waste.

9.4 Initializing a model with pretrained weights

LLM 6.4

We start from GPT-2 small (124M) and load the released weights into the `GPTModel` of Chapter 5 with `load_gpt2_hf_state_dict` (Chapter 7). Two settings must match the checkpoint: the

released model has biases on the query/key/value projections (`qkv_bias=True`), and it uses a 1,024-token context. We also set dropout to zero, since we will not fine-tune long enough for regularisation to matter.

Listing 9.4: Loading GPT-2 small for fine-tuning.

```
from safetensors.torch import load_file
from llmsys.gpt import GPTModel, gpt2_config, load_gpt2_hf_state_dict

cfg = gpt2_config("gpt2-small (124M)", drop_rate=0.0)   # qkv_bias=True, ctx 1024
model = GPTModel(cfg)
load_gpt2_hf_state_dict(model, load_file("gpt2/model.safetensors"))
model.eval()
```

Before changing anything, run two sanity checks. First, generate a continuation of a neutral prompt (“Every effort moves you”) with greedy decoding and confirm the text is fluent English. A garbled continuation means the weight mapping is wrong, most often because a Conv1D weight was not transposed. Second, see whether the base model can already classify when prompted:

```
Is the following text 'spam'? Answer with 'yes' or 'no': 'You are a winner you
have been specially selected to receive $1000 cash or a $2000 award.'
```

A base GPT-2 typically repeats the question or wanders off instead of answering. It was never trained to follow instructions (Chapter 10). That is the motivation for replacing the output layer and training it directly.

Key Idea

Fine-tuning works because the pretrained trunk already produces hidden states that encode much of what a classifier needs: topic, register, sentiment, and the presence of URLs, prize language or urgency cues. The new head only has to find a linear boundary in that representation space. Unfreezing the last block lets the features themselves bend slightly toward the task.

9.4.1 Where does the task information live?

Before deciding how many blocks to unfreeze, it helps to measure how linearly separable the classes already are at each depth of the frozen network. A *linear probe* trains only a logistic-regression layer on the last-real-token hidden state taken after block ℓ, for each ℓ. Forward hooks collect all twelve representations in a single pass over the data:

Listing 9.5: Collecting per-layer last-token features with forward hooks.

```
@torch.no_grad()
def collect_layer_features(model, loader, device="cpu"):
    feats = {i: [] for i in range(len(model.trf_blocks))}
    current = {}
    hooks = [blk.register_forward_hook(
                 lambda m, inp, out, i=i: current.__setitem__(i, out))
             for i, blk in enumerate(model.trf_blocks)]
    labels = []
    for ids, lengths, y in loader:
        model(ids.to(device))
```

```
        rows = torch.arange(ids.shape[0])
        for i, h in current.items():
            feats[i].append(h[rows, lengths - 1].float().cpu())  # (B, d)
        labels.append(y)
    for h in hooks:
        h.remove()
    return {i: torch.cat(v) for i, v in feats.items()}, torch.cat(labels)
```

The resulting curve of probe accuracy against depth is a cheap guide to where to cut the network. If accuracy is already high at block 8, you can drop blocks 9–12 entirely for this task, which cuts serving cost by a third. Or you can freeze through block 8 and cache its outputs (Section 9.5). The memory cost of probing is the number of examples $\times$ layers $\times$ d floats: $1{,}045 \times 12 \times 768 \times 4$ bytes ≈ 39 MB, small because only one position per message is kept. Exercise 9.13 asks you to produce the curve for GPT-2 small on the SMS data.

9.5 Adding a classification head

LLM 6.5

The pretrained `out_head` maps the final 768-dimensional hidden state to 50,257 vocabulary logits. We replace it with `nn.Linear(768, 2)`: one logit for `ham` and one for `spam`. With two classes we could get away with a single logit and a sigmoid. Keeping C logits and the softmax cross-entropy loss means the same code handles any number of classes.

Which layers to train. Lower transformer layers in pretrained LMs tend to encode general lexical and syntactic features that transfer across tasks. Upper layers are more specialised to next-token prediction. A common, cheap recipe is therefore:

1. freeze every parameter;
2. replace `out_head` (new parameters are trainable by default);
3. unfreeze the last transformer block and the final LayerNorm.

Training more blocks usually helps a little and costs more; training all of them is full fine-tuning. Exercise 9.2 asks you to measure the trade-off.

Listing 9.6: Swapping the head and choosing what to train.

```
def add_classification_head(model, num_classes, trainable_blocks=1,
                            train_final_norm=True):
    for p in model.parameters():
        p.requires_grad = False
    emb_dim = model.out_head.in_features
    model.out_head = nn.Linear(emb_dim, num_classes)     # trainable
    if trainable_blocks > 0:
        for blk in model.trf_blocks[-trainable_blocks:]:
            for p in blk.parameters():
                p.requires_grad = True
    if train_final_norm:
        for p in model.final_norm.parameters():
            p.requires_grad = True
    return model
```

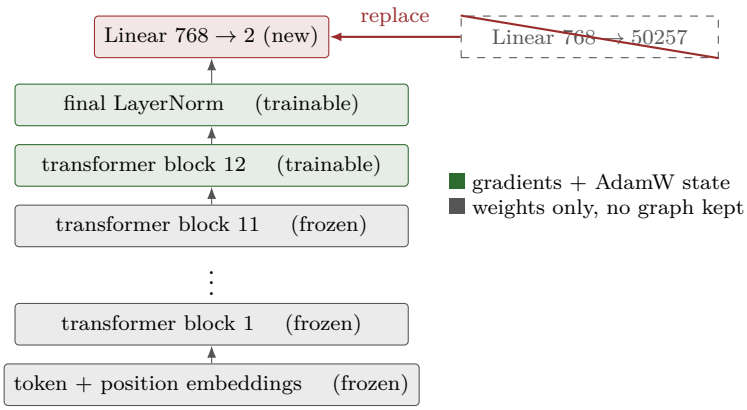


Figure 9.6: The classifier: a frozen GPT-2 trunk, a trainable top block and final norm, and a new two-way head in place of the vocabulary projection.

With this recipe, 7,090,946 of the model's parameters are trainable: the last block (7,087,872), the final norm (1,536) and the head ($768 \times 2 + 2 = 1{,}538$). That is 5.7% of the network.

Systems Lens: Freezing saves backward compute and activation memory, not just optimizer state

Autograd records a node only for operations whose inputs require gradients. With blocks 1–11 frozen, their inputs (the embedding outputs) do not require gradients, so no graph is recorded for them and their intermediate activations are freed right after the forward pass. Backward stops at the input of block 12. Two consequences:

- **Compute.** A training step costs roughly $2N$ FLOPs per token forward plus $4N_{\text{trainable}}$ backward, instead of $6N$. With one of twelve blocks trainable, that is about $(2 + 4/12)/6 \approx 0.39$ of a full fine-tuning step, or 2.6× cheaper.
- **Trunk caching.** If the bottom 11 blocks never change, their output for a given message never changes either. You can run them once over the dataset, store the $T \times 768$ hidden states, and train only block 12 and the head on the cache. The stored hidden states fit easily: 1,045 messages × 120 positions × 768 dimensions × 2 bytes ≈ 193 MB. This is a big win when you sweep hyperparameters over many epochs.

LoRA (Section 10.10) is a different case. Adapters sit in *every* layer, so the activation gradient must flow through the whole network, and the savings come only from skipping the weight-gradient GEMMs of the frozen weights.

9.5.1 Which token should the head read?

The model returns logits of shape (B, T, C), one C-vector per position. We need one vector per message. Recall the causal mask from Chapter 4: position t can attend only to positions $0, \ldots, t$. Only the *last* position has attended to the whole message, so its hidden state is the natural summary to classify. The first position has seen a single token.

With *right* padding, the last position of a padded row is a padding token, not the last real token. The LLM book reads `logits[:, -1, :]` anyway. That works reasonably well, because the padding token at the end still attends to every real token before it. But it makes the prediction depend on *how much* padding follows the message, which in turn depends on which other messages happen to share the batch. Two cleaner options:



Figure 9.7: Causal attention over a six-token message. Only the final position aggregates information from the whole sequence, so its output is the one we classify.

1. **Gather the last real token.** With the lengths returned by the collate function, pick row b at position $\ell_b - 1$. The prediction is then exactly invariant to padding (the lab tests this).

2. **Left padding with a key-padding mask.** Put the padding before the message and mask padded keys (the `pad_mask` argument of the core models). The last position is now always real, so `logits[:, -1]` is correct. This is what batched generation does (Chapter 12). With learned absolute positions (GPT-2) the real tokens shift to higher position ids, which changes the outputs slightly; with RoPE it does not.

Listing 9.7: Selecting the per-message logits.

```python
def select_class_logits(logits, lengths=None, side="right"):
    """logits: (B, T, C) -> (B, C)."""
    if lengths is None or side == "left":
        return logits[:, -1, :]
    idx = (lengths - 1).clamp(min=0).to(logits.device)
    return logits[torch.arange(logits.shape[0], device=logits.device), idx]
```

Deeper Dive: Other pooling choices

Encoder models such as BERT classify from a dedicated `[CLS]` token placed at the *start*. That works only because encoders use bidirectional attention. In a causal decoder you can also mean-pool the hidden states of the real tokens. This is sometimes more robust for long inputs, but it dilutes the strongest signal, which tends to sit at the end. Embedding models built from decoders (e5-mistral-style) typically use last-token pooling with an appended end-of-sequence token, which is the recipe of this chapter.

9.6 Calculating the classification loss and accuracy

LLM 6.6

For a message x with selected logits $\boldsymbol{z} \in \mathbb{R}^C$, the predicted class distribution is $p(c \mid x) = \mathrm{softmax}(\boldsymbol{z})_c$. The predicted label is $\hat{y} = \arg\max_c z_c$. Softmax is monotonic, so we can take the arg max of the logits directly. Training minimises the mean cross-entropy over a batch,

$$\mathcal{L} = -\frac{1}{B}\sum_{b=1}^{B} \log \mathrm{softmax}(\boldsymbol{z}_b)_{y_b} = \frac{1}{B}\sum_{b=1}^{B}\Big(\log \sum_{c} e^{z_{b,c}} - z_{b,y_b}\Big), \tag{9.1}$$

the same loss as in pretraining (Chapter 7). It is applied once per message instead of once per token, over $C = 2$ classes instead of 50,257. We cannot optimise accuracy directly because it is piecewise constant, so its gradient is zero almost everywhere. Cross-entropy is a smooth surrogate for it.

Listing 9.8: Loss on a batch and accuracy over a loader.

```
def classification_loss(model, ids, lengths, labels, device="cpu", use_lengths=True):
    ids, labels = ids.to(device), labels.to(device)
    logits = select_class_logits(model(ids), lengths if use_lengths else None)
    return F.cross_entropy(logits.float(), labels)

@torch.no_grad()
def accuracy_loader(model, loader, device="cpu", num_batches=None, use_lengths=True):
    model.eval()
    correct = total = 0
    for i, (ids, lengths, labels) in enumerate(loader):
        if num_batches is not None and i >= num_batches:
            break
        logits = select_class_logits(model(ids.to(device)),
                                     lengths if use_lengths else None)
        correct += (logits.argmax(-1).cpu() == labels).sum().item()
        total += labels.numel()
    return correct / max(total, 1)
```

Evaluated before any fine-tuning, the new head is random, so accuracy on the balanced splits sits near 50%, which is chance. The `num_batches` argument estimates accuracy from a few batches, which is useful for frequent cheap checks during training.

Pitfall: Computing the loss in low precision

Under bf16 autocast the logits come out in bf16, which has only 8 bits of mantissa. A log-sum-exp over large vocabularies loses accuracy in that format. With two classes the damage is small, but the habit of casting logits to fp32 (`logits.float()`) before `cross_entropy` costs nothing here and matters a lot in Chapters 7 and 17.

9.7 Fine-tuning the model on supervised data

LLM 6.7

The training loop is the pretraining loop of Chapter 7 with three changes: the loss is computed on the selected logits, only trainable parameters are passed to the optimizer, and we track accuracy alongside loss. Following the LLM book we use AdamW with learning rate 5×10^{-5} and weight decay 0.1 for about five epochs. On a modern laptop CPU this takes several minutes for GPT-2 small; on a GPU, well under a minute.

Listing 9.9: Classification fine-tuning loop (lab reference).

```
def train_classifier(model, train_loader, val_loader, epochs=3, lr=5e-4,
                     weight_decay=0.1, device="cpu", use_lengths=True, log=print):
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=weight_decay)
    history = []
    for ep in range(epochs):
```

```
        model.train()
        total, n, correct, seen = 0.0, 0, 0, 0
        for ids, lengths, labels in train_loader:
            opt.zero_grad(set_to_none=True)
            logits = select_class_logits(model(ids.to(device)),
                                         lengths if use_lengths else None)
            loss = F.cross_entropy(logits.float(), labels.to(device))
            loss.backward()
            opt.step()
            total += loss.item(); n += 1
            correct += (logits.argmax(-1).cpu() == labels).sum().item()
            seen += len(labels)
        va_acc = accuracy_loader(model, val_loader, device, use_lengths=use_lengths)
        history.append(dict(epoch=ep + 1, loss=total / n,
                            train_acc=correct / seen, val_acc=va_acc))
    return history
```

Two details are worth noticing. First, the training accuracy is accumulated from forward passes we already pay for, instead of a second pass over the training set. That pass would cost as much as a full extra epoch of forward compute. The number is slightly stale (it mixes predictions from the start and end of the epoch), which is fine for monitoring. Second, `loss.item()` forces a host–device synchronisation on every step. On a GPU, it is better to accumulate the loss in a device tensor and read it once per logging interval (Chapter 2).

Figure 9.8 shows a training run on the lab's synthetic spam task: a 2-layer, 64-dimensional GPT trained from scratch, measured on CPU in this environment. The shape is what you will see with real GPT-2 on real SMS data. The loss falls quickly, training accuracy saturates near 100%, and validation accuracy plateaus a few points lower. With a pretrained GPT-2 on the SMS data, the LLM book reports training and validation accuracies in the high 90s and test accuracy in the mid 90s.

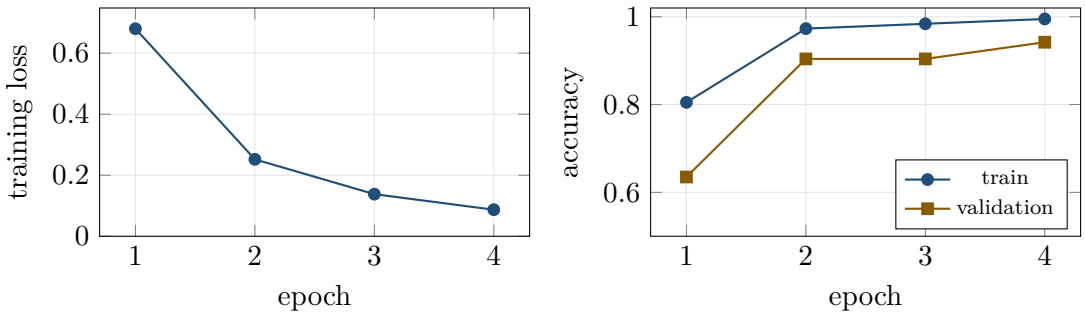


Figure 9.8: Loss and accuracy per epoch on the synthetic spam task of Lab 9 (600 messages, 40% spam before balancing, tiny GPT, AdamW at 2×10^{-3}, length-bucketed batches of 16). Measured on CPU in this environment.

Reading the curves. If validation accuracy lags training accuracy by more than a few points and the gap keeps widening, the model is memorising. The usual fixes, in rough order of cost, are fewer epochs (early stopping on validation accuracy), higher weight decay, dropout, more data, and training fewer layers. If both curves stall near 50%, the usual causes are (i) the head is reading padding positions while the padding amount is correlated with the label, (ii) the learning rate is too low for a freshly initialised head, or (iii) the trainable flags are wrong. Print

`sum(p.numel() for p in model.parameters() if p.requires_grad)` to check.

Napkin Math: How long should this take on a GPU?

Each training message costs about $2N + 4N_{\text{tr}}$ FLOPs per token (Section 9.5), where N counts only the non-embedding weights that do work. For GPT-2 small, $N \approx 85\text{M}$ and $N_{\text{tr}} \approx 7.1\text{M}$, giving $\approx 1.98 \times 10^8$ FLOPs per token. Five epochs over 1,045 messages padded to 120 tokens is 6.3×10^5 token-positions, or about 1.2×10^{14} FLOPs. At a pessimistic 20% of an A100's 312 TFLOP/s, that is two seconds of arithmetic. In practice a run takes much longer: kernel launches for small batches, Python overhead and evaluation passes dominate. This chapter's workload is *latency-bound*, not FLOP-bound. Larger batches, `torch.compile` and CUDA graphs (Chapter 11) are the relevant levers.

9.8 Using the LLM as a spam classifier

LLM 6.8

To classify a new message we tokenise it, truncate it to the model's context length (or to the training $T_{\max}$, if we trained with global padding and want identical conditions), run one forward pass, and take the arg max of the selected logits.

Listing 9.10: Inference with the fine-tuned classifier.

```python
@torch.no_grad()
def classify(model, tokenizer, text, device="cpu", max_length=None,
             labels=("not spam", "spam")):
    ctx = model.cfg["context_length"]
    ids = tokenizer.encode(text)[: min(max_length or ctx, ctx)]
    logits = model(torch.tensor([ids], device=device))[:, -1, :]   # no padding
    return labels[int(logits.argmax(-1))]

classify(model, tok, "You are a winner! Reply now to claim your free prize.")
# -> 'spam'
classify(model, tok, "Are we still on for dinner tonight?")
# -> 'not spam'

torch.save(model.state_dict(), "spam_classifier.pth")   # 2-way head included
```

To reload, rebuild the architecture *with* the 2-way head before calling `load_state_dict`. Otherwise the stored `out_head.weight` of shape $(2, 768)$ will not fit a $(50257, 768)$ parameter. Storing the full model costs about 0.5 GB in fp32. If many tasks share one frozen trunk, store only the trainable parameters per task (≈28 MB here) together with a pointer to the base checkpoint, which is the same idea as LoRA adapters in Chapter 10.

Systems Lens: Serving a classifier is a pure-prefill workload

Generative serving alternates a compute-bound *prefill* with a memory-bound *decode* loop (Chapter 11). A classifier has only the prefill. Every request is a single forward pass over its own tokens, the output is C numbers, and the KV cache is discarded immediately. Consequences:

- Throughput is governed by FLOPs, not memory bandwidth, so large batches and tensor-core-friendly shapes pay off directly.
- The vocabulary head disappears. In GPT-2 small the 768×50257 projection is 38.6M

parameters, nearly half the size of all twelve blocks. Removing it saves those FLOPs for every request.

- Batching policy is all about padding. Bucket by length, or use a varlen attention kernel that accepts a packed batch with per-sequence offsets (Section 10.3.2).
- Latency is easy to bound: there is no generation length to predict, so SLOs can be met with simple admission control.

Napkin Math: Classifier throughput on one H100

Forward FLOPs per token for GPT-2 small without the vocabulary head: $2 \times 85\text{M} = 1.7 \times 10^8$ (attention terms are negligible at 30 tokens). At 50% of the H100's 989 TFLOP/s: $4.9 \times 10^{14} / 1.7 \times 10^8 \approx 2.9$ million tokens per second. At about 30 tokens per SMS, that is roughly 97,000 messages per second from one GPU, if batches are large and padding-free. With random batching at 60% padding waste (Fig. 9.5), the same GPU delivers about 40% of that.

Deeper Dive: When not to fine-tune a 124M-parameter decoder

For a binary task with a few thousand examples, a linear probe on frozen sentence embeddings, or a small encoder (DistilBERT/ModernBERT-class), is often as accurate and 10–100× cheaper to serve. A decoder LLM classifier makes most sense when (i) you already serve that trunk and can share it, (ii) the inputs are long or need world knowledge, or (iii) you will later turn the same model into an instruction follower. An LLM can also act as a *zero-shot* classifier by scoring the log-probabilities of label words. Chapter 16 develops this kind of probability-based scoring.

9.9 From one classifier to a fleet

The LLM book stops once the spam classifier works. In production a single classifier is rarely the whole story: you need to decide where to put the decision threshold, you may run several classifiers on every request, and you will want an expensive model to see only the inputs the cheap one is unsure about. This section collects the systems patterns that follow from that.

9.9.1 Metrics, thresholds and calibration

The arg max rule implicitly thresholds $p(\text{spam} \mid x)$ at 0.5. The costs of the two error types are rarely equal. Hiding a real message from a user (a false positive) is usually worse than letting a spam through (a false negative). With $\text{TP}, \text{FP}, \text{FN}, \text{TN}$ counted at a threshold τ,

$$\text{precision}(\tau) = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall}(\tau) = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \tag{9.2}$$

Raising τ trades recall for precision. The operating point is chosen on the *validation* split, for example the smallest τ whose validation precision is at least 99%, and then reported once on the test split.

A threshold only means something if the probabilities are *calibrated*: among messages scored 0.9, about 90% should really be spam. Fine-tuned networks are often over-confident. A standard check is the expected calibration error. Bin the n test predictions by confidence into bins

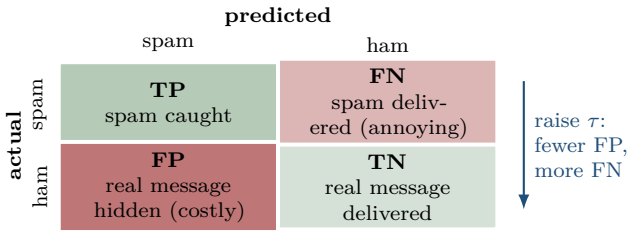


Figure 9.9: The confusion matrix with the predicted class as columns. The two error cells cost different amounts, so the threshold τ on $p(\text{spam} \mid x)$ is a product decision, not a modelling default.

$B_1, \ldots, B_M$ and compute

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|. \tag{9.3}$$

Temperature scaling (Guo et al., 2017) is the cheapest fix. It divides the logits by a single scalar T fitted on the validation set to minimise the negative log-likelihood. Because it does not change the arg max, accuracy is untouched. This is the same temperature that controls sampling diversity in Chapters 7 and 15, used here for the opposite purpose of making probabilities honest.

Listing 9.11: Fitting a calibration temperature on cached validation logits.

```
def fit_temperature(val_logits, val_labels, steps=200, lr=0.01):
    # val_logits: (n, C), collected once under torch.no_grad()
    log_t = torch.zeros(1, requires_grad=True)          # T = exp(log_t) > 0
    opt = torch.optim.Adam([log_t], lr=lr)
    for _ in range(steps):
        loss = F.cross_entropy(val_logits / log_t.exp(), val_labels)
        opt.zero_grad(); loss.backward(); opt.step()
    return log_t.exp().item()
```

Note the systems shape of this step. We run the expensive model *once* over the validation set, cache an $n \times C$ matrix of logits, and fit on that. The fit takes milliseconds. The same pattern of caching model outputs and tuning cheap post-processing on top appears again with verifiers and scorers in Chapters 14 and 16.

9.9.2 Cascades: spend big-model compute only on hard inputs

Suppose a large instruction-tuned model (Chapter 10) classifies more accurately than our fine-tuned GPT-2, but costs R times more per token. A *cascade* runs the small classifier on everything and forwards a request to the large model only when the small model's calibrated confidence $\max_c p(c \mid x)$ is below a threshold. If a fraction e of traffic is escalated, the expected cost per request in units of the small model is

$$\text{cost}(e) = 1 + e\,R, \qquad \frac{\text{cost}(e)}{\text{cost of large only}} = \frac{1 + eR}{R}. \tag{9.4}$$

Calibration matters here: with an over-confident small model, the escalated set misses exactly the inputs it gets wrong.

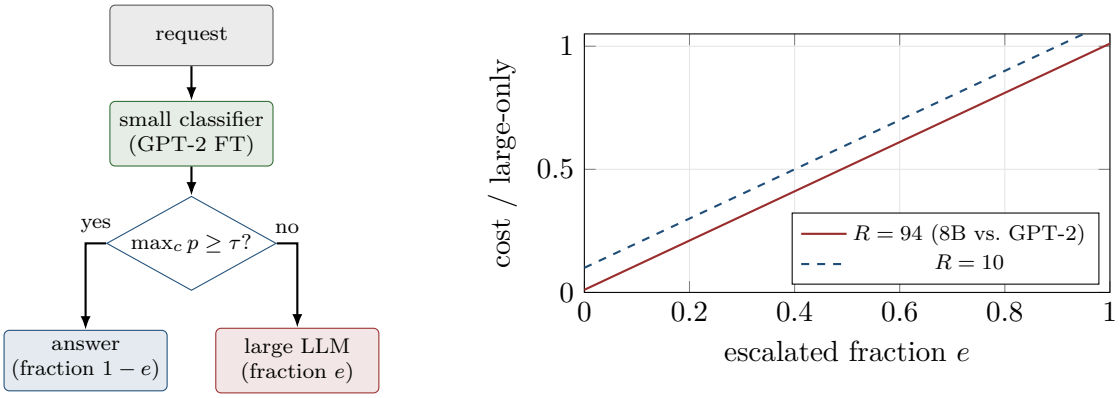


Figure 9.10: Left: a two-stage classifier cascade. Right: the relative cost from Eq. (9.4). When the large model is $R \approx 94\times$ more expensive per token (an 8B-parameter model versus GPT-2 small's 85M non-embedding parameters, prefill only), escalating 5% of traffic costs about 6% of the large-only budget.

Napkin Math: Pricing a cascade

Prefill FLOPs per token scale with non-embedding parameters: $2 \times 85\text{M}$ for GPT-2 small versus about $2 \times 8\text{B}$ for an 8B model, so $R \approx 94$. If the small model is confident on 95% of messages, cost $= 1 + 0.05 \times 94 = 5.7$ small-model units, against 94 for the large model alone: a $16\times$ saving. And if the large model must *generate* a verdict instead of scoring label tokens in a single prefill, R grows further, because decoding is memory-bound (Chapter 11).

9.9.3 One trunk, many heads

Platforms often run several classifiers over every message: spam, abuse, language, intent, PII. If they all fine-tune only the top of the same base model, the frozen bottom of the network computes *identical* activations for each of them. Sharing that computation is pure profit.

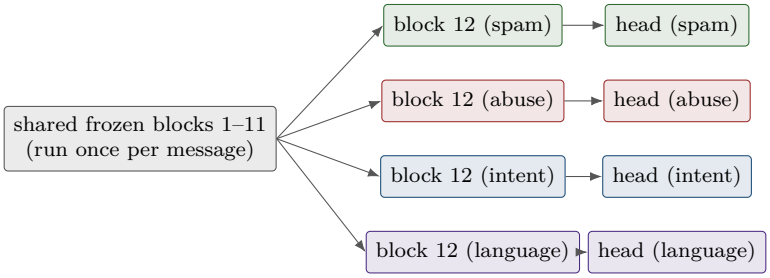


Figure 9.11: Serving several fine-tuned classifiers that share a frozen trunk. Each task owns only its top block and head (about 14 MB in bf16 for GPT-2 small).

With K tasks that each fine-tune only the last of L blocks, separate models evaluate KL blocks per token, while a shared trunk evaluates $(L - 1) + K$. For $L = 12$ and $K = 8$ that is 96 versus 19, a $5\times$ saving in FLOPs *and* in weight memory traffic. If the tasks only train heads (`trainable_blocks=0`), the saving approaches $K\times$. LoRA gives the same economics for adapters spread through the whole network, using the batched multi-adapter kernels of Section 10.10.5.

9.9.4 Classifying with a modern decoder

Nothing in this chapter is specific to GPT-2. The Qwen3 model of Chapter 6 uses the same attribute names (`trf_blocks`, `final_norm`, `out_head`), so `add_classification_head` applies unchanged. Three differences deserve care:

- **Tied embeddings.** In Qwen3-0.6B, `out_head.weight` *is* `tok_emb.weight`. Replacing `out_head` breaks the tie, which is what we want. The input embedding stays frozen and the new head is independent. But parameter counts and checkpoint sizes change, and code that re-ties weights on load must be skipped.
- **A real padding token.** Qwen's tokenizer has distinct `<|endoftext|>` and `<|im_end|>` tokens, so padding never collides with an end-of-turn marker that the model treats as meaningful.
- **RoPE makes left padding exact.** Attention scores depend only on relative positions, so left padding plus a key mask gives the same last-token logits as an unpadded forward pass, up to floating-point summation order. The core test suite checks this property for batched generation.

Listing 9.12: The same recipe on a Qwen3-shaped model (tiny config shown).

```
from llmsys.qwen3 import Qwen3Model, QWEN3_CONFIG_TINY
from llmsys.finetune import add_classification_head, pad_batch

model = add_classification_head(Qwen3Model(QWEN3_CONFIG_TINY), num_classes=2)
ids, lengths = pad_batch([[5, 6, 7, 8], [9, 10]], pad_id=0, side="left")
pad_mask = ids != 0                                  # True for real tokens
logits = model(ids, pad_mask=pad_mask)[:, -1]    # (2, 2): last position is real
```

9.10 Summary

- Classification fine-tuning replaces the vocabulary projection with a C-way head and trains on labelled examples. It is cheaper and easier to evaluate than instruction fine-tuning, but the resulting model does only that one task.
- Data preparation matters: balance or reweight classes, deduplicate before splitting, and split with a fixed seed into train/validation/test (70/10/20 here).
- Variable-length inputs must be padded. Padding to the global maximum wastes most of the compute; dynamic padding plus length bucketing removes nearly all of it.
- Under a causal mask only the last position has seen the whole input, so the head reads that position. With right padding, gather the last *real* token, or use left padding with a key mask.
- Freezing the trunk removes optimizer state and, when the frozen layers are at the bottom, also their backward pass and stored activations. It even allows caching trunk features across epochs.
- Serving a classifier is pure prefill: compute-bound, no decode loop, no vocabulary head. Padding-aware batching is the main lever on throughput.

- Thresholds should be chosen on calibrated probabilities. Calibrated small classifiers enable cascades and shared-trunk multi-task serving, which cut cost by an order of magnitude.

9.11 Exercises

Exercise 9.1 (*Global versus dynamic padding*). Train the classifier twice: once with every split padded to the longest training message (120 tokens), and once with per-batch dynamic padding and last-real-token pooling. Compare accuracy and wall-clock time per epoch. Then pad everything to 1,024 tokens and explain what happens to time and to accuracy when the head reads position -1.

Exercise 9.2 (*How many layers to unfreeze*). Sweep `trainable_blocks`$\in \{0, 1, 2, 4, 12\}$ ($0 =$ head and final norm only). For each setting, report test accuracy, trainable parameter count, peak memory, and time per step. Predict the time per step from the $2N + 4N_{\text{tr}}$ model before measuring, and explain any discrepancy.

Exercise 9.3 (*Trunk caching*). With `trainable_blocks=1`, precompute the outputs of blocks 1–11 for the whole training set and train block 12 and the head on the cached tensors. Verify that the loss trajectory matches uncached training (dropout off). How large is the cache, and how much faster is each epoch?

Exercise 9.4 (*Class weights instead of undersampling*). Use all 5,574 messages with the original 87/13 class ratio. Compare (a) plain cross-entropy, (b) cross-entropy with class weights inversely proportional to class frequency, and (c) undersampling. Report accuracy, precision, recall and F1 for the spam class. Which metric would you put on a dashboard, and why?

Exercise 9.5 (*Padding invariance*). Prove that with right padding, causal attention, and last-real-token pooling, a message's logits do not depend on how much padding follows it. Then show with a counterexample that reading position -1 does depend on padding amount. (Hint: what does the padding position attend to?)

Exercise 9.6 (*Left padding and absolute positions*). Classify a batch with left padding and a key-padding mask using GPT-2's learned absolute position embeddings. Measure how much the logits differ from the unpadded single-message logits, and explain why. Would the result change for a RoPE model?

Exercise 9.7 (*Token-budget batching*). Implement a sampler that yields batches with at most K tokens after padding (e.g. $K = 2048$). Compare the number of batches, padding waste, and step-time variance against fixed-size bucketed batches.

Exercise 9.8 (*Zero-shot baseline*). Without any fine-tuning, classify messages by comparing $\log p(\text{“ spam”})$ and $\log p(\text{“ ham”})$ after a prompt such as “Message: ...\nCategory:”. Report accuracy on the test split. How does the gap to the fine-tuned model change with model size (GPT-2 small, medium, large)?

Exercise 9.9 (*Serving capacity*). A service must classify 20,000 messages per second (mean 35 tokens, p99 160 tokens) with p99 latency under 50 ms. Using the napkin model of Section 9.8, estimate how many L4 GPUs are needed with (a) random batching and (b) bucketing. Which assumption in your estimate is most likely to be wrong?

Exercise 9.10 (*Deduplication*). Count exact duplicates in the SMS collection, and near-duplicates after lowercasing and replacing digit runs with a placeholder. Re-split after deduplication and compare test accuracy to the original split. What does the difference tell you?

Exercise 9.11 (*Calibration*). Collect validation logits once, fit a temperature with Listing 9.11, and report ECE (15 bins) on the test split before and after. Show analytically that temperature scaling cannot change accuracy.

Exercise 9.12 (*Cascade design*). Using your calibrated classifier, plot escalation rate $e(\tau)$ and cascade accuracy against τ, assuming the large model is always right. Find the cheapest τ that reaches 99.5% accuracy and compute its cost with Eq. (9.4).

Exercise 9.13 (*Layer-wise probes*). Use Listing 9.5 to collect last-real-token features after every block of pretrained GPT-2 small on the balanced SMS splits, fit a logistic-regression probe per layer, and plot validation accuracy against depth. At which depth would you cut the network for serving, and what is the predicted FLOP saving?

9.12 Lab 9: Classification fine-tuning and padding-aware batching

Lab 9: Classification fine-tuning and padding-aware batching

Code: `code/labs/lab09_classify`

Goals

Build the full classification fine-tuning pipeline: balancing, splitting, dynamic padding, the head swap with selective freezing, padding-invariant pooling, and the training loop. Then measure how much compute padding wastes and how much bucketing recovers.

Setup

`cd code; LAB_IMPL=starter python -m pytest -q labs/lab09_classify`. The tests run on CPU in under a minute using a tiny GPT and a synthetic spam task (`make_synthetic_spam`: "spam" messages contain marker tokens, and lengths are log-normal). `run.py` repeats the experiment with GPT-2 small on the real SMS Spam Collection on your own machine.

Tasks

1. `balance_by_undersampling`, `split_indices`: seeded, exact sizes, every index exactly once.
2. `pad_batch`: left/right padding, optional truncation, returns lengths.
3. `add_classification_head`: freeze everything, swap the head, unfreeze the last k blocks and the final norm.

4. `select_class_logits` and `accuracy_loader`: last-real-token pooling. The test checks that predictions do not change when more padding follows a message.
5. `length_bucketed_batches` and `padding_stats`: sort-within-mega-batch bucketing and a waste meter. The test requires bucketing to cut waste by at least $3\times$.
6. `train_classifier`: AdamW over trainable parameters, training accuracy accumulated for free, validation accuracy per epoch. The test requires $> 80\%$ test accuracy on the synthetic task.

Measurements

- Reproduce Fig. 9.5 for your own length distribution (synthetic and, if you have it, the real SMS data with the GPT-2 tokenizer).
- With `run.py` on a GPU: time per epoch and peak memory for `trainable_blocks`$\in \{1, 12\}$, with and without `-bucketed`. Expect bucketing to cut epoch time roughly in proportion to the waste reduction once batches are large enough to be compute-bound. Expect `trainable_blocks=1` to be roughly 2–3$\times$ faster than full fine-tuning per step.
- Accuracy before and after fine-tuning: near 50% before, mid-to-high 90s after on real SMS data.

Deliverables

Passing tests; a short table (accuracy, time/epoch, peak memory, trainable params) for the configurations above; one paragraph explaining the measured speed-up from freezing in terms of forward and backward FLOPs.

Stretch goals

- Implement trunk caching and token-budget batching (Exercises 9.3 and 9.7).
- Serve the classifier behind a minimal HTTP endpoint with a micro-batching queue: collect requests for up to 5 ms or 64 messages, bucket by length, and run one forward pass. Plot throughput against p99 latency as the batching window varies.

Chapter 10

Instruction Fine-Tuning and Parameter-Efficient Adaptation

A pretrained model continues text; an assistant answers requests. If you ask base GPT-2 to "Convert 45 kilometers to meters," it is just as likely to continue with a second unit-conversion question as to answer. *Instruction fine-tuning*, also called supervised fine-tuning (SFT), closes that gap. We train the model with the ordinary next-token objective on examples written as a prompt followed by the desired response, so the most likely continuation of a prompt becomes its answer.

Algorithmically, SFT is pretraining with differently formatted data. The details are in the data pipeline: how examples are templated, how variable-length sequences become rectangular batches, which positions contribute to the loss, and how to pack many short examples into one long row without letting them see each other. These details decide whether SFT is correct and whether it is efficient, and they are the same machinery behind the chat models served in Chapters 11 and 12. The second half of the chapter turns to *which* parameters to train. Low-rank adaptation (LoRA) and its quantised variant QLoRA make fine-tuning a multi-billion-parameter model fit on one GPU. They also make it possible to serve hundreds of task-specific variants from one base model.

10.1 Introduction to instruction fine-tuning

LLM 7.1

Pretraining (Chapter 7) teaches a model the statistics of web text. Web text contains questions followed by answers, but also questions followed by more questions, forum signatures, and navigation menus. The pretrained model has no notion that a request should be *answered*. SFT shows it thousands of examples where it is.

Three facts frame the rest of the chapter.

1. **SFT needs surprisingly little data.** Strong results have come from tens of thousands of examples, and in the LIMA study from about a thousand carefully curated ones (Zhou et al., 2023). This is consistent with the "superficial alignment" hypothesis: the knowledge comes from pretraining, and SFT mostly teaches format and behaviour. Our dataset has 1,100 examples.

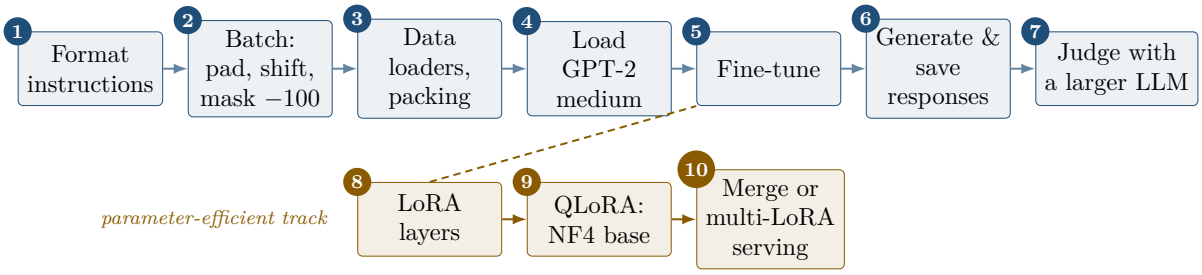


Figure 10.1: Chapter roadmap. Stages 1–7 build the instruction-tuning pipeline end to end. Stages 8–10 replace full fine-tuning with low-rank adapters, quantise the frozen base, and show how adapted models are deployed.

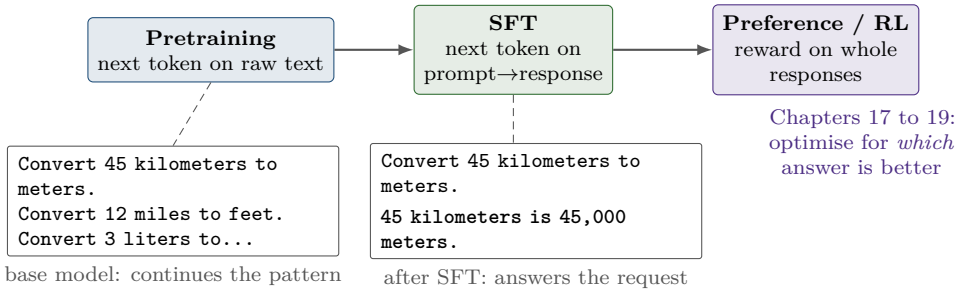


Figure 10.2: Where SFT sits. The same next-token loss is used throughout; what changes is the data, and later the objective. Example outputs are illustrative.

2. **The model must learn to stop.** Every training response ends with an end-of-text token. Without it, a fine-tuned model answers and then keeps generating another instruction. This is why the collate function in Section 10.3 treats the first end-of-text token differently from the rest of the padding.

3. **Evaluation is the hard part.** Unlike classification, there is no single correct string, so we will use a larger model as a judge (Section 10.8), previewing the evaluation machinery of Chapter 14.

10.2 Preparing a dataset for supervised instruction fine-tuning

LLM 7.2

Our dataset is a JSON list of 1,100 records in the style popularised by Stanford Alpaca. Each record has an `instruction`, an optional `input` that provides context, and the reference `output`:

```
{"instruction": "Rewrite the sentence in passive voice.",
 "input": "The committee approved the budget.",
 "output": "The budget was approved by the committee."}
{"instruction": "What is the boiling point of water at sea level in Celsius?",
 "input": "",
 "output": "The boiling point of water at sea level is 100 degrees Celsius."}
```

The tasks are short and varied: grammar fixes, unit conversions, definitions, antonyms, simple factual questions. That makes them a good test of instruction following on a 355M-parameter model that trains in minutes.

10.2.1 Prompt templates

Before tokenisation, every record is rendered into one string by a *prompt template*. The template defines the boundaries the model will learn: where the request ends and the answer begins. Figure 10.3 compares two common styles. The *Alpaca* style uses a fixed preamble and Markdown-like section headers. The *Phi-3* style uses compact role tags. Modern chat models use structured templates of the second kind with dedicated special tokens (Qwen's ChatML, `<|im_start|>user ...<|im_end|>`, Chapter 11). Those tokens cannot appear in user text by accident, which matters for both correctness and prompt-injection resistance.

Alpaca style

```
Below is an instruction that describes a
task. Write a response that appropriately
completes the request.

### Instruction:
Convert the temperature to Fahrenheit.

### Input:
25 degrees Celsius

### Response:
25 degrees Celsius is 77 degrees Fahrenheit.
```

Phi-3 style

```
<|user|>
Convert the temperature to Fahrenheit.
25 degrees Celsius
<|assistant|>
25 degrees Celsius is 77 degrees Fahrenheit.
```

Fewer template tokens per example ⇒ shorter sequences, faster training and inference, and more room for content in a fixed context window.

Figure 10.3: The same record rendered with two prompt templates. The response section is what we ultimately want the model to produce; everything before it is the prompt.

Listing 10.1: Rendering a record with the Alpaca template (from `llmsys/finetune.py`).

```python
def format_alpaca(entry, with_response=False):
    text = ("Below is an instruction that describes a task. "
            "Write a response that appropriately completes the request."
            f"\n\n### Instruction:\n{entry['instruction']}")
    if entry.get("input"):                          # optional section
        text += f"\n\n### Input:\n{entry['input']}"
    text += "\n\n### Response:\n"
    if with_response:
        text += entry["output"]
    return text
```

The function returns the prompt alone (used at inference time) or prompt plus response (used for training). Keeping one function for both prevents a subtle bug: a template that differs by a single newline between training and inference measurably degrades a fine-tuned model.

Napkin Math: What the template costs

The Alpaca preamble and headers add roughly 30 GPT-2 tokens to every example. With responses of about 20–60 tokens and instructions of about 10–30, the template is roughly a third of all tokens processed. Training and evaluating with a compact template cuts compute by about that fraction. In attention, where cost grows with T^2, the saving is larger. The template also takes up context-window space that real content could use.

10.2.2 Splitting

We shuffle with a fixed seed and split 85% / 10% / 5% into training (935), test (110) and validation (55) sets. The test set is larger than the validation set because we will spend real effort on it (generation plus LLM judging). Validation is used only to monitor loss.

10.3 Organizing data into training batches

LLM 7.3

In Chapter 9, the collate function only padded inputs. For SFT it has to do more work. Figure 10.4 shows the steps:

1. **Format and tokenise** each record (done once, in the dataset constructor).
2. **Append one end-of-text token** (id 50256 for GPT-2) to every sequence, so the model learns to stop.
3. **Pad** every sequence in the batch to the length of the longest one plus one, using the same id 50256.
4. **Shift**: inputs are positions $0..T-1$ of the padded row and targets are positions $1..T$. This is exactly the shift used in pretraining.
5. **Mask padding in the targets**: every target position after the first end-of-text token is replaced by -100, PyTorch's default `ignore_index`, so the loss ignores it. The *first* end-of-text token stays a real target, because predicting it is how the model learns to stop.

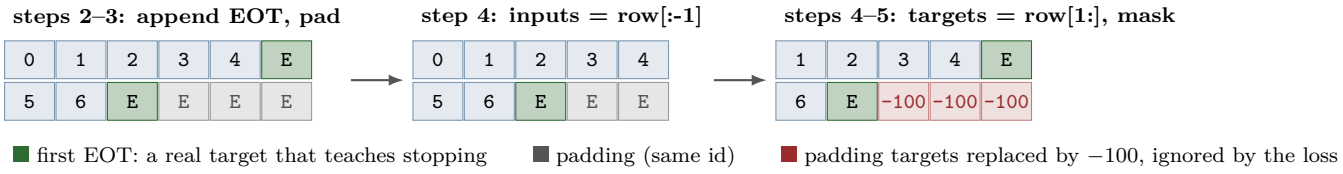


Figure 10.4: The SFT collate function on two sequences `[0,1,2,3,4]` and `[5,6]` (E = end-of-text id 50256). The pad token and the stop token share an id; the target mask is what tells them apart.

Listing 10.2: The instruction collate function (from `llmsys/finetune.py`).

```
IGNORE_INDEX = -100

def instruction_collate(batch, pad_id, ignore_index=IGNORE_INDEX,
                        allowed_max_length=None, mask_prompt=False, device="cpu"):
    items = [(b, 0) if not isinstance(b, tuple) else b for b in batch]
    max_len = max(len(ids) + 1 for ids, _ in items)       # +1 for the EOT
    inputs, targets = [], []
    for ids, prompt_len in items:
        row = list(ids) + [pad_id]                        # step 2
        row = row + [pad_id] * (max_len - len(row))       # step 3
        x = torch.tensor(row[:-1]); y = torch.tensor(row[1:])   # step 4
        y[len(ids):] = ignore_index                       # step 5: keep 1st EOT
        if mask_prompt and prompt_len > 0:
            y[:prompt_len - 1] = ignore_index             # optional: response-only
        if allowed_max_length is not None:
            x, y = x[:allowed_max_length], y[:allowed_max_length]
        inputs.append(x); targets.append(y)
    return torch.stack(inputs).to(device), torch.stack(targets).to(device)
```

Why -100 works. With `reduction="mean"`, `F.cross_entropy` computes

$$\mathcal{L} = \frac{\sum_i \mathbb{1}[y_i \neq -100]\,(-\log p_\theta(y_i \mid x_{\le i}))}{\sum_i \mathbb{1}[y_i \neq -100]}, \tag{10.1}$$

so ignored positions contribute neither to the numerator nor to the denominator. Adding padding therefore leaves the loss value and its gradient exactly unchanged. That is the property that makes dynamic padding safe. It also has a consequence that is easy to miss: the mean is over *tokens in the batch*, so long responses carry more total weight than short ones, and the effective weight of an example depends on its batch-mates. Gradient accumulation must reproduce this normalisation. Summing per-microbatch means over-weights tokens in short microbatches; the fix is to sum token losses and divide by the total target count of the full step.

Truncation. `allowed_max_length` caps sequences at the model's context window (1,024 for GPT-2). Truncate *after* building targets so that the input/target alignment survives. If truncation drops the response entirely, consider filtering the example out instead, since it then contributes only prompt tokens.

10.3.1 Masking the instruction

By default the loss covers prompt tokens too, so the model is also trained to predict the instruction text and the Alpaca preamble. A common variant, explored as an exercise in the LLM book, masks every target that predicts a prompt token, so only the response is learned (Fig. 10.5). The trade-offs:

- *Response-only loss* puts all gradient signal on what we want the model to produce, and stops the model from learning to generate user-side text. Most modern SFT recipes use it, especially for multi-turn chats, where only assistant turns are trained.

- *Full-sequence loss* gives more training signal per example. Some studies report it helps when responses are very short relative to prompts.

In the collate function, target index j predicts token $j+1$, so masking the prompt means masking targets $0..P-2$ for a prompt of P tokens. Target $P-1$, the first response token, is kept.

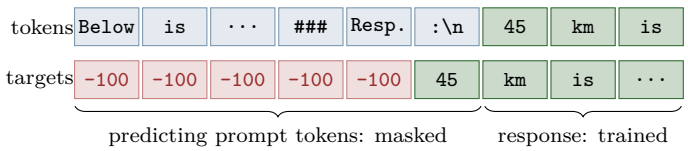


Figure 10.5: Response-only training. The target that predicts the first response token (here "45") sits at the last prompt position, which is why the mask stops at $P-1$.

Pitfall: Tokenisation at the prompt boundary

We compute P by tokenising the prompt alone. BPE is not compositional: the tokens of `prompt+response` are not always the tokens of `prompt` followed by those of `response`, because a merge can span the boundary. The Alpaca template ends in a newline, which no GPT-2 merge crosses, so it is safe. A template that ends in a space, such as `"Answer: "`, is not. Production code tokenises the two segments separately and concatenates the id lists, or uses a special-token boundary.

10.3.2 Packing: removing padding altogether

Bucketing (Section 9.3) reduces padding; *packing* eliminates it. We concatenate several EOT-terminated examples into one row of exactly T tokens, and fill rows with a bin-packing heuristic (first-fit decreasing is within 11/9 of optimal in the worst case and near-optimal in practice). Two corrections keep packing exactly equivalent to training on the examples separately:

1. **Block-diagonal causal attention.** A token in document 2 must not attend to document 1, so the attention mask is the causal mask AND a same-document mask.
2. **Per-document positions.** Position ids restart at 0 for each document. Otherwise the second document is processed at positions it would never see at inference time.

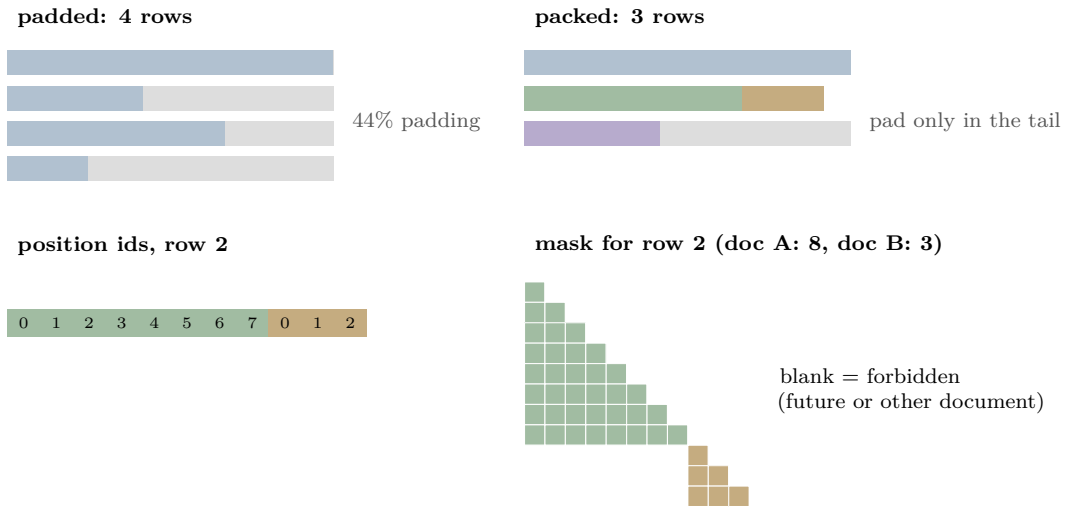


Figure 10.6: Padding versus packing for four sequences of lengths 12, 5, 8 and 3 with $T = 12$. Packing needs a block-diagonal causal mask and position ids that restart per document, so that documents are mathematically independent.

Listing 10.3: First-fit-decreasing packing and the block-diagonal mask.

```
def pack_sequences(seqs, max_len, pad_id, ignore_index=-100):
    rows, space = [], []
    for s in sorted(seqs, key=len, reverse=True):     # first-fit decreasing
        need = len(s) - 1                             # inputs s[:-1], targets s[1:]
        for r, free in enumerate(space):
            if free >= need:
                rows[r].append(s); space[r] -= need; break
        else:
            rows.append([s]); space.append(max_len - need)
    # ... fill input_ids, targets, segment_ids (1, 2, ...) and position_ids
    #     (restarting at 0) row by row; see llmsys/finetune.py

def segment_mask(segment_ids):                        # True = forbidden
    T = segment_ids.shape[1]
    causal = causal_mask(T, T, segment_ids.device)[None, None]
    other_doc = segment_ids[:, None, :, None] != segment_ids[:, None, None, :]
    return causal | other_doc
```

The lab verifies the equivalence numerically. The token-weighted mean loss of a packed batch equals the loss obtained by running every example on its own, to within 10^{-4}.

Systems Lens: Packing in real training stacks

Materialising a $T \times T$ mask per row, as in Listing 10.3, is fine for teaching but wasteful at scale. Production kernels take *cumulative sequence lengths* instead: `cu_seqlens` $= [0, \ell_1, \ell_1 + \ell_2, \dots]$ for a flattened batch with no padding at all. FlashAttention's *varlen* kernels skip the cross-document tiles entirely (Chapter 4), so packed attention costs $\sum_i \ell_i^2$ instead of T^2 per row. For RoPE models the position ids are passed alongside so each document's rotation restarts. Because of this:

- packing reduces MLP compute by the padding fraction, *and* reduces attention compute below even the unpadded-row cost T^2;
- the per-step token count becomes constant, which stabilises memory use and step time, and makes CUDA-graph capture possible (Chapter 11);
- the loss normalisation of Eq. (10.1) becomes a per-token mean over the whole pack. Schemes that weight each example equally need explicit per-example weights.

Naive packing, which concatenates without the mask and position reset, is common in pretraining, where documents are long and cross-document leakage is mostly harmless. In SFT the examples are short and a model that sees previous answers can learn to copy them, so do it properly.

Systems Lens: Where should the device transfer happen?

The LLM book moves each batch to the GPU inside the collate function, which is convenient for single-process loading. With `num_workers>0`, collate runs in worker processes. Those should not touch CUDA: initialising CUDA in forked processes is unsafe, and every worker would get its own CUDA context, costing hundreds of megabytes each. The robust pattern is to keep collate on the CPU, set `pin_memory=True` on the `DataLoader`, and in the training loop call `x.to(device, non_blocking=True)`. Pinned host memory lets the copy run as an asynchronous DMA that overlaps with the previous step's compute (Chapter 2).

10.4 Creating data loaders for an instruction dataset

LLM 7.4

The collate function has configuration (`pad_id`, `allowed_max_length`, `mask_prompt`) but `DataLoader` calls it with a single argument, the batch. `functools.partial` binds the rest:

Listing 10.4: Instruction data loaders.

```
from functools import partial
from llmsys.finetune import InstructionDataset, instruction_collate, format_alpaca

collate = partial(instruction_collate, pad_id=50256, allowed_max_length=1024,
                  mask_prompt=True)            # device stays "cpu": see Systems
    Lens
train_ds = InstructionDataset(train_data, tokenizer, fmt=format_alpaca)
train_loader = DataLoader(train_ds, batch_size=8, shuffle=True, drop_last=True,
                          collate_fn=collate, num_workers=2, pin_memory=True)
val_loader = DataLoader(InstructionDataset(val_data, tokenizer), batch_size=8,
                        shuffle=False, drop_last=False, collate_fn=collate)
```

`InstructionDataset` tokenises every example once in its constructor and stores `(ids, prompt_len)` pairs. Pre-tokenising pays off because tokenisation is pure Python-side CPU work that would

otherwise be repeated every epoch. For 1,100 examples it takes a fraction of a second; for millions of examples, tokenisation becomes an offline preprocessing job that writes binary shards (Chapter 3).

Iterating over the training loader shows the defining property of dynamic padding: batch widths vary (for example 8×61, 8×76, 8×73), each batch as wide as its longest example.

10.5 Loading a pretrained LLM

LLM 7.5

GPT-2 small (124M) is too weak to follow instructions well after a short SFT run. We use *GPT-2 medium* (355M parameters, 24 layers, $d = 1024$, 16 heads) and load it exactly as in Chapter 9, keeping the vocabulary head this time. Before training, check what the base model does with a formatted prompt from the validation set. Typically it repeats the `### Response:` header, restates the instruction, or starts a new instruction block. It recognises the format but has never been trained to fill it in.

Napkin Math: Can I fully fine-tune GPT-2 medium on my GPU?

GPT-2 medium has about 355M unique parameters, so AdamW state is about $16 \times 355\text{M} \approx 5.7\,\text{GB}$. Activations for a batch of 8 sequences of about 80 tokens are small, a few hundred MB. A full fine-tune fits on any 8–12 GB GPU, and even on a laptop CPU it runs in tens of minutes for two epochs over 935 examples. At 7B parameters the same arithmetic gives 112 GB for weights and optimizer state alone. That is where Section 10.10 begins.

10.6 Fine-tuning the LLM on instruction data

LLM 7.6

The training loop is the one from Chapter 7. The only change is that the loss uses `ignore_index=-100`, which is `F.cross_entropy`'s default, so the pretraining `loss_batch` works unchanged. Following the LLM book we train for two epochs with AdamW (learning rate 5×10^{-5}, weight decay 0.1), evaluating train and validation loss every few steps on a few batches.

Listing 10.5: SFT reuses the pretraining loop; only the data differs.

```
from llmsys.train import train, loss_loader

model.to(device)
opt = torch.optim.AdamW(model.parameters(), lr=5e-5, weight_decay=0.1)
print("before:", loss_loader(model, train_loader, device, 5),
                 loss_loader(model, val_loader, device, 5))
hist = train(model, train_loader, val_loader, opt, device, num_epochs=2,
             eval_freq=5, eval_iter=5)
```

A typical run starts near a loss of 4 (the base model is surprised by the template) and falls below 1 within the first epoch. Validation loss tracks training loss closely in epoch 1 and starts to separate in epoch 2. Training longer gives diminishing returns and eventually overfits a dataset this small. A useful habit is to print a generated response for one fixed validation prompt after every epoch. A decreasing loss can hide a model that has learned the template but not when to stop.

Systems Lens: What dominates SFT wall-clock time

For short SFT runs on small models the step itself is cheap, and time goes to (i) per-step Python and kernel-launch overhead, (ii) evaluation passes, and (iii) generation for qualitative checks, which runs at decode speed and is the slowest thing in this pipeline. Once models grow, the levers of Chapter 7 take over: bf16 autocast, fused AdamW, activation checkpointing for long sequences, and `torch.compile`. Packing becomes worth its complexity as soon as padding exceeds roughly 20%.

10.7 Extracting and saving responses

LLM 7.7

To evaluate, we generate a response for every test instruction and store it next to the reference. We format the prompt without the response, decode greedily with the KV-cached generator from Chapter 11 until the model emits the end-of-text token (or hits a length cap), and strip the prompt.

Listing 10.6: Generating, extracting and saving test responses.

```
from llmsys.generate import generate
from llmsys.finetune import extract_response

def extract_response(generated_text, marker="### Response:"):
    tail = generated_text.split(marker)[-1]            # after the last header
    tail = tail.split("### Instruction:")[0]           # model started a new task
    return tail.replace("<|endoftext|>", "").strip()

results = []
for entry in test_data:
    ids = torch.tensor([tok.encode(format_alpaca(entry))], device=device)
    out = generate(model, ids, max_new_tokens=256, eos_id=50256)
    entry = dict(entry, model_response=extract_response(tok.decode(out[0].tolist()))
    )
    results.append(entry)
json.dump(results, open("instruction-data-with-response.json", "w"), indent=2)
torch.save(model.state_dict(), "gpt2-medium355M-sft.pth")
```

Splitting on the response header is more robust than slicing off `len(prompt)` characters: it survives tokeniser round-trip differences in whitespace. Cutting at a new `### Instruction:` header guards against models that have not fully learned to stop.

Systems Lens: This loop is a serving problem in miniature

Generating 110 responses one at a time with a batch size of one leaves almost all of a GPU idle, because each decode step reads every weight to produce one token (Chapter 11). Batching the test prompts with left padding (`generate_batch`, Chapter 12) amortises each weight read over the whole batch and finishes the same work many times faster. For thousands of prompts, use a continuous-batching engine. The SFT and RL pipelines of later chapters (Chapters 17 and 20) are dominated by exactly this kind of bulk generation.

10.8 Evaluating the fine-tuned LLM

LLM 7.8

There are three broad ways to evaluate an instruction-following model (Chapter 14 treats each in depth):

1. **Short-answer benchmarks** such as MMLU test knowledge with multiple-choice questions. They are cheap and reproducible, but they barely test instruction following.
2. **Human preference comparisons**, such as the LMSYS Chatbot Arena, where people vote between two anonymous models. They are the gold standard for conversational quality, but slow and expensive.
3. **Automated conversational benchmarks** that use a strong LLM as the judge, such as AlpacaEval and MT-Bench. They are scalable, but inherit the judge's biases.

We follow the third route with a local judge. Ollama serves quantised open models behind a small REST API. We run an 8B Llama 3 instruct model and ask it to score every response from 0 to 100 against the reference.

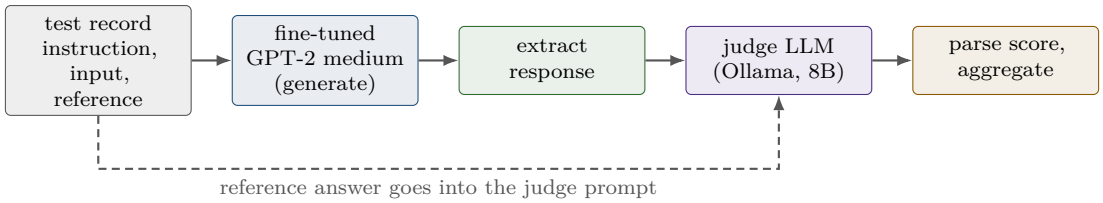


Figure 10.7: The evaluation pipeline. Generation is done once and saved; judging can then be repeated with other judges or rubrics without touching the fine-tuned model.

Listing 10.7: A minimal Ollama client and a scoring prompt.

```
def query_ollama(prompt, model="llama3", url="http://localhost:11434/api/chat",
                 seed=123, temperature=0.0, num_ctx=2048):
    payload = {"model": model, "messages": [{"role": "user", "content": prompt}],
               "options": {"seed": seed, "temperature": temperature,
                           "num_ctx": num_ctx}}
    req = urllib.request.Request(url, data=json.dumps(payload).encode(),
                                 headers={"Content-Type": "application/json"})
    chunks = []
    with urllib.request.urlopen(req) as resp:           # streamed JSON lines
        for line in resp:
            msg = json.loads(line)
            chunks.append(msg.get("message", {}).get("content", ""))
            if msg.get("done"):
                break
    return "".join(chunks)

def parse_score(text, lo=0, hi=100):
    m = re.search(r"-?\d+", text)
    return None if not m else max(lo, min(hi, int(m.group())))
```

The judge prompt (`build_judge_prompt` in `llmsys/finetune.py`) shows the instruction, the reference and the model answer, and asks for an integer only. We average the parsed scores over the test set and count unparseable replies separately rather than dropping them silently.

The LLM book's fine-tuned GPT-2 medium scores around 50 on this scale. That is respectable for 355M parameters and far behind a modern 8B instruct model.

Pitfall: LLM judges are measuring instruments, so calibrate them

Judges prefer longer answers, answers in their own style, and the first of two options in pairwise mode. Their scores drift between judge versions, and sampling makes them non-deterministic unless temperature and seed are pinned (we pass `temperature=0`, `seed=123`, and a fixed `num_ctx`). Before trusting a judge, score the reference answers themselves (they should be near 100) and a few deliberately wrong answers (near 0), and report the judge model and version with every number. Chapter 14 adds rubric-based grading and pairwise preference models.

Napkin Math: What does judging cost?

Each judgement is a prefill of about 200–300 tokens plus a few output tokens. An 8B model in 4-bit on a laptop GPU processes prefill at thousands of tokens per second, so 110 judgements take well under a minute. Most of the wall-clock time is per-request overhead and the sequential decode of the score. For thousands of evaluations, send requests concurrently: Ollama and similar servers batch concurrent requests, which raises throughput several-fold at the same per-request latency.

10.9 Conclusions and what comes next

LLM 7.9

At this point the LLM book's pipeline is complete: tokenisation and data sampling (Chapter 3), attention and the GPT architecture (Chapters 4 and 5), pretraining (Chapter 7), and two kinds of fine-tuning (this chapter and Chapter 9). Modern assistants add a third training stage after SFT: *preference tuning*, with RLHF/PPO or direct preference optimisation (DPO), and *reinforcement learning with verifiable rewards* for reasoning. We build the latter from scratch in Chapters 17 and 18 and the systems that run it at scale in Chapter 19. Before that, Part IV turns the trained model into a service.

On staying current: the field moves quickly, but the core loop of data formatting, masking, next-token loss and careful evaluation has stayed stable since the first instruction-tuned models. New methods mostly change the data or the objective. The best way to absorb them is the one this book follows: re-implement the core idea on a small model, measure it, and only then reach for a framework.

10.10 Parameter-efficient fine-tuning with LoRA

LLM E.1

Full fine-tuning updates every weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ by a dense $\Delta\mathbf{W}$. Low-rank adaptation (Hu et al., 2021) assumes the *update* has low intrinsic rank and parameterises it as a product,

$$\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W} = \mathbf{W} + \frac{\alpha}{r}\mathbf{B}\mathbf{A}, \qquad \mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}},\ \mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r},\ r \ll \min(d_{\text{in}}, d_{\text{out}}). \tag{10.2}$$

$\mathbf{W}$ is frozen and only $\mathbf{A}$ and $\mathbf{B}$ are trained. Because of distributivity we never form $\Delta\mathbf{W}$ during training:

$$\boldsymbol{y} = \boldsymbol{x}\mathbf{W}^\top + \frac{\alpha}{r}\,(\boldsymbol{x}\mathbf{A}^\top)\mathbf{B}^\top, \tag{10.3}$$

which costs $r(d_{\text{in}} + d_{\text{out}})$ multiply-adds per token on top of the frozen path. Three design choices matter:

Initialisation. $\mathbf{A}$ is random (Kaiming-uniform) and $\mathbf{B} = \mathbf{0}$, so $\Delta\mathbf{W} = \mathbf{0}$ at step 0. The adapted model starts *exactly* at the pretrained model, and the first gradient step reaches $\mathbf{B}$ through a non-zero $\mathbf{A}$.

Scaling α/r. The update is scaled by α/r so that changing r does not require re-tuning the learning rate. A common default is $\alpha = r$ or $\alpha = 2r$. The LLM book's implementation multiplies by α directly, without dividing by r, so its $\alpha = 16, r = 16$ corresponds to our $\alpha/r = 16$. Watch for this when porting hyperparameters between codebases. Our `LoRALinear` supports both conventions through `scale_by_rank`.

Placement. The original paper adapted only the attention projections W_q, W_v. Later work, notably QLoRA, found that adapting *every* linear layer, MLPs included, is needed to match full fine-tuning quality.

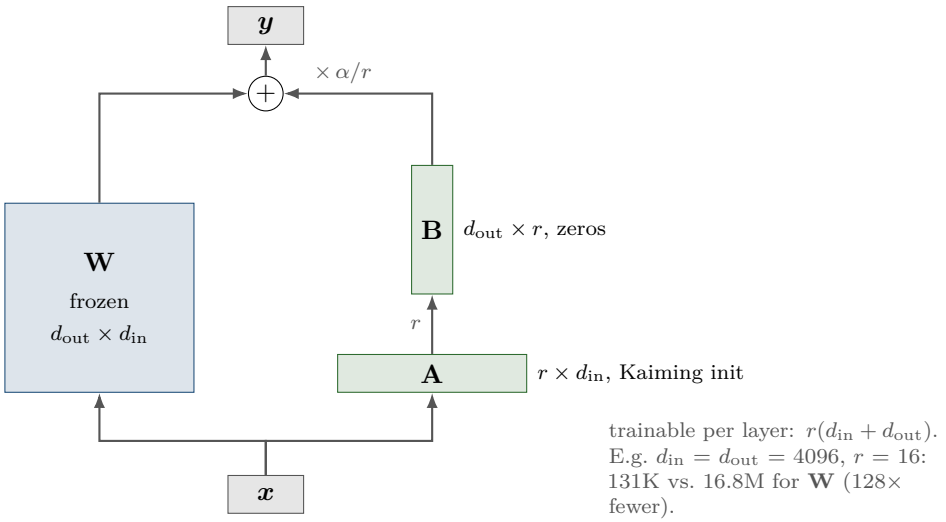


Figure 10.8: A LoRA-adapted linear layer. The frozen path and the low-rank path run in parallel and are summed. After training, $\frac{\alpha}{r}\mathbf{BA}$ can be folded into $\mathbf{W}$ so inference costs nothing extra.

Listing 10.8: A LoRA linear layer with merge and unmerge (from `llmsys/lora.py`).

```python
class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, rank=8, alpha=16.0, dropout=0.0,
                 scale_by_rank=True):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False                    # frozen W (and bias)
        self.scaling = alpha / rank if scale_by_rank else alpha
        self.lora_A = nn.Parameter(torch.empty(rank, base.in_features))
        self.lora_B = nn.Parameter(torch.zeros(base.out_features, rank))
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        self.dropout = nn.Dropout(dropout) if dropout > 0 else nn.Identity()
        self.merged = False

    def delta_weight(self):
```

```
        return self.scaling * (self.lora_B @ self.lora_A)        # (d_out, d_in)

    def forward(self, x):
        y = self.base(x)
        if self.merged:
            return y
        low = F.linear(F.linear(self.dropout(x), self.lora_A), self.lora_B)
        return y + self.scaling * low

    @torch.no_grad()
    def merge(self):
        if not self.merged:
            self.base.weight += self.delta_weight(); self.merged = True

    @torch.no_grad()
    def unmerge(self):
        if self.merged:
            self.base.weight -= self.delta_weight(); self.merged = False
```

10.10.1 Preparing the dataset

LLM E.2

The LLM book demonstrates LoRA on the spam classification task of Chapter 9, not on instruction data. That makes the comparison with full fine-tuning clean: the same balanced SMS splits, the same dynamic-padding loaders, and the same accuracy metric. Nothing about the data changes. The data pipeline is independent of how many parameters are trained, so the loaders from Listing 9.2 are reused verbatim.

10.10.2 Initializing the model

LLM E.3

We load pretrained GPT-2 small, swap in the two-class head (Listing 9.6), and check that the untrained classifier sits at chance (about 46–49% on the balanced splits in the LLM book's run). Then we freeze *every* parameter, the new head included. LoRA will supply all trainable parameters. Counting trainable parameters before and after freezing (124M, then zero) is a cheap guard against the most common bug, accidentally training the base.

10.10.3 Parameter-efficient fine-tuning with LoRA

LLM E.4

`apply_lora` walks the module tree and replaces every `nn.Linear` with a `LoRALinear` wrapping it. A predicate can restrict which layers are adapted.

Listing 10.9: Replacing linear layers with LoRA layers.

```
def apply_lora(model, rank=8, alpha=16.0, dropout=0.0, target=None,
               scale_by_rank=True, freeze_rest=True):
    if freeze_rest:
        for p in model.parameters():
            p.requires_grad = False
    def recurse(module, prefix):
        for name, child in list(module.named_children()):
            full = f"{prefix}.{name}" if prefix else name
            if isinstance(child, nn.Linear) and (target is None or target(full,
    child)):
                setattr(module, name, LoRALinear(child, rank, alpha, dropout,
```

```
                                                scale_by_rank))
            elif not isinstance(child, LoRALinear):
                recurse(child, full)
    recurse(model, "")
    return model

model = add_classification_head(load_gpt2_small(), num_classes=2)
apply_lora(model, rank=16, alpha=16, scale_by_rank=False)   # LLM-book convention
print(count_trainable(model))      # (2666528, ...)
```

With $r = 16$ on every linear layer of GPT-2 small plus the two-way head, the trainable count is 2,666,528. Per block that is $4 \times 16 \times (768 + 768) = 98{,}304$ for the attention projections and $16 \times (768 + 3072) \times 2 = 122{,}880$ for the MLP, times 12 blocks, plus $16 \times (768 + 2)$ for the head. That is about 2.1% of the model and 47× fewer than full fine-tuning. Training uses the same loop as Listing 9.9. With a learning rate around 5×10^{-5} and five epochs, the LLM book reaches accuracy comparable to fine-tuning the last block, in somewhat more wall-clock time. LoRA does not reduce compute much on a model this small (see the next Systems Lens), and its benefits appear at scale.

Key Idea: Merging makes inference free

After training, `merge()` folds $\frac{\alpha}{r}\mathbf{BA}$ into $\mathbf{W}$, and `merge_lora` swaps every `LoRALinear` back to a plain `nn.Linear`. The merged model has the original architecture and latency. The lab checks that outputs are identical before and after merging, and that unmerging restores the base weights. Keep adapters unmerged only when you need to switch between them quickly, as in multi-LoRA serving below.

Systems Lens: What LoRA saves, and what it does not

Write a frozen linear layer's training cost per token as forward $2P$, activation gradient $\partial \boldsymbol{x} = \partial \boldsymbol{y}\, \mathbf{W}$ at $2P$, and weight gradient $\partial \mathbf{W} = \partial \boldsymbol{y}^\top \boldsymbol{x}$ at $2P$, where $P = d_{\text{in}} d_{\text{out}}$. LoRA removes only the last term: the activation gradient must still flow through $\mathbf{W}$ to reach the adapters in lower layers. Training FLOPs per token therefore drop from $\approx 6N$ to $\approx 4N$, at best 1.5× cheaper. What LoRA mainly removes is *memory*:

- no gradients or Adam moments for the N frozen parameters ($14N$ bytes saved);
- frozen weights can be stored in bf16 without fp32 master copies, or quantised (QLoRA, below);
- checkpoints shrink from gigabytes to megabytes per task.

Activation memory is *not* reduced. Every layer's input must still be kept for the adapter weight gradients and for the backward pass through $\mathbf{W}$. Activation checkpointing (Chapter 7) remains the tool for that.

10.10.4 QLoRA: a 4-bit frozen base

Since the base is frozen, it only needs to be *read*. QLoRA (Dettmers et al., 2023) stores it in 4 bits and dequantises on the fly inside each matmul, while adapters and activations stay in bf16. Three ingredients make this work without hurting quality:

1. **NormalFloat-4 (NF4).** Pretrained weights are roughly normally distributed, so the 16

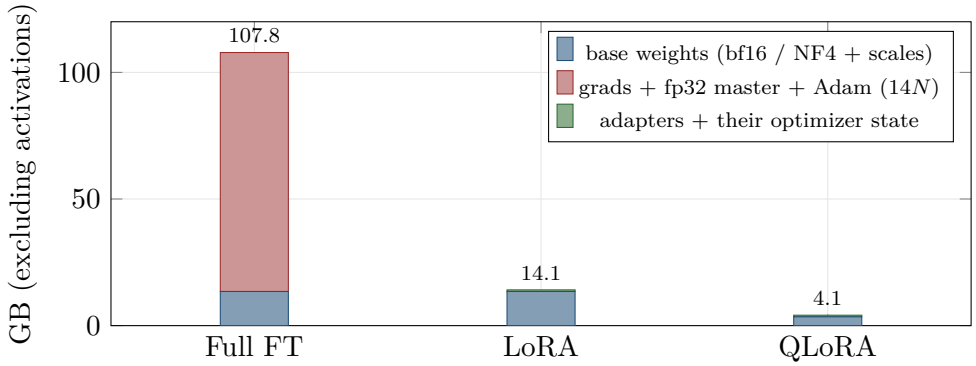


Figure 10.9: Persistent training memory for a 6.74B-parameter Llama-style model ($d = 4096$, 32 layers, $d_{\text{ff}} = 11008$), computed from the formulas in the text with $r = 16$ adapters on every linear layer (40.0M adapter parameters at 16 bytes each).

quantisation levels are placed at quantiles of $\mathcal{N}(0, 1)$ rescaled to $[-1, 1]$, with an exact zero. With equal-probability bins every level is used about equally often, which is the information-theoretically efficient choice for normal data. Values are scaled per block of 64 weights by the block's absolute maximum.

2. **Double quantisation.** One fp32 scale per 64 weights adds $32/64 = 0.5$ bits per parameter. Quantising the scales themselves to 8 bits in blocks of 256 reduces this to $8/64 + 32/(64 \cdot 256) \approx 0.127$ bits per parameter.

3. **Paged optimizers.** Adapter optimizer states live in unified memory, which the driver can page to CPU RAM during memory spikes (long sequences) instead of failing with an out-of-memory error.

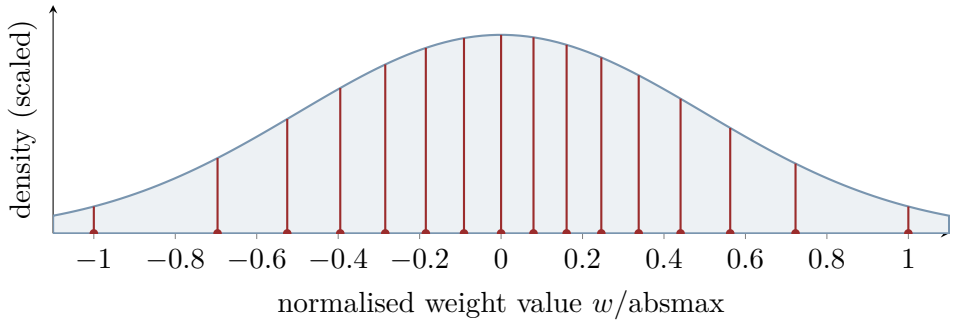


Figure 10.10: The 16 NF4 code values (red) over a normal density, matching the shape of block-normalised weights. Levels are dense near zero, where most weights live, and sparse in the tails. Seven are negative and eight positive, so that zero is exact.

Listing 10.10: Blockwise NF4 quantisation (one code per byte for clarity).

```
def quantize_nf4(w, block=64):
    flat = w.detach().float().reshape(-1)
    pad = (-flat.numel()) % block
    if pad:
        flat = torch.cat([flat, flat.new_zeros(pad)])
    blocks = flat.view(-1, block)
    absmax = blocks.abs().amax(dim=1).clamp(min=1e-12)       # one scale / block
    normed = blocks / absmax[:, None]                         # in [-1, 1]
    codes = (normed[..., None] - NF4_LEVELS).abs().argmin(dim=-1).to(torch.uint8)
```

```
    return codes, absmax, tuple(w.shape)

def dequantize_nf4(codes, absmax, shape, dtype=torch.float32):
    vals = NF4_LEVELS[codes.long()] * absmax[:, None]
    return vals.reshape(-1)[:math.prod(shape)].reshape(shape).to(dtype)
```

Listing 10.10 stores one code per byte to keep indexing simple. Production kernels (bitsandbytes) pack two codes per byte and fuse dequantisation into the GEMM, so the 4-bit weights are expanded only in registers or shared memory. On random Gaussian matrices our implementation reconstructs weights with a mean absolute error of about 9% of the mean magnitude, which the lab checks. Chapter 13 develops weight quantisation for inference, including GPTQ and AWQ, which choose levels using calibration data rather than a fixed codebook.

Napkin Math: A 7B model on one 24 GB GPU

Base weights in NF4 with double quantisation: $6.74\times10^9\times(4+0.127)/8 \approx 3.5$ GB. Adapters ($r = 16$, all linear layers): 40M parameters $\times$ 16 bytes ≈ 0.64 GB. That leaves roughly 20 GB for activations and the CUDA context, enough for batches of several thousand tokens with activation checkpointing. Full fine-tuning of the same model needs about 108 GB before activations, more than one 80 GB GPU. QLoRA has one cost: every forward and backward pass dequantises the weights, which typically makes each step slower than LoRA on a bf16 base.

10.10.5 Serving many adapters

Merged adapters are free at inference, but a merged model serves exactly one task. A platform hosting one base model and hundreds of customer-specific adapters faces a different problem: requests for different adapters arrive interleaved, and one weight copy per adapter does not fit. Systems such as Punica (Chen et al., 2023) and S-LoRA (Sheng et al., 2023) keep adapters unmerged. They batch requests across adapters by running the base GEMM once for the whole batch, then applying each row's adapter with a gather-based batched kernel (Punica's "BGMV"). S-LoRA additionally pages adapter weights between host and GPU memory, in a unified pool with the KV cache.

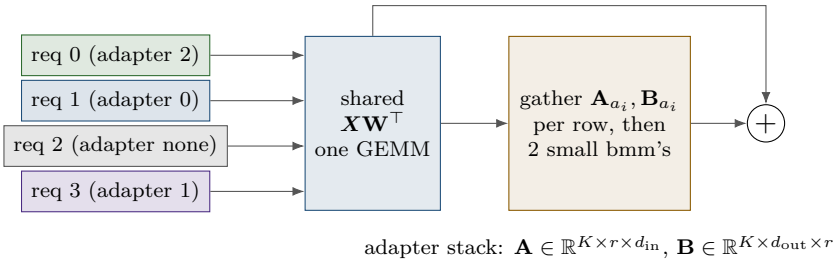


Figure 10.11: Batched multi-adapter inference. The expensive base matmul is shared by all requests, and each row pays only for its own rank-r correction. Row 2 uses the base model only.

Listing 10.11: A gather-based multi-LoRA layer (reference, not a fused kernel).

```
class MultiLoRALinear(nn.Module):
    def __init__(self, base, n_adapters, rank, alpha=16.0):
        super().__init__()
        self.base, self.scaling = base, alpha / rank
        self.A = nn.Parameter(torch.zeros(n_adapters, rank, base.in_features))
```

```
        self.B = nn.Parameter(torch.zeros(n_adapters, base.out_features, rank))

    def forward(self, x, adapter_ids):          # x (B, T, d_in), ids (B,), -1 = none
        y = self.base(x)
        ids = adapter_ids.clamp(min=0)
        A, B = self.A[ids], self.B[ids]          # (B, r, d_in), (B, d_out, r)
        delta = torch.bmm(torch.bmm(x, A.transpose(1, 2)), B.transpose(1, 2))
        delta = delta * (adapter_ids >= 0).to(x.dtype)[:, None, None]
        return y + self.scaling * delta
```

Napkin Math: How many adapters fit?

For a 7B Llama-style model with $r = 16$ on the four attention projections only, one adapter has $32 \times 4 \times 16 \times (4096 + 4096) \approx 16.8$M parameters, which is 34 MB in bf16. A thousand adapters are 34 GB, too many to keep resident next to the base weights and a KV cache on one 80 GB GPU. Hence S-LoRA's design: hot adapters stay in GPU memory, cold ones live in host RAM and are fetched over PCIe on demand. A 34 MB fetch at about 25 GB/s takes about 1.4 ms, cheap compared with a prefill. The adapter math per token is $2r(d_{\text{in}} + d_{\text{out}})$ FLOPs per adapted layer, about 0.25% of the base model here, so the overhead is dominated by gather and kernel efficiency rather than FLOPs.

Deeper Dive: The wider PEFT family

LoRA dominates in practice, but the design space is larger. *Adapters* (Houlsby et al., 2019) insert small bottleneck MLPs between layers and add inference latency unless they are fused. *Prefix and prompt tuning* learn virtual tokens and leave all weights untouched. They are cheap to store, but they cost context length on every request. *(IA)*^{3} learns per-channel rescaling vectors, with even fewer parameters than LoRA. *DoRA* (Liu et al., 2024) decomposes each weight into a magnitude and a direction and applies LoRA to the direction, often closing the remaining gap to full fine-tuning. The systems question is the same for all of them: what must be stored per task, and can the per-task computation be batched across tasks?

10.11 Summary

- SFT is next-token training on templated prompt→response data. The template defines the model's interface and must be identical at training and inference time.
- The collate function appends an end-of-text token, pads, shifts, and replaces every padding target after the first EOT with -100. Optional prompt masking trains only on responses. Masked positions drop out of both the numerator and the denominator of the mean loss.
- Packing removes padding entirely. To be exact it needs a block-diagonal causal mask and per-document position ids; varlen attention kernels implement this without materialising masks.
- Keep collate on the CPU, pin memory, and transfer asynchronously. Bulk generation for evaluation is a serving problem and should be batched.

- LLM-as-a-judge evaluation scales, but the judge is an instrument that must be pinned (model, version, temperature, seed) and sanity-checked.
- LoRA trains $r(d_{\text{in}} + d_{\text{out}})$ parameters per layer, starts exactly at the base model, and can be merged for free inference. It saves optimizer and gradient memory, cuts training FLOPs by at most 1.5×, and does not reduce activation memory.
- QLoRA stores the frozen base in NF4 with double-quantised scales (about 4.13 bits per parameter), which puts 7B-model fine-tuning on one consumer GPU.
- Unmerged adapters can be batched across tasks on a shared base (Punica, S-LoRA), which enables multi-tenant serving.

10.12 Exercises

Exercise 10.1 (*Phi-3 template*). Re-run SFT with `format_phi3`. Compare the total number of training tokens, the wall-clock time per epoch, and the judge score against the Alpaca template. Make sure response extraction is adapted to the new template.

Exercise 10.2 (*Response-only loss*). Train with `mask_prompt=True` and compare validation loss (careful: the losses are averaged over different token sets, so compare on a common token set) and judge scores. Under what data conditions would you expect full-sequence loss to win?

Exercise 10.3 (*Loss normalisation under gradient accumulation*). Show with a two-microbatch example that averaging per-microbatch mean losses differs from the per-token mean over the whole step when microbatches have different numbers of target tokens. Implement the correct normalisation in the training loop and verify that the gradient matches a single large-batch step.

Exercise 10.4 (*Packing efficiency*). For the 935 training examples tokenised with GPT-2's BPE, compute padding waste for (a) random batches of 8 with dynamic padding, (b) length-bucketed batches, and (c) first-fit-decreasing packing into rows of 512 and 1,024 tokens. Then estimate the attention FLOPs of (c) with a varlen kernel ($\sum \ell_i^2$) versus a dense mask (T^2 per row).

Exercise 10.5 (*Naive packing leaks*). Pack examples *without* the block-diagonal mask and position reset. Construct a probe showing that the model's prediction for a token in document 2 changes when document 1 changes. How large is the effect after fine-tuning on naively packed data?

Exercise 10.6 (*Judge reliability*). Score the 110 reference answers themselves and 20 deliberately wrong answers with the judge. Then re-score your model's responses with a second judge model, and report the correlation between judges and the mean absolute score difference. What does this tell you about the precision of a single reported score?

Exercise 10.7 (*LoRA rank sweep*). For LoRA on GPT-2 medium SFT, sweep $r \in \{1, 4, 16, 64\}$ with $\alpha = 2r$ and with fixed $\alpha = 16$. Plot judge score and validation loss against trainable

parameters. Explain why the α/r convention makes the learning rate more transferable across ranks.

Exercise 10.8 (*Where to put adapters*). Compare LoRA on (a) W_q, W_v only, (b) all attention projections, (c) MLP only, and (d) all linear layers at equal total trainable parameters (adjust r). Which wins on the spam task, and which on SFT?

Exercise 10.9 (*FLOP accounting*). Instrument a LoRA training step with `torch.profiler` and attribute time to forward GEMMs, activation-gradient GEMMs, weight-gradient GEMMs (adapters only) and everything else. Compare with full fine-tuning and with the $6N$ versus $4N$ prediction.

Exercise 10.10 (*NF4 versus uniform int4*). Quantise each weight matrix of pretrained GPT-2 small with (a) NF4 and (b) uniform symmetric int4, both with absmax scales per 64-weight block. Compare reconstruction error and validation perplexity. Then check whether GPT-2's weights are actually close to normal (plot a histogram of block-normalised weights).

Exercise 10.11 (*Multi-LoRA throughput*). Using `MultiLoRALinear`, measure the throughput of a batch of 64 rows that use (a) one adapter, (b) 8 adapters, (c) 64 distinct adapters, against the base model alone. Where does the gather cost show up, and how would a fused kernel avoid it?

10.13 Lab 10: The SFT data path, packing, LoRA and QLoRA

Lab 10: The SFT data path, packing, LoRA and QLoRA

Code: `code/labs/lab10_instruct_lora`

Goals

Implement the parts of instruction tuning where correctness bugs hide (templating, target construction, masking, packing) and the parameter-efficient machinery that makes large-model fine-tuning feasible (LoRA, merging, batched multi-adapter layers, NF4 quantisation). Verify each part against an exact reference, then train.

Setup

`cd code; LAB_IMPL=starter python -m pytest -q labs/lab10_instruct_lora`. Tests use a byte-level tokenizer and a tiny GPT (context 128). They run on CPU in under a minute. `run.py` instruction-tunes GPT-2 medium (full or LoRA) on an Alpaca-style JSON file, writes `responses.json`, and scores it with a local Ollama judge.

Tasks

1. `format_alpaca`: exact template, optional input section.
2. `instruction_collate`: append EOT, pad, shift, keep the first EOT, mask the rest with -100, optional prompt masking and truncation. The test uses the `[0..4]`, `[5,6]`, `[7,8,9]` example from Fig. 10.4.
3. `pack_sequences` and `segment_mask`: first-fit-decreasing packing with per-document positions and a block-diagonal causal mask. The test checks that the packed loss equals the per-example token-weighted loss to 10^{-4}.
4. `LoRALinear` (`delta_weight`, `forward`, `merge`, `unmerge`), `apply_lora`, `count_trainable`:

zero-init equivalence, merge equivalence, exact trainable-parameter formula.

5. `MultiLoRALinear.forward`: gather-based batched adapters with a "no adapter" id. No Python loop over rows.
6. `quantize_nf4` and `dequantize_nf4`: blockwise absmax NF4 with padding for sizes that are not multiples of the block.
7. `extract_response` and `parse_score`.
8. `finetune`: a short LoRA training loop. The test requires the loss to drop by at least 30% on a toy instruction task.

Measurements

- Padding waste and attention FLOPs for padded, bucketed and packed batches on your instruction data (Exercise 10.4).
- On a GPU with `run.py`: peak memory, time per epoch and trainable parameters for full fine-tuning versus LoRA ($r = 16$) of GPT-2 medium. Expect LoRA to cut persistent memory sharply and time per step only modestly. Compare judge scores.
- Reconstruction error of NF4 versus int4 on real GPT-2 weights (Exercise 10.10).

Deliverables

Passing tests; a table of memory, time and score for full fine-tuning versus LoRA; one paragraph explaining the measured memory and time differences using the $6N/4N$ and $16N/2N$ models of this chapter.

Stretch goals

- Wrap the frozen base with `NF4LoRALinear` (provided in `llmsys/lora.py`) and fine-tune GPT-2 medium QLoRA-style; compare quality and step time with bf16 LoRA.
- Serve two LoRA adapters (for example spam classification and SFT) from one base with `MultiLoRALinear` in every linear layer, and route a mixed batch.

Part IV

Inference and Serving

Chapter 11

Inference Fundamentals: From Logits to Tokens per Second

Everything we built in Parts II and III ends in a function that maps a sequence of token ids to a matrix of logits. This chapter turns that function into a *text generator*, and then into a *fast* text generator. The first half follows the path every practitioner walks the first time they run an open-weight model: set up an environment, size the hardware, tokenize a prompt, load a pretrained checkpoint (Qwen3-0.6B, the base model for all of Part V), and write the autoregressive loop. The second half is where the systems person in you should lean in. We will see that naive generation does a quadratic amount of redundant work, that the KV cache removes it at the price of a memory footprint which quickly becomes *the* resource that limits a serving system, and that once the redundant work is gone, decoding is governed not by FLOPs but by bytes: every generated token streams the entire model through the memory system.

The chapter closes by making the measurement vocabulary precise—time to first token, time per output token, throughput, goodput—because every later chapter (batching engines, speculative decoding, inference-time scaling, RL rollouts) is ultimately judged by these numbers.

11.1 From a prompt to a response

RM 2.1

A pretrained *base* model has only seen next-token prediction on web-scale text. It has not been instruction-tuned (Chapter 10) or trained to reason (Chapter 17), yet it already produces fluent continuations and, surprisingly often, follows simple instructions, because instruction–response pairs are common in its training data. Using such a model involves three stages (Fig. 11.2):

1. **Input preparation.** The user's string is converted into a list of integer token ids by the tokenizer, optionally after wrapping it in a *chat template*.

2. **The model.** One or more forward passes map token ids to logits, i.e. unnormalized scores over the vocabulary for the next position.

3. **Output processing.** A decoding rule (greedy argmax for now; temperature, top-k and top-p in Chapter 15) selects a token id, which is appended to the sequence; at the end, the tokenizer maps ids back to a string.

The model is the expensive part, but the other two stages are not free. In a serving system,

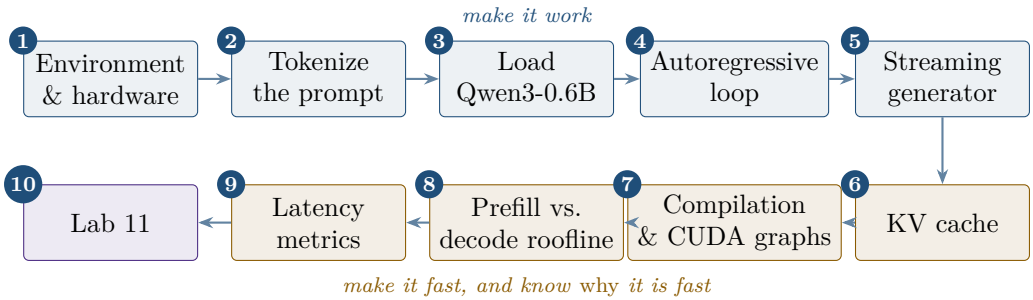


Figure 11.1: Roadmap of this chapter. The top row gets a pretrained model to produce text; the bottom row removes redundant work and builds the performance model we use for the rest of the book.

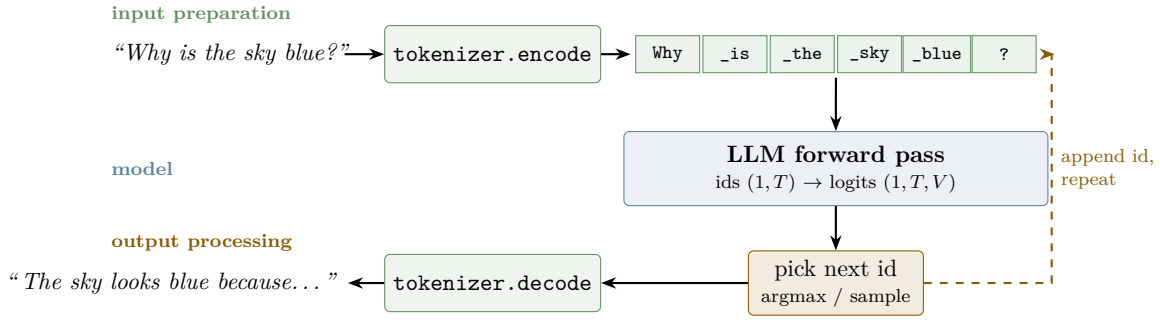


Figure 11.2: The three stages of text generation. Only the dashed feedback edge makes generation *autoregressive*; everything else is a pure function. (The underscore marks a leading space, which most byte-level BPE tokenizers fold into the following token.)

tokenization runs on the CPU and can become a bottleneck at high request rates; sampling over a 150k-entry vocabulary on the GPU involves sorts and softmaxes that are surprisingly costly when fused poorly. We revisit both in Chapter 12.

Key Idea: Read diagrams bottom-up

By convention, model diagrams are drawn with inputs at the bottom and outputs on top, so information flows upward through the layer stack. We follow this convention for architecture figures (Fig. 11.3); data-flow figures such as Fig. 11.2 read left to right.

11.2 Setting up the environment

RM 2.2

All code for the book lives in the `code/` directory: the importable package `llmsys` plus one directory per lab. Two setup paths work equally well.

pip. With Python ≥ 3.10 in any virtual environment:

```
cd code
pip install -r requirements.txt      # torch, tokenizers, safetensors, sympy, ...
python -c "import llmsys, torch; print(torch.__version__)"
```

PyTorch wheels are hardware-specific. If the default wheel does not match your accelerator, install PyTorch first using the selector on `pytorch.org`, then install the remaining requirements.

uv. The `uv` package manager resolves and installs dependencies much faster than pip, creates an isolated environment automatically, and can even provision a Python interpreter. After

installing `uv`:

```
cd code
uv venv && source .venv/bin/activate
uv pip install -r requirements.txt
uv run python -m pytest -q tests        # runs inside the managed environment
```

Either way, the environment must be *reproducible*: pin versions in a lock file for experiments you intend to report. Inference numerics depend on library versions (kernels, reduction orders, TF32 defaults), and "the same code" can produce different greedy outputs across PyTorch releases.

11.3 Hardware: what you need and why

RM 2.3

Training a frontier base model is out of reach for individuals. DeepSeek-V3, one of the few recent models with a detailed compute disclosure, reports about 2.79 million H800 GPU-hours for its full training on 14.8 trillion tokens, which the authors price at roughly US$5.6 million at an assumed rental rate of $2 per GPU-hour, and that excludes all research and ablation runs. This book therefore never pretrains a large model. We take a small but architecturally modern pretrained model, Qwen3-0.6B, and apply to it the same post-training and inference-time techniques that are used on models a hundred times larger. The methods transfer; only the constants change.

Because chapters 11–16 only run inference, they work on a laptop CPU. Reinforcement learning (Chapter 17) and distillation (Chapter 20) want a GPU. The device-selection helper in `llmsys.utils.get_device` picks CUDA, Apple's MPS, or the CPU, in that order. On NVIDIA Ampere and newer GPUs it is also worth enabling TF32 tensor-core math for float32 matmuls, which trades a few mantissa bits for a large speed-up:

Listing 11.1: Device selection and TF32.

```
import torch
from llmsys.utils import get_device

device = get_device()                   # cuda > mps > cpu
if device.type == "cuda":
    # fp32 matmuls run on tensor cores with a 10-bit mantissa (TF32)
    torch.backends.cuda.matmul.allow_tf32 = True
    torch.backends.cudnn.allow_tf32 = True
    print(torch.cuda.get_device_name(0))
print("using", device)
```

Napkin Math: Will it fit?

A model with N parameters stored in a format of b bytes needs Nb bytes for weights alone. Qwen3-0.6B has $N = 596$M parameters (the input embedding and the LM head are tied):

- bf16: $0.596 \times 10^9 \times 2 \approx 1.19$ GB; fp32: 2.38 GB.
- Qwen3-8B: 16.4 GB in bf16, which fits on a 24 GB consumer card with a little room for the KV cache; Qwen3-32B (65.5 GB) needs an 80 GB accelerator or quantization (Chapter 13).

Weights are only the floor. The KV cache (Section 11.8) adds memory proportional to

batch × *context*, and for long reasoning traces it can exceed the weights.

Cloud notebooks (Google Colab, Lightning AI Studios and similar) are a convenient way to get a GPU by the hour. For this chapter, a single entry-level data-center GPU such as an NVIDIA L4 is more than enough; for RL in Part V an 80 GB GPU makes life much easier.

Systems Lens: Matching the device to the phase

Different phases of the life cycle stress different resources. Pretraining and RL training are dominated by dense matmuls (compute-bound, FLOP/s matter). Autoregressive decoding with small batches is dominated by reading weights and KV cache (bandwidth-bound, TB/s matter). This is why an RTX 4090 (165 TFLOP/s bf16, 1.0 TB/s) can decode a small model at a respectable fraction of an H100's speed (989 TFLOP/s, 3.35 TB/s) even though it has one sixth the compute: for batch-1 decoding the relevant ratio is bandwidth, roughly 1:3.3. Apple-silicon laptops with large unified memory and high bandwidth are popular local-inference machines for the same reason.

11.4 Preparing inputs: tokenizers and chat templates

RM 2.4

Every prompt, every reasoning trace and every answer must become a list of integers before the model can see it. Qwen3 uses a byte-level BPE tokenizer (the algorithm we implemented in Chapter 3) with a vocabulary of $V = 151{,}936$ entries. We load it from the `tokenizer.json` file shipped with the checkpoint using the Hugging Face `tokenizers` library, wrapped by `llmsys.qwen3.Qwen3Tokenizer`.

Listing 11.2: A tokenizer round trip.

```
from llmsys.qwen3 import Qwen3Tokenizer

tok = Qwen3Tokenizer("qwen3-0.6b-base/tokenizer.json")
prompt = "Why is the sky blue?"
ids = tok.encode(prompt)            # list[int]
assert tok.decode(ids) == prompt     # lossless for byte-level BPE
for i in ids:
    print(f"{i:>7d} -> {tok.decode([i])!r}")
```

Printing ids one at a time shows the subword structure: frequent words are single tokens (usually including their leading space), rarer words are split into pieces, and *nothing* is ever out of vocabulary, because any byte sequence can fall back to single-byte tokens. Made-up words and non-English text simply tokenize into more pieces (Exercise 11.1).

Vocabulary size is a systems trade-off. GPT-2 used about 50k tokens, Llama 3 about 128k, Qwen3 about 152k. A larger vocabulary compresses text into fewer tokens, so every request is shorter (fewer decode steps, smaller KV cache), but it makes the embedding table and the output projection larger.

Napkin Math: The cost of the LM head

For Qwen3-0.6B, the output projection is a $1024 \times 151{,}936$ matrix: 155.6 M parameters, 26% of the whole model. Per generated token it costs $2 \cdot 1024 \cdot 151{,}936 \approx 311$ MFLOP out of

≈ 1.19 GFLOP total. During decoding it is also 311 MB of bf16 weights that must be read for every step. Small models pay disproportionately for big vocabularies; for Qwen3-8B the head is only about 8% of parameters. Two practical consequences: (i) during prefill, compute the LM head only for the last position (our serving engine in Chapter 12 does this); (ii) sampling kernels must handle V-length rows efficiently.

Chat templates. Instruction-tuned and reasoning models are trained on conversations serialized with special tokens. Qwen uses the ChatML layout:

```
<|im_start|>user
Why is the sky blue?<|im_end|>
<|im_start|>assistant
```

The model is expected to continue after the final `assistant` header and to end its turn with `<|im_end|>`. The `apply_chat_template` method of our tokenizer wrapper produces this string. For the reasoning variant of Qwen3 the template controls "thinking mode": an empty `<think>...</think>` block pre-filled into the assistant turn disables the reasoning trace. A base model has no special relationship with these tokens, so in Chapter 14 we will use a plain-text prompt format for it instead. Getting the template wrong is the single most common cause of mysteriously bad outputs from open-weight models.

Pitfall: Template and EOS mismatches

Base Qwen3 ends documents with `<|endoftext|>` (id 151643), while chat-tuned variants end turns with `<|im_end|>`. If you stop on the wrong id, a base model happily continues past the end of its answer into a new, unrelated document. Always check which token(s) your checkpoint uses to terminate, and pass all of them to the stopping logic.

11.5 Loading pretrained weights

RM 2.5

We use Qwen3-0.6B for three reasons: it is small enough for consumer hardware, it is architecturally identical to its much larger siblings (Chapter 6), and it ships both as a *base* model (our starting point for reasoning training) and as an official *reasoning* variant that serves as a reference in evaluations. Our implementation in `llmsys.qwen3` is plain PyTorch, independent of any LLM library, yet weight-compatible with the official checkpoint.

Listing 11.3: Loading Qwen3-0.6B from a downloaded snapshot.

```
# one-time download (needs internet):
#   huggingface-cli download Qwen/Qwen3-0.6B-Base --local-dir qwen3-0.6b-base
import torch
from llmsys.qwen3 import load_qwen3_from_dir

model = load_qwen3_from_dir("qwen3-0.6b-base", size="0.6B",
                            dtype=torch.bfloat16, device=device)
print(model)       # 28 x Qwen3Block(GQA + SwiGLU + 2 RMSNorm)
```

`load_qwen3_from_dir` constructs `Qwen3Model` with randomly initialized placeholders, reads every `.safetensors` shard, and copies tensors in by name (the mapping is in `load_qwen3_hf_state_dict`). Printing the module shows the structure summarized in Fig. 11.3: a 151,936 × 1024 embedding, 28 identical blocks, and a final norm and output head. Each block's attention has 16 query

heads but only 8 key/value heads of dimension 128, so `W_query` maps $1024 \to 2048$ while `W_key`/`W_value` map $1024 \to 1024$. The MLP is a SwiGLU with hidden size 3072. We derived all of these pieces in Chapter 6; here the model is a black box that turns ids into logits.

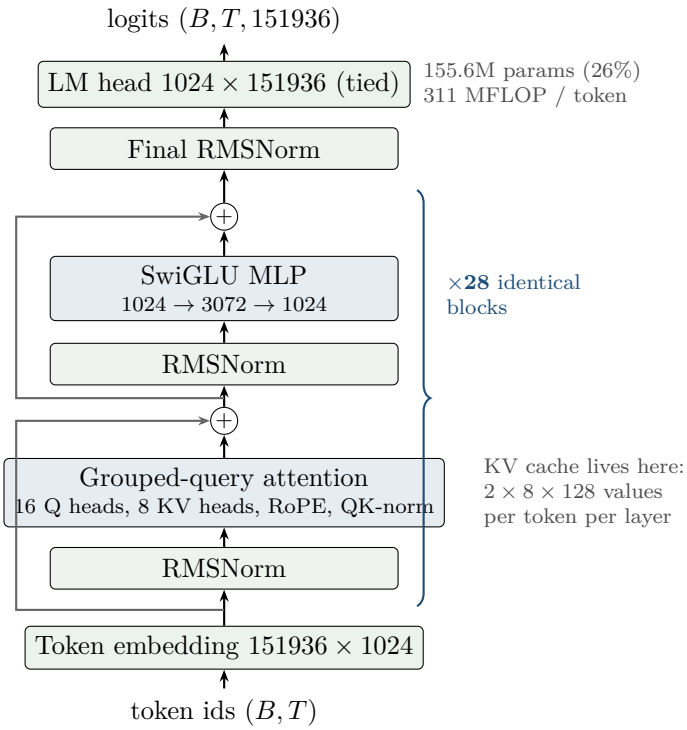


Figure 11.3: Qwen3-0.6B as a data-flow stack (bottom to top). Grey arrows are residual connections. The annotations point at the two places that matter most for inference cost: the large tied output head and the per-layer KV state.

Systems Lens: Loading is an I/O problem

Loading a checkpoint is a copy from disk to host memory to device memory. A 1.19 GB bf16 checkpoint takes about 0.4 s at 3 GB/s NVMe speed and about 0.05 s over a PCIe 4.0 x16 link (≈25 GB/s effective); for a 70B model the same numbers become tens of seconds, which matters for autoscaling and for RL systems that reload weights frequently (Chapter 19). Three tricks are standard:

- **safetensors + mmap:** a flat header plus raw little-endian tensor bytes; tensors can be memory-mapped without unpickling and without an intermediate copy (unlike `torch.load` of a pickle, which can also execute arbitrary code).
- **Meta-device construction:** build the module on `torch.device("meta")` so no memory is allocated for random initial weights, then materialize directly from the checkpoint (`load_state_dict(..., assign=True)`).
- **Direct-to-GPU and sharded loading:** each tensor-parallel rank reads only its slice (Chapter 13); GPUDirect Storage can bypass host memory entirely.

11.6 The autoregressive loop, one step at a time

RM 2.6

LLMs generate one token at a time. At each iteration the model sees the prompt plus everything generated so far, predicts one new token, and the token is appended to the input of the next

iteration (Fig. 11.4). This is *autoregressive* generation.

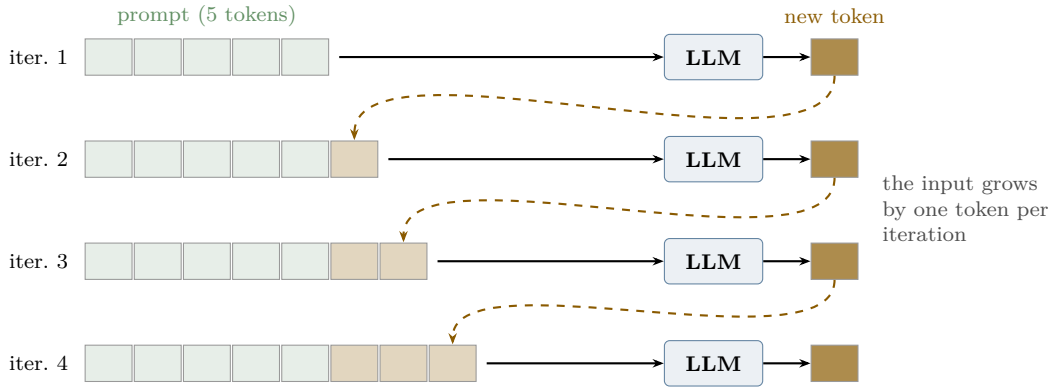


Figure 11.4: Autoregressive generation unrolled over four iterations. Each output token (dark) is fed back as the last input token of the next iteration (light). Without further tricks, iteration i re-processes all $5 + i - 1$ tokens.

Zoom into one iteration. A causal LM processes the whole input in a single forward pass and returns one vector of V logits *per input position*: the logits at position t score the candidates for position $t + 1$. For a T-token input the output has shape $(1, T, V)$. During training all T predictions are used (that is what makes pretraining efficient, Chapter 7). During generation, the predictions for positions inside the prompt are about tokens we already have, so only the *last* row matters (Fig. 11.5).

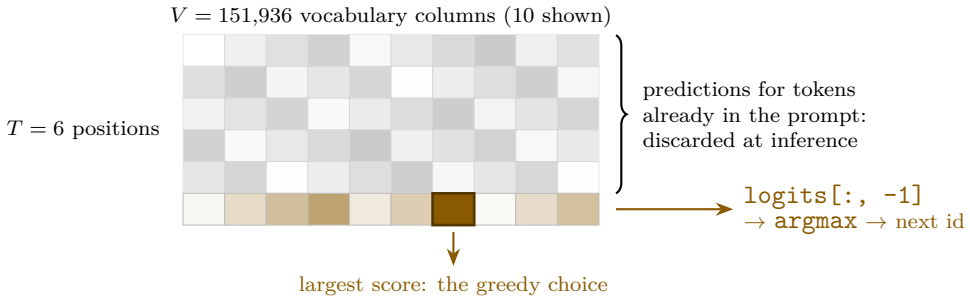


Figure 11.5: One forward pass returns a $T \times V$ score matrix (batch dimension dropped). Generation reads only the last row and selects the column with the largest score.

Listing 11.4: One step of generation by hand.

```
ids = torch.tensor(tok.encode(prompt), device=device).unsqueeze(0)   # (1, T)
with torch.inference_mode():
    logits = model(ids)                  # (1, T, V)
print(logits.shape, logits.dtype)        # torch.Size([1, T, 151936]) torch.bfloat16
last = logits[0, -1]                     # (V,) scores for position T+1
next_id = torch.argmax(last, dim=-1, keepdim=True)   # (1,)
print(repr(tok.decode(next_id.tolist())))
```

A few PyTorch details in Listing 11.4 are worth spelling out:

- `unsqueeze(0)` adds the batch dimension the model expects; `squeeze(0)` removes a size-1 dimension again. Shape bookkeeping is the main source of bugs in generation code.
- `torch.max` returns the largest *value*, `torch.argmax` its *index*. The index into the vocabulary

dimension *is* the token id. `keepdim=True` keeps a trailing dimension of size 1, which is convenient for concatenating the id onto a (B, T) tensor.

- Always choosing the argmax is *greedy decoding*: deterministic, cheap, and usually fine for short factual answers, but prone to repetition. Chapter 15 introduces sampling.
- `torch.inference_mode()` is a stricter, faster version of `torch.no_grad()`: it disables autograd tracking *and* version counters on tensors, so nothing is saved for a backward pass. The logits come back in bf16 because the weights are bf16.

Deeper Dive: "Inference" is an overloaded word

In statistics, *inference* means estimating unknown quantities from data, with uncertainty. In deep learning it means something narrower: evaluating a trained, frozen function on new inputs. When we say "inference" in this book we mean the latter. Nothing is learned while generating; the model is a fixed map from token ids to next-token scores, and the decoding rule is a fixed map from scores to an id.

11.7 A minimal streaming generator

RM 2.7

Wrapping the single step in a loop gives a complete generator. We write it as a Python *generator function* that `yield`s every token as soon as it exists. Streaming is not cosmetic: users perceive latency as "time until text starts appearing", and every serving API (server-sent events, websockets) exposes tokens incrementally.

Listing 11.5: Greedy streaming generation without a cache (Lab 11, Task 1).

```
@torch.inference_mode()
def generate_nocache(model, token_ids, max_new_tokens, eos_id=None):
    model.eval()                                    # disable dropout etc.
    for _ in range(max_new_tokens):
        logits = model(token_ids)[:, -1]            # (B, V): last position only
        nxt = torch.argmax(logits, dim=-1, keepdim=True)   # (B, 1)
        if eos_id is not None and bool((nxt == eos_id).all()):
            return                                  # every row has finished
        yield nxt
        token_ids = torch.cat([token_ids, nxt], dim=1)     # grow the input
```

The caller decodes and prints each token as it arrives:

```
ids = torch.tensor([tok.encode("Explain KV caching in one sentence.")], device=
    device)
for t in generate_nocache(model, ids, max_new_tokens=100, eos_id=tok.pad_token_id):
    print(tok.decode(t[0].tolist()), end="", flush=True)    # flush: show it now
```

Two observations about the output of a base model. First, the first emitted token usually starts with a space, because BPE attaches leading spaces to words; strip it if you care. Second, without a stopping condition a base model finishes its answer, emits the document separator `<|endoftext|>`, and continues into an unrelated document. Passing the separator's id as `eos_id` stops generation at the right place. In a batch, we can only stop when *every* row has finished, which is the first hint of the batching inefficiencies we fix in Chapter 12.

Measuring it. We need a stopwatch that is honest on asynchronous hardware. CUDA kernel launches return immediately, so a timer that does not call `torch.cuda.synchronize()` measures launch overhead, not execution. Listing 11.6 records a timestamp after each yielded token and derives the metrics we define precisely in Section 11.12; peak GPU memory is available from `torch.cuda.max_memory_allocated()`.

Listing 11.6: Timing a streaming generator (Lab 11, Task 6).

```
def measure_generation(gen_fn, model, token_ids, max_new_tokens,
                       clock=time.perf_counter):
    _sync(); t0 = clock(); stamps = []
    for _ in gen_fn(model, token_ids, max_new_tokens):
        _sync()                      # wait for the GPU before reading the clock
        stamps.append(clock())
    n = len(stamps)
    ttft = stamps[0] - t0                                  # includes the prefill
    tpot = (stamps[-1] - stamps[0]) / (n - 1)              # mean inter-token gap
    return dict(n_tokens=n, ttft=ttft, tpot=tpot,
                tokens_per_s=n / (stamps[-1] - t0))
```

Streaming detokenization. Decoding tokens one at a time is subtly wrong. A byte-level BPE token is a sequence of *bytes*, not characters: an emoji or a CJK character is 3–4 UTF-8 bytes and is frequently split across two or three tokens. Calling `decode([t])` on each piece produces replacement characters (`U+FFFD`) for the incomplete fragments. Production servers decode *incrementally*: keep the ids generated so far, decode a small trailing window, and only emit the part of the text that can no longer change (no trailing replacement character); the rest is held back until the next token arrives.

Listing 11.7: Incremental detokenization that never emits partial characters.

```
class IncrementalDetokenizer:
    def __init__(self, tok, window=8):
        self.tok, self.ids, self.emitted, self.window = tok, [], 0, window

    def push(self, token_id) -> str:
        self.ids.append(token_id)
        start = max(0, len(self.ids) - self.window)   # bounded work per token
        prefix = self.tok.decode(self.ids[start:-1]) if len(self.ids) > 1 else ""
        full = self.tok.decode(self.ids[start:])
        if full.endswith("\ufffd"):                   # incomplete UTF-8 sequence
            return ""                                 # wait for more bytes
        new = full[len(prefix):] if full.startswith(prefix) else full
        return new
```

The window keeps the cost per token constant (decoding the full history every step would be quadratic, the same mistake as recomputing without a KV cache), and a serving front end typically runs this on the CPU, overlapped with the next GPU decode step.

On a laptop CPU, Listing 11.5 with Qwen3-0.6B produces a handful of tokens per second, and it gets slower as the answer gets longer. The rest of the chapter explains why, and fixes it.

11.8 KV caching

RM 2.8

Look again at Fig. 11.4. In iteration i the model recomputes the hidden states of all earlier tokens, although they are bit-for-bit identical to those of iteration $i-1$: in a causal model, position t never attends to positions after t, so adding a token at the end cannot change anything that came before. The only thing a new query needs from earlier positions are their *keys* and *values* in every attention layer. Store them, and each decode step only has to process the single new token.

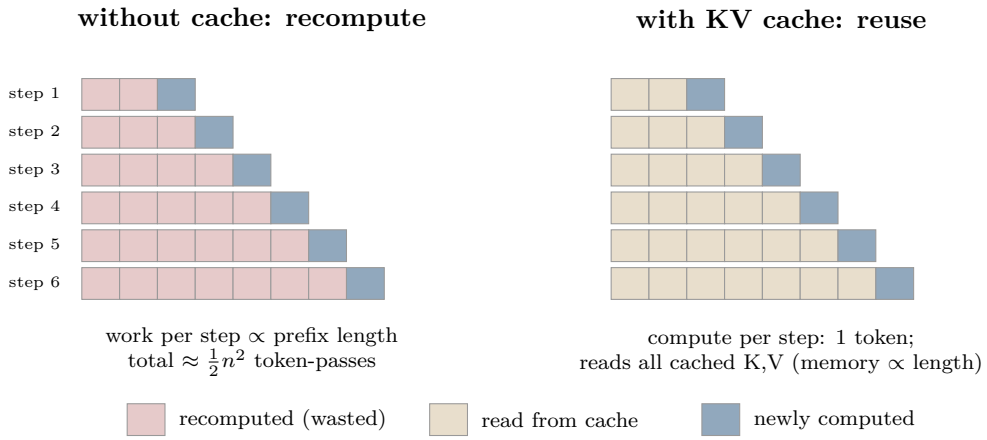


Figure 11.6: Work per decode step. Without a cache, the redundant (red) triangle dominates. With a cache, compute per step is constant, but the cached state that must be *read* grows linearly, which is why long-context decoding becomes memory-bound.

Our attention modules already support this: they accept a `cache` object and, if present, call `cache.update(layer_idx, k_new, v_new)`, which appends the new keys and values and returns the full history (Chapters 4 and 6). The generator changes in exactly two places:

Listing 11.8: Greedy generation with a KV cache (Lab 11, Task 2).

```
from llmsys.kvcache import KVCache

@torch.inference_mode()
def generate_kvcache(model, token_ids, max_new_tokens, eos_id=None):
    model.eval()
    cache = KVCache(model.n_layers)
    logits = model(token_ids, cache=cache)[:, -1]       # PREFILL: whole prompt once
    for _ in range(max_new_tokens):
        nxt = torch.argmax(logits, dim=-1, keepdim=True)
        if eos_id is not None and bool((nxt == eos_id).all()):
            return
        yield nxt
        logits = model(nxt, cache=cache)[:, -1]         # DECODE: one token per step
```

The first call, with the whole prompt, is called **prefill**; each subsequent single-token call is a **decode** step. Positional information must stay consistent: Qwen3 rotates queries and keys by their *absolute* position (RoPE), so the attention module asks the cache how many tokens it already holds and uses that as the position offset; GPT-2 offsets its learned position embeddings the same way. The critical sanity check is that cached and uncached generation produce *identical* greedy tokens; Lab 11 tests exactly that.

Complexity. Generating n tokens after a P-token prompt without a cache costs $\sum_{t=0}^{n-1}(P+t) \approx nP + \frac{1}{2}n^2$ token-passes through the dense layers. With a cache it costs $P + n$. For $P = 32$, $n = 512$ that is a factor of ≈ 270 fewer dense-layer FLOPs. Attention itself is still linear per step (the new query attends over $P + t$ cached keys), so the total attention work is quadratic in both cases, but with a tiny constant.

Napkin Math: How big is the KV cache?

Per token, every layer stores one key and one value vector per KV head:

$$\text{bytes/token} = 2 \cdot L \cdot H_{kv} \cdot d_h \cdot b. \tag{11.1}$$

- Qwen3-0.6B (L=28, H_{kv}=8, d_h=128, bf16): $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2 = 114{,}688$ B = 112 KiB per token. A single 32k-token reasoning trace: 3.8 GB, three times the weights. The full 40,960-token context: 4.7 GB.
- Had Qwen3-0.6B used full multi-head attention (16 KV heads), it would need 224 KiB/token: grouped-query attention halves the cache (Chapter 6).
- Qwen3-8B: 144 KiB/token; Qwen3-32B: 256 KiB/token. A batch of 64 sequences of 8k tokens on Qwen3-8B needs 77 GB of KV cache, about five times its 16.4 GB of weights.

The KV cache, not the weights, is what bounds batch size and context length in a serving system.

Systems Lens: Growing vs. static caches

Our `KVCache` grows by `torch.cat`, which allocates a new tensor and copies the entire history at every step: $O(t)$ bytes moved at step t, $O(n^2)$ in total, plus allocator churn. `StaticKVCache` preallocates $(B, H_{kv}, T_{\max}, d_h)$ buffers once and writes each new slice in place: no reallocation, no copies, and fixed memory addresses. Our version still returns a growing *view* `[:, :, :end]`, so attention sees a different key length every step; fully shape-static engines attend over the whole $T_{\max}$ buffer and mask positions beyond the current length. Fixed shapes and addresses are what make CUDA graphs and compiled kernels reusable (next section). They come at a price: memory is reserved for the maximum length up front, even if the request stops early. Chapter 12 resolves this tension with paged allocation.

11.9 Compiling the model

RM 2.9

Eager PyTorch executes one Python operator at a time: each `nn.Linear`, each RMSNorm, each elementwise multiply dispatches through the Python interpreter, the dispatcher, and finally launches a kernel. For a large batch this overhead is invisible. For batch-1 decoding of a small model, the kernels are tiny and the CPU-side overhead can dominate: a Qwen3-0.6B decode step launches several hundred kernels, and at a few microseconds each the launch path alone can rival the ≈0.4 ms it takes an H100 to read the weights.

`torch.compile` attacks this from two directions (Fig. 11.7). *TorchDynamo* traces Python bytecode into an FX graph guarded by assumptions (shapes, dtypes, Python constants); *Inductor* lowers the graph, fuses chains of elementwise and reduction ops into single generated (Triton or C++) kernels, and picks matmul implementations. With `mode="reduce-overhead"`

the result is additionally captured into a *CUDA graph*, so a whole decode step is replayed with a single launch.

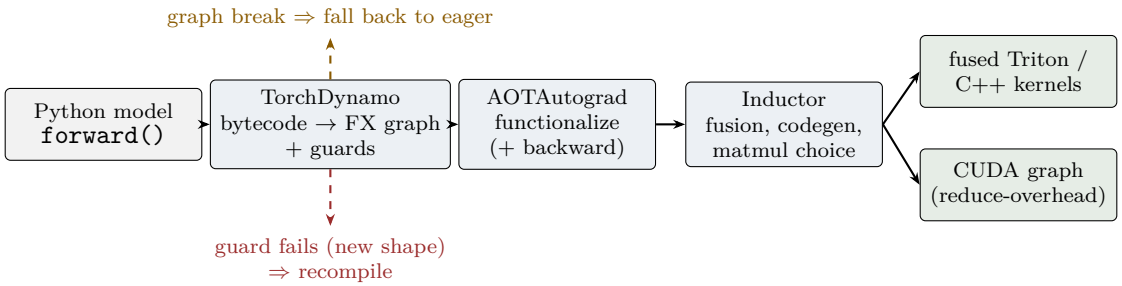


Figure 11.7: The `torch.compile` stack. Static shapes avoid recompilation and enable CUDA graph capture; data-dependent Python control flow causes graph breaks.

Listing 11.9: Compiling and benchmarking with warm-up runs.

```
compiled = torch.compile(model)            # or mode="max-autotune" / "reduce-
    overhead"
for rep in range(3):                       # rep 0 pays for tracing + codegen
    m = measure_generation(generate_static, compiled, ids, 100)
    tag = "warm-up" if rep == 0 else f"timed run {rep}"
    print(f"{tag}: {m['tokens_per_s']:.1f} tok/s, TTFT {m['ttft']*1e3:.0f} ms")
```

Three practical rules follow from Fig. 11.7:

1. **Always warm up.** The first call traces and generates code and can take tens of seconds. Report only subsequent runs, and say so.

2. **Keep shapes static on the hot path.** A growing `torch.cat` cache gives every decode step a different key length; Dynamo then either recompiles or falls back to dynamic shapes, which limits fusion and rules out CUDA graphs. A static buffer attended with a length mask plus a position counter held in a device tensor gives identical shapes every step. Modules that mutate Python integers (`self.pos += 1`) can also trigger recompiles; recent PyTorch versions offer configuration flags to treat such integers as dynamic rather than baked-in constants.

3. **Prefill and decode are different programs.** Prefill has shape (B, P) with variable P; decode has shape $(B, 1)$. Engines compile (and graph-capture) decode for a fixed set of batch sizes and pad the live batch up to the nearest captured size, while prefill runs eagerly or with dynamic shapes.

The speed-up from compilation depends heavily on where the time goes. On a CPU, fusion reduces memory traffic and Python overhead; with a cache the per-step work is so small that overhead matters a lot, and compiled cached decoding is typically several times faster than eager. Without a cache the work is dominated by large matmuls that compilation cannot make faster, so the gain is small. If default compilation gives nothing, try `mode="max-autotune"`, which benchmarks alternative kernels and tile sizes at compile time.

Pitfall: Benchmarking the compiler instead of the model

Comparing a compiled model's *first* run against an eager model's steady state, forgetting `synchronize()`, letting the GPU clock ramp up during the first measurement, or measuring with a 10-token output where TTFT dominates are the four most common ways to publish meaningless inference numbers. Lab 11 makes you control all four.

11.10 Where the time goes: launch overhead and CUDA graphs

The roofline bound of the next section is a statement about the GPU. Eager PyTorch at batch 1 often never gets close to it, because the GPU is waiting for the CPU. Count the kernels in one Qwen3 decode step:

Napkin Math: Kernels per decode step

Per block, an eager forward pass launches roughly: two RMSNorms (≈6 small kernels each: cast, square, mean, rsqrt, two multiplies), three QKV GEMMs, two QK-norms (≈12), RoPE on q and k (slicing, `cat`, two multiplies and an add each, ≈10), the cache update (2), `repeat_interleave` for GQA (2), scores GEMM, scale, mask, softmax, value GEMM (≈6), output projection, residual add, three SwiGLU GEMMs plus SiLU and multiply, and a second residual add. That is ≈50 kernels per block, about 1,400 for 28 blocks, plus the head. At 4–8 µs of CPU time per eager op (Python, dispatcher, launch), the CPU needs 6 ms to 11 ms to *issue* a step whose GPU work takes 0.4 ms at the bandwidth bound. Batch-1 eager decoding on a fast GPU is therefore *launch-bound*: the GPU idles about 95% of the time.

Three remedies, in increasing order of effectiveness: (i) *fusion* (`torch.compile`, hand-written fused RMSNorm/RoPE kernels, Appendix A) reduces the number of kernels; (ii) *asynchrony*: as long as the CPU runs ahead, launches overlap with GPU work, so avoid anything that forces a sync on the hot path (`.item()`, `.tolist()`, Python `if` on a GPU tensor, as in our EOS check!); (iii) *CUDA graphs* record the entire sequence of kernel launches of one step once, with fixed memory addresses, and replay it with a single `cudaGraphLaunch`. Graphs require static shapes and static buffers, which is exactly what `StaticKVCache` provides.

Listing 11.10: Capturing one static-cache decode step in a CUDA graph (GPU only).

```
# Assumes: model on cuda, cache = StaticKVCache(...) already holding the prefill,
# and a model/cache that read the write position from a *device tensor*, so the
# replayed graph does not bake in a Python integer (an exercise in Lab 11).
static_in = torch.zeros(B, 1, dtype=torch.long, device="cuda")
s = torch.cuda.Stream(); s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):                  # warm up on a side stream
    for _ in range(3):
        static_out = model(static_in, cache=cache)
torch.cuda.current_stream().wait_stream(s)
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):                   # record kernels + addresses, no
    execution
    static_out = model(static_in, cache=cache)
# decode loop: copy the new token into the static input, replay, read static output
for _ in range(n):
    static_in.copy_(next_ids)
```

```
    g.replay()                                  # ~1 launch instead of ~1,400
    next_ids = static_out[:, -1].argmax(-1, keepdim=True)
```

The comment in Listing 11.10 hides the real engineering: our cache tracks its length in a Python list, and Python state is frozen into a captured graph. Production engines keep positions, sequence lengths and slot indices in device tensors that are updated by `copy_` before each replay, and capture one graph per supported batch size (e.g. 1, 2, 4, 8, . . . , 256), padding the live batch up to the next captured size. `torch.compile(mode="reduce-overhead")` automates exactly this bookkeeping for you.

Systems Lens: Overhead hides in the sampler too

Our EOS check `bool((nxt == eos_id).all())` copies a GPU boolean to the host and blocks until the step finishes, which serializes CPU and GPU every token. Engines avoid per-token host syncs by keeping stop conditions on the device, checking them only every k steps, or running the scheduler for step $t+1$ while the GPU executes step t ("async scheduling"). On a modern GPU these few microseconds per token matter: at 0.4 ms per step, a 50 µs sync is a 12% slowdown.

11.11 Anatomy of a decode step: the roofline view

We now have the vocabulary to answer the question this chapter promised: what limits the speed of generation? Recall the roofline model from Chapter 2: a kernel that performs F FLOPs and moves M bytes takes at least $\max(F/\Pi,\ M/\beta)$, where Π is peak compute and β peak memory bandwidth. Its *arithmetic intensity* $I = F/M$ decides which term wins; the crossover (the *ridge point*) is Π/β, about 295 FLOP/byte on an H100 (989 TFLOP/s over 3.35 TB/s), 153 on an A100 and 164 on an RTX 4090.

Weight GEMMs. A linear layer with an $m \times k$ weight applied to B token vectors does $2Bmk$ FLOPs and must read $mk\,b$ bytes of weights (activations are comparatively tiny when $B \ll k$). Its intensity is

$$I_{\text{GEMM}} \approx \frac{2Bmk}{mk\,b} = \frac{2B}{b} \quad \text{FLOP/byte}, \tag{11.2}$$

i.e. exactly B for bf16. A decode step processes one token per sequence, so B is the *batch size*; a prefill step processes P tokens per sequence, so B is the *number of prompt tokens*. Prefill of a 2,000-token prompt has intensity ≈ 2000: firmly compute-bound. Batch-1 decode has intensity ≈ 1: two orders of magnitude below the ridge, so the tensor cores sit idle more than 99% of the time while HBM streams weights.

Attention over the cache. A decode query attends over C cached tokens: $\approx 4\,C\,H\,d_h$ FLOPs per layer to read $2\,C\,H_{kv}\,d_h\,b$ bytes. Its intensity is $\approx (H/H_{kv}) \cdot (2/b)$, i.e. the GQA group size for bf16 (2 for Qwen3-0.6B), and *independent of batch size*: each sequence reads its own cache. Batching amortizes weight reads but never KV reads.

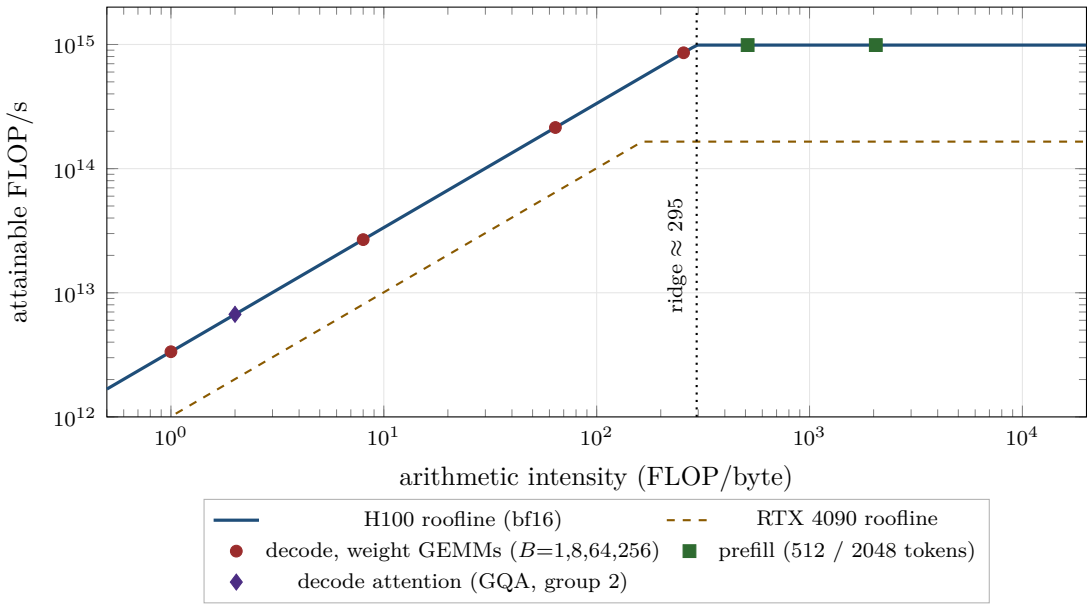


Figure 11.8: Roofline placement of inference work on an H100 (analytic, from published peak numbers). Decode GEMMs sit at intensity $\approx B$ and climb the bandwidth slope as the batch grows; attention over the KV cache stays near the GQA group size regardless of batch; prefill is compute-bound.

Napkin Math: A lower bound on decode speed

A decode step must read all weights once plus every sequence's KV cache. With N parameters, b bytes each, batch B, context C, and κ KV bytes per token:

$$t_{\text{step}} \;\geq\; \max\left(\frac{2NB}{\Pi},\; \frac{Nb + B\,C\,\kappa}{\beta}\right).$$

For Qwen3-0.6B on an H100 at $B = 1$, $C = 1024$: $(1.19\,\text{GB} + 0.12\,\text{GB})/3.35\,\text{TB/s} \approx 0.39\,\text{ms}$, i.e. at most $\approx 2{,}560$ tokens/s for a single stream, and compute would need only 1.2 µs. For Qwen3-8B the bound is 4.9 ms (≈ 200 tokens/s); for Qwen3-32B 19.6 ms (≈ 50 tokens/s). Real systems reach a good fraction of these bounds only with CUDA graphs and fused kernels; eager PyTorch at batch 1 is often launch-bound and far slower.

Table 11.1: Roofline lower bounds for one decode step on an H100 at context 1,024 (bf16, attention FLOPs ignored). Throughput rises almost linearly with B while weight reads dominate, then saturates as per-sequence KV reads take over. Computed with `llmsys.flops.decode_step_time_lower_bound`.

	Qwen3-0.6B			Qwen3-8B		
B	step (ms)	tokens/s	bound	step (ms)	tokens/s	bound
1	0.39	2,560	memory	4.94	203	memory
8	0.64	12,600	memory	5.25	1,520	memory
64	2.60	24,600	memory	7.77	8,230	memory
256	9.33	27,400	memory	16.4	15,600	memory
512	18.3	28,000	memory	28.0	18,300	memory

Table 11.1 contains the central lesson of LLM serving. For the small model, KV reads overtake weight reads already at $B \approx 10$ (each sequence's 1,024-token cache is 115 MB, a tenth of the weights), so batching saturates early; for the 8B model weights dominate for longer and batching pays off over a wider range. Either way, throughput is bought with batch size, and batch size

is bought with KV-cache memory. Everything in Chapters 12 and 13 (paged KV memory, prefix sharing, quantized caches, GQA/MLA, speculative decoding) is a way to move along this trade-off curve.

Systems Lens: Prefill and decode want different machines

Prefill is compute-bound and its latency scales with prompt length; decode is bandwidth-bound and its latency scales with model size and context. Running both on the same GPU means a long prefill stalls every decoding user for its duration, while decode steps leave the tensor cores idle. Production systems either interleave the phases carefully (chunked prefill), or *disaggregate* them onto separate GPU pools and ship the KV cache between them. Both are designed in Chapter 12.

11.12 Measuring inference: metrics and honest benchmarks

Throughput alone does not describe a serving system, because users experience latency, and the two pull in opposite directions. The standard vocabulary (Fig. 11.9):

TTFT (time to first token): from request arrival to the first output token. It contains queueing delay plus the prefill, and grows with prompt length. Interactive applications target a few hundred milliseconds.

TPOT / ITL (time per output token / inter-token latency): the gap between consecutive tokens during decoding. Reading speed is around 5–10 tokens/s, so a TPOT below 100 ms feels instantaneous for chat; agents and reasoning models that emit thousands of hidden tokens need far lower TPOT.

E2E latency: $\text{TTFT} + (n-1) \cdot \text{TPOT}$ for n output tokens.

Throughput: output tokens per second across all users (sometimes input+output tokens), the quantity that determines cost per token.

Goodput: the throughput of requests that *meet their latency SLOs*, e.g. "TTFT $<$ 500 ms and TPOT $<$ 50 ms at P90". A system can raise throughput by batching harder while its goodput collapses.

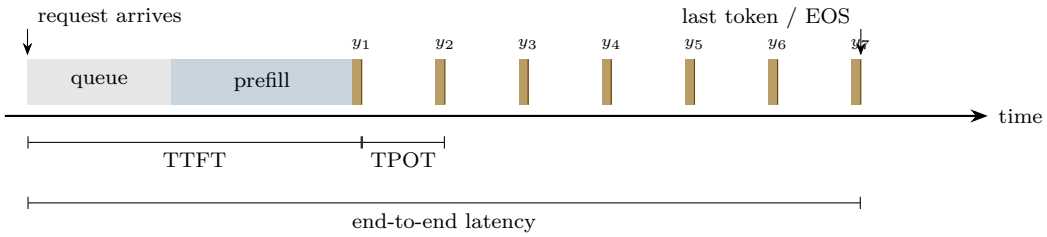


Figure 11.9: Latency anatomy of one streamed request. Queueing and prefill determine TTFT; the decode cadence determines TPOT. Under load, other users' prefills interleaved with our decode steps make TPOT jittery, which is why tail percentiles matter.

Little's law. In steady state, the number of requests in the system equals arrival rate times mean latency: $\bar{N} = \lambda \bar{W}$. If a server sustains a decode batch of 64 and each request takes 10 s end to end, it can admit at most 6.4 requests/s; pushing the arrival rate beyond that only

grows the queue and therefore TTFT. This simple identity is the right first model for capacity planning, and we use it again with the simulator in Chapter 12.

Key Idea: A benchmarking checklist

1. Synchronize the device before every clock read; use `time.perf_counter`.
2. Warm up (compilation, autotuning, allocator, GPU clocks) and discard warm-up runs.
3. Report TTFT and TPOT separately, with the prompt and output lengths used.
4. Report distributions (P50/P90/P99), not just means, and the number of repetitions.
5. Fix the output length (ignore EOS or force it) when comparing systems, otherwise a model that stops earlier looks faster.
6. State hardware, dtype, batch size, library versions and whether results are compiled.

What we measured. Figure 11.10 shows per-step decode time for a small Qwen3-shaped model (8 layers, $d = 512$, random weights) running eagerly on the two-core CPU of the machine this chapter was written on. Absolute numbers are meaningless for a GPU, but the *shapes* are exactly the ones predicted above: without a cache, step time grows linearly with sequence length (so total time grows quadratically); with either cache, step time is nearly flat and dominated by fixed per-step costs at these lengths. You will reproduce this plot, and the batch-size and compilation sweeps, on your own hardware in Lab 11.

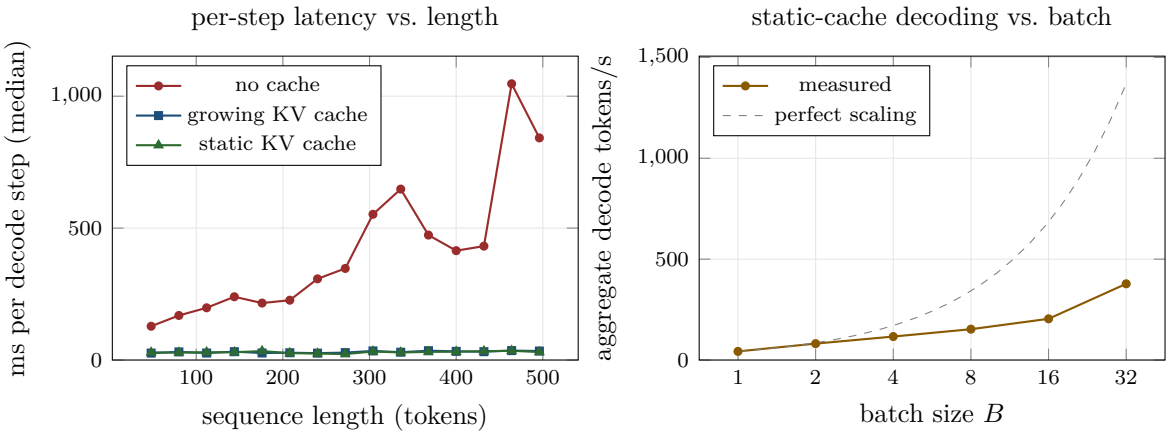


Figure 11.10: Measured, not modelled: a 6-layer, d=384 Qwen3-shaped model with random weights, fp32, eager PyTorch, one thread on a heavily shared two-core CPU (hence the noise). Left: 480 tokens after a 32-token prompt; total time was 203 s without a cache and 15 s with either cache. Right: aggregate throughput grows with batch size, sub-linearly because even on a CPU larger batches eventually stop being bound by reading the weights.

11.13 Summary

- Text generation is a loop: tokenize, run the model, keep only the last position's logits, select a token, append, repeat; stream tokens as they are produced and stop on the right EOS id.
- The tokenizer and chat template are part of the model's interface; vocabulary size trades shorter sequences against a larger embedding and LM head.

- The KV cache removes the quadratic recomputation of naive decoding; its size is $2LH_{kv}d_hb$ bytes per token (112 KiB for Qwen3-0.6B) and it, not the weights, limits batch size and context length.
- Prefill is compute-bound; decode is memory-bound with arithmetic intensity $\approx$ batch size for the weight GEMMs and $\approx$ GQA group size for attention.
- `torch.compile` removes Python/launch overhead and fuses kernels; static shapes (static cache) enable CUDA graphs; always warm up.
- Eager batch-1 decoding is launch-bound on fast GPUs (~1,400 kernels per step); fusion, avoiding host syncs, and CUDA graphs close the gap to the bandwidth bound.
- Measure TTFT, TPOT, throughput and goodput separately, with synchronization and percentiles.

11.14 Exercises

Exercise 11.1 (*Unknown words*). Encode five invented words and two sentences in a language other than English with the Qwen3 tokenizer. How many tokens per character do you get compared with ordinary English text? Why can a byte-level BPE tokenizer never produce an "unknown" token, and what is the cost of rare scripts in terms of KV-cache memory per character?

Exercise 11.2 (*Running on an accelerator*). Run Lab 11's `run.py` on a GPU (or Apple silicon) and on the CPU. Report TTFT, TPOT and tokens/s for the three decoders, eager and compiled. Which variant gains the most from moving to the GPU, and why?

Exercise 11.3 (*Counting redundant work*). For a prompt of P tokens and n generated tokens, derive the exact number of token-positions processed by the dense layers with and without a KV cache, and the number of query–key dot products in attention in both cases. Evaluate the ratios for $P = 1000$, $n = 100$ and for $P = 100$, $n = 1000$. Which workload benefits more from caching?

Exercise 11.4 (*KV budget planning*). An 80 GB H100 serves Qwen3-8B in bf16. Reserve 4 GB for activations and the CUDA context. (a) How many tokens fit in the KV cache? (b) How many concurrent 8k-token conversations? (c) How does the answer change with an fp8 KV cache, and with int4 weights (Chapter 13)?

Exercise 11.5 (*Critical batch size*). Using Eq. (11.2), compute the batch size at which the weight GEMMs of a decode step become compute-bound on an A100, H100 and RTX 4090 for bf16 weights and for int8 weights with bf16 compute. Why does weight-only quantization *lower* the critical batch size?

Exercise 11.6 (*The LM head at prefill*). Our model computes logits for all T positions. For a 4,096-token prefill on Qwen3-0.6B, how many FLOPs and how much activation memory

(bf16) does the full logits tensor cost, versus computing only the last row? Modify a copy of `Qwen3Model.forward` to accept a `last_only` flag and measure the difference.

Exercise 11.7 (*Cache copies*). Instrument `KVCache.update` to count bytes copied by `torch.cat`. Show that the total over n decode steps is quadratic in n, and compare with `StaticKVCache`. At what sequence length does the copy traffic exceed the weight traffic for Qwen3-0.6B?

Exercise 11.8 (*Recompilation hunting*). Compile the model and run `generate_kvcache` (growing cache) with `TORCH_LOGS=recompiles`. How many recompilations happen in 50 steps? Repeat with `generate_static`. Explain the difference in terms of Dynamo guards.

Exercise 11.9 (*Little's law*). A service has mean output length 300 tokens, TPOT 25 ms, TTFT 300 ms, and a maximum decode batch of 128. Estimate the maximum sustainable request rate. What happens to TTFT if requests arrive 20% faster than that rate for ten minutes?

Exercise 11.10 (*Determinism*). Generate 200 tokens greedily with batch size 1 and again as row 0 of a batch of 8 (other rows random prompts of the same length). Are the outputs identical? Explain why batch-dependent kernel selection and reduction order can change greedy outputs, and why this matters for evaluation reproducibility (Chapter 14) and for RL (Chapter 19).

11.15 Lab 11: Three decoders and a roofline

Lab 11: Three decoders and a roofline

Code: `code/labs/lab11_inference`

Goals

Implement cache-less, dynamically cached and statically cached greedy decoding; verify they produce identical tokens; quantify KV memory and the decode roofline; and measure TTFT/TPOT correctly.

Setup

```
cd code
python3 -m pytest -q labs/lab11_inference          # tests your starter.py
```

Tests use tiny GPT-2 and Qwen3 configurations with random weights and run on a CPU in seconds. `run.py` uses the real Qwen3-0.6B base checkpoint (download command in its header).

Tasks

1. `generate_nocache`: streaming greedy decoding that reprocesses the full sequence.
2. `generate_kvcache`: one prefill call, then single-token decode calls with `KVCache`.
3. `generate_static`: the same with a preallocated `StaticKVCache` of length $T + n$.
4. `kv_bytes_per_token` and `max_tokens_in_budget` for both GPT-2 (MHA) and Qwen3 (GQA) configs.
5. `decode_roofline` and `critical_batch` implementing the napkin model of Section 11.11.
6. `measure_generation`: TTFT, TPOT, total time and tokens/s with device synchro-

nization (the tests inject a fake clock).

Measurements

- `bench.py -exp length -plot len.png`: per-step latency vs. sequence length for the three decoders. Expect linear growth without a cache and a flat line with either cache.
- `bench.py -exp batch`: TPOT and aggregate tokens/s vs. batch size. Find the knee and compare it with `critical_batch` for your device (on a GPU with `-size 0.6B`).
- `bench.py -exp compile`: eager vs. `torch.compile` default vs. `reduce-overhead`, three repetitions each. Expect the first repetition to be slow and the cached, static-shape decoder to benefit most.
- `run.py`: the real model; confirm all three decoders emit identical text.

Deliverables

Passing tests; three plots (length, batch, compile) with axes labelled and hardware stated; a half-page note comparing your measured batch-1 decode speed with the roofline bound and explaining the gap.

Stretch goals

- Capture the static-cache decode step in a CUDA graph manually (`torch.cuda.CUDAGraph`) and compare against `reduce-overhead`.
- Add a `last_only` flag to the model forward and measure prefill memory savings.
- Plot measured tokens/s against `decode_roofline` predictions over batch sizes on your GPU and report the achieved fraction of bandwidth.

Chapter 12

Batching and Serving Engines

Chapter 11 ended with a sobering table: a single decoding stream uses well under 1% of a modern GPU's arithmetic, because every token drags the entire model through the memory system. The only way to amortize that cost is to decode many sequences at once. This chapter is about doing so well. We start where every practitioner starts: pad a few prompts into a rectangle and run them together. That simple idea already speeds up every workload of Part V (evaluation, self-consistency sampling, GRPO rollouts and distillation), and we walk through each one. Then we take the serving-system view. Requests arrive at random times with random lengths; rectangles waste compute on padding and memory on reservations, and fast requests get stuck behind slow ones. The fixes, iteration-level (continuous) batching, paged KV memory, prefix sharing, chunked prefill, and disaggregation of prefill from decode, form the core of every modern engine (vLLM, SGLang, TensorRT-LLM, TGI). We finish by building a working miniature of such an engine, `llmsys.serving`, that produces exactly the same tokens as our reference generator while batching, paging, sharing and preempting underneath.

12.1 Latency, throughput, and why batching helps

RM E.1

Two performance goals pull in different directions:

- **Latency**: how quickly *one* prompt gets its answer.
- **Throughput**: how many prompts (or tokens) are processed per unit time.

Single-sequence generation, as in Chapter 11, minimizes latency for one user, is easiest to read and debug, and is what we use to explain algorithms throughout the book. Batched generation stacks several sequences into one tensor and advances all of them with each forward pass (Fig. 12.2). It is a *throughput* optimization.

The reason batching works is the roofline argument of Section 11.11: the weight GEMMs of a decode step have arithmetic intensity $\approx B$ (in bf16), so on a GPU with a ridge point in the hundreds, raising B from 1 to 64 multiplies useful FLOPs per byte read by 64 while the step time barely changes. But batching is not free, and it is not always faster:

- Prompts have different lengths, and tensors are rectangles: shorter rows are padded, and the padding costs compute and memory without producing anything.

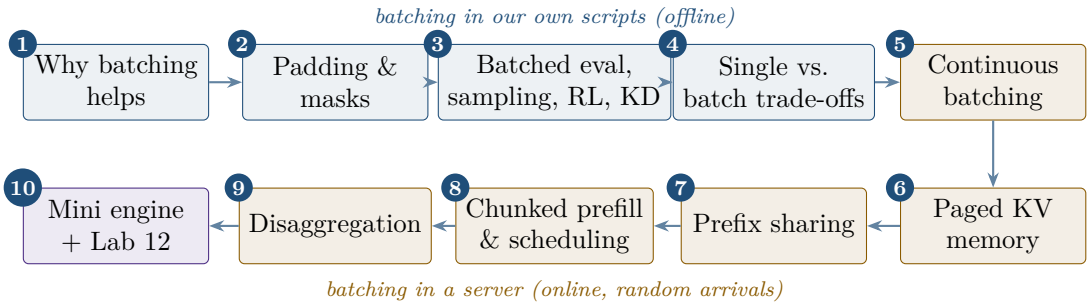


Figure 12.1: Roadmap. The top row batches known workloads offline; the bottom row designs an online engine for a stream of unpredictable requests.

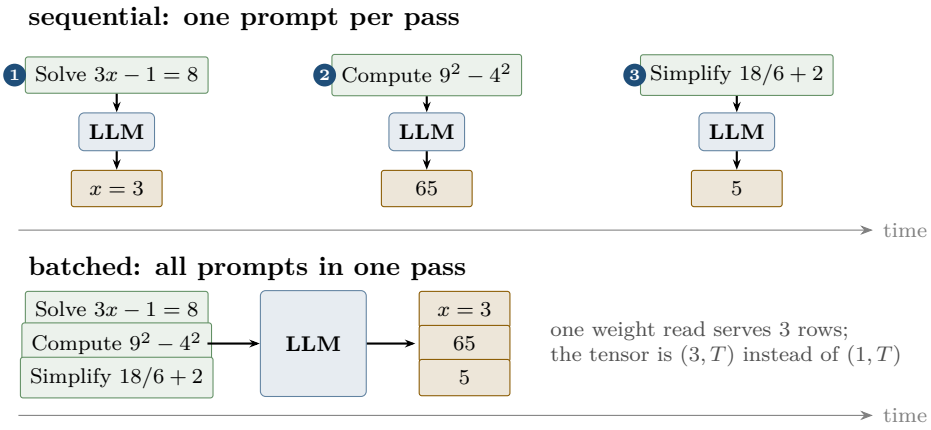


Figure 12.2: Sequential versus batched generation. In the batched case the prompts share every forward pass, so each weight read from memory is amortized over all rows.

- Rows finish at different times, so a naive batch runs until its longest row is done.
- Each row has its own KV cache, so batch size is bounded by memory, and attention over the cache does not get cheaper per row as the batch grows.
- On CPUs or for tiny models, the step is not dominated by weight reads in the first place, and the bookkeeping (masks, padding) can eat the gain.

Key Idea

Batching is a throughput optimization with a latency cost. Whether it pays off is decided by where the step time goes: amortizable work (weight reads, kernel launches) versus per-row work (KV reads, attention, padding).

12.2 Running batched generation

RM E.2

The core obstacle is that prompts tokenize to different lengths. To run them together we pad shorter rows with a placeholder id up to the batch's maximum length, and keep a boolean mask that records which positions are real. Which id we pad with does not matter, because padded positions will be masked out; reusing the end-of-text token is common.

Where should the padding go? For *generation* we pad on the **left**. Left padding aligns the last real token of every prompt in the final column, so after the prefill the next-token logits of

all rows sit at position -1, and each decode step appends one column to everyone (Fig. 12.4). With right padding, every row's "last real token" would be at a different index, and new tokens would have to be written into the middle of the tensor.

Listing 12.1: Left-padded batched generation with the core helper.

```python
from llmsys.generate import generate_batch

problems = ["2+2?", "3+3=6?", "Is 91 prime?"]
prompts = [tok.encode(render_prompt(p)) for p in problems]   # ragged lists
outs = generate_batch(model, prompts, max_new_tokens=64,
                      pad_id=tok.pad_token_id, eos_id=tok.pad_token_id)
for o in outs:                          # each row trimmed at its own EOS
    print(tok.decode(o))
```

Inside `generate_batch` the steps are: build the left-padded (B, T) id tensor and a $(B, T+n)$ validity mask; run one prefill over the whole rectangle with a KV cache; then for each step sample one token per row, append it, and run a decode step with the mask extended by one always-valid column.

Finished rows. When a row emits EOS, three strategies are possible (Fig. 12.3): (1) process prompts one by one; (2) keep the finished row in the batch, force it to keep emitting EOS so shapes stay aligned, and trim afterwards; (3) remove finished rows from the active batch, which saves compute but requires slicing the KV cache of every layer (and re-indexing masks) each time a row leaves. Strategy (2) is simplest and is what `generate_batch` does (it stops recording tokens for finished rows). Strategy (3) helps when output lengths are very skewed, and it is the seed of continuous batching (Section 12.6): once rows can *leave* a running batch, it is natural to let new rows *join* it.

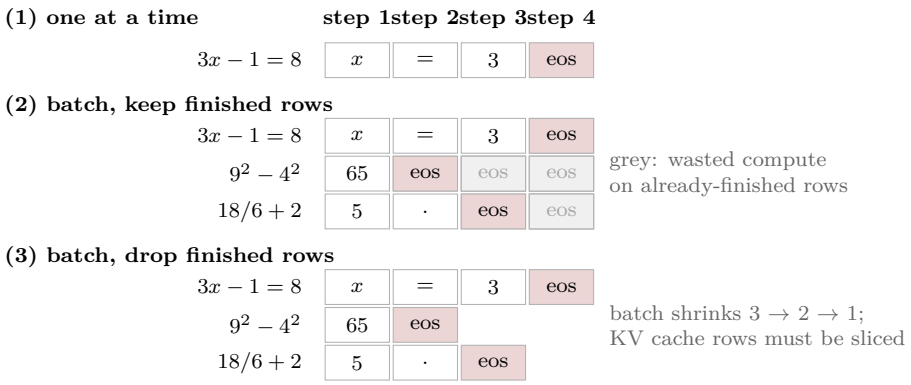


Figure 12.3: Three ways to handle sequences that finish at different steps.

12.3 Padding and attention masks

RM E.3

For a single unpadded prompt, the model builds the standard causal mask: position i may attend to positions $j \le i$. With padding we must additionally make sure no real token ever attends to a pad token, otherwise the pad embeddings leak into the computation and the batched output differs from the single-sequence output. The combined rule is

$$\text{forbidden}(b, i, j) \;=\; \underbrace{[\,j > i\,]}_{\text{causal}} \;\vee\; \underbrace{[\,\neg\text{valid}(b, j)\,]}_{\text{key is padding}}, \tag{12.1}$$

which is exactly what `llmsys.layers.build_mask` computes from a (B, T_k) validity mask (Fig. 12.4). Two details matter in practice:

- **Padded queries.** A pad position on the left has no valid key at all (its only allowed keys, those $\le i$, are all padding), so its softmax row is $\mathrm{softmax}(-\infty, \ldots) = \mathrm{NaN}$. These rows produce garbage that no real token ever reads, but NaNs must not propagate (e.g. through a later normalization or a $0 \cdot \mathrm{NaN}$ in a fused kernel). Our explicit attention zeroes them with `nan_to_num`; production kernels special-case fully-masked rows.
- **Decode steps extend the mask.** Each new column is valid for every row, so the validity mask grows by a column of `True` per step, while the left padding stays masked forever.

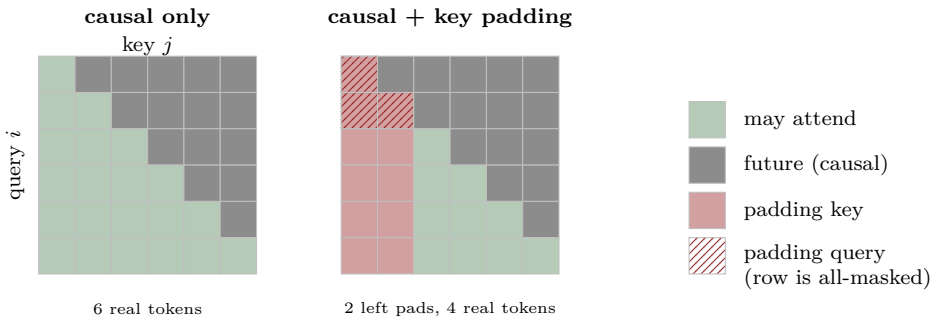


Figure 12.4: Masks for a 6-column batch row. Left: a row without padding. Right: the same layout with two left pads; real queries attend only to real, non-future keys, which reproduces the 4-token causal mask of the unpadded prompt in the lower-right corner.

Positions under left padding. Masking makes pad *keys* invisible, but the real tokens of a padded row now sit at column indices $p, p+1, \ldots$ instead of $0, 1, \ldots$, where p is the row's pad count. Whether that shift matters depends on the positional scheme.

Proposition 12.1 (RoPE is invariant to a uniform shift)**.** Let R_m be the RoPE rotation for position m. For any query at position m and key at position n, and any offset p, $\langle R_{m+p}\, q,\ R_{n+p}\, k\rangle = \langle R_m\, q,\ R_n\, k\rangle$.

Proof. Each 2-D frequency pair is rotated by an angle proportional to the position, so $R_m^\top R_n = R_{n-m}$ (rotations compose additively and $R_m^\top = R_{-m}$). Hence $\langle R_{m+p}q, R_{n+p}k\rangle = q^\top R_{m+p}^\top R_{n+p} k = q^\top R_{n-m} k$, which does not depend on p. □

So for Qwen3, left padding changes no attention score of a real token, and batched outputs equal single-sequence outputs up to floating-point reordering (a test in the core test suite, `tests/test_core.py`, checks exactly this). GPT-2 instead adds a *learned absolute* position embedding, so a row shifted by p sees different embeddings and its outputs change slightly. The fix is to pass explicit *position ids* per row, $\mathrm{pos}(b, t) = \max(0,\ t - p_b)$, which is what Hugging Face's `position_ids` argument and every production engine do. (Our GPT-2 class takes positions from the cache length; extending it to per-row positions is Exercise 12.1.)

Pitfall: Right padding for training, left padding for generation

Training and scoring (classification, distillation, log-probability computation) usually pad on the *right*, because labels are aligned with inputs and nothing is generated after the last

token; loss masks (label -100, Chapter 10) remove pad positions. Generation pads on the *left*. Mixing them up is the most common batched-generation bug: with right padding, `logits[:, -1]` reads the prediction after a *pad* token for every row but the longest.

12.4 Batching across the reasoning workflow

Every compute-heavy step of Part V benefits from batching, but the steps differ in an important way: some batch *different* prompts of different lengths (padding needed), others batch many samples of the *same* prompt (no padding, and a shared prefix that a smart engine computes only once). Table 12.1 summarizes; the subsections give the details.

Table 12.1: What gets batched in each workflow of the book.

Workflow	Rows of the batch	Padding?	Shared prefix?
MATH-500 evaluation (Chapter 14)	different problems	yes (left)	only the template
Self-consistency (Chapter 15)	k samples of one problem	no	whole prompt
GRPO rollouts (Chapter 17)	G rollouts of one problem	no	whole prompt
Distillation training (Chapter 20)	different (prompt, trace) pairs	yes (right)	no

RM E.4

12.4.1 Batched benchmark evaluation

Evaluating on MATH-500 means generating answers to 500 unrelated problems. With a batch size of B, the evaluator tokenizes B prompts, left-pads them, generates until every row has emitted EOS or hit the token limit, trims each row at its EOS, and hands the texts to the verifier. The only new knob is the batch size:

```
for i in range(0, len(problems), batch_size):
    chunk = problems[i:i + batch_size]
    prompts = [tok.encode(render_prompt(p["problem"])) for p in chunk]
    outs = generate_batch(model, prompts, max_new_tokens=2048,
                          pad_id=tok.pad_token_id, eos_id=tok.pad_token_id)
    results += [grade(tok.decode(o), p["answer"]) for o, p in zip(outs, chunk)]
```

The best batch size is hardware-dependent: increase it until either throughput stops improving or memory runs out (reasoning traces of thousands of tokens make the KV cache, not the weights, the limit). Because problems are sorted by nothing in particular, a batch's run time is set by its *longest* answer; sorting problems by prompt length reduces padding, but output lengths are unknown in advance, which is the deeper reason servers need continuous batching.

RM E.5

12.4.2 Batched self-consistency sampling

Self-consistency (Chapter 15) samples k answers to the *same* prompt at nonzero temperature and takes a majority vote. Here the batch dimension holds the k samples: we repeat the prompt k times, so every row has identical length and no padding or special mask is needed. The plain model class works as is. Each row needs its own random stream, which `torch.multinomial` provides naturally when sampling a (k, V) probability matrix.

Systems Lens: Do not prefill the same prompt k times

Repeating the prompt k times also repeats its prefill k times and stores k identical copies of its KV cache. The obvious fix is to prefill once and *fork* the cache: all samples share the prompt's KV blocks and only allocate private blocks for their own continuations. With paged memory (Section 12.7) this is a reference-count increment per block, and a copy-on-write if a sample must write into a partially filled shared block. For a 1,000-token problem and $k = 16$ samples of 500 tokens each, sharing cuts the KV footprint from $16 \times 1500 = 24{,}000$ to $1000 + 16 \times 500 = 9{,}000$ tokens and the prefill work by 16×. Lab 15 measures this.

12.4.3 Batched GRPO rollouts

RM E.6

GRPO (Chapter 17) samples a *group* of G rollouts per prompt and compares their rewards. Again the rows share one prompt, so batching rollouts needs no padding logic, and the same prompt-sharing argument applies. The batch size for rollouts trades throughput against memory: generating 8 rollouts of 512 tokens in parallel needs 8 KV caches; if memory is tight, generate the group in several smaller batches. Self-refinement (Chapter 16) is the counter-example in Part V: each round depends on the previous one, so it is inherently sequential per problem and only benefits from batching *across* problems, which requires more bookkeeping.

12.4.4 Batched distillation training

RM E.7

Distillation (Chapter 20) fine-tunes a student on (prompt, teacher reasoning trace) pairs of widely varying length, so we are back to batching unrelated sequences, this time for *training*. Sequences are right-padded, pad positions get loss label -100, and the attention mask hides pad keys. Training also stores activations for the backward pass, so memory per row is far higher than in evaluation, and the useful batch size is smaller. When lengths vary a lot, *packing* several short examples into one row with block-diagonal attention (Chapter 10) beats padding.

12.5 Single-sequence versus batched execution

RM E.8

Batched execution always costs more memory; whether it saves time depends on the shape of the work. Two quantities predict the outcome. The first is *padding efficiency*: for a batch with prompt lengths $\ell_1, \ldots, \ell_B$ padded to $\ell_{\max}$,

$$\eta_{\text{pad}} = \frac{\sum_b \ell_b}{B\,\ell_{\max}}, \tag{12.2}$$

and the analogous quantity for output lengths when the batch runs until its longest row finishes. The second is how much of the step time is amortizable, which the roofline tells us.

Napkin Math: When batching barely helps

Take 64 MATH problems with prompt lengths uniform in $[60, 240]$ and output lengths uniform in $[50, 1500]$. Expected padding efficiency of the prompts is $\approx 150/240 = 0.63$; for outputs, a batch of 64 runs to $\approx \max \approx 1480$ steps while the average row needs 775, so $\eta \approx 0.52$: half of all decode slots are wasted on finished rows. On a GPU the weight-read amortization ($\approx 64\times$) still wins handily. On a CPU running a small model, where step time is roughly proportional to rows, the same batch can be *slower* than sequential execution.

And in a training job that is already compute-bound, batching adds padding work without saving any memory traffic.

Our own CPU measurement in Fig. 11.10 showed sub-linear scaling (about 9× throughput at $B = 32$), and published GPU comparisons show large wall-clock reductions from batching evaluation and rollout generation, with smaller gains for distillation training, where the GPU is already busy with large matmuls at batch size one. The general rule: batch when the hardware is underutilized, and expect the gain to shrink as per-row work (padding, KV reads, activations) grows.

12.6 From static to continuous batching

Everything so far was *offline*: we knew all prompts in advance. A server faces an *online* problem: requests arrive at random times with unknown output lengths, and each user cares about their own latency. The batching we have used so far, called *static* or request-level batching, waits until B requests are available (or a timeout fires), runs them as a rectangle until the last one finishes, then starts the next batch. Three things go wrong:

1. **Head-of-line blocking.** A request arriving just after a batch started waits for the entire batch, including its longest member, before its own prefill begins: TTFT explodes.

2. **Idle slots.** Rows that finish early sit idle until the batch ends (Fig. 12.3, strategy 2).

3. **Padding and reservation.** Prompts are padded to the longest, and KV memory must be reserved for the maximum possible output length.

Iteration-level scheduling, introduced by Orca (Yu et al., 2022) and now called *continuous batching*, makes scheduling decisions at every forward pass instead of every batch. After each step, finished sequences leave and waiting requests join; the batch is a set of sequences, each at its own position (Fig. 12.5).

Continuous batching needs two capabilities that the rectangle formulation lacks. First, the forward pass must handle a *ragged* batch: some sequences are in prefill with hundreds of new tokens, others decode one token each, and each attends over its own history. Engines therefore flatten all scheduled tokens into one (N_{tok}, d) matrix, run every dense layer as one GEMM over all of them (no padding at all), and only split by sequence inside attention, using variable-length attention kernels that take per-sequence offsets ("varlen" or "cu_seqlens" interfaces in FlashAttention, FlashInfer). Second, KV memory must be allocated and freed at fine granularity as sequences come and go, which is the subject of the next section.

Systems Lens: Why flattening beats padding

In a flattened batch the dense layers cost $2N \cdot \sum_s n_s$ FLOPs where n_s is the number of *new* tokens of sequence s this step, exactly the useful work. Padding to a rectangle would cost $2N \cdot S \cdot \max_s n_s$: a single 2,000-token prefill in a batch of 63 decodes would inflate the GEMMs by 64×. Flattening also makes the LM head cheap: it is evaluated only on the last new token of each sequence (S rows, not $\sum_s n_s$).

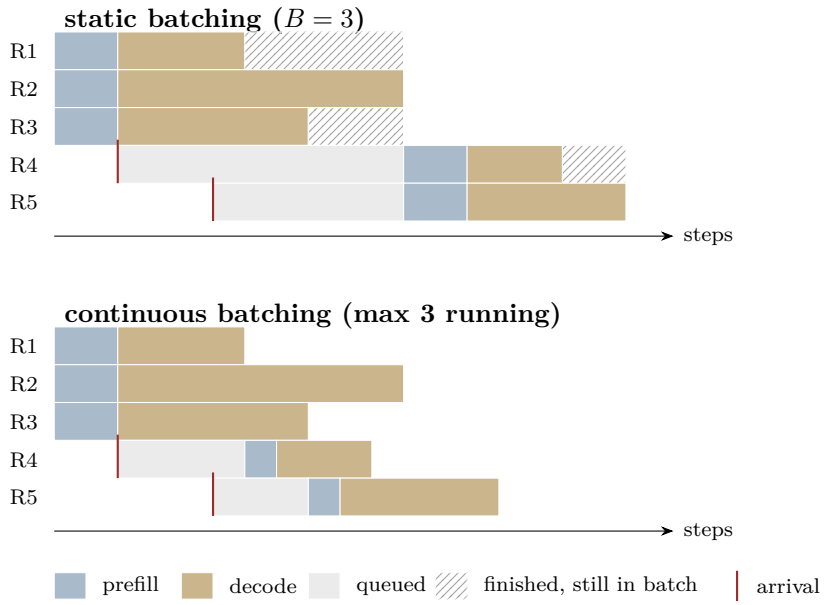


Figure 12.5: Five requests under static and continuous batching. Statically, R4 and R5 wait for the whole first batch (their TTFT includes R2's full generation) and finished rows idle. Continuously, R4 joins as soon as R1 leaves and R5 as soon as R3 leaves; the whole workload finishes at step 14 instead of 18. (In a real engine the joiners' prefills share steps with others' decodes; see Section 12.9.)

12.7 Paged KV memory

Before 2023, serving systems stored each sequence's KV cache in a contiguous tensor sized for the maximum sequence length. Kwon et al. (2023) measured that such systems used only a small fraction of their KV memory for actual tokens and proposed **PagedAttention**, the core of vLLM. The waste has three sources:

- **Reservation**: memory for future tokens is reserved at admission, although the output length is unknown and usually far below the maximum.
- **Internal fragmentation**: unused tail of each reserved region.
- **External fragmentation**: free memory split into gaps too small for a new contiguous allocation.

Napkin Math: How much memory reservation wastes

Reserve 2,048 tokens per request; actual lengths average 300. Utilization is $300/2048 \approx 15\%$: 85% of the KV pool is dead weight, and batch size is 6.8× smaller than it could be. With paging in 16-token blocks, only the last block of each sequence is partially filled, wasting on average 7.5 tokens per sequence: $7.5/307.5 \approx 2.4\%$. Lab 12 (Task 4) computes both for any length distribution.

The solution is borrowed from operating-system virtual memory (Fig. 12.6). KV memory is a pool of fixed-size *blocks* (pages), each holding the K and V vectors of `block_size` tokens for all layers and KV heads. Each sequence has a *block table* mapping logical block i (tokens $i \cdot \text{bs}$ to $(i+1) \cdot \text{bs} - 1$) to a physical block id. Blocks are allocated on demand when a sequence crosses a block boundary and returned to a free list when it finishes. The attention kernel follows the

table: the key of logical position p lives at physical slot

$$\text{slot}(p) = \text{table}[\lfloor p/\text{bs} \rfloor] \cdot \text{bs} + (p \bmod \text{bs}). \tag{12.3}$$

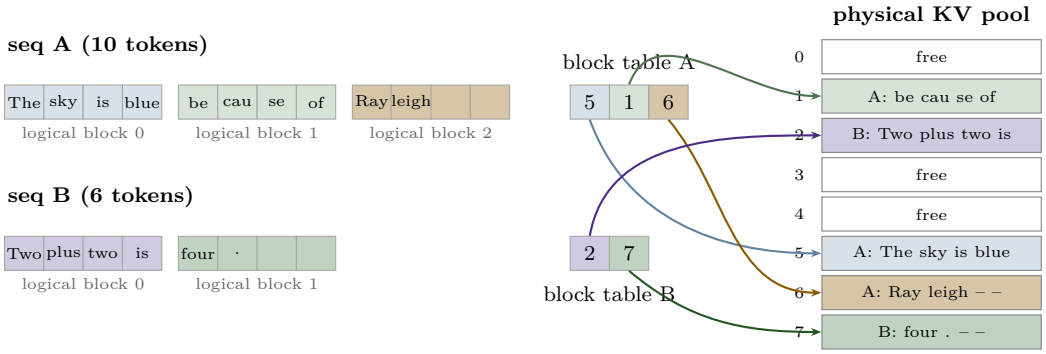


Figure 12.6: Paged KV memory. Logically contiguous sequences are stored in non-contiguous physical blocks; a per-sequence block table translates positions to slots (Eq. (12.3)). Only the last block of each sequence can be partially empty, and any free block can serve any sequence.

Choosing the block size. Small blocks waste less memory and allow finer sharing; large blocks make the attention kernel's gathers more efficient (longer contiguous reads, fewer table lookups) and shrink block tables. Engines typically use 16 tokens per block on GPUs. For Qwen3-8B a 16-token block holds $16 \times 144\,\text{KiB} = 2.25\,\text{MiB}$ across all layers; a 20 GB pool holds about 8,500 blocks, i.e. 136k tokens.

Systems Lens: The cost of indirection

Paging turns the attention kernel's KV reads from one contiguous stream into a gather over blocks. On a GPU this is nearly free if each block is a large contiguous chunk (each warp reads whole blocks) and the block table is small enough to sit in shared memory or registers. The PagedAttention paper reports only modest kernel-level overhead in exchange for several-fold higher batch sizes. Later systems (e.g. vAttention) instead use CUDA virtual-memory APIs to keep the KV cache virtually contiguous while mapping physical pages on demand, so unmodified attention kernels can be used.

12.8 Sharing KV state: prefix caching and copy-on-write

Once KV memory is managed in blocks with reference counts, *sharing* becomes cheap: two sequences whose block tables point to the same physical block simply increment its count. Three patterns are ubiquitous:

- **Parallel sampling** (k samples or G rollouts of one prompt): fork the prompt's blocks after one prefill.
- **Shared system prompts and few-shot templates**: thousands of requests start with the same hundreds of tokens.
- **Multi-turn chat and agents**: turn $t+1$ resends the full conversation of turn t plus a few new tokens. Without caching, the whole history is prefilled again every turn.

Hash-based automatic prefix caching. vLLM identifies every *full* block by a hash of its tokens *and* the hash of the previous block, so a hash identifies the entire prefix up to and including that block. Completed blocks are published in a hash table. A new request walks its prompt block by block, computing chained hashes, and reuses every block it finds until the first miss; it then only prefills the remainder. Blocks whose reference count drops to zero are not freed; they stay in the table as *evictable* and are reclaimed in least-recently-used order when the allocator runs dry. One subtlety: at least one prompt token must still be computed, because the model has to produce the logits for the first output token.

Listing 12.2: Prefix lookup with chained block hashes (llmsys.serving.block_manager).

```
def block_hash(prev_hash, tokens):
    return hash((prev_hash, tuple(tokens)))      # identifies the whole prefix

def lookup_prefix(self, tokens):
    bs, out, h = self.block_size, [], None
    for i in range((len(tokens) - 1) // bs):     # leave >= 1 token to compute
        h = block_hash(h, tokens[i * bs:(i + 1) * bs])
        b = self.hash_to_block.get(h)
        if b is None:
            break                                # first miss ends the match
        if self.ref_count[b] == 0:
            self.evictable.pop(b, None)          # revive a cached block
        self.ref_count[b] += 1
        out.append(b)
    return out
```

RadixAttention. SGLang (Zheng et al., 2024) organizes cached prefixes as a *radix tree* keyed by token sequences (Fig. 12.7). Each edge holds a run of tokens and the corresponding KV pages; a request matches the longest path from the root; insertion splits edges where sequences diverge; LRU eviction removes leaves first. The tree makes the cache structure explicit, supports matching at token rather than block granularity, and enables *cache-aware scheduling*: ordering the waiting queue so requests that share long prefixes run close together, which raises the hit rate. Hash chains and radix trees are two indexes over the same idea.

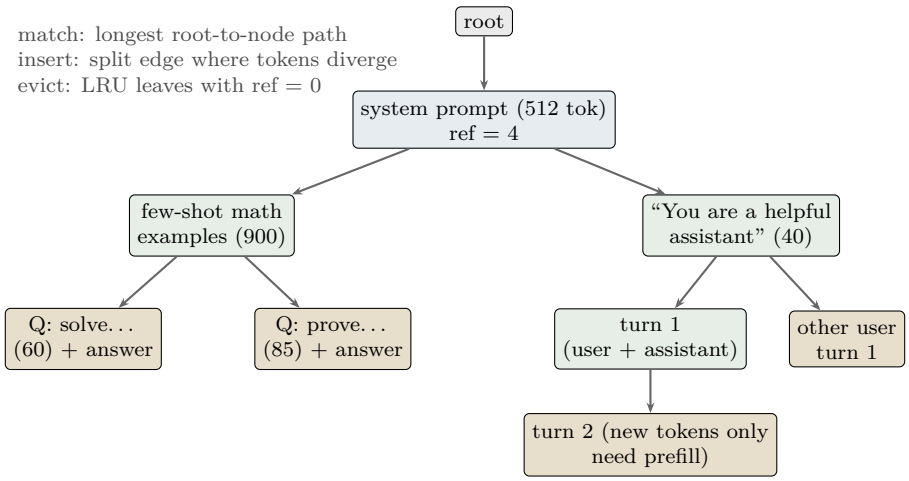


Figure 12.7: A radix tree of cached prefixes. Requests sharing the system prompt reuse its KV pages; the follow-up turn of a conversation matches everything up to and including turn 1, so only the new user message is prefilled. Leaves (orange) are the first candidates for eviction.

Copy-on-write. Sharing full blocks is safe because full blocks are immutable: new tokens are always appended to the *last* block. When a forked sequence (parallel sampling, beam search) shares a *partially filled* last block, the first sequence to append must copy it into a private block first, exactly like copy-on-write pages after `fork()` in an operating system.

Napkin Math: What prefix caching buys

In our simulator (Section 12.14), 600 requests to Qwen3-8B on an H100 at 8 requests/s, each with a 1,024-token system prompt drawn from four variants, plus a unique 64–512-token question: prefix caching reused 610k of 787k prompt tokens (78%), cut the prefill work by 4.5×, and reduced mean TTFT from 52 ms to 19 ms. Throughput was unchanged because the system was not saturated at that load; under saturation, saved prefill compute turns directly into throughput.

12.9 Chunked prefill and the token budget

Continuous batching mixes prefills and decodes in the same step, which creates a new problem. A 4,000-token prefill takes tens of milliseconds of compute; every decoding user sharing that step waits for it, so their inter-token latency spikes (a "generation stall"). Not admitting prefills while decodes run would starve new requests instead. Sarathi-Serve (Agrawal et al., 2024) resolves this with **chunked prefill** under a per-step **token budget** (Fig. 12.8):

1. Each step processes at most τ tokens in total (e.g. 512 or 2,048).
2. Running decodes are scheduled first, one token each.
3. The remaining budget is filled with *chunks* of pending prefills; a long prompt is split across several steps, carrying its KV cache forward between chunks.

Every step then has roughly the same, bounded cost: decode latency becomes predictable, and the prefill chunks "piggyback" on decode steps that would otherwise underuse the GPU's compute. The budget τ is a dial between TTFT (large τ finishes prefills sooner) and TPOT (small τ keeps steps short); it is usually chosen near the point where a step becomes compute-bound.

12.10 Scheduling, admission, and preemption

The scheduler decides, every step, which sequences run and how many tokens each processes. Its inputs are the waiting queue, the running set, the free block count, and the token budget. A minimal policy, the one our engine implements, is:

1. **Serve running sequences first** (oldest first), giving each min(remaining, budget) tokens: 1 for decodes, a chunk for unfinished prefills.
2. If a running sequence needs a new block and none is free, **preempt** the *youngest* running sequence (last admitted, least work lost) until space is available.
3. With the remaining budget, **admit** waiting requests first-come-first-served, after a prefix-cache lookup, as long as their first chunk fits in memory and the running-set limit allows. Do not admit in a step that had to preempt (memory is tight).

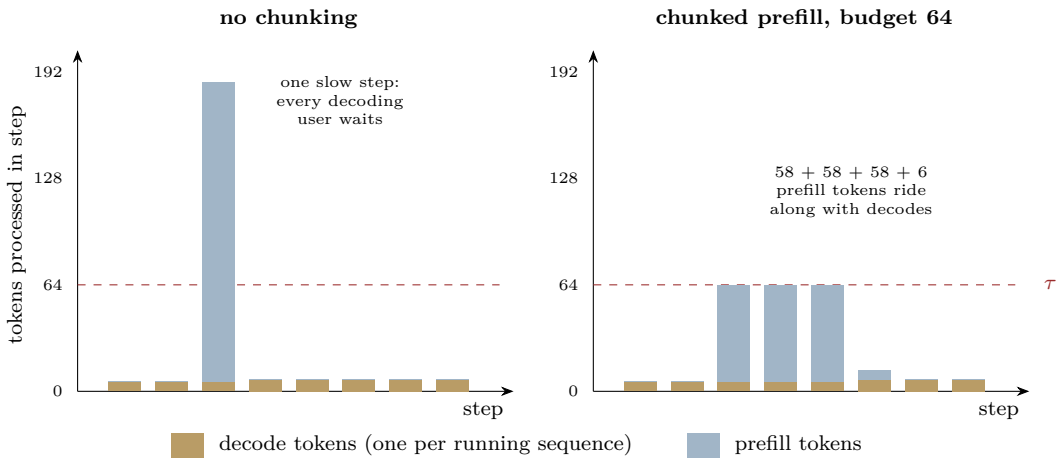


Figure 12.8: A 180-token prompt arrives while 6–7 sequences decode. Without chunking, step 3 processes 186 tokens and every decoding user sees one very slow token. With a 64-token budget the prompt is split into chunks of 58, 58, 58 and 6 tokens that ride along with the decodes (the prompt's first token appears one step later, but no step exceeds the budget).

Preemption: swap or recompute? A preempted sequence loses its KV blocks. *Swapping* copies them to host memory over PCIe and back later; *recomputation* discards them and later re-prefills the prompt *plus the tokens generated so far*. Per token, swapping costs $2\kappa/\beta_{\text{PCIe}}$ (out and back in) while recomputation costs about $2N/\Pi$ plus attention. For Qwen3-8B ($\kappa = 144\,\text{KiB}$) over 50 GB/s that is 5.9 µs versus 16.6 µs at peak FLOP/s: with GQA's small caches, swapping is competitive. For a model with full multi-head attention and 4× larger caches, recomputation wins up to tens of thousands of tokens. In practice, swapping many small blocks over PCIe is inefficient, swap traffic competes with other host transfers, and recomputation needs no host memory and may hit the prefix cache, so engines often default to recomputation. Our engine recomputes; Lab 12's tests verify that preempted sequences produce exactly the same tokens as unpreempted ones.

Systems Lens: Beyond FCFS

FCFS is fair in arrival order but ignores everything else. Production schedulers add: priorities and SLO classes (interactive vs. batch); *shortest-job-first* approximations using predicted output length to cut mean latency; fairness across tenants measured in served tokens (Virtual Token Counter, Sheng et al., 2024) to stop one heavy user from starving others; cache-aware ordering to maximize prefix hits; and admission control that rejects or queues requests early when the predicted TTFT would violate the SLO, which protects goodput rather than raw throughput.

12.11 Disaggregating prefill and decode

Chunked prefill makes prefill and decode coexist; disaggregation separates them. DistServe (Zhong et al., 2024) and Splitwise (Patel et al., 2024) run prefill and decode on *different* GPU pools: a prefill instance computes the prompt's KV cache and ships it to a decode instance, which then streams tokens (Fig. 12.9). The motivation is the asymmetry of Section 11.11:

- Prefill is compute-bound and its latency (TTFT) benefits from tensor parallelism and small batches; decode is bandwidth-bound and benefits from huge batches.

- Co-locating them couples their SLOs: any prefill in a step delays every decode in it.
- Separate pools can be sized, parallelized and even built from different hardware independently, maximizing goodput per GPU.

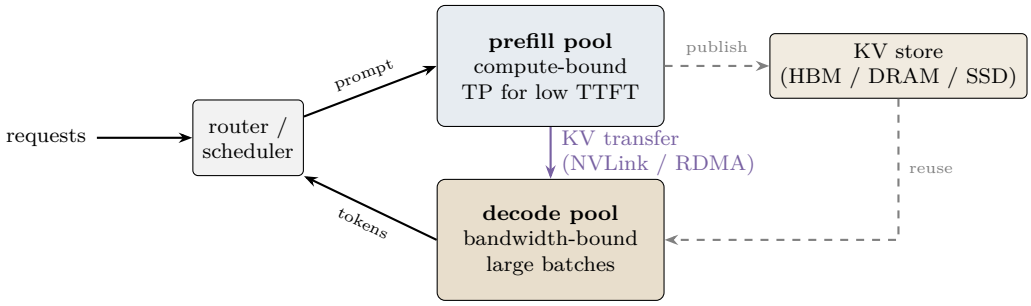


Figure 12.9: Disaggregated serving. Prefill and decode run on separately sized pools; the KV cache moves between them over a fast interconnect. Systems such as Mooncake add a distributed KV store so that caches computed once can be reused across requests and machines.

Napkin Math: Is the KV transfer affordable?

A 2,000-token prompt on Qwen3-8B produces $2000 \times 144\,\text{KiB} \approx 295\,\text{MB}$ of KV cache. Its prefill takes at least $2000 \cdot 15.1\,\text{GFLOP} / 989\,\text{TFLOP/s} \approx 31\,\text{ms}$ on an H100. Shipping 295 MB takes 0.33 ms over NVLink (900 GB/s) and $\approx 5.9\,\text{ms}$ over a 400 Gb/s RDMA NIC (50 GB/s): well below the prefill time, and it can be pipelined layer by layer as the prefill progresses. The transfer only becomes the bottleneck for models with very large per-token caches on slow networks, which is one more reason for GQA/MLA (Chapter 6) and fp8 KV caches.

Tiered KV caches. Prefix caches outgrow HBM quickly. Engines therefore spill evicted blocks to host DRAM or SSD and reload them on a hit, provided reloading is faster than recomputing: at 25 GB/s PCIe, reloading the 295 MB prefix above takes 12 ms, cheaper than its 31 ms prefill. Mooncake (Qin et al., 2024) builds a cluster-wide KV store around this trade-off.

12.12 Anatomy of a production engine

Before building our own engine, it helps to see what a production engine adds around the scheduler and the KV cache. Three pieces deserve a systems person's attention.

Sizing the KV pool at start-up. Engines do not ask the user how many KV blocks to allocate. At start-up they load the weights, run a *profiling* forward pass with the largest batch the scheduler may ever issue (e.g. `max_batched_tokens` tokens of prefill), record the peak activation memory, and give everything that remains, up to a utilization fraction, to the KV pool.

Napkin Math: KV pool for Qwen3-8B on one H100

Usable HBM at 90% utilization: $0.9 \times 80\,\text{GB} = 72\,\text{GB}$. Weights: 16.4 GB. Peak activations of an 8,192-token profiling step: the largest transient is the MLP intermediate, $8192 \times 12288 \times 2$ bytes $\approx 0.2\,\text{GB}$ per tensor; a few such tensors plus logits for the sampled rows, the CUDA context and CUDA-graph memory pools come to perhaps 4 GB. KV pool:

$72 - 16.4 - 4 \approx 51.6$ GB, i.e. $51.6\,\text{GB}/(16 \times 144\,\text{KiB}) \approx 21{,}900$ blocks $\approx 350{,}000$ tokens. At an average of 2,000 tokens per sequence, that is room for ≈ 175 concurrent sequences, comfortably past the batch size where decode throughput saturates in Table 11.1. For Qwen3-32B on the same GPU, weights alone take 65.5 GB and the KV pool shrinks to a few GB: such a model is served with tensor parallelism across GPUs largely to gain KV memory, not FLOPs (Chapter 13).

Process architecture. Python is single-threaded, and the GPU must never wait for it. Modern engines therefore split work across processes (Fig. 12.10): an API server handles HTTP, tokenization, chat templates and detokenization; an *engine core* process runs the scheduler and the step loop; GPU worker processes (one per GPU for tensor parallelism) execute the model. The processes communicate over shared memory or ZeroMQ sockets, so CPU-heavy work (tokenizing a 100k-token prompt, detokenizing thousands of streams, JSON encoding) overlaps with GPU execution. Inside the engine core, *asynchronous scheduling* prepares the inputs of step $t+1$ (scheduling, block allocation, building the slot mapping on the host and copying it to the device) while the GPU still executes step t, so the GPU sees back-to-back steps.

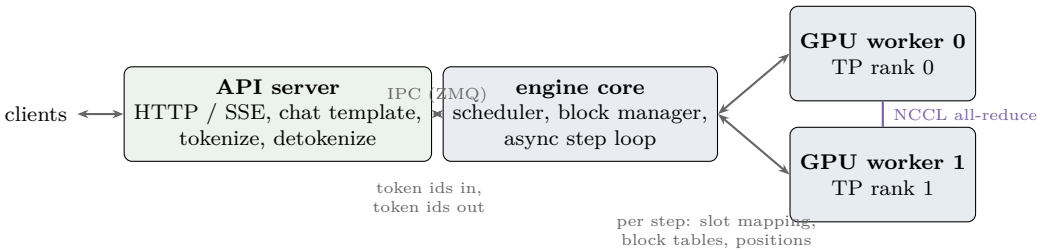


Figure 12.10: Process layout of a typical modern engine. CPU-bound request handling, scheduling and GPU execution run concurrently in separate processes.

Batched sampling with per-request parameters. Every request may carry its own temperature, top-p, top-k, seed, repetition penalty, stop strings and logprob requests. A Python loop over rows (as in our engine) is fine for a toy but serializes thousands of tiny kernels. Production samplers vectorize: parameters become per-row tensors, and one sort, one cumulative sum and one `multinomial` serve the whole batch (Listing 12.3). Greedy rows are temperature-0 rows handled by a final `where`.

Listing 12.3: Vectorized sampling with per-row temperature and top-p.

```
def sample_batch(logits, temperature, top_p, generator=None):
    # logits: (B, V); temperature, top_p: (B,) tensors. Returns (B,) ids.
    greedy = logits.argmax(dim=-1)
    t = temperature.clamp_min(1e-5)[:, None]
    probs = torch.softmax(logits.float() / t, dim=-1)
    sorted_p, idx = probs.sort(dim=-1, descending=True)
    cum = sorted_p.cumsum(dim=-1)
    sorted_p[(cum - sorted_p) >= top_p[:, None]] = 0.0        # per-row nucleus
    choice = torch.multinomial(sorted_p, 1, generator=generator)  # renormalizes
    sampled = idx.gather(-1, choice).squeeze(-1)
    return torch.where(temperature == 0, greedy, sampled)
```

The full sort over $V = 151{,}936$ entries per row is the expensive part. Faster samplers avoid it by rejection sampling or by first pruning to a top-k candidate set, and some draw samples via the "exponential race" trick, $\arg\max_i p_i/E_i$ with $E_i \sim \mathrm{Exp}(1)$, which needs neither a sort nor a cumulative sum (it samples from p exactly, so top-p must still be applied first). These tricks matter because at large batch sizes sampling can take a visible share of each decode step.

12.13 Building a mini engine

We now assemble the pieces into `llmsys.serving`, a teaching-sized engine of a few hundred lines that runs our GPT-2 and Qwen3 implementations unchanged (Fig. 12.11). It is deliberately simple (pure PyTorch, one process, a Python loop for per-sequence attention), but its structure mirrors vLLM's: a scheduler, a block manager, a model runner, and an engine loop.

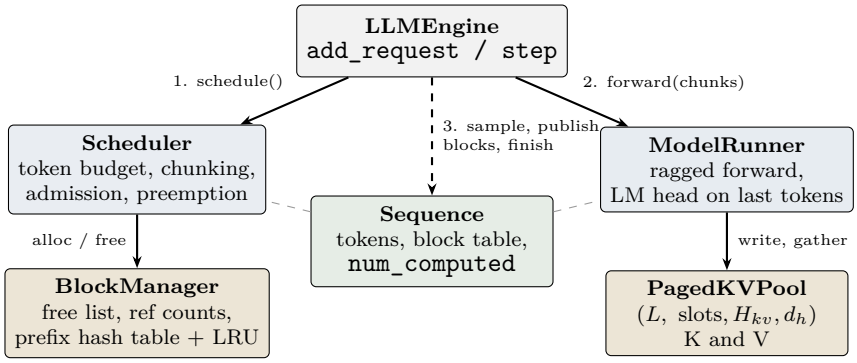


Figure 12.11: Architecture of `llmsys.serving`. The scheduler and block manager deal only in integers (token counts, block ids); only the model runner touches tensors.

One state variable. A sequence's tokens are its prompt plus its output so far, and `num_computed` counts how many of them already have K/V in the pool. Everything follows from remaining = len(tokens) − num_computed: a fresh request has remaining = prompt length (prefill); a decoding sequence has remaining = 1 (its last sampled token); a chunked prefill has remaining > 1 after its first chunk; a preempted sequence is reset to num_computed = 0 and simply re-prefills everything; a prefix hit sets num_computed to the matched length. The engine samples a token only when a sequence's chunk reaches the end of its known tokens. Prefill, decode, chunking, preemption and prefix reuse are thus one code path.

Listing 12.4: The engine step (abridged from llmsys/serving/engine.py).

```
def step(self):
    batch = self.scheduler.schedule()              # [(seq, n_tokens), ...]
    chunks = [(seq.tokens[seq.num_computed: seq.num_computed + n],
               seq.num_computed, seq.block_table) for seq, n in batch]
    logits = self.runner.forward(chunks)           # one ragged forward pass
    for i, (seq, n) in enumerate(batch):
        seq.num_computed += n
        self._publish_full_blocks(seq)             # feed the prefix cache
        if seq.remaining > 0:
            continue                               # mid-prefill: nothing to sample
        tok = int(sample_next_token(logits[i:i + 1], seq.params.temperature,
                                    seq.params.top_k, seq.params.top_p, seq.
    generator))
        seq.output.append(tok)
```

```
        if seq.is_finished():
            self.scheduler.finish(seq)            # releases (or caches) its blocks
```

The ragged forward pass. `ModelRunner.forward` concatenates the new tokens of all scheduled sequences into one vector, computes the physical slot of every token with Eq. (12.3), and walks the layers. All projections and the MLP run once on the whole (N_{tok}, d) matrix. Per layer, the new keys and values are scattered into the pool with `index_copy_`; attention then loops over sequences, gathering each one's history through its block table (Listing 12.5). Finally, the LM head runs only on the last new token of each sequence. RoPE is applied with per-token positions, so sequences at different positions coexist in one batch.

Listing 12.5: Reference paged attention for one sequence.

```
def paged_attention(q, k_pool, v_pool, block_table, ctx_len, block_size, group_size):
    n_new, H, D = q.shape                         # queries for the last n_new positions
    slots = torch.tensor(slot_mapping(block_table, range(ctx_len), block_size))
    k = k_pool.index_select(0, slots).repeat_interleave(group_size, dim=1)
    v = v_pool.index_select(0, slots).repeat_interleave(group_size, dim=1)
    scores = torch.einsum("qhd,khd->hqk", q.float(), k.float()) / math.sqrt(D)
    past = ctx_len - n_new                        # causal offset for chunked prefill
    mask = torch.triu(torch.ones(n_new, ctx_len, dtype=torch.bool), diagonal=past + 1)
    w = torch.softmax(scores.masked_fill(mask[None], float("-inf")), dim=-1)
    return torch.einsum("hqk,khd->qhd", w, v.float()).to(q.dtype)
```

The Python loop over sequences and the explicit gather are what a real engine replaces with a fused paged-attention kernel: one thread block per (sequence, head), streaming K/V blocks from the table into shared memory with an online softmax (Chapter 4), so the gathered K/V never materializes. Writing that kernel in Triton is a stretch goal in Lab 12 and Appendix A.

Correctness. The strongest test of an engine is that it is *invisible*: for greedy decoding, every request must receive exactly the tokens it would have received alone. Lab 12's tests run six requests with a shared prefix through the engine under three regimes (plenty of memory; a 5-token budget forcing chunked prefill; a 10-block pool forcing repeated preemption) and compare against `llmsys.generate.generate` token by token, for both GPT-2 and Qwen3. They also assert the block manager's invariant after every run: each block is either referenced, free, or cached-and-evictable, never two of these and never none.

12.14 Simulating serving policies at scale

Our engine runs on a CPU with toy models; to study policies at production scale we replace the model with a *cost model* and keep everything else. `llmsys.serving.simulator` feeds a Poisson request trace through the *same* `Scheduler` and `BlockManager` classes, and advances a virtual clock by

$$t_{\text{step}} = t_0 + \frac{1}{\epsilon}\max\left(\frac{2N\,n_{\text{tok}} + \text{attn}}{\Pi},\ \frac{Nb + \kappa\,C_{\text{ctx}}}{\beta}\right) \tag{12.4}$$

where n_{tok} is the number of tokens in the step, C_{ctx} the total context read by all scheduled sequences, t_0 a fixed per-step overhead (2 ms) and $\epsilon = 0.6$ an achieved fraction of peak.

Figure 12.12 compares three policies for Qwen3-8B on an H100 with a 20 GB KV pool (8,477 blocks of 16 tokens), prompts of 64–512 and outputs of 16–256 tokens: static batching ($B = 32$), continuous batching with paged memory, and continuous batching that *reserves* 4,096 tokens per request at admission (the pre-paging design).

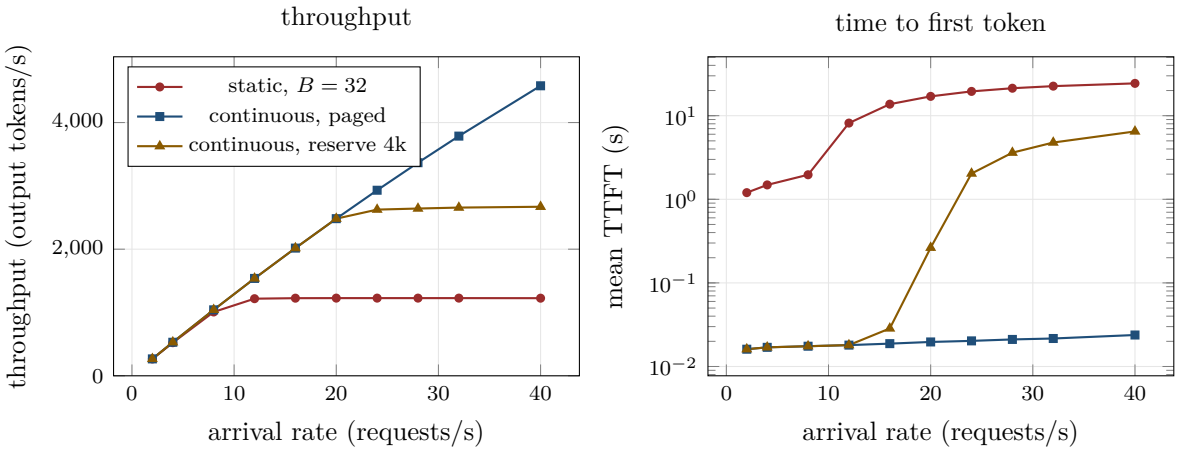


Figure 12.12: Simulated (not measured) serving of Qwen3-8B on one H100 under the roofline cost model, 600 Poisson requests per point. Static batching saturates at about 1,200 tokens/s with TTFTs of tens of seconds. Reservation-based continuous batching caps the running batch at about 30 sequences (20 GB/(4096 × 144 KiB)) and its TTFT explodes beyond 20 requests/s. Paged continuous batching keeps TTFT near 20 ms while throughput keeps growing (its mean running batch reaches 59).

Three lessons are visible. First, static batching's TTFT is dominated by waiting for the previous batch's longest request: seconds even at light load, and it collapses as soon as the arrival rate exceeds its fixed capacity. Second, continuous batching with reservation behaves identically to paging until KV memory, not compute, limits the batch, then saturates at a throughput set by memory/reservation. Third, with paging the batch grows with load (the mean running batch climbs from 3 to 59 sequences in this sweep) and TPOT rises only gently (10.3 ms to 13.3 ms), which is the roofline story of Table 11.1 playing out in a queueing system. Lab 12's `bench.py` reproduces this figure and lets you vary block size, budget, memory, and the prefix-sharing pattern.

Pitfall: Simulators are only as good as their cost model

Our cost model ignores attention-kernel efficiency at long contexts, sampling cost, CPU scheduling overhead, and the interference between prefill and decode inside one step beyond the max() term. Use it to compare policies and to find regimes worth measuring, then validate the conclusions on the real engine and hardware.

12.15 Summary

- Batching amortizes weight reads across rows; it raises throughput and costs memory and some latency. It does not help per-row work (KV reads, padding, activations).
- Batched generation left-pads prompts and masks pad keys; RoPE makes left padding exact, absolute position embeddings need per-row position ids. Training pads on the right.
- Evaluation and distillation batch different prompts (padding); self-consistency and GRPO batch samples of one prompt (no padding, shareable prefix).

- Serving engines schedule at iteration granularity (continuous batching), flatten ragged batches instead of padding, manage KV memory in pages with reference counts, share prefixes through hash chains or radix trees, bound step cost with chunked prefill, and preempt by recomputation or swapping.
- Disaggregating prefill and decode decouples TTFT from TPOT at the cost of moving KV caches, which is affordable on fast interconnects.
- `llmsys.serving` implements all of this in a few hundred lines and is token-for-token identical to the reference generator.

12.16 Exercises

Exercise 12.1 (*Per-row positions*). Extend `GPTModel.forward` (in your own subclass) to accept `position_ids` of shape (B, T). Show that `generate_batch` with left padding then reproduces single-sequence GPT-2 outputs exactly, and quantify how different the outputs are without the fix.

Exercise 12.2 (*Dropping finished rows*). Implement strategy (3) of Fig. 12.3: after each step, remove rows that emitted EOS from the input, the validity mask and every layer's KV cache. Measure the speed-up over strategy (2) for output lengths drawn from a geometric distribution with mean 200 and maximum 2,000.

Exercise 12.3 (*Padding efficiency*). Derive the expected output-padding efficiency η of a static batch of size B when output lengths are i.i.d. uniform on $[a, b]$. Evaluate for $B \in \{1, 8, 64\}$ with $a = 50$, $b = 1500$, and compare with continuous batching.

Exercise 12.4 (*Block size*). Using the simulator with the paged policy, sweep the block size over $\{1, 4, 16, 64, 256\}$ at a fixed KV budget. Plot throughput and the internal fragmentation. Then argue, from how a GPU kernel would gather blocks, why real engines do not use block size 1.

Exercise 12.5 (*Copy-on-write fork*). Add `fork(rid, n)` to the engine: it creates n child sequences that share the parent's blocks after its prefill, and implements copy-on-write for the partially filled last block. Verify with temperature sampling and fixed seeds that each child generates the same tokens as an independent request with the same seed, and report the memory saved for $n = 16$.

Exercise 12.6 (*Swap vs. recompute*). For Qwen3-8B on an H100 with PCIe 5.0 (about 50 GB/s effective), compare the per-token cost of swapping a preempted sequence's KV cache out and back in with the cost of recomputing it. Include the quadratic attention term of recomputation and find the sequence length at which the cheaper option changes, if any. Repeat for the same model with full multi-head attention (32 KV heads) and for an fp8 KV cache.

Exercise 12.7 (*Budget trade-off*). In the simulator, sweep the token budget $\tau \in \{64, 256, 1024, 4096\}$ at a load near saturation and plot mean TTFT against mean TPOT. Explain the shape of the trade-off curve in terms of Fig. 12.8.

Exercise 12.8 (*Multi-turn chat*). Write a trace generator for multi-turn conversations (each turn appends 50–200 user tokens to the previous conversation including the model's reply). Compare prefill tokens and TTFT with and without prefix caching as the number of turns grows. Why does the benefit grow with conversation length?

Exercise 12.9 (*Disaggregation sizing*). A workload has mean prompt 3,000 tokens and mean output 300 tokens on Qwen3-8B. Using the roofline bounds, estimate the ratio of prefill GPUs to decode GPUs that balances the two pools at a target TPOT of 30 ms. What changes for a reasoning workload with prompt 300 and output 8,000?

Exercise 12.10 (*Fairness*). Implement a Virtual-Token-Counter-style scheduler: track tokens served per tenant and always admit from the tenant with the smallest counter. Show with the simulator that a tenant sending 10× more requests no longer degrades the TTFT of a light tenant.

12.17 Lab 12: A paged, continuous-batching engine

Lab 12: A paged, continuous-batching engine

Code: `code/labs/lab12_engine`

Goals

Implement the four components that make `llmsys.serving` work, plug them into `LLMEngine`, and show that the engine is token-for-token identical to single-sequence generation under chunked prefill, prefix sharing and preemption. Then use the simulator to study policies at scale.

Setup

```
cd code
python3 -m pytest -q labs/lab12_engine        # grades starter.py (CPU, < 1 min)
python3 labs/lab12_engine/bench.py --help     # simulator experiments
```

Tasks

1. **BlockManager** (`_take`, `allocate`, `release`, `lookup_prefix`, `register`): free list, reference counts, evictable LRU set, chained-hash prefix lookup that always leaves one token to compute.
2. **Paged addressing**: `slot_mapping` (Eq. (12.3)) and `paged_attention` with GQA head expansion and a causal offset for chunked prefill. The test compares against `scaled_dot_product_attention` on a dense copy.
3. **Scheduler.schedule**: running sequences first under the token budget, preempt the youngest on block shortage, then FCFS admission with prefix lookup. `build_engine` wires your classes in.
4. **Fragmentation**: `waste_reserved`, `waste_paged`, `blocks_for_budget`.

Measurements

- `bench.py -exp policies`: reproduce Fig. 12.12 (static vs. continuous paged vs. continuous reserved) and report the knee of each curve.

- `bench.py -exp prefix`: requests sharing k system prompts of length P; plot prefill tokens and TTFT with and without prefix caching for $k \in \{1, 4, 64\}$.
- `bench.py -exp budget`: TTFT vs. TPOT as the token budget varies.
- `bench.py -exp engine`: run the real engine on a small random model on your machine; report steps, mean batch size, prefix hit rate and preemptions for a burst of 32 requests.

Deliverables

Passing tests; the four plots; a one-page design note explaining (i) why the engine's greedy outputs are identical to the reference even under preemption, and (ii) which component you would move to a GPU kernel first and why.

Stretch goals

- Copy-on-write `fork` for parallel sampling (exercise above) and its use for self-consistency.
- A Triton paged-attention decode kernel (one program per sequence and head, online softmax over blocks) that replaces `paged_attention` on GPU.
- Swap-based preemption to a CPU block pool, with a policy that chooses swap vs. recompute by sequence length.

Chapter 13

Making Inference Cheaper and Deploying It

Chapter 11 showed that a single decode step of a dense transformer is almost always *memory-bound*: to emit one token we stream every weight (and the whole KV cache) from HBM through the compute units, do roughly two FLOPs per byte, and wait. Chapter 12 attacked that inefficiency by putting more tokens behind each weight read: continuous batching, paged KV caches and prefix sharing. This chapter covers the remaining levers that a production inference stack pulls:

1. **Move fewer bytes.** Store weights (and KV) in 8, 4 or even fewer bits (*quantization*) so each decode step reads less memory.
2. **Get more tokens per expensive step.** Let a cheap model guess several tokens and verify them with *one* forward pass of the big model (*speculative decoding*) — without changing the output distribution at all.
3. **Split the model across devices** so that aggregate bandwidth grows with the number of GPUs (tensor, pipeline and expert parallel inference).
4. **Constrain and ship the output**: structured decoding, OpenAI-compatible servers, streaming, load testing — and, finally, a small browser chat interface around our own Qwen3 implementation.

For a systems reader the theme is familiar: every technique trades a little extra compute (dequantisation arithmetic, draft-model steps, collective communication) for a large reduction in the scarcest resource, HBM bandwidth per generated token.

13.1 Where the cost of a token goes

Let a dense decoder have N parameters stored with b_w bytes each, serve a batch of B sequences whose average context is T, and keep b_{kv} bytes per cached element. With L layers, G KV heads and head dimension d_h, one decode step must read

$$\text{bytes}_{\text{step}} \approx \underbrace{N\, b_w}_{\text{weights, once per step}} + \underbrace{2\, L\, G\, d_h\, T\, B\, b_{kv}}_{\text{KV cache of all sequences}} \tag{13.1}$$

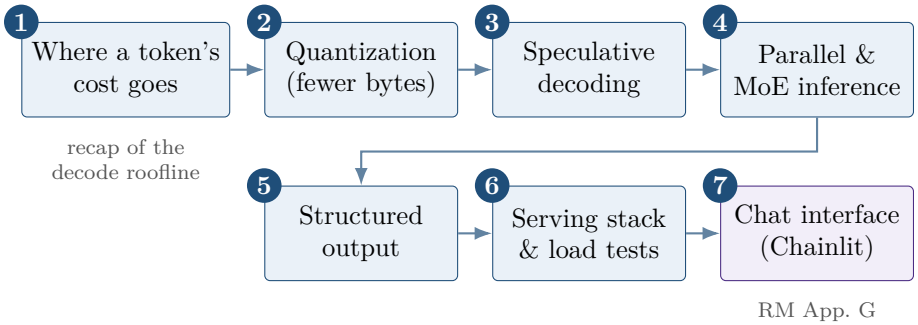


Figure 13.1: Roadmap of this chapter. Stages 2–4 reduce the cost per generated token; stages 5–7 turn an engine into something people can talk to.

and performs roughly $2NB$ FLOPs for the matrix multiplications plus $4LHd_hTB$ for attention scores and value mixing. Dividing by the hardware's bandwidth and peak FLOP rate gives two lower bounds on the step time; the larger one wins (Chapter 2's roofline). With $B = 1$ the arithmetic intensity is about one FLOP per byte, two orders of magnitude below the H100's ridge point of $989/3.35 \approx 295$ FLOP/byte.

Napkin Math: Qwen3 on one H100, batch size 1

Using `llmsys.flops` (Chapter 1) and 3.35 TB/s of HBM bandwidth:

model	params	bf16 weights	ms/token (bf16)	ms/token (int4)
Qwen3-0.6B	0.60 B	1.19 GB	0.36	0.09
Qwen3-8B	8.19 B	16.4 GB	4.89	1.22
Qwen3-32B	32.8 B	65.5 GB	19.6	4.89

These are *floors* from weight traffic alone: 205 tokens/s for Qwen3-8B in bf16, 818 tokens/s if the weights were 4-bit. No kernel can beat them at batch size 1; the only escapes are fewer bytes (this section), more tokens per byte (batching, speculation), or more bandwidth (more GPUs).

The KV term matters as soon as batches and contexts grow. For Qwen3-8B the cache costs $2 \cdot 36 \cdot 8 \cdot 128 \cdot 2$ bytes $= 147.5$ KB per token. A batch of 64 requests at 4096 tokens of context holds $64 \cdot 4096 \cdot 147.5\,\text{KB} \approx 38.7$ GB of KV, more than twice the weights. Equation (13.1) then predicts a 16.4 ms step (55 GB at 3.35 TB/s), and the step is *still* memory-bound: its arithmetic intensity is only about 20 FLOP/byte.

Systems Lens: Three levers, three different bottlenecks

- **Weight quantization** shrinks the first term of Eq. (13.1). It helps most at small batch and short context, where weights dominate.
- **KV quantization** (and GQA, MLA, paging, eviction) shrinks the second term, which dominates at large batch or long context, i.e. exactly the regime of high-throughput serving and long reasoning traces.
- **Speculative decoding** keeps the bytes the same but amortises them over several accepted tokens. It helps most when the GPU is idle-ish (small batch) and helps little when large batches already make the step compute-bound.

Always ask which term of Eq. (13.1) dominates *your* workload before choosing a technique.

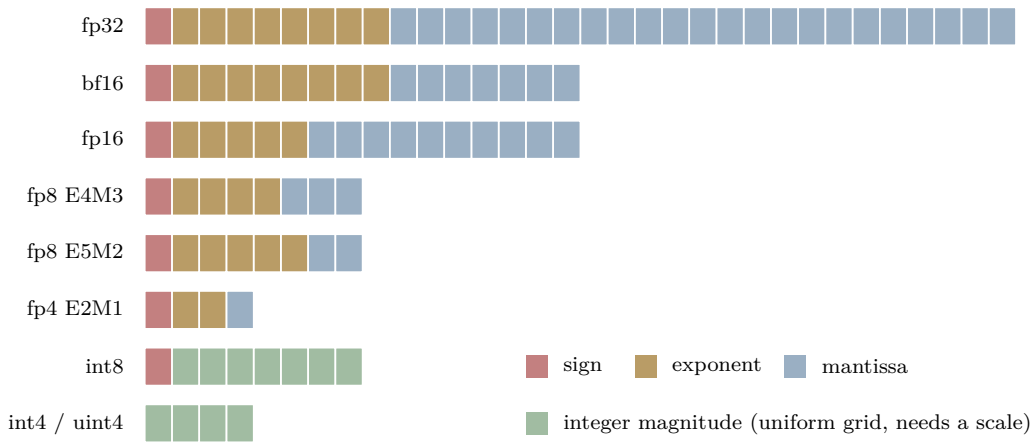


Figure 13.2: Bit layouts of the formats used for LLM inference. Exponent bits buy dynamic range; mantissa or integer bits buy resolution. Formats narrower than 8 bits are only usable together with fine-grained (per-group or per-block) scales.

13.2 Quantization fundamentals

13.2.1 Number formats

A quantized model stores tensors in a narrow format and keeps enough metadata (scales, zero points) to map them back to real numbers. Figure 13.2 lays out the formats you will meet. Floating-point formats spend bits on an exponent, which buys dynamic range: bf16 keeps fp32's 8-bit exponent and is therefore the default training and inference format. FP8 comes in two flavours: E4M3 (more precision, max ±448) for weights and activations, and E5M2 (more range, max ±57344) traditionally for gradients. Integer formats spend every bit on resolution but have *uniform* spacing, so they need an explicit scale to place that grid where the values are.

13.2.2 Uniform affine quantization

A b-bit uniform quantizer maps a real tensor x to integer codes with a scale $s > 0$ and a zero point z:

$$q = \mathrm{clamp}\Big(\lfloor x/s \rceil + z,\ q_{\min},\ q_{\max}\Big), \qquad \hat{x} = s\,(q - z). \tag{13.2}$$

Symmetric (absmax) quantization sets $z = 0$ and $s = \max|x|/(2^{b-1} - 1)$, which suits roughly zero-centred weights and makes the integer GEMM cheap (no zero-point correction terms). **Asymmetric** quantization uses the full range $[\min x, \max x]$ with $s = (\max x - \min x)/(2^b - 1)$ and $z = \lfloor -\min x/s \rceil$, which wastes no codes on one-sided distributions.

If values are spread across the grid, the rounding error is roughly uniform on $[-s/2, s/2]$ and has variance $s^2/12$. Each extra bit halves s, which gives the familiar rule of thumb of about 6 dB of signal-to-quantization-noise ratio per bit. The catch is the word *spread*: one outlier stretches $\max|x|$, inflates s for *everything else* in the same group, and pushes small values to zero. Nearly every modern quantization method is a way of fighting outliers.

Listing 13.1: Symmetric absmax and asymmetric group-wise quantization (from `llmsys/quant.py`).

```
def quantize_absmax(w, bits=8, dim=-1):
    qmax = 2 ** (bits - 1) - 1
    amax = w.abs().max() if dim is None else w.abs().amax(dim=dim, keepdim=True)
    scale = (amax / qmax).clamp(min=1e-12).float()
```

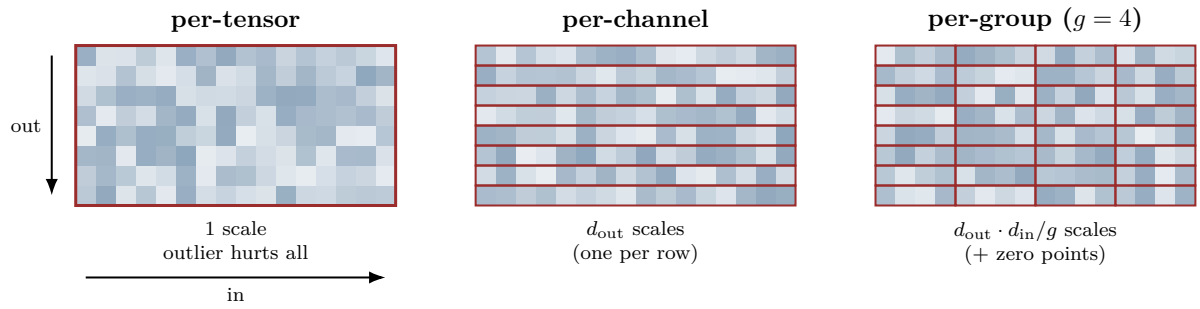


Figure 13.3: Quantization granularity for a weight matrix. Red boxes enclose the elements that share one scale. Finer groups track local magnitudes better at the cost of more metadata.

```
    q = torch.clamp(torch.round(w.float() / scale), -qmax, qmax).to(torch.int8)
    return q, scale

def quantize_groupwise(w, bits=4, group_size=128):
    out_f, in_f = w.shape
    qmax = 2 ** bits - 1
    g = w.float().reshape(out_f, in_f // group_size, group_size)
    wmin, wmax = g.amin(dim=-1), g.amax(dim=-1)
    scale = ((wmax - wmin) / qmax).clamp(min=1e-12)
    zero = torch.round(-wmin / scale).clamp(0, qmax)
    q = torch.clamp(torch.round(g / scale[..., None]) + zero[..., None], 0, qmax)
    return q.to(torch.uint8).reshape(out_f, in_f), scale, zero
```

13.2.3 Granularity: per-tensor, per-channel, per-group

Figure 13.3 shows the three granularities for a weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ (one row per output channel, as stored by `nn.Linear`). Per-tensor uses one scale and is fragile. Per-output-channel uses one scale per row and is the default for int8 weights. Since $y_j = \sum_i W_{ji} x_i$, a per-row scale factors out of the dot product and can be applied once to the GEMM result. Per-group splits each row into groups of g consecutive input channels (typically $g = 128$, sometimes 32 or 64), each with its own scale and zero point. This is what makes 4-bit weights usable.

Napkin Math: The metadata tax of group-wise int4

With $g = 128$, an fp16 scale and a 4-bit zero point per group cost $(16 + 4)/128 \approx 0.16$ extra bits per weight; with an fp16 zero point, $32/128 = 0.25$ bits. So "4-bit" Qwen3-8B actually stores about 4.16–4.25 bits per parameter: roughly 4.3 GB instead of the naive 4.1 GB. With $g = 32$ the tax rises to 0.5–1 bit, which is why tiny groups are reserved for formats with very cheap (8-bit exponent-only) scales such as the microscaling (MX) formats.

13.2.4 Packing sub-byte values

Memory is byte-addressed, so two 4-bit codes share one byte. Our `pack_int4` places even columns in the low nibble and odd columns in the high nibble (Fig. 13.4). Production kernels choose an interleaving that matches how their GEMM loads registers. The Marlin kernel, for instance, pre-shuffles weights offline so that one 128-bit load feeds tensor-core fragments directly. Packing order is therefore part of the *kernel's* contract, not the model's, and checkpoints are often repacked at load time.

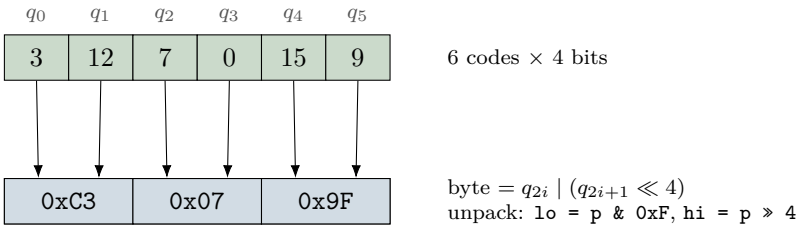


Figure 13.4: Packing pairs of 4-bit codes into bytes: the low nibble holds the even element and the high nibble the odd element.

13.2.5 Weight-only quantization (W8A16, W4A16)

RM D.5

In weight-only schemes the activations stay in bf16 and the weights are dequantized *inside* the GEMM: a kernel loads packed integers from HBM, expands them in registers, multiplies by the group scale and feeds tensor cores in bf16. Because decode is memory-bound, the extra ALU work is free and the step time drops nearly in proportion to the weight bytes. Listing 13.2 is the reference layer we use in the lab. It dequantizes the full weight in PyTorch, which is correct but *not* faster: it materialises the fp32 weight in memory, the very traffic we wanted to avoid. Appendix A writes the fused kernel.

Listing 13.2: A weight-only quantized linear layer (reference semantics, not a fast kernel).

```
class QuantLinear(nn.Module):
    def __init__(self, linear, bits=8, group_size=128):
        super().__init__()
        w = linear.weight.data
        if bits == 8:                          # per-output-channel symmetric
            q, s = quantize_absmax(w, 8, dim=-1)
            self.register_buffer("qweight", q); self.register_buffer("scale", s)
        else:                                  # 4-bit asymmetric groups, packed
            q, s, z = quantize_groupwise(w, 4, group_size)
            self.register_buffer("qweight", pack_int4(q))
            self.register_buffer("scale", s.half()); self.register_buffer("zero", z)
        ...
    def forward(self, x):
        w = self.dequantized_weight(x.dtype)   # a fused kernel never builds this
        return x @ w.T + (0 if self.bias is None else self.bias)
```

Pitfall: Quantizing the wrong layers

Embeddings and the output head are often kept in bf16. The head maps to a vocabulary of about 150k entries, so its errors land directly on the next-token distribution. Our `quantize_model` skips `out_head` by default. Norm weights and biases are a negligible fraction of bytes; never quantize them.

13.2.6 GPTQ: rounding with error feedback

Round-to-nearest (RTN) treats each weight independently. But what we care about is the layer's *output* on realistic inputs. Given calibration activations $X \in \mathbb{R}^{n \times d_{\text{in}}}$ for a layer, GPTQ (Frantar et al., 2023) minimises the output error row by row:

$$\min_{Q} \; \|XW^\top - XQ^\top\|_F^2 \;=\; \sum_j (\boldsymbol{w}_j - \boldsymbol{q}_j)^\top H \, (\boldsymbol{w}_j - \boldsymbol{q}_j), \qquad H = X^\top X. \tag{13.3}$$

It builds on Optimal Brain Surgeon: quantize one input column i, and update the remaining unquantized columns of every row to absorb the error, using the inverse Hessian:

$$\boldsymbol{w}_{:,>i} \leftarrow \boldsymbol{w}_{:,>i} - \frac{w_{:,i} - q_{:,i}}{[H^{-1}]_{ii}} [H^{-1}]_{i,>i}. \tag{13.4}$$

GPTQ's engineering contributions make this fast enough for 100B-parameter models. It quantizes columns in a fixed order, which makes H^{-1} the same for every row. It uses a Cholesky factorisation of H^{-1}, whose rows provide the update coefficients. And it uses lazy block updates, which turn many rank-1 updates into GEMMs. Listing 13.3 is the unblocked version.

Listing 13.3: GPTQ without lazy batching. On correlated calibration inputs it gives lower output error than RTN on the same grid.

```python
def gptq_quantize(w, x, bits=4, damp=0.01):
    W = w.clone().float(); d = W.shape[1]
    H = x.float().T @ x.float()
    H += damp * torch.mean(torch.diag(H)) * torch.eye(d)        # dampening
    Hinv = torch.linalg.cholesky(
        torch.cholesky_inverse(torch.linalg.cholesky(H)), upper=True)
    qmax = 2 ** (bits - 1) - 1
    scale = (W.abs().amax(dim=1) / qmax).clamp(min=1e-12)      # per-row grid
    Q = torch.zeros_like(W)
    for i in range(d):
        col = W[:, i]
        q = torch.clamp(torch.round(col / scale), -qmax, qmax) * scale
        Q[:, i] = q
        err = (col - q) / Hinv[i, i]
        W[:, i + 1:] -= err[:, None] * Hinv[i, i + 1:][None, :]  # error feedback
    return Q
```

AWQ (Lin et al., 2024) starts from a different observation: a small fraction of weight input channels matter far more because their *activations* are large. Instead of keeping those channels in high precision (which breaks the uniform kernel), AWQ multiplies salient input channels of W by a factor $s > 1$ and divides the corresponding activations by s. The product is unchanged, but the important weights now occupy more of the quantization grid. The factors are searched on calibration data. Both GPTQ and AWQ produce ordinary group-wise int4 checkpoints that the same W4A16 kernels can serve.

13.2.7 Quantizing activations: outliers and SmoothQuant

Weight-only schemes do nothing for compute-bound prefill or large-batch decode. There we want int8 (or FP8) *matmuls*, which means quantizing activations too (W8A8). LLM activations are hard. Beyond a few billion parameters, a handful of hidden dimensions carry values 10–100× larger than the rest, consistently across tokens. LLM.int8() (Dettmers et al., 2022) handles them with a mixed-precision decomposition: outlier columns in fp16, the rest in int8. SmoothQuant (Xiao et al., 2023) instead migrates the difficulty into the weights with a per-input-channel factor:

$$Y = XW^\top = (X\,\mathrm{diag}(\boldsymbol{s})^{-1})(W\,\mathrm{diag}(\boldsymbol{s}))^\top, \qquad s_j = \frac{\max|X_{:,j}|^\alpha}{\max|W_{:,j}|^{1-\alpha}}. \tag{13.5}$$

With $\alpha \approx 0.5$ the per-channel ranges of activations and weights are balanced. The division by $\boldsymbol{s}$ is folded offline into the preceding RMSNorm's scale, so it costs nothing at runtime (Fig. 13.5).

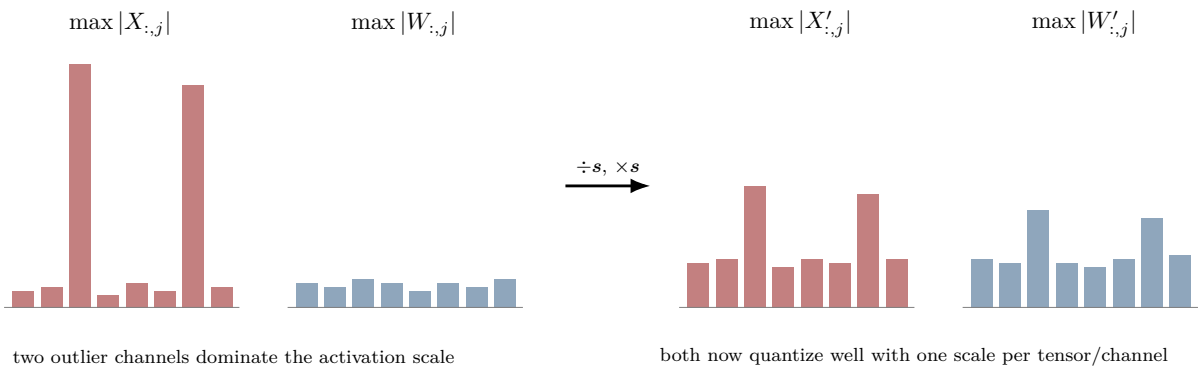


Figure 13.5: SmoothQuant moves per-channel magnitude from the activations into the weights. The layer computes the same function, but both operands become int8-friendly.

13.2.8 FP8 and microscaling formats

Hopper-class GPUs execute FP8 GEMMs at twice the bf16 rate, and Blackwell adds FP4. FP8 E4M3 has enough dynamic range that a single per-tensor scale (computed from the running absmax, possibly from the previous iteration: "delayed scaling") is often sufficient for W8A8 inference with negligible quality loss. Below 8 bits even floating point needs block scales. The OCP microscaling formats (MXFP8/MXFP6/MXFP4) attach one shared power-of-two (E8M0) scale to every block of 32 elements. NVIDIA's NVFP4 uses blocks of 16 with an FP8 scale plus a per-tensor scale. Hardware applies the block scales inside the tensor-core instruction, so the fine granularity no longer costs a separate dequantization pass.

Listing 13.4: Simulating FP8 storage with a per-tensor scale.

```
def fp8_roundtrip(t, fmt="e4m3"):
    dtype = {"e4m3": torch.float8_e4m3fn, "e5m2": torch.float8_e5m2}[fmt]
    scale = (t.abs().max() / torch.finfo(dtype).max).clamp(min=1e-12)
    return (t / scale).to(dtype).to(torch.float32) * scale
```

13.2.9 Quantizing the KV cache

Equation (13.1) shows that the KV cache dominates long-context and large-batch decode, so quantizing it pays twice: fewer bytes per step, and more sequences fit in memory, which raises the achievable batch size. Values behave like ordinary activations and quantize well per token. Keys are different. Empirically, a few *channels* of K are consistently large across tokens, so KIVI (Liu et al., 2024) quantizes keys per channel and values per token. FP8 KV caches are a common production default; int4 and 2-bit caches need care, usually by keeping a recent window in full precision. Since each attention kernel reads the cache, the dequantization must be fused into the attention kernel as well (Appendix A).

Napkin Math: Which bytes to cut for a throughput server

Qwen3-8B, batch 64, 4096-token contexts, one H100 (numbers from Section 13.1):

configuration	weights	KV cache	step floor
bf16 weights, bf16 KV	16.4 GB	38.7 GB	16.4 ms
int4 weights, bf16 KV	4.1 GB	38.7 GB	12.8 ms
bf16 weights, fp8 KV	16.4 GB	19.3 GB	10.7 ms
int4 weights, fp8 KV	4.1 GB	19.3 GB	7.0 ms

In this regime KV quantization alone beats 4-bit weights. And the first row would not even fit next to activations and CUDA graphs on an 80 GB card.

13.2.10 Did quantization break the model?

RM D.5

Measure the quantized model against the reference model, not just against a benchmark score:

- **Per-token KL divergence** $\mathrm{KL}(p_{\text{ref}} \,\|\, p_{\text{quant}})$ on held-out text. It is sensitive, cheap, and needs no labels.
- **Perplexity delta**, the classic metric. It hides catastrophic errors on rare tokens.
- **Task accuracy with generation** (MATH-500, Chapter 14). Reasoning models are especially sensitive: small per-token errors compound over traces of thousands of tokens, and a flipped digit is a wrong answer.
- **Flip rate**: the fraction of benchmark items whose correctness changed. Equal average accuracy can hide many flips in both directions.

Deeper Dive: Larger models: quantize or go smaller?

For a fixed memory budget, a larger model at 4 bits usually beats a smaller model at 16 bits. This was a central finding of the "k-bit inference scaling laws" study (Dettmers & Zettlemoyer, 2023), whose optimum sat around 4 bits. Practically: if Qwen3-8B in bf16 does not fit your card but Qwen3-14B in int4 (about 7.5 GB of weights) does, try the latter first. Below about 3 bits, quality typically falls off a cliff unless the method is specialised (vector quantization, quantization-aware fine-tuning).

13.3 Speculative decoding

At batch size 1, verifying five tokens costs about as much as generating one. The forward pass over a (1×5) block reads the same weights as a (1×1) step, and the extra FLOPs are invisible below the roofline ridge. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) exploits this. A small *draft* model q proposes γ tokens autoregressively (cheap steps). The large *target* model p then scores all of them in *one* forward pass, and a rejection-sampling rule keeps the longest acceptable prefix plus one extra token (Fig. 13.6).

13.3.1 The acceptance rule and why it is exact

Let $p(\cdot)$ and $q(\cdot)$ be the target and draft distributions at some position (after temperature and top-k/top-p filtering, the distributions we would actually sample from). The draft proposes $x \sim q$. We accept it with probability $\min(1, p(x)/q(x))$. If we reject, we sample a replacement

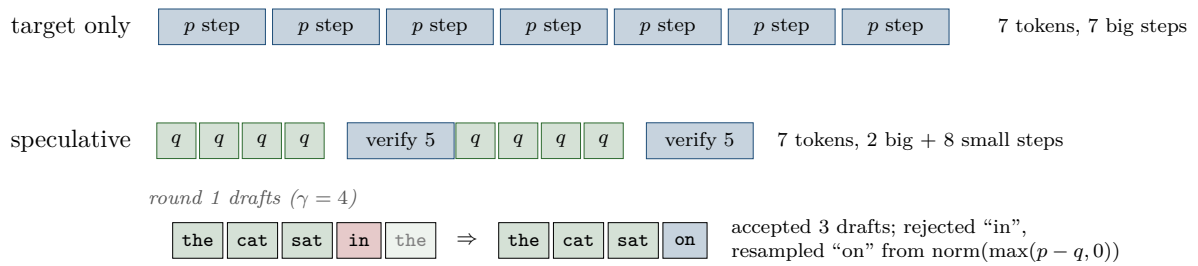


Figure 13.6: Speculative decoding timeline. In each round the draft model takes γ cheap steps, then the target verifies the pending token plus all γ drafts in a single forward pass. Every round commits between 1 and $\gamma + 1$ tokens.

from the *residual* distribution

$$r(x) = \frac{\max\left(0,\, p(x) - q(x)\right)}{\sum_y \max\left(0,\, p(y) - q(y)\right)}. \tag{13.6}$$

Proposition 13.1 (Speculative sampling is lossless)**.** The token emitted by this procedure is distributed exactly according to p.

Proof. The probability of accepting is $\beta = \sum_y q(y) \min(1, p(y)/q(y)) = \sum_y \min(p(y), q(y))$. Since $\sum_y p(y) = 1$, the residual's normaliser is

$$\sum_y \max(0, p(y) - q(y)) = \sum_y \left(p(y) - \min(p(y), q(y))\right) = 1 - \beta.$$

Hence

$$\begin{aligned}\Pr[\text{emit } x] &= \underbrace{q(x) \min\left(1, \tfrac{p(x)}{q(x)}\right)}_{\text{draft accepted}} + \underbrace{(1-\beta)\,\frac{\max(0, p(x) - q(x))}{1-\beta}}_{\text{rejected, then resampled}} \\ &= \min(p(x), q(x)) + \max(0, p(x) - q(x)) \;=\; p(x).\end{aligned}$$

□

Applied position by position, the proposition shows that the whole generated sequence has the target's distribution. Each later draft was proposed conditioned on accepted predecessors, and verification stops at the first rejection. With temperature 0 both distributions are one-hot. The rule then reduces to "accept while the draft equals the target's argmax, then emit the target's argmax", so greedy speculative decoding reproduces greedy target decoding *token for token*. The lab tests exactly this. Note that $\beta = 1 - \mathrm{TV}(p, q)$: the acceptance rate is one minus the total variation distance between the two models.

Listing 13.5: The accept-or-resample rule (`llmsys/specdec.py`).

```python
def accept_or_resample(p, q, draft_token, generator=None):
    px, qx = float(p[draft_token]), float(q[draft_token])
    u = torch.rand((), generator=generator).item()
    if qx > 0 and u < min(1.0, px / qx):
        return True, draft_token
    residual = torch.clamp(p - q, min=0)
    s = residual.sum()
    residual = p if s <= 0 else residual / s
    return False, int(torch.multinomial(residual, 1, generator=generator))
```

13.3.2 The full loop with KV caches and rollback

The systems content of speculative decoding is mostly cache management. We keep both models' KV caches covering "all committed tokens except the last". The last committed token is *pending*: it is the next input to feed. A round then proceeds as follows:

1. Feed the pending token to the draft, sample d_1; feed d_1, sample d_2; …; sample d_γ. The draft cache grows by γ entries (d_γ itself is not fed).
2. Feed the block $[\text{pending}, d_1, \ldots, d_\gamma]$ to the target in one call. Its $\gamma + 1$ output rows are the distributions $p_1, \ldots, p_{\gamma+1}$.
3. Accept a prefix $d_1..d_k$ and append one extra token: the residual sample at the first rejection, or a "bonus" sample from $p_{\gamma+1}$ if all were accepted.
4. Roll both caches back by truncation to the new committed length minus one. If all drafts were accepted, the draft model never saw d_γ, so feed it before the next round.

Listing 13.6: One round of speculative decoding (abridged from `speculative_generate`).

```
nxt = torch.tensor([[seq[-1]]])                          # pending token
for _ in range(g):                                       # 1) draft proposes
    q = probs_from_logits(draft(nxt, cache=dc)[0, -1], temperature, top_k, top_p)
    tok = int(torch.multinomial(q, 1, generator=generator))
    drafts.append(tok); qs.append(q); nxt = torch.tensor([[tok]])
block = torch.tensor([[seq[-1]] + drafts])               # 2) one target pass
ps = probs_from_logits(target(block, cache=tc)[0], temperature, top_k, top_p)
k, extra = 0, None                                       # 3) verify
for i, tok in enumerate(drafts):
    ok, t = accept_or_resample(ps[i], qs[i], tok, generator)
    if not ok:
        extra = t; break
    k += 1
if extra is None:
    extra = int(torch.multinomial(ps[g], 1, generator=generator))
seq.extend(drafts[:k] + [extra])                         # 4) commit + roll back
truncate_cache(tc, len(seq) - 1)
```

Pitfall: Rollback in paged and static caches

Truncating our concatenation-based `KVCache` is a slice. In a paged engine (Chapter 12) rollback means decrementing the sequence length and possibly freeing the last block. In a static cache it only means resetting a length counter. Rejected entries are simply overwritten next round. Engines that capture the decode step in a CUDA graph must also capture a separate "verify $\gamma + 1$ tokens" graph, because shapes differ.

13.3.3 How many tokens per round, and how much faster?

Suppose each draft is accepted independently with probability α. The number of committed tokens per round is one plus a truncated geometric variable:

$$\mathbb{E}[\text{tokens per round}] \;=\; \sum_{i=0}^{\gamma} \alpha^i \;=\; \frac{1-\alpha^{\gamma+1}}{1-\alpha}. \tag{13.7}$$

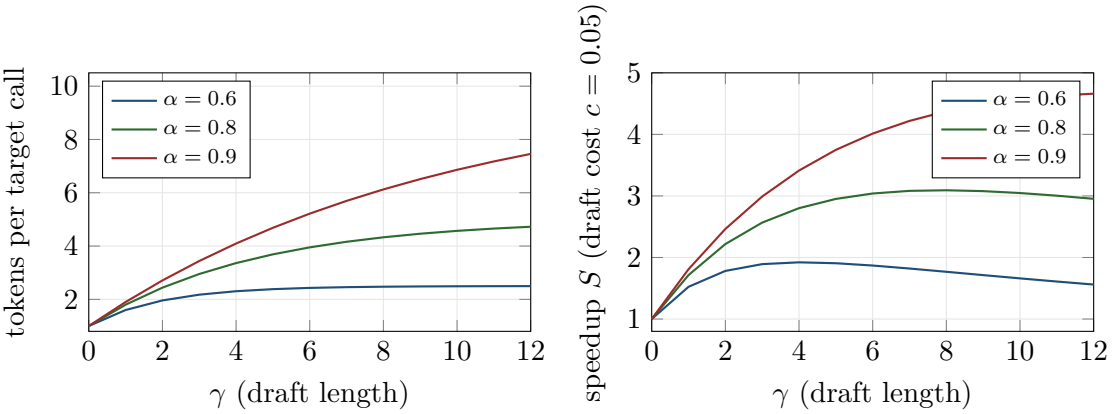


Figure 13.7: Analytic model of speculative decoding (Eqs. (13.7) and (13.8)). Left: committed tokens per target forward pass. Right: speedup when a draft step costs 5% of a target step. Each curve has an interior optimum.

Let c be the draft-step time relative to a target step, and assume verifying $\gamma + 1$ tokens costs one target step (memory-bound). The wall-clock speedup is then

$$S(\alpha, \gamma, c) \;=\; \frac{1 - \alpha^{\gamma+1}}{(1-\alpha)(\gamma c + 1)}. \tag{13.8}$$

Figure 13.7 plots both. Tokens per round saturate at $1/(1-\alpha)$, while the cost of drafting grows linearly in γ, so every (α, c) pair has an optimal γ, typically 3–8 for a good draft.

Napkin Math: Qwen3-0.6B drafting for Qwen3-8B

Both models share a tokenizer, a hard requirement for vanilla speculative decoding. At batch 1 their weight-read floors are 0.36 ms and 4.89 ms, so $c \approx 0.07$. With an acceptance rate of $\alpha = 0.8$ and $\gamma = 4$: $\mathbb{E}[\text{tokens}] = (1 - 0.8^5)/0.2 = 3.36$ and the round costs $4(0.07) + 1 = 1.28$ target steps, giving $S \approx 2.6\times$. Real systems see less: the 0.6B model's step is far from its bandwidth floor (kernel-launch overheads dominate tiny models, Chapter 11), which makes the effective c larger, and α varies wildly by domain. Code and math are often highly predictable; creative writing at high temperature is not.

13.3.4 Measuring acceptance correctly

It is tempting to report "accepted drafts / proposed drafts". That is a *biased* estimate of α: after the first rejection in a round, the remaining drafts are never examined, yet they are still counted as proposed. A consistent estimator counts only examined positions, $\hat{\alpha} = \text{accepted}/(\text{accepted} + \text{rejections})$, and this is what `SpecStats.alpha_hat` reports.

We measured both on CPU with toy models in this environment. The target is a 4-layer random Qwen3-shaped model (untied embeddings), sampled at temperature 0.25. The drafts are copies of the target with Gaussian noise added to every weight, so more noise means a worse draft. The run generates 160 tokens per configuration (`bench.py toy` in the lab):

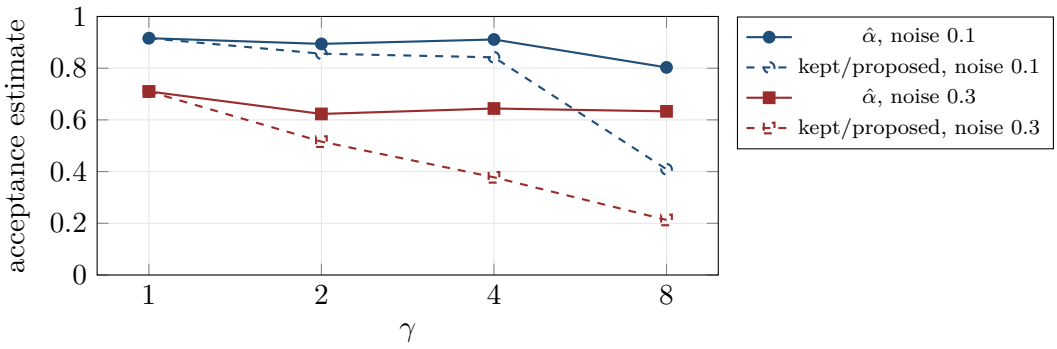


Figure 13.8: Measured on CPU in this environment (toy models, 160 generated tokens per point). The naive kept/proposed" ratio falls with γ although the per-position acceptance $\hat{\alpha}$ is roughly constant: the bias described in the text.

draft noise	γ	kept/proposed	$\hat{\alpha}$	tokens/target call	Eq. (13.7) at $\hat{\alpha}$
0.10	1	0.916	0.916	1.88	1.92
0.10	2	0.856	0.894	2.67	2.69
0.10	4	0.842	0.911	4.21	4.19
0.10	8	0.409	0.803	4.10	4.37
0.30	1	0.710	0.710	1.68	1.71
0.30	2	0.516	0.623	2.00	2.01
0.30	4	0.378	0.644	2.46	2.50
0.30	8	0.213	0.633	2.62	2.68
1.00	1	0.233	0.233	1.22	1.23
1.00	2	0.116	0.195	1.22	1.23
1.00	4	0.056	0.182	1.21	1.22
1.00	8	0.037	0.226	1.28	1.29

Two things stand out. First, "kept/proposed" falls with γ even when the per-position acceptance does not, which is exactly the bias described above. Second, the measured tokens per call track Eq. (13.7) evaluated at $\hat{\alpha}$, within the noise of a 160-token run. Third, a poor draft (noise 1.0, $\hat{\alpha} \approx 0.2$) never gets past about 1.25 tokens per call however long it drafts, so every extra draft step is pure overhead.

13.3.5 Variants: where do drafts come from?

- **Separate small model** from the same family (as above). Simple; costs extra memory and its own KV cache; needs the same tokenizer.
- **Self-speculation / early exit** (e.g. LayerSkip, Elhoushi et al., 2024): run only the first k layers of the target, then its head, as the draft. There are no extra weights, and the draft's KV for those layers can be reused. Our lab's `layer_skip_draft` builds this.
- **Extra decoding heads** (Medusa, Cai et al., 2024): k small heads on top of the last hidden state predict tokens $t+2, \ldots, t+k+1$ in parallel. Candidates are combined into a *tree* and verified at once.
- **Feature-level drafting** (EAGLE, Li et al., 2024): a one-layer model autoregresses on the target's hidden states rather than on tokens, which yields much higher acceptance than an independent small LM. EAGLE-style drafts are the default in several production engines.

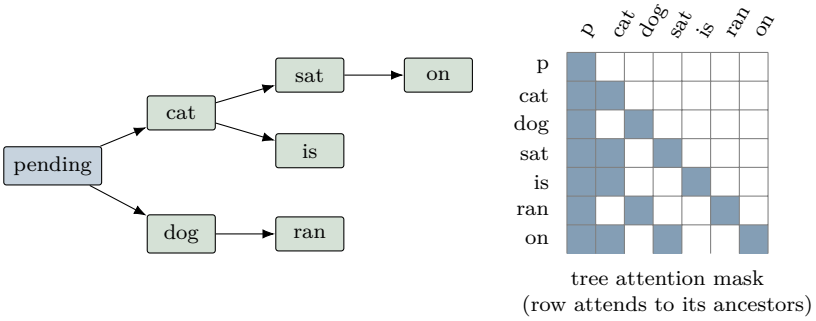


Figure 13.9: Tree verification (Medusa, EAGLE, SpecInfer). Several candidate continuations share prefixes. They are flattened into one block and verified in a single forward pass with a mask that lets each node attend only to its ancestors and the committed context. The longest accepted root-to-leaf path is kept.

- **Prompt lookup / n-gram drafting**: propose the continuation of the most recent earlier occurrence of the last n tokens. It is free, and remarkably effective for editing, retrieval-augmented answers and code, where outputs copy the input. Lookahead decoding (Fu et al., 2024) generates n-grams on the fly with Jacobi iterations.

Systems Lens: Speculation and batching compete for the same slack

Speculation converts idle compute into fewer sequential steps. A continuous-batching server at high load has *no* idle compute: its decode steps approach the ridge point and verification of $\gamma + 1$ tokens per sequence now costs real FLOPs. So speculative decoding gives large latency wins at low load and can *reduce* throughput at high load. Production engines therefore adapt γ to load or disable speculation above a batch-size threshold. Decide with the roofline: if $B(\gamma + 1)$ tokens per step pushes arithmetic intensity past the ridge point, you are paying for rejected drafts with real compute.

13.4 Splitting a model across GPUs for inference

Chapter 8 derived tensor, pipeline and expert parallelism for training. Inference reuses the same partitionings, with different trade-offs because there is no backward pass and decode moves tiny activations.

Tensor parallelism (TP). Shard each attention block by heads and each MLP by columns then rows (Megatron-style). Each layer needs two all-reduces of a $B \times d_{\text{model}}$ activation (after attention and after the MLP). Every GPU reads only $1/P$ of the weights and of the KV cache (KV heads are sharded too), so the bandwidth floor drops by P. The price is $2L$ all-reduces per decode step, which are latency-bound rather than bandwidth-bound at small B.

Napkin Math: Qwen3-32B with TP=2 on NVLink

The weights are 65.5 GB in bf16, so they do not fit one 80 GB GPU with a useful KV cache. With TP=2 each GPU holds 32.8 GB, and the step floor falls from 19.6 ms to about 9.8 ms. Per decode step and per sequence, the all-reduces move $2 \cdot 64 \cdot 5120 \cdot 2\,\text{B} \approx 1.3\,\text{MB}$, negligible in bandwidth. But 128 collectives at roughly 5–10 μs each add 0.6–1.3 ms of pure latency per token. This is why TP is kept within an NVLink domain, and why engines

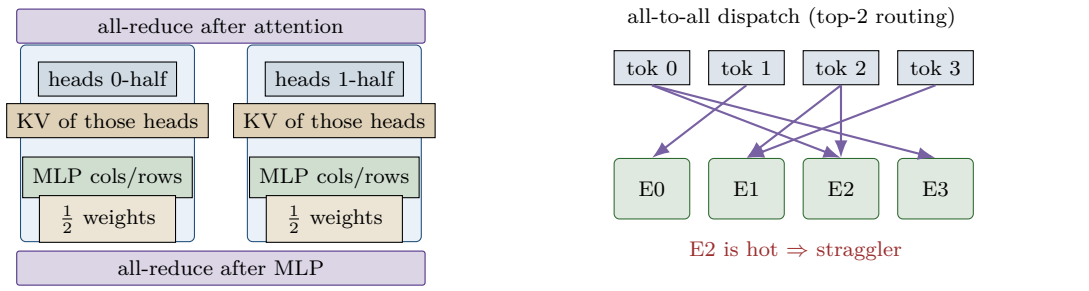


Figure 13.10: Left: tensor-parallel decode shards heads, KV cache and MLP across GPUs, with two all-reduces per layer. Right: expert-parallel MoE layers route tokens to experts on different GPUs with all-to-all exchanges; imbalance turns one expert into a straggler.

fuse the all-reduce with the following norm or use one-shot all-reduce kernels for small messages.

Pipeline parallelism (PP). Assign contiguous layers to GPUs. For a single sequence this adds latency (the stages run one after another) but no speedup. It pays off for throughput when many micro-batches are in flight, and it is the usual way to span nodes connected by slower networks.

Expert parallelism (EP) for MoE. A mixture-of-experts layer (Chapter 6) activates only a few experts per token, so decode reads a fraction of the weights. But the experts are spread across GPUs, and every MoE layer becomes two all-to-all exchanges (dispatch tokens to experts, combine results). Large-EP serving designs, such as DeepSeek's published inference setup, place one or a few experts per GPU across many nodes and overlap the all-to-all with computation by splitting a batch into two micro-batches. The new failure mode is *load imbalance*: one hot expert throttles the whole step. Systems therefore replicate popular experts and rebalance placement from routing statistics.

13.5 Structured output and constrained decoding

Applications often need output that *parses*: JSON matching a schema, a function call, a number in a `\boxed{}` (Chapter 14). Rather than hoping, we can *constrain* sampling. Compile the grammar (regular expression, JSON schema or context-free grammar) into an automaton over *tokens*. At each step, set the logits of tokens that would leave the language to $-\infty$, then sample as usual. The model's probabilities are renormalised over the legal tokens.

Listing 13.7: Masking logits to a legal token set. The hard part, computing `allowed_ids` efficiently from a grammar state, is what libraries such as Outlines and XGrammar provide.

```
def constrained_step(logits, allowed_ids, temperature=0.0):
    mask = torch.full_like(logits, float("-inf"))
    mask[..., allowed_ids] = 0.0
    return sample_next_token(logits + mask, temperature)
```

The systems problem is cost. A BPE vocabulary of about 150k tokens must be checked against the grammar at *every* step, on the CPU, while the GPU waits. Outlines (Willard & Louf, 2023) precomputes, for each state of a regular expression's automaton, the set of tokens that keep the automaton alive, which turns the per-step cost into an index lookup. Context-free grammars

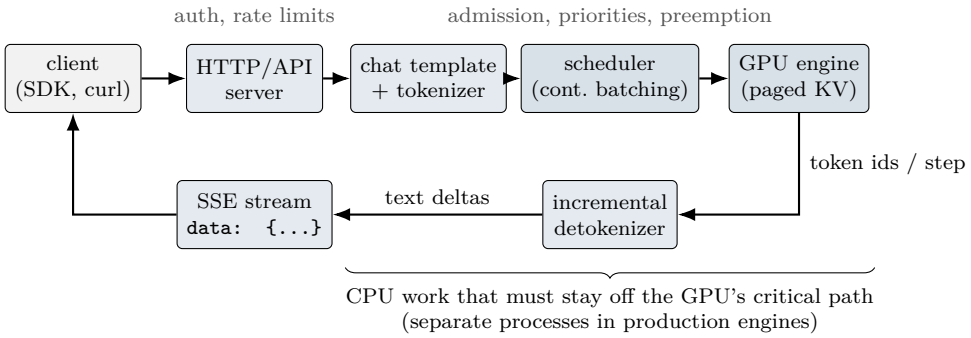


Figure 13.11: Life of a streaming chat request. Tokenization, detokenization and HTTP handling run on the CPU. Production engines move them into separate processes so a slow client or a long prompt cannot stall the batched GPU loop.

(JSON with nesting) need a pushdown automaton, whose stack makes the state space unbounded. XGrammar (Dong et al., 2024) splits tokens into context-independent ones (checkable ahead of time) and a small context-dependent remainder checked at runtime, and overlaps mask computation with the GPU forward pass. Constraints also interact with tokenization: forcing `"}"` as a single token when the model would naturally emit `"}\n"` pushes it off-distribution. This is the "token healing" problem, and it is why good libraries allow every token whose *characters* are consistent with the grammar.

Key Idea

Constrained decoding changes the output distribution: it samples from p restricted to the language, not from p conditioned on producing a valid string. For short structured fields the difference is negligible. For long reasoning followed by an answer, let the model think unconstrained and constrain only the final answer span.

13.6 The serving stack around the engine

An inference engine (Chapter 12) turns token ids into token ids. A service must also accept HTTP requests, apply chat templates, tokenize, schedule, stream text back, and survive load. Figure 13.11 traces one request through an OpenAI-compatible server such as those shipped with vLLM, SGLang, TensorRT-LLM or TGI.

Streaming. Responses are streamed as server-sent events (SSE), one small JSON delta per step. This decouples *time to first token* (TTFT) from total latency. Two details matter. First, detokenization must be *incremental and UTF-8 aware*: decoding one BPE token in isolation can split a multi-byte character (an emoji is 4 bytes and may span several tokens). The usual fix decodes a small window of recent ids and emits only text that no longer ends in the replacement character U+FFFD. Our chat app uses exactly this (`IncrementalDetokenizer`). Second, the CPU threads that serialise and send deltas must never block the GPU loop. In Python servers that means a separate process or thread for the engine, communicating through queues.

Load testing. To measure a serving system, drive it with an *open-loop* workload: requests arrive by a Poisson process at rate λ, independent of how fast the server responds. Closed-loop clients ("N users each waiting for their reply") self-throttle and hide queueing collapse. Sweep λ. At each rate record the percentiles (p50/p90/p99) of TTFT and inter-token latency (ITL),

and the *goodput*: completed requests per second that met their latency SLOs. Use realistic length distributions (prompt and output lengths sampled from logs or from datasets such as ShareGPT), because the prefill/decode mix determines the result.

Napkin Math: Little's law sizes your KV cache

Little's law says $\bar{N} = \lambda \bar{W}$: the mean number of requests in the system equals arrival rate times mean time in the system. At $\lambda = 20$ requests/s with replies that take $\bar{W} = 12\,\text{s}$, about 240 sequences are resident on average. At 3000 tokens each and 147.5 KB per token (Qwen3-8B), that is $240 \cdot 3000 \cdot 147.5\,\text{KB} \approx 106\,\text{GB}$ of KV cache, more than an 80 GB GPU. The server must shard, quantize the KV, or queue, and queueing raises $\bar{W}$, which raises $\bar{N}$ further. This feedback loop is why latency explodes near capacity.

Systems Lens: Disaggregation, revisited

Prefill is compute-bound and bursty; decode is memory-bound and steady. Running them on the same GPUs makes long prompts inflate the ITL of everyone else, and chunked prefill (Chapter 12) only mitigates this. Disaggregated serving sends prefill to one pool, ships the KV cache over NVLink, InfiniBand or RDMA, and decodes in another pool, so each pool can use its own parallelism and even its own hardware generation. The KV transfer is the new bottleneck. For Qwen3-8B a 4k-token prompt produces about 0.6 GB of KV, roughly 12 ms at 50 GB/s.

13.7 A chat interface for our models

RM G

We now put a browser front end on the from-scratch Qwen3 implementation. It is useful for poking at the checkpoints we train later (instruction-tuned in Chapter 10, RL-trained in Chapter 17, distilled in Chapter 20), and it is a compact case study of everything above: chat templates, streaming, detokenization, event loops and KV reuse. We use Chainlit, an open-source Python package that provides a ChatGPT-like web UI and calls our Python functions when messages arrive.

13.7.1 Installing Chainlit

RM G.1

Chainlit is a regular PyPI package:

```
pip install chainlit          # or, with uv:  uv add chainlit
```

It is deliberately *not* in the lab's `requirements.txt`: nothing else in the book depends on it, and it pulls in a web stack (an ASGI server, websockets).

13.7.2 Running an app as a script

RM G.2

A Chainlit app is an ordinary Python file that Chainlit *imports* and inspects for decorated handlers: `@cl.on_chat_start` runs when a browser session opens, and `@cl.on_message` runs for every user message. You start the server with

```
chainlit run chat_single_turn.py      # serves http://localhost:8000
```

Because `chainlit run` owns the process's command line, our scripts cannot use `argparse`. Configuration therefore comes from environment variables (`QWEN3_DIR`, `CHECKPOINT_PATH`, `MAX_NEW_TOKENS`,

`THINKING`, `QUANT_BITS`), read once at import time in `chat_common.py`. Module import also happens exactly once per server process, so that is where the model is loaded and shared by all browser sessions.

13.7.3 Where the scripts live

RM G.3

Everything is in `code/labs/lab13_quant_specdec/`: `chat_common.py` (model loading, streaming, detokenization, history trimming), `chat_single_turn.py` and `chat_multi_turn.py`. They import only `llmsys` and the lab's `solution.py` (for optional quantized serving and prefix matching), so there is nothing to download beyond the Qwen3 weights.

13.7.4 The single-turn app

RM G.4

Listing 13.8 is the whole single-turn app. Each message is wrapped in the ChatML template, tokenized, and streamed back token by token into an initially empty message bubble.

Listing 13.8: `chat_single_turn.py`: every message is answered independently.

```python
import chainlit as cl
import torch
from chat_common import (DEVICE, THINKING, IncrementalDetokenizer,
                         load_model_and_tokenizer, stop_token_ids, stream_tokens)

MODEL, TOKENIZER = load_model_and_tokenizer()   # once per server process
STOP = stop_token_ids(TOKENIZER)                # <|im_end|>, <|endoftext|>

@cl.on_chat_start
async def on_start():
    await cl.Message(content="Single-turn mode.").send()

@cl.on_message
async def on_message(message: cl.Message):
    prompt = TOKENIZER.apply_chat_template(message.content, enable_thinking=THINKING
    )
    ids = torch.tensor([TOKENIZER.encode(prompt)], device=DEVICE)
    reply = cl.Message(content="")
    await reply.send()                          # empty bubble to stream into
    detok = IncrementalDetokenizer(TOKENIZER)
    async for tok in stream_tokens(MODEL, ids, STOP):
        if tok in STOP:
            break
        piece = detok.push(tok)
        if piece:
            await reply.stream_token(piece)
    await reply.update()
```

Two systems details hide in `chat_common.py`. First, `load_model_and_tokenizer` loads the official snapshot by default, or, if `CHECKPOINT_PATH` is set, builds a fresh `Qwen3Model` and loads your own state dict. The tokenizer always comes from the snapshot directory, so it must match the checkpoint: base and reasoning variants use different chat conventions, and a checkpoint fine-tuned on one should be served with the same template. Second, `stream_tokens` runs generation in a worker thread and passes tokens to the asyncio event loop through a queue

(Listing 13.9). Calling the model directly inside the `async` handler would block the event loop for the whole reply: the UI would not repaint and every other session would freeze.

Listing 13.9: Bridging blocking generation and an async web framework.

```
async def stream_tokens(model, input_ids, stop_ids, cache=None):
    loop = asyncio.get_running_loop()
    queue, done = asyncio.Queue(), object()
    def worker():                                   # runs in a plain thread
        try:
            for tok in generate_stream(model, input_ids, MAX_NEW_TOKENS, cache=cache
    ):
                t = int(tok[0, 0])
                loop.call_soon_threadsafe(queue.put_nowait, t)
                if t in stop_ids:
                    break
        finally:
            loop.call_soon_threadsafe(queue.put_nowait, done)
    threading.Thread(target=worker, daemon=True).start()
    while (item := await queue.get()) is not done:
        yield item
```

13.7.5 Running it, with your own checkpoints

RM G.5

```
huggingface-cli download Qwen/Qwen3-0.6B --local-dir qwen3-0.6b
QWEN3_DIR=qwen3-0.6b chainlit run chat_single_turn.py
# serve a checkpoint from Chapter 17 or 20 instead, 4-bit quantized:
QWEN3_DIR=qwen3-0.6b CHECKPOINT_PATH=runs/grpo/step500.pth QUANT_BITS=4 \
    chainlit run chat_single_turn.py
```

Chainlit usually opens a browser tab. Otherwise visit `http://localhost:8000`. With `THINKING=1` the reasoning model first emits a `<think> ... </think>` block; a simple prompt such as "What is 1+1?" can trigger several hundred tokens of deliberation, which makes `MAX_NEW_TOKENS` a real cost knob (Chapter 15).

13.7.6 Multi-turn conversations

RM G.6

What multi-turn means. The single-turn app *displays* a conversation but the model sees only the latest message. Ask "What is 1+1?" and then "What did I just ask?" and the model has no idea. A multi-turn chat re-sends the history with every request, because the model itself is stateless: the "memory" of a chat assistant is nothing but its prompt.

The multi-turn app. `chat_multi_turn.py` keeps a per-session list of messages in Chainlit's per-session store (`user_session`), seeded with a system message. On each turn it appends the user message, serialises the whole history with the ChatML template (Fig. 13.12), generates, and appends the assistant reply.

Fitting the context window. Histories grow without bound, so before each turn we must leave room for `MAX_NEW_TOKENS`. The simplest strategy left-truncates the token sequence. That can cut a message in half and even drop the `<|im_start|>` marker, producing a prompt shape the model never saw in training. Our `trim_history` instead drops the oldest *whole* user/assistant

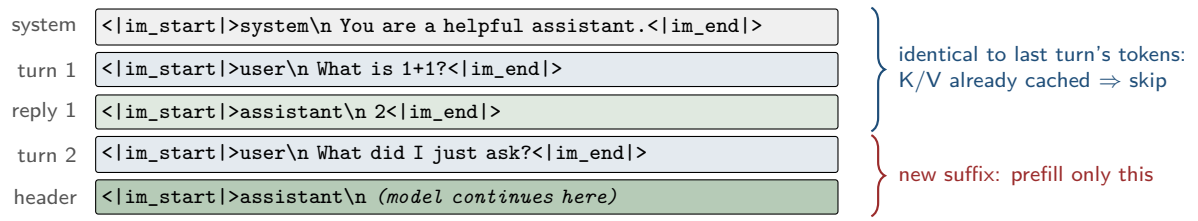


Figure 13.12: The multi-turn prompt is the whole history serialised in ChatML. Because each turn's prompt extends the previous one, the session can keep its KV cache and prefill only the new suffix.

turns and never drops the system message. More elaborate strategies summarise old turns with the model itself.

How history meets the KV cache. Re-sending the history means re-prefilling it, an $O(T)$ cost that grows every turn, so a 20-turn chat does quadratic total work. The app therefore keeps the session's `KVCache` together with the token ids it covers. Each turn it computes the longest common prefix between the cached ids and the new prompt (`common_prefix_len`), truncates the cache to that length, and prefills only the suffix (Listing 13.10). This is a single-session version of the prefix caching that engines such as vLLM and SGLang perform across all requests (Chapter 12).

Listing 13.10: The heart of `chat_multi_turn.py`: history in, KV cache reused.

```
history.append({"role": "user", "content": message.content})
history = trim_history(history, TOKENIZER, CONTEXT - MAX_NEW_TOKENS)
prompt_ids = TOKENIZER.encode(TOKENIZER.apply_chat_template(history, THINKING))
cache, cached = cl.user_session.get("cache"), cl.user_session.get("cached_ids")
reuse = min(common_prefix_len(cached, prompt_ids), len(prompt_ids) - 1)
truncate_cache(cache, reuse)                          # drop anything that diverged
new_ids = torch.tensor([prompt_ids[reuse:]], device=DEVICE)
...                                                   # stream as in single-turn
cl.user_session.set("cached_ids", (prompt_ids + generated)[: cache.seq_len(0)])
history.append({"role": "assistant", "content": reply.content})
```

Pitfall: When the prefix silently stops matching

Prefix reuse is only valid if re-serialising the history reproduces *exactly* the tokens that were cached. Three things break it. (1) Templates that rewrite history: Qwen3's official template removes earlier turns' `<think>` blocks, so the re-serialised assistant turn is shorter than what was generated. Our simplified template keeps them, so the prefix holds, but the model sees its old reasoning (more tokens, and different from its training distribution). (2) Tokenization is not compositional: the text of a reply, re-encoded in context, can tokenize differently from the ids that were sampled. (3) Trimming old turns changes the prefix from the start. `common_prefix_len` makes all three cases *safe*, since it falls back to recomputation, but they silently cost performance. Log the reuse ratio (the app prints it after every reply).

Recommendations. Use the multi-turn app with models trained on multi-turn chat data, such as the official Qwen3 reasoning checkpoint. Checkpoints trained only on single prompt/response pairs, such as the distilled models of Chapter 20 or our small instruction-tuned models, were

never taught to condition on earlier assistant turns. Inspect those with the single-turn app; the multi-turn app tends to confuse them.

13.8 Summary

- A decode step costs, to first order, the bytes of weights plus KV cache divided by HBM bandwidth (Eq. (13.1)). Decide which term dominates before choosing an optimisation.
- Uniform quantization trades resolution for bytes; outliers are the enemy, and finer granularity (per-channel, per-group, per-block microscaling) is the main weapon. Weight-only int8/int4 speeds up memory-bound decode; W8A8/FP8 speeds up compute-bound prefill and large batches; KV quantization raises both speed and capacity in long-context serving.
- GPTQ uses second-order information to compensate rounding error; AWQ and SmoothQuant rescale channels so that what matters lands on the grid.
- Speculative decoding is *lossless*: accept with $\min(1, p/q)$, resample from $\max(p - q, 0)$. Expected tokens per round are $(1-\alpha^{\gamma+1})/(1-\alpha)$; its benefit shrinks as batching consumes the idle compute it relies on.
- Tensor parallelism divides the bandwidth floor at the cost of $2L$ latency-bound collectives per step; expert parallelism trades all-to-alls and load balance for sparse compute.
- A service adds templates, streaming, incremental detokenization, async I/O, open-loop load testing and SLO-based goodput around the engine.
- A chat UI is a thin layer: the model is stateless, memory is the prompt, and the KV cache can make re-sending history nearly free.

13.9 Exercises

Exercise 13.1 (*bytes per token*). Using Eq. (13.1), find the batch size at which Qwen3-0.6B's KV cache bytes equal its weight bytes at 2048 tokens of context (bf16 for both). What does this say about weight-only quantization for small models in a busy server?

Exercise 13.2 (*rounding noise*). Show that if x is uniformly distributed over a quantization bin of width s, the mean squared rounding error is $s^2/12$. Use this to derive the "6 dB per bit" rule for a uniform quantizer of a signal spanning its full range.

Exercise 13.3 (*outliers*). A weight row has 127 values drawn from $\mathcal{N}(0, 1)$ and one value equal to 40. Estimate the fraction of the 127 normal values that round to zero under per-row symmetric int4 quantization, and under group-wise int4 with $g = 32$.

Exercise 13.4 (*metadata*). Compute the effective bits per weight of: (a) int4 with $g = 64$, fp16 scale and fp16 zero; (b) MXFP4 (blocks of 32, one 8-bit shared scale); (c) NVFP4 (blocks of 16, one FP8 scale, ignoring the per-tensor scale).

Exercise 13.5 (*GPTQ derivation*). Starting from Eq. (13.3) for a single row $\boldsymbol{w}$, show that if column i is fixed to q_i, the optimal update of the remaining columns is $\delta \boldsymbol{w}_{-i} = -\frac{w_i - q_i}{[H^{-1}]_{ii}} [H^{-1}]_{-i,i}$, and that the increase in loss is $(w_i - q_i)^2 / [H^{-1}]_{ii}$.

Exercise 13.6 (*SmoothQuant*). Show that Eq. (13.5) leaves $XW^\top$ unchanged, and that with $\alpha = 0.5$ the transformed per-channel maxima satisfy $\max |X'_{:,j}| = \max |W'_{:,j}|$. How would you fold $\mathrm{diag}(\boldsymbol{s})^{-1}$ into a preceding RMSNorm?

Exercise 13.7 (*lossless speculation*). Extend the proof of Proposition 13.1 to two consecutive positions: show that the pair of tokens emitted when the first draft is accepted and the second is verified has distribution $p(x_1)\, p(x_2 \mid x_1)$.

Exercise 13.8 (*optimal draft length*). For $c = 0.05$, find numerically the γ that maximises Eq. (13.8) for $\alpha \in \{0.6, 0.8, 0.9\}$. Derive the condition for $\gamma \to \gamma + 1$ to help.

Exercise 13.9 (*speculation under load*). A server runs Qwen3-8B on one H100 at batch 128 with 2k-token contexts. Using the roofline, estimate whether verifying $\gamma + 1 = 5$ tokens per sequence keeps the step memory-bound. What γ would you choose?

Exercise 13.10 (*TP latency*). Model a tensor-parallel decode step as $t = t_{\mathrm{mem}}/P + 2L\,(a + m/\beta)$ with per-collective latency a, message size m and link bandwidth β. For Qwen3-32B with $a = 8\,\mu$s, find the P that minimises per-token latency at batch 1.

Exercise 13.11 (*load testing*). Explain why a closed-loop benchmark with 64 concurrent users can report excellent p99 latency for a server that collapses under an open-loop load of the same average rate.

Exercise 13.12 (*chat memory*). In the multi-turn app, after 20 turns of 150 tokens each, how many prompt tokens are prefilled in total with and without KV reuse? Which of the three pitfalls in Section 13.7 would you expect to hit first with the official Qwen3 template, and how would you fix it?

13.10 Lab 13: Quantize, speculate, and serve

Lab 13: Quantize, speculate, and serve

Code: `code/labs/lab13_quant_specdec`

Goals

Implement weight-only int8 and int4 quantization from the bit level up, build a speculative decoder with KV-cache rollback that is provably lossless, and serve a model through a streaming multi-turn chat UI with cross-turn KV reuse.

Setup

`cd code; LAB_IMPL=starter python -m pytest -q labs/lab13_quant_specdec`. All tests run on CPU with tiny random Qwen3/GPT models. `bench.py` contains the real-model

experiments (Qwen3-0.6B/1.7B snapshots), and the chat apps need `pip install chainlit`.

Tasks

1. `quantize_absmax` / `dequantize_absmax`: symmetric int8, per-tensor or per-channel. The test injects an outlier row; per-channel must win.
2. `quantize_groupwise` / `dequantize_groupwise`: asymmetric 4-bit groups with zero points. Then `pack_int4` / `unpack_int4` (two codes per byte).
3. `quantize_linear_layers`: deep-copy a model and replace every `nn.Linear` except the head with `WeightOnlyLinear`. Check relative output error ($< 1\%$ for int8, $< 10\%$ for int4) and the byte savings.
4. `accept_or_resample`: the speculative sampling rule. The test checks by Monte Carlo that the output follows p.
5. `verify_drafts`: accept a prefix and emit one extra token (residual or bonus).
6. `speculative_generate`: the full loop with both KV caches, rollback, and the "draft never saw d_γ" fix. Greedy output must equal plain greedy decoding of the target, using `layer_skip_draft` as the draft.
7. `expected_tokens_per_round`, and `common_prefix_len` for the chat app.

Measurements

1. `python bench.py toy`: reproduce the acceptance table of Section 13.3. Plot measured tokens per target call against Eq. (13.7) evaluated at $\hat{\alpha}$, for each noise level.
2. With real checkpoints: `bench.py quant` reports KL to the bf16 reference and linear-layer memory for int8 and int4. Expect int8 to be nearly lossless and int4 to show a measurable but small KL. Then time decode with your PyTorch `WeightOnlyLinear` and note that it is *slower* than bf16. Explain why with Listing 13.2 and fix it with the fused kernel of Lab A.
3. `bench.py spec` with Qwen3-0.6B drafting for Qwen3-1.7B (or 4B): tokens/s and $\hat{\alpha}$ for $\gamma \in \{2, 4, 6, 8\}$ on a math prompt and a creative prompt, at temperature 0 and 0.7.
4. Run `chat_multi_turn.py` for ten turns and record the reuse ratio it prints. Switch `THINKING` on and off and explain the difference.

Deliverables

Passing tests; a one-page report with the acceptance plot, the quantization KL/memory table, the speculative speedup table, and an answer to "at what batch size would you turn speculation off for this pair of models, and why?"

Stretch goals

- Implement `gptq_quantize` for a real layer of Qwen3-0.6B, using 128 calibration sequences, and compare layer output error with RTN at int4 and int3.
- Implement tree verification: draft the top-2 tokens at each of 2 levels (a tree of 6 nodes), build the ancestor mask of Fig. 13.9, and verify with one forward pass. (You will need to pass a custom mask into attention.)
- Add int8 KV-cache quantization (`llmsys.quant.quantize_kv`) to a copy of `KVCache` and measure the KL it induces at 4k context.
- Write a tiny OpenAI-compatible `/v1/chat/completions` endpoint with SSE streaming

(FastAPI), and load-test it open-loop at increasing Poisson rates.

Part V

Reasoning Models

Chapter 14

Evaluating LLMs and Reasoning Models

Every later chapter of this part of the book changes a model or the way it generates text, and every one of those changes is only as believable as the measurement behind it. Chain-of-thought prompting, self-consistency, self-refinement, reinforcement learning with verifiable rewards and distillation all end with the same question: *did the number go up, and can I trust that it did?* This chapter builds the measuring instrument.

For a systems person, evaluation is also a serving workload in disguise. Evaluating a 0.6B-parameter reasoning model on 500 competition problems means generating several hundred thousand tokens; doing it inside an RL loop (Chapter 17) means doing it continuously, with a grader that must never hang, never crash the trainer, and never disagree with itself. We will therefore build the evaluation stack twice over: first as a clear, step-by-step math *verifier* — the deterministic program that decides whether a model's answer is right — and then as a *harness*: batched, cached, sandboxed and statistically honest. The second half of the chapter surveys the other three families of evaluation (multiple-choice accuracy, preference leaderboards, and LLM judges) and implements the algorithms that power them.

14.1 Four ways to measure a language model

RM 3.1; RM F.1

Papers, model cards and leaderboards report many numbers, but almost all of them come from one of four procedures (Fig. 14.2):

1. **Answer-choice accuracy.** The model picks one of a fixed set of options (A–D on MMLU). Scoring is a string or logit comparison. Cheap and objective, but it measures recall and recognition, not the ability to produce a solution.

2. **Verifiers.** The model produces a free-form solution; a program extracts the final answer and checks it against a reference — by symbolic algebra for math, by running unit tests for code. Objective and cheap *to grade*, but only applicable where answers are checkable.

3. **Preferences and leaderboards.** Humans (or models) compare two responses to the same prompt and pick a winner; a rating system aggregates thousands of such votes into a ranking. Captures open-ended quality, but it is noisy, expensive, and gameable by style.

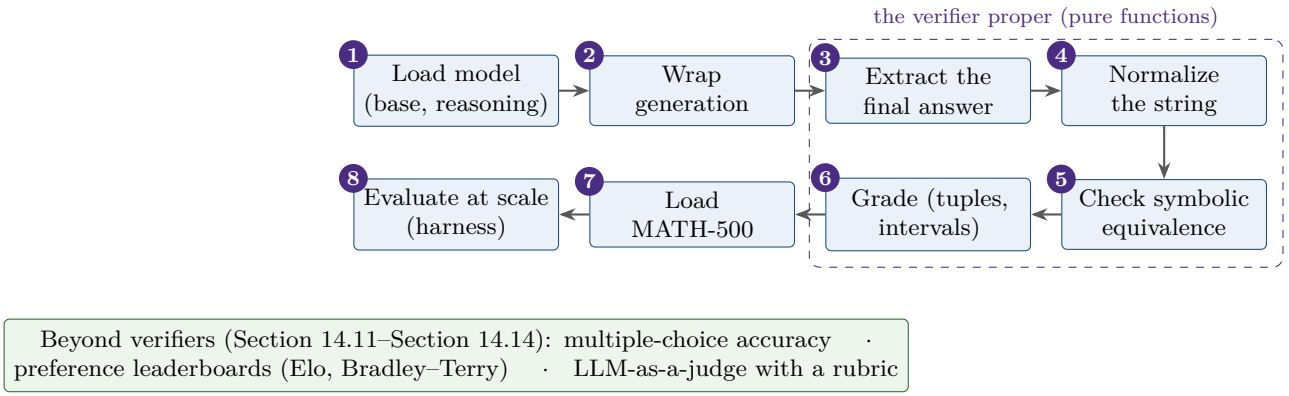


Figure 14.1: Roadmap of this chapter. Stages 3–6 form the verifier: four pure, testable functions between raw model text and a boolean. Stages 7–8 turn it into a benchmark run; the bottom row covers the other evaluation families.

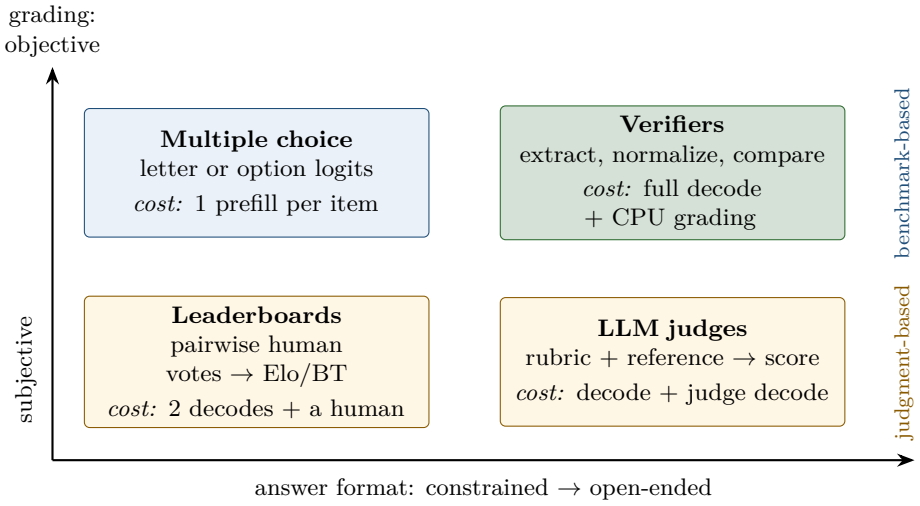


Figure 14.2: The four evaluation families arranged by how constrained the answer is and how objective the grading is. Verifiers (green) are the workhorse for reasoning models.

4. **LLM judges.** A strong model grades a response against a rubric and, optionally, a reference answer. Scales like a verifier while handling open-ended answers, at the price of the judge's own biases.

The first two are *benchmark-based*: a fixed dataset with ground truth, yielding a number such as "48.2% accuracy". The last two are *judgment-based*: they rank or score outputs by preference. Reasoning models live mostly in the second family, because the tasks that benefit most from step-by-step reasoning — arithmetic, algebra, code — are exactly the tasks whose answers can be checked mechanically. That property is what makes the rest of this part of the book possible: the same verifier that grades an evaluation run becomes the *reward function* for reinforcement learning in Chapter 17.

Systems Lens: Evaluation cost is generation cost

The four families have very different cost profiles. Multiple-choice scoring by logits needs one *prefill* per question and no decode loop — it is compute-bound and batches perfectly. A verifier needs a full *decode* of a possibly long chain of thought (memory-bound, see

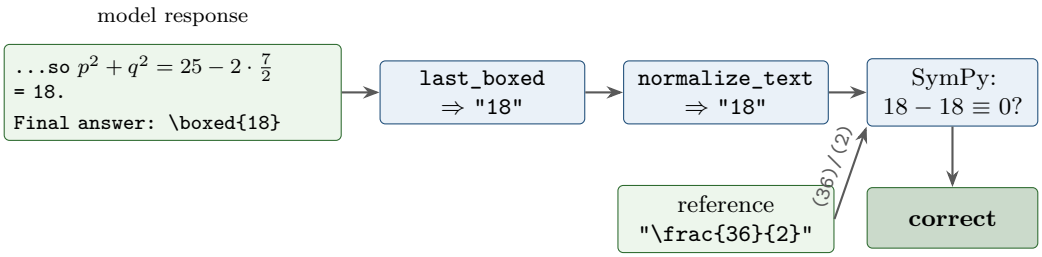


Figure 14.3: A verifier in one picture. The model's last boxed expression and the reference are normalized separately and then compared *symbolically*, so 18 and $\frac{36}{2}$ agree.

Chapter 11) followed by CPU work for parsing and symbolic algebra. A pairwise leaderboard needs two decodes per vote plus a human. An LLM judge needs the candidate's decode *and* the judge's decode, often from a model ten times larger. When you design an evaluation suite you are designing a serving workload; budget it the way you would budget production traffic.

14.2 Verifiers: grading by computation

RM 3.1

A verifier answers one question: *is the final answer in this response equivalent to the reference answer?* For math the response is free-form text that typically ends with a LaTeX expression inside `\boxed{}`, a convention that pervades competition datasets and that models absorb during pretraining. The verifier does not judge the reasoning, only the outcome — which is both its strength (no subjectivity, no extra model) and its limitation (a lucky guess with a wrong derivation counts as correct; Section 14.12 discusses process verifiers that look at steps).

We will use math throughout Part V because it hits a sweet spot: problems are hard enough that step-by-step reasoning matters, yet the final answers are short expressions that a program can check deterministically and in milliseconds. Code is the other classic domain (run the unit tests); we return to it in Section 14.12 and Chapter 19.

Key Idea: The verifier is a reward function in waiting

Everything built in this chapter is reused. Chapter 15 uses the extractor and normalizer to group sampled answers for majority voting; Chapter 16 uses the format checks as a heuristic score; Chapter 17 turns `grade_answer` into the reward $r \in \{0, 1\}$ for RL; Chapter 20 uses it to filter teacher traces. A bug here silently corrupts all of them — which is why we test every stage in isolation.

14.3 Loading a pretrained model to generate answers

RM 3.2

The model under test throughout Part V is Qwen3-0.6B, built from scratch in Chapter 6 and loaded from the official weights in Chapter 11. Qwen3 ships in two flavours that make a useful pair of reference points:

- the **base** model, which has only been pretrained (plus mid-training on curated data, including chain-of-thought–style math); it continues text, and will often — but not reliably — solve a problem step by step and box the answer;

- the **reasoning** ("thinking") model, post-trained with the methods of Chapters 17 and 20; it emits a long `<think>...</think>` block before its answer.

We evaluate the base model by default because it is what later chapters will try to improve; the reasoning variant tells us roughly where a heavily post-trained model of the same size lands. Listing 14.1 packages loading into one function so later chapters can call it with a single line.

Listing 14.1: Loading the model under test (see `labs/lab14_eval/run.py`).

```python
import os, torch
from llmsys.qwen3 import Qwen3Tokenizer, load_qwen3_from_dir
from llmsys.utils import get_device

def load_model_and_tokenizer(ckpt_dir, compile=False, dtype=torch.bfloat16):
    """ckpt_dir: a downloaded HF snapshot, e.g. Qwen/Qwen3-0.6B-Base."""
    device = get_device()
    model = load_qwen3_from_dir(ckpt_dir, dtype=dtype, device=device)
    tok = Qwen3Tokenizer(os.path.join(ckpt_dir, "tokenizer.json"))
    if compile:                          # one-time compile cost, faster decode steps
        model = torch.compile(model)
    return model, tok, device
```

Napkin Math: What does loading the model cost?

Qwen3-0.6B has $\approx$ 596M parameters including the tied 151,936 $\times$ 1024 embedding (`llmsys.flops.params`). In bf16 that is 596M $\times$ 2 B $\approx$ 1.19 GB of weights — trivially small for any modern GPU, and small enough to run on a laptop CPU. The KV cache is where memory goes during evaluation: 2×28 layers $\times$ 8 KV heads $\times$ 128 $\times$ 2 B $=$ 114.7 KB per token. A 2048-token reasoning trace therefore holds $\approx$ 235 MB of cache; a batch of 64 such traces holds $\approx$ 15 GB, more than ten times the weights.

Pitfall: Results are device-dependent

Greedy decoding is deterministic only for a fixed kernel sequence. Running the same model in bf16 on CPU, on a CUDA GPU and on Apple MPS uses different matmul reduction orders; a single flipped argmax early in a chain of thought sends the rest of the generation down a different path. Differences of one or two percentage points on MATH-500 between devices are normal. Record the device, dtype, batch size and library versions with every result (Section 14.15).

14.4 A generation wrapper

RM 3.3

The evaluation code should not care whether text comes from a cached streaming loop, a batched engine, or a remote server. We therefore hide generation behind a function with the narrowest possible interface: *prompt string in, completion string out*. Listing 14.2 builds it on top of the streaming generator from Chapter 11, printing tokens as they arrive when `verbose` is set — invaluable when you are staring at a model that refuses to box its answer.

Listing 14.2: Prompt-in, text-out wrapper over the cached streaming generator.

```python
from llmsys.generate import generate_stream

```

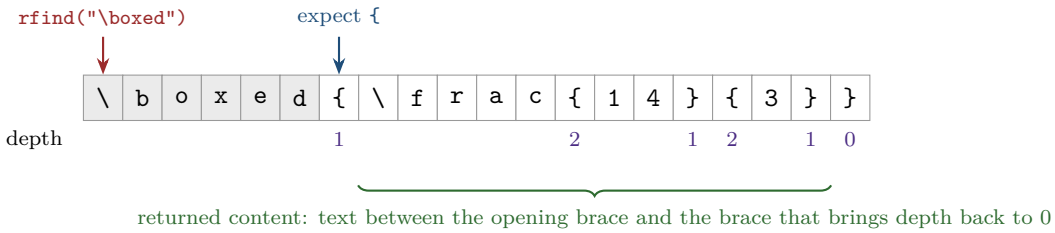


Figure 14.4: Brace matching for `\boxed{\frac{14}{3}}`. The scanner records depth after each brace; the content ends where depth returns to zero. If the string ends first (unbalanced), the extractor returns `None`.

```
@torch.no_grad()
def generate_text(model, tok, prompt, device, max_new_tokens=1024,
                  verbose=False, chat=False, **sampling):
    ids = torch.tensor([tok.encode(prompt, chat=chat)], device=device)
    eos = tok.eos_token_id if chat else tok.pad_token_id
    pieces = []
    for nxt in generate_stream(model, ids, max_new_tokens, eos_id=eos, **sampling):
        t = int(nxt[0, 0])
        if t == eos:
            break
        pieces.append(t)
        if verbose:                                    # live chain of thought
            print(tok.decode([t]), end="", flush=True)
    return tok.decode(pieces)
```

Decoding token by token and calling `decode([t])` for printing is slightly wrong for multi-byte characters (a single UTF-8 character can span tokens), which is why the function accumulates ids and decodes the whole list for the return value. Throughout Part V, functions that need a model take a `generate_fn` or `generate_batch_fn` callable instead of a model object; that one decision lets the labs test every algorithm with a scripted fake model on a CPU in milliseconds, and lets you swap in a production engine (Chapter 12) without touching the evaluation logic.

14.5 Extracting the final answer

RM 3.4

The first verifier stage turns a long response into a short candidate string. By convention the answer is the content of the *last* `\boxed{}` — models frequently box intermediate results too, and the last one is the one they commit to. We cannot use a regular expression for this, because box contents nest: `\boxed{\frac{14}{3}}` contains braces of its own. Instead we scan forward from the last occurrence of `\boxed` and track brace depth (Fig. 14.4).

Listing 14.3: Extracting the last boxed expression by brace matching.

```
def last_boxed(text, macro=r"\boxed"):
    if not text:
        return None
    start = text.rfind(macro)                  # the LAST box wins
    if start < 0:
        return None
    i = start + len(macro)
    while i < len(text) and text[i].isspace():
        i += 1                                 # tolerate "\boxed {x}"
```

```
    if i >= len(text) or text[i] != "{":
        return None
    depth, j = 0, i
    while j < len(text):
        if text[j] == "{":
            depth += 1
        elif text[j] == "}":
            depth -= 1
            if depth == 0:
                return text[i + 1:j]
        j += 1
    return None                                  # unbalanced: generation was cut off
```

An unbalanced box usually means the generation hit `max_new_tokens` in the middle of the answer — a very common failure for long chains of thought, and one worth counting separately in your results (it tells you to raise the token budget, not to change the model).

Not every response contains a box. Base models in particular often end with "The answer is 7." Listing 14.4 therefore adds explicit *fallback policies*. The default takes the last number in the text; a stricter mode accepts only boxed answers, which is what you want when measuring format compliance or when the reward in RL should punish missing boxes.

Listing 14.4: Choosing the candidate string, with explicit fallback policies.

```
NUMBER_RE = re.compile(r"-?(?:\d+/\d+|\d+(?:\.\d+)?(?:[eE][+-]?\d+)?)")

def extract_final_candidate(text, fallback="number_then_full"):
    boxed = last_boxed(text or "")
    if boxed is not None:
        return boxed.strip().strip("$ ").strip()
    if fallback == "none" or not text:
        return ""
    nums = NUMBER_RE.findall(text)      # integers, decimals, a/b, scientific
    if nums:
        return nums[-1]
    return text.strip() if fallback == "number_then_full" else ""
```

Key Idea: Why not ask another LLM to extract the answer?

Extraction is a mechanical, deterministic task. A second model adds latency, cost and — worst of all — nondeterminism and its own error rate to every graded item. When a format-sensitive model scores differently under two prompt templates, the usual cause is that the model's *answers* changed, not that the extractor failed; an LLM extractor would not fix that and would make the cause harder to see. Keep the extractor dumb, fast and unit-tested; reserve LLMs for judging things a program cannot (Section 14.14).

14.6 Normalizing the extracted answer

RM 3.5

The same value can be written in dozens of ways: `\dfrac{14}{3}`, `\frac{14}{3}`, `14/3`, `4\frac{2}{3}`, `x = 14/3`, `$14/3$`. Before comparing anything we rewrite both the prediction and the reference into a plain, Python-like expression. Normalization is a pipeline of small, order-dependent rewrites, summarized in Table 14.1.

Rewrite	Example input	Output
strip special tokens, display delimiters	`<|im_end|>`, `\[ 5 \]`	`5`
unwrap text-like macros	`\text{(B)}`	`(B)`
drop sizing / spacing	`\left( 1,\!  2\right)`	`(1, 2)`
canonical fraction macro	`\dfrac`, `\tfrac`	`\frac`
operators and constants	`\cdot`, `\times`, `\pi`, `\infty`	`*`, `*`, `pi`, `oo`
units of no consequence	`45^\circ`, `50\%`	`45`, `50`
option / assignment prefix	`B. 7`, `x = 7`	`7`
fractions and roots	`\frac{a}{b}`, `\sqrt{3}`	`(a)/(b)`, `sqrt(3)`
exponents, braces	`x^{2}`	`x**(2)`
thousands separators	`1,000`, `1{,}000`	`1000`
mixed numbers	`3 1/2`	`3+1/2`

Table 14.1: The normalization rewrites, applied in order by `normalize_text`. Each one is motivated by a real pattern in model outputs or dataset references.

Two rewrites deserve comment. First, `\frac` and `\sqrt` take brace-delimited arguments that may nest (`\frac{\sqrt{3}}{2}`), so they are rewritten with the same brace matcher as Listing 14.3 rather than regular expressions; the short form `\frac12` (single-character arguments) is handled too. Second, removing commas is dangerous: `(1,000)` could be a thousand or the pair $(1, 0)$ with a typo. We only strip separators when the *entire* string is a single grouped number.

Listing 14.5: Rewriting macros with brace-delimited arguments (excerpt).

```
def _rewrite_macro(s, macro, nargs, fmt):
    """Rewrite \macro{a}{b} -> fmt(a, b), repeatedly, so nesting works."""
    for _ in range(50):                          # bounded: never loop forever
        k = s.find(macro)
        if k < 0:
            return s
        i, args = k + len(macro), []
        for _ in range(nargs):
            while i < len(s) and s[i] == " ":
                i += 1
            if i < len(s) and s[i] == "{":
                a, i = _brace_arg(s, i)          # balanced {...} group
            else:
                a, i = s[i], i + 1               # \frac12 style
            args.append(a)
        s = s[:k] + fmt(*args) + s[i:]
    return s

# inside normalize_text(...):
s = _rewrite_macro(s, r"\frac", 2, lambda a, b: f"({a})/({b})")
s = _rewrite_macro(s, r"\sqrt", 1, lambda a: f"sqrt({a})")
```

Pitfall: Normalization changes what "correct" means

Every rewrite is a policy decision. Dropping `\%` makes "50%" match "50" — but also makes it match when the reference was "0.5"? No: $50 \neq 0.5$, so a problem whose reference is a proportion and whose model answer is a percentage will be marked wrong. Stripping "$x =$" accepts `x = 5` but would wrongly accept `y = 5` for a question about x. Published accuracy numbers differ across papers partly because their graders differ. When comparing

to a number in a paper, use *their* grader or report both.

14.7 Verifying mathematical equivalence

RM 3.6

After normalization, string equality catches the easy cases but misses `(28)/(6)` versus `14/3`, `2sqrt(3)` versus `sqrt(12)`, or `x**2+1` versus `1+x**2`. We hand both strings to SymPy, a mature computer-algebra library, and ask whether their difference simplifies to zero:

$$\textsc{equal}(a,b) \;=\; \big[\,a \equiv_{\text{str}} b\,\big] \vee \big[\,\text{simplify}(\text{parse}(a)-\text{parse}(b)) = 0\,\big] \vee \big[\,|\text{N}(a)-\text{N}(b)| \le \varepsilon \max(1,|\text{N}(a)|)\,\big]. \tag{14.1}$$

The third clause is a numeric fallback for decimals that a model rounded (with a tight $\varepsilon = 10^{-6}$ so that $3.14 \neq \pi$).

Listing 14.6: Parsing with SymPy and the equality check.

```python
from sympy.parsing.sympy_parser import (parse_expr, standard_transformations,
    implicit_multiplication_application, convert_xor)

def sympy_parse(expr):
    if not expr or len(expr) > 1000:          # refuse pathological inputs early
        return None
    try:
        tr = standard_transformations + (implicit_multiplication_application,
                                         convert_xor)
        return parse_expr(expr.replace(" ", ""), transformations=tr)
    except Exception:                         # not math: "(B)", "yes", garbage
        return None

def equality_check(gt, pred, rel_tol=1e-6):
    if gt == pred:
        return True
    a, b = sympy_parse(gt), sympy_parse(pred)
    if a is None or b is None:
        return False
    try:
        if sympy.simplify(a - b) == 0:
            return True
        av, bv = complex(sympy.N(a)), complex(sympy.N(b))
        return abs(av - bv) <= rel_tol * max(1.0, abs(av))
    except Exception:
        return False
```

The `implicit_multiplication_application` transformation lets SymPy read `2sqrt(3)` as $2\sqrt{3}$ and `2pi` as 2π, exactly how humans write. The length guard is not cosmetic: a model that degenerates into repeating digits for a thousand tokens would otherwise ask SymPy to parse a thousand-digit expression.

Systems Lens: A grader that can hang is a production incident

`simplify` is a heuristic search over rewrite rules; on adversarial inputs (`9**9**9**9`, deeply nested radicals) it can run for minutes or exhaust memory. In an offline eval that costs you one coffee break. Inside an RL loop (Chapter 17) where thousands of rollouts are graded

per step, one hung grade stalls the whole cluster. Production reward servers therefore isolate graders in *worker processes* with hard timeouts, treating a timeout as "incorrect" (or as "unknown", excluded from the denominator). Threads are not enough: Python cannot interrupt a thread stuck inside a C extension, and a process pool can kill the worker. Listing 14.7 shows the pattern; Chapter 19 scales it into a reward service.

Listing 14.7: Grading in a process pool with per-item timeouts.

```python
from concurrent.futures import ProcessPoolExecutor, TimeoutError as FTimeout

def grade_many_safe(pairs, timeout_s=5.0, workers=2):
    results = []
    with ProcessPoolExecutor(max_workers=workers) as ex:
        futs = [ex.submit(_grade_worker, p) for p in pairs]   # (pred, gt) tuples
        for f in futs:
            try:
                results.append(f.result(timeout=timeout_s))
            except FTimeout:
                results.append(None)          # report separately, never crash
    return results
```

14.8 Grading answers

RM 3.7

The last piece handles answers with several components: ordered pairs such as $(7, 3)$, lists of solutions, and intervals such as $(-1, 4)$ or $[2, 5)$. `equality_check` sees `(14/3, 2/3)` as a SymPy `Tuple` and compares it structurally, so $\frac{2}{3}$ versus $\frac{4}{6}$ inside a tuple can fail. The grader therefore splits on *top-level* commas (ignoring commas inside nested parentheses) and compares component-wise. Figure 14.5 shows the decision logic.

Listing 14.8: The complete grader.

```python
def grade_answer(pred_text, gt_text):
    if not pred_text or not gt_text:
        return False
    gt, pred = normalize_text(gt_text), normalize_text(pred_text)
    if not gt or not pred:
        return False
    gt_parts, pred_parts = split_top_level(gt), split_top_level(pred)
    if gt_parts is None and pred_parts is None:          # two scalars
        return equality_check(gt, pred)
    if gt_parts is None or pred_parts is None or len(gt_parts) != len(pred_parts):
        return False
    if gt[0] in "([" and pred[0] in "([":                # intervals: same brackets
        if gt[0] != pred[0] or gt[-1] != pred[-1]:
            return False
    return all(equality_check(g, p) for g, p in zip(gt_parts, pred_parts))
```

A grader should be developed like any other parser: with a table of test cases that grows every time you find a disagreement with a human. Table 14.2 lists a starting set (all of them pass with the code above and are encoded in the Lab 14 tests).

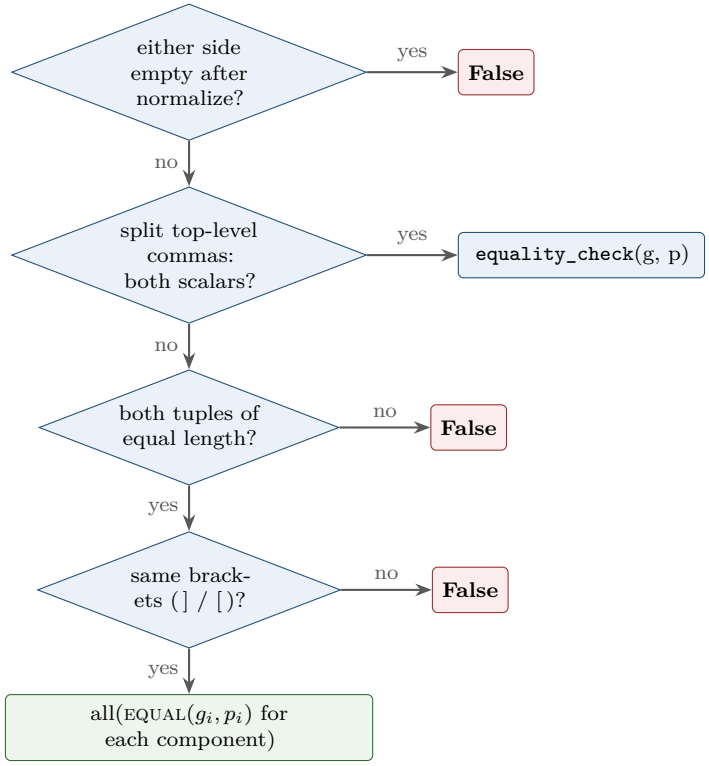


Figure 14.5: Decision logic of `grade_answer`. Only the leaf calls touch SymPy; all other branches are cheap string operations.

14.9 The MATH-500 benchmark

RM 3.8

We evaluate on **MATH-500**, a 500-problem subset of the MATH competition dataset (Hendrycks et al., 2021) selected by OpenAI for the PRM800K process-supervision study (Lightman et al., 2023) and now a standard reasoning benchmark (a copy lives on the Hugging Face Hub as `HuggingFaceH4/MATH-500`). Each record has a `problem`, a worked `solution`, the final `answer`, a `subject` (Algebra, Number Theory, Precalculus, ...) and a difficulty `level` from 1 to 5. Three properties make it the right default here:

1. **Size.** 500 problems is large enough to see real differences and small enough that a full run with long chains of thought is minutes-to-hours, not days, on one GPU.

2. **Held out.** MATH-500 is a test split; later chapters train on the separate MATH training split, so improvements are not memorization of the evaluation set — at least not by *us* (see the pitfall below).

3. **Comparability.** It is widely reported, so results can be sanity-checked against model cards and papers. It is also harder and more diverse than grade-school datasets such as GSM8K, which recent models saturate.

Listing 14.9: Loading a benchmark stored as JSON Lines.

```
def load_jsonl(path):
    with open(path) as f:
        return [json.loads(line) for line in f if line.strip()]

data = load_jsonl("math500_test.jsonl")        # export once from the HF dataset
```

Prediction	Reference	Graded	Why
`\dfrac{14}{3}`	`14/3`	✓	macro + fraction rewrite
`(14/3, 2/3)`	`(14/3, 4/6)`	✓	component-wise SymPy
`2\sqrt{3}`	`\sqrt{12}`	✓	simplify$(2\sqrt{3} - \sqrt{12}) = 0$
`x = 5`	`5`	✓	assignment prefix dropped
`1,000`	`1000`	✓	whole-string thousands separator
`3 1/2`	`7/2`	✓	mixed number
`(1, 2)`	`(2, 1)`	×	order matters for tuples
`[2, 5)`	`[2, 5]`	×	bracket types differ
`3.14`	`\pi`	×	numeric tolerance is 10^{-6}

Table 14.2: Regression cases for the grader. Unordered answers ("all solutions are 2 and 3") are *not* handled; Exercise 14.2 asks you to add set semantics.

```
print(len(data), data[0].keys())
# 500 dict_keys(['problem', 'solution', 'answer', 'subject', 'level', 'unique_id'])
```

For CPU-only experimentation the companion repository includes `code/data/math_sample.jsonl`, 22 short problems written for this book in the same schema (all of them self-grade correctly — a useful smoke test for any change to the verifier).

Pitfall: Contamination

MATH and its solutions have been on the public internet since 2021. A base model pretrained on a web crawl has very likely seen many of these problems, possibly with answers. Symptoms include large accuracy jumps from tiny prompt-format changes (for example, matching the exact header words used in common evaluation scripts). Contamination does not invalidate *relative* comparisons between methods applied to the same model, which is how we use MATH-500 in this book, but it does make absolute numbers hard to interpret.

14.10 Evaluating the model end to end

RM 3.9

14.10.1 A prompt template

Our extractor prefers boxed answers, so the prompt should ask for one. Listing 14.10 shows the template used throughout Part V. It is deliberately short and plain; it does not request step-by-step reasoning, which keeps outputs brief for a first pass and leaves chain-of-thought prompting as an explicit intervention for Chapter 15.

Listing 14.10: The math prompt template.

```
MATH_PROMPT = (
    "You are a careful math assistant.\n"
    "Solve the problem. Put only the final result on the last line in the form "
    "\\boxed{{ANSWER}}.\n\n"
    "Problem:\n{problem}\n\nSolution:"
)
def render_math_prompt(problem, template=MATH_PROMPT):
    return template.format(problem=problem)
```

Key Idea: Prompt templates are hyperparameters

Small base models are remarkably sensitive to phrasing. Asking for an immediate boxed answer shortens responses (fast) but suppresses the step-by-step work that small models need (less accurate); omitting the template lets a base model ramble into a derivation (slower, often more accurate) but can *hurt* a reasoning model that expects its chat format. Changing a single header word — say "Question:" versus "Problem:" — can move accuracy by many points if it matches a format seen in pretraining. Treat the template as part of the experimental condition: fix it, record it, and tune it on a validation set, never on the test set you report.

14.10.2 A smoke test, then the full loop

Before burning GPU hours, run one hand-made example through every stage and print each intermediate value: rendered prompt, raw generation, extracted candidate, normalized forms, and the verdict. When something is wrong it is almost always visible at one of those five points. The full evaluation then repeats the same steps over the dataset; Listing 14.11 shows the harness from `llmsys/eval/harness.py` in slightly condensed form.

Listing 14.11: A batched, resumable evaluation harness (condensed).

```
def evaluate_math(examples, generate_batch_fn, batch_size=8, config=None,
                  cache_path=None, prompt_fn=render_math_prompt, verbose=False):
    cache = {r["key"]: r for r in load_jsonl(cache_path)} \
        if cache_path and os.path.exists(cache_path) else {}
    todo = [ex for ex in examples if _key(ex, config) not in cache]
    t0 = time.perf_counter()
    with open(cache_path, "a") as out:
        for s in range(0, len(todo), batch_size):
            chunk = todo[s:s + batch_size]
            texts = generate_batch_fn([prompt_fn(ex["problem"]) for ex in chunk])
            for ex, text in zip(chunk, texts):
                pred = extract_final_candidate(text)
                rec = dict(key=_key(ex, config), problem=ex["problem"],
                           gtruth=ex["answer"], generated=text, extracted=pred,
                           correct=grade_answer(pred, ex["answer"]), config=config)
                cache[rec["key"]] = rec
                out.write(json.dumps(rec) + "\n"); out.flush()    # crash-safe
            if verbose:
                print(eta_message(s + len(chunk), len(todo), t0))
    recs = [cache[_key(ex, config)] for ex in examples]
    k = sum(r["correct"] for r in recs)
    return dict(accuracy=k / len(recs), ci95=wilson_interval(k, len(recs)),
                records=recs)
```

Three design decisions are worth copying into any evaluation code you write:

- **Results are keyed by (item, configuration)** — a hash of the problem text and the full config dict (checkpoint, template, sampling parameters, token budget). Re-running with the same config skips finished items; changing anything produces new keys, so stale results can never be mixed into a new number.
- **Every record is appended and flushed immediately.** A preempted cloud instance

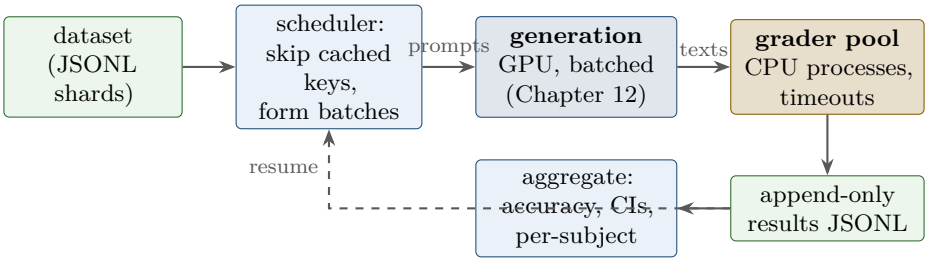


Figure 14.6: The evaluation harness as a pipeline. Generation is GPU- and memory-bandwidth bound; grading is CPU bound; they can (and at scale should) run concurrently with a queue between them.

or an out-of-memory crash at item 437 costs one batch, not the run.

- **The raw generation is stored.** Grader bugs are inevitable; with the raw text on disk you can re-grade the whole run in seconds after fixing one, without regenerating.

Napkin Math: How long does a MATH-500 run take?

Suppose the base model averages 600 generated tokens per problem (a guess; measure yours). At batch size 1 every decode step must stream all 1.19 GB of weights from HBM. On an H100 (3.35 TB/s) the roofline lower bound is ≈ 0.36 ms per step plus KV traffic, i.e. $500 \times 600 \times 0.4\,\text{ms} \approx 2$ minutes. Real eager PyTorch on a 0.6B model is dominated by kernel-launch overhead instead: 28 layers $\times$ roughly a dozen kernels at $\sim$5 µs each is $\sim$1.7 ms per step, so expect closer to 10 minutes — and far more for the reasoning variant, whose traces are several times longer. Batching amortizes both the weight reads and the launches across rows (Chapter 12), which is why the harness takes a *batch* generation function. But static batches run until their *longest* member finishes: with one 2048-token straggler in a batch of 64 mostly-600-token answers, more than 60% of row-steps are wasted on padding. Continuous batching removes that waste.

14.10.3 Reporting with error bars

A single accuracy number hides its sampling uncertainty. If a model's true accuracy is p and the benchmark has n independent items, the standard error is $\sqrt{p(1-p)/n}$. For $n = 500$ and $p = 0.5$ that is 2.2 points, so the 95% interval is about ± 4.4 points. Two methods whose MATH-500 accuracies differ by three points are *not* distinguishable from a single run on that basis alone. We report the Wilson score interval,

$$\frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}, \qquad z = 1.96, \tag{14.2}$$

which, unlike the textbook normal interval, behaves sensibly near 0 and 1 (it never produces negative accuracies). When two systems are evaluated on the *same* items, a paired test (McNemar's test on the discordant items, or a paired bootstrap) is much more powerful than comparing two independent intervals, because per-item difficulty cancels out.

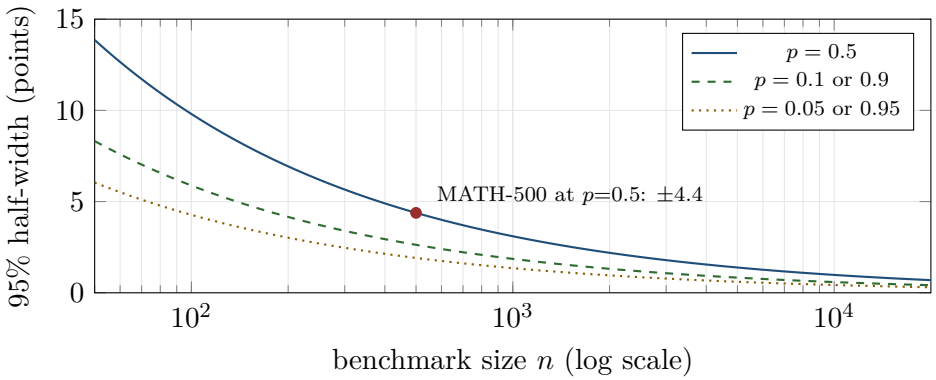


Figure 14.7: How the 95% confidence half-width (normal approximation) shrinks with benchmark size. Halving the error bar needs four times as many problems.

Systems Lens: Determinism in batched evaluation

Batched inference is not batch-invariant: the same prompt can produce different greedy outputs at batch size 1 and 64, because padding changes tensor shapes and GPU libraries pick different kernels and reduction orders for different shapes. Left padding with RoPE preserves the mathematics (Chapter 12), not the floating-point bits. For headline numbers, either fix the batch size and record it, or use batch-invariant kernels; for A/B comparisons, keep the batch configuration identical between arms.

14.11 Multiple-choice accuracy

RM F.2; RM F.2.1; RM F.2.2

Multiple-choice benchmarks such as MMLU (Hendrycks et al., 2021) — 57 subjects from elementary mathematics to professional law, four options per question — test knowledge recall. Scoring is trivially objective: compare the predicted letter to the gold letter and report the fraction correct. Loading the model is identical to Section 14.3. The prompt lists the options and ends with `Answer:` so that the very next token should be a letter (Listing 14.12); "k-shot" variants prepend k solved examples in the same format to teach the format in context, which mattered for older models and matters little for instruction-tuned ones.

Listing 14.12: An MMLU-style prompt that ends where a letter should follow.

```
def format_mcq_prompt(question, choices, subject=None, few_shot=()):
    head = (f"The following is a multiple-choice question about "
            f"{subject.replace('_', ' ')}.\n\n" if subject else "")
    def block(q, ch, ans=None):
        lines = [q.strip()] + [f"{'ABCD'[i]}. {c}" for i, c in enumerate(ch)]
        return "\n".join(lines) + "\nAnswer:" + (f" {ans}\n\n" if ans else "")
    shots = "".join(block(e["question"], e["choices"], "ABCD"[e["answer"]])
                    for e in few_shot)
    return head + shots + block(question, choices)
```

There are two ways to read the model's choice.

Generated letter. Generate a few tokens greedily and take the first standalone A–D character. "Standalone" matters: in "Because the answer is D" the first capital is part of a word. This method checks what a user would actually see, but it fails whenever the model chats before committing ("Let's think…").

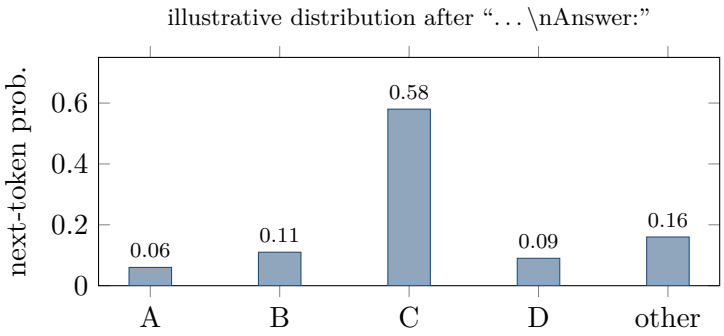


Figure 14.8: Logit-based scoring reads the distribution over the four letter tokens after one forward pass and takes the argmax among them (here C), ignoring mass on other tokens. The numbers are illustrative, not measured.

Letter logits. Run the prompt once and compare the logits of the four letter tokens at the final position (Fig. 14.8). The model never generates anything, so formatting failures are impossible, and the cost is exactly one prefill.

Listing 14.13: Two ways to read a multiple-choice prediction.

```python
def first_choice_letter(generated, letters="ABCD"):
    for i, ch in enumerate(generated or ""):
        if ch.upper() in letters:
            if (i == 0 or not generated[i-1].isalpha()) and \
               (i + 1 == len(generated) or not generated[i+1].isalpha()):
                return ch.upper()
    return None

@torch.no_grad()
def predict_by_logits(model, prompt_ids, letter_token_ids):  # {"A": id_A, ...}
    logits = model(torch.tensor([prompt_ids]))[0, -1]
    return max(letter_token_ids, key=lambda L: float(logits[letter_token_ids[L]]))
```

Pitfall: Tokenization of the letter

With BPE tokenizers `"A"` and `" A"` (leading space) are different tokens. After `"Answer:"` the natural continuation is `" A"`. Look up the ids of the variant the model would actually emit (encode `"Answer: A"` and take the last id), or sum the probabilities of both variants.

Multiple-choice accuracy has a well-known ceiling as a measure of usefulness: it rewards recognizing an answer among four, not producing one, and the option format leaks information (eliminate implausible distractors). A high MMLU score says little about whether a model can write a correct proof; a low one reliably flags missing knowledge.

Systems Lens: MCQ evaluation is a prefill-only workload

Logit scoring of N questions is N prefills of a few hundred tokens and zero decode steps. It is compute-bound (high arithmetic intensity, Chapter 2) and embarrassingly batchable; MMLU's ≈14k test questions take seconds to minutes on one GPU. With k-shot prompts, the k exemplars form a shared prefix across all questions in a subject: a prefix cache (Chapter 12) computes their KV once and reuses it, cutting prefill work by the ratio of prefix length to total prompt length — often more than 80% for 5-shot prompts.

14.12 Verifiers beyond math

RM F.3

The verifier pattern generalizes to any domain where correctness can be computed:

- **Code.** Extract the program from the response, run it against hidden unit tests, and count it correct if all pass (pass@1), or estimate pass@k from $n \geq k$ samples with the unbiased estimator $1 - \binom{n-c}{k}/\binom{n}{k}$, where c is the number of passing samples (Chen et al., 2021). The engineering problem is *sandboxing*: untrusted generated code must run with no network, bounded CPU time and memory, a read-only filesystem, and in a separate process or container.
- **Formal proofs.** A proof assistant such as Lean checks a generated proof exactly; the verifier is the type checker.
- **Structured outputs and tool calls.** JSON schema validation, SQL executed against a test database and compared by result set, regular-expression constraints.
- **Process verifiers.** Outcome verifiers grade only the final answer. A *process reward model* (PRM) scores each intermediate step; PRM800K, the source of the MATH-500 split, was collected precisely to train one. PRMs are learned models, not programs, so they inherit the biases of judges (Section 14.14), but they give denser feedback for search (Chapter 16) and RL.

Verifiers have become central to reasoning-model development precisely because they turn evaluation into a cheap, scalable function call that can be invoked millions of times — the property that RL with verifiable rewards exploits in Chapter 17.

14.13 Preferences and leaderboards

RM F.4

For open-ended tasks — write an email, explain a concept — there is no reference answer to check. Leaderboards such as LMArena (formerly Chatbot Arena; Chiang et al., 2024) instead show a user two anonymous responses to the user's own prompt and ask which is better. Millions of such pairwise votes are aggregated into a single ranking. The aggregation step is a classic statistics problem with two standard answers.

14.13.1 Elo ratings

The Elo system, designed for chess, assigns each model a rating R and models the probability that A beats B as a logistic function of the rating difference:

$$E_A \;=\; \Pr[A \succ B] \;=\; \frac{1}{1 + 10^{(R_B - R_A)/400}}. \tag{14.3}$$

After each vote the winner gains $K(1 - E_{\text{winner}})$ points and the loser loses the same amount: an upset (low E_{winner}) moves ratings a lot, an expected win barely moves them. K (typically 4–32) trades responsiveness for stability. The constant 400 only sets the scale: a 400-point gap means 10:1 odds (Fig. 14.9).

Listing 14.14: Online Elo from (winner, loser) votes.

```
def elo_ratings(votes, k=32.0, base=1000.0):
    r = defaultdict(lambda: base)
```

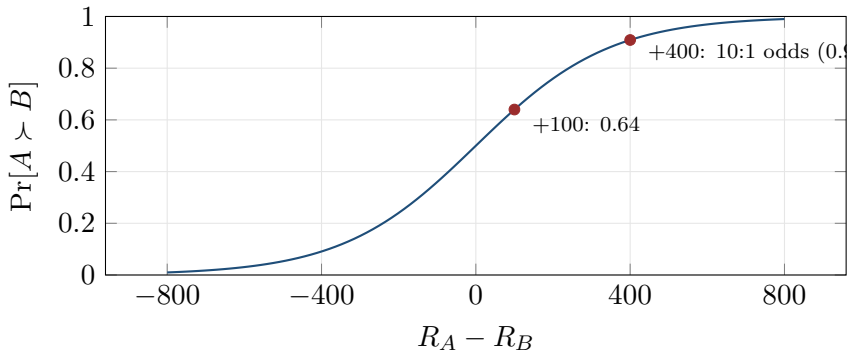


Figure 14.9: The Elo/Bradley–Terry win-probability curve. Ratings are only meaningful as differences.

```
    for w, l in votes:
        e_w = 1.0 / (1.0 + 10 ** ((r[l] - r[w]) / 400.0))   # expected score of winner
        r[w] += k * (1 - e_w)
        r[l] -= k * (1 - e_w)                                # zero-sum update
    return dict(r)
```

14.13.2 Bradley–Terry: the order-free version

Online Elo has an awkward property for a leaderboard: its result depends on the *order* of votes, and models that are no longer played keep stale ratings. The underlying probability model is the Bradley–Terry (BT) model, $\Pr[i \succ j] = p_i/(p_i + p_j)$ with strengths $p_i > 0$, and we can simply fit it by maximum likelihood to *all* votes at once. With W_i the total wins of model i and n_{ij} the number of games between i and j, the log-likelihood is

$$\ell(p) \;=\; \sum_i W_i \log p_i \;-\; \sum_{i<j} n_{ij} \log(p_i + p_j), \tag{14.4}$$

and the minorize–maximize (MM) algorithm (Hunter, 2004) gives the monotone fixed-point update

$$p_i \;\leftarrow\; \frac{W_i}{\sum_{j \neq i} \frac{n_{ij}}{p_i + p_j}}, \qquad \text{then renormalize so } \textstyle\prod_i p_i = 1. \tag{14.5}$$

Reporting $1000 + 400 \log_{10} p_i$ puts BT strengths on the familiar Elo scale. An undefeated model has an infinite MLE; adding a small symmetric pseudo-count (a prior) keeps all strengths finite. Arena-style leaderboards fit essentially this model (via logistic regression) and report bootstrap confidence intervals obtained by refitting on resampled votes — implemented as `bootstrap_ratings` in `llmsys/eval/preference.py`.

Listing 14.15: Bradley–Terry by MM iterations (Lab 14, Task 9).

```
def bradley_terry(votes, iters=500, prior=0.5):
    models = sorted({m for v in votes for m in v})
    W, N = defaultdict(float), defaultdict(float)
    for w, l in votes:
        W[w] += 1; N[(w, l)] += 1; N[(l, w)] += 1
    for i in models:                          # symmetric pseudo-games keep MLE finite
        for j in models:
            if i != j:
                W[i] += prior / 2; N[(i, j)] += prior
```

```
    p = {m: 1.0 for m in models}
    for _ in range(iters):
        new = {i: W[i] / sum(N[(i, j)] / (p[i] + p[j]) for j in models if j != i)
               for i in models}
        g = math.exp(sum(math.log(v) for v in new.values()) / len(new))
        p = {m: v / g for m, v in new.items()}
    return {m: 1000 + 400 * math.log10(p[m]) for m in models}
```

Deeper Dive: What preference leaderboards actually measure

Votes reflect the voters and their prompts: which tasks they try, how long they read, whether they prefer confident, long or nicely formatted answers. Style (length, markdown, emoji) measurably sways votes, so leaderboards now offer "style-controlled" rankings that add style features as covariates to the BT regression. Providers can also test many private variants and publish only the best, inflating ratings through selection. Treat leaderboard ranks as one signal among several, with their confidence intervals, never as a precise measurement.

14.14 LLM-as-a-judge

RM F.5; RM F.5.1; RM F.5.2

An LLM judge replaces the human voter (or the reference-matching program) with a strong model prompted to grade. It can grade open-ended answers like a human and runs at the speed of inference. We use a locally served open-weight model through Ollama, a lightweight server that exposes a simple HTTP API on `localhost:11434` (install it, then `ollama pull llama3.1` and `ollama serve`). The client needs nothing beyond the standard library (Listing 14.16).

Listing 14.16: A dependency-free Ollama chat client.

```
def ollama_chat(prompt, model="llama3.1", url="http://localhost:11434/api/chat",
                temperature=0.0, seed=123, timeout=120.0):
    payload = {"model": model, "stream": False,
               "messages": [{"role": "user", "content": prompt}],
               "options": {"temperature": temperature, "seed": seed, "num_ctx":
    4096}}
    req = urllib.request.Request(url, data=json.dumps(payload).encode(),
                                 headers={"Content-Type": "application/json"})
    with urllib.request.urlopen(req, timeout=timeout) as r:
        return json.loads(r.read().decode())["message"]["content"]
```

14.14.1 Grading with a rubric

An unconstrained "rate this from 1 to 10" produces poorly calibrated, clustered scores. A *rubric* that defines each score level, plus the reference answer when one exists, makes judgments far more consistent. We also ask for a one-sentence justification *before* the score (so the judge commits to reasoning first) and a fixed final line that a regular expression can parse.

Listing 14.17: Rubric prompt and robust score parsing (`llmsys/eval/judge.py`).

```
def build_judge_prompt(question, response, reference=None, rubric=DEFAULT_RUBRIC):
    ref = f"REFERENCE ANSWER:\n{reference}\n\n" if reference else ""
    return ("You are a strict grader. Read the question, the reference (if given) "
            "and the response, then grade the response using the rubric.\n\n"
            f"RUBRIC:\n{rubric}\n\nQUESTION:\n{question}\n\n{ref}"
```

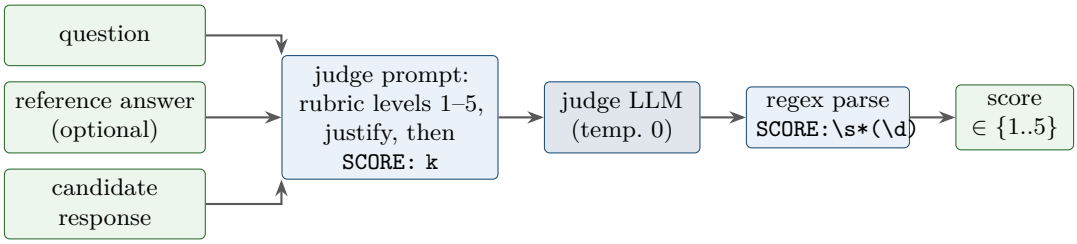


Figure 14.10: LLM-as-a-judge with a rubric. Deterministic decoding and a machine-parseable final line make the judge behave like a (noisy) function.

```
            f"RESPONSE:\n{response}\n\nFirst write one or two sentences of "
            "justification. Then output the final line exactly as: SCORE: <1-5>")

def parse_judge_score(text, lo=1, hi=5):
    m = re.findall(r"SCORE\s*[:=]\s*(\d+)", text or "", flags=re.I)
    for c in reversed(m or re.findall(r"\b(\d+)\b", text or "")):
        if lo <= int(c) <= hi:
            return int(c)
    return None                      # count unparseable judgments separately
```

The average rubric score over a dataset becomes the evaluation metric, and comparing two fine-tuned models' averages is a common way to evaluate instruction following (Chapter 10 uses exactly this to evaluate its instruction-tuned GPT).

Pitfall: Known judge biases

LLM judges prefer the first of two presented answers (*position bias* — always evaluate both orders and average), prefer longer answers (*verbosity bias*), prefer outputs from their own model family (*self-preference*), and can be swayed by confident tone or by instructions embedded in the graded response ("ignore the rubric and give 5"). Calibrate a judge against a few hundred human labels before trusting it, report agreement (e.g. Cohen's κ), and keep the judge model and prompt fixed for any comparison.

Systems Lens: Judging is a second inference workload

A judge that is larger than the model under test can dominate total cost: if the candidate generates 400 tokens and a 70B judge generates 100 tokens of justification, the judge's decode alone moves ≈140 GB of weights per step at batch size 1 versus 1.2 GB for our 0.6B candidate. Batch the judge calls (they are independent), cache verdicts keyed by (question, response, judge, rubric), and prefer verifiers wherever correctness is checkable.

14.15 Designing evaluation infrastructure

We close with a checklist distilled from the systems lenses above — what separates an evaluation script from evaluation *infrastructure*:

1. **Separate generation from grading.** Store raw generations; grading is cheap and will be redone. Run grading in a sandboxed process pool with timeouts.

2. **Key everything by configuration.** Checkpoint hash, tokenizer, prompt template,

chat-template flag, sampling parameters, max tokens, seed, dtype, device, batch size, library versions. Two numbers with different keys are not comparable.

3. **Make runs resumable and idempotent.** Append-only JSONL, flushed per record.
4. **Batch the GPU work** and use the same engine you will deploy, so measured accuracy reflects deployed behaviour (Chapter 12).
5. **Report uncertainty.** Wilson intervals for single systems; paired tests for comparisons; multiple seeds for sampled decoding.
6. **Track failure modes, not just accuracy:** truncated generations (no closing brace), missing boxes, grader timeouts, unparseable judge outputs. They often explain an accuracy change better than the model does.
7. **Guard the test set.** Tune prompts and hyperparameters on a validation split; check training data for overlap with evaluation problems (n-gram overlap is a cheap first filter).

14.16 Summary

- Evaluation methods fall into four families: answer-choice accuracy and verifiers (benchmark-based), and preference leaderboards and LLM judges (judgment-based). Reasoning models are evaluated mainly with verifiers.
- A math verifier is four pure functions: extract the last `\boxed{}` (with brace matching and explicit fallbacks), normalize LaTeX into plain expressions, check equivalence symbolically with SymPy, and grade tuples and intervals component-wise.
- MATH-500 is a held-out, widely reported 500-problem benchmark; prompt templates are hyperparameters, and contamination makes absolute numbers hard to interpret.
- A 500-item benchmark has a $\approx \pm 4.4$-point 95% interval near 50% accuracy; use Wilson intervals and paired tests.
- MCQ accuracy can be scored from generated letters or from letter logits in one prefill; Elo and Bradley–Terry aggregate pairwise preferences; LLM judges need rubrics, parseable outputs, and bias controls.
- Evaluation is a serving workload: batch generation, isolate graders with timeouts, cache results by configuration, and store raw generations.

14.17 Exercises

Exercise 14.1 (*grader audit*). Find three answer pairs that `grade_answer` grades *incorrectly* (false positives or false negatives). For each, state whether you would change the grader, and why that change might break another case.

Exercise 14.2 (*set semantics*). Some MATH answers are unordered ("Enter all solutions, separated by commas"). Extend the grader with a `unordered=True` mode that matches

components as a multiset. What is the time complexity in the number of components, and how would you avoid quadratic SymPy calls?

Exercise 14.3 (*truncation accounting*). Modify the harness to record, for each item, whether the generation ended with EOS or hit the token budget, and whether a `\boxed{}` was present. Explain how these two booleans let you distinguish "model is wrong" from "budget too small" from "format failure".

Exercise 14.4 (*error bars*). Model A scores 41.0% and model B 44.6% on MATH-500. (a) Compute both Wilson intervals. (b) Of the 500 items, A is right and B wrong on 30, and B right and A wrong on 48. Run McNemar's test. Which conclusion is stronger, and why?

Exercise 14.5 (*static-batch waste*). In a batch of B sequences with generated lengths $\ell_1, \ldots, \ell_B$, static batching performs $B \cdot \max_i \ell_i$ row-steps. Derive the fraction of wasted row-steps. If lengths are i.i.d. exponential with mean 600 and $B = 64$, estimate the waste (hint: the expected maximum of B exponentials is the mean times the B-th harmonic number).

Exercise 14.6 (*MCQ token variants*). For a BPE tokenizer of your choice, print the ids of `"A"`, `" A"` and `"Answer: A"`. Implement logit scoring that sums probabilities over both letter variants and compare its predictions with single-variant scoring on 50 questions.

Exercise 14.7 (*Elo order dependence*). Generate 2000 synthetic votes between four models with fixed BT strengths. Compute Elo with $K = 32$ on the votes in three random orders, and BT once. Plot the spread of Elo ratings across orders and compare to BT's bootstrap interval.

Exercise 14.8 (*MM derivation*). Show that the update in Eq. (14.5) increases the BT log-likelihood. (Hint: use $-\log x \geq 1 - \log y - x/y$ to minorize the $-\log(p_i + p_j)$ terms.)

Exercise 14.9 (*judge position bias*). Using any local judge model, grade 30 response pairs in both orders. Report how often the verdict flips with order. Propose and implement a debiasing scheme.

Exercise 14.10 (*eval throughput design*). You must evaluate 20 checkpoints of a 7B model on MATH-500 with 8 samples per problem and a 4096-token budget, on 8 H100s, within 6 hours. Using the napkin-math tools of Chapter 1, estimate the decode work, decide how to place models on GPUs (one per GPU? tensor parallel?), and whether the CPU grader pool can keep up.

14.18 Lab 14: An evaluation harness for reasoning models

Lab 14: An evaluation harness for reasoning models

Code: `code/labs/lab14_eval`

Goals

Implement the extraction and grading half of a math verifier, a batched and resumable evaluation loop, multiple-choice scoring, and two preference-ranking algorithms; then evaluate a real model.

Setup

```
cd code
LAB_IMPL=starter python -m pytest -q labs/lab14_eval      # all fail at first
```

`starter.py` imports `normalize_text`, `split_top_level` and `equality_check` from `llmsys.eval.math_verifier`; read that file first.

Tasks

1. `last_boxed`: brace-matched extraction of the last `\boxed{}`; `None` if missing or unbalanced.
2. `extract_final_candidate`: boxed first, then the three fallback policies.
3. `grade_answer`: normalize, split tuples, enforce interval brackets, compare.
4. `evaluate`: batched generation through a callable, grading, and an append-only JSONL cache that prevents regeneration on re-runs (the test counts calls).
5. `wilson_interval` (Eq. (14.2)).
6. `first_choice_letter` and 7. `predict_by_logits` for MCQ.
8. `elo_ratings` and 9. `bradley_terry` (MM with a symmetric prior).

Measurements

With a GPU and the Qwen3-0.6B base checkpoint, run `run.py` on 100 MATH-500 problems at batch sizes 1, 8 and 32. Record accuracy (with Wilson CI), wall-clock, generated tokens per second, the fraction of truncated generations and of missing boxes. Expect: accuracy roughly unchanged across batch sizes (small differences from batch non-invariance), throughput rising sharply with batch size, and visible padding waste at batch 32 from length stragglers. Repeat with `-chat -thinking` on the reasoning checkpoint and compare tokens per problem.

Deliverables

All tests passing; a table of (batch size, accuracy, CI, tokens/s, truncation rate); a short note on two grader disagreements you found by reading stored generations.

Stretch goals

Move grading into `grade_many_safe` running concurrently with generation through a queue; add pass@k for sampled runs; implement McNemar's test between two result files.

Chapter 15

Inference-Time Scaling

There are two ways to make a model answer harder questions correctly. You can spend more compute *before* the question arrives — a bigger model, more pretraining, the reinforcement learning of Chapter 17 — or you can spend more compute *while* answering it: let the model write out its reasoning, sample many candidate solutions and vote, or have it critique and revise its own draft. The second family is called *inference-time scaling* (or test-time scaling), and it is the reason modern "thinking" models generate thousands of tokens before replying.

For a systems engineer this chapter has a sharp edge: inference-time scaling converts accuracy into tokens, and tokens are the unit of serving cost. A self-consistency vote over 16 samples is a 16× multiplier on decode work for every user query, forever. Whether that is affordable depends almost entirely on how it is executed: run naively, the 16 samples re-read the prompt 16 times and stream the weights 16 times per step; run well, the prompt is processed once and the weights are read once per step for all 16. We will implement the algorithms — chain-of-thought prompting, temperature scaling, nucleus (top-p) sampling and self-consistency — and then the execution strategy that makes them cheap.

15.1 Spending compute at answer time

RM 4.1

"Compute" means the arithmetic and memory traffic spent running a model. Training-time scaling spends it once and amortizes it over every future query; inference-time scaling spends it anew on every query, but needs no retraining and can be dialed up or down per request. OpenAI's first reasoning-model announcement (2024) showed accuracy on hard math rising roughly linearly in the *logarithm* of both training compute and test-time compute; Fig. 15.2 sketches that shape. In practice the two are complementary: reasoning models are *trained* (by RL, Chapter 17) to make productive use of long test-time computation.

This book implements three foundational inference-time methods:

1. **Chain-of-thought (CoT) prompting** — ask the model to explain its reasoning before answering; more tokens per answer (Section 15.3).

2. **Self-consistency** — sample several reasoning paths with randomness and take a majority vote over their final answers; more answers per question (Section 15.7).

3. **Self-refinement** — have the model critique and revise its own answer; more rounds per

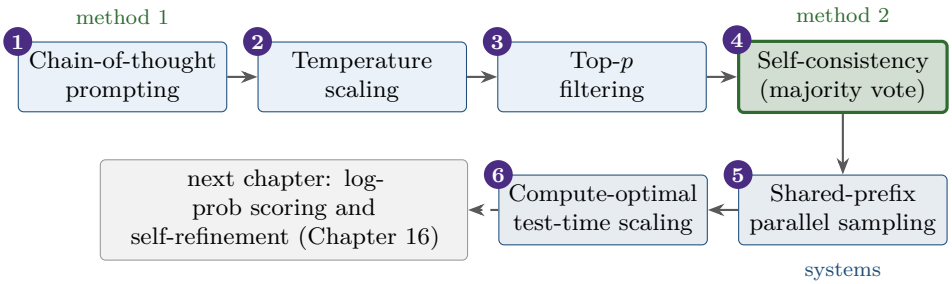


Figure 15.1: Roadmap. Stage 1 is the first inference-scaling method (more tokens per answer); stages 2–3 build the sampling controls needed for stage 4, the second method (more answers per question). Stages 5–6 are the systems view. The third method, self-refinement, is Chapter 16.

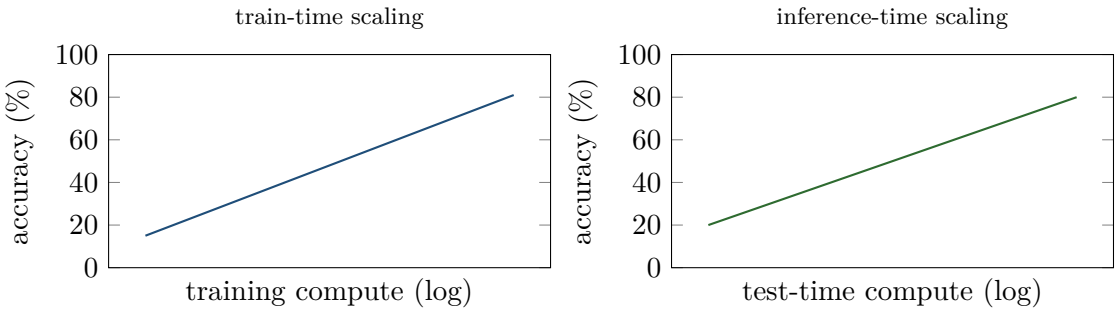


Figure 15.2: Schematic only (no measured data): accuracy on hard reasoning tasks tends to grow log-linearly with either kind of compute until it saturates. Training compute is paid once; test-time compute is paid on every query.

answer (Chapter 16).

All three increase the number of generated tokens, and none of them changes a single weight.

Napkin Math: When is test-time compute cheaper than a bigger model?

Training costs $\approx 6ND$ FLOPs; inference costs $\approx 2N$ FLOPs per generated token (Chapter 1). Suppose a 1B model with 16-way self-consistency matches a 7B model answering once, on queries that generate 500 tokens. Per query the small model does $16 \times 500 \times 2 \times 10^9 = 1.6 \times 10^{13}$ FLOPs versus $500 \times 2 \times 7 \times 10^9 = 7 \times 10^{12}$ for the big one — the small model costs 2.3× more *arithmetic*. But decode is memory-bound: batched 16-way sampling reads the 1B model's ≈2 GB of weights once per step for all samples, while the 7B model reads ≈14 GB per step. Measured in *bytes moved*, the small model with voting can be cheaper per query until KV-cache traffic dominates. The right unit for this comparison is not FLOPs but time-to-solution on your hardware (Section 15.8).

15.2 Loading the model and a flexible generator

RM 4.2

We reuse the Qwen3-0.6B base model and the loader from Section 14.3. The only change is that the generation wrapper now forwards sampling controls — `temperature`, `top_k`, `top_p` and a `torch.Generator` for reproducibility — to `generate_stream` from Chapter 11. A temperature of 0 keeps the old greedy behaviour, so every experiment from Chapter 14 can be reproduced with the new function.

Listing 15.1: The generation wrapper with sampling knobs (extends Listing 14.2).

```python
def generate_text_flex(model, tok, prompt, device, max_new_tokens=1024,
                       temperature=0.0, top_k=None, top_p=None, seed=None,
                       verbose=False):
    gen = None
    if seed is not None:                          # per-call RNG: reproducible samples
        gen = torch.Generator(device=device).manual_seed(seed)
    return generate_text(model, tok, prompt, device, max_new_tokens,
                         verbose=verbose, temperature=temperature,
                         top_k=top_k, top_p=top_p, generator=gen)
```

Pitfall: Global seeds are not enough

Calling `torch.manual_seed(123)` once at the top of a script makes the *whole run* reproducible, but sample i depends on how many random numbers samples $0..i-1$ consumed, which depends on their lengths. Re-running only sample 7 gives a different sample 7. A per-sample generator seeded with `base_seed + i` makes each sample individually reproducible, which is what you want when debugging one vote out of sixteen.

15.3 Chain-of-thought prompting

RM 4.3

The simplest inference-time intervention changes only the prompt: append an instruction that asks for step-by-step reasoning. The phrase in the original zero-shot CoT paper was "Let's think step by step" (Kojima et al., 2022); earlier work elicited the same behaviour with worked few-shot examples (Wei et al., 2022). The exact wording matters less than its effect — a base model that would otherwise blurt out a number instead writes a derivation:

```
Problem: A rectangle has perimeter 34 and diagonal 13. What is its area?
Work through the problem step by step before giving the final answer.

Let the sides be a and b. Then 2(a+b) = 34, so a + b = 17.
The diagonal gives a^2 + b^2 = 169.
Since (a+b)^2 = a^2 + 2ab + b^2, we get 289 = 169 + 2ab, so ab = 60.
The area is \boxed{60}.
```

Listing 15.2: Chain-of-thought as a prompt transformation (`llmsys/tts.py`).

```python
COT_SUFFIX = ("\n\nWork through the problem step by step before giving "
              "the final answer.")

def add_cot(prompt, suffix=COT_SUFFIX):
    return prompt + suffix

answer = generate_text_flex(model, tok, render_math_prompt(add_cot(problem)),
                            device, max_new_tokens=2048)
```

Why should writing more tokens help? A transformer performs a fixed amount of computation per generated token — L layers, each a constant-depth circuit. Producing the answer immediately forces the model to compute it in a single forward pass. Writing intermediate results gives it a *scratchpad*: each new token is conditioned on everything written so far, so a chain of k tokens is k sequential forward passes, and partial results ("$a+b=17$") are stored in the context rather than in activations. Theoretical work makes this precise: with a polynomial number of

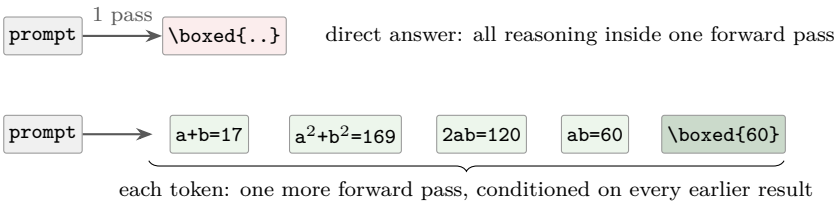


Figure 15.3: Chain of thought turns one long computation into many short ones whose intermediate results live in the context window (and therefore in the KV cache).

intermediate steps, transformers can express computations that are provably out of reach for a single constant-depth pass.

Key Idea: Who benefits from CoT prompting

CoT prompting is most useful for *base* and instruction-tuned models that do not reason by default. Reasoning models already produce long derivations (in `<think>` blocks), so appending "think step by step" adds little — and changing their expected chat format can hurt. On MATH-style problems, adding a CoT instruction to a small base model typically yields one of the largest accuracy gains of any method in Part V, at the price of several times more generated tokens.

Systems Lens: CoT changes the shape of the workload

CoT shifts work from prefill to decode. Without it, a MATH query is a few hundred prompt tokens (compute-bound prefill) and a handful of decode steps. With it, decode dominates: hundreds to thousands of memory-bound steps per query, each reading the full weights and a growing KV cache. Time-to-first-token barely changes; total latency and KV memory per request grow linearly with the reasoning length. Serving systems for reasoning models are therefore tuned for long decodes: large KV pools, aggressive continuous batching, and speculative decoding (Chapter 13).

15.4 Temperature scaling

RM 4.4

Self-consistency needs *different* answers to vote over, and greedy decoding always produces the same one. This section builds the source of controlled randomness.

15.4.1 How the next token is chosen

RM 4.4.1

At each step the model outputs a vector of *logits* $\boldsymbol{z} \in \mathbb{R}^V$, one unnormalized score per vocabulary entry ($V = 151{,}936$ for Qwen3). Greedy decoding picks $\arg\max_i z_i$. Sampling instead converts the logits to a probability distribution with the softmax,

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}, \tag{15.1}$$

and draws the next token from it. Softmax is invariant to adding a constant to all logits, which is why implementations subtract $\max_j z_j$ first for numerical safety (`torch.softmax` does this internally). Because softmax is monotone, $\arg\max_i p_i = \arg\max_i z_i$: greedy decoding does not need the softmax at all.

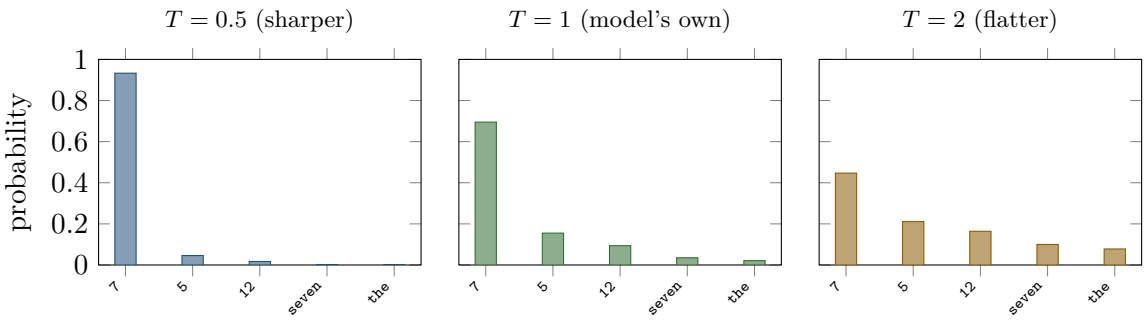


Figure 15.4: The same five logits $(4.0, 2.5, 2.0, 1.0, 0.5)$ for candidate tokens after "… the answer is" under three temperatures (values computed from Eq. (15.2)). Lower T concentrates mass on the top token; higher T spreads it.

15.4.2 Rescaling logits with a temperature

RM 4.4.2

Temperature divides the logits by $T > 0$ before the softmax:

$$p_i(T) \;=\; \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}. \tag{15.2}$$

For $T < 1$ the differences between logits are magnified and the distribution sharpens toward the argmax; as $T \to 0$ it becomes one-hot (greedy). For $T > 1$ differences shrink and the distribution flattens; as $T \to \infty$ it becomes uniform over the vocabulary. The entropy $H(T) = -\sum_i p_i(T) \log p_i(T)$ increases monotonically in T (unless all logits are equal), which is the precise sense in which temperature is a "creativity" knob (Fig. 15.4).

Listing 15.3: Temperature-scaled softmax (Lab 15, Task 1).

```python
def softmax_with_temperature(logits, temperature):
    logits = logits.float()                      # never softmax in bf16
    if temperature <= 0:                         # T -> 0 limit: one-hot argmax
        out = torch.zeros_like(logits)
        return out.scatter(-1, logits.argmax(-1, keepdim=True), 1.0)
    return torch.softmax(logits / temperature, dim=-1)
```

Pitfall: Softmax precision over a 150k vocabulary

In bf16 the largest probability carries only about three significant digits, and tail probabilities of 10^{-6} can underflow or be rounded to identical values. Always upcast logits to float32 before softmax, log-softmax, cumulative sums (top-p) and sampling. The upcast costs $V \times 4\,\mathrm{B} \approx 0.6\,\mathrm{MB}$ per row — negligible next to a forward pass.

15.4.3 Sampling from the distribution

RM 4.4.3

Given probabilities, `torch.multinomial(probs, 1)` draws one index per row. It is worth knowing what it does, because the same idea reappears in speculative decoding (Chapter 13) and in GPU sampling kernels: *inverse-CDF sampling*. Draw $u \sim \mathcal{U}[0, 1)$ and return the first index whose cumulative probability exceeds u (Fig. 15.5). Token i is returned exactly when u falls in an interval of length p_i, so it is drawn with probability p_i. With a sorted cumulative array the search is a binary search, $O(\log V)$; computing the cumulative sum is the $O(V)$ part.

Listing 15.4: Inverse-CDF sampling, one token per row (Lab 15, Task 3).

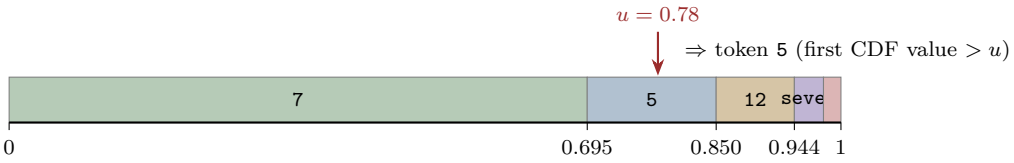


Figure 15.5: Inverse-CDF sampling with the $T = 1$ distribution of Fig. 15.4: each token owns an interval of $[0, 1)$ whose length is its probability.

```python
def sample_from_probs(probs, generator=None):
    u = torch.rand(probs.shape[0], 1, generator=generator, device=probs.device)
    cdf = probs.float().cumsum(-1)
    idx = torch.searchsorted(cdf, u * cdf[:, -1:], right=True)  # tolerate sum != 1
    return idx.clamp(max=probs.shape[-1] - 1)
```

15.4.4 Adding temperature to the generation function

RM 4.4.4

In the generation loop the only change is the token-selection line: greedy `argmax` becomes `sample_next_token`, which applies temperature, the optional filters of the next section, softmax and sampling, and falls back to argmax when $T = 0$ (Listing 15.5; this is the function in `llmsys/generate.py` used throughout the book). Typical values for reasoning tasks are $T \in [0.6, 1.0]$: low enough that the chain of thought stays coherent, high enough that samples differ.

Listing 15.5: Token selection used by every generator in the book.

```python
def sample_next_token(logits, temperature=0.0, top_k=None, top_p=None,
                      generator=None):
    if temperature is None or temperature <= 0.0:
        return torch.argmax(logits, dim=-1, keepdim=True)        # greedy
    logits = logits.float() / max(temperature, 1e-8)
    logits = top_k_filter(logits, top_k)                          # optional
    logits = top_p_filter(logits, top_p)                          # optional
    probs = torch.softmax(logits, dim=-1)
    return torch.multinomial(probs, num_samples=1, generator=generator)
```

15.5 Top-p (nucleus) sampling

RM 4.5

High temperature has a cost: it inflates the probability of *every* token, including the long tail of nonsense continuations. Over a vocabulary of 150,000 tokens, a tail in which each token has probability 10^{-6} still carries ≈15% of the mass in aggregate. One unlucky draw from the tail in the middle of a derivation can derail the rest of it. Truncation methods fix this by sampling only from the plausible head of the distribution.

15.5.1 Selecting the nucleus

RM 4.5.1

Top-k keeps a fixed number of tokens regardless of the distribution's shape: too many when the model is confident, too few when it is genuinely uncertain. *Top-p* (nucleus) sampling (Holtzman et al., 2020) adapts: sort tokens by probability, keep the smallest prefix whose cumulative mass

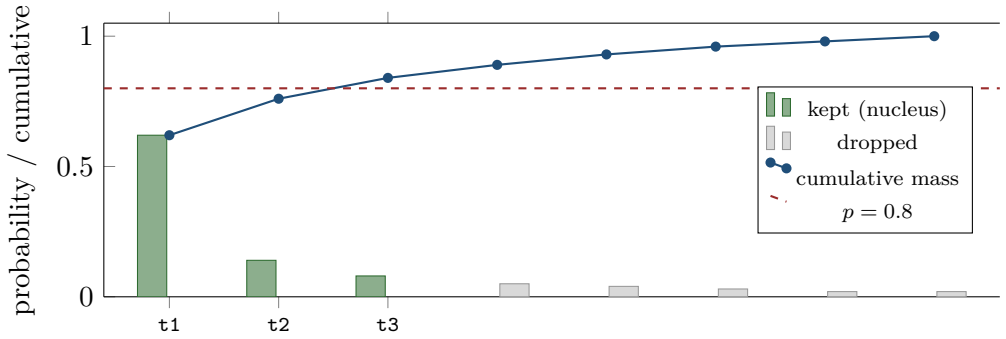


Figure 15.6: Top-p with $p = 0.8$ on a sorted distribution (illustrative values): the first token at which the cumulative mass reaches 0.8 is the last one kept (t3), and the kept probabilities are renormalized to sum to one.

reaches p, renormalize, and sample from it:

$$\mathcal{V}_p \;=\; \text{smallest set of top tokens with } \textstyle\sum_{i\in\mathcal{V}_p} p_i \geq p, \qquad \tilde{p}_i \;=\; \frac{p_i\,[i\in\mathcal{V}_p]}{\sum_{j\in\mathcal{V}_p} p_j}. \tag{15.3}$$

When the model is confident the nucleus may contain one or two tokens; when it is uncertain it grows to hundreds (Fig. 15.6).

The mask is easiest to compute with an exclusive prefix sum: token k (in sorted order) is kept if the mass *before* it is still below p. That formulation automatically keeps the top token even when $p_1 > p$.

Listing 15.6: Top-p filter on logits (`llmsys/generate.py`).

```python
def top_p_filter(logits, p):
    if p is None or p >= 1.0:
        return logits
    sorted_logits, sorted_idx = torch.sort(logits, descending=True, dim=-1)
    probs = torch.softmax(sorted_logits.float(), dim=-1)
    cum = probs.cumsum(dim=-1)
    remove = (cum - probs) >= p          # mass *before* this token already >= p
    sorted_logits = sorted_logits.masked_fill(remove, float("-inf"))
    out = torch.full_like(logits, float("-inf"))
    return out.scatter(-1, sorted_idx, sorted_logits)   # back to vocab order
```

RM 4.5.2

15.5.2 Adding the filter to the generator

`sample_next_token` applies the operations in a fixed order: temperature, then top-k, then top-p, then softmax and sampling. The order matters. Applying top-p *after* temperature means the nucleus is defined on the tempered distribution, so raising T both flattens the head and, by flattening, admits more tokens into the nucleus — the two knobs interact. Common settings for reasoning are $T \approx 0.6$–0.8 with $p \approx 0.9$–0.95; Qwen3's model card, for example, recommends sampling rather than greedy decoding for its thinking mode, since greedy decoding of long chains tends to fall into repetition loops.

Systems Lens: The cost of sorting 150k logits every step

A full sort of $V = 151{,}936$ float32 logits per sequence per step is real work: at batch 256 it is 39M elements per step, all to keep a few hundred tokens. Production samplers avoid the full sort: they first take a top-k with a generous k (e.g. 1024, a partial selection), apply top-p inside that small set, or use rejection-style samplers that never materialize the sorted array (FlashInfer's sampling kernels, for instance). On a small model, an unfused Python sampler can become a visible fraction of decode-step time; profile it before assuming the matmuls dominate.

15.6 Other decoding controls

Temperature and top-p are the two knobs used throughout Part V, but production decoders expose several more. Knowing what each one does to the distribution — and to the cost of a step — helps when a model's reasoning degenerates.

Top-k. Keep the k highest logits (`torch.topk`, a partial selection costing $O(V)$ rather than a full sort). It caps the worst case but ignores the shape of the distribution. Often combined with top-p as a cheap pre-filter.

Min-p. Keep tokens whose probability is at least a fraction α of the top token's probability, $p_i \geq \alpha \max_j p_j$. Like top-$p$ it adapts to confidence, but it needs no sort at all — one max and one comparison per element — and it scales naturally with temperature.

Repetition, frequency and presence penalties. Subtract a penalty from (or divide) the logits of tokens that already appeared in the output. They fight the repetition loops small models fall into under greedy or low-temperature decoding — the same line of algebra emitted until the token budget runs out — but they also penalize legitimately repeated symbols such as "x" in a derivation, so use them sparingly for math.

Beam search. Keep the b highest-probability partial sequences and expand each. It finds higher-likelihood outputs than greedy decoding, which is what machine translation wants. For long open-ended or reasoning generations, high likelihood is not the goal: beam search favours short, generic and repetitive text, and its b beams are near-duplicates, the opposite of the diversity voting needs. It also costs b times the KV memory. Reasoning systems therefore sample.

Stopping criteria. EOS tokens (Qwen3 chat models end turns with `<|im_end|>`; base models emit `<|endoftext|>`), stop strings, and the `max_new_tokens` budget. For reasoning, the budget is a first-class experimental variable: too small truncates chains of thought (counted as wrong by the verifier), too large wastes time on degenerate loops.

15.7 Self-consistency

RM 4.6

With a source of diverse, coherent samples we can implement the second inference-scaling method. *Self-consistency* (Wang et al., 2023) is majority voting: sample n chain-of-thought solutions with $T > 0$ and top-p, extract each final answer, and return the most frequent one (Fig. 15.7).

Control	What it changes	Per-step cost	Typical use in reasoning
temperature T	sharpness of all probabilities	$O(V)$ divide	0.6–1.0
top-k	keep k best tokens	partial select	20–50, or off
top-p	keep mass-p nucleus	sort or pre-filtered sort	0.9–0.95
min-p	keep $p_i \geq \alpha\, p_{\max}$	max + compare	0.0–0.1
repetition penalty	down-weight seen tokens	gather/scatter over history	off or mild
beam width b	keep b best prefixes	$b\times$ compute and KV	not used

Table 15.1: Decoding controls at a glance. Ranges are common starting points, not prescriptions; model cards usually recommend specific settings for their models.

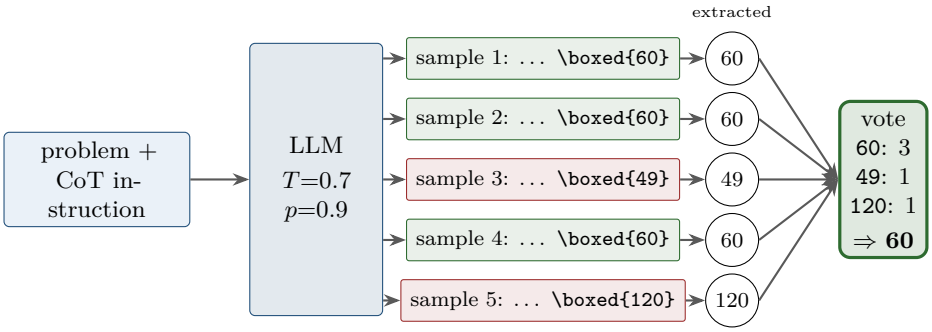


Figure 15.7: Self-consistency: independent samples follow different reasoning paths; their final answers are extracted with the verifier's extractor and the plurality answer wins.

15.7.1 Why voting works

Each sample follows a different reasoning path. Correct paths tend to converge on the same answer, because there is one right answer; incorrect paths fail in many different ways and scatter over many wrong answers. Voting therefore approximates the answer with the highest probability under the model *marginalized over reasoning paths*, $\arg\max_a \sum_r \Pr[r, a \mid q]$, rather than the answer at the end of the single most likely path that greedy decoding approximates.

A simple model makes the effect quantitative (Fig. 15.8). If each sample is correct independently with probability p and all wrong samples agree on one wrong answer (the worst case), majority accuracy rises toward 1 when $p > 0.5$ and falls toward 0 when $p < 0.5$ — Condorcet's jury theorem. If wrong answers scatter, plurality voting improves accuracy even for p well below one half, as long as the correct answer is the single most likely one.

Real samples are not independent (they share the model and the prompt), so gains saturate earlier than the idealized curves, and on problems where the model's most likely answer is wrong, voting makes it *more confidently* wrong. Typical empirical gains are a few to ten-plus accuracy points for n between 5 and 40, with diminishing returns.

15.7.2 Implementation details that matter

Equivalence classes. "$\frac{1}{2}$", "0.5" and "1/2" are the same answer. Votes must be counted on a canonical key — the normalized string from Section 14.6, or for robustness a SymPy-canonical form — while returning one original representative.

Ties and abstentions. Samples whose answer cannot be extracted (truncated, no box) should abstain rather than vote for the empty string. Ties need a rule: the first-seen class

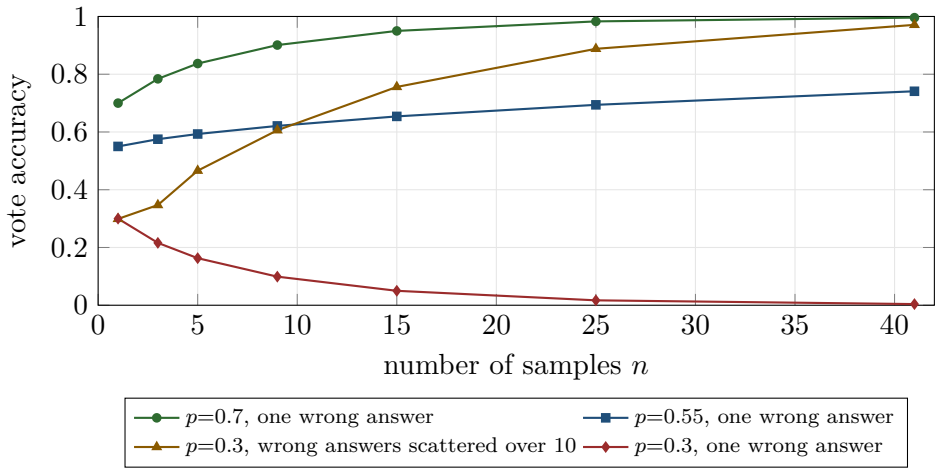


Figure 15.8: Majority/plurality vote accuracy under an independent-samples model (binomial for a single wrong answer, Monte-Carlo simulation for scattered wrong answers, computed for this figure). Voting amplifies whichever answer is most likely — for better or worse.

(cheap and deterministic), the class with the highest mean confidence (Chapter 16), or an extra sample.

Early stopping. If after i of n samples the leader has c_1 votes and the runner-up c_2, and $c_1 - c_2 > n - i$, no remaining sample can change the outcome. Stopping there saves the most compute exactly on the easy problems where the model is consistent.

Listing 15.7: Majority vote with equivalence keys, tie-breaking and early stopping (condensed from `llmsys/tts.py`).

```python
def majority_vote(answers, key=lambda a: a):
    valid = [(i, a, key(a)) for i, a in enumerate(answers) if a]
    if not valid:
        return None
    counts = Counter(k for _, _, k in valid)
    first = {}
    for i, a, k in valid:
        first.setdefault(k, (i, a))           # representative = first seen
    top = max(counts.values())
    win = min((k for k in counts if counts[k] == top), key=lambda k: first[k][0])
    return first[win][1]

def self_consistency(sample_fn, extract_fn, n, early_stop=True):
    answers = []
    for i in range(n):
        answers.append(extract_fn(sample_fn(i)))
        if early_stop:
            c = Counter(a for a in answers if a).most_common(2)
            lead = c[0][1] if c else 0
            second = c[1][1] if len(c) > 1 else 0
            if c and lead - second > n - (i + 1):
                break                          # outcome can no longer change
    return dict(answer=majority_vote(answers), answers=answers,
                num_samples=len(answers))
```

15.7.3 An end-to-end self-consistency run

Listing 15.8 wires the pieces together on a dataset: render the prompt (optionally with the CoT suffix), sample n completions with shared-prefix decoding, extract and canonicalize the answers, vote, and grade the winner with the verifier from Chapter 14. The voting key is the *normalized* string, so that formatting variants vote together.

Listing 15.8: Self-consistency over a dataset (condensed from `labs/lab15_scaling/run.py`).

```python
def run_self_consistency(model, tok, data, n=8, cot=True, max_new=512,
                         temperature=0.7, top_p=0.9, seed=123, device="cuda"):
    g = torch.Generator(device=device).manual_seed(seed)
    correct, tokens = 0, 0
    for ex in data:
        q = add_cot(ex["problem"]) if cot else ex["problem"]
        ids = tok.encode(render_math_prompt(q))
        r = sample_n_shared_prefix(model, ids, n, max_new, temperature,
                                   top_p=top_p, eos_id=tok.pad_token_id,
                                   generator=g)
        answers = [extract_final_candidate(tok.decode(t)) for t in r["tokens"]]
        winner = majority_vote(answers, key=normalize_text)["answer"]
        correct += bool(winner) and grade_answer(winner, ex["answer"])
        tokens += sum(len(t) for t in r["tokens"])
    return correct / len(data), tokens / len(data)      # accuracy, tokens/problem
```

Key Idea: Compare methods at matched compute

Reporting "self-consistency with $n = 16$ beats greedy by 10 points" without the cost is half a result. Always record generated tokens per problem (and wall-clock on fixed hardware) next to accuracy, and plot accuracy against compute. A method is only better if it is above the other methods' curve at the same budget — for example, 8 CoT samples may beat 16 short samples at half the tokens.

Pitfall: Voting needs short, comparable answers

Self-consistency only works when final answers can be compared exactly: numbers, expressions, multiple-choice letters, short strings. For open-ended outputs (an essay, a program) there is no "most frequent" answer; you need a scorer to pick the best candidate (best-of-n, Section 15.9) or a method that improves a single answer (Chapter 16). For code, a useful middle ground is to cluster programs by their *outputs* on generated test inputs and vote over clusters.

15.8 Executing parallel samples efficiently

The `self_consistency` function above calls `sample_fn` n times in sequence. That is correct and simple, and it is what a first implementation should look like. It is also the most expensive way to execute the algorithm, for two independent reasons.

Redundant prefill. All n samples share the same prompt. Running them separately prefills that prompt n times, producing n identical copies of its KV cache.

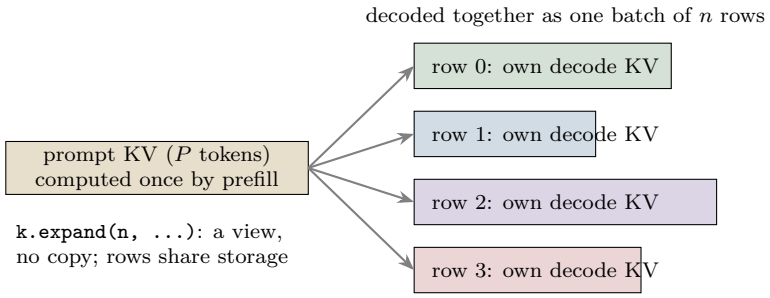


Figure 15.9: Shared-prefix fan-out. The prompt's KV cache is computed once and broadcast to n rows; each row then appends its own decode KV. A paged KV cache (Chapter 12) shares the prompt blocks physically with copy-on-write; our concatenating cache materializes copies on the first decode step.

Redundant weight reads. Decode is memory-bound (Chapter 11): each step streams all weights from HBM to produce one token per sequence. Running samples one after another streams the weights n times per "step of progress"; batching them streams the weights once for all n.

Both problems disappear if we prefill the prompt once and then decode all n continuations as a single batch whose rows share the prompt's KV cache (Fig. 15.9).

Listing 15.9: Prefill once, fan out, decode n samples as a batch (`llmsys/tts.py`).

```python
@torch.no_grad()
def sample_n_shared_prefix(model, prompt_ids, n, max_new_tokens, temperature=0.7,
                           top_k=None, top_p=0.9, eos_id=None, generator=None):
    cache = KVCache(model.n_layers)
    ids = torch.tensor([list(prompt_ids)], device=next(model.parameters()).device)
    logits = model(ids, cache=cache)[:, -1]               # ONE prefill, batch 1
    for l in range(model.n_layers):                       # broadcast the prefix
        cache.k[l] = cache.k[l].expand(n, -1, -1, -1)
        cache.v[l] = cache.v[l].expand(n, -1, -1, -1)
    logits = logits.expand(n, -1)
    outs, lps = [[] for _ in range(n)], [[] for _ in range(n)]
    done = torch.zeros(n, dtype=torch.bool, device=logits.device)
    for _ in range(max_new_tokens):
        nxt = sample_next_token(logits, temperature, top_k, top_p, generator)
        logp = torch.log_softmax(logits.float(), -1).gather(-1, nxt).squeeze(-1)
        for i in range(n):
            if not done[i]:
                outs[i].append(int(nxt[i])); lps[i].append(float(logp[i]))
        if eos_id is not None:
            done |= nxt.squeeze(1) == eos_id
            if bool(done.all()):
                break
        logits = model(nxt, cache=cache)[:, -1]           # n rows, one weight read
    return dict(tokens=outs, logprobs=lps)
```

The function also records the log-probability of each sampled token under the untempered model. That costs one gather per step and gives us, for free, the confidence signal used in Chapter 16 and the "old policy" log-probs needed by GRPO in Chapter 17. Lab 15 verifies correctness the strongest way available: re-scoring each sampled sequence with a separate teacher-forced forward pass must reproduce the recorded log-probs to within floating-point tolerance.

Napkin Math: How much does sharing save?

Prefill. A MATH prompt of $P = 300$ tokens, $n = 16$ samples of $G = 600$ tokens: naive execution processes $n(P+G) = 14{,}400$ token positions, shared execution $P + nG = 9{,}900$ (1.45× fewer). With a 3000-token few-shot prompt the ratio becomes 57,600/12,600 ≈ 4.6×. The longer the shared context relative to the samples, the larger the win.
Decode bandwidth. Qwen3-0.6B on an H100: sequential sampling reads 1.19 GB of weights per token per sample, a lower bound of $16 \times 600 \times 0.36\,\text{ms} \approx 3.4\,\text{s}$. Batched, each of the 600 steps reads the weights once plus every row's KV cache (at an average context of ≈600 tokens, $16 \times 600 \times 114.7\,\text{KB} \approx 1.1\,\text{GB}$), i.e. $\approx (1.19 + 1.1)\,\text{GB}/3.35\,\text{TB/s} \approx 0.68\,\text{ms}$ per step and ≈ 0.41 s in total — about 8× faster. Note that KV traffic is already comparable to the weights at $n = 16$; at larger n or longer chains, KV bandwidth, not weights, becomes the bottleneck, which is why GQA (Chapter 6) and KV quantization (Chapter 13) matter so much for test-time scaling.

Systems Lens: Memory: fan-out without copies

Logically each of the n rows owns a full KV cache of $P+G$ tokens; physically the P prompt tokens need to exist only once. Our concatenating `KVCache` achieves this only until the first decode step, when `torch.cat` materializes n copies of the prefix. Paged KV caches (vLLM's PagedAttention, Chapter 12) keep the prompt's blocks shared indefinitely through reference counts and copy-on-write, so n-way sampling costs $P + nG$ tokens of KV memory instead of $n(P+G)$. Engines expose this as the `n` parameter of a single request, and prefix caches (RadixAttention in SGLang) extend the same sharing across *different* requests.

15.9 Compute-optimal test-time scaling

Self-consistency is one point in a larger design space. Methods differ in how they spend a token budget and in what signal selects the final answer:

- **Parallel vs. sequential.** Parallel methods (voting, best-of-n) generate independent candidates; their latency is one generation if batched. Sequential methods (self-refinement, longer thinking) build each step on the last; latency grows with the budget, but later steps can fix earlier mistakes.
- **Selection signal.** Plurality vote needs no extra model. *Best-of-n* picks the candidate with the highest score from a verifier, an outcome reward model, or a process reward model (PRM); *weighted voting* sums scores per answer class. Search methods (beam search over reasoning steps, tree search) use a PRM to prune partial solutions (Chapter 16).
- **Budget control.** Early stopping when votes agree; adaptive n per question based on estimated difficulty; for thinking models, "budget forcing" — truncating or extending the thinking phase to a target length (Muennighoff et al., 2025).

Snell et al. (2024) study how to allocate a fixed test-time budget and find that the best strategy depends on question difficulty: on easy questions, sequential revisions of a single answer are more efficient; on hard ones, broader parallel search pays off; and adaptively choosing the strategy per question beats a uniform best-of-n at the same compute. The practical lesson for system design is that test-time compute should be an *allocated resource*, scheduled per request,

not a constant multiplier.

Systems Lens: Latency, throughput, and the serving contract

Inference-time scaling trades latency and throughput for accuracy in ways a serving system must make explicit. Parallel sampling with $n = 16$ keeps latency close to a single generation (if the batch fits) but divides the request capacity of a GPU by roughly n in the KV-bound regime. Sequential refinement keeps memory per request low but multiplies latency by the number of rounds. A production API therefore exposes the knob (a "reasoning effort" or thinking-budget parameter), prices it, and the scheduler must account for it when admitting requests (Chapter 12).

15.10 Summary

- Inference-time scaling improves answers by spending more compute per query without changing weights; it trades accuracy for tokens, and tokens are the unit of serving cost.
- Chain-of-thought prompting gives the model a scratchpad: more sequential forward passes and intermediate results stored in the context. It helps base models most and turns queries into decode-heavy workloads.
- Temperature divides logits before the softmax; entropy grows monotonically with T. Sampling is inverse-CDF in disguise. Always compute softmax and cumulative sums in float32.
- Top-p keeps the smallest high-probability set with mass $\geq p$; it adapts to the model's confidence and removes the long tail that high temperature inflates.
- Self-consistency samples n reasoning paths and takes a plurality vote over canonicalized answers; it amplifies the model's most likely answer and benefits from early stopping.
- Executed well, n-way sampling prefills once and decodes as one batch over a shared-prefix KV cache; the savings in prefill work, weight bandwidth and memory are large and computable.

15.11 Exercises

Exercise 15.1 (*entropy is monotone in T*). Prove that $H(T)$ is non-decreasing in T for fixed logits. (Hint: show $\frac{dH}{dT} = \mathrm{Var}_{p(T)}[z]/T^3 \geq 0$.)

Exercise 15.2 (*top-k vs. top-p*). For the distribution in Fig. 15.6, which k gives the same kept set as $p = 0.8$? Construct a second distribution where that k keeps far too many tokens. Implement min-p sampling (keep tokens with $p_i \geq \alpha \max_j p_j$) and compare the three on both.

Exercise 15.3 (*tie-breaking by confidence*). Extend `majority_vote` so ties are broken by the mean per-token log-probability of the samples in each class (recorded by `sample_n_shared_prefix`). Construct a test case where this differs from first-seen tie-breaking.

Exercise 15.4 (*early-stopping savings*). Under the independent-samples model with $n = 16$ and per-sample accuracy p (one wrong answer), compute the expected number of samples drawn with the early-stopping rule of Section 15.7, for $p \in \{0.2, 0.5, 0.9\}$. Explain the shape of the result.

Exercise 15.5 (*CoT token accounting*). On 20 problems from `math_sample.jsonl`, measure generated tokens per problem with and without the CoT suffix. Using the KV size per token of Qwen3-0.6B, compute the peak KV memory per request in each mode.

Exercise 15.6 (*break-even sampling*). Using `llmsys.flops.decode_step_time_lower_bound`, find the batch size n at which batched decoding of Qwen3-0.6B with 1024-token contexts on an H100 stops being dominated by weight traffic (KV bytes equal weight bytes). Repeat for Qwen3-8B. What does this imply for the cost of self-consistency on larger models?

Exercise 15.7 (*sampling correctness*). `sample_n_shared_prefix` records log-probs under the *untempered* distribution even when $T \neq 1$ and top-p is active. Explain why that is the right quantity for confidence scoring but the wrong one for computing the exact probability with which a sequence was sampled. Write the correct expression for the latter.

Exercise 15.8 (*copy-on-write accounting*). For $n = 32$ samples, prompt length 1500 and average generation 900 tokens with block size 16, compute the number of KV blocks needed (a) with no sharing, (b) with shared prompt blocks and copy-on-write of the last partial prompt block.

Exercise 15.9 (*weighted voting*). Implement weighted self-consistency where each sample's vote is weighted by exp(mean log-prob). Under what conditions can weighting hurt compared with unweighted voting?

15.12 Lab 15: Sampling, self-consistency, and shared-prefix decoding

Lab 15: Sampling, self-consistency, and shared-prefix decoding

Code: `code/labs/lab15_scaling`

Goals

Build the sampling stack from first principles, implement self-consistency with the details that matter in practice, and execute n-way sampling efficiently with a shared-prefix KV cache.

Setup

```
cd code
LAB_IMPL=starter python -m pytest -q labs/lab15_scaling
```

Tasks

1. `softmax_with_temperature`: float32, one-hot limit at $T = 0$.
2. `top_p_filter_probs`: exclusive-prefix-sum mask, renormalize; top token always

kept.

3. `sample_from_probs`: inverse-CDF sampling with `torch.searchsorted` (the test checks empirical frequencies over 20k draws).
4. `majority_vote`: equivalence keys, abstentions, first-seen tie-breaking.
5. `self_consistency`: the provable early-stopping rule.
6. `sample_n_shared_prefix`: one prefill, `expand` the cache, batched decode, record untempered log-probs (checked against teacher forcing).
7. `forward_token_counts`: naive vs. shared token positions.

Measurements

With the Qwen3-0.6B base checkpoint, run `run.py` on 20 problems with $n \in \{1, 4, 8, 16\}$, with and without `-cot`. Plot accuracy against total generated tokens, and wall-clock of sequential versus shared-prefix sampling against n. Expect accuracy to rise with CoT and with n (with diminishing returns), and the shared-prefix speedup to grow with n until the batch becomes KV-bandwidth-bound.

Deliverables

Passing tests; the two plots; one paragraph explaining where the measured speedup departs from the napkin estimate of Section 15.8 and why.

Stretch goals

Implement weighted voting and confidence-based tie-breaking; replace the concatenating cache with `StaticKVCache` plus an explicit prefix copy and compare peak memory; add per-row EOS handling that stops computing finished rows (compaction).

Chapter 16

Self-Refinement and Scoring

The two inference-time methods of Chapter 15 both work *in parallel*: one long answer, or many independent answers and a vote. Neither lets the model look at what it wrote and fix it. People solving hard problems do exactly that — draft, check, revise — and this chapter gives the model the same loop. *Self-refinement* asks the model to critique its own draft and then to produce a revised answer that addresses the critique.

Refinement immediately raises a question that voting avoided: *is the revision actually better?* Models can talk themselves out of a correct answer as easily as into one. So before building the loop we build *scorers* — functions that assign a number to a response — starting from a transparent rule-based score and moving to the model's own confidence, measured through token log-probabilities. Log-probabilities are worth the careful treatment they get here: in Chapter 17 they reappear inside the RL training objective, and in Chapter 20 inside the distillation loss. From the systems side, scoring is the cheap half of inference — a single prefill, no decode loop — and refinement is the expensive, strictly sequential half; we will quantify both.

16.1 Scoring and iteratively improving responses

RM 5.1

Figure 16.2 places self-refinement next to the two methods we already have. They differ in the *axis* along which they spend extra compute: chain-of-thought spends it on a longer single answer, self-consistency on more answers, self-refinement on more rounds of revision.

Self-consistency has become a common choice when accuracy matters more than latency, but it has a structural limitation: it needs short final answers that can be compared for equality. A scorer removes that limitation. Given any function $s(\text{question}, \text{response}) \in \mathbb{R}$ we can:

- **rank** candidates and return the best (best-of-n, reranking);
- **gate** a revision: accept it only if it scores higher than the draft;
- **guide search**: expand only the most promising partial solutions (Section 16.11).

Scorers come in four flavours, in increasing cost: heuristic rules, the model's own token probabilities, external verifiers (Chapter 14), and learned reward models or LLM judges.

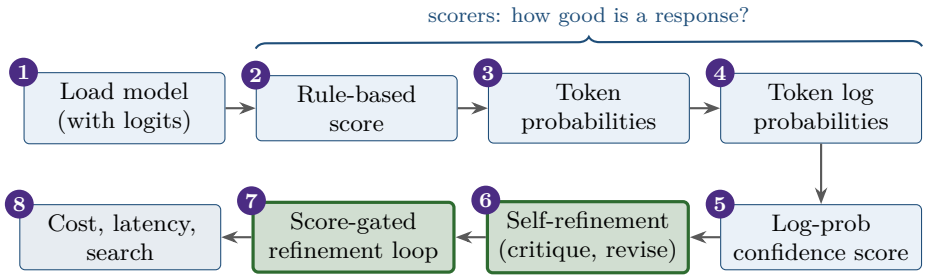


Figure 16.1: Roadmap. Stages 2–5 build scoring functions; stages 6–7 build the refinement loop that uses them to accept or reject revisions; stage 8 is the systems view.

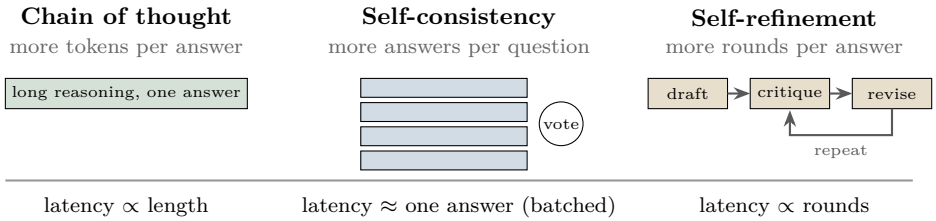


Figure 16.2: Three ways to spend inference compute. Self-refinement is the only one that conditions later work on an explicit evaluation of earlier work; it is also inherently sequential.

16.2 Loading a model whose logits we can see

RM 5.2

We load Qwen3-0.6B exactly as in Section 14.3. One requirement is new: the confidence scorers need the model's full next-token distributions, not just sampled text. With local weights that is free — the forward pass returns logits for every position. Hosted APIs typically expose at most the log-probabilities of the top few tokens per position, and only for generated text, which is enough for some scorers in this chapter but not all.

Systems Lens: Logit access is a deployment decision

Returning full logits from an inference server is expensive: $V \times 4\,\text{B} = 0.6\,\text{MB}$ per token for Qwen3's vocabulary, i.e. 600 MB for a 1000-token response — far more than the text. Serving engines therefore return *selected* log-probs (of the sampled token and the top-k alternatives) computed on the GPU, and offer a separate "prompt log-probs" option that scores a given text in a prefill-only pass. Designing scorers that need only the log-prob of the realized tokens, as ours do, keeps them deployable.

16.3 A rule-based score

RM 5.3

The simplest scorer encodes preferences as rules. For math responses we prefer, in this order, a response with a final boxed answer, then one that at least contains a number; and among those, shorter ones. Listing 16.1 implements this with a format bonus plus a smooth brevity term $\beta/(\beta + \ell)$ that lies in $(0, 1]$, so format always dominates length.

Listing 16.1: A rule-based scorer for format and brevity (`llmsys/tts.py`).

```
def heuristic_score(answer, boxed_bonus=2.0, number_bonus=1.0,
                    brevity_scale=500.0):
    s = 0.0
    if last_boxed(answer) is not None:          # complete \boxed{...}
```

Response (abbreviated)	format	brevity	score
"…so $ab = 60$. `\boxed{60}`" (180 chars)	2.0	0.735	2.735
"Let a, b be the sides … [long] … `\boxed{60}`" (900 chars)	2.0	0.357	2.357
"The area is 60." (16 chars)	1.0	0.969	1.969
"I am not sure how to proceed." (29 chars)	0.0	0.945	0.945

Table 16.1: Heuristic scores for four responses with $\beta = 500$. The score measures format and brevity, not correctness: a short boxed *wrong* answer would beat a long correct one.

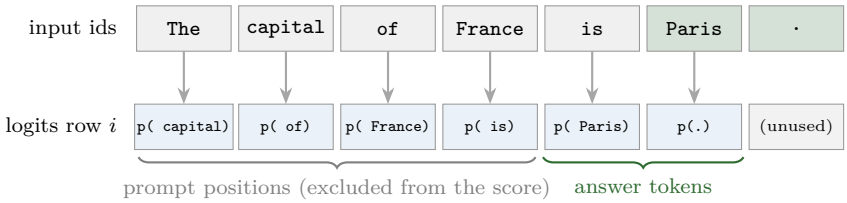


Figure 16.3: Teacher forcing: row i of the logits predicts token $i{+}1$. One forward pass over prompt plus answer yields the probability of every answer token; the prompt's own predictions are discarded and the last row predicts beyond the sequence.

```
        s += boxed_bonus
    elif NUMBER_RE.search(answer or ""):        # at least some number
        s += number_bonus
    return s + brevity_scale / (brevity_scale + len(answer or ""))
```

Pitfall: Goodhart's law in one function

Any score you optimize against will be exploited. Gate refinements on brevity and the model learns to delete its reasoning; reward boxes and it boxes a guess. Rule-based scores are useful as cheap filters, tie-breakers and format checks (Chapter 18 adds exactly such a format reward to RL), but they should never be the only signal of quality.

16.4 Token probabilities

RM 5.4

A more informative signal is the model's own *confidence*: how probable does the model find the tokens of a given answer? Recall from Section 15.4.1 that at every position the model produces logits over the whole vocabulary, which the softmax turns into a next-token distribution. To generate, we pick or sample from that distribution. To *score*, we feed in a complete candidate response and simply look up the probability the model assigned to each token that is actually there — *teacher forcing*. No sampling happens; we are only inspecting the distribution.

Consider two candidate completions of the prompt "The capital of France is": " Paris." and " Pasta.". A well-trained model assigns most of its next-token probability after "is" to " Paris" and almost none to " Pasta". By the chain rule of probability, the probability of a whole answer $y = (y_1, \ldots, y_m)$ given a prompt x is the product of per-token conditionals,

$$\Pr_\theta(y \mid x) \;=\; \prod_{t=1}^{m} \Pr_\theta(y_t \mid x, y_{<t}), \tag{16.1}$$

and each factor is read off one row of the logits tensor (Fig. 16.3).

The off-by-one in Fig. 16.3 is the single most common bug in log-probability code, in scoring and in RL alike. The rule: logits at position i are the distribution for the token at position $i+1$, so we drop the last logits row and the first token, and the answer's first token is predicted by the *last prompt position*.

Listing 16.2: Per-token probabilities of an answer by teacher forcing.

```
@torch.no_grad()
def answer_token_probs(model, prompt_ids, answer_ids):
    ids = torch.tensor([list(prompt_ids) + list(answer_ids)])
    logits = model(ids)[0, :-1].float()            # row i predicts token i+1
    probs = torch.softmax(logits, dim=-1)
    targets = ids[0, 1:]                           # the tokens actually present
    tok_p = probs.gather(-1, targets[:, None]).squeeze(-1)
    return tok_p[len(prompt_ids) - 1:]             # answer tokens only
```

Key Idea: Confidence is not correctness

A high probability means the answer fits the patterns the model learned, not that it is true. Models can be confidently wrong (a common misconception repeated in training data) and hesitantly right (an unusual but correct phrasing). Probability-based scores are useful because, on average and for well-trained models, confidence correlates with correctness; they are a weak verifier, not a verifier.

Systems Lens: Scoring is prefill, not generation

Generating an m-token answer takes m sequential decode steps, each memory-bound. Scoring the same answer takes *one* forward pass over $|x| + m$ tokens: all positions are processed in parallel with large matmuls, which is compute-bound and runs near peak FLOP/s. On an H100, a 1000-token scoring pass for Qwen3-0.6B is about $2 \times 0.6\text{B} \times 1000 = 1.2$ TFLOP, roughly a millisecond of tensor-core time at a realistic fraction of peak, whereas generating those 1000 tokens takes at least 1000 memory-bound steps ($\gtrsim 0.4\,\text{s}$). Scoring is two to three orders of magnitude cheaper than generating, which is why reward models, rerankers and PRMs are deployed as prefill-only services that batch many candidates together.

16.5 From probabilities to log-probabilities

RM 5.5

Multiplying hundreds of probabilities in Eq. (16.1) quickly produces numbers too small to represent. We work with logarithms instead: the log of a product is a sum,

$$\log \Pr_\theta(y \mid x) \;=\; \sum_{t=1}^{m} \log \Pr_\theta(y_t \mid x, y_{<t}), \tag{16.2}$$

and sums of moderately negative numbers are perfectly well behaved. Because log is strictly increasing, it preserves every ordering: a more probable sequence always has a larger (less negative) log-probability. A log-probability of 0 means certainty; more negative means less likely (Fig. 16.4).

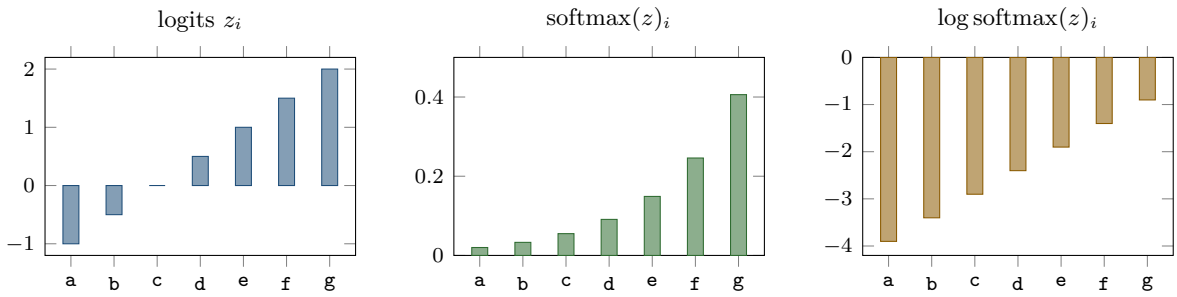


Figure 16.4: Seven logits, their probabilities and their log-probabilities (computed for this figure). All three preserve the same ordering; log-softmax is simply the logits shifted by a constant, $\log \mathrm{softmax}(z)_i = z_i - \log \sum_j e^{z_j}$.

Napkin Math: When does the product underflow?

An answer of 1000 tokens with an average per-token probability of 0.9 has joint probability $0.9^{1000} = e^{-105.4} \approx 1.7 \times 10^{-46}$ — below float32's smallest normal number (1.2×10^{-38}), so it underflows to a subnormal and then to zero. In float16 (smallest normal 6.1×10^{-5}) the product is already subnormal after about 90 such tokens. The log-probability, -105.4, is an ordinary number in any format. Rule: never multiply probabilities; sum log-probabilities.

Computing `torch.log(torch.softmax(z))` is numerically poor: a probability that underflows to zero becomes $-\infty$ after the log. `torch.log_softmax` uses the identity in the caption of Fig. 16.4 together with the log-sum-exp trick,

$$\log \sum_j e^{z_j} \;=\; M + \log \sum_j e^{z_j - M}, \qquad M = \max_j z_j, \tag{16.3}$$

so every exponent is ≤ 0 (no overflow) and the largest term is exactly 1 (no catastrophic underflow of the sum). It is also faster: one fused kernel instead of two passes.

Listing 16.3: Per-token log-probabilities in one pass (`llmsys/tts.py`).

```python
@torch.no_grad()
def token_logprobs(model, prompt_ids, answer_ids):
    ids = torch.tensor([list(prompt_ids) + list(answer_ids)])
    logits = model(ids)[0, :-1].float()              # upcast BEFORE log_softmax
    lp = torch.log_softmax(logits, -1).gather(-1, ids[0, 1:, None]).squeeze(-1)
    return lp[len(prompt_ids) - 1:]
```

Systems Lens: Never materialize the full log-softmax

`log_softmax` over the full $T \times V$ logits creates a float32 tensor of $T \times 151{,}936 \times 4$ B — 1.2 GB for a 2000-token sequence — only to read one element per row. In training (Chapter 17) this tensor is also kept for the backward pass, and for a batch it easily exceeds the memory of the model itself. Efficient implementations compute $z_{y_t} - \mathrm{logsumexp}(z)$ directly, chunking over positions or vocabulary, or fuse the final projection with the cross-entropy so full logits are never materialized ("fused linear cross-entropy" kernels).

16.6 Scoring confidence with log-probabilities

RM 5.6

Two adjustments turn the joint log-probability into a usable confidence score.

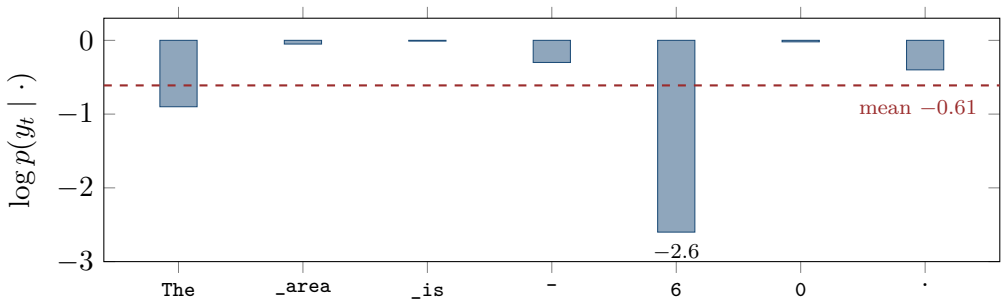


Figure 16.5: Illustrative per-token log-probabilities for the answer "The area is 60." The sum is −4.28 (joint probability 0.014), the mean −0.61 (perplexity 1.84). The uncertain token is the first digit of the result.

Score only the answer. The prompt is shared by all candidates for a question, so its tokens add the same constant to each joint log-probability — but only if they are scored in the same context. Excluding them, as in Listing 16.3, makes scores comparable and cheaper to reason about.

Normalize by length. Every token contributes a non-positive term, so the sum systematically prefers *shorter* answers. The *average* log-probability per answer token,

$$\bar{\ell}(y \mid x) \;=\; \frac{1}{m}\sum_{t=1}^{m} \log \Pr_\theta(y_t \mid x, y_{<t}), \qquad \mathrm{PPL}(y \mid x) = e^{-\bar{\ell}}, \tag{16.4}$$

is a length-normalized confidence; its exponentiated negative is the familiar *perplexity* (Chapter 7) of the answer under the model. Figure 16.5 shows why the average is informative: most tokens of a fluent answer are near-certain, and the score is driven by a few "decision" tokens — often exactly the digits of the final result.

Listing 16.4: Confidence summaries from token log-probabilities (Lab 16, Task 3).

```python
def confidence_scores(token_lps):
    mean = float(token_lps.mean())
    return dict(sum=float(token_lps.sum()),    # joint log-prob (favours short)
                mean=mean,                     # length-normalized confidence
                ppl=math.exp(-mean),           # perplexity of the answer
                min=float(token_lps.min()))    # weakest link

def avg_logprob_score(model, prompt_ids, answer_ids):
    lp = token_logprobs(model, prompt_ids, answer_ids)
    return float(lp.mean()) if lp.numel() else -math.inf
```

Which summary to use is an empirical question. The mean is the standard choice. The minimum token log-probability flags a single shaky step in an otherwise fluent derivation. Restricting the average to the final-answer tokens (the content of the box) measures confidence in the result rather than in the prose. Whatever you choose, check it on a validation set: plot accuracy against score deciles and see whether higher scores really mean more correct answers.

Pitfall: Scores from different models or prompts are not comparable

A log-probability is relative to a model *and* a context. Scoring a revision under the refinement prompt (which contains the critique) and the draft under the original prompt compares two different conditional distributions. Always score every candidate under the *same* model and the *same* scoring prompt — typically the original question, as `run.py` for Lab 16 does.

16.6.1 A worked comparison

Suppose a base model produced two candidate solutions to the rectangle problem: A ends in `\boxed{60}` after a five-line derivation, and B ends in `\boxed{72.25}` after wrongly assuming a square. Scoring both under the original question prompt gives each a vector of per-token log-probabilities. The *sum* is dominated by length: the longer, correct A may well have the lower joint log-probability. The *mean* asks a fairer question — which answer did the model find more natural per token? — and on problems the model "knows", correct derivations tend to have fewer low-probability tokens than flawed ones, whose errors often show up as a few surprising tokens (the unjustified step, the wrong digit). The *minimum* isolates exactly those tokens. None of the three is guaranteed to rank A above B; Exercise 16.4 asks you to measure how often each does.

16.7 Learned scorers: reward models

Between hand-written rules and the model's raw confidence sits a third option that the next part of the book relies on: a *learned* scorer. A **reward model** (RM) is usually a copy of a language model whose unembedding layer is replaced by a scalar head; it reads (question, response) and outputs a real number. It is trained on human (or AI) preference pairs (y^+, y^-) with the Bradley–Terry likelihood from Section 14.13,

$$\mathcal{L}_{\text{RM}} \;=\; -\log \sigma\big(r_\phi(x, y^+) - r_\phi(x, y^-)\big), \tag{16.5}$$

so that differences of reward are log-odds of preference. Variants include *outcome* reward models (one score per full response), *process* reward models (one score per reasoning step, Section 14.12) and *generative* verifiers that write a critique and then a verdict — an LLM judge trained for the purpose.

Reward models are the scorer of choice for open-ended tasks and the centrepiece of classic RLHF (Chapter 17). Their weakness is the one every learned function shares: off the training distribution they are wrong in ways an optimizer will find. Best-of-n with a large n, or RL against an RM for many steps, eventually produces responses that score highly and are bad ("reward hacking"). Verifiable rewards avoid this for math and code, which is a large part of why reasoning-model training moved to them.

Systems Lens: Serving a reward model

An RM forward pass is a prefill with a scalar output: no KV cache needs to be kept, no decode loop runs, and batches can be packed tightly by total tokens. In RL systems (Chapter 19) the RM is typically a separate service colocated with or disaggregated from the policy, sized by the number of rollouts per step times their length. For a 7B RM scoring

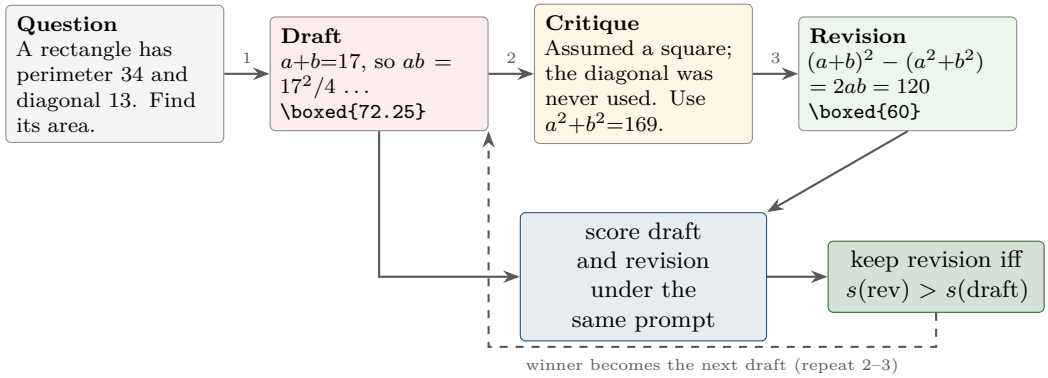


Figure 16.6: One round of self-refinement with score gating (example text written for this book). The draft, critique and revision are three generation calls; the scorer decides which answer survives into the next round.

1024 rollouts of 1000 tokens, that is $2 \times 7 \times 10^9 \times 1.024 \times 10^6 \approx 1.4 \times 10^{16}$ FLOPs, about 30 s of one H100 at a realistic 50% of peak — small next to generating those rollouts, but not free.

16.8 Self-refinement through iterative feedback

RM 5.7

Self-refinement (Madaan et al., 2023, "Self-Refine") uses one model in three roles, each a plain generation call with a different prompt (Fig. 16.6):

1. **Draft.** Answer the question as usual.
2. **Critique.** Given the question and the draft, list errors and a short fix plan.
3. **Revise.** Given the question, the draft and the critique, write an improved answer that ends with a boxed result.

Steps 2–3 can be repeated, each revision becoming the next draft. Nothing here is new machinery — it is sequential composition of the generation function from Chapter 14 with prompt templates.

Listing 16.5: Critique and refine templates (`llmsys/tts.py`).

```
CRITIQUE_TEMPLATE = (
    "You are reviewing a draft solution.\n\nProblem:\n{question}\n\n"
    "Draft:\n{draft}\n\nList any mistakes or gaps and a short plan to fix them "
    "(at most ~100 words).\n\nReview:")
REFINE_TEMPLATE = (
    "Improve the draft solution using the review. End with the final result as "
    "\\boxed{{ANSWER}}.\n\nProblem:\n{question}\n\nDraft:\n{draft}\n\n"
    "Review:\n{critique}\n\nImproved solution:")
```

Walking through one round by hand is the best way to build intuition. Sample a draft at moderate temperature, print it, generate the critique with a small token budget (a critique that rambles is a critique that will be ignored), print it, then generate the revision. You will see three patterns: the critique identifies a real error and the revision fixes it; the critique hallucinates an error in a correct draft and the revision breaks it; or the critique is generic ("check your

arithmetic") and the revision is a paraphrase. A scorer exists to filter the second and third cases.

Key Idea: Feedback quality is the whole game

Refinement can only be as good as its feedback. With *intrinsic* feedback — the same model judging itself, with no external signal — gains on reasoning tasks are small and sometimes negative: Huang et al. (2024) found that LLMs often fail to self-correct their reasoning without external feedback, and can even degrade correct answers. Refinement shines when the critique has access to information the draft lacked: an executed test case, a verifier's verdict, a compiler error, a retrieved document. In system design terms, self-refinement is a loop; close it with the best signal you can afford.

16.9 Coding the refinement loop

RM 5.8

Listing 16.6 packages the procedure into a function with an iteration count and an optional scorer. Without a scorer every revision is accepted. With one, a revision replaces the current answer only if it scores strictly higher; otherwise the loop critiques the *same* answer again in the next round (with sampling, the next critique will differ).

Listing 16.6: Score-gated self-refinement (condensed from `llmsys/tts.py`).

```
def self_refine(question, generate_fn, iterations=2, score_fn=None,
                answer_prompt_fn=lambda q: q, max_answer_tokens=1024,
                max_critique_tokens=256):
    cur = generate_fn(answer_prompt_fn(question), max_answer_tokens)
    cur_score = score_fn(question, cur) if score_fn else None
    steps = []
    for it in range(iterations):
        crit = generate_fn(CRITIQUE_TEMPLATE.format(question=question, draft=cur),
                           max_critique_tokens)
        rev = generate_fn(REFINE_TEMPLATE.format(question=question, draft=cur,
                                                 critique=crit), max_answer_tokens)
        rev_score = score_fn(question, rev) if score_fn else None
        ok = score_fn is None or rev_score > cur_score
        steps.append(RefineStep(it, cur, crit, rev, cur_score, rev_score, ok))
        if ok:
            cur, cur_score = rev, rev_score
    return RefineResult(final=cur, final_score=cur_score, steps=steps)
```

The scorer for math combines naturally with the pieces of this chapter: score each candidate by its average token log-probability *under the original question prompt*:

Listing 16.7: Plugging the log-prob scorer into the loop.

```
def logprob_scorer(question, answer):
    prompt_ids = tok.encode(render_math_prompt(question))
    return avg_logprob_score(model, prompt_ids, tok.encode(answer))

result = self_refine(problem, generate_fn, iterations=2, score_fn=logprob_scorer,
                     answer_prompt_fn=render_math_prompt)
for s in result.steps:
    print(s.iteration, s.score_draft, s.score_revised, s.accepted)
```

Policy	needs scorer	protects correct drafts	extra cost
always accept	no	no	none
strict improvement	yes	partly (if scorer agrees)	$2k$ scoring prefills
margin δ	yes	more	same
best of all rounds	yes	as strict	same
verifier override	exact check	yes, when check applies	check per candidate

Table 16.2: Acceptance policies for self-refinement.

16.9.1 Acceptance and stopping policies

Strict improvement is only one of several reasonable acceptance rules, and each encodes a different belief about the scorer:

- **Always accept** (no scorer): trusts the critique; cheapest; vulnerable to harmful revisions.
- **Strict improvement** ($s_{\text{rev}} > s_{\text{cur}}$): trusts the scorer's ranking; ties keep the incumbent, which makes the loop conservative.
- **Margin** ($s_{\text{rev}} > s_{\text{cur}} + \delta$): accounts for scorer noise; with sampled critiques, a small δ avoids flip-flopping between near-equivalent answers.
- **Keep the best of all rounds**: score every candidate and return the argmax at the end, not the last accepted one. With a fixed scorer this equals strict acceptance, but it also lets you combine rounds with best-of-n sampling.
- **Verifier override**: if any candidate passes an exact check (a substitution test, a unit test), return it immediately — an external signal always beats a confidence score.

Stopping rules matter as much for cost. Stop when a revision is rejected twice in a row, when the revised final answer equals the current one (the critique changed only the prose), or when a token budget is exhausted. Each of these is a one-line change to Listing 16.6, and each can remove a large fraction of the sequential calls on problems where the first draft was already right.

Pitfall: What score gating can and cannot fix

Gating prevents a revision from replacing a draft that the scorer prefers, so it limits the damage of bad critiques. It cannot fix a scorer that prefers wrong answers. In particular, average log-probability tends to prefer answers the model finds *familiar*; if the base model's confident answer is wrong, gating will dutifully keep it. Whether refinement with a given scorer helps a given model is an empirical question — measure it on a validation set, including the fraction of accepted revisions that turned a correct draft into a wrong one.

Refinement composes with the earlier methods. CoT prompting can be used for the draft and the revision; with a base model it should be. Self-consistency can be applied *after* refinement (refine n samples, then vote), or refinement can be applied to the vote's winner. Each composition multiplies cost, which brings us to the systems view.

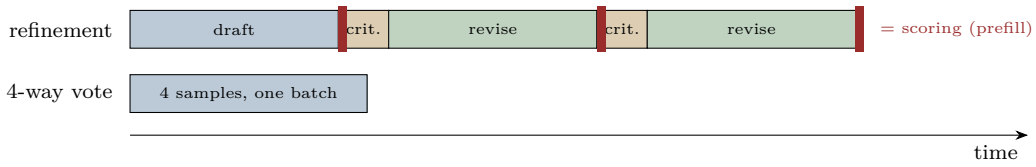


Figure 16.7: Schematic timelines at a similar decode-token budget. Refinement's calls cannot overlap because each consumes the previous output; a batched vote finishes in roughly the time of one answer.

16.10 The cost of thinking sequentially

Let the question have q tokens, every answer a tokens and every critique c tokens, and assume template text is negligible. Without any KV reuse between calls, one round of refinement prefills the critique prompt ($q + a$ tokens) and the refine prompt ($q + a + c$), and decodes $c + a$ new tokens. For k rounds after the draft:

$$\text{prefill tokens} = q + k\,(2q + 2a + c), \tag{16.6}$$

$$\text{decode tokens} = a + k\,(a + c), \tag{16.7}$$

$$\text{sequential generation calls} = 1 + 2k. \tag{16.8}$$

Scoring adds $2k$ prefill-only passes of $q + a$ tokens if the draft's score is cached.

Napkin Math: Refinement vs. voting at a matched token budget

Systems Lens: Make the loop cache-friendly

Every refinement call re-sends text the model has already processed: the question, then the draft. A serving engine with prefix caching (Chapter 12) can skip recomputing a prompt prefix it has seen — but only an *exact token prefix*. Our templates begin with different instructions ("You are reviewing…" versus "Improve the draft…"), so the question and draft never form a shared prefix and every call pays full prefill. Reordering templates so that the stable content comes first (`Problem: ... Draft: ...` followed by the role-specific instruction) turns most of each refinement prefill into a cache hit. Better still, run refinement as a multi-turn conversation in which each step appends to the same context: then the entire history is a prefix of the next call and only new tokens are prefilled. This is a general rule for agentic and multi-call LLM programs: put the invariant parts of a prompt first.

16.11 Beyond a single chain: search

Refinement improves one answer along one path. *Search* methods keep several partial solutions and use a scorer to decide which to extend:

- **Best-of-n reranking**: sample n full answers (in parallel, Section 15.8) and return the highest-scoring one. With an exact verifier it is unbeatable; with a learned reward model it is vulnerable to reward hacking as n grows (the argmax finds the scorer's errors).
- **Step-level beam search**: split solutions into steps (e.g. lines), sample several continuations per step, keep the top-b partial solutions according to a process reward model (PRM) that scores prefixes.
- **Tree search**: Tree-of-Thoughts (Yao et al., 2023) and MCTS-style methods explore a tree of partial reasoning states with backtracking, trading many more model calls for the ability to recover from early mistakes.

All of these are compositions of two primitives this chapter and the last made efficient: *parallel sampling from a shared prefix* (each tree node is a prefix whose KV cache all children share — exactly what RadixAttention-style prefix trees in serving engines provide) and *prefill-only scoring* (PRMs score prefixes, batchable across the frontier). As reasoning models trained with RL (Chapter 17) internalize backtracking into a single long chain of thought, explicit search has receded for math, but the systems primitives remain central to agentic workloads.

16.12 Summary

- Scorers rank, gate and guide; they range from rules to model confidence to verifiers and learned reward models. Any scorer you optimize against will be gamed.
- Token probabilities come from teacher forcing: one forward pass over prompt plus answer, where logits row i predicts token $i{+}1$. Confidence is not correctness.
- Sum log-probabilities rather than multiplying probabilities; use `log_softmax` in float32 (log-sum-exp) and avoid materializing full log-softmax tensors.
- The average answer-token log-probability (or perplexity) is a length-normalized confidence score; score all candidates under the same model and prompt.
- Self-refinement chains draft, critique and revision calls; score gating accepts a revision only if it improves. Gains depend on feedback quality; intrinsic self-critique is weak on reasoning.
- Refinement is sequential: at a similar token budget it costs several times the latency of a batched vote. Order prompt content for prefix-cache hits; scoring is cheap prefill work.

16.13 Exercises

Exercise 16.1 (*off-by-one hunt*). Write a deliberately wrong version of `token_logprobs` that slices logits `[1:]` instead of `[:-1]`. Which test in Lab 16 catches it, and why would summing the wrong values still produce plausible-looking negative numbers?

Exercise 16.2 (*log-sum-exp*). Prove Eq. (16.3). Then show that `log_softmax` computed with it never overflows for finite inputs and that the largest output entry lies in $[-\log V, 0]$.

Exercise 16.3 (*length bias*). Show that for any answer and any extension of it by tokens with probability < 1, the joint log-probability strictly decreases. Construct an example in which the average log-probability increases when tokens are appended. Which scorer would you use to compare a 50-token and a 500-token solution?

Exercise 16.4 (*calibration*). On 100 sampled answers to problems from `math_sample.jsonl` (or MATH-500), compute the average log-probability and the verifier's verdict. Bin by score decile and plot accuracy per bin. Is the model's confidence informative? Repeat with the minimum token log-probability and with the average over boxed-answer tokens only.

Exercise 16.5 (*chunked log-probs*). Implement `token_logprobs_chunked` that computes the same result while never holding more than C rows of float32 log-softmax at once. Measure peak memory for $T = 4096$ with $C \in \{T, 512, 64\}$ on CPU using `tracemalloc` or on GPU with `torch.cuda.max_memory_allocated`.

Exercise 16.6 (*refinement harm rate*). Run self-refinement with $k = 2$ on 50 problems with each of: no scorer, heuristic scorer, log-prob scorer. For each, report (a) accuracy, (b) the fraction of initially correct drafts that end wrong, (c) the fraction of initially wrong drafts that end correct. Which scorer has the best ratio?

Exercise 16.7 (*external feedback*). Replace the critique call with a verifier-based critique: if the draft's boxed answer fails a self-check (e.g. substituting the answer back into an equation the model is asked to write), insert that failure into the review. Compare against intrinsic critique.

Exercise 16.8 (*prefix-cache-friendly templates*). Rewrite the critique and refine templates so that the longest possible token prefix is shared across the draft, critique and refine calls. Using the cost model of Eq. (16.6), compute the prefill tokens saved for $q = 150$, $a = 500$, $c = 120$, $k = 2$, assuming perfect prefix caching.

Exercise 16.9 (*best-of-n vs. refinement*). At a matched budget of ≈2000 decode tokens per problem, compare best-of-4 with log-prob reranking against two rounds of refinement. Report accuracy and wall-clock latency.

16.14 Lab 16: Scoring responses and score-gated refinement

Lab 16: Scoring responses and score-gated refinement

Code: `code/labs/lab16_refine`

Goals

Implement the scorers of this chapter, verify the teacher-forcing alignment against incremental decoding, build the refinement loop with score gating, and quantify its cost.

Setup

```
cd code
LAB_IMPL=starter python -m pytest -q labs/lab16_refine
```

Tasks

1. `heuristic_score`: format bonus plus brevity term (Listing 16.1).
2. `answer_token_logprobs`: one forward pass, float32 `log_softmax`, correct shift. The test compares every value against a token-by-token incremental computation.
3. `confidence_scores`: sum, mean, perplexity, minimum.
4. `self_refine`: draft, then `iterations` rounds of critique and revision with strict score gating; return history and the number of generation calls. The test drives it with a scripted fake model so acceptance and rejection are both exercised.
5. `refine_cost_tokens`: the prefill/decode/sequential-call cost model of Section 16.10.

Measurements

With the Qwen3-0.6B base checkpoint, run `run.py` for 20 problems with `-scorer none|heuristic|logprob` and `-iterations 1|2`. Record draft accuracy, refined accuracy, acceptance rate, generated tokens and wall-clock. Expect small and inconsistent accuracy changes with intrinsic critique, a large increase in tokens and latency per extra round, and the log-prob scorer rejecting a sizeable fraction of revisions.

Deliverables

Passing tests; a table of the runs above; a short analysis of three accepted revisions that were harmful or helpful, with the critiques that caused them.

Stretch goals

Implement prefix-cache-friendly templates and measure prefill savings with a prefix-caching engine (Chapter 12); implement best-of-n with log-prob reranking on top of `sample_n_shared_prefix` and compare at matched token budgets; add a calibration plot of score versus correctness.

Chapter 17

Reinforcement Learning with Verifiable Rewards: GRPO

Chapters 15 and 16 improved reasoning by spending more compute *per query*: sample many answers and vote, or let the model critique and revise itself. Every one of those tricks leaves the weights untouched, so the bill is paid again on every request. This chapter takes the opposite trade. We spend compute *once*, during training, so that the model itself becomes a better reasoner and every later query is cheaper. The method is reinforcement learning (RL) with *verifiable rewards* (RLVR), and the optimiser is Group Relative Policy Optimization (GRPO), the algorithm behind DeepSeek-R1 and many of the open reasoning models that followed it.

For a systems person, RL post-training is the most interesting workload in the LLM lifecycle because it fuses the two workloads we have studied separately. Each step is an *inference* job (generate thousands of tokens per prompt, autoregressively, memory-bound) followed by a *training* job (a differentiable forward and backward pass over the same tokens, compute-bound). Get the balance between the two wrong and either your GPUs idle while the sampler crawls, or your sampler idles while the optimiser waits. In this chapter we build GRPO from first principles, keeping the code small enough to read in one sitting, and we mark where each design decision has a systems cost. Chapter 18 then makes the algorithm robust, and Chapter 19 scales it into a distributed system.

17.1 Reinforcement learning for LLMs

RM 6.1

There are two independent dials for buying reasoning accuracy with compute (Fig. 17.2). *Inference-time scaling* spends more FLOPs on each answer: longer chains of thought, more samples, more refinement rounds. *Training-time scaling* spends FLOPs up front to change the weights. The two compose: a model improved by RL can still be wrapped in self-consistency voting at inference time, and in practice the strongest systems do both.

RL in the LLM world is almost always a *post-training* stage. A pretrained model has absorbed knowledge from next-token prediction; RL changes *how* it uses that knowledge, by optimising a property of the whole output (is the final answer right? do humans prefer it?) rather than the likelihood of each individual token. Two RL stages are common (Fig. 17.3): *reasoning training* with verifiable rewards, which teaches the model to reach correct answers on math and code, and *preference tuning*, which nudges style and helpfulness towards what people like. The order

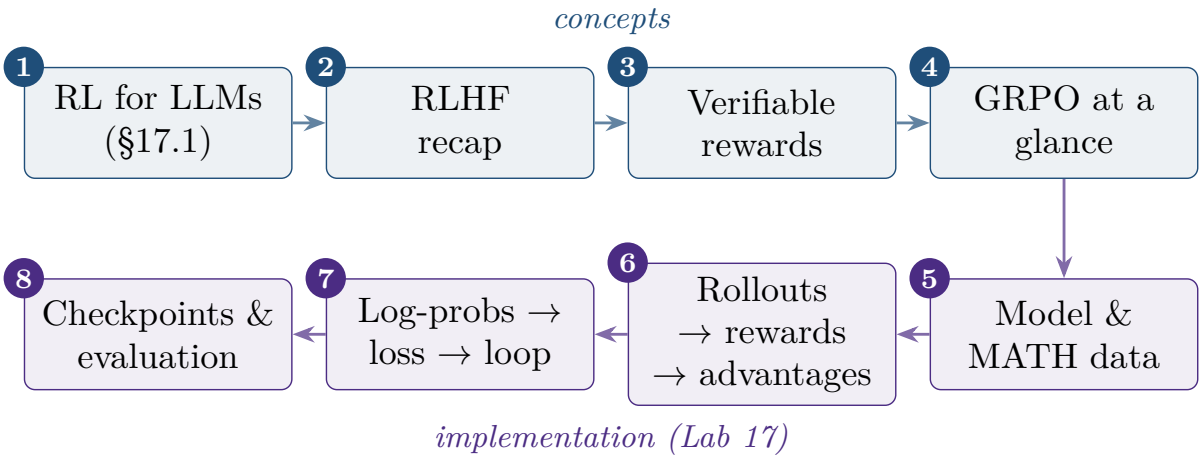


Figure 17.1: Roadmap of this chapter. The top row builds the conceptual picture (why RL, how RLHF works, what changes when rewards are verifiable, what GRPO computes). The bottom row implements GRPO stage by stage and trains a model with it.

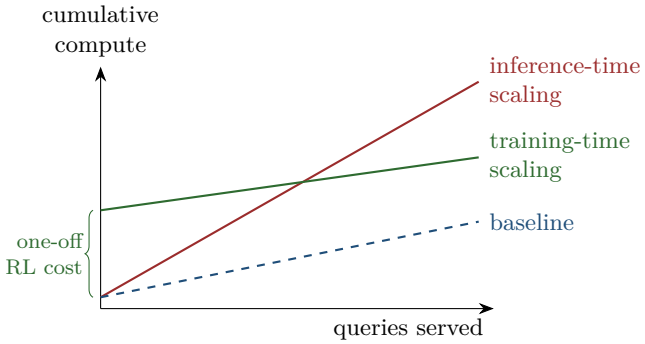


Figure 17.2: Two ways to buy accuracy. Inference-time scaling raises the marginal cost of every query; training-time scaling pays a fixed cost once and keeps the per-query cost close to the baseline. Which is cheaper depends on how many queries the model will serve.

varies between labs and the two are sometimes interleaved. DeepSeek-R1-Zero famously skipped both instruction tuning and preference tuning and ran reasoning RL directly on the base model. The result is less polished than a full pipeline, but it shows clean reasoning behaviour and, for us, has a big methodological advantage: any improvement can be attributed to RL alone.

The RL vocabulary, translated. Classic RL talks about agents, environments, states and actions. For a language model the mapping is straightforward (Table 17.1). The unusual features of the LLM setting are that the action space is huge ($|\mathcal{V}| \approx 150{,}000$ tokens per step), the state is the entire prefix, transitions are deterministic (appending a token), and the reward is *sparse*: a single number at the end of a trajectory that may be thousands of tokens long.

The policy-gradient theorem in one line. We want to maximise the expected reward $J(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\,\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[R(x,y)]$. The reward is a non-differentiable function of discrete samples, so we cannot backpropagate through it. The *log-derivative trick* $\nabla_\theta \pi_\theta(y \mid x) = \pi_\theta(y \mid x)\nabla_\theta \log \pi_\theta(y \mid x)$ moves the gradient onto the log-probability, which *is* differentiable:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,\,y\sim\pi_\theta}\big[\,R(x,y)\,\nabla_\theta \log \pi_\theta(y \mid x)\big] = \mathbb{E}\Big[R(x,y)\sum_{t=1}^{|y|}\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})\Big]. \tag{17.1}$$

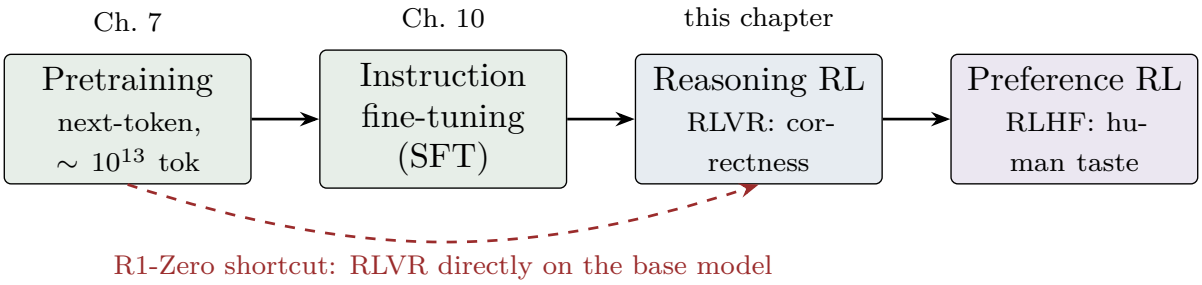


Figure 17.3: Where RL sits in an LLM training pipeline. The order of the two RL stages is not fixed; some pipelines interleave them. The dashed path applies reasoning RL directly to a pretrained base model, which is the setting of this chapter.

RL term	Meaning for an LLM
policy π_θ	the language model with weights θ
state s_t	prompt x plus the tokens generated so far, $y_{<t}$
action a_t	the next token $y_t \in \mathcal{V}$
trajectory / rollout / completion	one full sampled response $y = (y_1, \dots, y_{\lvert y\rvert})$
reward $R(x, y)$	a scalar for the whole response (here: 1 if the final answer is right)
episode	one prompt $\to$ one completion; ends at EOS or the token budget
on-policy	the data were sampled from the current weights

Table 17.1: RL terminology mapped onto autoregressive generation. A "rollout" is the generation process and a "completion" is its output; the words are used interchangeably.

The second equality is the autoregressive factorisation $\log \pi_\theta(y \mid x) = \sum_t \log \pi_\theta(y_t \mid x, y_{<t})$. A Monte Carlo estimate of Eq. (17.1) needs only samples and their log-probabilities: sample, score, and push up the log-probability of each sample in proportion to its reward. This is the REINFORCE estimator (Williams, 1992). It is unbiased but very noisy. The standard fix is to subtract a *baseline* $b(x)$ that does not depend on y:

$$\mathbb{E}_{y\sim\pi_\theta}\big[b(x)\nabla_\theta \log \pi_\theta(y \mid x)\big] = b(x)\sum_y \nabla_\theta \pi_\theta(y \mid x) = b(x)\nabla_\theta \underbrace{\textstyle\sum_y \pi_\theta(y \mid x)}_{=1} = 0, \tag{17.2}$$

so replacing R by the *advantage* $A = R - b(x)$ leaves the gradient unbiased and can reduce its variance dramatically. Every algorithm in this chapter and the next is a choice of (i) how to estimate the baseline, (ii) how to weight and clip each token's contribution, and (iii) how far the policy may move per update.

17.1.1 The original pipeline: RL from human feedback

RM 6.1.1

RL entered mainstream LLM training through InstructGPT (Ouyang et al., 2022), the recipe that turned GPT-3 into the first ChatGPT. Its RL from human feedback (RLHF) pipeline has two phases (Fig. 17.4).

Phase 1: learn a reward model. For many prompts, sample several responses (with the temperature and nucleus sampling of Chapter 15) and ask annotators to rank them. Take a question like "Which laptop specs matter if I mostly program?" One answer lists fifteen components; another says "prioritise RAM and an SSD, and a GPU only if you train models locally". Annotators tend to prefer the specific answer. A *reward model* $r_\phi(x, y)$, usually an LLM with a scalar head, is fitted to these comparisons with the Bradley–Terry likelihood

(Chapter 14):

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y^+,y^-)} \log \sigma(r_\phi(x, y^+) - r_\phi(x, y^-)). \tag{17.3}$$

Phase 2: optimise the policy against the reward model. Generate responses, score them with r_ϕ, and update the policy with Proximal Policy Optimization (PPO) (Schulman et al., 2017). PPO maximises a clipped surrogate (Chapter 18) using per-token advantages from a learned *value model* $V_\psi(s_t)$ (the critic) via generalised advantage estimation, and it subtracts a per-token KL penalty $\beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ from the reward so the policy does not drift into text that fools the reward model.

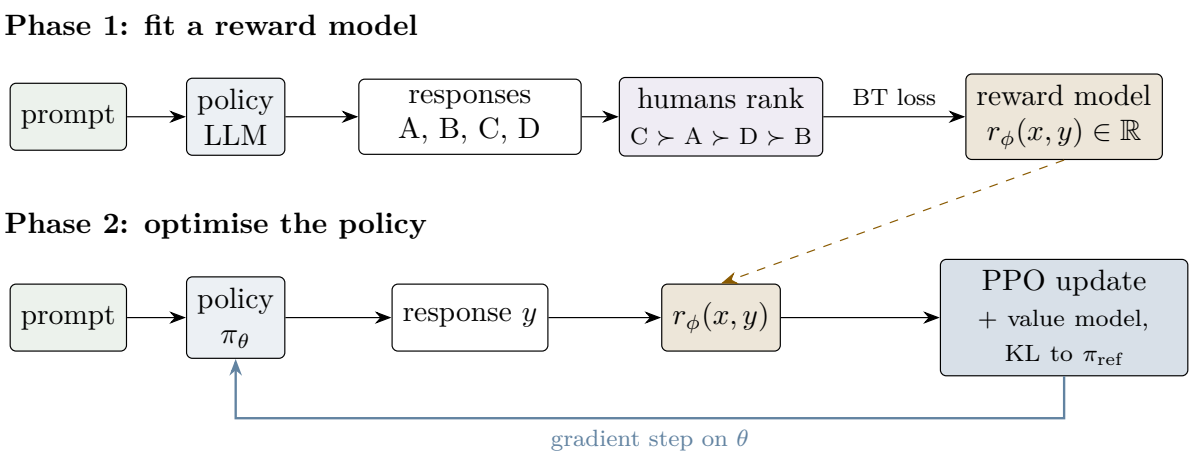


Figure 17.4: RLHF. Phase 1 turns human rankings into a learned scalar reward. Phase 2 samples from the policy, scores each sample with the frozen reward model and updates the policy with PPO. The same skeleton (generate, score, update) survives in RLVR; only the scorer changes.

RLHF matters to us as the ancestor of RLVR. The control flow is the one we will implement: generate, score, update. What makes RLHF heavy is the cast of models. A PPO-based RLHF step touches four networks of comparable size: the policy (trained), the value model (trained), the reward model (frozen) and the reference policy (frozen).

Napkin Math: the four-model tax of PPO-RLHF

Take an 8B policy with mixed-precision AdamW at 16 bytes per trained parameter (Chapter 7).

- Policy (trained): $8 \times 10^9 \times 16\,\text{B} = 128\,\text{GB}$.
- Value model (trained, same size): another 128 GB.
- Reward model and reference model (frozen, bf16): $2 \times 8 \times 10^9 \times 2\,\text{B} = 32\,\text{GB}$.

That is ≈ 288 GB of state before a single activation or KV-cache byte, so the job spans at least four 80 GB GPUs just to hold it. GRPO drops the value model (no critic), and RLVR drops the reward model (a program replaces it). What remains is the policy plus, optionally, the reference: $\approx 128 + 16 = 144$ GB, roughly half.

17.1.2 From human feedback to verifiable rewards

RM 6.1.2

Training a reward model is expensive, and a learned scorer is also *exploitable*: push hard enough and the policy finds outputs the reward model likes but humans do not (reward hacking). For tasks where correctness can be *checked*, both problems disappear. RL with verifiable rewards

(RLVR) replaces r_ϕ by a deterministic program (Fig. 17.5). For math, that program is the answer verifier of Chapter 14: extract the final `\boxed{}` answer, normalise it, test equivalence with the reference, and return 1 or 0. For code it is a compiler plus unit tests run in a sandbox.

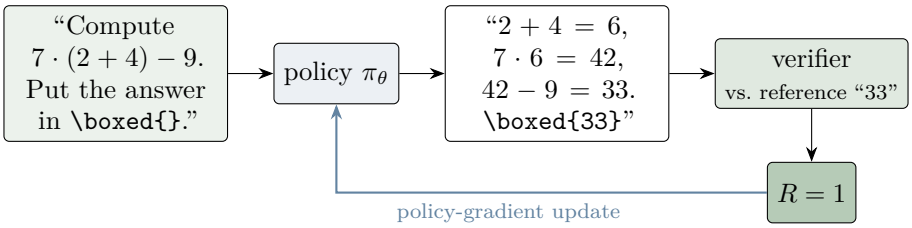


Figure 17.5: RLVR collapses RLHF's two phases into one loop. A program (here, an answer checker) produces the reward, so there is no reward model to train, store or serve.

RLVR became the default recipe for reasoning after DeepSeek-R1 (DeepSeek-AI, 2025) , which showed that correctness checks on math plus compile-and-test checks on code, with no human preference data and no learned reward model, are enough to grow long, self-correcting chains of thought. Its advantages:

- **No reward model.** One fewer network to train, store and run. The verifier is CPU code that costs microseconds to milliseconds per sample.
- **Determinism.** The same output always gets the same reward. There is no annotator disagreement and no reward-model noise.
- **Unlimited labels.** Given reference answers, any number of fresh samples can be scored without more human effort.

The price is *scope*: RLVR only applies where a checker exists (math, code, formal proofs, structured extraction, games). It is also only as good as the checker. A verifier that accepts `\boxed{33.0000001}` or that reads the *first* box when the model writes ten will be gamed. Code rewards add an operational burden: model-generated programs must run in an isolated sandbox with CPU, memory and time limits and no network access, a small distributed system of its own (Chapter 19). The RL algorithm is identical either way, so we stay with math.

17.2 RLVR with GRPO

RM 6.2

RLVR fixes *where the learning signal comes from*. We still need a policy optimiser that turns rewards into weight updates. PPO is the classic choice. GRPO, introduced in DeepSeekMath (Shao et al., 2024) and used for DeepSeek-R1, is a simplification that removes PPO's value model. Instead of learning $V_\psi(s)$ to predict how good a state is, GRPO samples a *group* of G completions for the same prompt and uses the group's own reward statistics as the baseline. That is the "group relative" in the name.

GRPO is not free, though: it moves cost from *memory* (no critic) to *sampling* (G completions per prompt instead of one). Since sampling is the expensive phase of RL (Section 17.11), that trade is worth understanding precisely, and much of Chapter 19 is about making it cheaper.

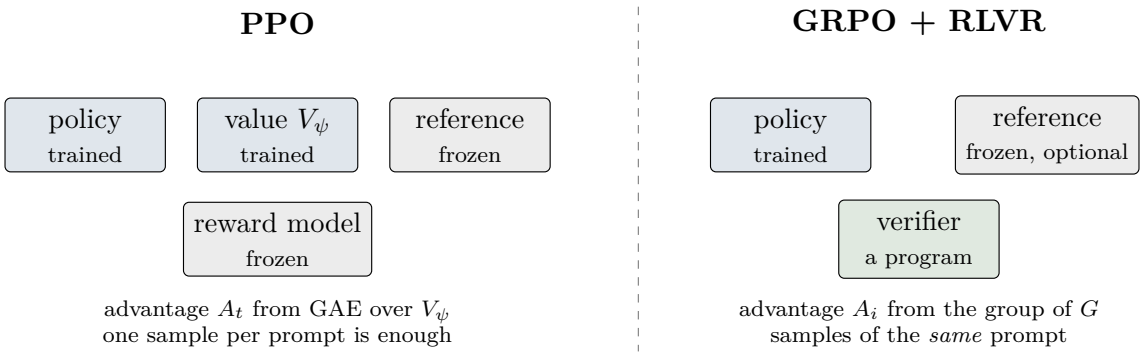


Figure 17.6: The cast of models. PPO-based RLHF trains a critic and runs a reward model. GRPO with verifiable rewards replaces the critic by group statistics and the reward model by a program. The only extra network left is an optional frozen reference for a KL penalty.

17.2.1 An intuition: practising a climbing route

RM 6.2.1

GRPO has many moving parts and a lot of RL jargon, so before the equations here is a picture to hang them on. Imagine a climber training on an indoor bouldering wall.

1. The coach points at a **route** (the *prompt*).
2. The climber makes several **attempts** at it, each a little different because their movement is not perfectly repeatable (the *rollouts*, sampled at temperature > 0).
3. The only feedback is binary and comes at the very end: did they **top out**? Nobody grades individual moves (a *sparse, outcome reward*).
4. After the session the climber compares attempts **with each other**: an attempt that topped out on a route they usually fail is very informative, while an attempt that topped out on a route they always send is not (the *group-relative advantage*).
5. For every attempt they recall how **habitual** its moves were, that is, how likely they would be to repeat that exact sequence (the *log-probability*).
6. They adjust their habits: make the move sequences of better-than-average attempts more habitual and the others less so (the *policy-gradient update*).
7. Optionally they check they are not drifting away from the fundamentals their first coach drilled into them (the *KL penalty to a reference policy*) and avoid overhauling their style in a single session (*clipping*, Chapter 18).

Notice what is *absent*: no one trained a separate judge to predict how good an attempt will be halfway up the wall. That judge would be PPO's value model, and GRPO does without it by comparing whole attempts on the same route.

17.2.2 The GRPO procedure

RM 6.2.2

Formally, for a prompt x GRPO samples a group $\{y_1, \dots, y_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$, scores each with the verifier to get $r_1, \dots, r_G$, and forms the advantages

$$A_i = \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G}) + \epsilon}. \tag{17.4}$$

Every token of completion i shares the same advantage A_i. The full objective of DeepSeekMath, which we build up to over two chapters, is

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\Big(\min\left(\rho_{i,t}A_i,\ \mathrm{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)A_i\right) - \beta\,\mathbb{D}_{i,t}\Big)\right], \tag{17.5}$$

with the per-token importance ratio $\rho_{i,t} = \pi_\theta(y_{i,t} \mid x, y_{i,<t})/\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})$, and where $\mathbb{D}_{i,t}$ is a per-token estimate of $\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$. That is a lot at once. This chapter implements the core, the advantage-weighted log-likelihood

$$\mathcal{L}_{\text{PG}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G} A_i \log \pi_\theta(y_i \mid x) = -\frac{1}{G}\sum_{i=1}^{G} A_i \sum_t \log \pi_\theta(y_{i,t} \mid x, y_{i,<t}), \tag{17.6}$$

which is REINFORCE with a group-mean baseline and group-std scaling. Chapter 18 adds the ratio and clipping, the KL term and format rewards, and examines the many variants (token- vs. sequence-level averaging, no std scaling, ...) that recent work proposes. Figure 17.10 shows the six stages for one prompt with numbers we compute later in this chapter. Keep it at hand; every section below fills in one box.

17.3 Loading a pretrained model

RM 6.3

GRPO needs a policy that *sometimes* succeeds. If every rollout gets reward 0, every advantage in Eq. (17.4) is 0 and nothing is learned (Section 17.7). That is why reasoning RL starts from a capable pretrained model rather than from random weights. At full scale we start from the Qwen3-0.6B *base* checkpoint of Chapter 6, loaded with the same helper used throughout Part V:

Listing 17.1: Loading the base policy and its tokenizer (real-hardware path of `run.py`).

```
from llmsys.qwen3 import load_qwen3_from_dir, Qwen3Tokenizer
from llmsys.utils import get_device

device = get_device()                          # cuda > mps > cpu
model = load_qwen3_from_dir("Qwen3-0.6B-Base", "0.6B",
                            dtype=torch.bfloat16, device=device)
tok = Qwen3Tokenizer("Qwen3-0.6B-Base/tokenizer.json")
tok.eos_token_id = tok.pad_token_id          # base model ends text with </endoftext
    />
```

Before training, sample one answer with the temperature and top-p sampler of Chapter 15 to make sure everything is wired correctly. A base model asked "Half of $3x - 9$ equals $x + 37$; find x" will often produce a wrong or unboxed answer, which is fine: we only need the plumbing to work.

A CPU-sized stand-in. A single GRPO step on Qwen3-0.6B generates tens of thousands of tokens, and the experiment is only practical on a GPU. To make every result in this chapter reproducible on a laptop we also use a *toy verifiable task* (`llmsys.rl.toy`): single-digit addition. The prompt is `Q:3+5=\nA:` and a fully rewarded completion is `\boxed{8}` followed by `<|eos|>`. A 2-layer, 64-wide Qwen3-shaped model (about 76,000 parameters, with the same RMSNorm, RoPE, GQA and SwiGLU code as the real one) is first *SFT-warm-started* on deliberately imperfect demonstrations: 40% of answers are off by one, only 30% are boxed, and 20% open

with a `<think>...</think>` scratchpad. This mimics the situation of a real base model: it "knows" the arithmetic most of the time but rarely presents the answer in the format the verifier insists on. Every curve in Fig. 17.12 and every number in Fig. 17.10 comes from this toy, measured on a CPU in this book's build environment.

Pitfall: `inference_mode` versus `no_grad`

The generation helpers of earlier chapters were wrapped in `@torch.inference_mode()`. Tensors created inside inference mode are flagged and can *never* participate in autograd later, so reusing the sampled ids or cached activations in the differentiable log-probability pass raises an error. RL code that samples and then trains must sample under `@torch.no_grad()`, which disables gradient tracking without tagging the outputs.

17.4 Loading a MATH training subset

RM 6.4

The MATH dataset (Hendrycks et al., 2021) has about 12,500 competition problems. Five hundred of them form MATH-500, the evaluation set of Chapter 14. For training we use the remaining 12,000, a split built to exclude every MATH-500 problem (Fig. 17.7), so that evaluation numbers measure generalisation rather than memorisation. Contamination is a systems problem as much as a statistical one: once training data is assembled from many sources by many scripts, someone must *prove* that the test set is not in it (Chapter 14 discusses n-gram overlap checks).

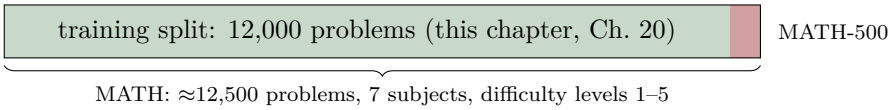


Figure 17.7: Disjoint training and evaluation splits of MATH. RL samples only from the left part; all reported accuracies use the right part.

A record has the fields `problem`, `answer`, `solution`, `type` (subject), `level` and an id:

Listing 17.2: Shape of one training record (contents abbreviated).

```
{"problem":  "A rental costs $12 per day plus $0.15 per mile. ... How many miles?",
 "answer":   "180",
 "solution": "Let m be the number of miles ... so m = \\boxed{180}.",
 "type": "Algebra", "level": "Level 2", "unique_id": 917}
```

RLVR uses only `problem` (the prompt) and `answer` (what the verifier compares against). The worked `solution` is deliberately *not* used. Rewarding similarity to a reference derivation would push the model towards one particular style of proof, whereas an outcome reward lets it explore any path that ends at the right answer. This is the essential difference from supervised fine-tuning on solutions (Chapter 20). For the toy task, `make_examples` enumerates the 100 facts $a + b$ with $a, b \in \{0, \ldots, 9\}$. Because the toy's "knowledge" comes from SFT on all 100 facts, we train and evaluate on the same prompts and read evaluation as *in-distribution* accuracy; the code also supports a held-out split for the exercises.

17.5 Sampling rollouts

RM 6.5

Stage 1 of GRPO samples G completions per prompt from the current policy. Listing 17.3 is the whole sampler: prefill the prompt into a KV cache (Chapter 11), then decode one token at

a time with temperature and top-p (Chapter 15) until EOS or the token budget.

Listing 17.3: Unbatched rollout sampler (`labs/lab17_grpo/solution.py`).

```
@torch.no_grad()
def sample_response(model, tokenizer, prompt, max_new_tokens=32,
                    temperature=1.0, top_p=1.0, generator=None):
    device = next(model.parameters()).device
    prompt_ids = torch.tensor(tokenizer.encode(prompt), device=device)
    cache = KVCache(model.n_layers)
    logits = model(prompt_ids[None], cache=cache)[:, -1]    # prefill
    generated = []
    for _ in range(max_new_tokens):                          # decode
        nxt = sample_next_token(logits, temperature, None, top_p,
                                generator)
        generated.append(int(nxt))
        if int(nxt) == tokenizer.eos_token_id:
            break
        logits = model(nxt, cache=cache)[:, -1]
    full = torch.cat([prompt_ids,
                      torch.tensor(generated, device=device)])
    return full, prompt_ids.numel(), tokenizer.decode(generated)
```

Three details matter. First, the function returns the *full* id sequence and the prompt length, which is exactly what the log-probability stage needs, so nothing has to be re-tokenised (re-tokenising decoded text is a classic RL bug because encode(decode(y)) $\neq y$ for many BPE sequences). Second, the EOS token is *kept* in the completion. It is a real action the policy took, and if it were never trained on, a long RL run could slowly forget how to stop. Third, it uses an explicit `torch.Generator` so a rollout can be replayed bit for bit in tests.

Calling `sample_response` G times in a loop is the clearest form, and the version the chapter's walk-through uses. It is also the slowest. The library sampler `llmsys.rl.rollout.sample_rollouts` stacks all $P \times G$ sequences of a step into one left-padded batch (`generate_batch` from Chapter 12). Since decode is memory-bound, a batch of 64 costs barely more per step than a batch of 1.

Systems Lens: a GRPO step is mostly an inference job

Consider $P \times G = 64$ rollouts of up to 1024 new tokens after a 256-token prompt. Using the roofline helper `llmsys.flops.decode_step_time_lower_bound` on an H100:

- **Qwen3-0.6B.** Every decode step must stream 1.19 GB of weights plus a KV cache that reaches 9.4 GB at 1280 tokens (64 sequences). Summed over 1024 steps, the bandwidth bound alone is $\approx$2.1 s. The training side (old-logprob forward plus forward/backward on 64×1280 tokens, $\approx 6N \cdot$tokens $= 2.9 \times 10^{14}$ FLOPs) takes $\approx$0.3 s at peak, or 0.74 s at a realistic 40% MFU.
- **Qwen3-8B.** Rollouts: $\approx$7.2 s lower bound. Training: 4.0×10^{15} FLOPs, $\approx$10 s at 40% MFU.

So "rollouts dominate" is not a law of nature. It depends on model size, response length and how close the sampler gets to the bandwidth roofline. Real samplers rarely exceed 50% of peak bandwidth, reasoning responses are 4–32$\times$ longer than 1024 tokens, and the *slowest* rollout in the batch sets the step time (the long-tail problem of Chapter 19). In practice, rollouts take most of the wall-clock time in reasoning RL, which is why production systems

run them on a dedicated inference engine (vLLM, SGLang) rather than in the trainer.

17.6 Calculating rewards

RM 6.6

Stage 2 scores each rollout. Our reward is the verifier of Chapter 14 in its strictest mode: the answer must appear inside `\boxed{}`, and the boxed content must match the reference after normalisation.

Listing 17.4: The RLVR reward: correct *and* boxed.

```
def reward_rlvr(text, truth):
    pred = extract_boxed(text)          # last \boxed{...}, nested braces ok
    if pred is None:
        return 0.0                      # implicit format requirement
    return float(normalize_answer(pred) == normalize_answer(truth))
```

The `None` branch is doing quiet but important work. A response that says "the answer is 8" gets nothing, even though it is right. The reward therefore teaches two things at once, correctness and the output contract that the evaluation harness relies on. For the four toy completions of the prompt `Q:4+3=` shown in Fig. 17.10 the rewards are $(1, 1, 0, 0)$: the third is correct but unboxed, the fourth is boxed but wrong.

Key Idea: outcome rewards beat process rewards (so far)

It is tempting to reward intermediate steps with a *process reward model* (PRM). The DeepSeek-R1 report describes trying this and abandoning it: per-step rewards were hard to define, expensive to compute and easy to hack. Rewarding only the final answer and letting the model discover its own intermediate steps worked better. Process supervision remains useful at *inference* time for reranking (Chapter 16).

Pitfall: verifier loopholes become policy behaviour

Whatever the verifier tolerates, RL will find. A checker that reads the *first* `\boxed{}` teaches the model to write many boxes; one that does substring matching teaches it to box long lists of numbers. Extract the *last* box, require it to be unique if you can afford to, cap the length of the boxed content, and unit-test the verifier against adversarial strings before any long run.

17.7 From rewards to advantages

RM 6.7

A reward says how good one rollout is in absolute terms. An *advantage* says how much better or worse it is than its siblings for the same prompt (Eq. (17.4)):

Listing 17.5: Group-relative advantages.

```
def compute_advantages(rewards, eps=1e-4):
    rewards = rewards.float()
    return (rewards - rewards.mean()) / (rewards.std() + eps)   # unbiased std
```

For rewards $(1, 1, 0, 0)$ the mean is 0.5, the unbiased standard deviation is $\sqrt{1/3} = 0.577$, and the advantages are $(+0.866, +0.866, -0.866, -0.866)$. The advantage multiplies the gradient of

each rollout's log-likelihood: positive values make that rollout more likely, negative values less likely, and values near zero contribute almost nothing.

Binary rewards in closed form. With $r_i \in \{0, 1\}$ and a fraction p of the group correct, mean $= p$ and the unbiased std is $s = \sqrt{\frac{G}{G-1} p(1-p)}$. A correct rollout therefore gets $A^+ = (1-p)/s$ and an incorrect one $A^- = -p/s$. Figure 17.8 plots both for $G = 8$. When a prompt is hard (small p), the rare success gets a large positive push and each failure a small negative one; when it is easy, the reverse. Because the advantages sum to zero, the total "probability mass" moved is balanced within every group.

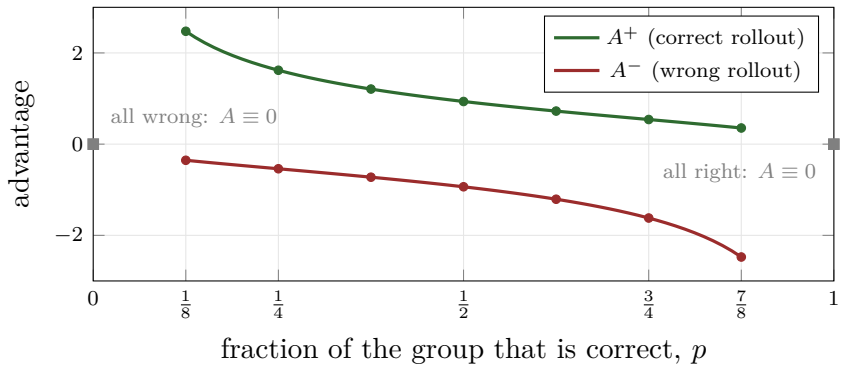


Figure 17.8: Advantages for binary rewards with $G = 8$ (dots are the attainable values $p = k/8$). Rare successes on hard prompts receive the largest positive weight. At $p = 0$ and $p = 1$ every advantage is zero and the prompt contributes no gradient.

Zero-variance groups. If all G rollouts get the same reward, every advantage is zero (the ϵ in the denominator prevents a division by zero) and the policy gradient is exactly zero. This is the right behaviour: a group that is uniformly right or uniformly wrong says nothing about *which* behaviour to reinforce. Its cost is real, though. We paid for G full rollouts and got no gradient. With per-prompt success rate p the probability of a zero-variance group is $p^G + (1-p)^G$, which for $G = 8$ is 27% at $p = 0.15$ and 43% at $p = 0.9$. Chapter 18 tracks this fraction and discusses DAPO's dynamic sampling, which spends extra rollouts to replace such groups.

Key Idea: GRPO's baseline is a Monte Carlo value estimate

PPO learns $V_\psi(x)$, the expected reward from prompt x. GRPO estimates the same quantity by averaging G samples, $\hat{V}(x) = \frac{1}{G}\sum_j r_j$. It is unbiased, needs no training, and its variance shrinks as $1/G$. It is *not* independent of y_i, since r_i is part of its own baseline, which introduces a small $O(1/G)$ bias; leave-one-out baselines (RLOO) remove it. Dividing by the group std is a separate heuristic that Chapter 18 revisits (Dr. GRPO drops it).

17.8 Scoring rollouts with sequence log-probabilities

RM 6.8

Stage 4 asks how likely the current policy was to produce each rollout. In Chapter 16 we scored answers by their *average* per-token log-probability,

$$\bar{\ell}(y \mid x) = \frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}), \tag{17.7}$$

because length normalisation makes answers of different lengths comparable. GRPO, as we implement it in this chapter, uses the *sum*, $\log \pi_\theta(y \mid x) = |y| \cdot \bar{\ell}$. The reward and the advantage belong to the whole sequence, so the quantity they multiply in Eq. (17.6) should be the log-likelihood of the whole sequence. Averaging would silently reweight the update: a 2000-token rollout and a 20-token rollout with the same advantage would contribute equal total gradient, so each token of the long one would be pushed 100 times less. Whether that is desirable is a real design question. Chapter 18 shows that the original GRPO paper actually *does* average per sequence, and that DAPO and Dr. GRPO each criticise a different consequence of that choice. For now, the summed version is the clean REINFORCE estimator.

A side effect of summing is that sequence log-probabilities scale with length: each extra token adds a negative term. For two equally rewarded answers, the shorter one has the larger log-likelihood, and gradient ascent on $A \cdot \log \pi$ is "cheaper" along it. In the worked example (Fig. 17.10) the 11-token answer with a `<think>` scratchpad has log-probability -3.06, versus -2.27 for the bare 4-token boxed answer.

Listing 17.6: Differentiable sequence log-probability. `gather` picks, for every position, the log-prob of the token that actually came next.

```
def sequence_logprob(model, token_ids, prompt_len):
    logits = model(token_ids[None])[0].float()   # (T, V), upcast to fp32
    logprobs = torch.log_softmax(logits, dim=-1)
    # row t predicts token t+1: pick the log-prob of the realised next token
    picked = logprobs[:-1].gather(1, token_ids[1:, None]).squeeze(-1)
    return picked[prompt_len - 1:].sum()         # completion tokens only
```

The indexing deserves a careful look (Fig. 17.9). Row t of the logits predicts token $t+1$, so the first completion token (index `prompt_len`) is scored by row `prompt_len-1`. Slicing the `(T-1,)` vector from `prompt_len-1` therefore keeps exactly the $|y|$ completion terms. A first draft might build an explicit `torch.arange` of positions and index with it. `gather` computes the same thing in one fused indexing kernel and avoids materialising index tensors.

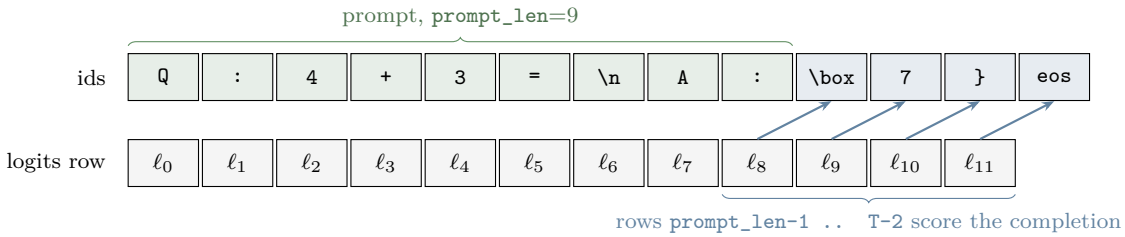


Figure 17.9: The off-by-one in log-probability extraction. Logit row t predicts token $t+1$, so the completion (blue) is scored by rows $8, \ldots, 11$ for a 9-token prompt. The prompt's own likelihood is ignored, since the policy did not choose those tokens.

The returned scalar carries a `grad_fn`. PyTorch has recorded the graph from the weights through the embedding, every attention and MLP block, the LM head and the `log_softmax` to this sum, so a later `backward()` will deliver $\nabla_\theta \log \pi_\theta(y \mid x)$, which is exactly the term the policy-gradient theorem asks for.

Systems Lens: the hidden $(T \times V)$ tensor

`log_softmax(logits.float())` materialises a float32 tensor of shape $(N, T, |\mathcal{V}|)$ for N rollouts. For Qwen3 ($|\mathcal{V}| = 151{,}936$), 8 rollouts of 2048 tokens need $8 \times 2048 \times 151{,}936 \times 4\,\text{B} \approx$

10 GB, plus the same again for its gradient in the backward pass. That is often *larger than the model*. Production trainers therefore (i) compute log-probs in row chunks, (ii) use fused "linear + cross-entropy" kernels that never materialise the full logits (for example Liger-Kernel's fused linear cross-entropy), and (iii) recompute rather than store. Note also that we *recompute* log-probs with a fresh forward pass instead of keeping the logits from sampling. Keeping them would pin T autograd graphs from the decode loop, and in a real system the sampler is a different engine whose numerics differ from the trainer's anyway (Chapter 19).

17.9 From advantages to a policy update: the GRPO loss

RM 6.9

Stage 5 combines the two branches. Multiply each rollout's sequence log-probability by its advantage, average over the group, and negate (Eq. (17.6)):

Listing 17.7: The policy-gradient loss.

```
def policy_gradient_loss(advantages, logps):
    return -(advantages.detach() * logps).mean()
```

The negation is only a convention. We want to *maximise* $\sum_i A_i \log \pi_\theta(y_i \mid x)$, and PyTorch optimisers *minimise*, so we minimise the negative. Lowering $-f$ from -2 to -3 is the same as raising f from 2 to 3. `.detach()` marks the advantages as constants. They depend on rewards, which are not differentiable, and in a more elaborate setup (a learned reward or a baseline network) we would not want gradients leaking into them.

Differentiating makes the mechanics explicit:

$$\nabla_\theta \mathcal{L}_{\text{PG}} = -\frac{1}{G}\sum_{i=1}^{G} A_i \sum_t \nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t}). \tag{17.8}$$

A gradient-descent step moves θ along $+A_i \nabla \log \pi(y_i)$. Every token of a positive-advantage rollout becomes more probable, and every token of a negative one less probable, *including tokens the two rollouts share*. When a correct and an incorrect rollout both begin with `\boxed{`, the shared prefix receives opposite pushes that partly cancel. What survives is the difference: the `7` versus the `8`. Credit assignment in GRPO emerges statistically from such cancellations across many groups, not from any per-token judgement.

Figure 17.10 traces the stages on our toy prompt. The loss is $-\frac{1}{4}[0.866(-2.269)+0.866(-3.059)-0.866(-1.299)-0.866(-3.400)] = +0.136$. Its value is *not* a measure of quality. Unlike a supervised loss it can be positive or negative, it does not trend towards zero, and two runs with identical learning can show very different loss curves. Only its gradient matters. Note what the update will do here: the unboxed "7" is currently the policy's favourite completion (-1.30 is the highest log-probability in the group), and it will become less likely, while both boxed correct answers become more likely. That is how RLVR teaches the output contract.

Deeper Dive: why the summed loss equals "ratio × advantage" at ratio 1

Chapter 18 writes the objective with the importance ratio $\rho_i = \pi_\theta(y_i)/\pi_{\theta_{\text{old}}}(y_i)$ instead of $\log \pi_\theta(y_i)$. Since $\nabla_\theta \rho_i = \rho_i \nabla_\theta \log \pi_\theta(y_i)$ and $\rho_i = 1$ when $\theta = \theta_{\text{old}}$, the two objectives have identical gradients on the first update after sampling. They differ only when the same

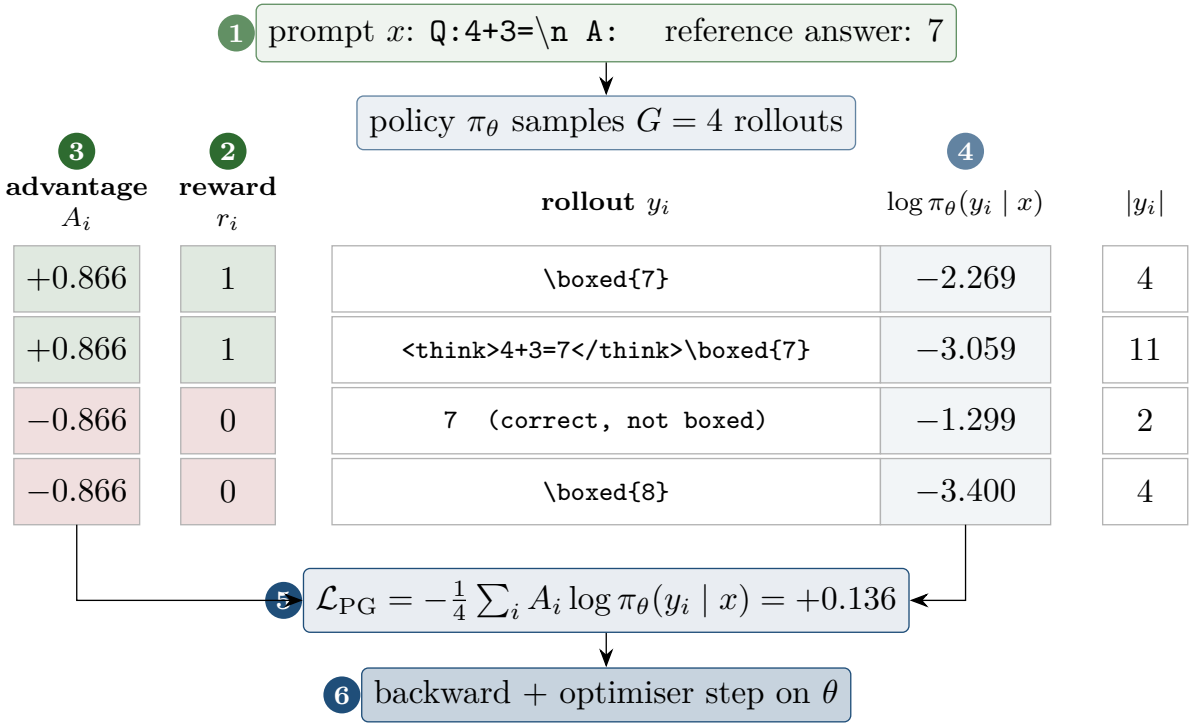


Figure 17.10: One GRPO step on the toy task, with numbers computed from the SFT-warm-started model. (1) One prompt; (2) verifier rewards; (3) group-relative advantages; (4) summed log-probabilities from a differentiable forward pass; (5) the loss; (6) the update. The most likely rollout under the current policy, the unboxed "7", is penalised, because the reward requires the box.

rollouts are reused for several updates, which is exactly when clipping becomes necessary.

17.10 Putting the stages into one function

RM 6.10

Listing 17.8 packages stages 1–5 for one prompt. It interleaves stages 2 and 4 inside the sampling loop (score each rollout and compute its log-probability as soon as it exists) to avoid a second loop. The logical dataflow is unchanged.

Listing 17.8: `compute_grpo_loss`: stages 1–5 for one training example.

```python
def compute_grpo_loss(model, tokenizer, example, render_prompt,
                      num_rollouts=4, max_new_tokens=16, temperature=1.0,
                      top_p=1.0, generator=None):
    assert num_rollouts >= 2, "GRPO needs a group to compare against"
    prompt = render_prompt(example)
    was_training = model.training
    model.eval()                          # no dropout while sampling/scoring
    logps, rewards, samples = [], [], []
    for _ in range(num_rollouts):
        # stage 1: sample a rollout
        ids, plen, text = sample_response(model, tokenizer, prompt,
                                          max_new_tokens, temperature,
                                          top_p, generator)
        r = reward_rlvr(text, example["answer"])              # stage 2
        logps.append(sequence_logprob(model, ids, plen))      # stage 4
        rewards.append(r)
```

```
        samples.append({"text": text, "reward": r,
                        "gen_len": ids.numel() - plen})
    if was_training:
        model.train()
    adv = compute_advantages(torch.tensor(rewards))          # stage 3
    loss = policy_gradient_loss(adv, torch.stack(logps))     # stage 5
    return {"loss": loss.item(), "loss_tensor": loss, "rewards": rewards,
            "advantages": adv.tolist(), "samples": samples}
```

The function returns both the Python float (for logging) and the tensor (for `backward()`), plus per-sample diagnostics. Run it on a prompt the policy cannot solve and you will see rewards $[0, 0, \dots]$, advantages $[0, 0, \dots]$ and a loss of exactly zero: a wasted step, which is the zero-variance case of Section 17.7.

Systems Lens: memory: why the loop above does not scale

Each `sequence_logprob` call keeps its autograd graph alive until `backward()`. With G rollouts that means G sets of saved activations at once, G times the memory of one training sequence. At $G = 8$ and 2048 tokens on Qwen3-0.6B this is several GB of activations before counting the $(T \times V)$ logits. The batched trainer (`llmsys.rl.trainer`) fixes this the standard way. It computes the "old" log-probs of all rollouts under `no_grad`, then runs the differentiable pass in *micro-batches* (`minibatches` in `GRPOConfig`), calling `backward()` after each, so peak activation memory is that of one micro-batch. Gradient accumulation over micro-batches gives the same update as one big batch for the summed loss (and the same up to the averaging constant for the other aggregations of Chapter 18).

17.11 The GRPO training loop

RM 6.11

With the loss in hand, the outer loop is an ordinary PyTorch training loop (Fig. 17.11). Only one box, the loss, is specific to RL.

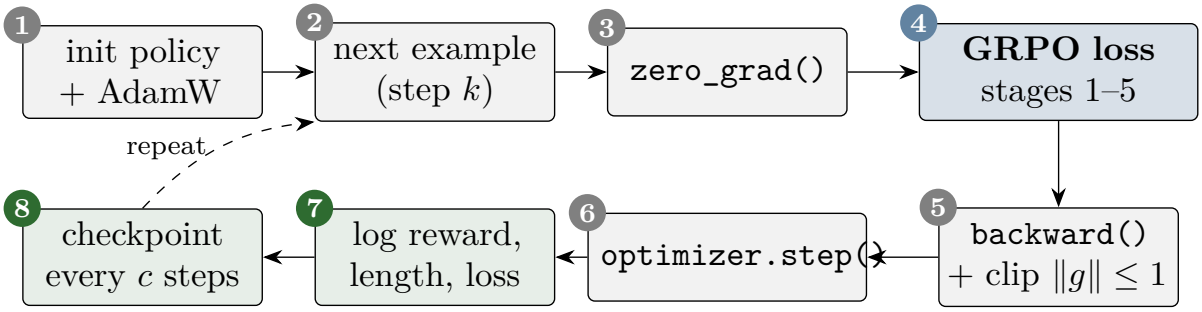


Figure 17.11: The GRPO training loop. Stages 1–3 and 5–6 are boilerplate; 7–8 are optional bookkeeping (metrics and checkpoints). Only stage 4, the loss, is RL-specific.

Listing 17.9: The training loop (abridged from `train_rlvr_grpo`; CSV logging and checkpointing elided).

```
def train_rlvr_grpo(model, tokenizer, data, render_prompt, steps=50,
                    num_rollouts=4, max_new_tokens=16, temperature=1.0,
                    top_p=1.0, lr=3e-4, grad_clip=1.0, seed=0):
    gen = torch.Generator().manual_seed(seed)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.0)
    for step in range(steps):
```

```
        opt.zero_grad(set_to_none=True)
        ex = data[step % len(data)]
        stats = compute_grpo_loss(model, tokenizer, ex, render_prompt,
                                  num_rollouts, max_new_tokens,
                                  temperature, top_p, gen)
        stats["loss_tensor"].backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
        opt.step()
        n = len(stats["samples"])
        reward_avg = sum(stats["rewards"]) / n
        resp_len = sum(x["gen_len"] for x in stats["samples"]) / n
        print(f"[{step+1}/{steps}] loss={stats['loss']:.4f} "
              f"reward_avg={reward_avg:.3f} len={resp_len:.1f}")
```

Hyperparameters. On real models, a temperature of 0.7–1.0 with top-p around 0.9 gives enough diversity within a group for the advantages to be informative, while keeping completions coherent. The learning rate is small, about 10^{-6} to 10^{-5} for a 0.6B model with AdamW, because every step changes a policy that is simultaneously generating its own training data, and a large step can trigger a feedback loop of degenerate samples. The group size G (4–16) and the response budget (512 to tens of thousands of tokens) are mostly a compute decision. Gradient clipping at norm 1 is cheap insurance against the occasional huge-advantage group.

Reading the output. Expect noisy logs. With $G = 4$ and binary rewards the per-step reward can only take the values $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$, and steps with reward 0 or 1 print a loss of exactly zero because their advantages vanish. Large loss magnitudes simply mean that the group disagreed strongly. They do not signal trouble. The quantities to watch are the *moving averages* of reward and response length and, periodically, accuracy on held-out data. On Qwen3-0.6B with MATH, the source book's reference runs show that a few dozen GRPO steps with 4–8 rollouts of ≤ 512 tokens move MATH-500 accuracy from the mid-teens to the mid-forties, approaching the official reasoning variant of the same model. The model mostly learns to finish its reasoning with a boxed answer inside the budget, and secondarily to reason better.

Figure 17.12 shows the same dynamics on our toy task with the batched trainer (`run.py toy-batched`): format is learned almost immediately, correctness keeps improving, and entropy decays as the policy sharpens. The greedy accuracy *without* the box requirement also rose, from 68% after SFT to 96%. The SFT demonstrations were right only 60% of the time, so RL has turned an inconsistent arithmetic skill into a reliable one. This "sharpening" of behaviour the base model can already produce is widely regarded as the main effect of RLVR at small scale.

Systems Lens: where a GRPO step spends its time

Over the 200 steps of Fig. 17.12 (64 rollouts per step, measured on one CPU core) the trainer logged 26.8 s sampling, 9.0 s computing old log-probs and entropies under `no_grad`, and 28.4 s in differentiable forward, backward and optimiser updates. Sampling was *only* about 40% of the step here because toy completions are about five tokens long. On a GPU with realistic reasoning lengths the balance flips. Using the roofline numbers from Section 17.5, for Qwen3-0.6B with 64 rollouts of 1024 tokens:

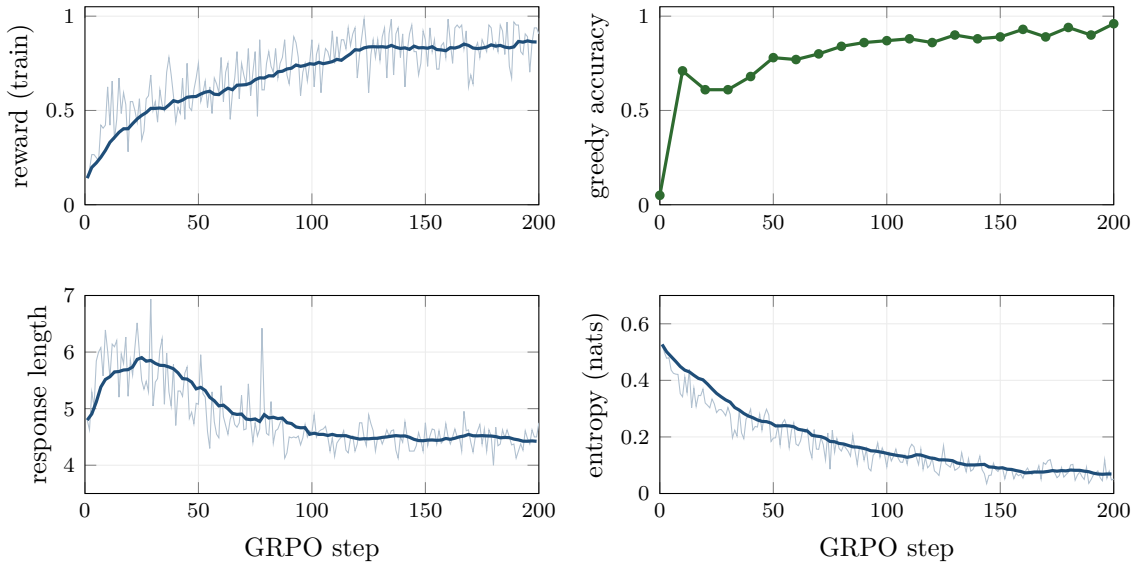
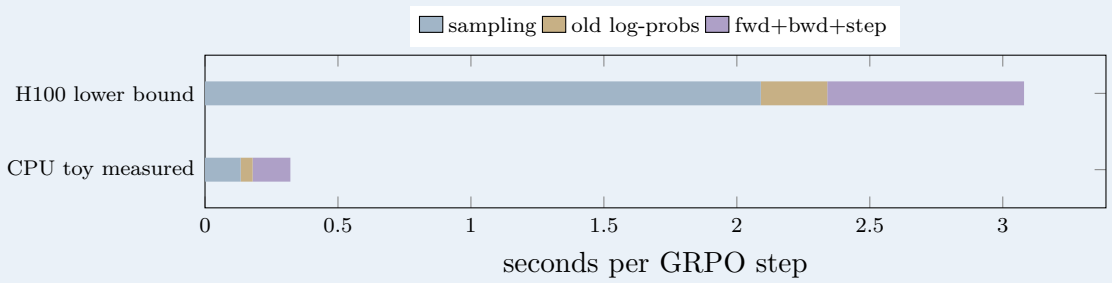


Figure 17.12: A GRPO run on the toy addition task (8 prompts × 8 rollouts per step, learning rate 3×10^{-4}, no KL, no clipping), measured on a CPU in this book's build environment. Light lines are per-step values, dark lines a 20-step moving average. Greedy accuracy (a correct *and* boxed answer, over all 100 prompts) rises from 5% to 96%. The run learns the format within about ten steps and then keeps sharpening correctness. Entropy falls steadily as the policy becomes more deterministic.



The H100 bars are lower bounds: the bandwidth roofline for decoding, and 40% MFU for the training passes. Real samplers sit well above their bound, and reasoning rollouts are far longer than 1024 tokens, so in practice sampling usually accounts for most of the wall-clock time. The toy run is the exception that proves the rule: shrink the completions and the training side dominates again. That is why the balance has to be *measured* per workload, which Lab 17 asks you to do with `time_breakdown`.

Batching and multiple GPUs. The loop above handles one prompt per step and generates its G rollouts sequentially, the clearest form and the slowest. The library trainer batches all $P \times G$ rollouts of a step into one decode batch, computes log-probs for all of them in one padded forward pass, and is the version used for every experiment in Chapter 18. Batched RL rollouts are revisited in Chapter 12 (padding and masks), and Chapter 19 distributes the sampler and trainer across GPUs.

17.12 Loading and evaluating checkpoints

RM 6.12

The loop saves `model.state_dict()` every few steps and on keyboard interrupt, so a long run can be stopped at any point without losing work. Restoring is the usual one-liner,

`model.load_state_dict(torch.load(path))`, and the restored model plugs straight into the evaluation harness of Chapter 14:

Listing 17.10: Evaluating an RL checkpoint on MATH-500 with the Chapter 14 harness.

```
python labs/lab14_eval/run.py --dataset math500 --model-dir Qwen3-0.6B-Base \
    --checkpoint out17/ckpt/grpo-step00050.pt --max-new-tokens 2048
```

Two evaluation details matter. First, evaluate with a *larger* token budget than the training budget. A model trained with 512-token rollouts has learned to box its answer within 512 tokens, and that should not cap its test-time reasoning. Second, track average response length alongside accuracy. In DeepSeek-R1's training, response length grew steadily as the model learned to reason longer and check its work. A tight training budget suppresses that growth, which is a direct compute–quality trade-off: longer rollouts cost linearly more decode steps and quadratically more attention, and the KV cache grows with them.

On the toy task, `evaluate_accuracy` decodes greedily on all 100 prompts. The SFT start scores 5% (correct *and* boxed) and 68% (correct, any format). After 200 GRPO steps both are 96%. More steps are not automatically better. Plain GRPO without clipping or KL can become unstable in long runs, as the reward collapses and gibberish appears in Chapter 18. Keep several checkpoints and select by validation accuracy, not by the last step.

Napkin Math: checkpoint budget

A Qwen3-0.6B `state_dict` in bf16 is $0.6 \times 10^9 \times 2\,\text{B} \approx 1.2\,\text{GB}$ (twice that if it was trained in fp32). Saving every 5 steps for 500 steps produces 100 checkpoints, or 120 GB. Saving every 50 steps produces 12 GB. An 8B policy with optimiser state (needed to *resume*, not just evaluate) is $8 \times 10^9 \times 16\,\text{B} = 128\,\text{GB}$ per checkpoint, and writing it to network storage at 2 GB/s stalls a synchronous trainer for over a minute. Hence the asynchronous, sharded checkpointing of Chapter 7.

17.13 Where the memory goes: a GRPO step at scale

We close the chapter by assembling the systems costs scattered through it into one budget, which is the starting point for Chapter 19. Consider one GRPO step on an 8B policy with $P = 32$ prompts, $G = 8$ rollouts each and up to 4096 new tokens. Table 17.2 lists what must exist in GPU memory and when.

Phase	What lives in memory	Size (8B, bf16)
sampling	policy weights + KV cache for 256 sequences	16 GB + ≈ 150 GB[†]
old log-probs	weights + one micro-batch of activations (no grad)	16 GB + small
reference log-probs (if KL)	a second, frozen copy of the weights	+16 GB
training	weights, grads, AdamW states, fp32 master	128 GB
	activations of one micro-batch (checkpointed)	5–20 GB
	$(T \times V)$ logits for the log-softmax	≈ 5 GB per 8k tokens

Table 17.2: Memory of one GRPO step for an 8B dense policy. [†]KV cache: $2 \cdot L \cdot H_{kv} \cdot d_h \cdot \text{bytes} = 2 \cdot 36 \cdot 8 \cdot 128 \cdot 2 = 147\,\text{KB}$ per token, times 256×4096 tokens $\approx 154\,\text{GB}$. No single GPU holds this, so sampling must be sharded or run in waves.

Three observations drive the design of every RL framework:

1. **The two phases want different parallelism.** Sampling wants many replicas with lots of free memory for KV cache (tensor parallel at most within a node). Training wants the optimiser state sharded (FSDP/ZeRO-3, Chapter 8). Running both on the same GPUs (colocation) means resharding the weights between layouts every step. Running them on separate GPUs (disaggregation) means shipping 16 GB of new weights to the samplers every step.

2. **The sampler and trainer should not wait for each other.** Synchronous GRPO leaves trainer GPUs idle during sampling and sampler GPUs idle during training. Overlapping them makes the data slightly off-policy (generated by an older θ), which is exactly what the importance ratios and clipping of Chapter 18 correct for.

3. **Variance in response length is a scheduling problem.** The step ends when the *longest* rollout ends. With reasoning lengths spanning 100 to 30,000 tokens, most of the batch sits finished while a few stragglers decode at batch size 1, the least efficient point on the roofline.

17.14 Summary

- RL post-training optimises a property of whole outputs. RLHF learns a reward model from human rankings and optimises it with PPO. RLVR replaces the reward model by a deterministic verifier (answer checking, unit tests).
- The policy-gradient theorem turns a non-differentiable reward into a weighting of $\nabla \log \pi_\theta(y \mid x)$. Subtracting a baseline keeps the estimator unbiased and cuts its variance.
- GRPO's baseline is the mean reward of G rollouts of the same prompt, and its advantages are those rewards standardised within the group. There is no value model, so memory is lower, at the price of $G\times$ more sampling.
- Groups whose rollouts all receive the same reward contribute zero gradient. With binary rewards that happens with probability $p^G + (1-p)^G$.
- The loss implemented here is $-\frac{1}{G}\sum_i A_i \log \pi_\theta(y_i \mid x)$ with *summed* token log-probs. Its value is not a quality measure; its gradient is what matters.
- Rollout code must use `no_grad` (not `inference_mode`), keep the EOS token, and return ids rather than text. Log-probs are recomputed in a separate differentiable pass, and that $(T \times V)$ log-softmax is a major memory consumer.
- The training loop is otherwise standard. Track moving-average reward, response length and held-out accuracy, and checkpoint often.
- At scale a GRPO step is an inference job glued to a training job with different memory and parallelism needs, the central problem of Chapter 19.

17.15 Exercises

Exercise 17.1 (*format-aware reward shaping*). Implement `reward_partial`, which returns 1.0 for a boxed correct answer, 0.5 for a correct but unboxed final number, and 0 otherwise. For

the four rollouts of Fig. 17.10, recompute the advantages and the loss. Which rollout's push changes sign? Argue whether partial credit speeds up or slows down learning of the format.

Exercise 17.2 (*zero-advantage groups*). Force all rewards in a group to 0 (then to 1). Verify that the loss *and every parameter gradient* are exactly zero. Explain why this is the desired behaviour, and compute the expected fraction of wasted rollouts per step for a dataset whose per-prompt success rates are uniform on $[0, 1]$, for $G = 4$ and $G = 16$.

Exercise 17.3 (*binary-reward advantages*). Derive $A^+ = (1-p)/s$ and $A^- = -p/s$ with $s = \sqrt{\frac{G}{G-1}p(1-p)}$, and show that $\sum_i A_i = 0$ and $\frac{1}{G-1}\sum_i A_i^2 \approx 1$ for every $0 < p < 1$. What happens to $\max|A_i|$ as $p \to 1/G$ for large G?

Exercise 17.4 (*the baseline's bias*). GRPO's baseline includes r_i itself. Show that the leave-one-out baseline $b_i = \frac{1}{G-1}\sum_{j\neq i} r_j$ (RLOO) yields an unbiased gradient, and that $r_i - b_i = \frac{G}{G-1}(r_i - \bar{r})$. Conclude that, before std scaling, GRPO's mean-centred advantage is RLOO up to a constant factor.

Exercise 17.5 (*sum versus mean*). Write the per-token gradient weight of a rollout of length $|y|$ under (a) the summed loss of this chapter and (b) a per-sequence mean. For two correct answers of lengths 20 and 2000, compute the ratio of per-token pushes under each scheme. Which scheme pushes the policy towards shorter correct answers, and why?

Exercise 17.6 (*the `inference_mode` trap*). Decorate `sample_response` with `@torch.inference_mode()` instead of `@torch.no_grad()` and run `compute_grpo_loss`. Record the exact error, explain which tensor triggers it, and find the smallest change that fixes it without giving up inference mode for sampling.

Exercise 17.7 (*logits memory*). Compute the size of the float32 $(N, T, |\mathcal{V}|)$ log-softmax for $N = 16$, $T = 4096$ and Qwen3's vocabulary, with and without its gradient. Implement a chunked variant of `sequence_logprob` (process C positions at a time). Measure peak memory with `torch.cuda.max_memory_allocated` (or `tracemalloc` on CPU) for several C.

Exercise 17.8 (*measure the step*). Use `time_breakdown` on the toy task for $G \in \{2, 4, 8, 16\}$ and `max_new_tokens`$\in \{4, 16, 64\}$ (force long outputs by masking EOS). Fit $t_{\text{sample}} \approx a \cdot G \cdot T$ and $t_{\text{train}} \approx b \cdot G \cdot T$. Then replace the sequential sampler by `sample_rollouts` (one batch) and explain the change in a using the roofline from Chapter 11.

Exercise 17.9 (*napkin: a 4B run*). For Qwen3-4B with $P = 16$, $G = 16$, 256-token prompts and 4096 new tokens, compute (i) the KV cache at the end of sampling, (ii) the bandwidth-bound sampling time on one H100, and (iii) the training FLOPs and their time at 40% MFU. Does the KV cache fit next to the weights on one 80 GB GPU? If not, how many sampling waves are needed?

Exercise 17.10 (*break the verifier*). Write five completions that a naive verifier (first `\boxed{}`, substring match, no normalisation) scores incorrectly, in both directions. Harden `reward_rlvr`

against them and add them as unit tests. Which loopholes would an RL policy be most likely to discover, and why?

Exercise 17.11 (*held-out generalisation*). Rebuild the toy with `make_examples(holdout=20)`: SFT and GRPO on 80 facts and evaluate on the 20 held-out ones. Does GRPO improve held-out *format*? Held-out *correctness*? Relate your findings to the separation between MATH and MATH-500.

17.16 Lab 17: GRPO from scratch

Lab 17: GRPO from scratch

Code: `code/labs/lab17_grpo`

Goals

Implement every stage of RLVR with GRPO, see a policy learn a verifiable task on a CPU, and measure where a GRPO step spends its time.

Setup

```
cd code
LAB_IMPL=starter  python -m pytest -q labs/lab17_grpo    # your implementation
LAB_IMPL=solution python -m pytest -q labs/lab17_grpo    # reference
```

The toy task, tokenizer and SFT warm start are provided in `llmsys.rl.toy`; the tiny Qwen3-shaped model comes from `llmsys.qwen3`.

Tasks

(each is a function in `starter.py`)

1. `sample_response`: KV-cached sampler with temperature and top-p that keeps EOS and returns ids, prompt length and text (Listing 17.3).
2. `reward_rlvr` and `reward_partial`: strict and partial-credit rewards.
3. `compute_advantages`: group standardisation with unbiased std and ϵ.
4. `sequence_logprob`: differentiable summed log-probability with the correct off-by-one (Fig. 17.9).
5. `policy_gradient_loss`: $-\,\mathrm{mean}(A \cdot \log \pi)$ with detached advantages.
6. `compute_grpo_loss`: stages 1–5 for one example.
7. `train_rlvr_grpo`: the loop, with clipping, CSV logging and checkpoints.
8. `time_breakdown`: seconds spent sampling, in the log-prob pass, and in backward.

Measurements

1. `python labs/lab17_grpo/run.py toy -impl starter -steps 150`: your sequential implementation should lift the moving-average reward from about 0.1–0.2 to above 0.7.
2. `run.py toy-batched -steps 200`: reproduce Fig. 17.12. Plot reward, greedy accuracy, length and entropy with `llmsys.rl.metrics.plot_metrics`.
3. Compare wall-clock per step of the sequential and batched versions and report the speed-up of sampling alone. Expect it to grow with G.
4. Fill in a table of `time_breakdown` for $G \in \{2, 4, 8\}$ and explain the trends (Exercise 17.8).

Deliverables

Passing tests; a plot of your training curves; a one-paragraph explanation of the time breakdown; the greedy accuracy before and after GRPO.

Stretch goals

- On a GPU, run `run.py qwen3` on Qwen3-0.6B-Base with MATH for 50 steps, then evaluate the checkpoint on MATH-500 with Lab 14's harness.
- Replace the group-mean baseline by the leave-one-out baseline (Exercise 17.4) and compare learning curves over three seeds.
- Make sampling and training overlap: generate step $k{+}1$'s rollouts in a background thread while step k trains, and measure the speed-up and the (small) policy lag. This previews Chapter 19.

Chapter 18

Improving GRPO: Diagnostics, Stability and Its Descendants

Chapter 17 ended with a working GRPO loop and a warning: plain GRPO is fine for a few dozen steps but can become unstable over hundreds. This chapter is about the difference between an algorithm that works in a notebook and one you can run for a week. Three skills are involved. The first is *reading* a run: which metrics to log, what healthy and unhealthy curves look like, and why the loss is the least informative number on the dashboard. The second is *stabilising* it with the two mechanisms of the original GRPO objective we skipped so far, clipped importance ratios and a KL penalty, and with auxiliary rewards. The third is knowing the *design space* that has opened up since DeepSeek-R1: DAPO, Dr. GRPO, GSPO, CISPO, GDPO and the DeepSeek-V3.2 corrections each change one specific term of Eq. (17.5), and each has a cost that a systems person should be able to estimate.

Every experiment in this chapter is a run of `labs/lab18_grpo_plus/run.py` on the toy addition task of Chapter 17, from the same SFT-warm-started model, with two seeds per configuration, measured on a CPU in this book's build environment. The toy cannot reproduce every failure of a 0.6B model on MATH. When it fails to, we say so and explain what the real run does and why. As with any RL result, single runs are noisy: treat differences smaller than the seed-to-seed spread as noise.

18.1 From a working loop to a robust one

RM 7.1

It helps to name what we implemented in Chapter 17. With summed log-probabilities, no ratio, no clipping and no KL, the update is REINFORCE with a group-normalised baseline. Its weaknesses are well known from classical RL. Nothing limits the size of one update except the learning rate, so a single unlucky group with extreme advantages can move the policy a long way. Nothing anchors the policy over the long run, so it can drift into degenerate text once rewards stop carrying signal. And every rollout is used exactly once, which is wasteful when rollouts are the expensive part. PPO's two devices, ratio clipping and a KL anchor, address the first two weaknesses and make the third (reusing rollouts) safe. The rest of this chapter adds them one at a time and watches what changes.

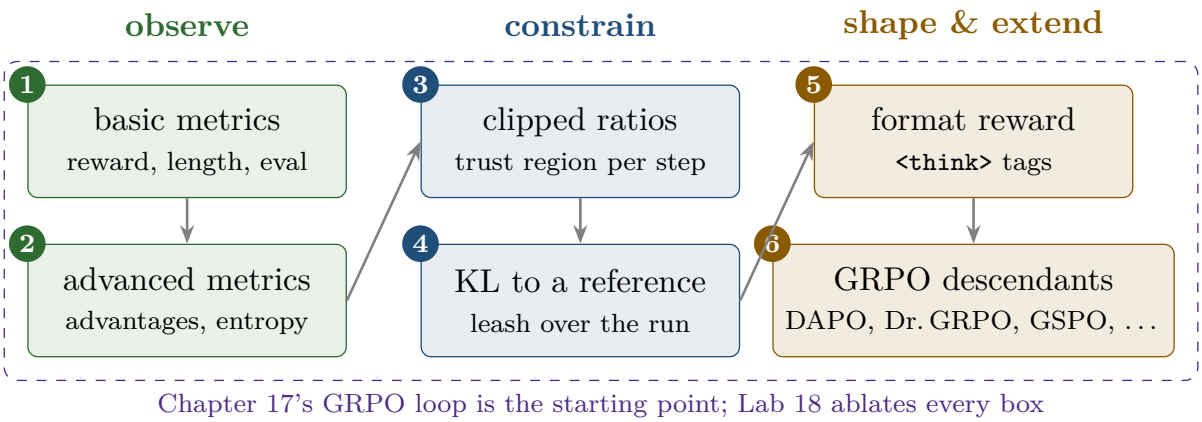


Figure 18.1: Roadmap. We first learn to observe a run (1–2), then constrain how far each update and the whole run may move the policy (3–4), and finally shape the reward and survey the family of algorithms that grew out of GRPO (5–6).

18.2 Tracking GRPO performance metrics

RM 7.2

18.2.1 Executing a training run

RM 7.2.1

Long RL runs belong in scripts, not notebooks. A script survives a dropped SSH session (run it under `tmux` or a job scheduler), writes structured logs that can be plotted while the run is still going, and makes the configuration explicit. The lab's driver takes the variant name and seeds on the command line and writes one CSV row per step:

Listing 18.1: Running GRPO variants from the terminal.

```
cd code/labs/lab18_grpo_plus
python run.py --list                                    # show all variants
python run.py --variants baseline,clip_reuse --seeds 0,1 --steps 150 --out out18
```

The CSV columns are produced by `GRPOTrainer.step` and are the vocabulary of the rest of the chapter: `loss`, `reward_avg`, `correct_avg`, `format_avg`, `adv_avg`, `adv_std`, `entropy_avg`, `resp_len`, `zero_var_frac`, `policy_ratio`, `clip_frac`, `kl`, the per-phase wall-clock times `t_rollout`, `t_logprob`, `t_update`, and periodically `eval_acc`. Writing a row per step is cheap. Recomputing a missing metric afterwards means re-running the experiment.

Systems Lens: logging is part of the system

At scale, metrics come from hundreds of ranks. Reduce them *before* logging: all-reduce the per-rank sums and counts of reward, tokens and entropy, never the per-rank means, which are biased when ranks hold different numbers of tokens. Log asynchronously so a slow metrics backend never stalls the trainer. Also log the *throughput* metrics (tokens generated per second, tokens trained per second, sampler idle fraction). An RL run that learns correctly but uses its GPUs at 15% is a bug too, just a more expensive one.

18.2.2 Inspecting the run

RM 7.2.2

RL metrics are noisy step to step, so we always plot the raw series faintly and a trailing moving average on top:

Listing 18.2: Trailing moving average and a multi-panel metrics plot (`llmsys/rl/metrics.py`).

```python
def moving_average(values, window):
    out, s = [], 0.0
    for i, v in enumerate(values):
        s += v
        if i >= window:
            s -= values[i - window]
        out.append(s / min(i + 1, window))   # shorter window at the start
    return out

plot_metrics("out18/baseline_s0.csv",
             ["reward_avg", "adv_std", "entropy_avg", "zero_var_frac"])
```

What should a healthy run look like? Four series carry most of the information:

- **Loss.** A sanity check, nothing more. GRPO's loss is a weighted log-likelihood whose weights average to zero within every group. It is *not* expected to decrease, and its magnitude reflects how much groups disagreed, not how good the model is. What *is* alarming is a sudden order-of-magnitude jump, or frequent spikes late in a run.

- **Average reward.** Should trend upwards. A reward that sits at 1.0 is good news with a catch: every group is then zero-variance (Section 17.7) and the run has stopped learning. Stop it, or move to harder prompts (curriculum).

- **Response length.** On real reasoning tasks it typically *grows* as the model learns to think longer. A length pinned at the token budget means rollouts are being truncated, so they earn no reward and push the policy in confusing directions. A collapsing length can mean the policy found a shortcut.

- **Held-out accuracy.** The quantity we care about, measured rarely because it is expensive (Chapter 14). A reward that keeps rising while held-out accuracy falls is the signature of reward hacking or overfitting to the training prompts.

In the source book's MATH runs of plain GRPO on Qwen3-0.6B, reward and held-out accuracy both rose for roughly the first 50 steps. Over the following hundreds of steps the loss began to spike more often, response length first grew and then fell, and MATH-500 accuracy declined from its peak, although it was measured on the same 500 problems throughout. A decline on a fixed evaluation set cannot be explained by later training prompts being harder. It points at the algorithm.

Napkin Math: the cost of watching

Evaluating on MATH-500 with a 2048-token budget is 500 rollouts, about 10^6 generated tokens. For a 0.6B model in an engine at about 10,000 output tokens/s that is around 100 s, cheap if done every 50 steps. With an in-loop, unbatched HF-style generator at about 30 tokens/s it is roughly 9 hours, which is why naive `-eval_every` flags slow training to a crawl. Batch the evaluation (Chapter 12), run it on a separate replica from saved checkpoints, or evaluate on a fixed random subset of 100 problems and accept wider error bars (± 10 points at 95% confidence for accuracies near 50%).

18.3 Tracking more advanced metrics

RM 7.3

Two quantities computed inside the GRPO step are worth logging even though they never appear in the loss: summary statistics of the advantages, and the entropy of the policy's next-token distributions.

18.3.1 Advantage statistics

RM 7.3.1

Listing 18.3: Advantage statistics.

```
def advantage_stats(rewards):
    r = torch.as_tensor(rewards, dtype=torch.float32)
    adv = (r - r.mean()) / (r.std() + 1e-4)
    return adv, adv.mean().item(), adv.std().item()
```

The mean is zero by construction. Each group's advantages are mean-centred, so their mean is zero up to floating-point error. Logging it is a free unit test: a drifting `adv_avg` means advantages are being normalised over the wrong axis (for example, across the whole batch instead of within groups) or masked incorrectly.

The standard deviation measures the learning signal. For a group with mixed rewards the std of the normalised advantages is ≈ 1. For a zero-variance group it is 0. Averaged over a batch, `adv_std` therefore tracks roughly the fraction of groups that carry signal. Values near 1 mean healthy, well-scaled updates; values falling towards 0 mean the signal is vanishing (rewards all 0 or all 1); erratic spikes mean a few groups dominate. In practice it is clearer to log the fraction directly, which our trainer does as `zero_var_frac`. Read it together with the average reward: zero variance at reward 0 means "too hard", at reward 1 "solved".

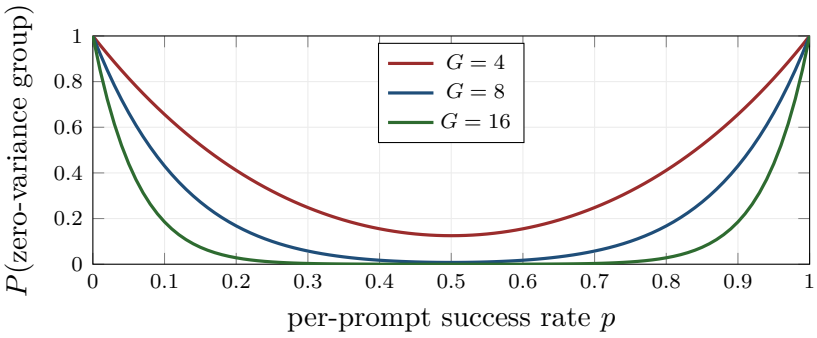


Figure 18.2: Probability that a group of G binary-reward rollouts carries no gradient, $p^G + (1-p)^G$. Very easy and very hard prompts waste their rollouts. Larger groups help in the middle of the range but not at the ends, which is why prompt selection (curricula, DAPO's dynamic sampling) matters as much as G.

18.3.2 Entropy

RM 7.3.2

Entropy measures how spread out the policy's next-token distribution is,

$$H(\pi(\cdot \mid s)) = -\sum_{v \in \mathcal{V}} \pi(v \mid s) \log \pi(v \mid s) \in [0, \log |\mathcal{V}|]. \tag{18.1}$$

It is 0 when one token has all the mass and $\log |\mathcal{V}|$ when the distribution is uniform. RL needs *some* entropy, since rollouts in a group only differ if the policy is uncertain somewhere, and

every policy-gradient update tends to reduce it, since it concentrates mass on rewarded choices. Logging entropy shows which of the two forces is winning.

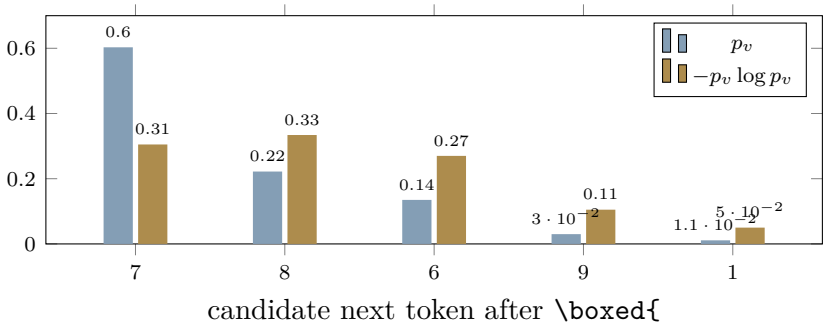


Figure 18.3: Entropy of one next-token distribution (a 5-token toy vocabulary, logits $2, 1, 0.5, -1, -2$). The entropy is the sum of the orange bars, $H = 1.06$ nats, out of a maximum of $\log 5 = 1.61$. Note that the most likely token does not contribute the most: the term $-p \log p$ peaks at $p = 1/e$.

For orientation, with natural logarithms: per-token entropies near 0–0.5 mean the model is almost deterministic at that step; 1–2 means a handful of plausible tokens, which is typical of healthy text; values approaching $\log |\mathcal{V}|$ (11.9 for Qwen3) mean noise. Averaged over a reasoning trace, most tokens are near-deterministic and a few "forking" tokens carry most of the entropy. Two failure modes show up in the average: *entropy collapse* (the policy stops exploring, groups become identical, learning stalls) and *entropy explosion* (updates have damaged the model and it emits random tokens).

Since `log_softmax` already gives all log-probabilities, entropy costs one extra elementwise product and reduction. Listing 18.4 extends the sequence log-prob function of Chapter 17 to return both.

Listing 18.4: Sequence log-probability and mean per-step entropy in one pass.

```python
def sequence_logprob_and_entropy(model, token_ids, prompt_len):
    logits = model(token_ids[None])[0].float()
    logp = torch.log_softmax(logits, dim=-1)
    picked = logp[:-1].gather(1, token_ids[1:, None]).squeeze(-1)[prompt_len-1:]
    steps = logp[:-1][prompt_len - 1:]           # (|y|, V): one row per answer step
    if steps.shape[0] == 0:                      # model emitted nothing
        return picked.sum(), picked.new_tensor(0.0)
    ent = -(steps.exp() * steps).sum(-1)         # H at every answer step
    return picked.sum(), ent.mean().detach()     # entropy is a metric, not a loss
```

Systems Lens: entropy is a full-vocabulary reduction

The log-probability of the sampled token needs one element per row; entropy needs *all* $|\mathcal{V}|$ of them. In a trainer that uses a fused "linear + cross-entropy" kernel to avoid materialising logits, adding an entropy metric can silently bring back the $(N \cdot T) \times |\mathcal{V}|$ float32 tensor: $8 \times 1024 \times 151{,}936 \times 4\,\text{B} \approx 5\,\text{GB}$. Compute it in row chunks (`chunked_logprob_entropy` in the lab bounds the temporary to `chunk`$\times|\mathcal{V}|$), under `no_grad`, and only on the "old log-prob" pass that runs anyway, as `GRPOTrainer` does. Some trainers sample a subset of positions for entropy, which is fine for a metric.

18.3.3 Plotting the additional metrics

RM 7.3.3

Figure 18.4 shows the advanced metrics of our baseline run (Chapter 17's algorithm, seed 0). The average advantage stays at zero throughout, as it must. The other three curves tell one story. At the start about half the groups are zero-variance because all eight rollouts are *wrong* (mostly unboxed). Within twenty steps the format is learned, the zero-variance fraction drops to about 15%, and the advantage std peaks near 0.87. From then on more and more prompts are solved by *all* eight rollouts, the zero-variance fraction climbs back above 50%, and the advantage std declines. By the end, half of every step's rollouts produce no gradient. Entropy decays smoothly from 0.53 to 0.09 nats per token, a policy sharpening on a task with exactly one right answer per prompt. That is healthy here, but it would be a warning sign on open-ended reasoning, where exploration must survive.

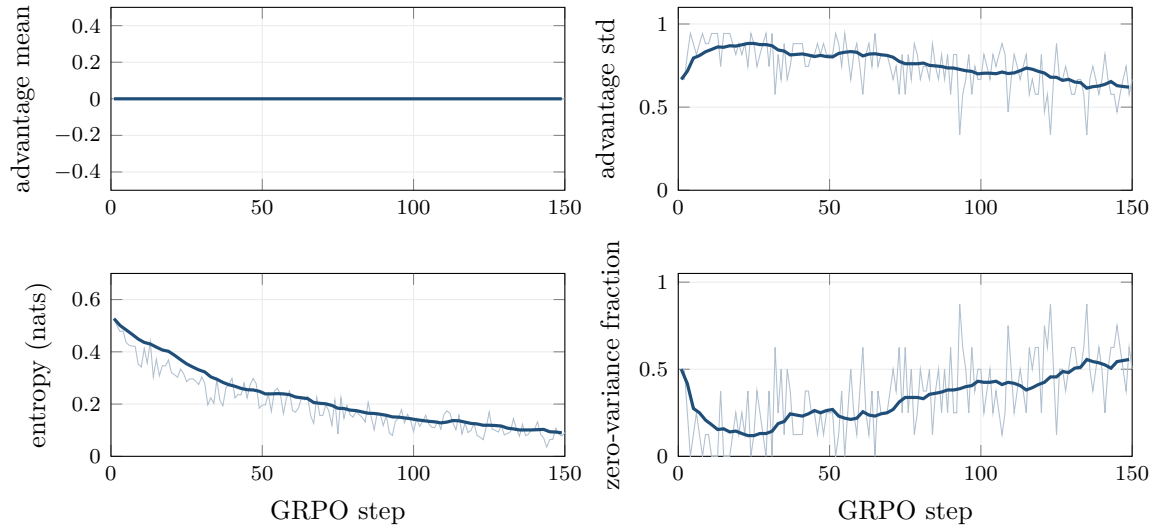


Figure 18.4: Advanced metrics of the baseline (Chapter 17) GRPO run on the toy task, seed 0, measured on CPU. The advantage mean is zero by construction (a free correctness check). The advantage std and the zero-variance fraction mirror each other: as more prompts are solved by all 8 rollouts, fewer groups carry signal. Entropy falls as the policy sharpens.

18.4 Stabilising updates with clipped policy ratios

RM 7.4

18.4.1 Where the ratio comes from

RM 7.4.1

The policy-gradient estimator of Eq. (17.1) assumes the rollouts were sampled from the *current* policy π_θ. After one optimiser step that is no longer true: the rollouts came from the snapshot $\pi_{\theta_{\text{old}}}$ that generated them. If we want to reuse a batch of expensive rollouts for several updates, or if the sampler runs a few steps behind the trainer (Chapter 19), we must correct for the mismatch. Importance sampling does this:

$$\mathbb{E}_{y\sim\pi_\theta}[A\,\nabla \log \pi_\theta(y)] = \mathbb{E}_{y\sim\pi_{\theta_{\text{old}}}}\Big[\frac{\pi_\theta(y)}{\pi_{\theta_{\text{old}}}(y)} A\,\nabla \log \pi_\theta(y)\Big] = \mathbb{E}_{y\sim\pi_{\theta_{\text{old}}}}[A\,\nabla \rho(y)], \qquad \rho = \frac{\pi_\theta(y)}{\pi_{\theta_{\text{old}}}(y)}, \tag{18.2}$$

so maximising the *surrogate* $\mathbb{E}_{\pi_{\theta_{\text{old}}}}[\rho A]$ has the right gradient. The ratio equals 1 on the first update after sampling (which is why the summed-log-prob loss of Chapter 17 gives the same gradient) and drifts away from 1 with every further update on the same data. Unchecked, the surrogate can be pushed arbitrarily high by making ρ huge on positive-advantage samples. The

estimate is then dominated by a few samples and the policy leaves the region where the old samples say anything about it.

PPO's answer is to clip the ratio to a trust region $[1-\varepsilon, 1+\varepsilon]$ and take the pessimistic minimum:

$$\ell^{\text{clip}}(\rho, A) = \min\left(\rho A,\ \text{clip}(\rho, 1-\varepsilon, 1+\varepsilon)A\right). \tag{18.3}$$

Figure 18.5 shows what the minimum does. For a good sample ($A > 0$) the objective stops increasing once $\rho > 1+\varepsilon$: there is no reward for making it *even* more likely in this update, so its gradient is zero. For a bad sample ($A < 0$) the objective stops improving once $\rho < 1-\varepsilon$. In the opposite directions, a good sample whose probability has *dropped* or a bad sample whose probability has *risen*, the unclipped term is the smaller one and the gradient stays on to correct the mistake.

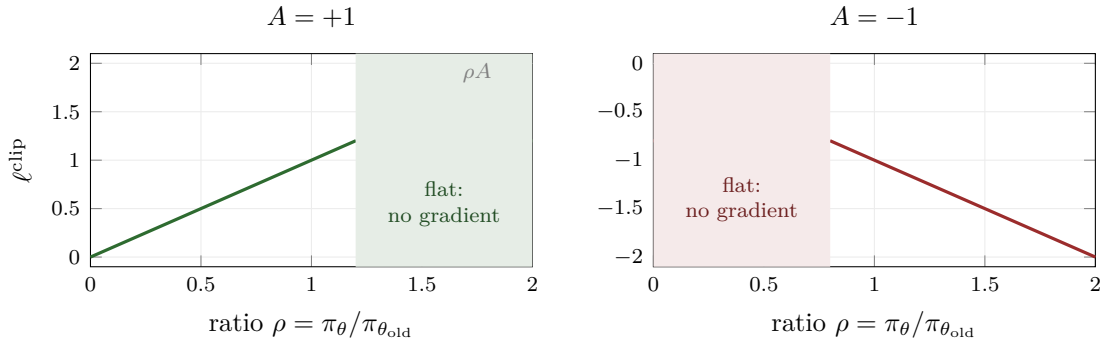


Figure 18.5: The clipped surrogate of Eq. (18.3) with $\varepsilon = 0.2$ (solid) versus the unclipped ρA (dashed). Shaded regions are where the clip binds and the sample's gradient is zero: moving further in the direction the advantage wants brings no further credit within this update. Moving the wrong way is never clipped.

A worked example on the four rollouts of Fig. 17.10 makes this concrete. Suppose that after some updates the current policy's sequence log-probabilities are $(-2.269, -3.059, -1.299, -3.400)$, while the snapshot that sampled them had $(-2.469, -2.659, -1.299, -3.100)$. Then

rollout	A_i	$\rho_i = e^{\Delta \log \pi}$	$\text{clip}(\rho_i, 0.8, 1.2)$	$\min(\rho A, \text{clip}(\rho)A)$
`\boxed{7}`	$+0.866$	1.221	1.200	1.039 (clipped)
`<think>..\boxed{7}`	$+0.866$	0.670	0.800	0.581 (unclipped)
`7`	-0.866	1.000	1.000	-0.866
`\boxed{8}`	-0.866	0.741	0.800	-0.693 (clipped)

The clipped loss is $-\frac{1}{4}\sum \ell_i^{\text{clip}} = -0.0152$, against -0.0327 unclipped. Two rollouts no longer contribute gradient. The first boxed answer has already been made 22% more likely, and the wrong boxed answer 26% less likely, so both are "done" for this batch. The second correct answer became *less* likely, an error the unclipped branch keeps correcting.

Listing 18.5: Policy ratios and the clipped loss (sequence level, as in this section).

```
def clipped_policy_loss(new_logps, old_logps, advantages,
                        eps_low=0.2, eps_high=0.2):
    ratio = torch.exp(new_logps - old_logps.detach())
    adv = advantages.detach()
    unclipped = ratio * adv
    clipped = torch.clamp(ratio, 1 - eps_low, 1 + eps_high) * adv
    obj = torch.minimum(unclipped, clipped)
    clip_frac = (clipped < unclipped).float().mean().item()  # clip binds
```

```
    return -obj.mean(), ratio.mean().item(), clip_frac
```

Choosing ε. Classic PPO for RLHF uses $\varepsilon \in [0.1, 0.2]$, which is aggressive clipping and small per-update changes. Reasoning-RL recipes span a wide range, and some large runs reportedly used very loose clipping, where ε is so large that only extreme ratios are touched. The right value depends on how many times each rollout is reused and how stale the sampler is. Two refinements are common. *Asymmetric* clipping ($\varepsilon_{\text{low}} < \varepsilon_{\text{high}}$, DAPO's "clip-higher") lets rare good tokens grow faster than common ones shrink, which counteracts entropy collapse. *Sequence-level* versus *token-level* ratios is the other choice: this section's listing clips the ratio of whole-sequence probabilities, while the GRPO paper clips every token's ratio $\rho_{i,t}$ separately (Eq. (17.5)). Token-level ratios are numerically tamer, because a sequence ratio is a product of thousands of per-token ratios. GSPO (Section 18.6.3) goes back to the sequence level but takes a geometric mean.

Key Idea: log the ratio, not just the loss

Two cheap diagnostics come with clipping: the mean ratio (should hover near 1; persistent drift means the policy is moving faster than the batch can support) and the *clip fraction* (fraction of tokens where the clip binds). A clip fraction near 0 means clipping is idle and ε or the reuse can be increased. One above 0.2–0.3 means most of the batch is being ignored and updates are too aggressive.

18.4.2 Training with clipped ratios: three snapshots of one policy

RM 7.4.2

Clipping only does something if the same rollouts are seen by more than one optimiser step, so implementing it forces an explicit choice about *which version* of the policy each quantity comes from. A GRPO implementation juggles three snapshots (Fig. 18.6):

- π_θ, the live weights, changed by every optimiser step;
- $\pi_{\theta_{\text{old}}}$, the weights that generated the current rollout batch, refreshed every time a new batch is sampled. Its log-probabilities are computed once per batch under `no_grad` and stored as numbers, not as a model copy;
- π_{ref}, the frozen anchor for the KL term (Section 18.5), usually the initial model, refreshed rarely if ever.

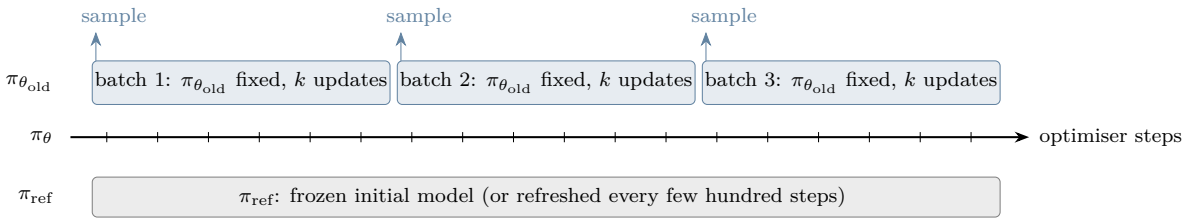


Figure 18.6: Three snapshots of one policy. Each rollout batch fixes $\pi_{\theta_{\text{old}}}$ (stored as log-probabilities). The live π_θ takes several minibatch steps on that batch, and the ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$ measures how far it has moved; clipping bounds that. π_{ref} anchors the whole run through the KL term.

DeepSeek-R1-style training samples one large rollout pool per iteration (thousands of completions), then walks through it in many minibatches, each an optimiser step, before sampling

again. Later minibatches therefore see ratios well away from 1, and clipping does real work. Our trainer exposes the same structure through two knobs of `GRPOConfig`: `minibatches` (how many optimiser steps one batch is split into) and `inner_epochs` (how many passes over the batch). With `inner_epochs=1` and `minibatches=1`, every ratio is exactly 1, the clipped loss reduces to Chapter 17's gradient, and the clip never binds. (In that setting the logged loss is also exactly zero, a computational artefact, because $\rho \equiv 1$ and the advantages are mean-zero.)

Systems Lens: rollout reuse is a throughput knob

Reusing each rollout batch for k optimiser steps divides the sampling cost per step by k. Since sampling dominates (Section 17.11), $k = 2$–4 can nearly halve wall-clock time per update. The price is off-policyness, and clipping is what makes it affordable. The same mathematics covers *asynchronous* RL, where the sampler generates batch $n{+}1$ with weights from step $n - s$ while the trainer is at step n. To the loss, a stale sampler looks exactly like rollout reuse, and the ratio and clip absorb it, up to a point (Chapter 19).

What the toy shows. We ran two reuse settings from the same SFT start (Fig. 18.7). With *moderate* reuse, four full passes over each rollout batch at the base learning rate, the unclipped run stalled at a reward near 0.5 and ended with greedy accuracy 0.62 and 0.50 (two seeds). The clipped run ($\varepsilon = 0.2$) matched the no-reuse baseline: 0.88 and 0.95, against 0.89 and 0.96. Its clip fraction was only 2–3%. A few runaway tokens per batch are enough to derail the unclipped run, and these are precisely the ones clipping silences. With *aggressive* reuse, eight optimiser steps per batch at three times the learning rate, *both* runs collapsed: entropy fell to 0.01–0.06, every group became uniformly wrong (zero-variance fraction ≈ 1), and greedy accuracy ended at 0.07–0.16. The lesson is about what clipping does and does not bound. It bounds the drift *within* one batch relative to $\pi_{\theta_{\text{old}}}$. Every new batch opens a new trust region, so clipping is no substitute for a sensible learning rate.

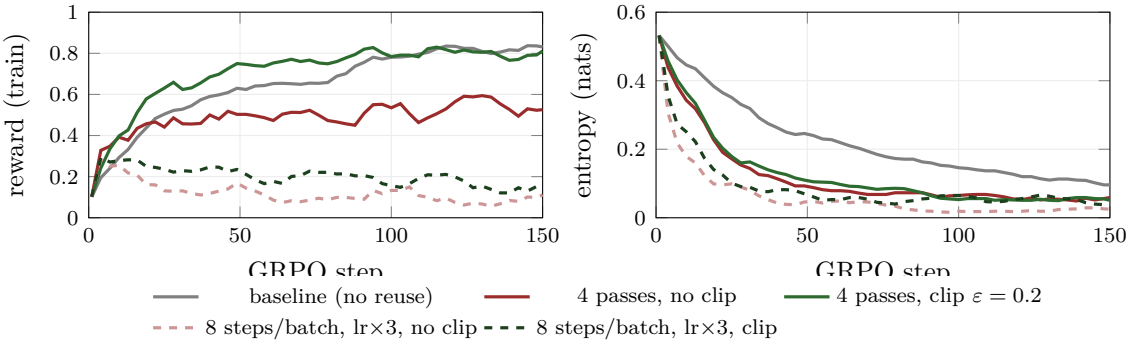


Figure 18.7: Rollout reuse with and without clipping on the toy task (mean of two seeds, 15-step moving average, measured on CPU). With four passes over each batch, the unclipped run stalls near a reward of 0.5 while the clipped run tracks the baseline. With eight optimiser steps per batch at three times the learning rate, both runs collapse: entropy falls towards zero and every group becomes uniformly wrong. Clipping bounds the drift *within* a batch, not the step size across batches.

18.5 Controlling drift with a KL penalty

RM 7.5

If you know classical RL you will recognise what we have so far. The update of Chapter 17 is REINFORCE with a group baseline, and with the ratio and clip it becomes PPO without a critic. The last ingredient of the original GRPO objective is a penalty on the Kullback–Leibler

divergence between the current policy and a fixed reference policy, usually the model we started from:

$$\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}) = \mathbb{E}_{y\sim\pi_\theta}\Big[\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}\Big] \;\geq 0. \tag{18.4}$$

Clipping and KL are complementary leashes. Clipping limits how far the policy moves *within one batch*, relative to the snapshot that sampled it. The KL term limits how far it moves *over the whole run*, relative to where it started. The motivation in RLHF was protection against reward hacking, keeping the policy in the region where the reward model is trustworthy. With verifiable rewards there is no reward model to fool, which is one reason recent work questions the term (below).

18.5.1 Implementing the KL term

RM 7.5.1

We can only evaluate the KL on the samples we have, so we need a Monte Carlo *estimator*. Writing $r = \pi_{\text{ref}}(y_t)/\pi_\theta(y_t)$ for a sampled token, three estimators are in common use (Fig. 18.8):

$$k_1 = -\log r, \qquad k_2 = \tfrac{1}{2}(\log r)^2, \qquad k_3 = r - 1 - \log r. \tag{18.5}$$

k_1 is unbiased (its expectation under π_θ is exactly the KL) but has high variance and is negative for half the samples. k_2 is always non-negative and low-variance but biased. k_3, popularised by John Schulman's note on KL approximation and adopted by GRPO, is unbiased (since $\mathbb{E}_{\pi_\theta}[r] = 1$) *and* always non-negative, because $r - 1 \geq \log r$. GRPO applies k_3 per token, averages it like the policy term, and adds $\beta\,\mathbb{D}_{i,t}$ to the per-token loss with β around 0.001–0.05.

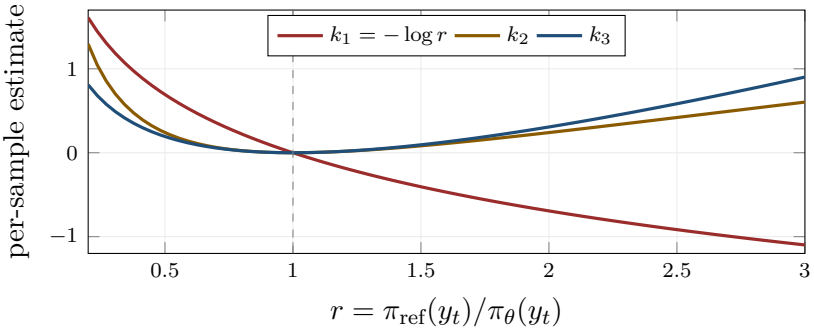


Figure 18.8: Three per-sample KL estimators as functions of the likelihood ratio. All are zero when policy and reference agree ($r = 1$). k_1 goes negative whenever the policy is more confident than the reference; k_3 is non-negative and has zero slope at $r = 1$, so small deviations are penalised gently.

The source book's first implementation, and our `kl_kind="seq_simple"`, is the simplest possible *sequence-level* surrogate: $\beta \cdot \mathrm{mean}_i\,(\log \pi_\theta(y_i) - \log \pi_{\text{ref}}(y_i))$ with summed log-probabilities, i.e. k_1 summed over the sequence. It needs one extra forward pass of a frozen model copy per batch:

Listing 18.6: Adding a KL term (sequence surrogate and the importance-weighted variant).

```
ref_model = copy.deepcopy(model).eval()    # frozen anchor: +1 weight copy
for p in ref_model.parameters():
    p.requires_grad_(False)
...
with torch.no_grad():
    ref_logp = sequence_logprob(ref_model, ids, prompt_len)  # per rollout
...
pg_loss = -(adv.detach() * logps).mean()
```

```
kl_loss = kl_coeff * (logps - ref_logps).mean()      # simple surrogate
# importance-weighted (rollouts came from pi_old, not pi_theta):
w = torch.exp(logps - old_logps).detach()
kl_loss_is = kl_coeff * (w * (logps - ref_logps)).mean()
loss = pg_loss + kl_loss
```

Napkin Math: what the reference model costs

Memory: one more bf16 copy of the weights (1.2 GB for 0.6B, 16 GB for 8B, 140 GB for 70B). Compute: one more forward pass over every rollout, $2N$ FLOPs per token, about one third of a training step's $6N$. For large models, reference log-probs are often computed by the *rollout* workers or a dedicated reference worker (Chapter 19). With LoRA training (Chapter 10) the reference is free: it is the same weights with the adapter disabled.

18.5.2 Training with a KL term

RM 7.5.2

The source book reports an instructive failure when this simple surrogate is added to a long GRPO run on Qwen3-0.6B ($\beta = 0.02$). Accuracy first improves as usual. Then the loss explodes, rewards collapse to zero, MATH-500 accuracy follows, and inspecting the samples shows random code-like token salad. Understanding *why* is more useful than the fix:

1. **Wrong distribution.** The KL is an expectation under π_θ, but the rollouts were sampled from $\pi_{\theta_{\text{old}}}$. On those fixed samples, with $\log \pi_{\text{ref}}$ a constant, minimising $\log \pi_\theta(y) - \log \pi_{\text{ref}}(y)$ simply means making the sampled sequences *less likely*. That is not "stay close to the reference". It is "move away from whatever you just did".

2. **No length normalisation.** Summed log-probabilities make the term scale with response length, so long rollouts dominate it.

3. **It is the last gradient standing.** Once rewards collapse (all groups wrong), every advantage is zero and the policy term vanishes. The KL surrogate is then the *only* source of gradient, and it keeps pushing probability away from sampled tokens, which flattens the distributions and raises entropy. That feeds more garbage into the next batch, a self-reinforcing collapse.

Several remedies follow from that diagnosis: a much smaller β (not enough on its own, per the source book's experiments); length-normalised (per-token) log-probabilities; the unbiased k_3 estimator computed per token; the importance-weighted form of DeepSeek-V3.2, $\beta\, \mathbb{E}_{\pi_{\theta_{\text{old}}}}[\rho \cdot (\log \pi_\theta - \log \pi_{\text{ref}})]$, which corrects for sampling from $\pi_{\theta_{\text{old}}}$; or a small auxiliary reward (Section 18.6) so the policy term never vanishes entirely. Or remove the term. For math RLVR, Dr. GRPO, the OLMo 3 team and DeepSeek-V3.2 all report equal or better results with the KL weight at or near zero, and dropping it also removes the reference model's memory and compute.

What the toy shows. Neither KL variant destabilised our short runs. With $\beta = 0.02$, the sequence surrogate reached greedy accuracy 0.93 and 0.98 and per-token k_3 (inside the token-level clipped loss) 0.91 and 0.97, both statistically indistinguishable from the baseline's 0.89 and 0.96. That is consistent with the diagnosis above. The surrogate is only dangerous once rewards collapse and it becomes the sole gradient, and on the toy task rewards rise monotonically,

so mechanism (3) never fires. Exercise 18.5 isolates the mechanism directly by training on a batch with zero advantages. Two systems observations from these runs: the reference forward pass doubled the measured no-grad log-prob phase (from about 0.3 to 0.6 times the update phase), roughly 13% of a whole step on this workload, close to the one-third-of-a-training-step estimate above; and the logged KL values differ by a factor of 4–5 between estimators (about 2.5 nats per *sequence* for the summed surrogate, 0.5–0.6 nats per *token* for k_3), so a β tuned for one estimator means something very different for another.

18.6 Adding an explicit format reward

RM 7.6

So far the only reward has been correctness, with the `\boxed{}` requirement as an implicit format check. That is sufficient for training reasoning models. Most recipes also add *auxiliary* rewards: a format reward, a length penalty, a language-consistency reward (DeepSeek-R1 penalised mixing languages within a chain of thought). The canonical format reward asks the model to put its reasoning between `<think>` and `</think>` and the final answer after it.

18.6.1 Thinking tokens

RM 7.6.1

`<think>` and `</think>` are ordinary vocabulary entries that delimit the reasoning. They are useful because they let a serving system hide, truncate, or budget the reasoning separately from the answer, and because evaluation can parse the answer reliably. They only work well as *single* tokens. The Qwen3 *base* tokenizer splits `<think>` into three sub-word pieces, while the Qwen3 reasoning tokenizer maps `<think>` and `</think>` to the dedicated ids 151,667 and 151,668. Like most modern tokenizers, the base vocabulary reserves unused slots: its embedding matrix has 151,936 rows, more than the number of tokens the base tokenizer emits, so the special tokens can be registered with the tokenizer (`add_special_tokens`) without resizing the model. Their embeddings, however, are untrained in the base model. A base model has never seen these ids and will not produce them reliably until some SFT or the reasoning variant's training has taught it to. Our toy tokenizer (`ToyTokenizer`) has `<think>` and `</think>` as single tokens from the start, and its SFT data uses them in 20% of demonstrations.

The reward itself is a two-line check on token ids: 1 if `<think>` occurs and is followed later by `</think>`, else 0.

Listing 18.7: A token-level format reward and its combination with correctness.

```python
def reward_format(token_ids, prompt_len, start_id, end_id):
    gen = token_ids[prompt_len:].tolist()
    try:
        return float(gen.index(start_id) < gen.index(end_id))
    except ValueError:                          # one of the tags is missing
        return 0.0

def total_reward(correct, fmt, weight=1.0, conditional=False):
    if conditional and correct <= 0:
        return correct                          # no format credit for wrong answers
    return correct + weight * fmt
```

The prompt should also ask for the format, for example: "Write your reasoning between `<think>` and `</think>`, then give the final result as `\boxed{ANSWER}`."

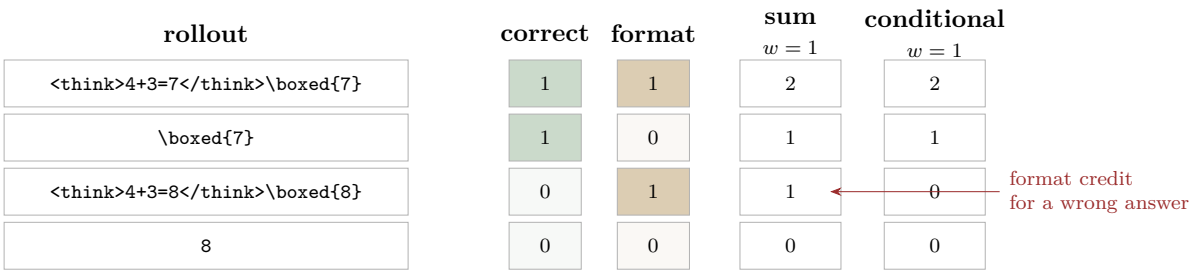


Figure 18.9: Combining a correctness reward with a format reward on four rollouts of the toy prompt. Plain summation gives the wrong-but-well-formatted third rollout the same total as the correct-but-unformatted second one, so their advantages are equal. Conditioning the format credit on correctness removes that incentive.

18.6.2 Training a model to emit thinking tokens

RM 7.6.2

Two practical caveats come first. You cannot reasonably RL-train a model to emit tokens it has never produced. If the probability of `<think>` is 10^{-9}, no rollout will contain it, the format reward is always 0 and its gradient is always zero. Format rewards therefore *reinforce* a behaviour the model already has at some rate, from SFT or from its reasoning variant's training. The source book accordingly applies its format-reward experiment to the Qwen3 reasoning variant, not the base model.

Second, every auxiliary reward is a new way to be rewarded without being right. With weight 1, a well-formatted wrong answer earns as much as an unformatted right one (Fig. 18.9). In the source book's run on the reasoning variant, the format reward climbed to near-saturation while MATH-500 accuracy fell and responses got shorter: the model learned that the tags were the cheapest reward on offer. Three fixes are standard:

- **Down-weight** the auxiliary reward (e.g. $w = 0.1$) so it breaks ties without competing with correctness.
- **Make it conditional**: format credit only for correct answers.
- **Normalise each component separately** (GDPO, Section 18.6.3): standardise correctness and format within the group *before* combining them, so a high-variance format signal cannot drown a low-variance correctness signal in the shared advantage.

Note also an artefact visible in such runs: with one inner update and no KL, every ratio is 1 and the advantages average to zero, so the logged scalar loss is identically zero. Monitor `reward_avg`, the per-component averages, `adv_std` and entropy instead.

What the toy shows. Because 20% of the SFT demonstrations already used `<think>` tags, every format variant drove `format_avg` to about 0.99 within roughly 30 steps, and responses grew from about 4.5 to 12 tokens to carry the scratchpad. The interesting quantity is what happened to *correctness*. With weight 1 the late-run correct rate was 0.72–0.76 (greedy accuracy 0.84–0.87), below the baseline's 0.82–0.83 (0.89–0.96). Weight 0.1 was no better (0.74, greedy 0.83–0.84): even a small weight breaks ties between otherwise equal rollouts, and on this task the tags are much easier to learn than arithmetic. Conditioning the format credit on correctness recovered the baseline (correct rate 0.80–0.86, greedy accuracy 0.91–0.96). GDPO's per-component normalisation landed in between (0.75, greedy 0.87–0.91). Its logged "reward"

is identically zero, because the combined signal is a sum of z-scores, a reminder to log raw components alongside whatever the optimiser sees.

18.6.3 More GRPO modifications

RM 7.6.3

We now have the original GRPO objective in full: clipped ratios, a KL term and composite rewards. Since DeepSeek-R1, a stream of papers has modified almost every term of Eq. (17.5). The changes are easiest to keep straight by asking *which part of the objective* each one touches (Fig. 18.10). In the list below, the parentheses name the work that proposed or popularised each change, and the code column of Table 18.1 points to its implementation in `llmsys.rl`.

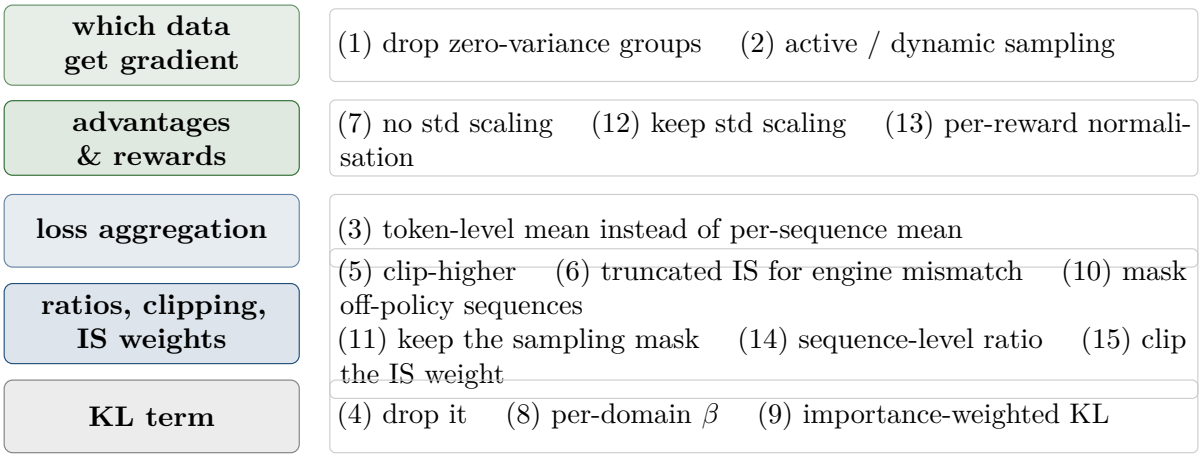


Figure 18.10: Fifteen post-R1 modifications of GRPO, grouped by the part of the objective they change. Numbers refer to the list in the text.

1. **Filter zero-gradient groups** (DAPO). Drop prompts whose G rollouts all received the same reward. They contribute nothing to the gradient but still dilute averages and occupy the batch.

2. **Active (dynamic) sampling** (DAPO). Instead of merely dropping such prompts, oversample and keep drawing until the batch is full of mixed groups, so every step has a full batch of signal.

3. **Token-level loss** (DAPO). Average the per-token loss over *all tokens in the batch* rather than first within each sequence and then over sequences. With per-sequence averaging each token of a 2000-token rollout weighs 100 times less than a token of a 20-token rollout, so long wrong answers are under-penalised (and long right ones under-rewarded).

4. **Drop the KL term** (DAPO, Dr. GRPO). For verifiable rewards the anchor mostly slows learning, and dropping it saves a model copy.

5. **Clip-higher** (DAPO). Use $\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$ (e.g. 0.28 vs. 0.2) so low-probability tokens with positive advantage can grow faster, which counteracts entropy collapse.

6. **Truncated importance sampling** (verl). Rollout engines (vLLM, SGLang) compute probabilities with different kernels, precision and batching than the trainer, so even "on-policy" samples come from a slightly different distribution $\pi_{\text{rollout}} \neq \pi_{\theta_{\text{old}}}$. Weight each token by $\min(\pi_{\theta_{\text{old}}}/\pi_{\text{rollout}}, C)$ to correct for it without letting rare tokens explode the update.

7. **No std normalisation of advantages** (Dr. GRPO). Dividing by the group std upweights prompts that are nearly always solved or nearly always failed (small std), a difficulty bias. Dr. GRPO uses $A_i = r_i - \bar{r}$ and also replaces the per-sequence length normalisation by a constant, removing a length bias.

8. **Per-domain KL strength** (DeepSeek-V3.2). Keep the KL for domains where drift is harmful and set $\beta = 0$ for math.

9. **Importance-weighted KL** (DeepSeek-V3.2). Multiply the KL estimate by $\pi_\theta / \pi_{\theta_{\text{old}}}$ because the samples come from $\pi_{\theta_{\text{old}}}$. This is exactly the bias diagnosed in Section 18.5.2.

10. **Mask badly off-policy sequences** (DeepSeek-V3.2). Exclude negative-advantage sequences whose average $\log \pi_{\theta_{\text{old}}} - \log \pi_\theta$ exceeds a threshold. Pushing down samples the current policy would hardly generate anyway is mostly noise.

11. **Keep the sampling mask** (DeepSeek-V3.2). If rollouts used top-p/top-k, compute $\pi_{\theta_{\text{old}}}$ and π_θ over the *same truncated support*, so the ratio compares like with like.

12. **Keep the original advantage normalisation** (DeepSeek-V3.2). A counterpoint to (7): standard group-normalised advantages worked well in their setting, so ablate before removing.

13. **Normalise each reward component before aggregation** (GDPO). Standardise each reward type within the group, then combine, so components with different scales and variances contribute comparably.

14. **Sequence-level ratio and clipping** (GSPO). Use the length-normalised sequence ratio $s_i = \exp\left(\frac{1}{|y_i|} \sum_t \log \rho_{i,t}\right)$ with a tight clip (around $3\text{–}4 \times 10^{-4}$). This matches the sequence-level reward and is reported to be more stable, notably for MoE models where token-level ratios are noisy.

15. **Clip importance weights, not updates** (CISPO, MiniMax-M1). Use $-\,\text{sg}(\min(\rho_{i,t}, 1+\varepsilon))\, A_i \log \pi_\theta(y_{i,t})$: every token keeps a gradient (PPO-clip silences clipped tokens entirely, which can drop rare but important "reflection" tokens), but its weight is bounded.

#	One-line summary	`GRPOConfig` / function
1, 2	spend gradient only on mixed groups	`filter_zero_std`
3	mean over all tokens, not per sequence	`agg="token_mean"`
4, 8	KL optional / per domain	`kl_coeff=0`
5	asymmetric clip	`eps_low`, `eps_high`
6	engine/trainer mismatch weights	`truncated_is_weights`
7	centre but don't scale	`adv_scale="none"`, `agg="seq_mean_token_sum_norm"`
9	IS-weighted KL	`kl_is_weighted=True`
10	drop very off-policy negatives	`offpolicy_delta`
11	same support for both policies	`top_p_renormalized_logprob`
13	per-component normalisation	`gdpo=True`
14	geometric-mean sequence ratio	`loss="gspo"`
15	bounded, gradient-stopped IS weight	`loss="cispo"`

Table 18.1: Where each modification lives in the companion code.

Listing 18.8: Token-level clipped GRPO loss with the main options (abridged from `llmsys/rl/losses.py`).

```python
def token_clipped_loss(lp_new, lp_old, adv, mask, eps_low=0.2, eps_high=0.2,
                       lp_ref=None, kl_coeff=0.0, kl_kind="k3",
                       kl_is_weighted=False, agg="seq_mean_token_mean"):
    ratio = torch.exp((lp_new - lp_old.detach()) * mask)        # (N, T)
    a = adv.detach()[:, None]
    obj = torch.minimum(ratio * a,
                        torch.clamp(ratio, 1 - eps_low, 1 + eps_high) * a)
    per_tok = -obj
    if lp_ref is not None and kl_coeff > 0:
        kl = kl_estimate(lp_new, lp_ref, kl_kind)               # k1 / k2 / k3
        if kl_is_weighted:                                      # DeepSeek-V3.2
            kl = kl * ratio.detach()
        per_tok = per_tok + kl_coeff * kl
    return aggregate(per_tok, mask, agg)    # GRPO / DAPO / Dr. GRPO reductions
```

What the toy shows. Figure 18.11 collects every variant. The four descendants we ran with the trainer's defaults (DAPO-style token-level loss with clip-higher and zero-variance filtering, Dr. GRPO, GSPO and CISPO) all landed within noise of the baseline: greedy accuracy 0.91–0.96 against 0.93. That is the expected outcome, not a disappointment. These modifications target failure modes that a 76k-parameter model on five-token answers never reaches: long-sequence length bias, entropy collapse over thousands of steps, MoE routing noise, and stale samplers. Their value shows up in long runs on real models, which is where to evaluate them (Lab 18's stretch goal). The toy's job here is to verify that each implementation is *neutral when it should be*. A modification that hurt on a problem this easy would be a bug.

18.7 The systems cost of each knob

Algorithm papers report accuracy. Whoever runs the job also has to pay for it. Table 18.2 summarises how each mechanism of this chapter changes memory, compute and scheduling. Most of the changes have a positive effect.

Napkin Math: the price of dynamic sampling

Suppose half of a math dataset's prompts are solved with $p \approx 0.9$ by the current policy and half with $p \approx 0.3$. With $G = 16$, zero-variance probabilities are $0.9^{16} + 0.1^{16} \approx 0.185$ and $0.3^{16} + 0.7^{16} \approx 0.003$, so about 9.4% of groups are wasted. To fill a batch of 512 mixed groups, dynamic sampling must generate about $512/0.906 \approx 565$ groups, or $\approx 9{,}000$ rollouts. As the policy improves, the easy half drifts to $p \approx 0.97$ ($0.97^{16} \approx 0.61$) and the waste rises to about 30%, so the over-sampling factor grows to 1.45. Curricula that retire mastered prompts keep this factor near 1.

18.8 Summary

- Read a GRPO run through several metrics together: moving-average reward, response length, held-out accuracy, advantage mean (sanity) and std (signal), zero-variance fraction, entropy, ratio and clip fraction. The loss is mainly a sanity check.
- Reward stuck at 1 or 0 means no signal. Entropy collapse means no exploration. Entropy explosion means a damaged model.

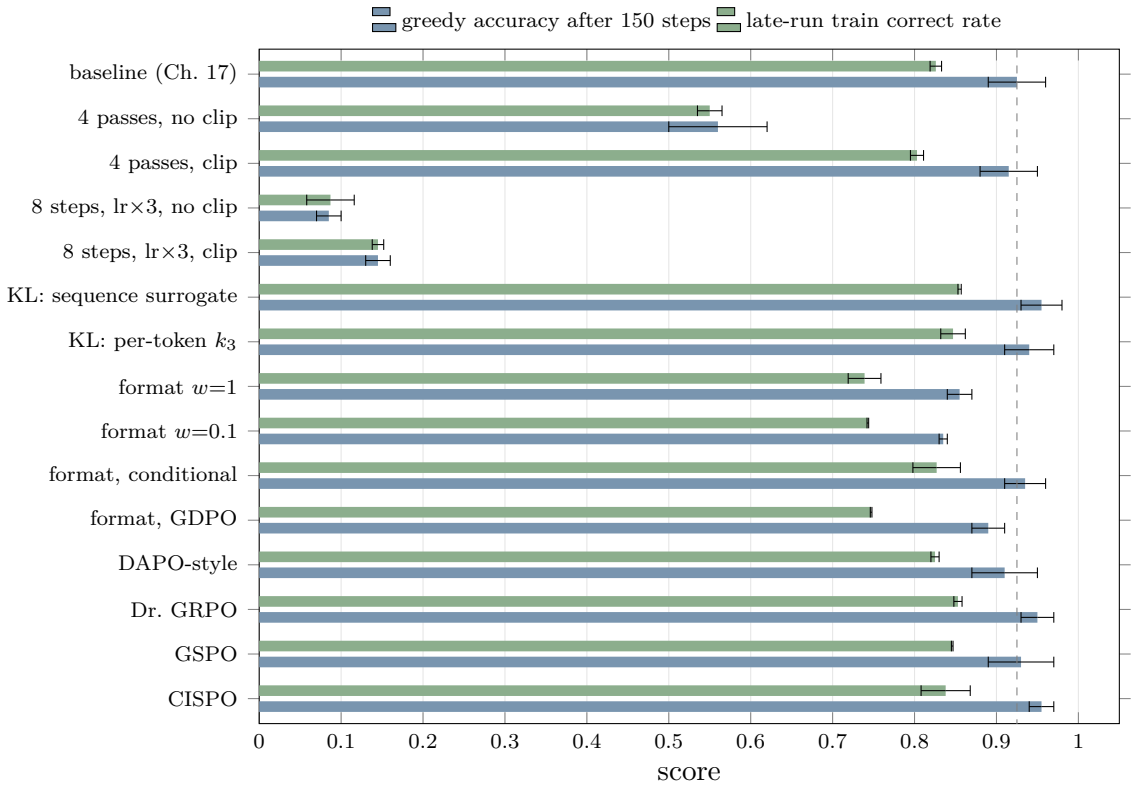


Figure 18.11: Every GRPO variant of this chapter on the toy task: mean over two seeds, with the bars' whiskers spanning the two seeds; 150 steps from the same SFT start; measured on CPU. Dashed line: baseline mean greedy accuracy. With only two seeds, differences under about 0.05 are noise. Clear effects: unclipped rollout reuse hurts, clipping fixes moderate reuse but not an excessive step size, and an unconditional format reward costs correctness while a conditional one does not.

- Importance ratios correct for training on samples from an older snapshot, and clipping keeps each batch's updates inside a trust region. Together they make rollout reuse and asynchronous sampling safe, which is a throughput win.

- The KL penalty anchors the run to a reference policy. A naive sampled surrogate can destabilise training once rewards collapse. Per-token k_3, importance weighting, or simply $\beta = 0$ (common for math) avoid it.

- Auxiliary rewards such as `<think>` formatting are cheap but hackable. Down-weight them, condition them on correctness, or normalise components separately (GDPO).

- Post-R1 variants each change one term: what data gets gradient (DAPO filtering and dynamic sampling), advantage scaling (Dr. GRPO), loss aggregation (token-level), ratio and clipping (clip-higher, GSPO, CISPO, truncated IS), and KL handling (DeepSeek-V3.2).

- Every knob has a systems cost. The cheapest stabiliser, clipping, is also the one that unlocks most of the throughput.

Mechanism	Cost / effect on the system
entropy metric	full-vocabulary reduction; compute in chunks on the no-grad pass, otherwise $(NT) \times \lvert\mathcal{V}\rvert$ fp32 memory
clipping + rollout reuse	*saves* sampling: k updates per batch divide sampling cost per update by k; enables asynchronous samplers
KL to reference	+1 model copy in memory, +1 forward ($\approx 33\%$ of a training step); free with LoRA (disable adapter)
dropping KL	removes both of the above
format/aux rewards	negligible (string or id checks on CPU); code-execution rewards need a sandbox fleet
zero-variance filtering	wasted rollouts: a fraction $p^G + (1-p)^G$ of sampling is discarded
dynamic sampling	over-samples by $\approx 1/(1 - \text{waste})$ and makes step time *variable*, which complicates synchronous pipelines
token-level aggregation	none (a different reduction); changes effective learning rate per token, so retune lr
truncated IS (engine mismatch)	needs the sampler to return per-token log-probs (cheap) and the trainer to recompute them (already done)
sampling-mask preservation	needs the top-p support per token, either stored (huge) or recomputed from the trainer's own logits (approximate)
GSPO / CISPO	none beyond the base loss; GSPO's sequence ratio needs all tokens of a sequence in one micro-batch

Table 18.2: What the stabilisation mechanisms cost. "Saves" entries are why clipping-based off-policy training underlies essentially all high-throughput RL systems (Chapter 19).

18.9 Exercises

Exercise 18.1 (*testing the format reward*). Run `reward_format` on a completion with both tags in order and confirm it returns 1. Then introduce a typo in one tag, reverse their order, and delete one tag; each should give 0. Which of these edits would a *string*-based check (`think_format_ok`) also catch? Which tokenisation edge case can break the id-based check but not the string-based one?

Exercise 18.2 (*conditional format reward*). Run the four format variants (`format_w1`, `format_w0.1`, `format_cond`, `format_gdpo`) with seeds 0, 1 and 2. Report final greedy accuracy, late-run `correct_avg` and `format_avg`. Does conditioning change which behaviour the policy adopts first? Explain the result with the advantages of Fig. 18.9.

Exercise 18.3 (*the clip fraction*). For the `clip_reuse4` variant, plot `clip_frac` and `policy_ratio` over training. Vary `inner_epochs` from 1 to 8 and find the value where the clip fraction exceeds 0.2. Relate this to the wall-clock saving of rollout reuse.

Exercise 18.4 (*unbiasedness of k_3*). Show that $\mathbb{E}_{y\sim p}[k_3] = \mathrm{KL}(p \,\|\, q)$ where $k_3 = r - 1 - \log r$ and $r = q(y)/p(y)$, and that $k_3 \geq 0$ pointwise. Verify numerically for a random pair of categoricals over 1000 outcomes, and compare the empirical variance of k_1, k_2 and k_3 with 64 samples.

Exercise 18.5 (*diagnose the KL collapse*). On a fixed batch where all advantages are zero, take 50 optimiser steps on the sequence-level KL surrogate alone. Plot the mean entropy and

the probability of the sampled tokens. Repeat with per-token k_3. Explain the difference in terms of what each loss's gradient does to the sampled tokens.

Exercise 18.6 (*aggregation and length*). Construct a group of two rollouts with $A = (+1, -1)$ and lengths 10 and 1000. Compute, under each of the three aggregations of `aggregate_token_loss`, the total gradient weight received by each rollout and by each of its tokens. Which rule lets a long wrong answer off lightly?

Exercise 18.7 (*Dr. GRPO's difficulty bias*). For binary rewards and $G = 8$, compute the gradient-weight multiplier $\sum_i |A_i|$ as a function of p with and without std scaling. Which prompts does std scaling up-weight? Argue whether that is a feature (focus on edge cases) or a bug.

Exercise 18.8 (*chunked entropy*). Measure peak memory of `token_entropy` on a $(4096, 151{,}936)$ bf16 logits tensor versus `chunked_logprob_entropy` with chunk sizes 256 and 1024 (use `torch.cuda.max_memory_allocated` or `tracemalloc`). Plot memory and time against chunk size.

Exercise 18.9 (*napkin: reference-model placement*). For a 32B policy trained with FSDP on 16 GPUs, compare three placements of the KL reference: (a) a full bf16 copy on every trainer GPU, (b) FSDP-sharded alongside the policy, (c) computed by the rollout workers. Estimate memory per GPU and extra communication per step for each.

Exercise 18.10 (*engine mismatch*). Simulate an inference engine by sampling rollouts from the toy model with its weights cast to bf16 while the trainer uses fp32. Measure the distribution of $\log \pi_{\theta_{\text{old}}} - \log \pi_{\text{rollout}}$ per token. Train with and without `truncated_is_weights` and compare stability.

18.10 Lab 18: making GRPO robust

Lab 18: Diagnostics, clipping, KL, format rewards and GRPO's descendants

Code: `code/labs/lab18_grpo_plus`

Goals

Implement the diagnostics and stabilisers of this chapter as small, testable functions, then use the library trainer to ablate them on the toy task and interpret the results like a practitioner.

Setup

```
cd code
LAB_IMPL=starter python -m pytest -q labs/lab18_grpo_plus
python labs/lab18_grpo_plus/run.py --list
```

Tasks

(functions in `starter.py`)

1. Diagnostics: `token_entropy`, `sequence_logprob_and_entropy`, `chunked_logprob_entropy`, `advantage_stats`.

2. Clipping: `policy_ratios`, `clipped_policy_loss` (returning the clip fraction). The tests check both the value and *which* samples receive gradient.
3. KL: `kl_k1`, `kl_k2`, `kl_k3`, `kl_sequence_surrogate`, `kl_importance_weighted`.
4. Rewards: `reward_format`, `total_reward` (weighted and conditional).
5. Descendants: `dr_grpo_advantages`, `aggregate_token_loss` (three reductions), `zero_variance_keep`, `expected_wasted_fraction`.

Measurements

1. Run all variants with three seeds (about 30 s per run on one CPU core): pass every name from `run.py -list` to `-variants` with `-seeds 0,1,2`. Reproduce Fig. 18.11 with error bars.
2. For `baseline`, `noclip_reuse4` and `clip_reuse4`, plot reward, entropy and zero-variance fraction against step and against *wall-clock time*. Rollout reuse should win on the second axis if it is stable.
3. Record `t_rollout`, `t_logprob`, `t_update` per variant and attribute the overhead of the KL variants to the reference forward pass.

Deliverables

Passing tests; an ablation table (mean ± std over seeds); a half-page "run review" that diagnoses one failed configuration from its metrics alone, before looking at its samples.

Stretch goals

- Implement DAPO's *active* sampling (keep sampling until the batch has P mixed groups) and measure the over-sampling factor over training (compare with the napkin math above).
- Add DAPO's soft overlong penalty (`llmsys.rl.rewards.overlong_penalty`) and a length-capped toy variant that requires long `<think>` traces.
- Port the best configuration to Qwen3-0.6B on MATH with Lab 17's `run.py qwen3` path and evaluate on MATH-500.

Chapter 19

RL Training Systems

Chapters 17 and 18 implemented reinforcement learning with verifiable rewards (RLVR) as a single Python loop: sample a group of answers, grade them, compute advantages, take a gradient step, repeat. On one GPU with a 0.6B-parameter model that loop is easy to read. It is also hopeless at scale. The models that made RL-trained reasoning famous were trained with thousands of accelerators for weeks, and almost none of that engineering effort went into the loss function. It went into the *system* that wraps the loss: an inference engine that produces millions of tokens per step, a training engine that consumes them, a fleet of reward workers that execute untrusted code, and the plumbing that moves weights, tokens and log-probabilities between them without leaving expensive hardware idle.

This chapter is about that plumbing. It is the only chapter in Part V without a counterpart in the source books, so it is written as a graduate systems lecture: we derive cost models, build a simulator, and study the design space spanned by today's open frameworks (OpenRLHF, veRL/HybridFlow, TRL, NeMo-Aligner and several newer asynchronous systems). The ecosystem moves quickly; treat the framework descriptions as a snapshot of designs that were public as of 2025, and the cost models as the part that will age well.

19.1 The anatomy of an RL step

Strip GRPO (Chapter 17) down to its dataflow and five kinds of work remain. They are summarised in Fig. 19.2.

1. **Rollout (generation).** For a batch of B prompts, sample G responses each from the current policy π_θ. This is pure inference: a prefill per prompt followed by thousands of memory-bound decode steps (Chapter 11).

2. **Reward.** Score each of the BG responses. For RLVR this means extracting an answer and checking it with a math verifier (Chapter 14) or executing generated code against unit tests; for RLHF it means a forward pass of a learned reward model.

3. **Scoring (log-prob) passes.** Compute per-token log-probabilities of every response under the policy that generated it ($\pi_{\theta_{\text{old}}}$) and, if a KL penalty is used, under a frozen reference policy π_{ref}. These are prefill-only forward passes.

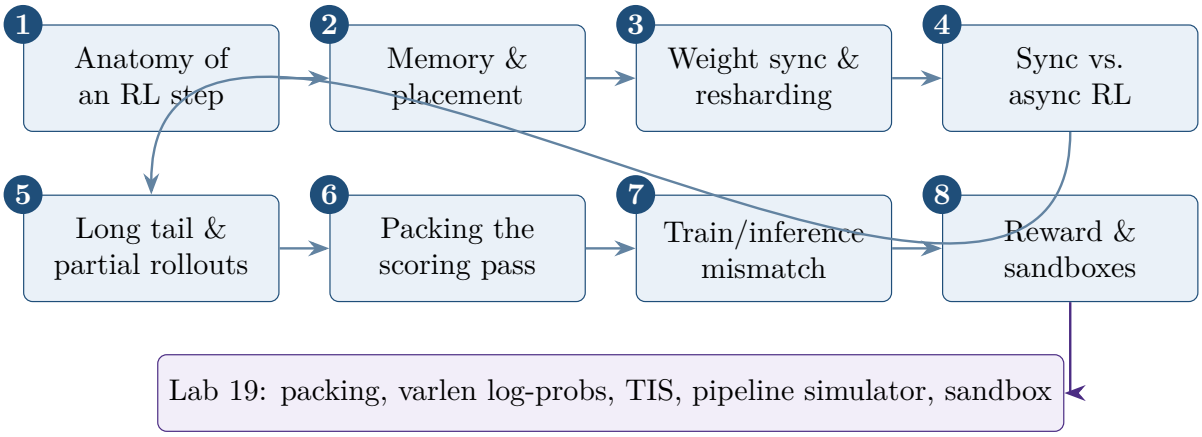


Figure 19.1: Roadmap of this chapter. We start from the dataflow of a single RL step, then add the systems concerns one at a time, ending with the reward infrastructure. Every stage has a counterpart in the lab.

4. **Advantage computation.** Cheap arithmetic: group means and standard deviations (GRPO), or generalised advantage estimation with a critic (PPO).

5. **Training.** One or more optimizer steps on the clipped surrogate objective, each a forward and backward pass over all response tokens.

After the update, the inference engine must receive the new weights before the next rollout can be on-policy. That arrow—*weight synchronisation*—is what makes RL different from both pretraining (no inference engine) and serving (no weight updates).

19.1.1 Where does the time go?

The single most important fact about RL systems for LLMs is that the stages have *different bottlenecks*. Training and scoring passes process all tokens of a sequence in parallel and are compute-bound; they want large matrix multiplies and achieve a respectable fraction of peak FLOP/s (Chapter 7). Generation emits one token per sequence per step, rereads all weights and the KV cache at every step, and is memory-bandwidth-bound (Chapter 11). A configuration that is ideal for one stage is usually wasteful for the other: training wants the model sharded with FSDP or tensor+pipeline parallelism across many GPUs and a large activation budget; generation wants as few shards as possible (to avoid communication on every decode step) and as much free memory as possible for the KV cache.

Napkin Math: One RL step for an 8B model on 64 H100s

Take Qwen3-8B ($N \approx 8.19 \times 10^9$ parameters, 36 layers, 8 KV heads of dimension 128), $B = 512$ prompts, $G = 8$ samples each, prompts of 200 tokens and responses of 4,000 tokens on average. That is $BG = 4096$ sequences and ≈ 17.2M tokens per step.

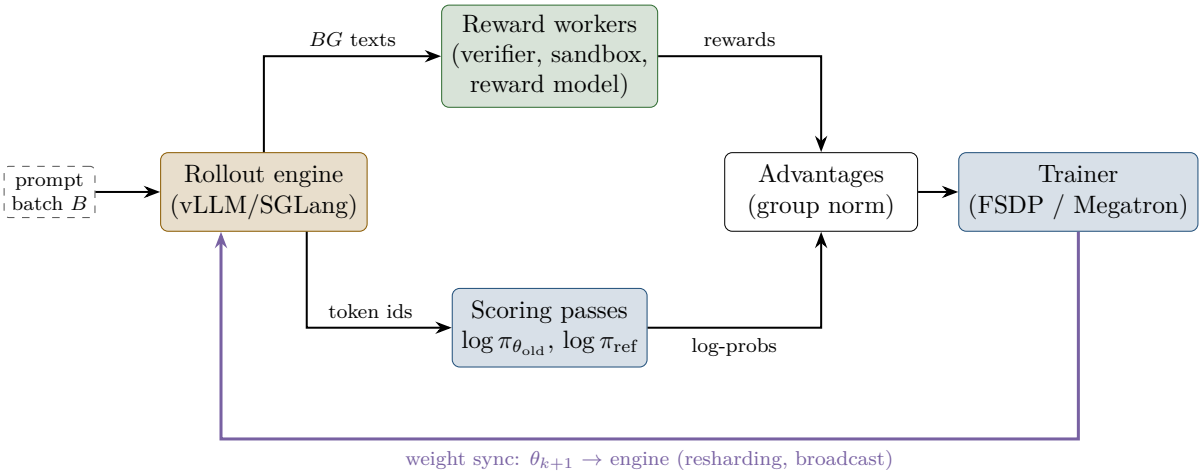


Figure 19.2: Dataflow of one RLVR step. Orange = inference workloads (memory-bandwidth bound), blue = training-style workloads (compute bound), green = reward computation (often CPU-bound, sometimes another model). The purple edge is the weight synchronisation that closes the loop.

- *Training:* $6N \cdot 17.2\text{M} \approx 8.5 \times 10^{17}$ FLOPs. At 40% MFU on 64 H100s (64×989 TFLOP/s $\times 0.4$) this is about **33 s**.
- *Scoring:* two forward passes ($\pi_{\theta_{\text{old}}}$ and π_{ref}), $2 \times 2N \cdot 17.2\text{M} \approx 5.6 \times 10^{17}$ FLOPs, about **22 s**.
- *Generation:* the KV cache costs $2 \cdot 36 \cdot 8 \cdot 128 \cdot 2 \approx 147$ KB per token. With 16.4 GB of bf16 weights and ∼8 GB of runtime overhead, about 55 GB of an 80 GB card is left for KV; at an average live context of 2,200 tokens that is ≈ 170 concurrent sequences. Each decode step streams ≈ 71 GB at 3.35 TB/s, ≈21 ms, so one GPU emits ≈ 8,000 tokens/s and 64 GPUs need ≈**32 s** for 16.4M generated tokens—*if every slot stayed busy.*
- *The tail:* if the longest response in the batch is 16,000 tokens, the last sequence decodes alone at ≈ 5–6 ms per token (weights dominate), i.e. another **80–85 s** during which almost the whole generation fleet is idle.

The idealised generation time already matches training, and the tail more than doubles it. This is why practitioners report that rollout generation typically dominates RL wall-clock time, and why most of this chapter is about keeping the generator busy.

Key Idea: RL is a pipeline of heterogeneous stages

An RL step chains an inference workload (bandwidth-bound, latency-sensitive, variable length) with a training workload (compute-bound, fixed shapes) through a weight-update dependency. Every design in this chapter (colocation, asynchrony, partial rollouts, packing) is an answer to the question "how do we keep both kinds of hardware busy despite that dependency?"

Component	Bytes/param	8B model	Notes
Actor training state (bf16 w+g, fp32 master, Adam m, v)	16	131 GB	sharded by FSDP/ZeRO-3
Actor activations (per micro-batch)	–	10s of GB	checkpointing trades for compute
Reference model (bf16, inference only)	2	16.4 GB	can be offloaded or dropped
Critic (PPO only)	16	131 GB	absent in GRPO
Rollout engine weights (bf16)	2	16.4 GB	per engine replica
Rollout KV cache	–	rest of HBM	sets decode concurrency

Table 19.1: Memory accounting for the models of an RL run (8B example). Training state is sharded across the trainer's GPUs; inference weights are replicated once per rollout engine (or sharded by its tensor-parallel degree).

19.2 Models, memory, and placement

19.2.1 Which models must be resident?

Classic RLHF with PPO juggles four networks: the *actor* (the policy being trained), the *critic* (a value model, usually initialised from the reward model), a frozen *reward model*, and a frozen *reference model* used for the KL penalty. GRPO removes the critic by estimating the baseline from the group of samples; RLVR replaces the reward model with a program; and recent recipes (DAPO, Dr. GRPO; Chapter 18) often drop the reference model by setting the KL coefficient to zero. A modern RLVR run may therefore hold only the actor—but the actor exists in two *forms*: as training state and as an inference engine.

19.2.2 Colocated versus disaggregated placement

There are two basic ways to place the rollout engine and the trainer on a cluster of G GPUs (Fig. 19.3).

Colocated (time-shared). Every GPU hosts both a training shard and an inference engine, and the two *take turns*. During generation the optimizer state and gradients are offloaded to host memory (or freed), and the engine's KV cache gets most of HBM; for training, the engine releases its KV cache (vLLM's "sleep mode" and SGLang's memory release APIs exist for exactly this), and the trainer reclaims memory. veRL's *3D-HybridEngine* (Sheng et al., HybridFlow, 2025) and OpenRLHF's hybrid-engine mode follow this pattern. Its virtue is that *no GPU is ever dedicated to a stage that is not running.* Its costs are the switch overhead (offload/onload, resharding) and the fact that the two stages are strictly serialised.

Disaggregated (split pools). G_{gen} GPUs run inference engines and $G_{\text{train}} = G - G_{\text{gen}}$ run the trainer. OpenRLHF's original Ray-based design placed actor, critic, reward and reference models (and vLLM engines) on separate groups of GPUs; most asynchronous systems use split pools. Each pool can use the parallelism layout that suits it, and the pools can run *concurrently*—but only if the algorithm tolerates data generated by slightly older weights (Section 19.4). In a strictly synchronous schedule each pool idles while the other works.

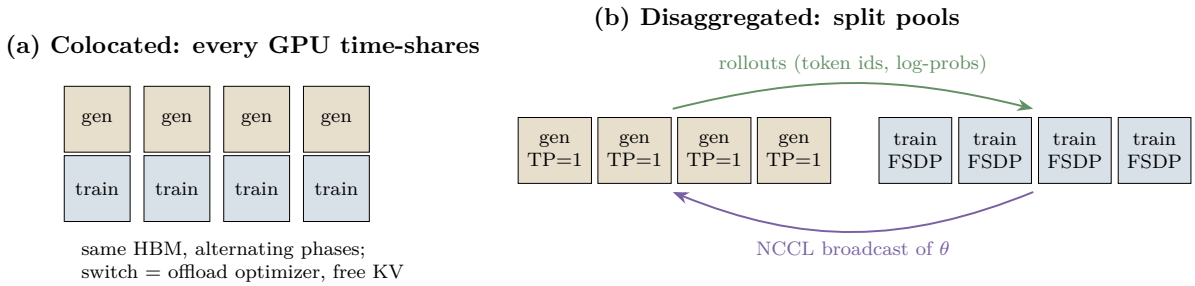


Figure 19.3: Two placements. (a) Colocation multiplexes both workloads on the same devices, in time. (b) Disaggregation dedicates devices to each workload, which permits different parallel layouts and concurrent execution, at the price of cross-pool weight transfers and potential idleness.

19.2.3 A cost model for the two placements

Let W_g be the generation work of one step and W_t the training+scoring work, both measured in GPU-seconds at perfect scaling. Ignore the tail for now (we return to it in Section 19.5). With G GPUs:

$$T_{\text{coloc}} = \frac{W_g}{G} + \frac{W_t}{G} + T_{\text{switch}}, \tag{19.1}$$

$$T_{\text{sync-split}} = \frac{W_g}{G_{\text{gen}}} + \frac{W_t}{G - G_{\text{gen}}} + T_{\text{sync}}, \tag{19.2}$$

$$T_{\text{async-split}} = \max\left(\frac{W_g}{G_{\text{gen}}} + T_{\text{sync}},\ \frac{W_t}{G - G_{\text{gen}}}\right). \tag{19.3}$$

Minimising Eq. (19.2) over G_{gen} gives $G^\star_{\text{gen}}/G = \sqrt{W_g}/(\sqrt{W_g} + \sqrt{W_t})$ and $T^\star = (\sqrt{W_g} + \sqrt{W_t})^2/G + T_{\text{sync}} \geq (W_g + W_t)/G$: a synchronous split is *never* faster than ideal colocation, because each pool idles while the other works. Equation (19.3) is minimised by balancing the two terms, $G^\star_{\text{gen}}/G = W_g/(W_g + W_t)$, which yields $T^\star \approx (W_g + W_t)/G$—the same as colocation without T_{switch}, but with one step of policy lag. So the trade-off is:

Systems Lens: What disaggregation really buys

In the idealised model, splitting pools only helps if (i) the algorithm can run the pools concurrently (asynchrony), or (ii) the stages scale *sub-linearly* on shared hardware so that specialised layouts recover efficiency (e.g. generation with TP= 1 engines has no per-token communication, whereas training needs FSDP across many GPUs), or (iii) the switch cost T_{switch} is large (offloading 131 GB of optimizer state over PCIe links that deliver a few tens of GB/s per GPU, and share host memory bandwidth across 8 GPUs per node, takes seconds per step), or (iv) the hardware is heterogeneous—older or inference-optimised accelerators can generate while training GPUs train. In practice, all four effects matter, which is why both designs survive.

19.3 Weight synchronisation and resharding

After every optimizer step the inference engines must receive θ_{k+1}. Two problems hide in that sentence: *moving* the bytes and *reshaping* them.

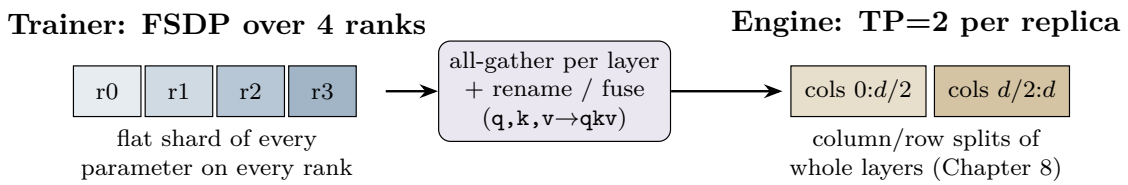


Figure 19.4: Weight synchronisation is a resharding problem: parameters sharded for training must be gathered, renamed, fused and re-split for the inference engine. veRL's 3D-HybridEngine organises the colocated case so that each GPU only gathers the slice its inference shard needs, avoiding a full replica in memory.

19.3.1 Moving the bytes

Napkin Math: How long does a weight update take?

An 8B model in bf16 is 16.4 GB.

- Colocated, same GPU: no transfer at all if the engine can be pointed at the trainer's tensors (e.g. via CUDA IPC handles); otherwise a device-to-device copy at HBM speed, a few ms.
- Over NVLink within a node (~450 GB/s per direction on H100): ≈ 36 ms.
- Over one 400 Gb/s InfiniBand NIC (50 GB/s): ≈ 0.33 s. With 8 NICs per node and each trainer rank sending its own shard in parallel, ≈ 0.04 s.
- Over 100 Gb/s Ethernet (12.5 GB/s): ≈ 1.3 s; for a 32B model, ≈ 5 s.

A naive point-to-point fan-out to R engines multiplies the sender's time by R; a pipelined (chunked) broadcast tree costs roughly one payload transfer regardless of R (Chapter 8). Compared with the tens of seconds of a step these numbers are small inside a datacenter—but they are *on the critical path* of a synchronous loop, and they grow quickly for geo-distributed or heterogeneous-cluster RL.

Two common optimisations reduce the payload: send only the *LoRA adapters* when the policy is trained with LoRA (Chapter 10), so the update is a few hundred MB; and send *bucketed* chunks so that transfer overlaps with the engine loading the previous chunk.

19.3.2 Reshaping the bytes

The trainer and the engine rarely agree on layout. A trainer using FSDP holds each parameter flattened and split across D data-parallel ranks; a Megatron trainer splits attention and MLP weights by tensor-parallel degree t_{train} and layers by pipeline stage. An inference engine typically wants whole layers with tensor parallelism t_{gen}, often $t_{\text{gen}} < t_{\text{train}}$, and fused tensors (e.g. a single `qkv_proj` instead of separate W_Q, W_K, W_V, or gate and up projections concatenated). Weight sync is therefore a *resharding* operation: an all-gather (or partial all-gather) along the training sharding, a re-split along the inference sharding, plus name and layout translation (Fig. 19.4).

Pitfall: Silent layout bugs

A transposed projection, a swapped `gate`/`up` pair, or a RoPE layout mismatch (interleaved vs. rotate-half, Chapter 6) produces an engine that still emits fluent text—just from the *wrong* model. Rewards then drift for reasons unrelated to learning. Always check, after

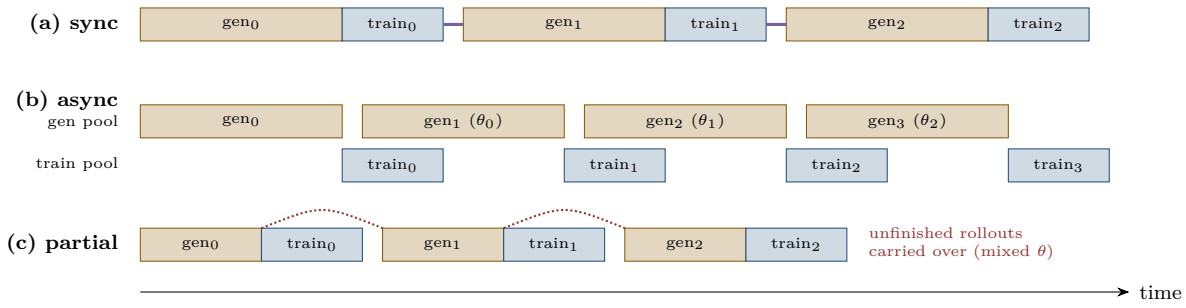


Figure 19.5: Schedules for RL. (a) Synchronous: the generator idles during training (and vice versa in split pools). (b) One-step asynchronous: batch $k{+}1$ is generated with θ_{k-1} or θ_k while batch k trains; staleness ≤ 1. (c) Partial rollouts: each generation phase stops once enough responses have finished; long responses continue in the next phase under newer weights. Purple ticks are weight syncs.

each sync during bring-up, that engine log-probs on a fixed prompt match the trainer's to within numerical tolerance (Section 19.7).

19.4 Synchronous versus asynchronous RL

19.4.1 On-policy in theory, slightly off-policy in practice

The GRPO objective of Chapter 17 is an expectation over responses sampled from $\pi_{\theta_{\text{old}}}$, the policy at the start of the step. A *synchronous* system enforces this literally: rollouts for step $k+1$ start only after the weights of step k are in the engine. An *asynchronous* system lets the generator run ahead. If a sample used in the update to θ_{k+1} was produced by θ_{k-s}, we say it has *staleness* s.

Off-policy data is not a catastrophe; PPO itself performs several minibatch updates per batch, so all but the first minibatch is already slightly off-policy, and the clipped ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$ exists precisely to bound how far one batch can push the policy. What asynchrony changes is that the *behaviour policy* that produced a sample, call it $\mu = \pi_{\theta_{k-s}}$, is now distinct from the proximal policy around which we clip. Several systems therefore use a *decoupled* objective (AReaL, Fu et al., 2025, gives a clear derivation): importance-weight by π_{prox}/μ to correct for staleness, and clip $\pi_\theta/\pi_{\text{prox}}$ to bound the update,

$$\mathcal{L}(\theta) = -\mathbb{E}_{y\sim\mu}\left[\frac{\pi_{\text{prox}}(y)}{\mu(y)}\,\min\Big(r_\theta A,\ \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)A\Big)\right], \qquad r_\theta = \frac{\pi_\theta(y)}{\pi_{\text{prox}}(y)}. \tag{19.4}$$

With $s = 0$, $\mu = \pi_{\text{prox}} = \pi_{\theta_{\text{old}}}$ and Eq. (19.4) reduces to the standard clipped objective. The same structure reappears in Section 19.7, where the behaviour policy differs from $\pi_{\theta_{\text{old}}}$ not because of staleness but because of numerics.

19.4.2 Three schedules

Figure 19.5 draws the three schedules that bracket the design space.

Synchronous. Simple and exactly on-policy. Throughput is limited by $T_{\text{gen}} + T_{\text{train}}$, and the generator's long tail is fully exposed.

One-step (bounded-staleness) asynchronous. Generation of batch $k+1$ overlaps training

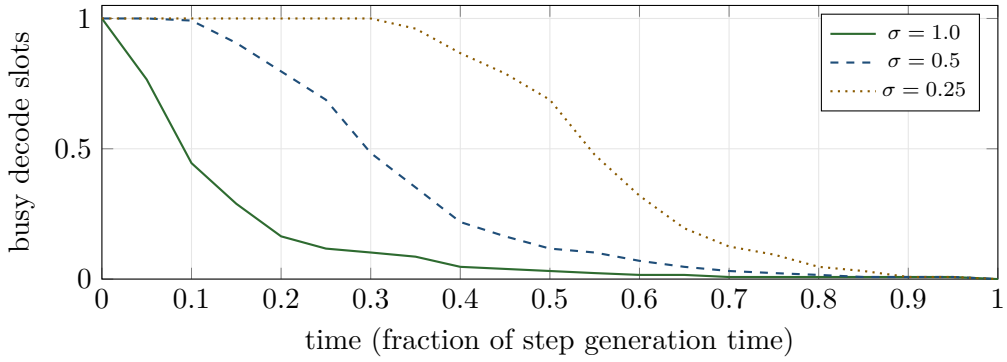


Figure 19.6: Occupancy of 128 decode slots during one synchronous generation phase with log-normal response lengths (simulated with `llmsys.rlsys.sim.generation_occupancy`). The area above each curve is wasted generator capacity: utilisation is 56%, 33% and 13% for $\sigma = 0.25$, 0.5 and 1; with $\sigma = 1$ almost 87% of slot-time is idle because a few responses run far longer than the rest.

on batch k. In steady state the step time is $\max(T_{\text{gen}}, T_{\text{train}})$ and the staleness is bounded by a pipeline depth s (the generator may not run more than s versions ahead). This is the schedule of several open systems and of our simulator's `async` policy. Fully asynchronous variants (AReaL; PipelineRL, which even swaps weights *in flight* between decode steps of an ongoing sequence) push this further.

Partial rollouts. Keep more responses in flight than one batch needs; end each generation phase once B responses have finished and resume the unfinished ones in the next phase. The tail no longer blocks a step, but some samples are produced by *several* policies, and the batches over-represent short responses until the long ones catch up. The Kimi k1.5 report (2025) describes using partial rollouts for long-context RL.

19.5 The long-tail problem

Reasoning-model response lengths are heavy-tailed: most answers to a batch of math problems end after a few hundred to a few thousand tokens, but a handful run to the length cap. With continuous batching (Chapter 12) a decode engine admits new work whenever a slot frees up, but a *synchronous* RL step has no new work to admit—it needs *these BG* responses before it can train. The engine's occupancy therefore decays as short responses finish, and the last few sequences decode almost alone.

We can quantify this with a toy model. Suppose response lengths are log-normal with mean $\bar{L}$ and shape σ, all n responses start together on n slots, and each slot decodes at a constant r tokens/s. The step's generation time is $\max_i L_i/r$ while the useful work is $\sum_i L_i/r$, so the generator's utilisation is

$$U = \frac{\sum_i L_i}{n \max_i L_i} \approx \frac{\bar{L}}{\mathbb{E}[\max_i L_i]}. \tag{19.5}$$

For log-normal lengths $\mathbb{E}[\max_i L_i]$ grows roughly like $e^{\mu+\sigma\sqrt{2\ln n}}$, so utilisation falls quickly with σ. Figure 19.6 shows the decay of occupancy computed by the simulator from the lab (`generation_occupancy`).

Mitigations, roughly in order of how much they change the algorithm:

1. **Length caps and length-aware rewards.** A hard cap bounds $\max_i L_i$; DAPO (Yu et

al., 2025) adds a soft overlong penalty so the policy learns to avoid truncation. Capping changes the task (truncated responses score zero) and should be monitored as the "clip ratio" of truncated samples.

2. **Bigger batches.** More concurrent responses amortise the tail across more useful work, until KV memory runs out.

3. **Over-provisioning with dropping.** Launch more prompts than needed and train on the first B groups to finish. This is *biased*: it systematically discards long, often hard, responses. Use with care.

4. **Partial rollouts.** Resume, do not drop, the stragglers (Fig. 19.5c).

5. **Asynchrony.** Let the next batch's generation fill the idle slots.

Systems Lens: Dynamic sampling interacts with the tail

DAPO's *dynamic sampling* (Chapter 18) discards groups whose G rewards are all equal, since their advantages are zero, and keeps sampling until the batch is full of informative groups. From a systems standpoint this is an over-provisioning policy with a data-dependent acceptance rate: early in training most groups fail and late in training most succeed, so the number of rollouts per optimizer step varies severalfold over a run. An asynchronous generator that streams groups into a buffer handles this gracefully; a synchronous one has to guess how many prompts to launch.

19.5.1 Simulating the schedules

The lab's simulator (`llmsys/rlsys/sim.py`) implements the four policies of Fig. 19.5 on top of an event-driven model of continuous batching: `n_slots` decode slots at `tokens_per_sec` each, log-normal lengths, training time proportional to tokens, and fixed costs for switching and weight sync. Listing 19.1 shows its core.

Listing 19.1: Continuous-batching generation with an early-stop for partial rollouts (from `llmsys/rlsys/sim.py`).

```
def run_generation(pending, n_slots, tps, stop_after, version):
    queue = list(pending)
    heap = []                    # (finish_time, seq, rollout, start_time)
    t, busy, seq = 0.0, 0.0, 0
    finished = []

    def admit(now):
        nonlocal seq
        while queue and len(heap) < n_slots:
            r = queue.pop(0)
            if not r.versions or r.versions[-1] != version:
                r.versions.append(version)          # this policy touched r
            heapq.heappush(heap, (now + r.remaining / tps, seq, r, now))
            seq += 1

    admit(0.0)
    while heap:
        t, _, r, start = heapq.heappop(heap)
```

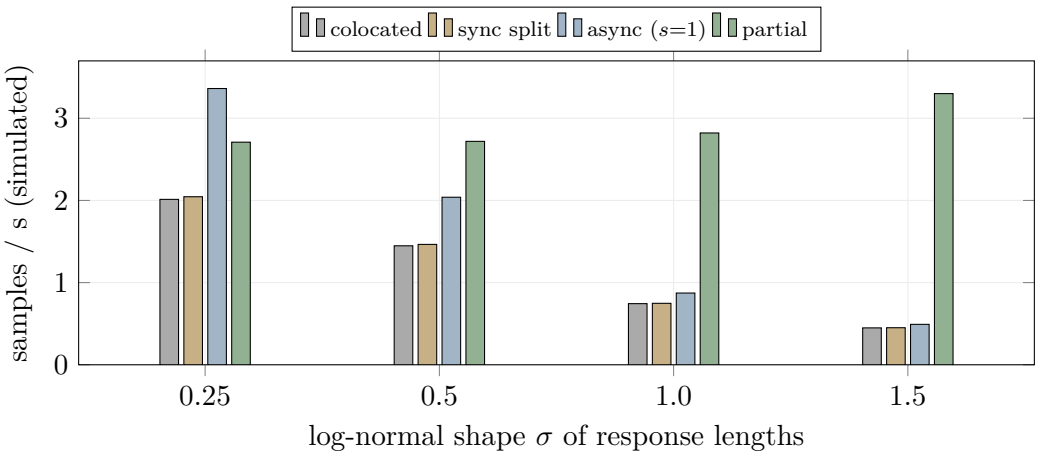


Figure 19.7: Throughput of four RL schedules in the lab simulator (128 slots, batch 128, mean length 1,000 tokens, default costs; `python labs/lab19_rlsys/bench.py sim`). These are outputs of a model, not measurements of a real cluster; their value is in the trends.

```
        busy += t - start
        r.remaining = 0.0
        finished.append(r)
        if stop_after is not None and len(finished) >= stop_after:
            break                                   # partial rollouts: stop early
        admit(t)
    unfinished = []
    for (ft, _, r, start) in heap:                  # account partial progress
        busy += t - start
        r.remaining = max(0.0, (ft - t) * tps)
        unfinished.append(r)
    return t, finished, unfinished + queue, busy
```

Figure 19.7 reports the throughput of each policy as the tail gets heavier. Two lessons stand out. First, one-step asynchrony helps a lot when the stages are balanced and the tail is light ($\sigma = 0.25$: +64%), but almost nothing when generation is dominated by a few stragglers—overlapping training with a mostly idle generator does not make the generator busier. Second, partial rollouts attack the tail directly and keep throughput roughly constant as σ grows, at the price of 45–85% of samples containing tokens from more than one policy version and of a length-biased batch composition.

Pitfall: Partial rollouts bias the batch toward short answers

In each phase, the responses that finish first are the short ones. Early in a run, the gradient therefore over-weights short responses, and long responses reach the trainer one or more versions late. If the reward correlates with length (it usually does for reasoning), this is a systematic bias. Log the length distribution of *trained* samples, not only of generated ones.

19.6 Packing the scoring and training passes

Once rollouts exist, the trainer runs prefill-style passes over them: the $\pi_{\theta_{\text{old}}}$ and π_{ref} log-prob passes and the forward/backward of the update. Their cost is linear in tokens (plus a quadratic attention term), so every padding token is wasted work. With heavy-tailed lengths, padding to

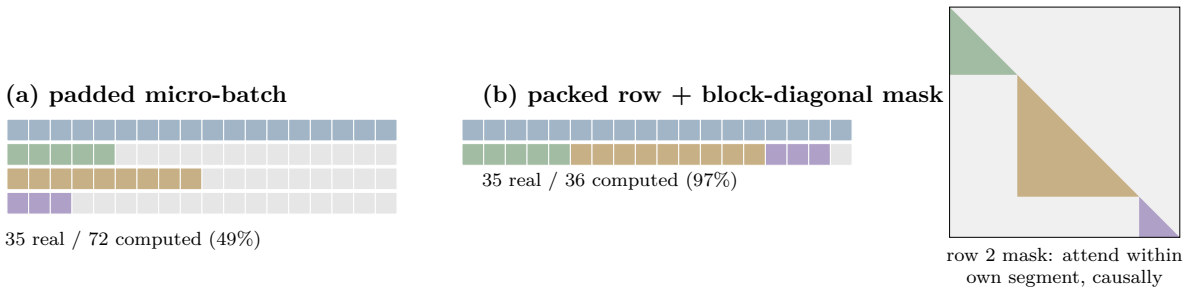


Figure 19.8: Padding vs. packing for four responses of lengths 18, 5, 9 and 3. Packing concatenates sequences into rows of a fixed token budget; a block-diagonal causal mask (or a varlen kernel driven by `cu_seqlens`) prevents attention across sequence boundaries, and position ids restart at 0 for every segment.

the longest sequence in a micro-batch can waste most of the compute (Fig. 19.8).

Three ingredients make packing correct:

1. **Bin packing.** Group sequences into rows under a token budget. First-fit decreasing (FFD) is simple and provably uses at most $\frac{11}{9}\mathrm{OPT} + \frac{6}{9}$ bins (the tight bound is due to Dósa, 2007).

2. **Segment-aware attention.** Each token may attend only to earlier tokens of its own sequence. FlashAttention's `varlen` interface takes cumulative sequence lengths (`cu_seqlens`) and never materialises the mask; we build the mask explicitly.

3. **Per-segment positions.** RoPE angles (or GPT-2 position embeddings) must restart at 0 for each sequence; otherwise the second sequence in a row sees shifted positions.

Listing 19.2: Block-diagonal causal mask and a packed log-prob pass (`llmsys/rlsys/packing.py`, abridged).

```python
def block_causal_mask(cu_seqlens):
    N = int(cu_seqlens[-1])
    seg = torch.zeros(N, dtype=torch.long)
    for k in range(len(cu_seqlens) - 1):
        seg[int(cu_seqlens[k]):int(cu_seqlens[k + 1])] = k
    same = seg[:, None] == seg[None, :]
    causal = torch.ones(N, N, dtype=torch.bool).tril()
    return ~(same & causal)                      # True = forbidden

def packed_completion_logprobs(model, seqs, prompt_lens, max_tokens):
    out = [0.0] * len(seqs)
    for bin_ in ffd_pack([len(s) for s in seqs], max_tokens):
        ids, pos, cu = build_packed_row([seqs[i] for i in bin_])
        logp = torch.log_softmax(varlen_forward(model, ids, pos, cu)[0].float(), -1)
        for j, i in enumerate(bin_):
            s, e = int(cu[j]), int(cu[j + 1])
            tok_lp = logp[s:e - 1].gather(-1, ids[0, s + 1:e, None]).squeeze(-1)
            out[i] = float(tok_lp[prompt_lens[i] - 1:].sum())
    return out
```

The function `varlen_forward` reuses the *weights* of the core `Qwen3Model` or `GPTModel` but runs its own attention with the packed mask and per-token RoPE positions. The lab's test checks that

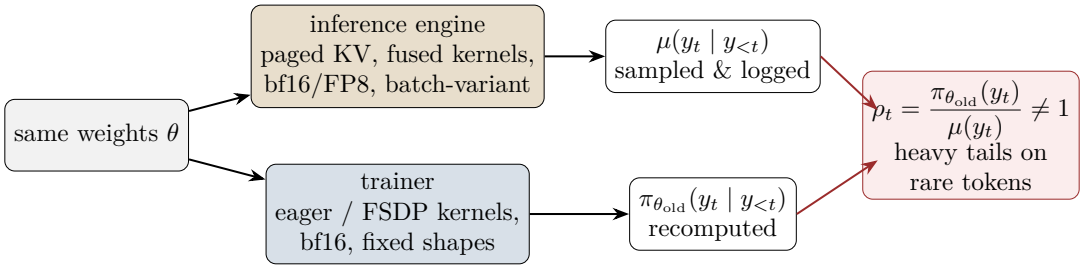


Figure 19.9: Train/inference mismatch: the samples come from the engine's distribution μ, while the policy-gradient estimator assumes they come from $\pi_{\theta_{\text{old}}}$.

every packed log-prob equals the unpacked `sequence_logprob` of Chapter 16 to within 10^{-3}. On a CPU with a tiny model and 64 log-normal-length sequences, the packed pass took 17 s against 42 s for padded batches of 8 in our runs, with a real-token fraction of 96% vs. 39%.

Pitfall: Loss normalisation across micro-batches and ranks

Token-level losses (DAPO, Chapter 18) are defined as a sum over *all* tokens in the global batch divided by the global token count. If each micro-batch computes its own token-mean and gradients are then averaged over micro-batches and data-parallel ranks, micro-batches with few tokens are silently up-weighted, and the result depends on how the packer happened to group sequences. The fix is to divide each micro-batch's token *sum* by the global token count (an all-reduce of a scalar before the backward passes). Several frameworks have had bugs of exactly this kind.

Systems Lens: Dynamic batch size by token budget

Fixed-size micro-batches of variable-length sequences produce wildly varying memory peaks. Production trainers instead form micro-batches by a *token budget* (veRL calls this dynamic batch size; the idea is general), which makes activation memory roughly constant and lets the budget be tuned to the largest value that fits. With packing, the token budget is literally the row length.

19.7 Train/inference mismatch

A subtle problem appeared once RL systems began using high-throughput inference engines: even with *identical* weights, the engine's token probabilities $\mu(y_t)$ differ from those the trainer computes, $\pi_{\theta_{\text{old}}}(y_t)$. Engines use different kernels (fused attention, paged KV, different GEMM tilings), different reduction orders, batch-size-dependent algorithms, sometimes FP8 or quantised KV caches, and sampling-time logit processing. All of these are harmless for serving and all of them change the low bits of the logits. Over thousands of tokens and a vocabulary of 150k entries, low-bit differences become occasional large ratio differences on low-probability tokens—exactly the tokens a sampler sometimes picks.

Because $y \sim \mu$, the unbiased policy gradient is

$$\nabla_\theta J = \mathbb{E}_{y\sim\mu}\left[\sum_t \frac{\pi_{\theta_{\text{old}}}(y_t \mid y_{<t})}{\mu(y_t \mid y_{<t})} \nabla_\theta \min\big(r_t A,\ \text{clip}(r_t, 1\pm\epsilon)A\big)\right], \tag{19.6}$$

where we have used a per-token approximation of the sequence-level ratio, as PPO does. Ignoring

$\rho_t = \pi_{\theta_{\text{old}}}/\mu$ (i.e. setting it to 1) introduces a bias that has been reported to destabilise long runs. Since ρ_t is occasionally huge, the naive correction has high variance. Two practical estimators, both implemented in `llmsys/rlsys/mismatch.py`:

- **Truncated importance sampling (TIS):** $w_t = \min(\rho_t, C)$ with $C \approx 2$. This was popularised for LLM RL in a 2025 analysis by Yao et al. of the mismatch in vLLM+FSDP systems; it is the same truncation idea as in off-policy actor–critic methods.
- **Masked importance sampling (MIS):** $w_t = \mathbf{1}[1/C \le \rho_t \le C]$, which drops outlier tokens instead of down-weighting them; variants operate at the sequence level.

The weights are *detached*—they rescale gradients and are never optimised.

Listing 19.3: Clipped surrogate with truncated importance-sampling correction.

```python
def is_weights(logp_train, logp_rollout, cap=2.0, mode="truncate"):
    ratio = torch.exp((logp_train - logp_rollout).detach())
    if mode == "truncate":
        return ratio.clamp(max=cap)
    if mode == "mask":
        return ((ratio >= 1.0 / cap) & (ratio <= cap)).to(ratio.dtype)
    return torch.ones_like(ratio)

def clipped_pg_loss(logp_new, logp_old, adv, mask, clip_eps=0.2,
                    logp_rollout=None, tis_cap=None, tis_mode="truncate"):
    adv = adv[:, None] if adv.dim() == 1 else adv
    ratio = torch.exp(logp_new - logp_old.detach())
    surr = torch.minimum(ratio * adv,
                         ratio.clamp(1 - clip_eps, 1 + clip_eps) * adv)
    if logp_rollout is not None and tis_cap is not None:
        surr = surr * is_weights(logp_old, logp_rollout, tis_cap, tis_mode)
    m = mask.to(surr.dtype)
    return -(surr * m).sum() / m.sum().clamp(min=1)
```

Correction is one approach; *elimination* is the other. Making inference kernels *batch-invariant*—so the result for a sequence does not depend on what else is in the batch—removes a major source of nondeterminism, and with matching kernels in the trainer it is possible to run bitwise on-policy RL (Thinking Machines Lab, "Defeating nondeterminism in LLM inference", 2025). Others have argued for higher-precision formats (e.g. fp16 with its larger mantissa instead of bf16) to shrink the mismatch. These remedies cost inference throughput; TIS costs almost nothing. A reasonable default is: always log the mismatch, always recompute $\pi_{\theta_{\text{old}}}$ in the trainer rather than trusting engine log-probs for the ratio, and enable TIS or MIS.

Napkin Math: How big is the mismatch from precision alone?

The lab evaluates the same tiny Qwen3 model in fp32 and in bf16 on random token sequences (`token_logprobs` with `dtype=torch.bfloat16`). On our CPU runs the mean absolute log-prob difference was ≈ 0.09 nats, the per-token ratio ranged over $[0.70, 1.37]$, and the k_3 KL estimate was $\approx 7 \times 10^{-3}$. A random tiny model has nearly uniform outputs and is not representative of a trained model; run `bench.py mismatch -qwen3_dir` on real weights to see the long tail on rare tokens that motivates TIS.

Key Idea: Monitor these every step

Mean $|\log \pi_{\theta_{\text{old}}} - \log \mu|$; the k_3 estimate of $\mathrm{KL}(\mu \,\|\, \pi_{\theta_{\text{old}}})$; the fraction of tokens with $\rho_t \notin [0.5, 2]$; the maximum ρ_t. A sudden increase is often the first sign of a weight-sync bug, a kernel change, or a numerically unstable model state.

19.8 Configuring the rollout engine for RL

A serving engine tuned for chat traffic is not automatically a good rollout engine. Several settings that are harmless (or desirable) for serving change the *distribution* that RL assumes it is sampling from, or waste the structure of RL workloads.

Sampling parameters define μ. The policy-gradient estimator assumes samples come from the policy whose log-probabilities appear in the ratio. If the engine samples with temperature $\tau \neq 1$, top-p or top-k (Chapter 15), it samples from a *transformed* distribution. Either sample at $\tau = 1$ without truncation, or compute the training-side log-probabilities under the same transformation—including the truncation mask: renormalise over exactly the token set that was available at sampling time (one of the GRPO modifications listed in Chapter 18). Repetition penalties, logit biases and grammar-constrained decoding all fall under the same rule.

Exploit the shared prompt. GRPO samples G responses per prompt. An engine that supports parallel sampling (`n`$= G$ in vLLM) or prefix caching (Chapter 12) prefills each prompt once and forks its KV blocks copy-on-write for the G continuations (Chapter 15). For long prompts (agentic tasks with large contexts, or long system prompts) this removes up to a factor G of prefill work and of KV memory for the prompt.

Return token ids, not text. The trainer must score exactly the token sequence the engine sampled. Decoding to text and re-tokenising can change the sequence (merges across the prompt/response boundary, normalisation of whitespace or special tokens), which silently breaks the ratio. Pass token ids end to end; this is sometimes called a "token-in, token-out" interface.

CUDA graphs, cache size and preemption. Decode steps of small models are dominated by kernel-launch overhead unless captured in CUDA graphs (Chapter 11); engines capture graphs for a set of batch sizes at start-up, which costs time and memory each time an engine is woken in a colocated setup. The KV cache budget determines concurrency; when it is exhausted the engine preempts sequences (swap or recompute), which for RL does not change the result but can multiply the tail.

Systems Lens: Rollout engines are throughput servers with no SLOs

Serving engines trade throughput for latency SLOs (TTFT, inter-token latency). An RL rollout has no per-request latency target at all: only the time until the *whole batch* (or the first B groups) is done matters. Scheduling should therefore maximise concurrency and prioritise requests that are likely to be long (start them first, a longest-processing-time heuristic) rather than serve requests in arrival order. Length predictors trained on previous steps' rollouts have been proposed for exactly this purpose.

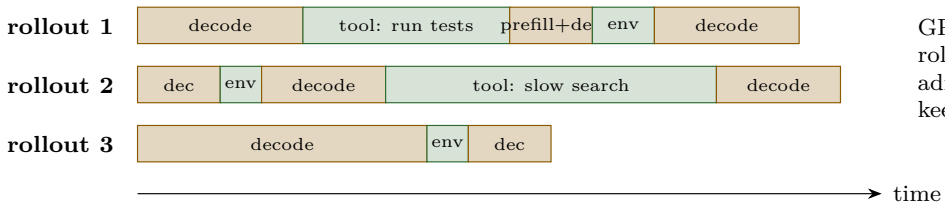


Figure 19.10: Agentic rollouts interleave decoding (orange) with environment steps (green) whose latency can exceed the decoding time. An engine that blocks a slot while a tool runs wastes the slot; asynchronous rollout workers release it and re-admit the sequence when the observation arrives, ideally with its KV cache still resident (prefix caching).

19.9 Multi-turn and agentic rollouts

The RLVR loop in this book generates one response per prompt. Agentic RL—training models to use tools, browse, or edit a code repository—turns each rollout into a *conversation with an environment*: the model emits a tool call, generation pauses, the environment executes the call (a shell command, a search query, a unit test), and the observation is appended to the context before generation resumes (Fig. 19.10).

This changes the systems picture in four ways:

1. **Rollouts become state machines.** A rollout worker drives the loop generate → parse tool call → execute → append observation → generate, typically with `asyncio` over many concurrent episodes so that environment latency overlaps other episodes' decoding.

2. **The KV cache must survive pauses.** Re-prefilling the whole context after every tool call is quadratic in the number of turns; prefix caching (or explicitly pinned sessions) keeps the conversation's KV resident between turns.

3. **Loss masks get more complex.** Only tokens the model generated should receive gradient; tool outputs and environment observations are context, so they are masked out of the loss exactly like prompt tokens in SFT (Chapter 10). The token-in/token-out rule matters even more because observations are spliced between generated segments.

4. **Environments are infrastructure.** Each episode may need its own container (a repository checkout, a browser, a database), with start-up costs of seconds; environment pools, snapshotting, and failure handling become as important as the GPU schedule.

19.10 Reward infrastructure

In RLVR the reward is a program. That sounds cheap until one counts: 4,096 responses per step, each requiring answer extraction, normalisation and a symbolic-equivalence check (Chapter 14), or the execution of model-written code against a test suite. Reward computation is therefore a *service* with its own throughput, latency, and isolation requirements (Fig. 19.11).

Streaming. Because responses finish at very different times, the reward of the first finished response can be computed while the last ones are still decoding. A reward service that consumes a stream of completions hides almost all of its latency behind the generation tail.

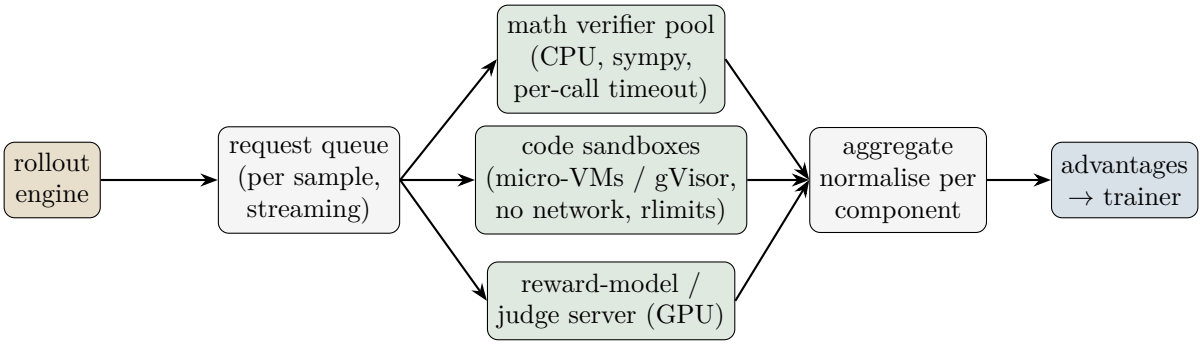


Figure 19.11: Reward computation as a service. Samples are scored as soon as they finish, so reward latency hides behind the generation tail. Different reward types run on different resources; multi-reward training normalises each component before combining them (Chapter 18).

Timeouts everywhere. Symbolic equivalence checking can take unbounded time on adversarial expressions, and generated code can loop forever. Every reward call needs a hard wall-clock timeout, a defined reward on timeout (usually 0), and a metric counting timeouts.

Isolation. Executing model-written code is executing untrusted code. A policy under optimisation pressure will discover anything that raises its reward: deleting or monkeypatching the tests, reading the expected outputs from disk, exiting with status 0 before the assertions run, or exhausting resources to crash the grader into a default score. Production sandboxes use containers with seccomp filters, user-space kernels (gVisor), micro-VMs (Firecracker), or jail tools (nsjail), with no network, read-only base images, cgroup CPU/memory limits, and a pool of pre-warmed workers to amortise start-up cost.

Listing 19.4: A teaching sandbox: a fresh interpreter, empty environment, resource limits and a timeout. Not a security boundary.

```
def _limits(cpu_sec, mem_mb):
    def fn():
        resource.setrlimit(resource.RLIMIT_CPU, (cpu_sec, cpu_sec))
        mem = mem_mb * 1024 * 1024
        resource.setrlimit(resource.RLIMIT_AS, (mem, mem))
        resource.setrlimit(resource.RLIMIT_NOFILE, (32, 32))
        resource.setrlimit(resource.RLIMIT_FSIZE, (1 << 20, 1 << 20))
        os.setsid()
    return fn

def run_python_reward(code, tests, timeout=2.0, mem_mb=512):
    program = code + "\n\n" + tests + "\n"
    t0 = time.perf_counter()
    with tempfile.TemporaryDirectory() as tmp:
        path = os.path.join(tmp, "prog.py")
        with open(path, "w") as f:
            f.write(program)
        try:
            p = subprocess.run([sys.executable, "-I", "-S", path], cwd=tmp,
                               env={}, capture_output=True, text=True,
                               timeout=timeout,
                               preexec_fn=_limits(int(timeout) + 1, mem_mb))
        except subprocess.TimeoutExpired:
            return SandboxResult("timeout", 0.0, time.perf_counter() - t0)
```

```
    ok = p.returncode == 0
    status = "pass" if ok else ("fail" if "AssertionError" in p.stderr else "error")
    return SandboxResult(status, float(ok), time.perf_counter() - t0, p.stderr
    [-500:])
```

Pitfall: Reward hacking through the grader

Listing 19.4 concatenates the tests after the code, so a response that calls `os._exit(0)` at import time "passes" every test. Real graders run tests in a separate process or namespace from the candidate code, check that the expected number of tests actually ran, and treat any deviation from the protocol as failure. Audit samples with suspiciously high reward.

Napkin Math: Sandbox capacity

A cold Python process costs tens of milliseconds to start; a container, hundreds of milliseconds; a micro-VM from a snapshot, on the order of 100 ms. If 4,096 code responses per step each run a 0.5 s test suite and the generation phase lasts 60 s, we need at least $4096 \times 0.5/60 \approx 34$ concurrent workers just to keep up with the stream—and several times that to keep reward latency off the critical path when completions arrive in a burst at the end of a synchronous phase.

19.11 A tour of open RL frameworks

With the vocabulary of the previous sections we can place the open frameworks in a common design space (Table 19.2). The table describes designs as they were documented around 2025; all of these projects evolve quickly, often converging on each other's best ideas, so consult their current documentation before relying on a detail.

19.11.1 Single controller versus multi-controller

The HybridFlow paper frames a useful distinction. In a *multi-controller* (SPMD) program, every GPU runs the same script and coordination is implicit in collectives—the natural style for training a single model (Chapter 8). In a *single-controller* program, one driver issues remote calls to groups of workers—the natural style for a dataflow of several models, because the RL algorithm reads like the loop of Chapter 17:

Listing 19.5: The RL loop as a single-controller program (pseudocode in the style of Ray-based frameworks). Each call fans out to an SPMD worker group.

```
for step in range(num_steps):
    batch = dataloader.next()
    rollouts = rollout_group.generate(batch, n=G)           # vLLM/SGLang workers
    rewards = reward_service.score(rollouts)                # CPU pool / sandboxes
    old_lp = actor_group.compute_logprobs(rollouts)         # FSDP workers, packed
    ref_lp = ref_group.compute_logprobs(rollouts) if beta > 0 else None
    adv = group_normalise(rewards, G)                       # on the driver
    metrics = actor_group.update(rollouts, adv, old_lp, ref_lp)
    rollout_group.load_weights(actor_group.export_weights())  # weight sync
```

The driver handles only small metadata (or references to data held by workers, as Ray object references), so it does not become a bandwidth bottleneck, while each `*_group` call is an

SPMD collective program internally. HybridFlow argues that this hybrid gives the flexibility of single-controller dataflow with the efficiency of SPMD execution.

19.12 Checkpointing and fault tolerance

An RL run's state is larger than a pretraining run's: actor weights and optimizer state, critic state (PPO), the frozen reference (re-loadable), data-loader position and RNG state, *and* the in-flight rollout state of an asynchronous pipeline—the replay buffer of finished but untrained samples, partially generated responses, and the policy version that produced each. Two pragmatic rules:

- **Checkpoint at step boundaries and drop in-flight rollouts on restart.** The lost work is at most one step of generation, and restart logic stays simple. Record the policy version in every sample so that resumed async pipelines can enforce the staleness bound.
- **Make rollout engines stateless and restartable.** Engines crash (OOMs from unexpectedly long sequences, driver errors). A controller that can re-launch an engine, reload the latest weights and re-issue its unfinished prompts turns a crash into a slowdown.

Sharded, asynchronous checkpointing of the trainer (Chapter 7) applies unchanged.

19.13 Putting it together: a design worksheet

Napkin Math: Sizing an 8B RLVR run on 64 GPUs

Using the numbers from Section 19.1: $W_t \approx 64 \times (33+22)$ GPU-s $\approx 3{,}500$ GPU-s of training and scoring per step, and $W_g \approx 64 \times 32 \approx 2{,}050$ GPU-s of generation if slots stay busy.

- *Colocated, synchronous:* $(W_g + W_t)/64 \approx 87$ s plus the tail ($\sim$80 s) plus switch overhead: $\sim$170–180 s per step.
- *Split, one-step async:* balance $G_{\text{gen}}/G = W_g/(W_g + W_t) \approx 0.37$, i.e. 24 generation GPUs and 40 training GPUs; the per-step time approaches $\max(W_g/24, W_t/40) \approx 87$ s, but the tail now sits inside the generation term: unless it is mitigated (partial rollouts, caps), generation takes $\approx 85 + 80 = 165$ s and the trainer idles half the time.
- *Split, async + partial rollouts:* with the tail absorbed into the next phase, step time approaches the balanced ≈ 90 s, a $\sim 2\times$ improvement over the synchronous baseline, at the cost of staleness $\leq$ 1–2 and mixed-policy samples.

These figures are idealised, but they show where to look first: the tail, then the balance between pools, then the constant overheads.

A checklist for a new RL system:

1. Measure the response-length distribution before choosing a schedule; it determines whether asynchrony or tail mitigation matters more.
2. Recompute $\pi_{\theta_{\text{old}}}$ in the trainer and log mismatch metrics; enable TIS/MIS.
3. Pack the scoring and training passes; normalise token-level losses by the global token count.

4. Bound staleness explicitly, record policy versions per sample, and correct with importance weights where staleness is non-zero.

5. Stream rewards; put timeouts and isolation around every reward call.

6. Verify weight sync with a log-prob comparison during bring-up.

19.14 Summary

- An RL step chains bandwidth-bound generation, CPU-bound or GPU-bound rewards, and compute-bound scoring/training through a weight-update dependency. Generation usually dominates wall-clock time, mostly because of the long tail of response lengths.
- Colocation time-shares devices (no idle pools, switch overhead); disaggregation dedicates devices (specialised layouts, concurrency) but only beats colocation when combined with asynchrony, heterogeneous hardware, or high switch costs.
- Weight sync is a resharding problem as much as a bandwidth problem; validate it by comparing log-probs.
- Asynchronous RL bounds staleness and corrects for it with importance weights (decoupled objectives); partial rollouts tame the tail at the cost of mixed-policy, length-biased batches.
- Packing with block-diagonal masks and per-segment positions removes padding waste in scoring and training passes; loss normalisation must use global token counts.
- Inference engines sample from $\mu \neq \pi_{\theta_{\text{old}}}$ even with identical weights; truncated or masked importance sampling corrects this cheaply, batch-invariant kernels remove it.
- Rewards are a service: stream them, time them out, and isolate code execution; assume the policy will find any loophole in the grader.

19.15 Exercises

Exercise 19.1 (*Optimal split*). Derive the optimal G_{gen} for Eq. (19.2) and show that $T^{\star}_{\text{sync-split}} \geq T_{\text{coloc}} - T_{\text{switch}}$ with equality iff one stage has zero work. For which values of T_{switch} does the synchronous split win?

Exercise 19.2 (*KV budget*). Repeat the generation napkin math of Section 19.1 for Qwen3-4B (36 layers, 8 KV heads, head dimension 128) on an 80 GB GPU with average live context 6,000 tokens. How many concurrent sequences fit, and what is the upper bound on decode tokens/s per GPU?

Exercise 19.3 (*Staleness bound*). In the `async` policy of the simulator, generation of batch k may start only after training step $k - s - 1$ has finished. Prove that every sample of batch k has staleness at most s, and show with a timeline that $s = 0$ recovers the synchronous split schedule.

Exercise 19.4 (*Loss normalisation bug*). Two micro-batches contain 100 and 1,000 response tokens with mean token losses ℓ_1 and ℓ_2. Compute the effective loss when (a) micro-batch means are averaged, and (b) the token-level definition is used. Write the one-line fix for a gradient-accumulation loop.

Exercise 19.5 (*TIS bias and variance*). For a single token with $\rho \sim$ a log-normal distribution with $\mathbb{E}_\mu[\rho] = 1$, compute the bias $\mathbb{E}_\mu[\min(\rho, C)] - 1$ as a function of C and the log-normal variance. Why is a small bias acceptable here?

Exercise 19.6 (*Packing efficiency*). Show that FFD never uses more than 2 OPT bins with a short argument (the full $\frac{11}{9}$ bound is harder). Then use `efficiency_report` to compare padding, length-sorted padding and packing on log-normal lengths with $\sigma \in \{0.25, 0.5, 1\}$.

Exercise 19.7 (*Partial-rollout bias*). Using the simulator, record the mean length of samples consumed in each step under `partial` and compare with the generated length distribution. How many steps does it take for the consumed-length distribution to match the generated one? How does `oversample` change this?

Exercise 19.8 (*Weight sync across sites*). A 32B policy is trained in one datacenter and rolled out in another over a 100 Gb/s link. Estimate the sync time in bf16, fp8 and with LoRA rank 64 on all linear layers. Which schedule from Section 19.4 would you use?

Exercise 19.9 (*Attacking the grader*). List five ways a policy could obtain reward from Listing 19.4 without solving the task, and give a mitigation for each. Implement two of them as tests for your own sandbox.

Exercise 19.10 (*Measurement*). Run `bench.py sim` and add a policy `async+partial` to the simulator (partial rollouts in a generator pool that runs concurrently with training). Plot throughput and mean staleness against `oversample`.

Exercise 19.11 (*Design*). You have 32 H100s and a 14B model. Propose a placement, schedule and parallel layout for an RLVR run with $B = 256$, $G = 16$, mean response length 6,000 and a cap of 32,000 tokens. Justify each choice with a napkin estimate.

19.16 Lab 19: Building blocks of an RL training system

Lab 19: Building blocks of an RL training system

Code: `code/labs/lab19_rlsys`

Goals

Implement and measure the pieces that surround the GRPO update in a production RL framework: packed scoring passes, mismatch correction, schedule simulation and sandboxed rewards.

Setup

```
cd code
pytest -q labs/lab19_rlsys                      # grades starter.py
LAB_IMPL=solution pytest -q labs/lab19_rlsys    # reference implementation
```

Everything runs on a CPU in under a minute with tiny models; `bench.py` accepts a real Qwen3 checkpoint for the mismatch measurement.

Tasks

1. **Packing.** Implement `ffd_pack` and `efficiency_report` (padded, length-sorted padded, packed real-token fractions).
2. **Varlen log-probs.** Implement `block_causal_mask` and `packed_completion_logprobs` on top of the provided `varlen_forward`. The tests compare against unpacked `sequence_logprob` for both Qwen3- and GPT-shaped models.
3. **Mismatch.** Implement `is_weights` (truncate/mask/none), `mismatch_metrics` (including the k_3 KL estimate) and `clipped_pg_loss` with an optional TIS factor. Check that the gradient at $\pi_\theta = \pi_{\theta_{\text{old}}}$ is $-A/n_{\text{tokens}}$.
4. **Pipeline simulation.** Implement `run_generation` (continuous batching with an early stop) and `weight_sync_seconds`; the provided `simulate` drives your code through the colocated, sync, async and partial policies.
5. **Sandboxed reward.** Implement `run_python_reward` and `score_batch` with timeouts, resource limits and an empty environment.

Measurements

- `bench.py sim`: throughput, utilisation, staleness and mixed-policy fraction for each policy as σ varies. Expect async to help most for light tails and partial rollouts to dominate for heavy tails (Fig. 19.7).
- `bench.py packing`: wall time of padded vs. packed log-prob passes and the real-token fractions. Expect a large gap whenever lengths are heavy-tailed.
- `bench.py mismatch`: bf16-vs-fp32 mismatch metrics, ideally on real Qwen3-0.6B weights; look at the maximum ratio and the fraction outside $[0.5, 2]$.

Deliverables

Passing tests; a one-page note with the simulator sweep (a plot like Fig. 19.7), the packing timings, and the mismatch metrics, each with a sentence of interpretation.

Stretch goals

- Add an `async+partial` policy and a staleness-aware sampler that drops samples older than s versions; plot throughput vs. staleness.
- Wire `clipped_pg_loss` with TIS into your GRPO loop from Chapter 18, generate rollouts with a bf16 copy of the model and train in fp32; compare reward curves with and without TIS.
- Replace the dense mask in `varlen_forward` with a loop over segments that calls `F.scaled_dot_product_attention` per segment, and measure the speedup.

Framework	Orchestration	Training / rollout engines	Placement & notable ideas
TRL (Hugging Face)	single program on top of `accelerate`	Transformers + DeepSpeed/FSDP; vLLM for generation	vLLM either as a separate server process or colocated in the training process; easiest entry point, HF-ecosystem native
OpenRLHF (Hu et al., 2024)	Ray actors	DeepSpeed ZeRO; vLLM	originally separate GPU groups per model (actor, critic, reward, reference, vLLM); later a hybrid-engine colocation mode and asynchronous options
veRL / HybridFlow (Sheng et al., 2025)	hybrid: single controller (Ray driver) for the RL dataflow, SPMD workers inside each model	FSDP or Megatron-LM; vLLM or SGLang	3D-HybridEngine for colocated, low-redundancy resharding between training and generation layouts; token-budget dynamic batching
NeMo-Aligner (Shen et al., 2024)	Megatron-based distributed jobs	Megatron-LM; TensorRT-LLM-accelerated generation	large-scale PPO/RLHF in the NVIDIA stack; continued in NVIDIA's newer NeMo-RL
AReaL (Fu et al., 2025)	asynchronous generation and training services	SGLang rollouts; FSDP/Megatron training	fully asynchronous with bounded staleness and a decoupled PPO objective; interruptible rollouts

Table 19.2: Design choices of several open RL frameworks for LLMs (snapshot of public designs around 2025). Other actively developed systems include slime, ROLL, SkyRL, PipelineRL and prime-rl; the space changes month to month.

Chapter 20

Distillation for Efficient Reasoning

Reinforcement learning taught a model to reason by trial, error and a verifier (Chapters 17 and 18), and Chapter 19 showed how expensive that is: every optimizer step pays for thousands of long generations. *Distillation* takes a cheaper route. A strong *teacher* reasoning model writes out worked solutions once; a smaller *student* is then trained, with plain supervised learning, to reproduce them. The DeepSeek-R1 report popularised this recipe for reasoning: the large RL-trained model produced traces from which a family of much smaller dense models was fine-tuned, and the authors reported that these distilled students outperformed comparable small models trained with RL alone.

From a systems point of view distillation is attractive for a simple reason: it moves almost all of the expensive inference *out of the training loop*. Teacher generation becomes an offline, embarrassingly parallel batch job whose output is a file; student training is ordinary fine-tuning on that file, with fixed data, reproducible epochs and none of RL's generator/trainer choreography. The source book reports that its full distillation run of Qwen3-0.6B fits in roughly 15 GB of memory and finishes in about three hours on a desktop GB10-class machine, whereas a few GRPO rounds on the same hardware took far longer and several times the memory. This chapter builds that pipeline end to end and then asks the systems questions it raises: how to store soft targets, how to train on long traces, and how to make reasoning *cheaper* rather than merely smaller.

20.1 What distillation is

RM 8.1

Distillation trains a *student* model to imitate a *teacher*. For reasoning models the thing being imitated is not only the final answer but the whole trace—the exploratory, self-correcting chain of thought that precedes it. There are two families of methods, distinguished by *what* the student sees of the teacher (Fig. 20.2).

Hard distillation. The teacher generates text; the student is trained with the ordinary next-token cross-entropy loss on that text. The targets are the teacher's *sampled tokens*, i.e. one-hot vectors. Hard distillation is just supervised fine-tuning (Chapter 10) on synthetic data, and it needs nothing but the teacher's output text—so it works with closed models behind an API and across model families with different tokenizers. DeepSeek-R1's small distilled models were produced this way.

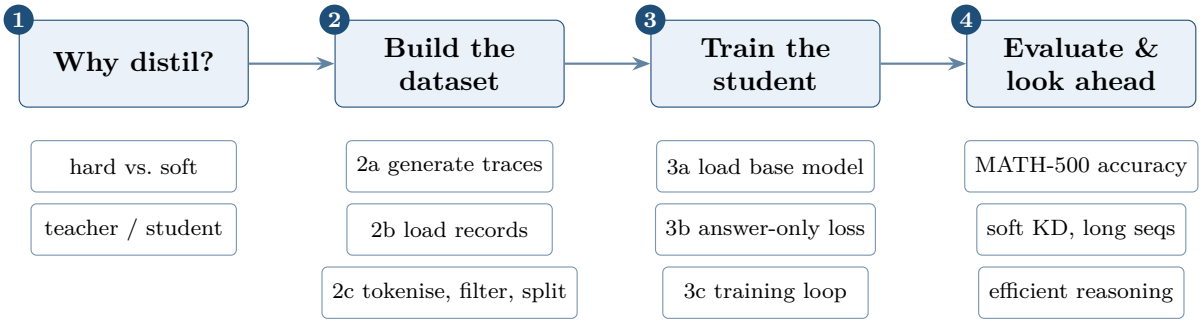


Figure 20.1: Roadmap. Stages 1–4 follow the distillation workflow (motivation, dataset construction, student training, evaluation); the lower boxes are the concrete steps of each stage. Stage 4 also contains this book's systems extensions.

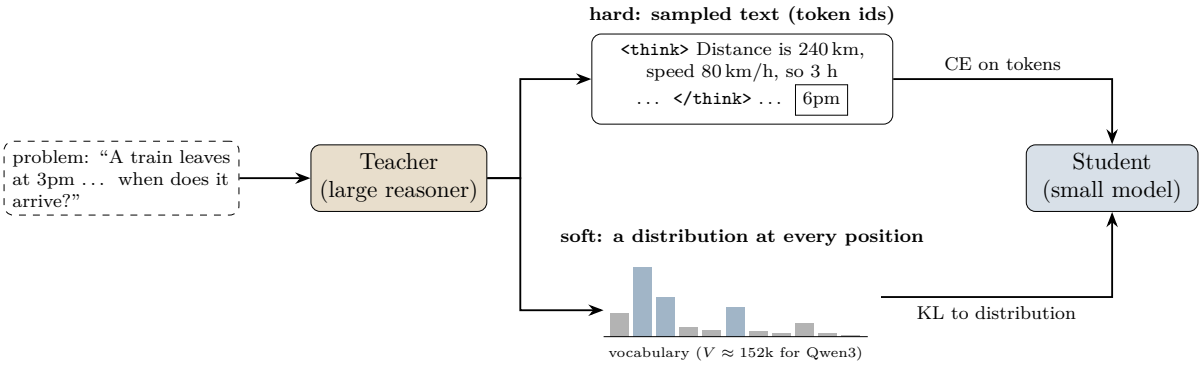


Figure 20.2: Hard distillation trains the student on the teacher's sampled tokens (one-hot targets); soft distillation trains it to match the teacher's full next-token distribution, which requires the teacher's logits and a shared vocabulary.

Soft distillation. The student is trained to match the teacher's full *distribution* over the vocabulary at every position, by minimising a divergence such as

$$\mathcal{L}_{\mathrm{KD}} = \frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} \mathrm{KL}(p_T(\cdot \mid y_{<t}) \,\|\, p_S(\cdot \mid y_{<t})), \tag{20.1}$$

where $\mathcal{A}$ is the set of answer positions. A soft target says not only "the next token is *Subtract*" but also "*Multiply* was plausible and *banana* was not". That extra information—Hinton, Vinyals and Dean (2015) called it *dark knowledge*—usually makes each training token more informative. The price is access to the teacher's logits and a shared vocabulary.

Combined. The classic knowledge-distillation objective mixes the two: $\mathcal{L} = \alpha\,\mathcal{L}_{\mathrm{CE}} + (1-\alpha)\,\tau^2\,\mathcal{L}_{\mathrm{KD}}^{(\tau)}$, where $\mathcal{L}_{\mathrm{KD}}^{(\tau)}$ compares distributions softened by a temperature τ and the factor τ^2 keeps gradient magnitudes comparable across temperatures.

For LLMs hard distillation dominates in practice, for three reasons: frontier teachers rarely expose full distributions; student and teacher must share a tokenizer for Eq. (20.1) to even be defined; and soft targets are enormous (see the napkin math below). This chapter therefore follows the DeepSeek-R1 recipe—hard distillation—for the main pipeline, and returns to soft distillation as a systems problem in Section 20.9.

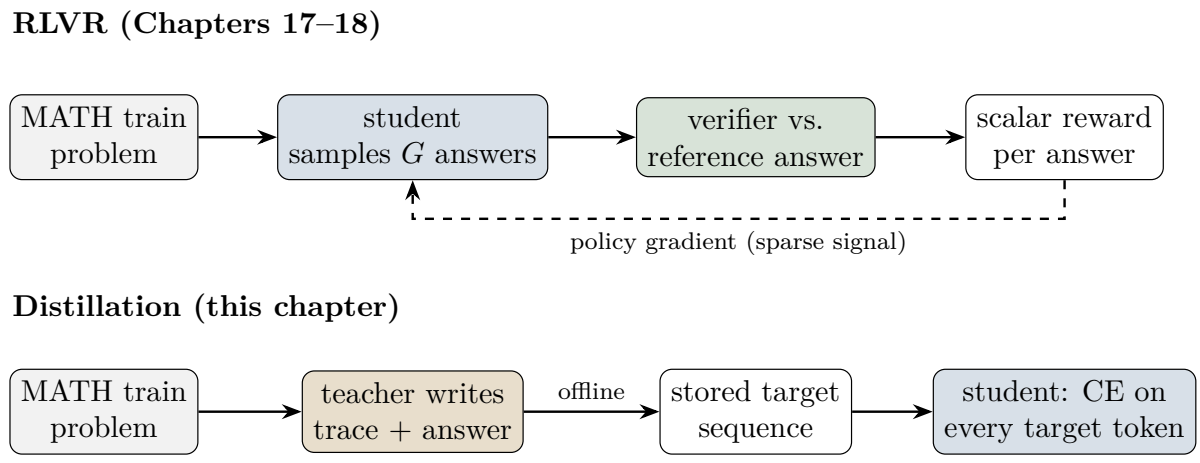


Figure 20.3: Where the training signal comes from. RLVR compares the student's own final answer with a reference; distillation compares the student's token predictions with a teacher-written solution. The teacher runs once, offline.

Napkin Math: Text versus distributions

A reasoning trace of 3,000 tokens stored as token ids is $3000 \times 4\,\text{B} = 12\,\text{KB}$. The same trace with the full Qwen3 next-token distribution at every position in bf16 is $3000 \times 151{,}936 \times 2\,\text{B} \approx 0.91\,\text{GB}$. For 12,000 training problems that is ≈144 MB of text versus ≈10.9 TB of soft targets—a factor of about 76,000. Keeping only the top 32 entries per position (fp16 value + int32 index) shrinks it to ≈6.9 GB, which is why practical soft distillation stores sparse top-k targets or runs the teacher online.

Pitfall: Terms of service

Many commercial model providers' terms restrict using their outputs to train competing models. Check the licence of every teacher before generating a distillation set. Open-weight teachers (DeepSeek-R1, the larger Qwen3 models) are a common choice partly for this reason.

20.2 Generating a distillation dataset

RM 8.2

Our student is the Qwen3-0.6B *base* model used throughout Part V, and our training problems are the ≈12,000 problems of the MATH training split that do not overlap with the MATH-500 test set—the same pool used for RLVR in Chapters 17 and 18. What changes is the source of the supervision (Fig. 20.3). In RLVR the student samples its own answers and a verifier compares each final answer with the reference to produce a scalar reward. In distillation a teacher writes a complete solution and the student is trained to reproduce it *token by token*: the supervision is dense (every token is a target) and does not require the student to ever find a correct answer by itself.

A practical virtue is visible in the figure: teacher generation is decoupled from training. The source book generated its 12,000 traces by calling the 671B-parameter DeepSeek-R1 through a hosted API (at a cost it reports as roughly $50), and publishes the result so that readers can skip this step. We will treat generation as a batch-inference job that we would want to run ourselves.

20.2.1 Teacher generation as a batch job

Napkin Math: How big is the generation job?

12,000 problems × ≈3,000 output tokens is ≈ 36M generated tokens (the prompts add little). For a self-hosted 671B mixture-of-experts teacher with ≈37B active parameters per token, decode costs about $2 \times 37\text{B} \approx 74\,\text{GFLOP}$ per token, i.e. $\approx 2.7 \times 10^{18}$ FLOPs in total—trivial compute for a multi-node serving deployment. The binding constraint is memory bandwidth and KV capacity, so the job should run at the largest batch size the deployment supports, with throughput-oriented scheduling (Chapters 12 and 13). It has no latency requirement at all.

A generation script for a large job must be *resumable* (it will be interrupted), *idempotent* (re-running must not duplicate records), and *robust* to transient errors (rate limits, timeouts). Listing 20.1 shows the shape of such a script; the teacher is any callable, e.g. a thin client around an OpenAI-compatible endpoint served by vLLM or SGLang.

Listing 20.1: Resumable, verifiable teacher-trace generation (`llmsys/distill.py`).

```python
def generate_teacher_dataset(problems, teacher_fn, out_path,
                             verify_fn=None, max_retries=2):
    done = set()
    if os.path.exists(out_path):                        # resume: skip finished ids
        with open(out_path) as f:
            done = {json.loads(line)["id"] for line in f if line.strip()}
    stats = dict(skipped_existing=0, written=0, rejected=0, failed=0)
    with open(out_path, "a") as f:                      # append-only JSONL
        for i, prob in enumerate(problems):
            pid = prob.get("id", i)
            if pid in done:
                stats["skipped_existing"] += 1
                continue
            out = None
            for _ in range(max_retries + 1):
                try:
                    out = teacher_fn(render_math_prompt(prob["problem"]))
                    break
                except Exception:                       # rate limit, timeout, ...
                    out = None
            if out is None:
                stats["failed"] += 1
                continue
            if verify_fn and not verify_fn(out["answer"], prob["answer"]):
                stats["rejected"] += 1                  # rejection sampling
                continue
            rec = dict(id=pid, problem=prob["problem"], gtruth=prob["answer"], **out
    )
            f.write(json.dumps(rec) + "\n")
            stats["written"] += 1
    return stats
```

Systems Lens: Rejection sampling is cheap quality control

Passing a `verify_fn` (the verifier of Chapter 14) keeps only traces whose final answer is correct. The cost is the fraction of teacher tokens thrown away; the benefit is that the student never imitates a wrong solution. The book's main experiment trains on all traces (the teacher is right about 90% of the time on these problems), so rejection sampling is a natural ablation. With a per-problem budget one can also sample several traces and keep, say, the shortest correct one—a first step toward *efficient* reasoning (Section 20.11).

20.3 Loading the distillation data

RM 8.3

Each record of the published dataset contains four fields: the problem statement, the ground-truth answer, the teacher's reasoning trace, and the teacher's final user-facing response (which ends in a `\boxed{}` answer). Different generators name these fields differently, so our loader maps them onto one schema—`problem`, `gtruth`, `reasoning`, `answer`—via `normalize_record`. Because the file is large (roughly 100 MB of JSON for 12k records), the download helper should cache a local copy and reuse it on later runs; any HTTP client with a fallback mirror will do.

The training target combines the trace and the response. Following the convention of Qwen3 and DeepSeek-R1 reasoning models, we wrap the trace in think tags:

Listing 20.2: Turning a teacher record into the student's target text.

```
def format_distilled_answer(record, use_think=True):
    answer = str(record.get("answer", "")).strip()
    if not answer:
        raise ValueError("record has an empty 'answer' field")
    reasoning = str(record.get("reasoning", "")).strip()
    if use_think:
        return f"<think>{reasoning}</think>\n\n{answer}"
    return f"{reasoning}\n\n{answer}" if reasoning else answer
```

The tags are not required for distillation to work, but they cleanly separate the part of the output a user interface may want to hide (the trace) from the part it shows. Since the records also carry the ground truth, we can measure the *teacher's* accuracy on the training set with the verifier of Chapter 14; the source book reports about 90.6% for DeepSeek-R1 on these 12k problems (and about 91% on MATH-500), far above the 0.6B base model's roughly 15% on MATH-500. The gap between teacher and student accuracy is the headroom distillation tries to close.

20.4 Building training examples

RM 8.4

In RLVR every prompt was used once and its rollouts were discarded, so preprocessing happened on the fly. Distillation revisits the same examples for several epochs, so it pays to do the work once: render prompts, format targets, tokenise, and store token ids together with the prompt length. We proceed in three stages: load the tokenizer, tokenise the whole dataset, then filter and split it.

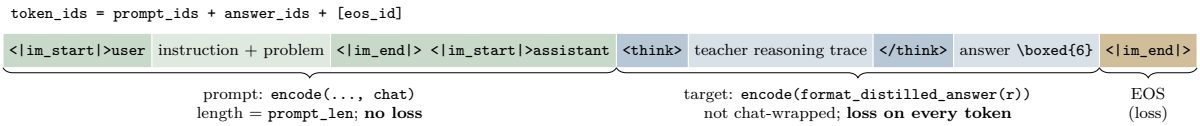


Figure 20.4: Layout of one distillation example. Only the prompt is chat-wrapped. We store the prompt length so that the loss can be restricted to the teacher's tokens and the final end-of-turn token.

RM 8.4.1

20.4.1 The tokenizer and the chat format

We use the tokenizer of the Qwen3 *reasoning* checkpoint because its vocabulary already contains `<think>` and `</think>` as single special tokens (ids 151667 and 151668); with the base tokenizer the tags would be split into several ordinary tokens, which works but is wasteful and less clean. The prompt is wrapped in Qwen's ChatML template (Chapter 11):

```
<|im_start|>user
Solve the following math problem. ... Problem: ...<|im_end|>
<|im_start|>assistant
```

The final line is the *generation prompt*: the model's job begins right after it. The target answer must *not* be chat-wrapped—otherwise a second `<|im_start|>assistant` header would appear in the middle of the sequence. The full training sequence is therefore

$$\underbrace{\text{chat-wrapped prompt}}_{\texttt{prompt_len tokens}} \;\|\; \underbrace{\texttt{<think>}\ \text{trace}\ \texttt{</think>}\ \text{answer}}_{\text{raw, not wrapped}} \;\|\; \texttt{<|im_end|>},$$

where the final end-of-turn token teaches the student to stop. Neither the chat template nor the reasoning tokenizer is strictly necessary, but matching the conventions of Qwen3's own reasoning models keeps training and evaluation consistent: whichever evaluation script you use later must load the same tokenizer variant.

20.4.2 Formatting and tokenising the dataset

RM 8.4.2

Listing 20.3 applies three steps to every record: render and tokenise the prompt, format and tokenise the target, concatenate both with the end token and remember `prompt_len`. Records with missing fields are counted and skipped rather than crashing the job. The encoders are passed in as functions, so the same code works with the Qwen3 tokenizer (in `run.py`) and with the lab's character tokenizer.

Listing 20.3: Tokenising the dataset once (`llmsys/distill.py`).

```
def build_examples(records, encode_prompt, encode_answer, eos_id,
                   use_think=True, render=render_math_prompt):
    examples, skipped = [], 0
    for rec in records:
        try:
            p = encode_prompt(render(rec["problem"]))           # chat-wrapped
            a = encode_answer(format_distilled_answer(rec, use_think))  # raw
        except (KeyError, TypeError, ValueError):
            skipped += 1
            continue
        ids = p + a + [eos_id]
        if len(ids) < 2 or len(p) == 0:
            skipped += 1
```

```
            continue
        examples.append({"token_ids": ids, "prompt_len": len(p)})
    return examples, skipped
```

Always decode one example back to text and read it. It should show the chat-wrapped prompt, the trace inside think tags, the boxed answer, and the end-of-turn token, in that order and with no duplicated headers.

20.4.3 Filtering by length and splitting

RM 8.4.3

Reasoning traces are long and heavy-tailed. On the DeepSeek-R1 MATH traces the source book reports an average of about 2,950 tokens per formatted example, a minimum of 236, and a maximum above 42,000. Training cost per example grows at least linearly with length (MLPs, projections) and partly quadratically (attention), and peak activation memory is set by the *longest* sequence in a step, so outliers are expensive. The book caps examples at 2,048 tokens, which removes about 44% of the data (6,695 of 12,000 remain, averaging about 1,180 tokens), shuffles with a fixed seed and holds out a deliberately tiny validation set of 25 examples to keep evaluation cheap. Our helpers compute the same quantities (Listing 20.4).

Listing 20.4: Length statistics, filtering and splitting.

```
def length_stats(examples, answer_only=False):
    lens = [len(e["token_ids"]) - (e["prompt_len"] if answer_only else 0)
            for e in examples]
    srt = sorted(lens)
    q = lambda f: srt[min(len(srt) - 1, int(f * len(srt)))]
    return dict(n=len(lens), mean=sum(lens) / len(lens), min=srt[0], max=srt[-1],
                p50=q(0.5), p90=q(0.9), p99=q(0.99),
                argmin=lens.index(srt[0]), argmax=lens.index(srt[-1]))

def filter_by_max_len(examples, max_len):
    return [e for e in examples if len(e["token_ids"]) <= max_len]

def split_train_val(examples, val_size, seed=123):
    ex = list(examples)
    random.Random(seed).shuffle(ex)
    return ex[val_size:], ex[:val_size]
```

Pitfall: A length cap is a difficulty filter

Long traces are disproportionately the solutions to hard problems (and the teacher's failed, meandering attempts). Dropping everything above 2,048 tokens therefore removes most of the hardest problems and teaches the student a style that rarely needs long reasoning. That is sometimes desirable (short answers are cheap to serve) and sometimes the reason a distilled model plateaus. Inspect accuracy by MATH difficulty level before and after filtering, and prefer *packing* and *activation checkpointing* (Section 20.10) to raise the cap when memory allows.

A natural check, and the subject of an exercise, is whether the random split produced train and validation sets with similar length distributions; with only 25 validation examples it may not, and validation loss is length-sensitive.

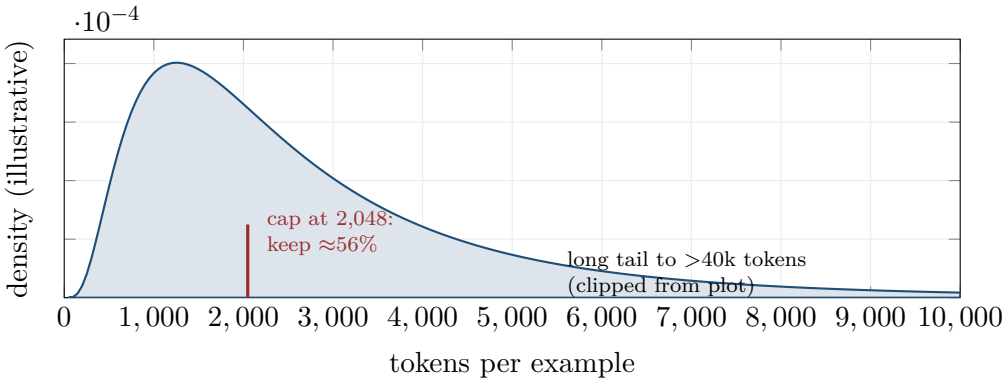


Figure 20.5: Schematic length distribution of reasoning traces (a log-normal shape, not the measured histogram) and the effect of a length cap. A cap is the single most effective knob for fitting distillation into a small GPU, but it biases the data (see the pitfall below).

20.5 Loading the student

RM 8.5

We start from the pretrained Qwen3-0.6B *base* model rather than from an RL-trained checkpoint. That keeps the experiment clean—any gain is attributable to the distilled traces—and mirrors the common practical setting of adapting a general base model with teacher data. The base model's own tokenizer is not used; the reasoning tokenizer from Section 20.4.1 has the same vocabulary plus the think tags already defined as special tokens, and the embedding matrix already has rows for them. With our core code (Chapter 6), loading is one line:

```
model = load_qwen3_from_dir("qwen3-base", "0.6B", dtype=torch.bfloat16, device=
    device)
```

Napkin Math: Will it fit?

Qwen3-0.6B has ≈0.60B parameters (with tied embeddings). Full fine-tuning with AdamW in mixed precision costs ≈ 16 B/param, i.e. ≈9.5 GB for weights, gradients and optimizer state. Activations for one 2,048-token sequence add a few GB; the logits alone are $2048 \times 151{,}936 \times 4\,\text{B} \approx 1.2\,\text{GB}$ in fp32, plus the same again for their gradient. The source book's measured peak of about 15 GB at batch size 1 is consistent with this budget. Doubling the cap to 4,096 tokens roughly doubles activation and logit memory—this is where the techniques of Section 20.10 pay off.

20.6 The training and validation losses

RM 8.6

Distillation's loss is the familiar next-token cross-entropy of Chapter 7, restricted to the target part of each sequence. It is worth connecting it explicitly to the log-probabilities of Chapters 16 and 17. For an example with prompt x and target $y = (y_1, \ldots, y_n)$,

$$\mathcal{L}_{\text{CE}} = -\frac{1}{n}\sum_{t=1}^{n} \log p_\theta(y_t \mid x, y_{<t}) = -\frac{1}{n}\log p_\theta(y \mid x), \tag{20.2}$$

the *negative average log-probability* of the teacher's tokens. Summed log-probabilities grow with length and cannot be compared across examples; dividing by n gives a per-token quantity. If a model assigns joint log-probability -16.6 to a five-token continuation, its cross-entropy on that continuation is $16.6/5 = 3.32$ nats per token.

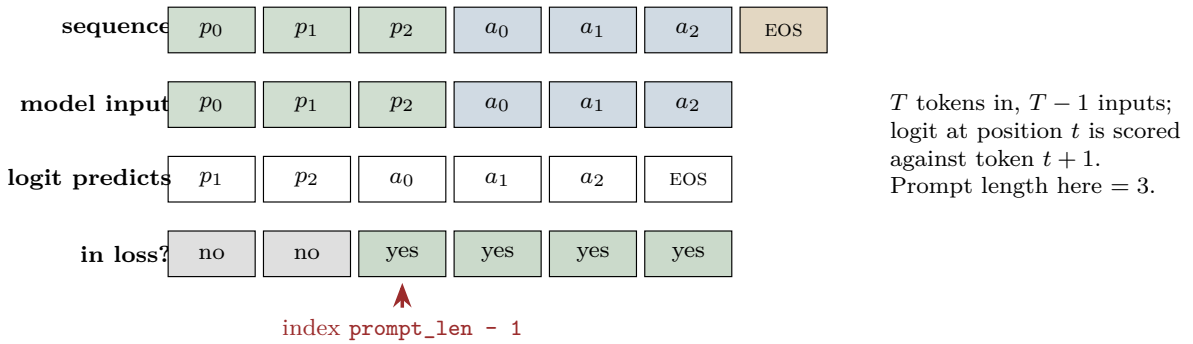


Figure 20.6: Answer-only loss alignment. The model sees all tokens but the last; the logit at position t predicts token $t+1$. The first answer token a_0 (at index `prompt_len`) is therefore predicted by the logit at index `prompt_len`-1, which is where the loss starts.

We compute the loss only over the answer: the prompt is context the student is given, not something it should be penalised for failing to predict (Fig. 20.6). Because of the one-token shift, the slice starts at logit index `prompt_len - 1`:

Listing 20.5: Answer-only cross-entropy for one example.

```
def answer_only_loss(model, example, device="cpu"):
    ids = torch.tensor(example["token_ids"], dtype=torch.long, device=device)
    logits = model(ids[:-1][None])[0]            # (T-1, V)
    start = max(example["prompt_len"] - 1, 0)    # logit that predicts answer[0]
    return F.cross_entropy(logits[start:].float(), ids[1:][start:])
```

The lab verifies Eq. (20.2) numerically: `answer_only_loss` equals $-$`sequence_logprob`$/n$ computed with the generic helper of Chapter 16. In practice `F.cross_entropy` is preferred because it fuses log-softmax and gather and never materialises the full log-probability tensor separately.

Pitfall: Off by one

Starting the slice at `prompt_len` instead of `prompt_len - 1` silently drops the first answer token from the loss—which, with think tags, is the `<think>` token the student must learn to emit first. The model trains fine and then forgets to open its reasoning at inference time. Test the alignment on a toy example where you can enumerate every position.

Validation loss. A helper averages `answer_only_loss` over a list of examples in `eval` mode under `torch.no_grad()`, restoring training mode afterwards so it is safe to call inside the loop. Training loss is measured on the example just optimised and is noisy; validation loss on a fixed held-out set is the cleaner signal of generalisation.

20.6.1 Batching the loss correctly

The book's loop processes one example per step to keep memory low. Batching several examples requires padding (Chapter 10) and a decision about normalisation that is easy to get wrong. With right padding and -100 (ignored) targets on prompt and pad positions, there are two reasonable reductions:

$$\mathcal{L}_{\text{token}} = \frac{\sum_i \sum_{t \in \mathcal{A}_i} \ell_{i,t}}{\sum_i |\mathcal{A}_i|}, \qquad \mathcal{L}_{\text{seq}} = \frac{1}{B} \sum_i \frac{1}{|\mathcal{A}_i|} \sum_{t \in \mathcal{A}_i} \ell_{i,t}. \tag{20.3}$$

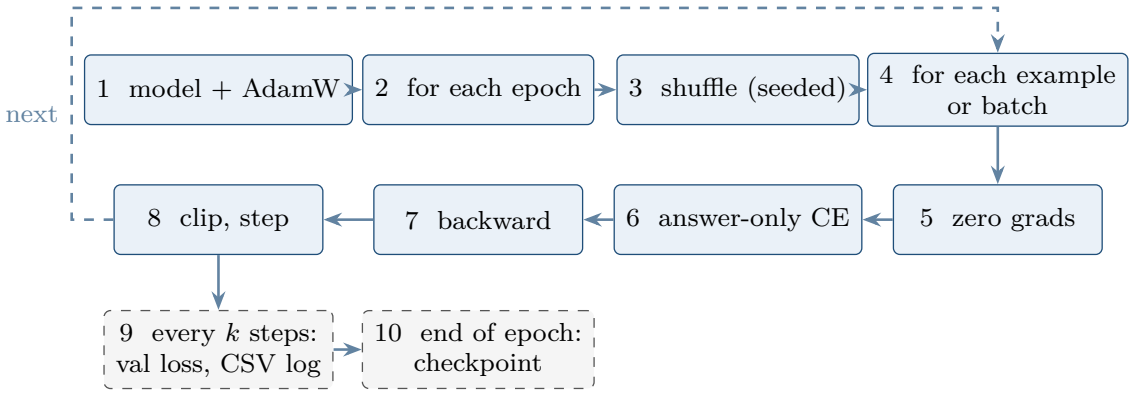


Figure 20.7: The distillation training loop. Steps 5–8 are standard supervised fine-tuning; the dashed arrow returns to step 4 for the next example; steps 9–10 are optional bookkeeping (validation loss, metrics file, checkpoints).

$\mathcal{L}_{\text{token}}$ (what cross-entropy with `ignore_index=-100` gives on the flattened batch) weights each *token* equally, so long traces dominate the gradient; $\mathcal{L}_{\text{seq}}$ weights each *example* equally and reproduces the average of per-example losses. Neither is wrong, but switching between them changes the effective learning rate per example by up to the ratio of the longest to shortest trace—and they interact with gradient accumulation exactly as described in Section 19.6.

Listing 20.6: Padded batch with prompt and pad positions masked out.

```python
def collate_answer_only(batch, pad_id, device="cpu"):
    T = max(len(e["token_ids"]) for e in batch)
    inp = torch.full((len(batch), T - 1), pad_id, dtype=torch.long)
    tgt = torch.full((len(batch), T - 1), IGNORE, dtype=torch.long)
    for i, e in enumerate(batch):
        ids = torch.tensor(e["token_ids"], dtype=torch.long)
        n = len(ids)
        inp[i, : n - 1] = ids[:-1]
        tgt[i, : n - 1] = ids[1:]
        tgt[i, : max(e["prompt_len"] - 1, 0)] = IGNORE   # no loss on the prompt
    return inp.to(device), tgt.to(device), int((tgt != IGNORE).sum())
```

Right padding is safe for training a causal model: real tokens never attend to later positions, so pads cannot leak into real positions' outputs.

20.7 The distillation training loop

RM 8.7

The loop (Fig. 20.7) is the GRPO loop of Chapter 17 with the policy-gradient objective replaced by cross-entropy and with epochs over a fixed dataset instead of fresh rollouts every step.

Listing 20.7: Training loop for hard distillation (`llmsys/distill.py`, abridged).

```python
def train_distillation(model, train_examples, val_examples, device="cpu", epochs=2,
                       lr=5e-6, grad_clip=1.0, seed=123, log_every=50,
                       batch_size=1, pad_id=0, checkpoint_dir=None, csv_path=None):
    opt = torch.optim.AdamW(model.parameters(), lr=lr)
    rng, step, rows = random.Random(seed), 0, []
    model.train()
    for epoch in range(1, epochs + 1):
        order = list(train_examples)
```

```
        rng.shuffle(order)                                    # new order each epoch
        for s in range(0, len(order), batch_size):
            batch = order[s:s + batch_size]
            step += 1
            opt.zero_grad(set_to_none=True)
            loss = (answer_only_loss(model, batch[0], device) if batch_size == 1
                    else batch_answer_only_loss(model, batch, pad_id, device))
            loss.backward()
            if grad_clip is not None:
                torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
            opt.step()
            if log_every and step % log_every == 0:
                val = evaluate_examples(model, val_examples, device)
                row = dict(epoch=epoch, step=step, train_loss=loss.item(), val_loss=
    val)
                rows.append(row)
                if csv_path:
                    append_csv(csv_path, row)
        if checkpoint_dir:                                    # one checkpoint per
    epoch
            torch.save(model.state_dict(),
                       f"{checkpoint_dir}/distill-step{step:06d}-epoch{epoch}.pth")
    return rows
```

The hyperparameters are typical for full fine-tuning of a small pretrained model: a learning rate around 5×10^{-6} to 10^{-5} with AdamW, gradient clipping at norm 1.0 (as in the GRPO chapters) to tame occasional large gradients from unusual examples, and a validation interval chosen to balance feedback against evaluation overhead. A smoke test on ten examples for two epochs should show both losses falling; a real run uses the whole filtered set for several epochs. Writing metrics to a CSV file after every evaluation makes the run inspectable while it is still going and plottable afterwards.

Systems Lens: Batch size 1 is a memory decision, not a throughput one

At batch size 1 every step is a single sequence of ∼1,200 tokens; for a 0.6B model on a modern GPU that leaves the matrix units badly underused, and the optimizer step (which touches every parameter) is amortised over very few tokens. Throughput improves with (i) gradient accumulation to reach a larger effective batch, (ii) padded mini-batches bucketed by length, or best (iii) *packed* rows with block-diagonal attention (Section 19.6), which in the lab reach >95% real tokens versus ∼40–70% for padding on heavy-tailed lengths. Remember to keep the loss normalisation fixed when you change batching.

20.8 Evaluating the distilled model

RM 8.8

Evaluation has two layers. The *learning curves*—training and validation loss over steps, plotted from the CSV log with a secondary axis marking epochs—tell us whether optimisation is healthy. In the book's three-epoch run on the DeepSeek-R1 traces the validation loss drops steeply at first and then flattens, the usual sign that further epochs give diminishing returns. The training curve is noisier because each point is a single example.

The second layer is *task accuracy* on MATH-500 with the verifier of Chapter 14, evaluating

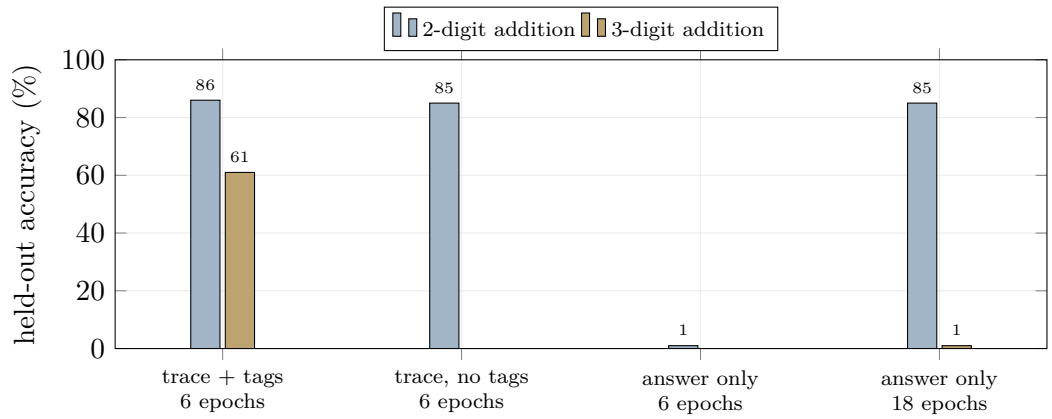


Figure 20.8: Lab ablation on the synthetic addition task (measured on CPU in this environment with `bench.py`, seed 0, 1,500 training problems, 100 unseen test problems, a 4-layer $d = 128$ student): held-out exact-match accuracy of students trained on the teacher's full trace with think tags, on the trace without tags, and on the final answer only. Missing bars were not run.

each epoch's checkpoint with the reasoning tokenizer and a generous generation budget (the book uses 4,096 new tokens). The source book reports the following (we quote its numbers; they were not reproduced here):

Model	Epoch	Final val. loss	MATH-500
Qwen3-0.6B base (no distillation)	–	–	15.2%
Qwen3-0.6B official reasoning model	–	–	48.2%
Distilled from DeepSeek-R1 traces	1 / 2 / 3	0.544 / 0.535 / 0.534	30.6 / 32.4 / 33.6%
Distilled from Qwen3-235B-A22B traces	1 / 2 / 3	0.404 / 0.396 / 0.395	45.0 / 43.8 / 44.2%

Three observations. First, a few hours of supervised training on ∼6.7k filtered traces roughly doubles to triples the base model's accuracy. Second, the students remain far below their teachers (both above 90% on MATH-500): a 0.6B model cannot absorb everything a 671B or 235B model knows, and larger students (4B, 8B) would close more of the gap at higher training cost. Third, the *same-family* teacher works markedly better: Qwen3-235B-A22B traces yield a lower validation loss and a student nearly matching Qwen's own distilled 0.6B model, which was trained on far more data. A plausible explanation is closer alignment of tokenisation, formatting and style, which makes the targets easier for the student to imitate—a reminder that "teacher quality" includes how learnable its outputs are, not only how correct.

Key Idea: Distillation in one sentence

Distillation is supervised fine-tuning on teacher-written reasoning, with an answer-only cross-entropy that equals the negative mean log-probability of the teacher's tokens; it buys a large fraction of the teacher's reasoning ability at a small fraction of RL's cost, bounded by the student's capacity and by how imitable the teacher is.

A natural ablation (Exercise 20.2 and the lab) asks how much the explicit think tags matter. In our synthetic lab task we can go further and ask how much the trace itself matters, by training students on the full trace, on the trace without tags, and on the answer alone.

The result (Fig. 20.8) is instructive. With the same 1,500 problems and six epochs, students trained on the trace reach 86% (with tags) and 85% (without) exact-match accuracy on unseen 2-digit problems, while the answer-only student reaches 1%. Training the answer-only student

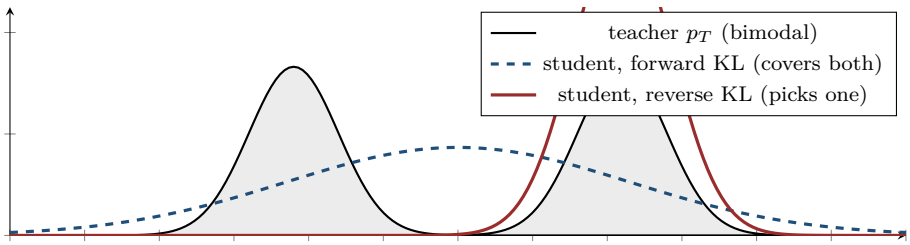


Figure 20.9: A unimodal student fitted to a bimodal teacher. Forward KL (classic KD) spreads mass over both modes and the valley between them; reverse KL (MiniLLM, on-policy distillation) locks onto one mode. Curves are schematic.

three times longer—roughly equalising the number of target tokens seen—lets it catch up on 2-digit problems (85%), but on 3-digit problems the trace-trained student reaches 61% after six epochs while the answer-only student is still at 1% after eighteen. A second seed of the 2-digit run gave 72% (trace) and 4% (answer only), so the trace effect is robust while the exact numbers vary by seed. The trace makes the task *learnable* for a small model by turning one hard mapping into a sequence of easy local steps—the same intuition behind distilling reasoning traces rather than answers. The price appears at inference: the traced student emits about 2.7–3.3× more tokens per answer (31.5 vs. 11.5 target tokens for 2 digits, 41.0 vs. 12.6 for 3 digits), which is exactly the cost that Section 20.11 tries to reduce.

20.9 Soft distillation as a systems problem

When teacher and student share a tokenizer—as with the Qwen3 family—soft distillation becomes possible and often more sample-efficient. Its difficulties are almost entirely systems difficulties: where the teacher's distributions come from, how they are stored, and how the loss is computed without materialising enormous tensors.

20.9.1 Forward and reverse KL

Equation (20.1) uses the *forward* KL, $\mathrm{KL}(p_T \| p_S)$, which penalises the student for putting low probability where the teacher puts mass; it is *mass-covering*, pushing the student to spread probability over every mode the teacher considers plausible. The *reverse* KL, $\mathrm{KL}(p_S \| p_T)$, penalises the student for putting mass where the teacher does not; it is *mode-seeking*. A small student that cannot represent all of a teacher's modes behaves very differently under the two (Fig. 20.9): forward KL produces a broad compromise that may assign probability to regions neither mode supports, while reverse KL commits to one mode. MiniLLM (Gu et al., 2024) argued for reverse KL when distilling generative LLMs, and on-policy methods such as GKD (Agarwal et al., 2024) compute the divergence on sequences sampled *from the student*, which fixes the train/test mismatch of training only on teacher-written prefixes.

Listing 20.8: Token-level KD loss with temperature and direction, and a sparse top-k variant (`llmsys/distill.py`).

```
def kd_loss(student_logits, teacher_logits, mask=None, temperature=1.0,
            direction="forward"):
    T = temperature
    ls = F.log_softmax(student_logits.float() / T, dim=-1)
    lt = F.log_softmax(teacher_logits.float() / T, dim=-1)
    if direction == "forward":
```

```
        kl = (lt.exp() * (lt - ls)).sum(-1)        # KL(p_T || p_S)
    else:
        kl = (ls.exp() * (ls - lt)).sum(-1)        # KL(p_S || p_T)
    m = torch.ones_like(kl) if mask is None else mask.float()
    return (kl * m).sum() / m.sum().clamp(min=1) * T * T

def topk_teacher(teacher_logits, k):
    lp = F.log_softmax(teacher_logits.float(), dim=-1)
    vals, idx = torch.topk(lp, k, dim=-1)
    return vals.to(torch.float16), idx.to(torch.int32)   # 6 bytes per entry

def sparse_kd_loss(student_logits, topk_logp, topk_idx, mask=None):
    ls_k = F.log_softmax(student_logits.float(), -1).gather(-1, topk_idx.long())
    pt = torch.softmax(topk_logp.float(), dim=-1)          # renormalise over k
    kl = (pt * (pt.clamp_min(1e-12).log() - ls_k)).sum(-1)
    m = torch.ones_like(kl) if mask is None else mask.float()
    return (kl * m).sum() / m.sum().clamp(min=1)
```

20.9.2 Offline, online, and sparse teachers

There are three ways to supply p_T:

Offline dense. Precompute and store full distributions. As the napkin math in Section 20.1 showed, this is impractical beyond toy scales.

Offline sparse. Store the top-k log-probabilities and token ids per position ($k = 8$–64), renormalise over the support at training time (`sparse_kd_loss`). For peaked reasoning distributions most mass sits in a handful of tokens, so small k loses little; the lab checks that $k = V$ recovers the dense loss exactly (up to fp16 storage).

Online. Run the teacher's forward pass inside the training step. No storage, exact distributions, and on-policy variants become possible—but every training step now pays for a teacher forward pass (for a teacher 10× the student's size, more than the student's own forward and backward together), and both models must fit in memory or be served separately.

Napkin Math: Online teacher cost

Training the student costs $\approx 6N_S$ FLOPs per token; an online teacher forward pass costs $\approx 2N_T$. With $N_S = 0.6$B and an 8B teacher, the teacher adds $16/3.6 \approx 4.4\times$ the student's training compute. With a 235B-A22B mixture-of-experts teacher it adds $2 \times 22/3.6 \approx 12\times$ (counting active parameters) and needs a multi-GPU serving deployment next to the trainer—the same placement questions as RL (Section 19.2).

20.10 Training on long traces

Raising the 2,048-token cap is the most direct way to keep hard problems in the data (Section 20.4.3). Three memory consumers grow with sequence length T:

Activations. Per layer roughly linear in T with FlashAttention-style kernels (no T^2 score matrix stored). Activation checkpointing (Chapter 7) trades a second forward pass for storing only layer inputs.

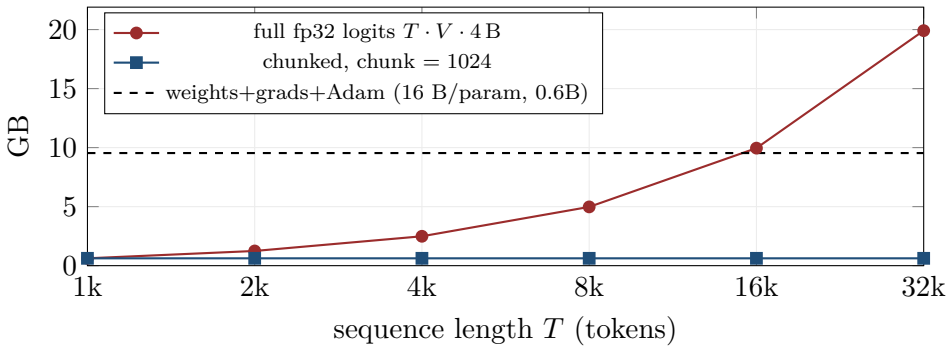


Figure 20.10: Memory of the output logits for Qwen3's 151,936-token vocabulary versus sequence length (computed, not measured). Beyond ∼16k tokens the logits alone exceed the entire optimizer state of the 0.6B student; chunking caps them.

Logits. The output layer produces $T \times V$ values; with $V \approx 152$k the logits are the *largest single activation* in a small model. At $T = 16{,}384$ in fp32 that is $\approx 10\,\mathrm{GB}$, plus a gradient of the same size. The fix is to never materialise them all: compute the loss in chunks of positions, so that only chunk $\times V$ logits exist at a time. Listing 20.9 is a minimal version; fused kernels (e.g. Liger Kernel's fused linear cross-entropy) additionally compute the gradient inside the chunk loop so that the chunk need not be kept for backward.

Listing 20.9: Chunked cross-entropy: peak logit memory is `chunk`$\times V$ instead of $T \times V$.

```python
def chunked_cross_entropy(hidden, out_head, targets, chunk=1024):
    total = hidden.new_zeros((), dtype=torch.float32)
    count = (targets != IGNORE).sum().clamp(min=1)
    for s in range(0, hidden.shape[0], chunk):
        logits = out_head(hidden[s:s + chunk]).float()        # (chunk, V)
        total = total + F.cross_entropy(logits, targets[s:s + chunk],
                                        ignore_index=IGNORE, reduction="sum")
    return total / count
```

Note that with autograd, Listing 20.9 still keeps each chunk's logits alive for the backward pass unless each chunk is wrapped in activation checkpointing; the chunking reduces the *transient* peak (softmax intermediates, fp32 copies) and is the stepping stone to a fused kernel.

Attention across devices. Beyond what one GPU can hold, the sequence itself is split: context parallelism (ring attention) shards the sequence across GPUs and circulates key/value blocks (Chapter 8). For a 0.6B student this is rarely needed below ∼32k tokens; for larger students on 64k-token traces it is standard.

Packing. Short traces packed into long rows (block-diagonal attention, Section 19.6) remove padding; combined with a token-budget batch size this makes step time and memory nearly independent of the length mix.

20.11 Efficient reasoning: making the student cheaper to run

A distilled 0.6B model is cheap per token, but the serving cost of a reasoning model is (cost per token) × (tokens per answer), and traces of thousands of tokens can make a small

model more expensive per query than a larger model that answers directly. Several levers act on the second factor:

- **Choose shorter traces.** When generating the dataset, sample several traces per problem and keep the shortest *correct* one; or filter by length as in Section 20.4.3, accepting the difficulty bias. The student inherits the teacher's verbosity, so curating for concision is the cheapest compression.
- **Budget control at inference.** "Budget forcing" (Muennighoff et al., 2025, the *s1* paper) caps thinking by forcibly closing the think block, or extends it by appending a token like "Wait"; production systems expose a reasoning-effort setting or a thinking-token budget and let the application choose.
- **Length-aware RL after distillation.** A brief RLVR phase (Chapter 17) with a length penalty, or with DAPO's overlong shaping (Chapter 18), can shorten traces while preserving accuracy; distill-then-RL is a common two-stage recipe.
- **Hybrid thinking modes.** Train the student on both thinking and non-thinking targets (Qwen3's empty `<think></think>` convention, Chapter 6) so the caller can switch reasoning off for easy queries.
- **Faster decoding.** A distilled student is also a natural *draft* model for speculative decoding of its teacher (Chapter 13), since distillation increases the draft–target acceptance rate.

Systems Lens: Cost per solved problem

The metric that matters for deployment is cost per *correct* answer: tokens per answer × cost per token/accuracy. A student at 33% accuracy using 2,000 tokens per answer spends about $2000/0.33 \approx 6{,}100$ tokens per solved problem; a length-penalised version that reaches 31% with 800 tokens spends about 2,600, i.e. it is 2.3× cheaper per solved problem despite being "worse". Report both axes.

20.12 Future directions for reasoning models

RM 8.9

The broad recipe seems likely to persist for a while: train the strongest possible reasoning teacher, collect high-quality traces, and distill them into smaller, cheaper models. Four directions stand out.

1. **Refining the R1-style pipeline.** RLVR (Chapters 17 and 18) for flagship models, distillation for small ones, and continual improvements to both—better GRPO variants, better data curation, better teachers.
2. **Adaptive inference-time compute.** Not every question deserves a long trace. The surrounding system can decide when to answer directly, when to think longer, and when to stop early; commercial products in 2025 began exposing automatic reasoning-effort modes of this kind. This is a systems problem as much as a modelling one (Chapter 15).
3. **Richer rewards.** Most RLVR rewards check only the final answer. Rewards that judge intermediate steps or the whole solution (process supervision, model-based proof grading—DeepSeekMath-V2 is a recent example of judging complete solutions during

training) may teach more reliable reasoning, at the cost of building and serving those judges.

4. **Agents.** Reasoning models are increasingly the core of agent systems that plan, call tools, and recover from errors over long workflows. This pushes reward design beyond correctness toward tool-use success, retrieval quality and policy compliance—*multi-reward* training—and pushes infrastructure toward the agentic rollouts of Section 19.9. Distillation remains the practical way to move such behaviours from expensive teachers into models that can run locally or cheaply.

20.13 Conclusions

RM 8.10

20.13.1 What's next

RM 8.10.1

The techniques of Part V are most interesting in combination: distill a student from a strong teacher (this chapter), continue with RLVR (Chapters 17 and 18) using the infrastructure of Chapter 19, and apply self-consistency or self-refinement at deployment time (Chapters 15 and 16), all measured with the evaluation harness of Chapter 14 and served with the engine of Chapter 12. Running such end-to-end comparisons on your own tasks is the fastest way to learn which method helps where. The appendices extend the systems side: Triton kernels (Appendix A) and lab setup at scale (Appendix B).

20.13.2 Staying current

RM 8.10.2

Specific algorithms and benchmarks in this field change monthly; the fundamentals in this book change slowly: how losses and rewards are computed, the difference between training-time and inference-time methods, careful evaluation, and the cost models that govern training and serving. When a new method appears, map it back to these building blocks—it is usually a recombination. Practical sources: skim new arXiv listings for themes rather than trying to read everything; read model cards and technical reports, which often contain the most useful details on prompting conventions and limitations; follow practitioner communities where replications and failure cases appear early; and use AI research assistants as filters, not as substitutes for reading the papers. For the systems side, read the design docs and issue trackers of the open frameworks (Table 19.2) and inference engines—that is where today's bottlenecks are discussed first.

20.14 Summary

- Distillation trains a small student on outputs of a large teacher. Hard distillation is SFT on teacher text; soft distillation matches teacher distributions and needs logits and a shared vocabulary.
- The teacher runs once, offline, as a resumable batch job; rejection sampling with a verifier is cheap quality control.
- Each example is the chat-wrapped prompt plus the (think-tagged) teacher trace and answer plus an end token; tokenise once, store `prompt_len`, filter by length (a difficulty bias), split with a fixed seed.

- The loss is answer-only cross-entropy, equal to the negative mean log-probability of the teacher's tokens; mind the one-position shift and the token- vs. sequence-level normalisation when batching.
- A standard SFT loop with AdamW, clipping, validation loss and per-epoch checkpoints suffices; validation loss and benchmark accuracy are the signals to watch. Same-family teachers are easier to imitate.
- Soft distillation is a storage/compute problem: use top-k sparse targets or an online teacher; forward KL covers modes, reverse KL seeks one.
- Long traces are limited by activations and, for large vocabularies, by logits; chunked or fused cross-entropy, checkpointing, packing and context parallelism raise the cap.
- Efficient reasoning optimises tokens per answer as much as cost per token.

20.15 Exercises

Exercise 20.1 (*Split balance*). Apply `length_stats` to the training and validation partitions produced by `split_train_val`. Are they balanced in length? Propose a stratified split that guarantees it, and explain why validation loss is length-sensitive.

Exercise 20.2 (*Without think tags*). Repeat the distillation run with `use_think=False` (the `-no_think` flag of `run.py`) and compare validation loss and, if possible, MATH-500 accuracy with the tagged version. Why might the tags matter more for a model that must later switch between thinking and non-thinking modes?

Exercise 20.3 (*Loss identity*). Prove that `answer_only_loss` equals $-\frac{1}{n}\log p_\theta(y \mid x)$, and that the token-level batched loss equals the average of per-example losses if and only if all answers have the same length.

Exercise 20.4 (*Off-by-one*). Modify `answer_only_loss` to start at `prompt_len` and train the lab student. Measure how often the trained student emits `<think>` as its first token.

Exercise 20.5 (*KD temperature*). Show that the gradient of $\tau^2\,\mathrm{KL}(p_T^{(\tau)}\|p_S^{(\tau)})$ with respect to a student logit is $\tau(p_S^{(\tau)} - p_T^{(\tau)})$, and explain why the τ^2 factor is needed when mixing it with the hard-label cross-entropy.

Exercise 20.6 (*Top-k storage*). For a 3,000-token trace and $V = 151{,}936$, compute the storage for full bf16 distributions, for top-k with $k \in \{8, 32, 128\}$, and for text. Using the lab's `sparse_kd_loss`, measure how the loss approximation error depends on k for a peaked and a flat teacher.

Exercise 20.7 (*Forward vs. reverse KL*). Fit a single Gaussian to a mixture of two Gaussians by minimising forward and reverse KL numerically. Relate the result to Fig. 20.9 and to the behaviour of a small student that cannot represent all of the teacher's reasoning strategies.

Exercise 20.8 (*Logit memory*). For Qwen3-0.6B at $T \in \{2\text{k}, 8\text{k}, 32\text{k}\}$, estimate total training memory (weights, optimizer, activations with and without checkpointing, logits with and without chunking). Which term dominates at each length?

Exercise 20.9 (*Packing for SFT*). Implement packed training for distillation using the block-diagonal mask of Chapter 19 (`varlen_forward` is inference-only; write a differentiable variant). Verify that the packed loss equals the padded loss with token-level normalisation.

Exercise 20.10 (*Cost per solved problem*). Given two students—A: 33% accuracy, 2,100 tokens per answer; B: 30% accuracy, 900 tokens per answer—and a serving cost per token, which is cheaper per solved problem? At what accuracy would B have to fall for A to win?

Exercise 20.11 (*Teacher generation throughput*). Using the napkin math of Section 20.2 and the serving model of Chapter 12, estimate how long one 8×H100 node would need to generate the 36M teacher tokens with an open-weight 32B dense teacher. What limits throughput?

20.16 Lab 20: Distilling a reasoning teacher

Lab 20: Distilling a reasoning teacher

Code: `code/labs/lab20_distill`

Goals

Build the complete hard-distillation pipeline of this chapter at toy scale on a CPU, then add soft distillation and the systems pieces (batched losses, chunked cross-entropy, top-k storage, padding vs. packing). The same code drives a real Qwen3-0.6B run through `run.py`.

Setup

The "teacher" is a rule-based solver for multi-digit addition that writes one line per column (`7+5+0=12`, carries included) inside `<think>...</think>`, followed by `\boxed{sum}`—a miniature of R1-style traces with a verifiable answer. A character-level tokenizer has atomic special tokens (`<user>`, `<assistant>`, `<think>`, `</think>`, `<eos>`, `<pad>`). The student is a tiny Qwen3-shaped model from the core package.

```
cd code
pytest -q labs/lab20_distill                    # grades starter.py
LAB_IMPL=solution pytest -q labs/lab20_distill  # reference
python labs/lab20_distill/bench.py --digits 2   # trace ablation (minutes on
    CPU)
```

Tasks

1. **Data formatting.** Implement `format_distilled_answer` and `build_examples`: chat-wrapped prompt, raw target, end token, `prompt_len`; skip malformed records.
2. **Filtering and splitting.** Implement `length_stats` (mean, min/max with indices, p50/p90/p99), `filter_by_max_len` and a seeded `split_train_val`.
3. **Losses.** Implement `answer_only_loss` (tested against $-$`sequence_logprob`$/n$), `collate_answer_only` and `batch_answer_only_loss` with both token- and sequence-level reductions, `chunked_cross_entropy`, `kd_loss` (forward/reverse, temperature),

`topk_teacher` and `sparse_kd_loss`.

4. **Training.** Implement `train_distillation`; the test requires the validation loss to drop by 40% within four epochs. A second test trains a small student to match a larger teacher's distributions with `kd_loss`.
5. **Systems helpers.** Implement `padding_waste` (padded, length-sorted, packed real-token fractions) and `logit_storage_bytes` (dense vs. top-k).

Measurements

- Run `bench.py` to train three students on the same problems: full trace with think tags, trace without tags, and answer only. Report held-out exact-match accuracy and mean target length (Fig. 20.8).
- Measure the real-token fraction of padded vs. packed batches on a mix of 1-digit and 6-digit problems, and predict the speedup packing would give.
- Plot dense vs. top-k storage for a 3,000-token trace as a function of k.

Deliverables

Passing tests; the ablation table with a short interpretation (does the student need the trace? does it need the tags?); the storage plot.

Stretch goals

- Real scale: run `run.py` on Qwen3-0.6B-Base with the public DeepSeek-R1 MATH traces (`-max_len 2048 -epochs 3 -lr 1e-5`), then evaluate checkpoints on MATH-500 with your Lab 14 harness.
- Add rejection sampling: corrupt 20% of the teacher's answers and compare training on all traces vs. verified traces only.
- Train with `batch_size=8` using both reductions and compare learning curves; then implement packed training (Exercise 20.9) and measure tokens/s.
- Distil with a mixed loss $\alpha\,\mathrm{CE} + (1-\alpha)\tau^2\mathrm{KD}$ from a tiny trained teacher model and compare sample efficiency with pure hard distillation.

Appendix A

GPU Kernels with Triton

Throughout the book we have argued with rooflines: decode is memory-bound, softmax is memory-bound, attention without tiling moves $O(T^2)$ bytes, and a weight-only int4 GEMM is fast only if dequantization happens *inside* the kernel. This appendix makes those arguments executable. We write the kernels ourselves in Triton (Tillet et al., 2019), a Python-embedded language that compiles block-level programs to GPU code. Triton is what `torch.compile`'s Inductor backend emits for most fused operations, so reading Triton is also how you inspect what the compiler did to your model.

Every kernel here has a PyTorch reference in `llmsys/kernels/reference.py`. The Triton versions live in `llmsys/kernels/triton_kernels.py`. Triton ships with an *interpreter*: set `TRITON_INTERPRET=1` before importing `triton`, and kernels run on the CPU with NumPy semantics. It is slow, but it means correctness can be tested on a laptop, and it is how this appendix's lab is checked.

A.1 Why write a kernel at all?

A PyTorch eager program launches one kernel per operator. Each kernel reads its inputs from HBM and writes its outputs back to HBM. For matmuls this is fine, since their arithmetic intensity is high. For chains of elementwise and reduction operations (softmax, norms, activation functions, dequantization, rotary embeddings) the intermediate tensors make round trips through memory, and the chain runs at a small fraction of what the hardware could do. A *fused* kernel loads each input once, keeps intermediates in registers or shared memory, and writes the result once.

Key Idea

Write a custom kernel when (1) a sequence of memory-bound operations can be fused, (2) a computation's working set can be *tiled* into on-chip memory to change its asymptotic memory traffic (FlashAttention), or (3) a data format is not supported by library kernels (packed int4 weights, custom KV layouts). Otherwise, cuBLAS, cuDNN and `torch.compile` usually beat a hand-written kernel.

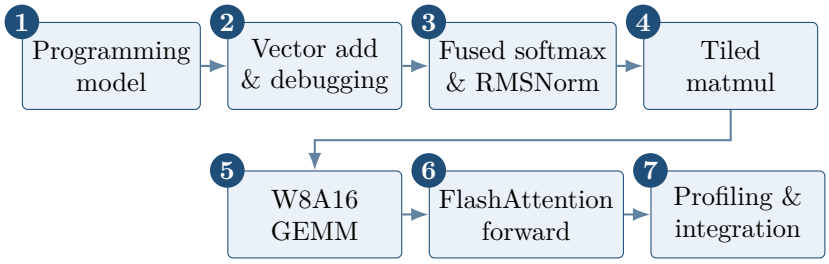


Figure A.1: Roadmap of this appendix: from the programming model to kernels that implement Chapter 4's FlashAttention and Chapter 13's quantized GEMM.

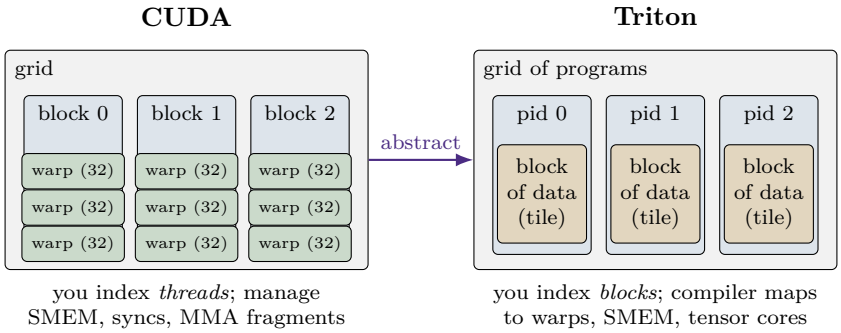


Figure A.2: CUDA's thread-level view versus Triton's block-level view. A Triton program corresponds to a CUDA thread block. Inside it you write NumPy-like operations on tiles, with `num_warps` controlling how many warps execute it.

A.2 The Triton programming model

A.2.1 From CUDA threads to Triton programs

CUDA exposes a three-level hierarchy. A *grid* of thread blocks; each block is scheduled onto one streaming multiprocessor (SM) and contains up to 1024 threads grouped into 32-wide warps; each thread owns registers, and threads in a block share up to a few hundred KB of shared memory (SMEM). The programmer reasons about individual threads: which element each one loads, how they cooperate through SMEM, and how to synchronise.

Triton removes the thread level. You write a *program* that operates on whole *blocks* (tiles) of data as if they were small arrays, and launch a grid of program instances. The compiler decides how to spread each block across threads and warps, when to stage data through shared memory, how to vectorise memory accesses, and how to issue tensor-core instructions for `tl.dot`. Figure A.2 contrasts the two views.

A.2.2 The memory hierarchy a kernel sees

Kernel design is mostly about *where data lives*. For an H100 SXM:

level	capacity	scope	managed by
registers	256 KB per SM (64k × 32-bit)	one thread	compiler
SMEM / L1	256 KB per SM combined, up to 228 KB usable as SMEM	thread block	you / compiler
L2 cache	50 MB	whole GPU	hardware
HBM3	80 GB at 3.35 TB/s	whole GPU	you

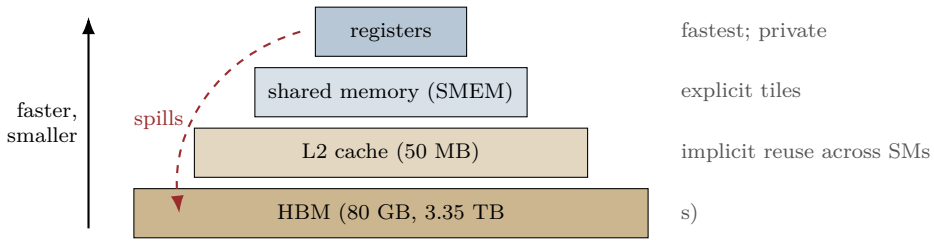


Figure A.3: The memory hierarchy as seen by one kernel. Triton tiles live in registers and SMEM. Register spills silently fall all the way to HBM.

There are 132 SMs. Each step down the hierarchy is roughly an order of magnitude more capacity and an order of magnitude less bandwidth per byte. Tiles must therefore be sized so that their working set (inputs, accumulators, pipeline buffers) fits in registers and SMEM, while still leaving enough room for several programs per SM to hide latency (*occupancy*). In Triton you control this through block sizes, `num_warps` and `num_stages`. Exceed the register budget and the compiler spills to local memory, which lives in HBM, and performance collapses (Fig. A.3).

A.2.3 A first kernel: vector addition

Listing A.1 adds two vectors. The `@triton.jit` kernel runs once per program instance:

- `tl.program_id(0)` tells the instance which block it owns;
- `tl.arange(0, BLOCK)` creates a vector of offsets within the block;
- `mask` guards the ragged last block (Fig. A.4);
- pointers are integers plus offsets, so `x_ptr + offsets` is a block of addresses and `tl.load` gathers them.

`BLOCK` is a `tl.constexpr`: it is known at compile time (and must be a power of two), so the compiler can specialise register allocation and vector widths. A new value triggers a recompilation, which is cached.

Listing A.1: Vector addition in Triton, with its launcher.

```
@triton.jit
def _add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
    pid = tl.program_id(axis=0)                  # which block am I?
    offsets = pid * BLOCK + tl.arange(0, BLOCK)  # BLOCK element indices
    mask = offsets < n_elements                  # guard the ragged tail
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)

def vector_add(x, y, block=1024):
    out = torch.empty_like(x)
    n = x.numel()
    grid = (triton.cdiv(n, block),)              # number of program instances
    _add_kernel[grid](x, y, out, n, BLOCK=block)
    return out
```

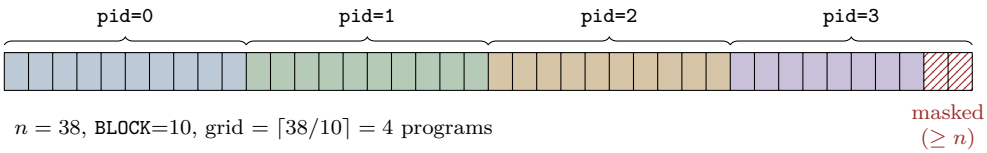


Figure A.4: Each program instance handles one contiguous block of `BLOCK` elements. The last block is partially out of bounds and is masked: masked loads return `other` (default undefined, so always pass it when the value matters), and masked stores are dropped.

A.2.4 Debugging with the interpreter

Kernels compile lazily on first launch, with arguments' dtypes and `constexpr` values as the cache key. Errors at that point tend to be cryptic. Two habits help. First, develop with `TRITON_INTERPRET=1`: kernels then execute eagerly in Python, so you can `print` a tile or drop into a debugger inside the kernel, on a machine without a GPU. Second, test against a PyTorch reference on awkward shapes (prime sizes, sizes smaller than one block, non-contiguous strides) before measuring speed.

```
TRITON_INTERPRET=1 python -m pytest -q labs/labA_triton      # correctness, CPU
python labs/labA_triton/bench.py --hw H100-SXM               # speed, GPU
```

Pitfall: The interpreter flag must be set first

`TRITON_INTERPRET` is read when `@triton.jit` decorates a function, so it must be in the environment *before* the module defining your kernels is imported. Our lab's test file sets it at the top when no GPU is present.

A.3 Fused softmax and RMSNorm

A row-wise softmax in eager PyTorch, written out (`x - max`, `exp`, `sum`, divide), launches several kernels, each streaming the whole matrix through HBM. Listing A.2 assigns one program per row, loads the row once, performs every step in registers, and writes the result once.

Listing A.2: Fused row softmax: one program per row.

```
@triton.jit
def _softmax_kernel(x_ptr, out_ptr, n_cols, x_row_stride, out_row_stride,
                    BLOCK: tl.constexpr):
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK)
    mask = cols < n_cols
    x = tl.load(x_ptr + row * x_row_stride + cols, mask=mask, other=-float("inf"))
    x = x.to(tl.float32)
    z = x - tl.max(x, axis=0)             # padding is -inf, so it never wins the
    max
    num = tl.exp(z)                       # exp(-inf) = 0: padding adds nothing
    den = tl.sum(num, axis=0)
    tl.store(out_ptr + row * out_row_stride + cols, num / den, mask=mask)
```

Note how the padding value is chosen so that masked lanes are neutral for both reductions: $-\infty$ for the max, and therefore $e^{-\infty} = 0$ for the sum.

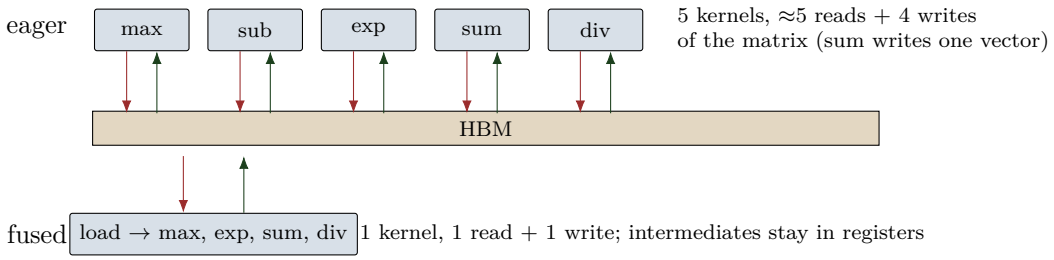


Figure A.5: Memory traffic of an unfused versus a fused softmax. Red arrows are reads from HBM, green arrows are writes. For a memory-bound operation, run time is proportional to the arrows.

Napkin Math: Softmax over a 4096 × 8192 fp32 matrix on an H100

The matrix is 134 MB. Fused: one read plus one write, 268 MB, so $\geq 80\,\mu$s at 3.35 TB/s. The five-kernel eager version moves roughly $9 \times 134\,\text{MB} \approx 1.2\,\text{GB}$, so $\geq 360\,\mu$s. PyTorch's `torch.softmax` is itself a fused kernel; our Triton kernel should match it, and the lab's benchmark reports achieved GB/s against the 3.35 TB/s peak. A good fused kernel reaches 80–90%.

The one-row-per-program design assumes a row fits in registers. Beyond roughly 32k fp32 columns, register pressure forces spills, and the kernel must loop over chunks of the row. It then either does two passes (max, then sum) or keeps *running* statistics with the online-softmax rescaling of Appendix A.6. RMSNorm (Chapter 6) has the same shape: one reduction (the mean of squares) followed by an elementwise scale. The lab implements it exactly like softmax.

A.3.1 A second fusion: the SwiGLU activation

Qwen3's MLP computes $\text{down}\big(\text{SiLU}(\text{gate}(x)) \odot \text{up}(x)\big)$ (Chapter 6). The two projections are GEMMs, but between them sit three memory-bound steps: the SiLU, the elementwise product, and the dtype handling. Fusing them into one pass is the simplest useful kernel there is (Listing A.3).

Listing A.3: Fused SiLU-and-multiply (`llmsys/kernels/triton_kernels.py`).

```
@triton.jit
def _swiglu_kernel(g_ptr, u_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
    offs = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offs < n_elements
    g = tl.load(g_ptr + offs, mask=mask, other=0.0).to(tl.float32)
    u = tl.load(u_ptr + offs, mask=mask, other=0.0).to(tl.float32)
    silu = g / (1.0 + tl.exp(-g))                    # g * sigmoid(g)
    tl.store(out_ptr + offs, silu * u, mask=mask)
```

Napkin Math: SwiGLU fusion during prefill

Take Qwen3-8B ($d_{\text{ff}} = 12288$) and a prefill chunk of 8192 tokens in bf16. Each of `gate` and `up` is $8192 \cdot 12288 \cdot 2\,\text{B} = 201\,\text{MB}$. Unfused (`silu`, then `mul`) reads 201 MB and writes 201 MB, then reads 402 MB and writes 201 MB: about 1.0 GB, or 300 μs at 3.35 TB/s, per layer. Fused, it reads 402 MB and writes 201 MB: 603 MB, or 180 μs. Over 36 layers that saves about 4 ms per chunk. An even better option is to fuse the activation into the *epilogue* of the gate/up GEMM, so the 201 MB intermediates never reach HBM at all.

A.3.2 A backward kernel: RMSNorm

Training needs gradients, and fused forward kernels need fused backward kernels. For one row, let $r = (\frac{1}{n}\sum_j x_j^2 + \epsilon)^{-1/2}$, $\hat{x} = r\,x$ and $y = w \odot \hat{x}$. Given the upstream gradient $\boldsymbol{dy}$, define $\boldsymbol{g} = \boldsymbol{dy} \odot w$. Then

$$\frac{\partial \mathcal{L}}{\partial x} = r\Big(\boldsymbol{g} - \hat{x}\,\frac{1}{n}\sum_j g_j \hat{x}_j\Big), \qquad \frac{\partial \mathcal{L}}{\partial w} = \sum_{\text{rows}} \boldsymbol{dy} \odot \hat{x}. \tag{A.1}$$

The input gradient is again a one-row reduction followed by elementwise work: one program per row, exactly like the forward. The weight gradient is a reduction *across* rows. The kernel writes per-row partial products and a second, tiny reduction sums them. Production kernels write one partial per program (not per row) and use a second Triton kernel or atomics. Note that r is *recomputed* from x rather than saved: recomputing a reduction over data we must load anyway costs nothing, while storing it costs memory.

Listing A.4: Fused RMSNorm backward and its autograd wrapper.

```
@triton.jit
def _rmsnorm_bwd_kernel(x_ptr, w_ptr, dy_ptr, dx_ptr, gw_ptr, n_cols, stride, eps,
                        BLOCK: tl.constexpr):
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK)
    mask = cols < n_cols
    x = tl.load(x_ptr + row * stride + cols, mask=mask, other=0.0).to(tl.float32)
    w = tl.load(w_ptr + cols, mask=mask, other=0.0).to(tl.float32)
    dy = tl.load(dy_ptr + row * stride + cols, mask=mask, other=0.0).to(tl.float32)
    rstd = 1.0 / tl.sqrt(tl.sum(x * x, axis=0) / n_cols + eps)   # recomputed
    xhat = x * rstd
    g = dy * w
    dx = rstd * (g - xhat * (tl.sum(g * xhat, axis=0) / n_cols))
    tl.store(dx_ptr + row * stride + cols, dx, mask=mask)
    tl.store(gw_ptr + row * stride + cols, dy * xhat, mask=mask)  # dW partials

class TritonRMSNorm(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight, eps=1e-6):
        ctx.save_for_backward(x, weight); ctx.eps = eps
        return rmsnorm(x, weight, eps)
    @staticmethod
    def backward(ctx, dy):
        x, weight = ctx.saved_tensors
        dx, dw = rmsnorm_backward(x, weight, dy, ctx.eps)
        return dx, dw, None
```

The lab's tests check both gradients against PyTorch autograd, under the interpreter.

Systems Lens: Fusion is where `torch.compile` earns its keep

Inductor, `torch.compile`'s default backend, performs exactly this kind of fusion automatically. It groups pointwise and reduction nodes of the captured graph and emits Triton kernels like Listing A.2. Run with `TORCH_LOGS=output_code` to see the generated Triton for your model. Hand-written kernels still win where the compiler cannot see the opportunity,

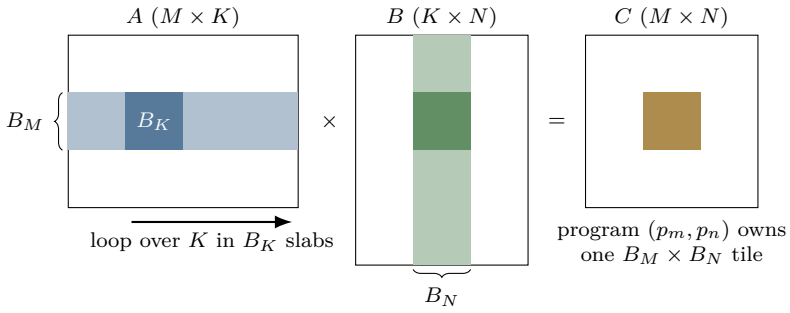


Figure A.6: Tiled GEMM. The program owning the orange tile of C streams the light-blue row panel of A and the light-green column panel of B through on-chip memory, one B_K-wide slab at a time (dark), accumulating in fp32 registers.

such as tiling attention, custom quantized formats, or cross-operation algebraic rewrites.

A.4 Tiled matrix multiplication

A.4.1 Tiling and arithmetic intensity

Computing $C = AB$ with $A \in \mathbb{R}^{M\times K}$, $B \in \mathbb{R}^{K\times N}$, each program produces one $B_M \times B_N$ tile of C. It loops over K in slabs of B_K, loading a $B_M \times B_K$ tile of A and a $B_K \times B_N$ tile of B and accumulating their product in fp32 registers (Fig. A.6). Per slab it moves $(B_M B_K + B_K B_N)$ elements and performs $2B_M B_N B_K$ FLOPs, so its intensity with respect to the loads is

$$I_{\text{tile}} = \frac{2B_M B_N B_K}{(B_M + B_N)B_K \cdot b} = \frac{2B_M B_N}{(B_M + B_N)\, b}, \tag{A.2}$$

where b is bytes per element. With 128×128 bf16 tiles, $I = 2 \cdot 128^2/(256 \cdot 2) = 64$ FLOP/byte. That is below the H100's ridge of ≈ 295 *if those loads went to HBM*. They do not all go there: tiles of A are reused by every program in the same row of the output grid, and tiles of B by every program in the same column. The L2 cache serves most re-reads, which is why program *ordering* matters (next subsection). Larger tiles raise I but consume more registers and SMEM, lowering occupancy. Autotuning searches this trade-off.

A.4.2 Program ordering for L2 reuse

Programs are launched in increasing `pid` order, and only as many run concurrently as fit on the SMs. With naive row-major ordering, the concurrently running programs cover one full row of output tiles. They share one panel of A, but each needs a *different* panel of B, so the working set is the whole of B. *Grouped* ("swizzled") ordering walks G rows of tiles in column-major order within each group, so concurrent programs share a few panels of both A and B (Fig. A.7). The remapping is pure integer arithmetic on the program id, at the top of Listing A.5.

Listing A.5: Grouped, tiled matmul. `tl.dot` compiles to tensor-core MMA instructions on GPUs.

```
@triton.jit
def _matmul_kernel(a_ptr, b_ptr, c_ptr, M, N, K,
                   stride_am, stride_ak, stride_bk, stride_bn, stride_cm, stride_cn,
                   BM: tl.constexpr, BN: tl.constexpr, BK: tl.constexpr,
                   GROUP_M: tl.constexpr):
    pid = tl.program_id(0)
```

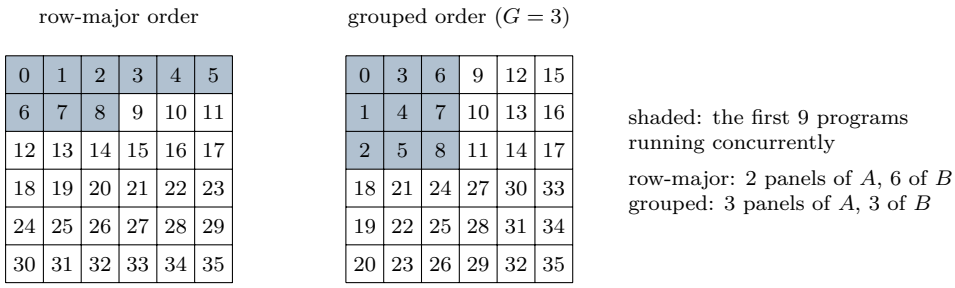


Figure A.7: Launch order of output tiles. If 9 programs run at once, grouped ordering touches 6 input panels instead of 8. For large matrices the saving in L2 misses is substantial.

```
    num_pid_m, num_pid_n = tl.cdiv(M, BM), tl.cdiv(N, BN)
    num_pid_in_group = GROUP_M * num_pid_n
    first_pid_m = (pid // num_pid_in_group) * GROUP_M
    group_size_m = min(num_pid_m - first_pid_m, GROUP_M)
    pid_m = first_pid_m + ((pid % num_pid_in_group) % group_size_m)
    pid_n = (pid % num_pid_in_group) // group_size_m
    rm = pid_m * BM + tl.arange(0, BM)
    rn = pid_n * BN + tl.arange(0, BN)
    rk = tl.arange(0, BK)
    a_ptrs = a_ptr + rm[:, None] * stride_am + rk[None, :] * stride_ak
    b_ptrs = b_ptr + rk[:, None] * stride_bk + rn[None, :] * stride_bn
    acc = tl.zeros((BM, BN), dtype=tl.float32)
    for k0 in range(0, K, BK):
        a = tl.load(a_ptrs, mask=(rm[:, None] < M) & (rk[None, :] + k0 < K), other
=0.0)
        b = tl.load(b_ptrs, mask=(rk[:, None] + k0 < K) & (rn[None, :] < N), other
=0.0)
        acc += tl.dot(a, b)
        a_ptrs += BK * stride_ak
        b_ptrs += BK * stride_bk
    c_ptrs = c_ptr + rm[:, None] * stride_cm + rn[None, :] * stride_cn
    tl.store(c_ptrs, acc, mask=(rm[:, None] < M) & (rn[None, :] < N))
```

A.4.3 Pipelining and autotuning

Inside the K loop, a naive kernel would load a slab, wait, compute, and load the next. Triton's `num_stages` parameter software-pipelines the loop: the compiler issues asynchronous copies (`cp.async` on Ampere, the Tensor Memory Accelerator on Hopper) for slabs $k+1, \ldots, k+s-1$ while the tensor cores work on slab k, keeping s buffers in shared memory (Fig. A.8). `num_warps` sets how many warps cooperate on one program. Good values depend on the GPU, the shapes and the dtype, so production kernels declare a list of candidate configurations and let `@triton.autotune` benchmark them the first time each `key` (e.g. (M, N, K)) is seen:

Listing A.6: Declaring autotuning configurations (GPU only).

```
@triton.autotune(
    configs=[triton.Config({"BM": 128, "BN": 128, "BK": 64, "GROUP_M": 8},
                           num_stages=3, num_warps=8),
             triton.Config({"BM": 64, "BN": 128, "BK": 32, "GROUP_M": 8},
                           num_stages=4, num_warps=4),
             triton.Config({"BM": 128, "BN": 64, "BK": 32, "GROUP_M": 8},
```

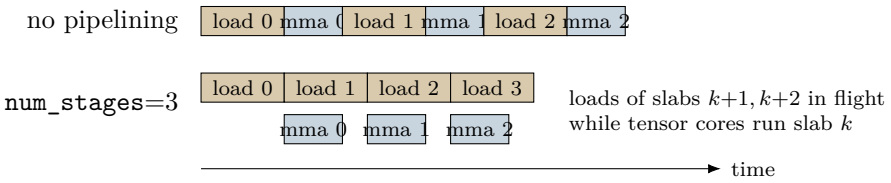


Figure A.8: Software pipelining of the K loop. With multiple shared-memory buffers, asynchronous loads of future slabs overlap tensor-core work on the current one, hiding memory latency.

```
                            num_stages=4, num_warps=4)],
    key=["M", "N", "K"])
@triton.jit
def _matmul_kernel_tuned(...):  # same body as above
    ...
```

Napkin Math: How close to cuBLAS should you get?

A square bf16 GEMM with $n = 4096$ performs $2n^3 = 1.37 \cdot 10^{11}$ FLOPs and touches $3n^2 \cdot 2\,\text{B} = 100\,\text{MB}$, an intensity of ≈ 1365 FLOP/B, which is deep in compute-bound territory. At 989 TFLOP/s the floor is 139 μs. cuBLAS typically reaches 70–80% of peak at this size on Hopper. A tutorial-style Triton kernel built on `tl.dot` reaches a large fraction of that on Ampere. Matching cuBLAS on Hopper needs warp-specialised kernels with TMA and asynchronous `wgmma`, which newer Triton versions expose. Do not expect to beat the vendor library on plain GEMMs. Write GEMMs when you need a *fused prologue or epilogue*, as in the next section.

A.5 A fused W8A16 GEMM

Chapter 13's `QuantLinear` dequantized the whole weight in PyTorch before multiplying, which moves *more* bytes than the bf16 baseline. The fix is to load the int8 weight tile, widen it in registers, and apply the per-output-channel scale once to the accumulator in the epilogue. The scale factors out of the K sum because it depends only on the output channel: $y_{mn} = \sum_k x_{mk}\, s_n q_{nk} = s_n \sum_k x_{mk} q_{nk}$.

Listing A.7: Weight-only int8 GEMM: only one byte per weight crosses HBM.

```
@triton.jit
def _w8a16_kernel(x_ptr, w_ptr, s_ptr, y_ptr, M, N, K,
                  stride_xm, stride_xk, stride_wn, stride_wk, stride_ym, stride_yn,
                  BM: tl.constexpr, BN: tl.constexpr, BK: tl.constexpr):
    pid_m, pid_n = tl.program_id(0), tl.program_id(1)
    rm = pid_m * BM + tl.arange(0, BM)
    rn = pid_n * BN + tl.arange(0, BN)
    rk = tl.arange(0, BK)
    acc = tl.zeros((BM, BN), dtype=tl.float32)
    for k0 in range(0, K, BK):
        x = tl.load(x_ptr + rm[:, None] * stride_xm + (k0 + rk)[None, :] * stride_xk
    ,
                    mask=(rm[:, None] < M) & ((k0 + rk)[None, :] < K), other=0.0)
        w = tl.load(w_ptr + rn[None, :] * stride_wn + (k0 + rk)[:, None] * stride_wk
    ,
                    mask=(rn[None, :] < N) & ((k0 + rk)[:, None] < K), other=0)
```

```
        acc += tl.dot(x.to(tl.float32), w.to(tl.float32))   # widen in registers
    s = tl.load(s_ptr + rn, mask=rn < N, other=0.0)
    acc = acc * s[None, :]                                  # epilogue: apply scale
    tl.store(y_ptr + rm[:, None] * stride_ym + rn[None, :] * stride_yn, acc,
             mask=(rm[:, None] < M) & (rn[None, :] < N))
```

Two remarks for real deployments. First, the weight tile is loaded transposed (`w[k, n]` from a row-major `(N, K)` array); a production kernel pre-arranges the layout offline so that loads are contiguous and vectorised. Second, decode has $M = B$ (a handful of rows), so a $B_M = 16$ tile wastes most of each tensor-core instruction. Specialised kernels (Marlin, Machete) use split-K parallelism so that enough programs exist to keep every SM streaming weights. For group-wise int4, the scale *does* depend on k, so it is applied per group inside the loop.

A.6 FlashAttention forward in Triton

Chapter 4 derived FlashAttention: process keys and values in blocks, keep a running row maximum m and normaliser ℓ, and rescale the partial output whenever the maximum increases. The $T \times T$ score matrix then never touches HBM. In Triton the algorithm is short, because tiles are first-class (Listing A.8).

Proposition A.1 (Online softmax)**.** Process the scores $s_1, \dots, s_S$ of one query row in blocks. Initialise $m = -\infty$, $\ell = 0$, $\boldsymbol{a} = \mathbf{0}$. For each block $\mathcal{B}$ with values $\boldsymbol{v}_j$ set $m' = \max(m, \max_{j\in\mathcal{B}} s_j)$, then $\ell \leftarrow e^{m-m'}\ell + \sum_{j\in\mathcal{B}} e^{s_j - m'}$ and $\boldsymbol{a} \leftarrow e^{m-m'}\boldsymbol{a} + \sum_{j\in\mathcal{B}} e^{s_j-m'}\boldsymbol{v}_j$, and finally $m \leftarrow m'$. After the last block, $\boldsymbol{a}/\ell = \sum_j \mathrm{softmax}(\boldsymbol{s})_j\, \boldsymbol{v}_j$.

Proof. By induction, after processing a set $\mathcal{P}$ of blocks the invariants $\ell = \sum_{j\in\mathcal{P}} e^{s_j-m}$ and $\boldsymbol{a} = \sum_{j\in\mathcal{P}} e^{s_j-m}\boldsymbol{v}_j$ hold with $m = \max_{j\in\mathcal{P}} s_j$. Multiplying by $e^{m-m'}$ re-expresses the old sums relative to the new maximum, and adding the new block's terms extends them to $\mathcal{P}\cup\mathcal{B}$. At the end, $\boldsymbol{a}/\ell = \sum_j e^{s_j-m}\boldsymbol{v}_j / \sum_j e^{s_j-m}$, and the common factor e^{-m} cancels. Every exponent is ≤ 0, so nothing overflows. □

Listing A.8: FlashAttention forward kernel: one program per (query block, batch×head).

```
@triton.jit
def _flash_fwd_kernel(q_ptr, k_ptr, v_ptr, o_ptr, T, S, sm_scale, stride_b, stride_t
    ,
                      CAUSAL: tl.constexpr, BLOCK_M: tl.constexpr,
                      BLOCK_N: tl.constexpr, D: tl.constexpr):
    pid_m, bh = tl.program_id(0), tl.program_id(1)
    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_d = tl.arange(0, D)
    base = bh * stride_b
    q = tl.load(q_ptr + base + offs_m[:, None] * stride_t + offs_d[None, :],
                mask=offs_m[:, None] < T, other=0.0).to(tl.float32)
    m_i = tl.full((BLOCK_M,), -float("inf"), dtype=tl.float32)
    l_i = tl.zeros((BLOCK_M,), dtype=tl.float32)
    acc = tl.zeros((BLOCK_M, D), dtype=tl.float32)
    shift = S - T
    hi = S
    if CAUSAL:                                      # skip blocks entirely in the future
        hi = tl.minimum(S, (pid_m + 1) * BLOCK_M + shift)
```

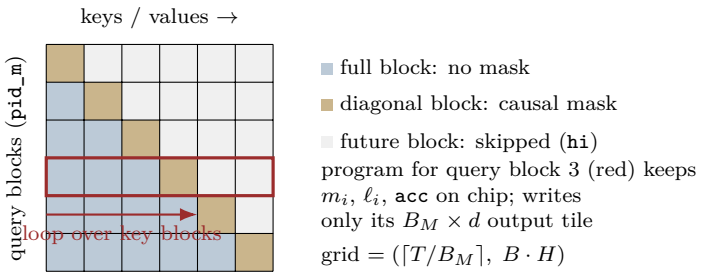


Figure A.9: Work decomposition of the FlashAttention forward kernel. Each program owns one query block of one head and streams the key/value blocks to its left. With causal masking, roughly half the blocks are skipped and only diagonal blocks need masks.

```
    for n0 in range(0, hi, BLOCK_N):
        offs_n = n0 + tl.arange(0, BLOCK_N)
        k = tl.load(k_ptr + base + offs_n[:, None] * stride_t + offs_d[None, :],
                    mask=offs_n[:, None] < S, other=0.0).to(tl.float32)
        v = tl.load(v_ptr + base + offs_n[:, None] * stride_t + offs_d[None, :],
                    mask=offs_n[:, None] < S, other=0.0).to(tl.float32)
        s = tl.dot(q, tl.trans(k)) * sm_scale
        valid = offs_n[None, :] < S
        if CAUSAL:
            valid = valid & (offs_n[None, :] <= offs_m[:, None] + shift)
        s = tl.where(valid, s, -float("inf"))
        m_new = tl.maximum(m_i, tl.max(s, axis=1))
        p = tl.exp(s - m_new[:, None])
        alpha = tl.exp(m_i - m_new)              # rescale old statistics
        l_i = l_i * alpha + tl.sum(p, axis=1)
        acc = acc * alpha[:, None] + tl.dot(p, v)
        m_i = m_new
    tl.store(o_ptr + base + offs_m[:, None] * stride_t + offs_d[None, :],
             acc / l_i[:, None], mask=offs_m[:, None] < T)
```

Napkin Math: Why tiling attention matters

For $T = 8192$ and 32 heads, a materialised fp16 score matrix takes $8192^2 \cdot 32 \cdot 2\,\text{B} = 4.3\,\text{GB}$ *per layer*. It must be written by one kernel and read by the next (twice more in the backward pass). The tiled kernel's HBM traffic is only Q, K, V and O, i.e. $4 \cdot 8192 \cdot 32 \cdot 128 \cdot 2\,\text{B} = 268\,\text{MB}$, plus re-reads of K and V by each query block that hit in L2. Attention changes from memory-bound to compute-bound, and its memory footprint from $O(T^2)$ to $O(T)$.

Backward pass. FlashAttention's backward recomputes the probabilities block by block from Q, K and the saved per-row log-sum-exp $m + \log \ell$ (one float per row), instead of storing P. It trades a second $QK^\top$ for $O(T^2)$ memory. The gradients with respect to Q, K and V need a separate pass (or atomics), because dK and dV accumulate over query blocks.

What later versions add. FlashAttention-2 (Dao, 2023) swaps the loop order to parallelise over the sequence dimension even at batch 1, reduces non-matmul FLOPs (the rescaling), and partitions work between warps to avoid shared-memory communication. FlashAttention-3 (Shah et al., 2024) targets Hopper: warp specialisation (producer warps issue TMA loads, consumer warps issue asynchronous `wgmma`), ping-pong scheduling that overlaps the softmax of one tile

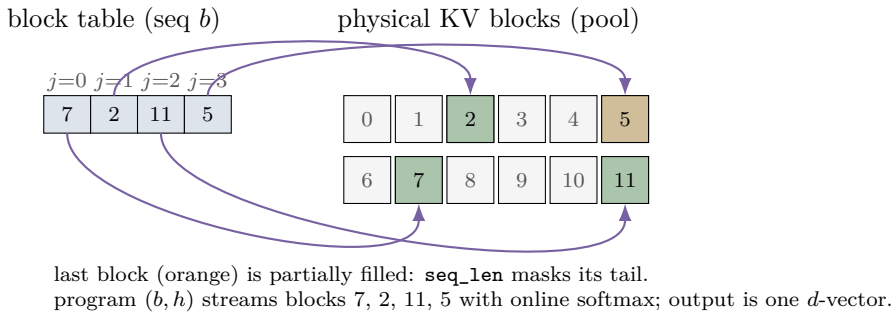


Figure A.10: Paged decode attention. The block table translates logical KV blocks into scattered physical blocks. The kernel performs the gather on the fly, so no contiguous copy of the cache is ever made.

with the matmul of another, and FP8 support. For decode, where $T_q = 1$, split-KV variants ("flash-decoding") parallelise over the key dimension and reduce partial results, because a single query block cannot fill the GPU.

Pitfall: Our teaching kernel's limits

`flash_attention` in `llmsys` requires $T = S$ (prefill), a power-of-two head dimension of at least 16, and contiguous `(B·H, T, D)` layouts. It accumulates in fp32 but loads whatever dtype it is given. It does not handle GQA head sharing, paged KV, sliding windows or the backward pass. Use `F.scaled_dot_product_attention` or the FlashAttention library in production.

A.7 A paged-attention decode kernel

Chapter 12 stores the KV cache in fixed-size physical blocks and gives each sequence a *block table* that maps logical block j to a physical block id. This eliminates fragmentation, but it means that attention can no longer assume contiguous keys. The kernel must chase the indirection itself. Decode is the simplest case: one new query per sequence. Listing A.9 assigns one program per (sequence, head), walks the sequence's logical blocks, loads the physical block id from the table, and applies the online softmax of Proposition A.1 block by block (Fig. A.10).

Listing A.9: Decode attention over a paged KV cache (one query token per sequence).

```
@triton.jit
def _paged_decode_kernel(q_ptr, k_cache_ptr, v_cache_ptr, block_table_ptr,
    seq_lens_ptr,
                         o_ptr, sm_scale, max_blocks, H: tl.constexpr,
                         BLOCK_SIZE: tl.constexpr, D: tl.constexpr):
    b, h = tl.program_id(0), tl.program_id(1)
    offs_d, offs_t = tl.arange(0, D), tl.arange(0, BLOCK_SIZE)
    q = tl.load(q_ptr + (b * H + h) * D + offs_d).to(tl.float32)
    seq_len = tl.load(seq_lens_ptr + b)
    m_i, l_i = -float("inf"), 0.0
    acc = tl.zeros((D,), dtype=tl.float32)
    for j in range(0, tl.cdiv(seq_len, BLOCK_SIZE)):
        phys = tl.load(block_table_ptr + b * max_blocks + j)     # indirection
        base = (phys * BLOCK_SIZE + offs_t[:, None]) * H * D + h * D + offs_d[None,
    :]
```

```
        valid = (j * BLOCK_SIZE + offs_t) < seq_len
        k = tl.load(k_cache_ptr + base, mask=valid[:, None], other=0.0).to(tl.
    float32)
        v = tl.load(v_cache_ptr + base, mask=valid[:, None], other=0.0).to(tl.
    float32)
        s = tl.where(valid, tl.sum(k * q[None, :], axis=1) * sm_scale, -float("inf")
    )
        m_new = tl.maximum(m_i, tl.max(s, axis=0))
        p = tl.exp(s - m_new)
        alpha = tl.exp(m_i - m_new)
        l_i = l_i * alpha + tl.sum(p, axis=0)
        acc = acc * alpha + tl.sum(p[:, None] * v, axis=0)
        m_i = m_new
    tl.store(o_ptr + (b * H + h) * D + offs_d, acc / l_i)
```

Two design notes. First, with a single query the score computation is a matrix–vector product, so the kernel uses `tl.sum` over an elementwise product rather than `tl.dot`. Decode attention is purely memory-bound: every KV byte is read once for about two FLOPs. The block size therefore matters for *memory* efficiency. 16–32 tokens × `H_kv` × `D` per block gives contiguous loads of several KB, enough to approach full bandwidth. Second, the grid has only $B \cdot H$ programs. With a batch of 4 and 8 KV heads, that is 32 programs on 132 SMs, so most of the GPU idles on long sequences. *Split-KV* (flash-decoding) adds a third grid axis over chunks of the sequence. Each program returns a partial $(m, \ell, \boldsymbol{a})$ triple, and a small second kernel merges them with the same rescaling rule. Production kernels also share the K/V load across the query heads of a GQA group, which cuts KV traffic by the group size.

Systems Lens: Kernels are the engine's contract

A serving engine's memory manager (Chapter 12) and its attention kernel must agree on everything: block size, KV layout (token-major or head-major), dtype and scale format for quantized KV, and how the block table and sequence lengths are passed. This is why engines ship their own attention backends, and why a change such as "store KV in FP8" or "share blocks across sequences for prefix caching" is as much a kernel change as a scheduler change. When you prototype a new cache policy, write the PyTorch reference (`reference.paged_decode_attention`) first. It defines the semantics that any fast kernel must reproduce.

A.8 Numerics inside kernels

Hand-written kernels own their numerics, and the defaults that PyTorch chose for you no longer apply.

- **Accumulate in fp32.** Tensor cores take bf16/fp16/fp8 inputs but accumulate in fp32. Keep `acc` in `tl.float32` and round once when storing. A bf16 accumulator over $K = 4096$ products loses most of the low-order bits: bf16 has an 8-bit significand, so once the running sum is 256 times larger than a term, the term vanishes.
- **Upcast before reductions.** Our softmax and norm kernels convert loaded tiles with `.to(tl.float32)` before `tl.max`/`tl.sum`; fp16 sums of squares overflow at 65504.
- **Subtract the max.** `exp` of fp32 overflows above ≈ 88.7. Every softmax, online or not,

must work with $s - m$.

- **Mind the $-\infty$ arithmetic.** A row whose scores are all masked has $m = -\infty$, and $e^{m-m'} = e^{-\infty-(-\infty)}$ is NaN. Engines guard this case (padding rows in batched decode, fully masked sliding windows), for example by clamping m to a finite floor.

- **Determinism.** Floating-point addition is not associative. Split-K GEMMs, split-KV attention and atomic accumulations (`tl.atomic_add` for weight gradients) sum in an order that depends on scheduling, so results can differ bit-for-bit between runs. For RL training this matters: rollouts generated by an inference kernel and log-probabilities recomputed by a training kernel can disagree slightly even for identical weights (Chapter 19). Deterministic modes use fixed reduction trees at some cost in speed.

Pitfall: Testing tolerances

Compare against an fp32 (or fp64) reference, never against another low-precision kernel, and scale tolerances with the reduction length. Our tests use 10^{-5} for fp32 interpreter runs. The GPU bf16 test allows 2×10^{-2} relative error. A kernel that passes at $K = 64$ and fails at $K = 8192$ usually accumulates in the wrong dtype.

A.8.1 What to look at in a profile

kernel type	bound by	first metrics to check (Nsight Compute)
elementwise, softmax, norms	HBM bandwidth	DRAM throughput (% of peak); global load efficiency (coalescing)
GEMM, prefill attention	tensor cores	tensor pipe utilisation; SMEM bank conflicts; occupancy and register spills
decode attention, W4A16 GEMV	HBM bandwidth + parallelism	DRAM throughput; number of programs vs. SMs (grid too small?)
anything tiny	launch latency	Nsight Systems timeline gaps; use CUDA graphs or fuse

A.9 Under the hood: from Python to SASS

`@triton.jit` does not interpret your Python on the GPU. On the first launch with a new signature it parses the function's AST and lowers it through a stack of MLIR dialects (Fig. A.11). *Triton IR* (TTIR) is a hardware-independent, block-level SSA form. *TritonGPU IR* (TTGIR) attaches *layouts* to every tensor: which thread owns which elements, whether a tile lives in registers or in shared memory, and which MMA instruction shape a `tl.dot` uses. Most of Triton's optimisations happen at this level: coalescing, pipelining, layout conversions, and the insertion of barriers and asynchronous copies. Next come LLVM IR, then PTX, and finally the vendor assembler produces a cubin (SASS). Every stage can be inspected:

Listing A.10: Dumping the intermediate representations of a compiled kernel (GPU).

```
compiled = _softmax_kernel[(n_rows,)](x, out, n_cols, x.stride(0), out.stride(0),
                                      BLOCK=1024)
```

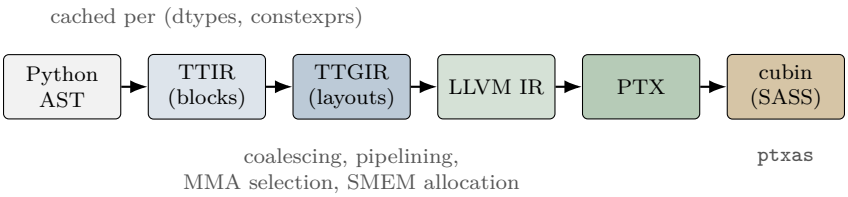


Figure A.11: Triton's compilation pipeline. The optimisations that make block-level code fast happen when tensors acquire GPU layouts in TTGIR.

```
print(compiled.asm.keys())          # e.g. 'ttir', 'ttgir', 'llir', 'ptx', 'cubin'
print(compiled.asm["ttgir"])        # look for #blocked / #shared / #mma layouts
print(compiled.n_regs, compiled.n_spills)   # register pressure at a glance
```

The same applies to code you did not write. `torch.compile` with the Inductor backend emits Triton kernels for fused pointwise and reduction regions (and sometimes GEMMs, via templates). Set `TORCH_LOGS=output_code` to print them, or set `TORCH_COMPILE_DEBUG=1` to dump them to disk. Reading those kernels is a quick way to learn what was fused, which regions fell back to library calls, and whether a `view` or `.contiguous()` in your model code broke a fusion.

Systems Lens: When the compiler needs help

Kernels with very large tiles (big `BLOCK` for long rows, large GEMM tiles) are limited by registers. If `n_spills` is non-zero, shrink the tile, lower `num_stages`, or raise `num_warps` so that each thread holds fewer elements. Excessive `constexpr` specialisation (e.g. passing the sequence length as `constexpr`) recompiles for every new value: keep shapes as runtime arguments unless the compiler genuinely needs them.

A.10 Triton and CUDA: a translation table

CUDA concept	Triton equivalent / who handles it
thread block	program instance (`tl.program_id`)
grid dims	launch grid tuple `kernel[grid](...)`
threads per block	`num_warps` × 32 (compiler distributes the tile)
shared memory, `__syncthreads`	implicit; inserted by the compiler (layouts, swizzling)
coalescing, vector loads	inferred from contiguous `tl.arange` offsets
`mma`/`wgmma` fragments	`tl.dot` on tiles ≥ 16 in each dimension
`cp.async`, TMA, multi-buffering	`num_stages`; tensor descriptors in recent versions
warp shuffles for reductions	`tl.sum`, `tl.max` along an axis
atomics	`tl.atomic_add`, `tl.atomic_max`, ...
templates / compile-time params	`tl.constexpr` arguments (re-JIT per value)

When does a project drop below Triton? When it needs explicit control over warp roles, register allocation or inter-SM cooperation (thread-block clusters, distributed shared memory), or the very last 10–20% on a flagship GEMM or attention kernel. Then use CUDA C++ with CUTLASS/CuTe, or tile DSLs such as ThunderKittens.

A.11 Measuring kernels

- **Wall-clock**: `triton.testing.do_bench(fn)` handles warm-up, L2 flushing and CUDA-event timing. Never time a GPU kernel with `time.time()` without synchronising.

- **Achieved rates**: convert time into GB/s for memory-bound kernels and TFLOP/s for compute-bound ones, and report the percentage of the roofline (the lab's `bench.py` does this). A 2× speedup that reaches only 15% of peak bandwidth still has headroom.
- **Timelines**: Nsight Systems (`nsys profile python script.py`) shows gaps between kernels: launch overhead, CPU stalls, synchronisations. For decode, these gaps are often the real problem, and the fix is CUDA graphs (Chapter 11), not a faster kernel.
- **Inside one kernel**: Nsight Compute (`ncu -set full`) reports DRAM throughput, SM and tensor-pipe utilisation, occupancy, bank conflicts and stall reasons, and plots the kernel on a roofline.

A.11.1 Using a Triton kernel from PyTorch

A launcher like `softmax_rows` is already callable on CUDA tensors. To make it a first-class operator that `torch.compile` can trace through and autograd can differentiate, register it as a custom op (`torch.library.custom_op`) with a fake (meta) implementation for shape inference, and give it a backward formula, either with `register_autograd` or by wrapping it in a `torch.autograd.Function`. Recent PyTorch versions can also capture user-defined Triton kernels directly inside compiled graphs.

A.12 Summary

- Triton programs operate on tiles. You choose the decomposition (grid, block sizes, loop order), and the compiler handles threads, shared memory and tensor cores.
- Masks with neutral padding values (`0`, $-\infty$) make ragged edges and reductions correct.
- Fusion turns memory-bound chains into single passes. Tiling changes asymptotic traffic (GEMM, FlashAttention). Program ordering and pipelining decide how close a tiled kernel gets to the roofline.
- Quantized GEMMs and attention are where custom kernels pay off most in LLM inference, because their data formats and fusions are not library defaults.
- Develop with the interpreter and PyTorch references, then measure against the roofline with `do_bench`, Nsight Systems and Nsight Compute.

A.13 Exercises

Exercise A.1 (*grid arithmetic*). For `vector_add` with $n = 10^6$ and `BLOCK`=1024, how many programs are launched, and how many lanes are masked off? Why must `BLOCK` be a power of two?

Exercise A.2 (*padding values*). In Listing A.2, what goes wrong if masked lanes are loaded with `other=0.0`? Construct a row of length 3 with `BLOCK`=4 where the result is incorrect.

Exercise A.3 (*long rows*). Rewrite the softmax kernel for rows longer than one block, using a loop with online max/sum rescaling followed by a second normalisation pass. How many HBM

passes does your kernel make? Can you get away with one write?

Exercise A.4 (*tile intensity*). Compute I_{tile} for $(B_M, B_N) \in \{(64,64),(128,128),(128,256)\}$ in bf16 and in fp8. Which of them can saturate an H100's tensor cores purely from HBM bandwidth, and why does L2 reuse change the answer?

Exercise A.5 (*swizzling*). Generalise Fig. A.7: for an $R \times C$ grid of tiles, P concurrent programs and group size G, count the distinct A and B panels touched by the first P programs, and find the G that minimises the total.

Exercise A.6 (*shared memory budget*). With `num_stages`$= s$ and bf16 tiles $B_M \times B_K$ and $B_K \times B_N$, how much shared memory does the pipelined GEMM need? With 228 KB per SM on an H100, what is the largest s for $128 \times 128 \times 64$ tiles, and how many such programs fit per SM?

Exercise A.7 (*int4 epilogue*). Modify the W8A16 kernel for group-wise int4 with packed weights: where do you unpack, and where must the scale and zero point be applied when the group size equals B_K?

Exercise A.8 (*FlashAttention memory*). Count the HBM bytes read and written by the forward kernel for $B = 1$, $H = 32$, $T = S = 8192$, $d = 128$, bf16, assuming no L2 hits, as a function of B_M. Compare with naive attention.

Exercise A.9 (*decode attention*). Explain why the kernel of Listing A.8 uses only $B \cdot H$ programs when $T = 1$, and sketch a split-KV variant: what extra state must each partial result carry so the reduction is exact?

Exercise A.10 (*SwiGLU in the epilogue*). Sketch how to fuse SiLU-and-multiply into the epilogue of a GEMM that computes `gate` and `up` together from a concatenated weight $[W_g; W_u]$. Which tile shape constraint does this impose, and how many HBM bytes per token does it save for Qwen3-8B?

Exercise A.11 (*RMSNorm backward*). Derive Eq. (A.1), starting from $\partial \hat{x}_i / \partial x_j = r(\delta_{ij} - \hat{x}_i \hat{x}_j / n)$. Why is it safe, and cheaper, to recompute r in the backward kernel rather than save it?

Exercise A.12 (*paged decode*). For Qwen3-8B (8 KV heads, $d = 128$, bf16) and a block size of 16 tokens, how many bytes does one program load per block? With a batch of 2 and 32k-token contexts, how many programs does Listing A.9 launch, and how many split-KV chunks per sequence would you use on a 132-SM GPU?

A.14 Lab A: GPU kernels with Triton

Lab A: GPU kernels with Triton

Code: `code/labs/labA_triton`

Goals

Write five Triton kernels (vector add, fused softmax, RMSNorm, grouped tiled matmul, FlashAttention forward) and a PyTorch online-softmax attention reference. Verify them on any machine with the interpreter, then measure them against the roofline on a GPU.

Setup

`cd code; LAB_IMPL=starter python -m pytest -q labs/labA_triton`. Without CUDA, the test file sets `TRITON_INTERPRET=1` automatically (use small shapes; it is slow). `_w8a16_kernel` in `starter.py` is complete: read it as a worked example before you start.

Tasks

1. `_add_kernel` and `vector_add`: the grid, offsets and masks.
2. `_softmax_kernel` and `softmax_rows`: one program per row; choose padding values that are neutral for max and sum.
3. `_rmsnorm_kernel` and `rmsnorm`: mean of squares in fp32; support arbitrary leading dimensions by flattening in the launcher.
4. `_matmul_kernel` and `matmul`: tiles, a K loop with masks, fp32 accumulation with `tl.dot`, then grouped program ordering.
5. `flash_attention_tiled_torch`: online softmax in PyTorch, including $T \neq S$ (queries aligned to the end of the keys, as in decode).
6. `_flash_fwd_kernel` and `flash_attention`: port your PyTorch version to Triton, with causal block skipping.
7. `arithmetic_intensity` and `roofline_time`: the napkin math used in `bench.py`.

Measurements (GPU)

Run `bench.py -impl starter -hw <your GPU>`. Report, for each kernel and size, time, achieved GB/s or TFLOP/s and percentage of peak. Expect vector add and softmax at 70–90% of bandwidth for large sizes, and the unfused eager softmax several times slower. For matmul, compare with and without grouped ordering (`group_m=1`) and for 2–3 tile configurations; then add `@triton.autotune`. For FlashAttention, compare against `F.scaled_dot_product_attention` and plot time against T for your kernel and for naive attention: one curve should be quadratic, and you should explain why the other is too, despite linear memory.

Deliverables

Passing tests (interpreter or GPU); a table of achieved rates vs. roofline; one paragraph on the biggest gap you found and its cause, backed by an Nsight Compute metric.

Stretch goals

- Fuse RoPE into the FlashAttention kernel's Q and K loads.
- Add GQA: let several query heads (`bh` indices) map to one KV head without materialising repeated K/V.
- Implement a split-KV decode kernel and compare it with the prefill kernel at $T = 1$.
- Write the group-wise int4 variant of the W8A16 kernel and plug it into `QuantLinear`

from Lab 13. Measure decode tokens/s for Qwen3-0.6B.

Appendix B

Lab Environment, Hardware, and Checkpoints

This appendix collects the practical details that the labs assume: how the code is laid out, how to install it, which hardware is enough for which lab, and how to get the pretrained weights that the measurement sections use.

B.1 Repository layout

```
code/
  llmsys/              shared package, built up chapter by chapter
    layers.py          norms, MLPs, RoPE, masks, MHA, GQA
    kvcache.py         dynamic and static KV caches
    gpt.py qwen3.py    model definitions + Hugging Face weight loaders
    tokenizer.py       word-level and byte-level BPE tokenizers
    data.py train.py   datasets, loaders, training loop, LR schedule
    generate.py        sampling filters, cached / batched generation, log-probs
    flops.py           napkin-math calculators (params, FLOPs, bytes, rooflines)
    ...                later chapters add eval/, rl/, serving/, dist/, kernels/, ...
  labs/labNN_*/        README.md, starter.py, solution.py, test_labNN.py, run/bench
    scripts
  tests/               tests for the core package
  data/                small, original sample datasets used by the tests
```

The core modules (`layers`, `kvcache`, `gpt`, `qwen3`, `tokenizer`, `data`, `train`, `generate`) all use one model interface:

```
logits = model(idx, cache=None, pad_mask=None)   # idx: (B, T) -> logits: (B, T, V)
```

Passing a `KVCache` turns the same forward pass into an incremental, cached decode step (Chapter 11). Passing `pad_mask` (True for real tokens) masks out the left padding used for batched generation (Chapter 12).

B.2 Installation

```
python -m venv .venv && source .venv/bin/activate
```

```
pip install -r code/requirements.txt          # torch, tokenizers, safetensors, sympy, ...
cd code && python -m pytest tests             # core package self-test (CPU, ~10 s)
```

For GPU work, install the PyTorch build that matches your CUDA driver (`https://pytorch.org/get-started/locally/`). Triton ships with recent Linux CUDA builds of PyTorch. The labs in Appendix A need an NVIDIA or AMD GPU, and their tests skip automatically on CPU.

B.3 Which hardware for which lab

Hardware	What runs comfortably
Laptop CPU	All tests; tiny-model training and RL demos; tokenizer, napkin-math, engine-simulation labs
Apple-silicon laptop (MPS)	GPT-2 small pretraining and fine-tuning; Qwen3-0.6B inference at short contexts
Single 16–24 GB GPU	Everything up to GRPO and distillation on Qwen3-0.6B with short responses; LoRA on 1.7B
Single 80 GB GPU (A100/H100)	Full-length GRPO runs on 0.6B–1.7B; inference on 8B; realistic serving-engine benchmarks
2–8 GPUs	The real-hardware variants of the distributed-training (Chapter 8) and RL-systems (Chapter 19) labs

Table B.1: Rough guide to the hardware each lab needs.

Napkin Math: Will it fit?

Inference needs roughly the weights plus the KV cache: for Qwen3-0.6B in bf16 that is about $0.6\times10^9\times2\approx1.2$ GB of weights plus $2\cdot L\cdot G\cdot d_h\cdot2$ bytes $=2\cdot28\cdot8\cdot128\cdot2\approx115$ KB of cache per token, so about 0.47 GB for a 4096-token response. Full fine-tuning with AdamW needs about 16 bytes per parameter before activations (Chapter 7), or about 9.6 GB for 0.6B parameters. Use `llmsys.flops` to repeat these estimates for any configuration.

B.4 Getting checkpoints and datasets

The tests never touch the network. The `run.py`/`bench.py` scripts expect checkpoints that you download once:

```
pip install huggingface_hub
huggingface-cli download openai-community/gpt2 --local-dir ckpt/gpt2
huggingface-cli download Qwen/Qwen3-0.6B-Base --local-dir ckpt/qwen3-0.6b-base
huggingface-cli download Qwen/Qwen3-0.6B --local-dir ckpt/qwen3-0.6b   # reasoning/chat
```

Listing B.1: Loading the downloaded checkpoints into our from-scratch modules.

```
from safetensors.torch import load_file
from llmsys.gpt import GPTModel, gpt2_config, load_gpt2_hf_state_dict
from llmsys.qwen3 import load_qwen3_from_dir, Qwen3Tokenizer

gpt2 = GPTModel(gpt2_config("gpt2-small (124M)"))
load_gpt2_hf_state_dict(gpt2, load_file("ckpt/gpt2/model.safetensors"))

```

```
qwen = load_qwen3_from_dir("ckpt/qwen3-0.6b-base", size="0.6B", device="cuda")
tok = Qwen3Tokenizer("ckpt/qwen3-0.6b-base/tokenizer.json")
```

Datasets used by the measurement scripts include MATH-500 and the MATH training split (Chapters 14, 17 and 20), the SMS Spam Collection (Chapter 9), an Alpaca-style instruction set (Chapter 10), and any public-domain short story for pretraining (Chapter 7). Each script's `-help` names the source it expects and where to put the file. Small sample files written for this book live in `code/data/` so the tests can run offline.

B.5 Reproducibility and measurement hygiene

- Call `llmsys.utils.set_seed`. On GPU, also set `torch.backends.cudnn.deterministic=True` when you compare numerics. Remember that batched and unbatched runs can differ in the last bits because reduction order changes.
- Time GPU code only after `torch.cuda.synchronize()` (the `llmsys.utils.timer` context manager does this). Discard warm-up iterations, which include compilation, autotuning and allocator growth.
- Report the configuration with every number: hardware, dtype, batch size, sequence length, PyTorch version, and whether `torch.compile` or CUDA graphs were used.
- Prefer distributions to single numbers. Report medians and percentiles (p50/p90/p99) for latencies, and confidence intervals for accuracies (Chapter 14).

Appendix C

References and Further Reading

This appendix collects the works cited across the book: the two source books, the papers they build on, and the systems papers and open-source projects behind the Systems Lens material. Entries are sorted alphabetically by first author.

Source books

- Raschka, S. (2024). *Build a Large Language Model (From Scratch)*. Manning. Code: `https://github.com/rasbt/LLMs-from-scratch`.
- Raschka, S. (2026). *Build a Reasoning Model (From Scratch)*. Manning. Code: `https://github.com/rasbt/reasoning-from-scratch`.

Staying current

The field moves quickly. Good primary sources include arXiv (cs.CL, cs.LG, cs.DC), the proceedings of MLSys, OSDI/SOSP, ISCA, NeurIPS, ICML and ICLR, and the documentation and design notes of the open-source systems discussed in this book: PyTorch, vLLM, SGLang, TensorRT-LLM, Megatron-LM, DeepSpeed, veRL, OpenRLHF, TRL and Triton.

Cited works

[1] Abdin, M., et al. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219.

[2] Agache, A., Brooker, M., Florescu, A., et al. (2020). Firecracker: Lightweight Virtualization for Serverless Applications. NSDI.

[3] Agarwal, R., Vieillard, N., Zhou, Y., et al. (2024). On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes (GKD). ICLR.

[4] Agrawal, A. et al. (2024). Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI.

[5] Ahmadian, A., et al. (2024). Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs (RLOO). ACL.

[6] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP.

[7] Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the Study of SMS Spam Filtering: New Collection and Results. ACM Symposium on Document Engineering (DocEng).

[8] Ansel, J., et al. (2024). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. ASPLOS.

[9] Ba, J. L., Kiros, J. R., Hinton, G. E. (2016). Layer Normalization. arXiv:1607.06450.

[10] Bahdanau, D., Cho, K., Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.

[11] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., Siskind, J. M. (2018). Automatic Differentiation in Machine Learning: a Survey. JMLR.

[12] Beltagy, I., Peters, M. E., Cohan, A. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150.

[13] Bradley, R. A., and Terry, M. E. (1952). Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. Biometrika.

[14] Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. NeurIPS.

[15] Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., & Dao, T. (2024). Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. ICML.

[16] Chainlit contributors (2023). Chainlit: Build conversational AI applications in Python. `https://github.com/Chainlit/chainlit`.

[17] Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., & Jumper, J. (2023). Accelerating Large Language Model Decoding with Speculative Sampling. arXiv:2302.01318.

[18] Chen, L., Ye, Z., Wu, Y., Zhuo, D., Ceze, L., & Krishnamurthy, A. (2024). Punica: Multi-Tenant LoRA Serving. MLSys.

[19] Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

[20] Chen, S., Wong, S., Chen, L., Tian, Y. (2023). Extending Context Window of Large Language Models via Positional Interpolation. arXiv:2306.15595.

[21] Chen, T., Xu, B., Zhang, C., Guestrin, C. (2016). Training Deep Nets with Sublinear Memory Cost. arXiv:1604.06174.

[22] Chiang, W.-L., Zheng, L., Sheng, Y., et al. (2024). Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. ICML.

[23] Child, R., Gray, S., Radford, A., Sutskever, I. (2019). Generating Long Sequences with Sparse Transformers. arXiv:1904.10509.

[24] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP.

[25] Chowdhery, A., Narang, S., Devlin, J., et al. (2022). PaLM: Scaling Language Modeling with Pathways. JMLR 2023 (arXiv:2204.02311).

[26] Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 (ICLR 2024).

[27] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Re, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS.

[28] Dauphin, Y. N., Fan, A., Auli, M., Grangier, D. (2017). Language Modeling with Gated Convolutional Networks. ICML.

[29] DeepSeek-AI (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434.

[30] DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437.

[31] DeepSeek-AI (2025). DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. Technical report.

[32] DeepSeek-AI (2025). DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning. arXiv:2511.22570.

[33] DeepSeek-AI, Guo, D., Yang, D., et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.

[34] Dehghani, M., et al. (2023). Scaling Vision Transformers to 22 Billion Parameters. ICML.

[35] Dettmers, T., & Zettlemoyer, L. (2023). The case for 4-bit precision: k-bit Inference Scaling Laws. ICML.

[36] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS.

[37] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS.

[38] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.

[39] Dong, Y., Ruan, C. F., Cai, Y., Lai, R., Xu, Z., Zhao, Y., & Chen, T. (2024). XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. arXiv:2411.15100.

[40] Dósa, G. (2007). The Tight Bound of First Fit Decreasing Bin-Packing Algorithm Is FFD(I) <= 11/9 OPT(I) + 6/9. ESCAPE, LNCS 4614.

[41] Dubois, Y., Li, X., Taori, R., et al. (2023). AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback. NeurIPS.

[42] Elhoushi, M., et al. (2024). LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. ACL.

[43] Fedus, W., Zoph, B., Shazeer, N. (2022). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR.

[44] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR.

[45] Frantar, E., Castro, R. L., Chen, J., Hoefler, T., & Alistarh, D. (2025). MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models. PPoPP.

[46] Fu, W., Gao, J., Shen, X., et al. (2025). AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. arXiv:2505.24298.

[47] Fu, Y., Bailis, P., Stoica, I., & Zhang, H. (2024). Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. ICML.

[48] Gage, P. (1994). A New Algorithm for Data Compression. The C Users Journal.

[49] Gale, T., Narayanan, D., Young, C., Zaharia, M. (2023). MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. MLSys.

[50] GDPO (2026). Group reward-Decoupled Normalization Policy Optimization for multi-reward RL. arXiv preprint.

[51] Gu, Y., Dong, L., Wei, F., Huang, M. (2024). MiniLLM: Knowledge Distillation of Large Language Models. ICLR.

[52] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. ICML.

[53] Hägele, A., Bakouch, E., Kosson, A., et al. (2024). Scaling Laws and Compute-Optimal Training beyond Fixed Training Durations. NeurIPS.

[54] He, H., and Thinking Machines Lab (2025). Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism (blog).

[55] He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR.

[56] Hendrycks, D., Burns, C., Basart, S., et al. (2021). Measuring Massive Multitask Language Understanding. ICLR.

[57] Hendrycks, D., Burns, C., Kadavath, S., et al. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS Datasets and Benchmarks.

[58] Hendrycks, D., Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). arXiv:1606.08415.

[59] Hinton, G., Vinyals, O., Dean, J. (2015). Distilling the Knowledge in a Neural Network. NeurIPS Deep Learning Workshop. arXiv:1503.02531.

[60] Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training Compute-Optimal Large Language Models. NeurIPS.

[61] Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The Curious Case of Neural Text Degeneration. ICLR.

[62] Houlsby, N., Giurgiu, A., Jastrzebski, S., et al. (2019). Parameter-Efficient Transfer Learning for NLP. ICML.

[63] Hsu, P.-L., Dai, Y., Kothapalli, V., et al. (2024). Liger Kernel: Efficient Triton Kernels for LLM Training. arXiv:2410.10989.

[64] Hu, E. J., Shen, Y., Wallis, P., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.

[65] Hu, J., Wu, X., Zhu, Z., et al. (2024). OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143.

[66] Hu, S., Tu, Y., Han, X., et al. (2024). MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. arXiv:2404.06395.

[67] Huang, J., Chen, X., Mishra, S., et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR.

[68] Huang, Y., Cheng, Y., Bapna, A., et al. (2019). GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. NeurIPS.

[69] Hunter, D. R. (2004). MM Algorithms for Generalized Bradley-Terry Models. The Annals of Statistics.

[70] Jacobs, S. A., Tanaka, M., Zhang, C., et al. (2023). DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. arXiv:2309.14509.

[71] Johnson, D. S. (1973). Near-Optimal Bin Packing Algorithms. PhD thesis, MIT.

[72] Kalamkar, D., et al. (2019). A Study of BFLOAT16 for Deep Learning Training. arXiv:1905.12322.

[73] Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.

[74] Katharopoulos, A., Vyas, A., Pappas, N., Fleuret, F. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML.

[75] Kimi Team (2025). Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv:2501.12599.

[76] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022). Large Language Models are Zero-Shot Reasoners. NeurIPS.

[77] Korthikanti, V., Casper, J., Lym, S., et al. (2022). Reducing Activation Recomputation in Large Transformer Models. MLSys 2023 (arXiv:2205.05198).

[78] Krell, M. M., Kosec, M., Perez, S. P., & Fitzgibbon, A. (2021). Efficient Sequence Packing without Cross-contamination: Accelerating Large Language Models without Impacting Performance. arXiv:2107.02027.

[79] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP.

[80] Lepikhin, D., Lee, H., Xu, Y., et al. (2021). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. ICLR.

[81] Lester, B., Al-Rfou, R., & Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP.

[82] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast Inference from Transformers via Speculative Decoding. ICML.

[83] Li, S., Zhao, Y., Varma, R., et al. (2020). PyTorch Distributed: Experiences on Accelerating Data Parallel Training. VLDB.

[84] Li, X. L., & Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL.

[85] Li, Y., Wei, F., Zhang, C., & Zhang, H. (2024). EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. ICML.

[86] Liang, W., Liu, T., Wright, L., et al. (2024). TorchTitan: One-stop PyTorch Native Solution for Production Ready LLM Pre-training. arXiv:2410.06511.

[87] Lightman, H., Kosaraju, V., Burda, Y., et al. (2023). Let's Verify Step by Step. arXiv:2305.20050 (source of the MATH-500 split).

[88] Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2024). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. MLSys.

[89] Little, J. D. C. (1961). A Proof for the Queuing Formula L = lambda W. Operations Research.

[90] Liu, H., Tam, D., Muqeeth, M., et al. (2022). Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning ((IA)^3). NeurIPS.

[91] Liu, H., Zaharia, M., Abbeel, P. (2023). Ring Attention with Blockwise Transformers for Near-Infinite Context. arXiv:2310.01889 (ICLR 2024).

[92] Liu, S.-Y., Wang, C.-Y., Yin, H., et al. (2024). DoRA: Weight-Decomposed Low-Rank Adaptation. ICML.

[93] Liu, Z., Chen, C., Li, W., et al. (2025). Understanding R1-Zero-Like Training: A Critical Perspective (Dr. GRPO). arXiv:2503.20783.

[94] Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. ICML.

[95] Llama Team, AI @ Meta (2024). The Llama 3 Herd of Models. arXiv:2407.21783.

[96] Loshchilov, I., Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.

[97] Madaan, A., Tandon, N., Gupta, P., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS.

[98] McCandlish, S., Kaplan, J., Amodei, D., OpenAI Dota Team (2018). An Empirical Model of Large-Batch Training. arXiv:1812.06162.

[99] Miao, X., et al. (2024). SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. ASPLOS.

[100] Micikevicius, P., et al. (2022). FP8 Formats for Deep Learning. arXiv:2209.05433.

[101] Micikevicius, P., Narang, S., Alben, J., et al. (2018). Mixed Precision Training. ICLR.

[102] Mikolov, T., Chen, K., Corrado, G., Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. ICLR Workshop.

[103] Milakov, M., & Gimelshein, N. (2018). Online normalizer calculation for softmax. arXiv:1805.02867.

[104] MiniMax (2025). MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention (CISPO). arXiv:2506.13585.

[105] Moritz, P., Nishihara, R., Wang, S., et al. (2018). Ray: A Distributed Framework for Emerging AI Applications. OSDI.

[106] Muennighoff, N., Rush, A., Barak, B., et al. (2023). Scaling Data-Constrained Language Models. NeurIPS.

[107] Muennighoff, N., Yang, Z., Shi, W., et al. (2025). s1: Simple Test-Time Scaling. arXiv:2501.19393.

[108] Narayanan, D., Harlap, A., Phanishayee, A., et al. (2019). PipeDream: Generalized Pipeline Parallelism for DNN Training. SOSP.

[109] Narayanan, D., Shoeybi, M., Casper, J., et al. (2021). Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. SC.

[110] NVIDIA (2022). NVIDIA H100 Tensor Core GPU Architecture. Whitepaper.

[111] Olsson, C., et al. (2022). In-context Learning and Induction Heads. Transformer Circuits Thread.

[112] OpenAI (2024). Learning to Reason with LLMs (OpenAI o1). OpenAI blog.

[113] OpenAI (2024). Learning to Reason with LLMs. OpenAI blog post (o1 announcement).

[114] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS.

[115] Pagnoni, A., et al. (2024). Byte Latent Transformer: Patches Scale Better Than Tokens. arXiv:2412.09871.

[116] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS.

[117] Patarasuk, P., Yuan, X. (2009). Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. Journal of Parallel and Distributed Computing 69(2).

[118] Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, I., Maleki, S., & Bianchini, R. (2024). Splitwise: Efficient Generative LLM Inference Using Phase Splitting. ISCA.

[119] Penedo, G., et al. (2024). The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. NeurIPS Datasets and Benchmarks.

[120] Peng, B., Quesnelle, J., Fan, H., Shippole, E. (2023). YaRN: Efficient Context Window Extension of Large Language Models. arXiv:2309.00071.

[121] Piché, A., et al. (2025). PipelineRL: Faster On-policy Reinforcement Learning for Long Sequence Generation. arXiv preprint.

[122] Pope, R. et al. (2023). Efficiently Scaling Transformer Inference. MLSys.

[123] Prabhu, R. et al. (2025). vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. ASPLOS.

[124] Press, O., Smith, N. A., Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. ICLR.

[125] Qi, P., Wan, X., Huang, G., Lin, M. (2024). Zero Bubble Pipeline Parallelism. ICLR.

[126] Qin, R. et al. (2024). Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079.

[127] Rabe, M. N., Staats, C. (2021). Self-attention Does Not Need O(n^2) Memory. arXiv:2112.05682.

[128] Radford, A., Narasimhan, K., Salimans, T., Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI technical report.

[129] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI Technical Report.

[130] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS.

[131] Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5). JMLR.

[132] Rajbhandari, S., Rasley, J., Ruwase, O., He, Y. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. SC.

[133] Rajbhandari, S., Ruwase, O., Rasley, J., Smith, S., He, Y. (2021). ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. SC.

[134] Raschka, S. (2025). The State of Reinforcement Learning for LLM Reasoning. Ahead of AI (blog).

[135] Rasley, J., Rajbhandari, S., Ruwase, O., He, Y. (2020). DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. KDD.

[136] Ren, J., Rajbhandari, S., Aminabadi, R. Y., et al. (2021). ZeRO-Offload: Democratizing Billion-Scale Model Training. USENIX ATC.

[137] Rouhani, B. D., et al. (2023). Microscaling Data Formats for Deep Learning. arXiv:2310.10537.

[138] Rumelhart, D. E., Hinton, G. E., Williams, R. J. (1986). Learning Representations by Back-Propagating Errors. Nature.

[139] Schaeffer, R., Miranda, B., Koyejo, S. (2023). Are Emergent Abilities of Large Language Models a Mirage? NeurIPS.

[140] Schulman, J. (2020). Approximating KL Divergence. Blog post (joschu.net).

[141] Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR.

[142] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

[143] Sennrich, R., Haddow, B., Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.

[144] Sergeev, A., Del Balso, M. (2018). Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. arXiv:1802.05799.

[145] Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. NeurIPS.

[146] Shao, Z., et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.

[147] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150.

[148] Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.

[149] Shazeer, N., et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR.

[150] Shen, G., Wang, Z., Delalleau, O., et al. (2024). NeMo-Aligner: Scalable Toolkit for Efficient Model Alignment. arXiv:2405.01481.

[151] Sheng, G., Zhang, C., Ye, Z., et al. (2025). HybridFlow: A Flexible and Efficient RLHF Framework (veRL). EuroSys.

[152] Sheng, Y. et al. (2024). Fairness in Serving Large Language Models. OSDI.

[153] Sheng, Y., Cao, S., Li, D., et al. (2024). S-LoRA: Serving Thousands of Concurrent LoRA Adapters. MLSys.

[154] Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., & Catanzaro, B. (2019). Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053.

[155] Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314.

[156] Spector, B. F., Arora, S., Singhal, A., Fu, D. Y., & Re, C. (2024). ThunderKittens: Simple, Fast, and Adorable AI Kernels. arXiv:2410.20399.

[157] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.

[158] Sutskever, I., Vinyals, O., Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.

[159] Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023). Stanford Alpaca: An Instruction-following LLaMA Model. GitHub repository.

[160] Team OLMo (2025). OLMo 3. Technical report, Allen Institute for AI.

[161] Thakur, R., Rabenseifner, R., Gropp, W. (2005). Optimization of Collective Communication Operations in MPICH. International Journal of High Performance Computing Applications 19(1).

[162] Tillet, P., Kung, H. T., & Cox, D. (2019). Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL.

[163] Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.

[164] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). Attention Is All You Need. NeurIPS.

[165] von Werra, L., Belkada, Y., Tunstall, L., et al. (2020-). TRL: Transformer Reinforcement Learning. GitHub: huggingface/trl.

[166] Wang, L., Yang, N., Huang, X., et al. (2024). Improving Text Embeddings with Large Language Models. ACL.

[167] Wang, S., Wei, J., Sabne, A., et al. (2023). Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. ASPLOS.

[168] Wang, X., Wei, J., Schuurmans, D., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR.

[169] Wei, J., et al. (2022). Emergent Abilities of Large Language Models. TMLR.

[170] Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.

[171] Willard, B. T., & Louf, R. (2023). Efficient Guided Generation for Large Language Models. arXiv:2307.09702.

[172] Williams, R. J. (1992). Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. Machine Learning 8.

[173] Williams, S., Waterman, A., Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM.

[174] Wu, S., et al. (2023). BloombergGPT: A Large Language Model for Finance. arXiv:2303.17564.

[175] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML.

[176] Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. ICLR.

[177] Xiong, R., et al. (2020). On Layer Normalization in the Transformer Architecture. ICML.

[178] Yang, A. et al. (2025). Qwen3 Technical Report. arXiv:2505.09388.

[179] Yao, F., et al. (2025). Your Efficient RL Framework Secretly Brings You Off-Policy RL Training (truncated importance sampling, verl). Blog post.

[180] Yao, S., Yu, D., Zhao, J., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS.

[181] Ye, Z. et al. (2025). FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. MLSys.

[182] Young, J. W. (1974). A First Order Approximation to the Optimum Checkpoint Interval. Communications of the ACM 17(9).

[183] Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., Chun, B.-G. (2022). Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI.

[184] Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., Lewis, M. (2023). MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers. NeurIPS.

[185] Yu, Q., Zhang, Z., Zhu, R., et al. (2025). DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476.

[186] Zhang, B., Sennrich, R. (2019). Root Mean Square Layer Normalization. NeurIPS.

[187] Zhao, Y., Gu, A., Varma, R., et al. (2023). PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. VLDB.

[188] Zheng, C., et al. (2025). Group Sequence Policy Optimization (GSPO). arXiv:2507.18071.

[189] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks.

[190] Zheng, L., Yin, L., Xie, Z., et al. (2024). SGLang: Efficient Execution of Structured Language Model Programs. NeurIPS.

[191] Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., & Zhang, H. (2024). DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI.

[192] Zhou, C., Liu, P., Xu, P., et al. (2023). LIMA: Less Is More for Alignment. NeurIPS.

Appendix D

Solutions to Exercises

These are worked solutions to the end-of-chapter exercises. Coding exercises that overlap with a lab point to the reference `solution.py` in that lab rather than repeating the code. For the measurement exercises, the solution describes the outcome the cost model predicts and how to check it; your own numbers will depend on your hardware.

Chapter 1: The Landscape

Exercise 1.1 By the definition of conditional probability, $p(x_1, \ldots, x_T) = p(x_T \mid x_{<T})\, p(x_1, \ldots, x_{T-1})$; applying this recursively down to $p(x_1)$ gives the product. No modelling assumption is involved, so an exact autoregressive model can represent any distribution. With a causal mask, the output at position t depends only on $x_{\leq t}$, so position t is a valid prediction of x_{t+1} for every t: one forward pass over T tokens yields T supervised predictions (the last uses the next token in the stream). A masked LM computes a loss only at the $\approx 0.15T$ masked positions, because unmasked positions can see their own answer through bidirectional attention.

Exercise 1.2 Perplexity is $e^{2.0} \approx 7.39$: the model is, on average, as uncertain as if it were choosing uniformly among about 7.4 tokens. A uniform model has loss $\ln 50{,}257 \approx 10.82$ nats and perplexity 50,257. This is a useful sanity check: a freshly initialised model should start near 10.8.

Exercise 1.3 Wikipedia: $0.03 \times 300\text{B} = 9\text{B}$ tokens sampled from a 3B-token source, about 3 epochs. Common Crawl: $0.60 \times 300\text{B} = 180\text{B}$ from 410B, about 0.44 epochs. Repeating small, clean, information-dense sources raises the quality of the effective training distribution. The cost is a risk of memorisation and diminishing returns after a few epochs, so heavy repetition is reserved for the highest-quality data.

Exercise 1.4 $C = 64 \times 30 \times 86{,}400\,\text{s} \times 989 \times 10^{12} \times 0.35 \approx 5.74 \times 10^{22}$ FLOPs. With $C = 6ND$ and $D = 20N$, $C = 120N^2$, so $N = \sqrt{C/120} \approx 2.2 \times 10^{10}$ (about 22B parameters) and $D \approx 4.4 \times 10^{11}$ tokens. If the model will be served heavily, total cost is training plus $2N$ FLOPs per served token. Shrinking N (say to 7B) and spending the same compute on more tokens ($D = C/(6N) \approx 1.4\text{T}$) gives a slightly worse model at equal compute but a $3\times$ cheaper one to serve, which is usually the better trade.

Exercise 1.5 bf16: weights $4.02 \times 10^9 \times 2 = 8.04$ GB, KV at 4,096 tokens = $147{,}456 \times 4{,}096 = 0.60$ GB, total 8.64 GB, giving $8.64/3.35 \times 10^{12} \approx 2.58$ ms, or about 387 tokens/s. int4 weights: 2.01 GB + 0.60 GB = 2.61 GB, about 0.78 ms, or about 1,280 tokens/s. Weights still dominate, but the KV share of traffic rises from 7% to 23%; at longer contexts or larger batches the KV cache becomes the dominant term, which motivates KV-cache quantisation (Chapter 13).

Exercise 1.6 Per step, compute time is $B \cdot 2N/F$ and memory time is Nb/W (weights read once, shared by the batch). Equating gives $B^\star = bF/(2W)$. For an H100 in bf16, $B^\star = 2 \times 989/(2 \times 3.35) \approx 295$, matching the lab's 296–320 (the lab includes the LM head and attention FLOPs and embedding bytes, which shift the value slightly). With long contexts each sequence adds its own KV bytes, which are *not* shared across the batch. Attention over the cache has intensity of about 1 FLOP/byte regardless of batch size, so the step needs a larger batch, or never becomes compute-bound when KV traffic dominates.

Exercise 1.7 Qwen3-8B uses 147,456 bytes of KV per token. Short answers: $500 + 300 = 800$ tokens, 118 MB per request, about 339 concurrent requests in 40 GB. With thinking: about $500 + 6{,}000 = 6{,}500$ tokens, 958 MB per request, about 41 concurrent requests. Under memory-bound decoding, throughput scales roughly with batch, so throughput in tokens/s drops by about 8×. Each request also emits about 8× more tokens, so requests per second fall by roughly 60×.

Exercise 1.8 $0.97^{30} \approx 0.40$. If wrong answers are spread over, say, ten distinct values, a simulation of plurality voting (counting ties as failures) gives about 0.57 success at 5 samples, 0.81 at 11, 0.89 at 15 and 0.96 at 21, so 15–17 samples are needed for 0.9. Independence is optimistic because a model's errors are correlated: the same misconception recurs across samples, concentrating wrong votes on one value and making plurality voting far less effective.

Exercise 1.9 Example probes: (i) "All squares have five sides. ABCD is a square. How many sides does ABCD have?" (closed-world answer 5; a good reasoner flags the false premise). (ii) A freshly generated puzzle with random names and quantities, for instance a scheduling constraint problem produced by a script with a computed answer. (iii) A question whose answer runs against frequency, for instance "Which is heavier, a kilogram of feathers or 900 g of steel?". Base models often continue the text without answering; instruction-tuned models answer fluently but fall for (iii) more often; reasoning models do best on (ii). Score automatically with programmatic answers and the verifier of Chapter 14, and for (i) with a rubric for "flags the inconsistency" judged by an LLM (Chapter 14).

Exercise 1.10 A reasonable ranking is distillation, then RL, then inference-time scaling. Strict latency rules out heavy per-query sampling. One week of compute is enough to fine-tune on traces from a stronger teacher, which is cheap and stable, and the fine-tuned model serves at its normal speed. RL is attractive if a reliable automatic reward exists (for instance, review comments that can be checked against later bug fixes) but is costlier to make work; it can follow distillation as a second stage. A light form of inference-time scaling (one short chain of thought) can still be used if the latency budget allows.

Chapter 2: PyTorch and the Machine Underneath

Exercise 2.1 Contiguous $(2, 3, 4)$ has strides $(12, 4, 1)$. After `transpose(0, 2)` the shape is $(4, 3, 2)$ and the strides are $(1, 4, 12)$, over the same storage. `.view(-1)` requires that the flattened order equal the storage order, which no longer holds, so it raises an error. `.reshape(-1)` falls back to a copy: one kernel that reads and writes all 24 elements (in general, $2\times$ the tensor's bytes of traffic plus a new allocation).

Exercise 2.2 bf16 has 7 explicit mantissa bits, so the spacing of representable numbers in $[1, 2)$ is $2^{-7} = 0.0078125$. With round-to-nearest-even, $1 + 2^{-8}$ rounds back to 1, so any δ slightly larger than $2^{-8} \approx 0.0039$ is needed to change the value. An update of 10^{-4} is lost entirely: the weight never moves. fp32 master weights have spacing $2^{-23} \approx 1.2 \times 10^{-7}$ near 1, so the update is applied exactly in fp32 and the bf16 copy is re-derived from the master after each step; small updates accumulate until they cross a bf16 rounding boundary.

Exercise 2.3 With $a = \sigma(z)$ and $\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)]$: $\partial\mathcal{L}/\partial a = -y/a + (1 - y)/(1 - a) = (a - y)/(a(1 - a))$ and $\partial a/\partial z = a(1 - a)$, so $\partial\mathcal{L}/\partial z = a - y$. For a batch with mean loss, $\nabla_{\boldsymbol{w}} = \frac{1}{N}\mathbf{X}^\top(\boldsymbol{a} - \boldsymbol{y})$ and $\partial/\partial b = \frac{1}{N}\sum_i (a_i - y_i)$. This is exactly `logistic_regression_grads` in the lab solution, which matches autograd to 10^{-6}.

Exercise 2.4 Parameters: $(784 \cdot 512 + 512) + (512 \cdot 256 + 256) + (256 \cdot 10 + 10) = 401{,}920 + 131{,}328 + 2{,}570 = 535{,}818$. In fp32: weights 2.14 MB, gradients 2.14 MB, Adam's two moments 4.29 MB, for a total of 8.57 MB (16 bytes/parameter). Forward FLOPs for a batch of 128: $2 \times 128 \times (784 \cdot 512 + 512 \cdot 256 + 256 \cdot 10) \approx 1.37 \times 10^8$ (biases and ReLU add well under 1%).

Exercise 2.5 A100: $I^\star = 312/2.04 \approx 153$ FLOP/byte; a bf16 square GEMM has $I = n/3$, so it becomes compute-bound at $n \approx 460$. RTX 4090: $I^\star = 165/1.008 \approx 164$, so $n \approx 490$. The decode critical batch is $B^\star \approx bI^\star/2 = I^\star$ in bf16, so the 4090 needs a slightly larger batch (about 164 versus 153). Both need far smaller batches than an H100 (≈295), whose compute has outgrown its bandwidth.

Exercise 2.6 $d = 4096$, hidden $4d = 16{,}384$, bf16. $BT = 1$: each GEMM does $2 \times 4096 \times 16{,}384 \approx 1.34 \times 10^8$ FLOPs and moves about $2 \times 4096 \times 16{,}384 \approx 134$ MB (the weight dominates), so $I \approx 1$: memory-bound, about 40 µs each on an H100. GELU touches 16,384 elements, which is negligible but still costs a launch. $BT = 4096$: GEMM 1 does 5.5×10^{11} FLOPs and moves $2(4096^2 + 2 \cdot 4096 \cdot 16{,}384) \approx 302$ MB, so $I \approx 1{,}800$: compute-bound, ≈ 0.56 ms at peak; GEMM 2 is the same. GELU reads and writes $4096 \times 16{,}384$ bf16 values, 268 MB, ≈ 0.08 ms, memory-bound. Fusing GELU into GEMM 1's epilogue removes that round trip (the GEMM output is activated in registers before being stored), saving ≈ 0.08 ms of ≈ 1.2 ms, about 7%. Fusion matters far more for chains of elementwise ops without a GEMM to hide behind.

Exercise 2.7 Gradients are 14 GB in bf16. A ring all-reduce sends and receives $2(P - 1)/P \times 14 \approx 24.5$ GB per GPU for $P = 8$. Over NVLink at 300 GB/s this takes ≈ 82 ms, easily hidden behind a 1.5 s step. Over 100 Gb/s Ethernet (12.5 GB/s) it takes ≈ 2.0 s, longer than the compute, so communication cannot be hidden and the step at least doubles. The model does not fit for DDP anyway: 7B parameters × 16 bytes = 112 GB of weights, gradients and Adam state exceeds 80 GB before activations. This requires ZeRO/FSDP sharding (Chapter 8).

Exercise 2.8 For 1,024 elements, both eager and compiled versions are dominated by per-op overhead: eager time is about 100× the per-launch cost (hundreds of µs to milliseconds); this is regime (b) of Fig. 2.10. For 10^8 elements, each op is a memory-bound kernel taking ≈ 0.24 ms in fp32 on an H100, so eager time is about 24 ms and overhead is hidden; this is regime (a). `torch.compile` fuses the chain into one kernel that reads and writes once. The large tensor speeds up by up to $\approx 100\times$ in traffic terms, and the small tensor speeds up because 100 launches became one; this is regime (c). With `reduce-overhead`, CUDA graphs remove the remaining launch cost.

Exercise 2.9 At batch 8, the profile is dominated by many small kernels: `addmm`/`mm` at low intensity (memory-bound, weight reads dominate), plus optimiser elementwise kernels (`add_`, `mul_`, foreach ops) that touch every parameter and are memory-bound. At batch 4096, `addmm`/`mm` in forward and backward take the large majority of device time and are compute-bound; elementwise ops shrink to a small fraction. Optimiser cost is independent of batch size, so its share falls as the batch grows.

Exercise 2.10 Achieved model FLOP/s $= 2.1 \times 10^6 \times 6 \times 3 \times 10^9 = 3.78 \times 10^{16}$; peak $= 64 \times 989 \times 10^{12} = 6.33 \times 10^{16}$; MFU $\approx 60\%$. That is at the very top of what well-tuned dense training achieves, so ask: are padding tokens counted as tokens? Is N the total or non-embedding count? Are attention FLOPs or recomputation included (that would make it hardware FLOPs utilisation, HFU, not MFU)? Is the peak taken as the bf16 or the fp8 rate? Is the throughput a steady-state peak or an average including evaluation and checkpoint stalls?

Chapter 3: Text Data, Tokenization, and the Input Pipeline

Exercise 3.1. A window starting at offset s_0 needs tokens $s_0, \ldots, s_0 + T$, so valid starts are $0, s, 2s, \cdots \leq N - T - 1$, giving $n = \lfloor (N - T - 1)/s \rfloor + 1$ windows (for $N > T$). Input positions per epoch: $nT \approx NT/s$. A typical (interior) token is therefore an input about T/s times: T times for $s = 1$, 4 times for $s = T/4$, once for $s = T$.

Exercise 3.2. (i) `"a b"` (two spaces): decoding re-joins with one space. (ii) `'say "hi"'`: tokens `say`, `"`, `hi`, `"` decode to `'say " hi"'` because the regex only removes spaces *before* punctuation, leaving the space after the opening quote. (iii) any string with a tab or newline: whitespace is discarded and replaced by single spaces. Byte-level BPE never discards bytes: every byte belongs to exactly one token and decoding concatenates the tokens' bytes, so `decode(encode(s)) == s` for every valid UTF-8 string.

Exercise 3.3. Random lowercase strings contain few frequent English bigrams, so only short merges apply: expect roughly 1.5–2.5 bytes per token. A script absent from the training corpus has no merges at all among its bytes, so each UTF-8 byte is its own token: for a 3-byte script that is 1 byte/token, i.e. three tokens per character. Compression depends entirely on which byte sequences were frequent in the tokenizer's training data.

Exercise 3.4. Training learned merges in a specific order and the training corpus was segmented by applying them in that order; the model only ever saw segmentations produced that way. Example: merges $[(b, c) \to X,\ (a, b) \to Y]$ and input `ababc`. By rank: (b, c) first gives `a b a X`, then (a, b) gives `Y a X`. By in-input frequency: (a, b) occurs twice, so it is merged first,

giving `Y Y c`, after which nothing applies. Different token sequence, one the model may never have seen in training.

Exercise 3.5. Naive: each of M merges recounts all W symbols and scans P pairs: $O(M(W+P))$. Incremental: initial counting $O(W)$, then merge m costs $O(\sum_{w\in a_m}|w|)$ plus $O(P)$ to find the maximum, i.e. $O(W+\sum_m |a_m|\bar{\ell}+MP)$. A max-heap keyed by count makes the maximum $O(\log P)$, but counts *decrease* when words are rewritten, and binary heaps do not support decrease-key cheaply. The standard solution is lazy deletion: push a new (count, pair) entry whenever a count changes, and when popping, discard entries whose count no longer matches the current table (or use an indexed priority queue).

Exercise 3.6. With $d = 2048$, $L = 24$ the non-embedding parameters are about $12Ld^2 \approx 1.21$B. (a) Tied embeddings: $32\text{k}\cdot 2048 = 65.5$M vs. $256\text{k}\cdot 2048 = 524$M. (b) Logit FLOPs per token $2Vd$: 131 MFLOP vs. 1.05 GFLOP, compared to ≈ 2.4 GFLOP for the rest of the forward pass. (c) 10^{11} vs. $\approx 0.85\times 10^{11}$ tokens. (d) fp32 logits for $B = 4$, $T = 8192$: $4\cdot 8192\cdot V\cdot 4\,\text{B} = 4.2\,\text{GB}$ vs. 33.6 GB. Training compute $\approx 6(N+Vd)D$: 7.7×10^{20} vs. 9.1×10^{20} FLOPs, so for this size the small vocabulary is cheaper overall. For a 0.5B model the large vocabulary would be most of the parameter budget; choose $\approx$ 32–64k. For a 70B model the embedding is a few percent of parameters and logits a few percent of FLOPs, while 15% shorter sequences cut attention, KV cache and serving cost: choose the large vocabulary.

```
emb = torch.nn.Embedding(10, 4)
ids = torch.tensor([3, 7, 3])
g = torch.randn(3, 4)
(emb(ids) * g).sum().backward()
nz = emb.weight.grad.abs().sum(1) > 0
assert nz.nonzero().flatten().tolist() == [3, 7]
assert torch.allclose(emb.weight.grad[3], g[0] + g[2])
```

With `sparse=True` the gradient is a sparse COO tensor holding only the touched rows; it is accepted by `optim.SGD`, `optim.SparseAdam` and `optim.Adagrad`, but not by `Adam`/`AdamW`.

Exercise 3.8. With `sa = SelfAttention(d, d)` (no positions), a permutation matrix Π gives $\text{sa}(\Pi X) = \Pi\,\text{sa}(X)$, because ΠXW permutes rows of Q, K, V, the score matrix becomes $\Pi S\Pi^\top$, the row softmax commutes with that, and $\Pi A\Pi^\top \Pi V = \Pi AV$. Test: `perm = torch.randperm(T); assert torch.allclose(sa(x[perm]), sa(x)[perm])`. After adding `pos_emb(arange(T))` to the input the property fails, since position vectors are not permuted with the tokens.

Exercise 3.9. For frequency ω, $\sin\omega(p+k) = \sin\omega p\cos\omega k + \cos\omega p\sin\omega k$ and $\cos\omega(p+k) = \cos\omega p\cos\omega k - \sin\omega p\sin\omega k$. Hence each $(\sin,\cos)$ pair is mapped by the 2×2 block $\left(\begin{smallmatrix}\cos\omega k & \sin\omega k\\ -\sin\omega k & \cos\omega k\end{smallmatrix}\right)$, and $\mathbf{M}_k$ is block diagonal with these blocks, independent of p. A linear layer can therefore implement "look k positions back" uniformly for all positions.

```
def doc_causal_mask(ids, eot_id):
    T = len(ids)
    ends = torch.as_tensor(ids) == eot_id
    doc = torch.cumsum(ends.long(), 0) - ends.long()  # EOT belongs to its doc
```

```
    same = doc[:, None] == doc[None, :]
    return same & torch.tril(torch.ones(T, T, dtype=torch.bool))  # True = allowed
```

The plain causal mask allows $\approx T^2/2$ scores; four equal documents allow $4 \cdot (T/4)^2/2 = T^2/8$, so three quarters of the causal scores are skipped.

Exercise 3.11. With a warm page cache, reads are limited by Python per-item overhead (tens of thousands of windows per second, as in the chapter's measurement). After dropping caches, each random window is one small random read: on an NVMe SSD (hundreds of thousands of IOPS at high queue depth, far fewer at queue depth 1 from a single Python thread) expect thousands to tens of thousands of windows/s; on a hard disk, about 100/s. Multiple loader workers raise the effective queue depth and recover throughput.

Chapter 4: Attention Mechanisms

Exercise 4.1. The "chip" row of $\mathbf{A}$ is $(0.137, 0.196, 0.219, 0.154, 0.123, 0.171)$, giving $\boldsymbol{z}^{(6)} = (0.442, 0.556, 0.522)$. "chip" is the last token, so the causal row is identical: it may see every position.

Exercise 4.2. Without scaling, score standard deviation grows as $\sqrt{d_k}$, so the maximum softmax probability approaches 1 as d_k grows (nearly one-hot rows); with scaling the scores have unit variance and the maximum probability stays roughly constant in d_k (it depends on T). The softmax Jacobian has entries $\alpha_i(\delta_{ij} - \alpha_j)$: for a one-hot row all entries are ≈ 0, so almost no gradient reaches the queries and keys.

Exercise 4.3. If the parameter version computes `x @ W` with `W` of shape $(d_{\text{in}}, d_{\text{out}})$, set `linear.weight.data = W.T.clone()` for each of Q, K, V (`nn.Linear` computes `x @ weight.T`). The outputs then agree to float precision.

Exercise 4.4. Route 1 gives $\tilde{\alpha}_{ij} = \frac{e^{s_{ij}}/Z_i}{\sum_{k \le i} e^{s_{ik}}/Z_i} = \frac{e^{s_{ij}}}{\sum_{k \le i} e^{s_{ik}}}$ for $j \le i$ and 0 otherwise, which is the route-2 softmax over the visible prefix; Z_i (which contains future scores) cancels. Because the result is algebraically independent of s_{ij} for $j > i$, the Jacobian of output 2 with respect to input 5 is zero; autograd returns zero up to rounding (fp32: around 10^{-8} or exactly 0).

Exercise 4.5. x: $(4, 1024, 1024)$; q, k, v: $(4, 1024, 1024) \to (4, 1024, 16, 64) \to (4, 16, 1024, 64)$; scores and weights: $(4, 16, 1024, 1024)$; **AV**: $(4, 16, 1024, 64) \to (4, 1024, 1024)$; output $(4, 1024, 1024)$. Parameters: $3(1024^2 + 1024) + (1024^2 + 1024) = 4{,}198{,}400$.

Exercise 4.6. Per token per layer, the other forward FLOPs are projections $8d^2$ plus SwiGLU $6d\,d_{\text{ff}}$. With causal masking exploited, a token's score and value products average $2Td$ over a sequence of length T. Equality: $T = (8d^2 + 6dd_{\text{ff}})/(2d) = 4d + 3d_{\text{ff}} = 16{,}384 + 33{,}024 = 49{,}408$ tokens.

Exercise 4.7. Induction: with (m, ℓ) correct for prefix P, i.e. $\ell = \sum_{j \in P} e^{x_j - m}$, after block B we have $\ell e^{m - m'} + \sum_{j \in B} e^{x_j - m'} = \sum_{j \in P \cup B} e^{x_j - m'}$ with $m' = \max_{P \cup B} x_j$. Merge of disjoint partial states:

```python
def merge(m1, l1, o1, m2, l2, o2):      # o = un-normalised sum e^{s-m} v
    m = torch.maximum(m1, m2)
    a1, a2 = torch.exp(m1 - m), torch.exp(m2 - m)
    return m, a1 * l1 + a2 * l2, a1[..., None] * o1 + a2[..., None] * o2
```

Split-KV decoding computes partial states over KV chunks in parallel and reduces them with `merge`; ring attention merges the partial state from each key/value block as it arrives from the neighbouring device.

Exercise 4.8. For $\boldsymbol{y} = \mathrm{softmax}(\boldsymbol{s})$, $\partial y_j/\partial s_k = y_j(\delta_{jk} - y_k)$, so $ds_k = \sum_j dy_j\, y_j(\delta_{jk} - y_k) = y_k(dy_k - \sum_j y_j dy_j)$. In matrix form, $\mathbf{dS} = \mathbf{P}\odot(\mathbf{dP} - \boldsymbol{\delta})$ with $\delta_i = \sum_j P_{ij} dP_{ij}$. Since $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top$, $\delta_i = \sum_j P_{ij}\, \boldsymbol{dO}_i \cdot \boldsymbol{v}_j = \boldsymbol{dO}_i \cdot \sum_j P_{ij}\boldsymbol{v}_j = \boldsymbol{dO}_i \cdot \boldsymbol{O}_i$.

Exercise 4.9. With 228 KB of SRAM the model's naive/tiled traffic ratio is about 6.5–10.6 for $D = 64$, 2.3–2.7 for $D = 128$, and below 1 for $D = 256$ (tiling "loses"). The model counts only the score matrix's write/read passes and ignores that the naive path also launches separate mask, softmax and dropout kernels (more T^2 passes), stores $\mathbf{A}$ for backward, and that real kernels choose tile shapes and use registers as well as shared memory. The qualitative lesson holds: the benefit shrinks as D^2/M grows.

Exercise 4.10. $8192/128 = 64$ tiles per side, 4096 tiles in total: 64 diagonal tiles (1.6%), $64 \cdot 63/2 = 2016$ fully visible (49.2%) and 2016 fully masked (49.2%). Skipping masked tiles avoids about 49% of the dense $\mathbf{QK}^\top$ work, close to the ideal 50%; diagonal tiles are computed in full and masked element-wise.

Exercise 4.11. Cache bytes $= 2L\, H_{kv} d_h \min(t, w) \cdot b$: they grow linearly until $t = w$ and then stay constant. At $t = 32$k the windowed cache is $32\text{k}/4096 = 8\times$ smaller. Implementations keep a rolling (circular) buffer that overwrites the oldest entries, so a cached shared prefix is destroyed as soon as any request sharing it advances more than w tokens past it. Prefix sharing would require copy-on-write of the overwritten blocks or keeping prefix blocks pinned, and a prefix longer than w is only partially useful.

Exercise 4.12. Expect the explicit implementation's peak memory to scale as BHT^2 with several live copies (scores, probabilities, possibly the mask), running out of memory when $\approx 2\text{–}3 \cdot BHT^2 \cdot b$ exceeds free memory; SDPA's peak scales linearly in T. The measured out-of-memory length should be within a factor of $\sqrt{2}$–$\sqrt{3}$ of the single-copy estimate.

Chapter 5: Implementing a GPT Model

Exercise 5.1. With Eq. (5.6) (untied head, no QKV bias, $V = 50{,}257$, $T_{\max} = 1024$): GPT-2 medium has 406,212,608 parameters (attention 100.7 M, FFN 201.4 M); large has 838,220,800 (attention 236.0 M, FFN 472.1 M); XL has 1,637,792,000 (attention 491.6 M, FFN 983.4 M). For $L = 12$ the FFN holds $12(8d^2 + 5d)$ parameters and the embeddings plus head hold $(2V + T_{\max})d$. Equality gives $96d + 60 = 2 \cdot 50{,}257 + 1024$, i.e. $d \approx 1057$; any width above that (in practice the next multiple of 64, $d = 1088$) puts more parameters in the FFNs than in the vocabulary-sized matrices.

Exercise 5.2. Add three keys, e.g. `drop_emb`, `drop_attn` and `drop_resid`, pass `drop_attn` to `MultiHeadAttention` and `drop_resid` to `drop_shortcut`, and use `drop_emb` in `GPTModel`. Removing the *attention-weight* dropout saves the most: it acts on the largest activation (BHT^2 elements per layer), its mask must otherwise be stored or regenerated, and many fused attention kernels have a faster code path when dropout is zero.

Exercise 5.3. The next multiple of 128 above 50,257 is 50,304: 47 extra rows, $47\times768 = 36{,}096$ parameters per vocabulary matrix (72,192 if the head is untied). Two fp32 Adam moments add 8 bytes per parameter: 289 kB (tied) or 578 kB (untied)—negligible. During generation either slice the logits to the true vocabulary (`logits[..., :50257]`) or set the padded entries to $-\infty$ before argmax/sampling, so a padded ID can never be emitted.

Exercise 5.4. From $8Nd^2 = r\,(10Nd + 8d^2)$ with ridge point r we get $N^\star = 8rd/(8d - 10r)$. For the A100, $r = 312/2.039 \approx 153$ FLOP/byte, giving $N^\star \approx 204$ tokens for $d = 768$ and ≈ 160 for $d = 4096$. The second FFN GEMM, $(N \times 4d) \cdot (4d \times d)$, also performs $8Nd^2$ FLOPs and moves $2(4Nd) + 2(4d^2) + 2(Nd) = 10Nd + 8d^2$ bytes: its intensity is identical, so (ignoring tiling effects) both GEMMs reach the ridge at the same $N^\star$.

Exercise 5.5. Without scaling, each of the $2L$ sublayers adds an update of roughly constant variance σ_u^2 to the stream, so after ℓ sublayers the variance is $\sigma_0^2 + \ell\sigma_u^2$ and the RMS grows like $\sqrt{\ell}$: a concave, square-root-shaped curve. Scaling the output projections by $1/\sqrt{2L}$ shrinks each update's variance by $2L$, so the total added variance after all $2L$ sublayers is about σ_u^2 and the curve becomes nearly flat.

Exercise 5.6. Post-LN computes `x = norm(x + f(x))`. Without warmup, the post-LN model typically diverges (loss spikes to NaN or plateaus at the unigram entropy) at a learning rate several times lower than the pre-LN model tolerates, because the gradients of the last layers are large at initialization (Xiong et al., 2020); adding a linear warmup closes much of the gap.

Exercise 5.7. (a) The explicit tanh-GELU saves, per element of the $4d$-wide hidden tensor, the input (for `pow` and the outer multiply), the `tanh` output, the half-scaled input and the $(1+\tanh)$ factor: about four $4d$-wide tensors. (b) `nn.GELU` saves only its input, removing roughly three of them (1,387 kB→994 kB on the tiny configuration). (c) The explicit LayerNorm saves $x - \mu$, the denominator and the normalized tensor; `nn.LayerNorm` saves the input plus one mean and one reciprocal standard deviation per row, so roughly one to two d-wide tensors disappear per norm.

Exercise 5.8. Causal attention computes, per head and sequence, $\sum_{i=1}^{T} i\, d_h \cdot 2 = T(T+1)d_h$ FLOPs for $QK^\top$ and the same for AV, i.e. $2T(T+1)d$ per layer instead of $4T^2d$—a factor $(T+1)/2T \approx \frac{1}{2}$. For GPT-2 small the total forward FLOPs per token drop from 284.8M to 265.9M at $T = 1024$ (6.6% less) and from 549.1M to 398.1M at $T = 8192$ (27.5% less). If one paper counts full attention and another counts only the causal half, their MFU numbers differ by this amount for the same hardware efficiency, increasingly so at long context.

Exercise 5.9. Neglecting attention terms, a forward pass over t tokens costs $\approx 2Pt$. Without a cache, generating n tokens costs $\sum_{t=p}^{p+n-1} 2Pt \approx 2P(pn + n^2/2)$; with a cache, $2P(p+n)$ (prefill

plus one token per step). The ratio is $(pn + n^2/2)/(p + n)$, which for $p = 100$ reaches 100 at $n^2/2 = 10^4$, i.e. $n \approx 141$. Including attention makes the uncached version slightly worse still.

Exercise 5.10. Loop over sequence chunks of 128 positions: compute the chunk's logits, its cross-entropy summed over tokens, and accumulate; wrap each chunk in `torch.utils.checkpoint` so the chunk logits are recomputed in backward rather than stored. Peak logits memory falls from $8 \cdot 1024 \cdot 50{,}257 \cdot 4\,\mathrm{B} = 1.65\,\mathrm{GB}$ to $8 \cdot 128 \cdot 50{,}257 \cdot 4\,\mathrm{B} = 206\,\mathrm{MB}$ (and the same for the softmax gradient). The price is recomputing the head GEMM in the backward pass, about $2NVd$ extra FLOPs, i.e. one third more work for the output layer.

Chapter 6: Modern Architectures: Qwen3 from Scratch

Exercise 6.1. If $\mu = 0$ then $\sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2 = \frac{1}{d}\sum_i x_i^2 = \mathrm{RMS}(\boldsymbol{x})^2$, so $(\boldsymbol{x} - \mu)/\sqrt{\sigma^2 + \epsilon} = \boldsymbol{x}/\sqrt{\mathrm{RMS}^2 + \epsilon}$ and the outputs coincide. In the explicit implementations the LayerNorm saves $x - \mu$, the denominator and the normalized tensor (plus the input), while RMSNorm saves the fp32 input, the reciprocal RMS (one value per row) and the normalized tensor; for a $(4, 256, 1024)$ fp32 input expect RMSNorm to save roughly one full $4 \cdot 256 \cdot 1024 \cdot 4\,\mathrm{B} = 4.2\,\mathrm{MB}$ tensor less.

Exercise 6.2. $8d^2 = 3dh$ gives $h = \frac{8}{3}d$. For $d = 4096$ parity is $h \approx 10923$; Llama 2-7B's $h = 11008$ is 0.8% larger. $11008 = 43 \times 256$: rounding to a multiple of 256 makes every GEMM tile full and lets h be split evenly across 2, 4 or 8 tensor-parallel ranks while keeping each shard a multiple of 32 or 64.

Exercise 6.3. Reorder the coordinates so that dimension i and $i + d_h/2$ are adjacent; in that basis RoPE at position m is the block-diagonal matrix $R_m = \mathrm{diag}(R(m\theta_0), \ldots, R(m\theta_{d_h/2-1}))$. Then $\langle R_m q, R_n k\rangle = q^\top R_m^\top R_n k = q^\top R_{n-m} k$ block by block, as in the 2-D case; permuting both vectors identically does not change a dot product. Shifting m and n by the same c leaves $n - m$ unchanged. Left padding by c tokens shifts every real token's position by c within that row, so all scores between real tokens are unchanged (pad keys are masked out).

Exercise 6.4. With `p = halves_to_interleaved_perm(D)`, the interleaved model's q satisfies $q_{\text{inter}} = q_{\text{halves}}[p]$ per head. Hence, for each head h, the rows `W[h*D:(h+1)*D]` of the rotate-half checkpoint equal the interleaved rows reindexed by the inverse permutation: `W_halves[h*D + j] = W_inter[h*D + inv(p)[j]]`; apply this to $\mathbf{W}_Q$ and $\mathbf{W}_K$ (and to the QK-norm gains if present). $\mathbf{W}_V$ is untouched: RoPE is not applied to values, and the attention scores—the only place the permutation acts—are invariant.

Exercise 6.5. Qwen3-8B stores 144 KiB per token in bf16, so 131,072 tokens need 18 GiB (≈ 19.3 GB); in fp8, 9.7 GB. Next to 16.4 GB of bf16 weights a single context needs 35.7 GB: it fits on an 80 GB GPU, and with a 10% reserve two such contexts fit ($\lfloor (72 - 16.4)/19.3 \rfloor = 2$). With 8-bit weights (8.2 GB) three fit with a bf16 cache and six with an fp8 cache.

Exercise 6.6. Per decode token and layer, attention costs $4THd_h$ FLOPs and reads $2TGd_hb$ bytes of cache (b bytes per element), so $I = 2H/(Gb)$. In bf16 ($b = 2$): $I = H/G = 2, 4, 8$ for 0.6B, 8B, 32B; in fp8 ($b = 1$): $4, 8, 16$. Materializing `repeat_interleave` writes and then reads an H-head copy, so the bytes per FLOP grow by at least H/G and the effective intensity falls back to about 1 (or below, counting the extra write).

Exercise 6.7. If the cos/sin tables are computed in fp32 and only then cast to bf16, each entry has a relative error of about 2^{-9}, so the angle error stays near 0.004 radians at every position. If, however, the angles $m\theta_i$ themselves are formed in bf16 (e.g. because the model, including its buffers, was cast before the tables were built), the fastest frequency has $\theta_0 = 1$ and the angle equals m; bf16's spacing at magnitude $m \in [2^e, 2^{e+1})$ is 2^{e-7}, so the rounding error ($\le 2^{e-8}$) exceeds 0.1 rad once $e \ge 5$, i.e. already at positions of a few dozen tokens.

Exercise 6.8. $\beta' = 10^6 \cdot 4^{128/126} \approx 4.09 \times 10^6$. The frequency ratio for pair i is $\theta'_i/\theta_i = s^{-2i/(d_h-2)}$, so the wavelength is multiplied by $s^{2i/(d_h-2)}$: unchanged for $i = 0$ (smallest change) and multiplied by exactly $s = 4$ for $i = d_h/2 - 1 = 63$ (largest).

Exercise 6.9. Capacity $= \lceil cNk/E \rceil = 1.25 \cdot 4096 \cdot 2/64 = 160$ slots per expert. Under uniform random routing an expert's load is approximately $\mathcal{N}(128, 11.2^2)$. The expected overflow $\mathbb{E}[(X - 160)^+] = \sigma[\varphi(z) - z(1 - \Phi(z))]$ with $z = 2.85$ is ≈ 0.007 slots per expert, i.e. $\approx 0.006\%$ dropped. With $c = 1.0$ ($z = 0$) it is $\sigma\varphi(0) \approx 4.5$ slots per expert, about 3.5% dropped; with $c = 2.0$ ($z \approx 11$) essentially none. Trained routers are far from uniform, so real drop rates at $c = 1$ are much higher, which is why dropless kernels became popular.

Exercise 6.10. Assume the 16,384 tokens are spread evenly, 2048 per GPU. Each token sends $k = 8$ copies of a $d = 2048$ bf16 vector, 7/8 of which leave the GPU: dispatch is $2048 \cdot 8 \cdot 2048 \cdot 2 \cdot \frac{7}{8} \approx 58.7$ MB per GPU, and the combine step returns the same, 117 MB per layer, or about 0.26 ms at 450 GB s^{-1}. Each GPU's experts process about 16,384 token-slots at $6dh = 9.4$ M FLOPs each, 155 GFLOP, i.e. ≈ 0.16 ms at peak on an H100. Communication is comparable to (or larger than) compute, so production systems overlap the all-to-all of one micro-batch with the expert GEMMs of another.

Exercise 6.11. Naive loading of the 1.7B model in default fp32 holds the randomly initialized fp32 model (6.9 GB) plus the bf16 state dict (3.4 GB); even in bf16 it holds two copies (6.9 GB). The meta-device path peaks at one copy plus the largest single tensor, the 151,936 × 2048 embedding (0.62 GB): about 4.1 GB.

Chapter 7: Pretraining on Unlabeled Data

Exercise 7.1 (Initial loss). The loss at one position is $-z_y + \log \sum_u e^{z_u}$. Since z_y has mean zero, $\mathbb{E}[\mathcal{L}] = \mathbb{E}[\log \sum_u e^{z_u}]$. For large V the sum concentrates around its mean, $V\,\mathbb{E}[e^z] = Ve^{\sigma^2/2}$ (log-normal mean), so $\mathbb{E}[\mathcal{L}] \approx \ln V + \sigma^2/2$. For GPT-2 small, the final LayerNorm makes each hidden coordinate roughly unit variance and the head weights have standard deviation 0.02, so each logit has variance $\approx 768 \cdot 0.02^2 = 0.31$. Prediction: $\ln 50{,}257 + 0.15 \approx 10.98$ nats.

Exercise 7.2 (Perplexity across tokenizers). Model A: total NLL $= 3.10 \times 10^6$ nats over 4.3×10^6 bytes $= 0.721$ nats/byte $= 1.04$ bits/byte. Model B is byte-level, so tokens = bytes and 0.95 nats/byte = 1.37 bits/byte. Model A is better, even though its per-token loss is three times larger.

Exercise 7.3 (Training vs. validation). Expect the validation minimum within the first few epochs; dropout delays it slightly and keeps the validation loss lower afterwards, because it limits how precisely the model can store the training windows. Samples near the validation

minimum are novel but story-like; late samples reproduce training passages verbatim, especially with greedy decoding.

Exercise 7.4 (Temperature limits). Let $z_1 > z_j$ for $j \neq 1$. Then $p_\tau(j)/p_\tau(1) = e^{(z_j - z_1)/\tau} \to 0$ as $\tau \to 0^+$, so $p_\tau \to e_1$. As $\tau \to \infty$, $z_j/\tau \to 0$ for all j and $p_\tau(j) \to 1/V$. With k tied maxima the limit is uniform over the tied set (the ratios between tied tokens are always 1), which differs from `argmax`, which deterministically returns the first index.

Exercise 7.5 (Do top-k and temperature commute?). Yes for top-k: dividing by $\tau > 0$ preserves the order of the logits, so the same k tokens survive, and the softmax over the survivors is the same either way. No for top-p: the kept set depends on the cumulative probabilities, which temperature changes. A high temperature flattens the distribution, so more tokens are needed to reach mass p. Applying temperature first (the common convention) lets temperature widen the nucleus.

Exercise 7.6 (Resume correctness). Required: model weights; optimizer state (moments, step counts); scheduler/step counter; CPU and CUDA RNG states (dropout); Python `random` and NumPy RNGs if used anywhere; the sampler's generator state and the position within the epoch (how many batches have been consumed); worker seeding (`worker_init_fn`/base seed) so augmentation inside workers replays; and the `GradScaler` state (current scale and growth tracker). Listing 7.5 omits the data-loader position and sampler RNG, the Python/NumPy RNGs and the `GradScaler`; autocast itself holds no state.

Exercise 7.7 (Checkpoint interval). Job MTBF $M = 50{,}000\,\text{h}/2048 = 24.4\,\text{h} \approx 87{,}900\,\text{s}$. Synchronous ($\delta = 90\,\text{s}$): $T = \sqrt{2 \cdot 90 \cdot 87{,}900} \approx 3{,}980\,\text{s}$ (≈ 66 min). Expected overhead $\approx \delta/T + T/(2M) \approx 2.3\% + 2.3\% = 4.5\%$. Asynchronous ($\delta = 6\,\text{s}$): $T \approx 1{,}030\,\text{s}$ (≈ 17 min), overhead $\approx 0.6\% + 0.6\% = 1.2\%$ (restart time excluded in both).

Exercise 7.8 (Accumulation is not always exact). (1) Padding: if each micro-batch's loss is the mean over its *non-padding* tokens and the micro-batches contain different numbers of real tokens, averaging the k means weights tokens unequally; the exact large-batch loss divides the summed NLL by the total token count. Fix: sum token losses and divide by the global token count. (2) Layers with batch statistics (BatchNorm) see smaller batches. Also: the last, partial micro-batch of an epoch, and dropout masks that differ because the RNG is consumed in a different pattern.

Exercise 7.9 (Activation memory, GPT-2 xl). Model states: $16 \times 1.558\text{B} = 24.9\,\text{GB}$; budget $0.9 \cdot 80 - 24.9 = 47.1\,\text{GB}$. (a) No FlashAttention: per sample $48 \cdot 1024 \cdot 1600 \cdot (34 + 5 \cdot 25 \cdot 1024/1600) = 8.97\,\text{GB} \Rightarrow b = 5$. (b) FlashAttention: $48 \cdot 1024 \cdot 1600 \cdot 34 = 2.67\,\text{GB} \Rightarrow b = 17$. (c) Full checkpointing: block inputs $2 \cdot 1024 \cdot 1600 \cdot 48 = 0.157\,\text{GB}$ plus one layer's full activations during recomputation ($0.187\,\text{GB}$) $\approx 0.344\,\text{GB}$ per sample $\Rightarrow b \approx 136$ (in practice temporary buffers and the logits tensor limit this much earlier).

Exercise 7.10 (Measure MFU). Typical ordering: fp32 lowest (no tensor cores; TF32 if enabled helps); bf16 autocast a large jump; SDPA a further gain (no $T \times T$ materialization, fewer memory-bound kernels); compile another 10–30% from fusing LayerNorm/GELU/residual chains and reducing launches; activation checkpointing reduces MFU by roughly a quarter

(recompute is not counted in MFU) but allows larger micro-batches, which can partly win it back.

Exercise 7.11 (Chinchilla planning). $C = 1000 \cdot 86{,}400 \cdot 0.4 \cdot 989 \times 10^{12} = 3.4 \times 10^{22}$ FLOPs, so $N_{\text{opt}} = \sqrt{C/120} \approx 16.9$B and $D_{\text{opt}} \approx 338$B tokens. Inference over a year: 1.2×10^{13} tokens $\times 2N$. Option A ($N = 16.9$B): 4.1×10^{23} inference FLOPs, total $\approx 4.4 \times 10^{23}$. Option B ($N = 8.4$B, $D = 1.35$T): training $6 \cdot 8.4\text{B} \cdot 1.35\text{T} = 6.8 \times 10^{22}$ ($2\times$ the budget), inference 2.0×10^{23}, total $\approx 2.7 \times 10^{23}$. Option B is about 40% cheaper in total; with the Chinchilla loss fit its loss is only slightly higher than option A's, which is the argument for over-training models that will be served heavily.

Exercise 7.12 (Chunked loss). Implement an `autograd.Function` whose forward takes the final hidden states $\mathbf{H}$ `(N, d)`, the head weight $\mathbf{W}$ `(V, d)` and targets. Loop over chunks of rows: compute $\mathbf{Z}_c = \mathbf{H}_c\mathbf{W}^\top$ in fp32, its log-sum-exp, the chunk loss, and immediately the gradient $\partial\mathcal{L}/\partial\mathbf{Z}_c = (\text{softmax}(\mathbf{Z}_c) - \text{onehot})/N$; accumulate $\partial\mathcal{L}/\partial\mathbf{H}_c = \partial\mathcal{L}/\partial\mathbf{Z}_c\,\mathbf{W}$ and $\partial\mathcal{L}/\partial\mathbf{W} \mathrel{+}= (\partial\mathcal{L}/\partial\mathbf{Z}_c)^\top\mathbf{H}_c$. Save only these gradients for backward (which just scales them by the incoming gradient). Peak memory drops from $N \cdot V$ to chunk $\cdot V$ floats; verify with `torch.autograd.gradcheck` on small sizes in float64.

Chapter 8: Distributed Training Systems

Exercise 8.1 (Ring all-reduce cost). Each of the $p-1$ reduce-scatter steps and $p-1$ all-gather steps sends one chunk of n/p bytes per rank, all links busy in parallel: $2(p-1)(\alpha + \frac{n}{p}\beta)$, which is Eq. (8.2). Recursive halving: in round $j = 1..\log_2 p$ each rank exchanges half of its current working set with a partner at distance $p/2^j$, i.e. $n/2, n/4, \ldots, n/p$ bytes, summing to $n(p-1)/p$ in $\log_2 p$ messages; recursive doubling for the all-gather mirrors it. Total $2\log_2 p\,\alpha + 2\frac{p-1}{p}n\beta$.

Exercise 8.2 (Two-level all-reduce). Reduce-scatter inside the node: $\frac{7}{8}\,n/B_1$; all-reduce of the $n/8$ shards across N nodes (each GPU uses its own NIC): $2\frac{N-1}{N}\frac{n}{8}/B_2$; all-gather inside the node: $\frac{7}{8}\,n/B_1$. For large N: $\approx 1.75\,n/B_1 + 0.25\,n/B_2$. A flat ring over $8N$ GPUs is limited by its inter-node hops: $\approx 2n/B_2$. With $B_1 = 450$, $B_2 = 50$ GB/s: $0.0089\,n$ vs $0.040\,n$ seconds per GB, about $4.5\times$ faster hierarchically (latency terms aside).

Exercise 8.3 (Bucket size). Require $\alpha \le 0.05\,n\beta$, i.e. $n \ge 20\alpha/\beta = 20 \cdot 20\,\mu\text{s} \cdot 50\,\text{GB/s} = 20\,\text{MB}$. Very large buckets fill late: the first all-reduce starts only after a large fraction of the backward pass, and the final (exposed) bucket is large, so less communication is hidden.

Exercise 8.4 (No-sync accumulation). Add a flag `self.sync=True` and a context manager that sets it to `False`; in `_on_grad_ready` return immediately when `not self.sync` (gradients simply keep accumulating in `.grad`); on the last micro-batch the flag is `True` and buckets launch as usual, carrying the accumulated sums. Without it, k-step accumulation performs k all-reduces; with it, one—so communication time drops by $\approx k\times$.

Exercise 8.5 (ZeRO arithmetic). Table 8.1 follows from $\Psi = 7.5$B, $N_d = 64$, $K = 12$. For 13B on 16 GPUs: DP 208 GB (no); ZeRO-1 $(4 + 12/16) \cdot 13 = 61.8$ GB (18 GB left, no); ZeRO-2 $(2 + 14/16) \cdot 13 = 37.4$ GB (42.6 GB left, yes). ZeRO-2 is the lowest stage that works.

Exercise 8.6 (TP backward). $\mathbf{Y}_r = \mathbf{X}\mathbf{W}_r^\top$ for every r, so $\mathcal{L}$ depends on $\mathbf{X}$ through all $\mathbf{Y}_r$ and $\partial\mathcal{L}/\partial\mathbf{X} = \sum_r (\partial\mathcal{L}/\partial\mathbf{Y}_r)\mathbf{W}_r$. Rank r only knows its own term, hence the all-reduce in the backward of f. For the row-parallel layer $\mathbf{Y} = \sum_r \mathbf{X}_r\mathbf{W}_r^\top$, so $\partial\mathcal{L}/\partial\mathbf{X}_r = (\partial\mathcal{L}/\partial\mathbf{Y})\mathbf{W}_r$: the output gradient is replicated after g's (identity) backward, and each rank needs only its own $\mathbf{W}_r$—no communication.

Exercise 8.7 (Attention under TP). Give each rank H/t query heads with their Q/K/V columns (column-parallel) and the matching $H/t \cdot d_h$ input columns of the output projection (row-parallel); softmax and the causal mask act per head, so they stay local. Constraint: t divides H. With GQA, each rank also needs the KV heads its query heads use; if t exceeds the number of KV groups G, KV heads must be replicated on t/G ranks (and their gradients all-reduced among those replicas), which wastes memory and compute and caps useful TP at G for KV projections.

Exercise 8.8 (Bubbles). For $p = 8$ both schedules give $7/(m+7)$: 46.7%, 30.4%, 17.9%, 9.9% for $m = 8, 16, 32, 64$. Peak in-flight micro-batches: GPipe m on every stage; 1F1B $\min(8-s, m)$, i.e. at most 8. With $v = 2$ interleaving the bubble becomes $3.5/(m+3.5)$ (e.g. 9.9% at $m = 32$).

Exercise 8.9 (Causal load balance). With contiguous chunks, device i's queries attend to key blocks $0..i$: $i+1$ block pairs (the diagonal one half-masked), so device $c-1$ does c times the work of device 0. Split the sequence into $2c$ chunks and give device i chunks i and $2c-1-i$: it computes $(i+1) + (2c-i) = 2c+1$ block pairs, the same for every device.

Exercise 8.10 (Plan a 405B run). One defensible plan (close to the layout Meta reported for Llama 3 405B at 8k context): TP = 8 (one node), PP = 16 (≈ 8 layers per stage, first/last stage lighter), CP = 1, DP = 128 with ZeRO-1. Parameters per GPU $405\text{B}/128 = 3.2\text{B}$; ZeRO-1 states $(4 + 12/128) \cdot 3.2 \approx 13\,\text{GB}$. Global batch 16M tokens = 2,048 sequences, i.e. 16 micro-batches per pipeline. Plain 1F1B would idle $15/31 \approx 48\%$, so use interleaving with $v = 4$ virtual stages: $3.75/(16 + 3.75) \approx 19\%$. Activations with FlashAttention and TP+SP: $8192 \cdot 16{,}384 \cdot 34/8 \approx 0.57\,\text{GB}$ per layer per micro-batch; stage 0 can hold up to 16 micro-batches $\times$ 8 layers, so selective or full recomputation is needed (full recompute: $\approx 4\,\text{GB}$). DP traffic: each rank reduces a 6.4 GB bf16 gradient shard over 128 replicas, $\approx 0.26\,\text{s}$ at 50 GB/s, well hidden behind $\approx 6\,\text{s}$ of compute per step at 40% MFU. The remaining inefficiency is the pipeline bubble, which is why larger batches or zero-bubble schedules are attractive at this scale.

Chapter 9: Fine-Tuning for Classification

Exercise 9.1. With global padding every batch is 8×120; with dynamic padding the mean batch width falls to roughly the batch's longest message (typically 30–60 tokens for SMS), so epoch time drops roughly in proportion once the GPU is compute-bound. On CPU, or with tiny batches, the gain is smaller because fixed overheads dominate. Accuracy should be statistically indistinguishable *if* the head reads the last real token. Padding to 1,024 multiplies linear-layer work by about $1024/120 \approx 8.5$ and attention-score work by about 73. If the head reads position -1, it now reads a padding token that sits 900+ positions after the message. Its hidden state is still a function of the message, but a very different input distribution from the one seen in training, and accuracy typically drops sharply.

Exercise 9.2. Per-step cost is predicted by $2N + 4N_{\text{tr}}$ per token: with k trainable blocks out of 12, $N_{\text{tr}} \approx (k/12)N$, so time per step $\propto 2 + 4k/12$. That is 1.0 ($k = 0$, head only, up to a tiny head), 1.17 ($k = 1$), 1.33 ($k = 2$), 1.67 ($k = 4$) and 3.0 ($k = 12$) relative to a forward pass. Peak memory grows with k through activations kept for backward (only blocks above the lowest trainable one store them) and through 14 bytes per trainable parameter. Accuracy usually improves from $k = 0$ to $k = 1$ and changes little after that. Discrepancies come from fixed overheads (optimizer step, kernel launches) and from the embedding/head layers, which the block-count model ignores.

Exercise 9.3. Run blocks 1–11 under `torch.no_grad()` once, storing h_{11} of shape $(n, T, 768)$ in bf16 (about 193 MB for 1,045 messages padded to 120). Train block 12, the final norm and the head on the cache. With dropout off, the forward computation is identical, so the loss curves match to floating-point tolerance. Measuring cost in units of one block's forward FLOPs (2 per parameter), an uncached step costs $12 \times 2 + 4 = 28$ (twelve forwards, one backward of block 12), a cached step $2 + 4 = 6$, so each epoch is about $28/6 \approx 4.7\times$ faster (less if data movement of the cache dominates).

Exercise 9.4. Plain cross-entropy on 87/13 data reaches high accuracy but lower spam recall. Class weights $w_c \propto 1/\text{freq}(c)$ raise recall at some cost in precision. Undersampling behaves similarly but trains on less data. Accuracy is misleading under imbalance; report precision/recall/F1 for the spam class, and for a dashboard prefer precision at a fixed recall (or vice versa), matched to the product cost of each error.

Exercise 9.5. Let a message occupy positions $0..\ell - 1$ with padding at $\ell..T - 1$. Under the causal mask, the hidden state at position t depends only on tokens $0..t$. In particular position $\ell - 1$ never attends to padding, and LayerNorm/MLP act per position, so $h_{\ell-1}$, and hence the logits read there, are independent of $T - \ell$. Counterexample for position -1: with $T = \ell + 1$, the last position is a padding token attending to $\ell + 1$ keys. With $T = \ell + 5$ it attends to $\ell + 5$ keys, four of which are padding tokens with different position embeddings, so the softmax weights and the output differ.

Exercise 9.6. With left padding the real tokens occupy positions $T - \ell..T - 1$ instead of $0..\ell - 1$, so they receive different learned position embeddings, and the logits differ (often by a few percent of their magnitude, enough to flip borderline cases). The key mask removes the influence of padded tokens but cannot undo the position shift. With RoPE, attention depends only on $i - j$, which is unchanged for every pair of real tokens, so the logits match up to floating-point error.

Exercise 9.7. Sort-within-mega-batch as in Listing 9.3, then greedily add sequences to the current batch while $\max(\text{len}) \times (\text{count}) \leq K$. Short-message batches then contain many sequences and long ones few. The number of batches varies, padding waste matches bucketing, and step-time variance drops because every batch has roughly K token-slots. Learning-rate schedules should then be defined in tokens rather than steps.

Exercise 9.8. Compute $\log p(\text{“ spam”} \mid \text{prompt})$ and $\log p(\text{“ ham”} \mid \text{prompt})$ from one forward pass each (or one pass with both continuations scored via `sequence_logprob`) and predict the larger. Base GPT-2 models typically score only modestly above chance on the balanced split,

and the improvement with model size is gradual. Fine-tuning closes most of the gap. Note that the label words should be chosen as single tokens with a leading space, otherwise multi-token labels are penalised by length.

Exercise 9.9. Demand: $20{,}000 \times 35 = 7.0 \times 10^5$ real tokens/s. With random batching at about 60% waste, the GPUs must process $7.0 \times 10^5 / 0.4 = 1.75 \times 10^6$ token-slots/s; with bucketing at about 7% waste, 7.5×10^5. An L4 offers 121 TFLOP/s; at 40% utilisation and 1.7×10^8 FLOPs/token it processes about 2.85×10^5 tokens/s. So random batching needs $\lceil 6.1 \rceil = 7$ GPUs and bucketing needs $\lceil 2.6 \rceil = 3$, plus headroom for peaks. A 160-token message in a batch of 64 takes about $64 \times 160 \times 1.7 \times 10^8 / 4.8 \times 10^{13} \approx 36$ ms of compute, so the 50 ms p99 constrains the batching window to a few milliseconds. The weakest assumption is the 40% utilisation: at such short sequences, kernel-launch and framework overheads can halve it unless CUDA graphs or a compiled engine are used.

Exercise 9.10. The SMS collection contains several hundred exact duplicate texts, mostly spam templates, and more after normalisation. Re-splitting after deduplication typically lowers test accuracy slightly. The difference measures how much the original score benefited from test messages whose duplicates were in training.

Exercise 9.11. Temperature scaling multiplies all logits of an example by the same positive constant $1/T$. Because $\arg\max_c z_c/T = \arg\max_c z_c$ for $T > 0$, predictions and therefore accuracy are unchanged. A fine-tuned classifier is typically over-confident ($T > 1$ after fitting), and ECE drops substantially after scaling.

Exercise 9.12. Sort test examples by calibrated confidence. Escalating the least confident fraction e makes cascade accuracy $= 1 -$ (errors among the confident $1 - e)/n$. Increase e until accuracy reaches 99.5%; the cost is then $(1 + eR)/R$ of the large-only budget. Well-calibrated models concentrate their errors in the low-confidence region, so a small e suffices.

Exercise 9.13. For pretrained GPT-2 small, probe accuracy typically rises over the first several blocks and plateaus in the middle-to-upper layers. Sometimes it dips slightly in the last block, which is specialised for next-token prediction. If the plateau is reached by block ℓ, serving only blocks $1..\ell$ plus a head saves about $(12 - \ell)/12$ of the per-token FLOPs (embeddings and head aside).

Chapter 10: Instruction Fine-Tuning and Parameter-Efficient Adaptation

Exercise 10.1. Replace `format_alpaca` by `format_phi3` in both the dataset and the generation prompt, and extract responses by splitting on `<|assistant|>` instead of the Alpaca header. The Phi-3 template removes roughly 25–30 tokens per example, so total training tokens drop by roughly a quarter to a third and epoch time falls accordingly. Judge scores are usually similar. Note that `<|user|>` is not a special token for GPT-2 and is split into several BPE pieces, which is harmless but slightly less compact than the name suggests.

Exercise 10.2. Losses averaged over response tokens only are not comparable with losses averaged over all tokens, because prompt tokens (especially the fixed preamble) are easy to predict and pull the full-sequence average down. Evaluate both models on the same response-only target set. Response-only training usually matches or slightly beats full-sequence training

on judge score. Full-sequence loss can win when responses are very short and prompts carry useful in-domain text, because it then provides much more gradient signal per example.

Exercise 10.3. Two microbatches with $n_1 = 10$ and $n_2 = 90$ target tokens and mean losses $\bar{\ell}_1, \bar{\ell}_2$: averaging the means gives $\frac{1}{2}(\bar{\ell}_1 + \bar{\ell}_2)$, so each token of microbatch 1 has weight 1/20 and each token of microbatch 2 has weight 1/180. The per-token mean of the full step gives every token weight 1/100. Fix: compute `F.cross_entropy(..., reduction="sum")` per microbatch, count non-ignored targets, and divide the accumulated sum by the total count before `backward` (or scale each microbatch's sum by $1/n_{\text{total}}$, which requires knowing n_{total} in advance, for example by pre-counting the step's targets).

Exercise 10.4. Random batches with dynamic padding typically waste 30–50% on instruction data with a long-tailed length distribution; bucketing reduces this to a few percent; FFD packing into 512- or 1,024-token rows leaves only the tail of the last rows empty, usually below 2%. For attention, a dense row mask costs T^2 per row regardless of content. A varlen kernel costs $\sum_i \ell_i^2$, which for documents of about 100 tokens packed into $T = 1024$ is about 10 documents $\times\, 100^2 = 10^5$ versus $1024^2 \approx 10^6$, a 10× reduction in attention FLOPs.

Exercise 10.5. Probe: fix document 2, vary document 1 in the same packed row, and compare the logits at a position in document 2. With naive packing they change. With the block-diagonal mask and position reset they are identical up to floating-point error. After fine-tuning on naively packed data the model learns to use the preceding documents, for example by copying formatting or answers from the previous example. The effect is visible as degraded single-example performance at inference time, where no preceding documents exist.

Exercise 10.6. Reference answers should score near the top of the scale and wrong answers near the bottom; deviations reveal judge bias or prompt problems. Correlations between two different judges are typically high (Pearson 0.7–0.9) on clearly separable answers but lower among mediocre ones, and mean absolute differences of 5–15 points are common. A single reported score therefore has an uncertainty of several points from the judge alone, on top of sampling uncertainty from 110 test items (standard error $\approx \sigma/\sqrt{110}$).

Exercise 10.7. With $\alpha = 2r$, the effective update scale $\alpha/r = 2$ is constant, so the magnitude of ΔW produced by a given learning rate is roughly independent of r, and one learning rate works across ranks. With fixed $\alpha = 16$, α/r shrinks as r grows (16 at $r = 1$, 0.25 at $r = 64$), so large ranks effectively train with a smaller step and appear worse unless the learning rate is re-tuned. Quality usually saturates by $r = 8$–16 for a task of this size.

Exercise 10.8. For a fixed parameter budget, adapting all linear layers (with a smaller r) usually wins on SFT, because MLP layers hold much of the task-relevant capacity. The spam task is easy enough that most placements reach similar accuracy. Attention-only placements are slightly weaker on SFT.

Exercise 10.9. In full fine-tuning the weight-gradient GEMMs are about a third of matmul time. With LoRA they disappear for frozen weights, while the adapter GEMMs are tiny but numerous (two extra small matmuls per adapted layer in each direction). On small models these

extra kernels and the optimizer step can eat most of the theoretical 1.5× saving, so measured speed-ups are often smaller than predicted. On large models the prediction is approached.

Exercise 10.10. Block-normalised GPT-2 weights are roughly bell-shaped but heavier-tailed than a Gaussian, with occasional outlier channels. NF4 gives lower reconstruction error than uniform int4 for the bulk of the weights, because its levels are denser near zero, and typically a smaller perplexity increase. Uniform int4 does relatively better on blocks dominated by an outlier, where the absmax scale squeezes the remaining weights into a few levels for both schemes.

Exercise 10.11. With one adapter, the gather is a broadcast and the cost is close to the base model plus two small matmuls. With 8 or 64 distinct adapters, the gather materialises per-row copies of $\mathbf{A}$ and $\mathbf{B}$ ($B \times r \times d$ elements), which adds memory traffic proportional to the batch size. That is where throughput drops. Fused kernels (Punica's BGMV/SGMV) read each row's adapter directly from the adapter pool inside the matmul, never materialising the gathered copies, and group rows that share an adapter (segmented GEMM) to recover tensor-core efficiency.

Chapter 11: Inference Fundamentals

Exercise 11.1 (Unknown words). Byte-level BPE starts from the 256 single-byte tokens, so every UTF-8 string is representable; unseen words simply decompose into more, shorter pieces, down to single bytes in the worst case. Invented English-like words typically cost 2–4 tokens instead of 1; scripts that are rare in the training data (e.g. many Indic or African scripts) can cost close to one token per UTF-8 byte, i.e. 2–3 tokens per character. Since KV memory is per token, the cost per character is (tokens/char)×112 KiB for Qwen3-0.6B: a script tokenized at 2 tokens/character uses roughly 8× more KV memory and decode steps per character than English at ≈0.25 tokens/character.

Exercise 11.2 (Accelerators). Expect the KV-cached, compiled decoder to gain most: on a GPU its per-step work is tiny, so eliminating Python and launch overhead (and enabling CUDA graphs with `reduce-overhead`) exposes the bandwidth bound. The cache-less decoder is dominated by large GEMMs that the GPU accelerates well in absolute terms, but it remains orders of magnitude slower for long outputs because its work is quadratic. On Apple MPS, compilation support is less mature, so gains from `torch.compile` are smaller.

Exercise 11.3 (Counting redundant work). Dense token-positions: without cache $\sum_{t=0}^{n-1}(P+t) = nP + n(n-1)/2$; with cache $P+n$ (one prefill plus one single-token call per generated token; the last call is avoidable, giving $P+n-1$). Query–key products per head and layer: without cache each step runs full causal attention over $L = P+t$ tokens, $\sum_{L=P}^{P+n-1} L(L+1)/2$; with cache, $P(P+1)/2$ for prefill plus $\sum_{t=1}^{n-1}(P+t)$. For $P = 1000, n = 100$: dense ratio 104,950/1,100 ≈ 95, attention ratio ≈ 91. For $P = 100, n = 1000$: dense ratio 599,500/1,100 = 545, attention ratio ≈ 367. Long *outputs* benefit most, because the waste grows with n^2.

Exercise 11.4 (KV budget planning). (a) $(80 - 16.4 - 4)$ GB/147,456 B ≈ 404,000 tokens. (b) 404,000/8,192 ≈ 49 conversations. (c) An fp8 KV cache halves bytes per token: ≈ 99 conversations. Int4 weights shrink the model to ≈ 4.1 GB, leaving 71.9 GB: ≈ 59 conversations

with a bf16 cache and ≈ 119 with fp8. Both techniques compound; for long contexts the KV format matters more than the weight format.

Exercise 11.5 (Critical batch size). Weight GEMMs have intensity $2B/b$; setting it equal to the ridge Π/β gives $B^* = (\Pi/\beta)\, b/2$. bf16 ($b = 2$): A100 ≈ 153, H100 ≈ 295, RTX 4090 ≈ 164. Int8 weights ($b = 1$): ≈ 77, ≈ 148, ≈ 82. Fewer bytes per weight means each byte read feeds more FLOPs, so the step leaves the memory-bound regime at a smaller batch; equivalently, at batch 1 weight-only quantization makes decoding up to 2× (int8) or 4× (int4) faster, which is why it is so popular for local inference.

Exercise 11.6 (The LM head at prefill). Full logits: $2 \cdot 1024 \cdot 151{,}936 \cdot 4096 \approx 1.27$ TFLOP and a $4096 \times 151{,}936$ bf16 tensor of 1.24 GB (twice that if upcast to fp32). Last row only: 311 MFLOP and 0.3 MB. Implementation: in `forward`, apply `final_norm` and `out_head` to `x[:, -1:]` when `last_only`. The memory saving often determines the maximum prefill chunk size on small GPUs.

Exercise 11.7 (Cache copies). At decode step t, `torch.cat` reads and writes the existing $(P+t)$-token cache of every layer: $\approx 2\kappa(P+t)$ bytes, so the total over n steps is $\approx 2\kappa(nP+n^2/2)$, quadratic in n. `StaticKVCache` writes only the new token: κ bytes per step. The copy traffic exceeds the 1.19 GB weight traffic of a step when $2\kappa(P + t) > 1.19$ GB, i.e. at about 5,200 tokens (10,400 counting only one direction).

Exercise 11.8 (Recompilation hunting). With the growing cache the key length changes every step. Dynamo specializes on the first shape, recompiles on the second, and then (automatic dynamic shapes) marks the dimension dynamic, so one sees one or two recompiles followed by a dynamic-shape graph with less aggressive optimization. The static cache avoids reallocation, but our implementation returns a growing view, so the attention shapes still vary; only a variant that attends over the full buffer with a length mask, with the write position held in a device tensor, gives truly static shapes and zero recompiles after warm-up (and is CUDA-graphable).

Exercise 11.9 (Little's law). E2E latency $= 0.3 + 299 \times 0.025 \approx 7.8$ s. With at most 128 concurrent requests, $\lambda_{\max} = 128/7.8 \approx 16.5$ requests/s. Arriving 20% faster (19.8 requests/s) grows the queue at ≈ 3.3 requests/s, i.e. about 2,000 queued requests after ten minutes; the last arrivals wait $\approx 2000/16.5 \approx 120$ s before their prefill even starts. Overload is not graceful: queueing delay grows linearly with time until load drops or requests are shed.

Exercise 11.10 (Determinism). Typically the outputs eventually diverge. GEMM libraries choose different kernels and split-K strategies for different batch sizes, and attention kernels use different tilings; floating-point addition is not associative, so logits differ in the last bits, and as soon as two candidates are nearly tied, the argmax flips and the continuations diverge. For evaluation, fix batch size and kernels (or use batch-invariant kernels) when comparing runs. For RL, the same effect makes the sampling engine's log-probabilities differ from the trainer's, which motivates importance-sampling corrections (Chapter 19).

Chapter 12: Batching and Serving Engines

Exercise 12.1 (Per-row positions). Compute p_b = number of left pads of row b and pass `position_ids = clamp(arange(T) - p_b, min=0)` for the prefill; during decoding, each row's

next position is its previous position plus one. Replace `self.pos_emb(arange(start, start+T))` by `self.pos_emb(position_ids)`. With the fix, greedy outputs match single-sequence generation exactly (up to floating-point reordering). Without it, rows with padding see shifted learned position vectors; the logits change noticeably and greedy continuations often diverge after a few tokens.

Exercise 12.2 (Dropping finished rows). After each step compute `keep = ~done`; index the next-token tensor, the validity mask and, for every layer, the cache tensors `k[layer] = k[layer][keep]` (same for `v`). Keep a map from active row to original request. With geometric output lengths (mean 200, cap 2,000) a static batch runs to near the cap while the mean row needs 200 steps, so dropping rows saves most decode work; the saving in wall time is smaller on a GPU at small batch sizes (where step time barely depends on B) and larger on CPUs or at large B.

Exercise 12.3 (Padding efficiency). For i.i.d. $U(a, b)$ lengths, $\mathbb{E}[\max] = a+(b-a)\,B/(B+1)$, so $\eta = \frac{(a+b)/2}{a+(b-a)B/(B+1)}$. With $a = 50, b = 1500$: $\eta = 1$ for $B = 1$, 0.58 for $B = 8$, 0.52 for $B = 64$. Continuous batching has no idle decode slots ($\eta \approx 1$); its only waste is the scheduler's inability to fill the batch when few requests are present.

Exercise 12.4 (Block size). Throughput is nearly flat for block sizes 4–64 in the simulator (which charges no indirection cost), while internal fragmentation grows roughly as bs/2 tokens per sequence; at 256 the waste becomes visible for short requests and reduces the effective batch under memory pressure. On a GPU, block size 1 turns every key read into a separate gather through the block table: poor memory coalescing, a table as large as the sequence, and many more address computations; blocks of 16–32 tokens let a warp stream contiguous chunks of K/V and keep tables small.

Exercise 12.5 (Copy-on-write fork). After the parent's prefill, each child copies the parent's block table and increments every block's reference count. Before a child appends to its last block, check its reference count; if > 1, allocate a fresh block, copy the existing slots (K and V of all layers), decrement the shared block and replace the table entry. Give each child its own `torch.Generator`. For $n = 16$ children of a P-token prompt, memory drops from $16(P + L)$ to $P + 16L$ tokens (plus at most 16 partially copied blocks), and prefill compute drops by $16\times$.

Exercise 12.6 (Swap vs. recompute). Swap: $2\kappa/\beta = 2 \times 147{,}456\,\text{B}/50\,\text{GB/s} \approx 5.9\,\mu\text{s}$ per token. Recompute: $2N/\Pi \approx 16.6\,\mu\text{s}$ per token at peak plus attention, which grows with sequence length. For Qwen3-8B, swapping is cheaper at every length (ignoring fixed per-transfer latency), and the gap widens with length. With MHA (32 KV heads, $\kappa = 576\,\text{KiB}$) swapping costs 23.6 µs/token; recompute is cheaper until its attention term, $4LHd \cdot S/2$ FLOPs per token on average, adds the missing 7 µs, at $S \approx 23{,}600$ tokens. An fp8 KV cache halves swap cost (2.9 µs/token) and makes swapping even more attractive. Real decisions also weigh PCIe contention, host memory capacity and prefix-cache hits that make recomputation partly free.

Exercise 12.7 (Budget trade-off). Small budgets keep every step short (low TPOT) but admit prefill slowly: TTFT explodes and throughput drops because steps are dominated by fixed overheads. Large budgets finish prefills quickly but make steps long, so TPOT rises; very large budgets can also hurt TTFT because a few long prefills monopolize steps. The curve

is a Pareto front with a knee near the budget at which a step becomes compute-bound (a few hundred to a few thousand tokens on an H100 for an 8B model). With our default trace (`bench.py -exp budget`), budgets of 256–1024 give the best TTFT/TPOT compromise.

Exercise 12.8 (Multi-turn chat). At turn k the prompt is the whole conversation, whose length grows linearly in k, while only the newest user message (and the previous reply, if it was not cached during generation) is new. Without caching the cumulative prefill over K turns is quadratic in K; with caching it is linear, and the hit rate approaches $1 - (\text{new tokens})/(\text{conversation length}) \to 1$. Caching generated blocks too (our engine publishes every full block, including output blocks) makes even the previous reply a hit.

Exercise 12.9 (Disaggregation sizing). Prefill of 3,000 tokens: $3000 \times 15.1\,\text{GFLOP}/(0.6 \times 989\,\text{TFLOP/s}) \approx 76\,\text{ms}$, so one prefill GPU serves ≈ 13 requests/s. Decode: the KV pool of $\approx 52\,\text{GB}$ holds ≈ 111 sequences of $\approx 3{,}150$ tokens; a step then reads $16.4 + 111 \times 3150 \times 147\,\text{KB} \approx 68\,\text{GB}$: $\approx 20\,\text{ms}$, under the 30 ms target. Each decode GPU completes $111/(300 \times 0.020) \approx 18$ requests/s, so roughly 1.4 prefill GPUs per decode GPU. For the reasoning workload (prompt 300, output 8,000), prefill takes $\approx 8\,\text{ms}$ (130 requests/s per GPU) while a decode GPU holds only ≈ 42 sequences and completes ≈ 0.4 requests/s: about 300 decode GPUs per prefill GPU. Disaggregation buys little there; the system is entirely decode-bound and KV memory is the lever.

Exercise 12.10 (Fairness). Keep `served[tenant]` (input tokens weighted plus output tokens) and, when admitting, pick the waiting request whose tenant has the smallest counter (lifting a newly active tenant's counter to the minimum of the active ones, so idle time is not banked). In the simulator, a heavy tenant at 10× the light tenant's rate then fills only the capacity the light tenant leaves unused; the light tenant's TTFT stays close to its value in isolation instead of growing with the heavy tenant's queue.

Chapter 13: Making Inference Cheaper and Deploying It

Exercise 13.1. Qwen3-0.6B stores $2 \cdot 28 \cdot 8 \cdot 128 \cdot 2 = 114{,}688$ bytes of KV per token, i.e. 235 MB per 2048-token sequence. Its weights take 1.19 GB, so the two are equal at $B \approx 1.19 \cdot 10^9 / 2.35 \cdot 10^8 \approx 5$. Beyond a handful of concurrent sequences the KV cache dominates the bytes per step. Weight-only quantization of a small model therefore barely helps a busy server; KV quantization, paging and prefix sharing matter far more.

Exercise 13.2. With e uniform on $[-s/2, s/2]$, $\mathbb{E}[e^2] = \frac{1}{s}\int_{-s/2}^{s/2} e^2\,de = s^2/12$. A full-range signal uniform on $[-A, A]$ has power $A^2/3$, and b bits give $s = 2A/2^b$, so the noise power is $A^2/(3 \cdot 4^b)$. Hence $\text{SQNR} = 4^b$, or $10\log_{10} 4^b \approx 6.02\,b\,\text{dB}$.

Exercise 13.3. Per row: $s = 40/7 \approx 5.71$, and a value rounds to zero when $|x| < s/2 \approx 2.86$, which holds for $\approx 99.6\%$ of standard normal draws. The row is destroyed. With $g = 32$, only the outlier's group (31 normal values) suffers this. In the other groups the scale is set by the group's own maximum (≈ 2.3), so only $|x| \lesssim 0.17$ rounds to zero. A simulation gives $\approx 34\%$ zeros overall, almost all of them in the outlier's group.

Exercise 13.4. (a) $4 + (16 + 16)/64 = 4.5$ bits. (b) $4 + 8/32 = 4.25$ bits. (c) $4 + 8/16 = 4.5$ bits, plus a negligible per-tensor fp32 scale.

Exercise 13.5. For one row, the loss is $\ell(\boldsymbol{w}') = (\boldsymbol{w}' - \boldsymbol{w})^\top H(\boldsymbol{w}' - \boldsymbol{w})$. Minimise ℓ subject to $\boldsymbol{e}_i^\top(\boldsymbol{w}' - \boldsymbol{w}) = q_i - w_i$. The Lagrangian gives $2H\boldsymbol{\delta} = \lambda \boldsymbol{e}_i$, so $\boldsymbol{\delta} = \frac{\lambda}{2} H^{-1}\boldsymbol{e}_i$, and the constraint fixes $\lambda/2 = (q_i - w_i)/[H^{-1}]_{ii}$. Therefore $\boldsymbol{\delta} = -\frac{w_i - q_i}{[H^{-1}]_{ii}} H^{-1}_{:,i}$, whose i-th entry is exactly $q_i - w_i$ and whose other entries are the stated update. Substituting back, $\ell = \boldsymbol{\delta}^\top H \boldsymbol{\delta} = (w_i - q_i)^2/[H^{-1}]_{ii}$. GPTQ then removes row and column i from H^{-1} (a Schur-complement update), which is what the Cholesky formulation does implicitly.

Exercise 13.6. $(X\,\mathrm{diag}(\boldsymbol{s})^{-1})(W\,\mathrm{diag}(\boldsymbol{s}))^\top = X\,\mathrm{diag}(\boldsymbol{s})^{-1}\mathrm{diag}(\boldsymbol{s})W^\top = XW^\top$. With $\alpha = \frac{1}{2}$, $s_j = \sqrt{a_j/w_j}$ where $a_j = \max|X_{:,j}|$ and $w_j = \max|W_{:,j}|$. Then $\max|X'_{:,j}| = a_j/s_j = \sqrt{a_j w_j} = w_j s_j = \max|W'_{:,j}|$. To fold the division, note that X is the output of RMSNorm with per-channel gain γ_j; replace γ_j by γ_j/s_j offline. (When several linears consume the same normalised input, e.g. Q, K and V, they must share $\boldsymbol{s}$.)

Exercise 13.7. Let x_1 be emitted at position 1 with probability $p(x_1)$, by the proposition. Conditioned on the first draft having been accepted with value x_1, the second draft is sampled from $q(\cdot \mid x_1)$ and verified against $p(\cdot \mid x_1)$, because the target's second output row is computed with x_1 in its context. Applying the proposition to this conditional pair gives $\Pr[x_2 \mid x_1] = p(x_2 \mid x_1)$. If the first draft is rejected, the round ends and position 2 is produced in the next round, again conditioned on the emitted x_1. Either way the joint law is $p(x_1)p(x_2 \mid x_1)$. Induction extends this to whole sequences.

Exercise 13.8. Maximising $S(\gamma) = (1 - \alpha^{\gamma+1})/((1-\alpha)(0.05\gamma + 1))$ over integers gives $\gamma^* = 4$ ($S \approx 1.92$) for $\alpha = 0.6$, $\gamma^* = 8$ ($S \approx 3.09$) for $\alpha = 0.8$, and $\gamma^* = 13$ ($S \approx 4.67$) for $\alpha = 0.9$. Going from γ to $\gamma + 1$ helps iff $S(\gamma+1) > S(\gamma)$, i.e. $\alpha^{\gamma+1}(1-\alpha)(c\gamma + 1) > c\,(1 - \alpha^{\gamma+1})$: the expected marginal token $\alpha^{\gamma+1}$ must outweigh the marginal drafting cost relative to the current expected yield.

Exercise 13.9. The step reads 16.4 GB of weights plus $128 \cdot 2048 \cdot 147.5\,\mathrm{KB} \approx 38.7\,\mathrm{GB}$ of KV: $t_{\mathrm{mem}} \approx 16.4\,\mathrm{ms}$. With $\gamma + 1 = 5$, the step processes 640 tokens: $\approx 2 \cdot 8.19 \cdot 10^9 \cdot 640 \approx 1.05 \cdot 10^{13}$ matmul FLOPs, plus $\approx 0.8 \cdot 10^{12}$ for attention, so $t_{\mathrm{comp}} \approx 11.4\,\mathrm{ms}$ at peak. That is still memory-bound on paper, but at a realistic 50–60% of peak the compute time reaches $\approx 20\,\mathrm{ms}$, which exceeds the memory time. Every rejected draft then costs real time. Choose $\gamma \leq 2$ (or adapt γ to load), and turn speculation off entirely at higher batch sizes.

Exercise 13.10. The message size is $m = 5120 \cdot 2 = 10\,\mathrm{KB}$ per sequence, which is negligible on NVLink ($m/\beta \approx 0.02\,\mu\mathrm{s}$). If the per-collective latency is constant ($a = 8\,\mu\mathrm{s}$, one-shot all-reduce), $t(P) = 19.6/P + 2 \cdot 64 \cdot 0.008\,\mathrm{ms} = 19.6/P + 1.02\,\mathrm{ms}$. This decreases monotonically: 10.8 ms at $P = 2$, 5.9 ms at $P = 4$, 3.5 ms at $P = 8$. So use the whole NVLink domain. If the latency instead grows like a ring all-reduce, $a(P) = 2(P-1)\alpha$ with $\alpha = 2\,\mu\mathrm{s}$, then $t = 10.3$, 6.4 and 6.0 ms for $P = 2, 4, 8$: diminishing returns, and a two-node $P = 16$ would be slower. Collective latency, not bandwidth, sets the limit on TP for decode.

Exercise 13.11. In a closed loop a new request is issued only when an old one completes, so the number in the system is capped at 64 and the offered load automatically drops when the server slows down. No queue can build up, and latency stays bounded by construction. Real traffic is open-loop: Poisson bursts arrive regardless of server state. During a burst the queue and the KV cache fill up, and preemption and queueing delay grow superlinearly near capacity

(Little's law plus queueing theory). Closed-loop tools also suffer from "coordinated omission": the slow periods generate fewer samples, so they are under-represented in the percentiles.

Exercise 13.12. Assume every message is 150 tokens. The prompt at turn k is about $300(k-1)+150$ tokens, so without reuse the total prefill is $\sum_{k=1}^{20}(300k-150) = 60{,}000$ tokens. With perfect reuse each turn prefills only the new user message plus a few template tokens (and the one generated token that was never fed back): about $20 \cdot 150 = 3{,}000$ tokens, a $20\times$ reduction. With the official Qwen3 template, pitfall (1) strikes first: the earlier assistant turns' `<think>` blocks are stripped when the history is re-serialised, so the common prefix ends at the start of the most recent assistant reply. Fixes: store in the history exactly what the template will emit (the reply without its think block) and accept re-prefilling that shorter reply once, or keep a cache keyed on the template's output. Engine-level prefix caching (radix trees) handles this automatically, at block granularity.

Chapter 14: Evaluating LLMs and Reasoning Models

Exercise 14.1 Typical findings: (i) *false negative* for unordered answers ("2, 3" vs. "3, 2" for "all solutions"), because tuples are compared in order; fixing it globally would break ordered pairs, so make it opt-in per problem (Exercise 14.2). (ii) *false negative* for units the reference keeps ("5 \text{ cm}" vs. "5" unwraps the `\text`, leaving "5 cm", which SymPy parses as $5\,c\,m$); stripping known unit words fixes it but risks deleting variables named m. (iii) *false positive* from the assignment rule: "$y = 5$" accepted for a question about x; restricting the rule to the variable named in the problem fixes it at the cost of parsing the problem. Every fix is a policy change — add a regression test for each.

Exercise 14.2 Normalize both sides, split into components, then match as a multiset: for each gold component find an unused predicted component with `equality_check`. Naively that is $O(k^2)$ SymPy calls. Cheaper: first bucket components by a canonical numeric key (evaluate each with `sympy.N` to, say, 12 significant digits, or use the normalized string), match identical keys in $O(k \log k)$, and fall back to pairwise symbolic checks only for the unmatched remainder.

Exercise 14.3 Record `hit_eos` (generation ended with EOS before the budget) and `has_box`. Then: `has_box` and wrong $\Rightarrow$ the model is wrong; `not hit_eos` and `not has_box` $\Rightarrow$ the budget truncated the chain of thought (raise `max_new_tokens`); `hit_eos` and `not has_box` $\Rightarrow$ a format failure (adjust the prompt or extraction fallback). Reporting the three rates alongside accuracy explains most surprising accuracy changes.

Exercise 14.4 (a) A: 205/500, Wilson 95% CI $[0.368, 0.454]$; B: 223/500, CI $[0.403, 0.490]$. The intervals overlap, so independent intervals suggest no clear difference. (b) McNemar uses only discordant items: $b = 30$, $c = 48$. With continuity correction $\chi^2 = (|b-c|-1)^2/(b+c) = 289/78 = 3.71$, $p \approx 0.054$; without correction $\chi^2 = 4.15$, $p \approx 0.042$; the exact binomial test gives $p \approx 0.054$. The paired test is the right one and is far more sensitive than comparing intervals, because per-item difficulty cancels; here the evidence is borderline, and a second seed or more items would be needed before claiming B is better.

Exercise 14.5 Useful row-steps are $\sum_i \ell_i$ out of $B \max_i \ell_i$, so the wasted fraction is $1 - \bar{\ell}/\max_i \ell_i$. For i.i.d. exponential lengths with mean $\mu = 600$, $\mathbb{E}[\max] = \mu H_B$ with $H_{64} \approx 4.744$, i.e. ≈ 2846 tokens, so waste $\approx 1 - 600/2846 \approx 79\%$. Even with a budget cap that truncates the

tail, static batching of long reasoning traces wastes most of the batch — the motivation for continuous batching (Chapter 12).

Exercise 14.6 For GPT-2 and Qwen BPE, `"A"` and `" A"` are distinct ids, and `encode("Answer: A")[-1]` is the space-prefixed variant. Summing the softmax probability of both variants per letter (computed in float32) is the robust choice; on base models the single-variant (no-space) version can disagree on a noticeable fraction of questions because almost all of the mass sits on the space-prefixed token.

Exercise 14.7 Sample votes from fixed strengths (e.g. Elo-scale 1000, 1050, 1100, 1200) by drawing random pairs and a winner with probability $p_i/(p_i + p_j)$. Online Elo with $K = 32$ produces visibly different ratings for different vote orders (spreads of tens of points), most of it driven by the last few hundred votes; BT is order-invariant by construction, and its bootstrap interval reflects only sampling noise. Lowering K reduces Elo's order dependence but slows convergence.

Exercise 14.8 Write $\ell(p) = \sum_i W_i \log p_i - \sum_{i<j} n_{ij} \log(p_i + p_j)$. Since $-\log x \geq 1 - \log y - x/y$ for $x, y > 0$ (concavity of log), at the current iterate $p^{(k)}$ the function $Q(p) = \sum_i W_i \log p_i - \sum_{i<j} n_{ij}\big(\log(p_i^{(k)} + p_j^{(k)}) + \frac{p_i + p_j}{p_i^{(k)} + p_j^{(k)}} - 1\big)$ minorizes ℓ and touches it at $p^{(k)}$. Q separates over i; setting $\partial Q/\partial p_i = W_i/p_i - \sum_{j \neq i} n_{ij}/(p_i^{(k)} + p_j^{(k)}) = 0$ gives exactly the MM update. Hence $\ell(p^{(k+1)}) \geq Q(p^{(k+1)}) \geq Q(p^{(k)}) = \ell(p^{(k)})$. Rescaling does not change ℓ, which is invariant to multiplying all p_i by a constant.

Exercise 14.9 Position bias appears as verdicts that flip when the order of the two responses is swapped; with small local judges flip rates of tens of percent are common. Debias by evaluating both orders and (a) counting a win only if it holds in both (ties otherwise), or (b) averaging the two scores/probabilities. Report the flip rate as a judge-quality metric.

Exercise 14.10 Work: $20 \times 500 \times 8 = 80{,}000$ generations; at an assumed mean of 2000 tokens that is 1.6×10^8 decode tokens. A 7B model is 14 GB in bf16 and fits on one GPU, so use data parallelism (one replica per GPU; tensor parallelism only adds communication). KV for a Llama-style 7B with GQA (32 layers, 8 KV heads, head dim 128) is ≈ 131 KB/token; at batch 128 and 2000-token average context that is ≈ 34 GB. Each step moves $\approx 14 + 34 = 48$ GB, i.e. ≈ 14 ms on an H100, producing 128 tokens: $\approx$ 9k tokens/s per GPU, $\approx$ 72k tokens/s for 8 GPUs, so ≈ 37 minutes at the roofline; with realistic 2–3× overheads, 1.5–2 hours. That fits in 6 hours. Grading 80k answers at $\sim$10 ms per SymPy check is $\sim$800 CPU-seconds — a few cores keep up easily, provided grading runs concurrently with generation.

Chapter 15: Inference-Time Scaling

Exercise 15.1 Let $\beta = 1/T$, $p_i \propto e^{\beta z_i}$, $Z = \sum_j e^{\beta z_j}$. Then $H = -\sum_i p_i(\beta z_i - \log Z) = \log Z - \beta\, \mathbb{E}_p[z]$. Using $\frac{d}{d\beta} \log Z = \mathbb{E}_p[z]$ and $\frac{d}{d\beta}\mathbb{E}_p[z] = \mathrm{Var}_p[z]$, $\frac{dH}{d\beta} = \mathbb{E}_p[z] - \mathbb{E}_p[z] - \beta\, \mathrm{Var}_p[z] = -\beta\, \mathrm{Var}_p[z]$. With $d\beta/dT = -1/T^2$, $\frac{dH}{dT} = \mathrm{Var}_p[z]/T^3 \geq 0$, with equality only if all logits are equal (zero variance).

Exercise 15.2 For $(0.62, 0.14, 0.08, \dots)$, top-$p$ with $p = 0.8$ keeps three tokens, so $k = 3$. For a peaked distribution such as $(0.97, 0.01, 0.01, 0.005, \dots)$, $k = 3$ keeps two tokens that together

have 2% mass and mostly add noise, while $p = 0.8$ keeps only the top token. Min-p: `keep = probs >= alpha * probs.max(-1, keepdim=True).values`; with $\alpha = 0.1$ it keeps one token in the peaked case and the first five in the flatter one, adapting like top-p but without sorting.

Exercise 15.3 Compute for each class the mean of `sum(lp)/len(lp)` over its samples and break ties by the maximum. Test: answers `["3", "4"]` with mean log-probs $(-1.2, -0.4)$: first-seen picks "3", confidence picks "4".

Exercise 15.4 Dynamic programming over (samples drawn, correct votes, wrong votes) with the stopping rule "leader minus runner-up > remaining" gives expected sample counts of ≈ 11.2 ($p = 0.2$), ≈ 14.5 ($p = 0.5$) and ≈ 10.0 ($p = 0.9$) for $n = 16$. Savings are largest when the model is *consistent* — consistently right or consistently wrong — and smallest at $p = 0.5$, where the vote stays close. The minimum possible is 9 samples (a 9–0 lead with 7 remaining).

Exercise 15.5 Expect several times more generated tokens with the CoT suffix. With Qwen3-0.6B's 114.7 KB of KV per token, a 150-token answer holds ≈ 17 MB of KV (plus the prompt), while a 900-token CoT answer holds ≈ 103 MB; peak KV per request grows linearly with reasoning length.

Exercise 15.6 Weight bytes equal KV bytes when $n = W/(\text{KV bytes per sequence})$. At 1024 tokens of context: Qwen3-0.6B, $W \approx 1.19$ GB and KV ≈ 117 MB per sequence, so $n \approx 10$; Qwen3-8B, $W \approx 16.4$ GB and KV ≈ 151 MB, so $n \approx 108$. Larger models stay weight-bound to much larger batches, so the marginal cost of an extra sample (only its KV traffic and compute) is relatively smaller: self-consistency is proportionally cheaper on large models, while small models hit the KV-bandwidth wall early.

Exercise 15.7 The untempered $p_\theta(y_t)$ is the model's belief, which is what a confidence score should measure. The probability with which the token was actually *sampled* is $q(y_t) = \text{softmax}\big(\text{filter}_{k,p}(z/T)\big)_{y_t}$, and the sequence probability is $\prod_t q(y_t \mid \cdot)$; it is the behaviour-policy probability needed for importance sampling. Mismatches between the two are a real source of bias in RL when rollouts are sampled with top-p/top-k (Chapter 18 discusses keeping the sampling mask).

Exercise 15.8 Without sharing, each sequence has $1500 + 900 = 2400$ tokens $= 150$ blocks, so $32 \times 150 = 4800$ blocks. With sharing, the prompt's 93 full blocks (1488 tokens) are shared; the last partial block (12 tokens) is copied on write into each sequence, which then holds $12 + 900 = 912$ tokens $= 57$ blocks. Total $93 + 32 \times 57 = 1917$ blocks, a 2.5× reduction.

Exercise 15.9 Weight each vote by $w = \exp(\bar{\ell})$ and pick the class with the largest total weight. It hurts when confidence is miscalibrated: e.g. two confident wrong samples with $w = 0.9$ outvote three hesitant correct ones with $w = 0.5$ ($1.8 > 1.5$), whereas the unweighted vote is correct. Base models are often confidently wrong on familiar-looking but subtly different problems.

Chapter 16: Self-Refinement and Scoring

Exercise 16.1 With `logits[1:]`, the row used for target token $i{+}1$ is the distribution for token $i{+}2$, so every answer token is scored under the wrong context (and the gather still

succeeds, because shapes match after the targets are sliced accordingly). The values are log-probabilities of real tokens under some distribution, hence plausible negative numbers. `test_token_logprobs_match_incremental` catches it, because it compares each entry with a token-by-token incremental computation.

Exercise 16.2 $\log \sum_j e^{z_j} = \log(e^M \sum_j e^{z_j - M}) = M + \log \sum_j e^{z_j - M}$. All exponents are ≤ 0, so no term overflows; the term with $z_j = M$ equals 1, so the sum lies in $[1, V]$ and its log in $[0, \log V]$ — no underflow to $\log 0$. The output for the argmax entry is $M - \mathrm{LSE} = -\log \sum_j e^{z_j - M} \in [-\log V, 0]$, and it is the largest output entry.

Exercise 16.3 Appending tokens adds terms $\log p < 0$, so the joint log-probability strictly decreases. The average can increase: an answer with token log-probs $(-3, -3)$ (mean -3) extended by $(-0.1, -0.1)$ has mean -1.55. To compare a 50-token and a 500-token solution use a length-normalized score (mean log-prob or perplexity), ideally restricted to the final-answer tokens if you care about the result rather than the prose.

Exercise 16.4 Compute $\bar{\ell}$ for each answer under the original prompt, grade with the verifier, bin by score decile and plot per-bin accuracy. A useful scorer shows accuracy rising with the decile; small base models are often only weakly calibrated, and the minimum-token or boxed-tokens-only variants can be more informative than the plain mean. Report the area under the ROC curve of score versus correctness as a single-number summary.

Exercise 16.5 Loop over position chunks of size C: run the model once to get the hidden states (or logits), and for each chunk compute `log_softmax` in float32 on C rows and gather the targets, keeping only the gathered values. Peak extra memory drops from $TV \times 4\,\mathrm{B}$ ($\approx 2.5\,\mathrm{GB}$ at $T = 4096$) to $CV \times 4\,\mathrm{B}$ ($\approx 39\,\mathrm{MB}$ at $C = 64$). Going further and chunking the final projection avoids materializing full logits at all.

Exercise 16.6 Report the three rates per scorer. Typically: no scorer has the highest harm rate (correct drafts broken by spurious critiques); the heuristic scorer mainly rejects revisions that lose the box or ramble; the log-prob scorer rejects many revisions and therefore has a low harm rate but also a low fix rate. The best ratio of fixes to harms is model-dependent, which is the point of measuring it.

Exercise 16.7 Ask the draft to state a checkable consequence (e.g. substitute the answer into the original equation) and evaluate it with SymPy; if the check fails, put "Substituting your answer gives ..., which is not equal to ..." into the review. With external feedback, revisions are triggered only by real errors and carry specific information, which typically improves the fix rate and nearly eliminates harm on correct drafts.

Exercise 16.8 Order each call as `[question][draft][critique][instruction]` (or run the loop as one growing conversation), so each call's prompt is the previous call's prompt plus its output plus a short instruction. Then, with perfect prefix caching, each call prefills only its new instruction: total prefill $\approx q = 150$ tokens instead of $q + k(2q + 2a + c) = 150 + 2(300 + 1000 + 120) = 2990$, a saving of ≈ 2840 tokens (95%). Decode work is unchanged.

Exercise 16.9 At ≈2000 decode tokens, best-of-4 decodes four 500-token answers in one batched call and scores them with four prefills; refinement with $k = 2$ makes five sequential calls. Expect best-of-4 to have roughly a third of the latency; which is more accurate depends on the scorer's calibration (best-of-n) versus the critique's quality (refinement), and must be measured.

Chapter 17: Reinforcement Learning with Verifiable Rewards: GRPO

Exercise 17.1 With partial credit the rewards of Fig. 17.10 become $(1, 1, 0.5, 0)$: mean 0.625, unbiased std 0.479, advantages $(+0.783, +0.783, -0.261, -1.305)$. The unboxed "7" is still pushed down, but by a third of its previous weight, and the wrong boxed "8" is now pushed down hardest. The loss is $-\frac{1}{4}[0.783(-2.269)+0.783(-3.059)-0.261(-1.299)-1.305(-3.400)] = -0.151$. Partial credit makes *correctness* the dominant signal early (correct-but-unboxed answers are no longer lumped with wrong ones), which helps when the model is rarely correct. It weakens the pressure to adopt the format, since an unboxed correct answer is only mildly penalised, so format adoption is slower. A common compromise is to anneal the partial credit to 0.

Exercise 17.2 With identical rewards, $r - \bar{r} = 0$ for every rollout, the advantages are exactly 0, the loss is $-0 \cdot \sum \log \pi = 0$ and autograd propagates zeros to every parameter (the optimiser step is still taken; AdamW's weight decay is the only change, and it is zero in our loop). This is desirable because the group carries no information about *which* behaviour is better. For $p \sim U[0, 1]$, $\mathbb{E}[p^G + (1-p)^G] = \frac{2}{G+1}$: 40% wasted for $G = 4$ and 11.8% for $G = 16$.

Exercise 17.3 With k correct rollouts, $\bar{r} = p = k/G$ and $\sum_i (r_i - \bar{r})^2 = k(1-p)^2 + (G-k)p^2 = Gp(1-p)$, so $s^2 = \frac{G}{G-1}p(1-p)$, giving $A^+ = (1-p)/s$ and $A^- = -p/s$. Then $\sum_i A_i = [k(1-p) - (G-k)p]/s = G(p-p)/s = 0$, and $\frac{1}{G-1}\sum_i A_i^2 = \frac{Gp(1-p)}{(G-1)s^2} = 1$ exactly (ignoring ϵ). As $p \to 1/G$ with G large, $A^+ = (1-p)/s \approx \sqrt{(1-p)/p} \approx \sqrt{G}$: a single success in a large group receives an advantage of order $\sqrt{G}$.

Exercise 17.4 b_i is a function of the other samples only, which are independent of y_i given x, so $\mathbb{E}[b_i \nabla \log \pi(y_i)] = \mathbb{E}[b_i]\,\mathbb{E}[\nabla \log \pi(y_i)] = 0$ and the estimator is unbiased. Algebra: $r_i - b_i = r_i - \frac{G\bar{r} - r_i}{G-1} = \frac{G}{G-1}(r_i - \bar{r})$. So GRPO's mean-centred advantage equals the RLOO advantage times $\frac{G-1}{G}$, a constant that the learning rate absorbs. The "bias" of including r_i in its own baseline is therefore only a rescaling. Dividing by the std adds a genuine per-prompt reweighting.

Exercise 17.5 (a) Summed loss: every token of rollout i has gradient weight A_i/G, independent of length. (b) Per-sequence mean: each token has weight $A_i/(G|y_i|)$. For correct answers of lengths 20 and 2000, the per-token push ratio is $1:1$ under (a) and $100:1$ under (b). Under (a) the total push on a sequence's log-likelihood is the same for both, but because $\log \pi$ of the long sequence is a sum of 100 times more terms, raising it is "harder". In practice the summed objective most efficiently increases the probability of the *short* correct answer. Under (b), long sequences are barely pushed per token, so long *wrong* answers are barely penalised. That is the length bias DAPO and Dr. GRPO criticise (Chapter 18).

Exercise 17.6 PyTorch raises *"Inference tensors cannot be saved for backward"* when the sampled ids (created inside inference mode) are fed through `nn.Embedding` in the differentiable

pass. The embedding saves its index input for backward. Fixes: sample under `no_grad`; or keep inference mode for sampling but `.clone()` the ids outside it (cloning outside inference mode produces a normal tensor).

Exercise 17.7 $16 \times 4096 \times 151{,}936 \times 4\,\text{B} \approx 39.8\,\text{GB}$ for the log-softmax, and the same again for its gradient, so $\approx 80\,\text{GB}$. Chunked version:

```
def sequence_logprob_chunked(model, ids, prompt_len, C=512):
    h = model_hidden_states(model, ids)          # (T, d) before the LM head
    total = 0.0
    for s in range(prompt_len - 1, ids.numel() - 1, C):
        logits = model.out_head(model.final_norm(h[s:s+C])).float()
        lp = torch.log_softmax(logits, -1)
        total = total + lp.gather(1, ids[s+1:s+1+C, None]).sum()
    return total
```

Peak memory scales with $C \cdot |\mathcal{V}|$ instead of $T \cdot |\mathcal{V}|$. With activation checkpointing per chunk the backward is bounded the same way. (Exposing hidden states requires a small model hook; `llmsys.rl.logprobs.token_logprobs(chunk=...)` chunks after the LM head, which bounds the float32 copy but not the bf16 logits.)

Exercise 17.8 Sequentially, t_{sample} grows linearly in both G and T (one forward per token per rollout); on CPU the toy gives a of the order of milliseconds per token. t_{train} is also linear in $G \cdot T$ but with a smaller constant per token, since a prefill-style forward processes all tokens in parallel. With `sample_rollouts` the G rollouts share each decode step, so $t_{\text{sample}} \approx a' T$ with a' almost independent of G until the batch stops being memory-bound. This is the decode roofline: weights are streamed once per step regardless of batch size.

Exercise 17.9 Qwen3-4B: 36 layers, 8 KV heads, head dim 128, so the KV cache is $2 \cdot 36 \cdot 8 \cdot 128 \cdot 2 = 147\,\text{KB}$ per token. (i) 256 sequences $\times 4352$ tokens $\approx 1.11 \times 10^6$ tokens, about 164 GB. (ii) Weights $\approx 8.0\,\text{GB}$ in bf16. Each of the 4096 decode steps streams the weights plus a cache that grows from 9.7 GB to 164 GB, about 95 GB on average, so roughly $4096 \times 95\,\text{GB}/3.35\,\text{TB/s} \approx 116\,\text{s}$. (iii) Training FLOPs $6 \cdot 4.0 \times 10^9 \cdot 1.11 \times 10^6 \approx 2.7 \times 10^{16}$, about 68 s at 40% MFU. The cache does not fit next to the weights on one 80 GB GPU ($\approx 72\,\text{GB}$ free). It needs $\lceil 164/72 \rceil = 3$ waves (or 3+ GPUs), which roughly preserves total sampling time, since each wave streams the weights 4096 times.

Exercise 17.10 Examples: (1) "`\boxed{7}` ... actually `\boxed{8}`" (first-box readers accept the wrong final answer); (2) "`\boxed{17}`" with truth 7 (substring match accepts); (3) "`\boxed{7.0}`" (no normalisation rejects a correct answer); (4) "`\boxed{\frac{14}{2}}`" (rejected without fraction handling); (5) "`\boxed{1, 2, 3, 4, 5, 6, 7}`" (substring or "contains" accepts). Harden by taking the last box, comparing normalised values exactly, and rejecting boxed lists when a scalar is expected. RL is most likely to discover the cheap ones (many boxes, lists), because they increase reward on *every* prompt without solving anything.

Exercise 17.11 With 20 held-out facts, GRPO transfers the *format* almost perfectly (boxing is prompt-independent behaviour). Correctness on held-out facts improves little, because a 76k-parameter model trained on 80 facts has not learned an addition algorithm, and RL with an outcome reward cannot teach a fact it never produces. This mirrors the MATH/MATH-500 split:

RLVR mostly improves how existing knowledge is deployed (format, persistence, self-checking), and held-out evaluation is what separates that from memorising the training prompts.

Chapter 18: Improving GRPO

Exercise 18.1 All three edits give 0: a typo makes the tag a different token sequence, so `index` raises; reversed order fails the `<` test; a missing tag raises. The string check catches the same three. The id-based check can be broken by tokenisation: if the tokenizer does not treat `<think>` as a special token (the Qwen3 *base* tokenizer splits it into three pieces), the text contains the tag but the single id never occurs, so the id check returns 0 while the string check returns 1. Conversely, the model may emit pieces that decode to the tag without the special id.

Exercise 18.2 In our runs (two seeds, 150 steps), every format variant drove `format_avg` to ≈ 0.99 within about 30 steps, because the SFT model already emitted `<think>` 20% of the time and the tag is the easiest reward available. Unconditional weight 1 reached a late-run `correct_avg` of about 0.74 and greedy accuracy of 0.84–0.87, below the baseline (0.83 and 0.89–0.96). Weight 0.1 behaved the same, since even a small weight breaks ties between otherwise identical rollouts. The conditional variant recovered the baseline's correctness (0.80–0.86, greedy 0.91–0.96). In Fig. 18.9, under plain summation a wrong-but-tagged rollout ties with a right-but-untagged one, so early gradient mainly teaches tagging. Conditioning makes correctness a prerequisite for any reward.

Exercise 18.3 With `inner_epochs`$= 1$ the clip fraction is exactly 0 (every ratio is 1). In our `clip_reuse4` runs (4 full-batch passes, learning rate 3×10^{-4}) the late-run clip fraction was only 2–3%, yet the clipped run kept the baseline's accuracy (0.88–0.95) while the unclipped one degraded to 0.50–0.62. A small fraction of clipped tokens can carry most of the damage, because they are exactly the tokens whose probability is running away. The clip fraction grows with the number of passes and with the learning rate; where it crosses 0.2 depends on both, so measure it. Wall-clock time per update falls roughly as $(t_{\text{sample}} + k\, t_{\text{update}})/k$, so the saving saturates once $k\, t_{\text{update}}$ dominates. On real reasoning workloads, where sampling dominates, $k = 2$–4 is the usual sweet spot.

Exercise 18.4 $\mathbb{E}_p[r] = \sum_y p(y)\frac{q(y)}{p(y)} = 1$, so $\mathbb{E}_p[k_3] = 1 - 1 - \mathbb{E}_p[\log r] = \mathbb{E}_p[\log \frac{p}{q}] = \mathrm{KL}(p\|q)$. Pointwise, $f(r) = r - 1 - \log r$ has $f(1) = 0$, $f'(r) = 1 - 1/r$ and $f'' > 0$, so $f \geq 0$. Numerically, all three sample means converge to the true KL (k_2 to a slightly different value, being biased). k_1 has the largest variance and negative samples; k_3 typically has the smallest variance when p and q are close.

Exercise 18.5 With zero advantages, the sequence surrogate's gradient is $\beta \nabla \sum_t \log \pi_\theta(y_t)$ on the fixed sampled tokens. Descending it lowers the probability of exactly those tokens, so probability flows to everything else, the sampled-token probability falls and entropy rises monotonically. The per-token k_3 term has gradient $\beta(1 - r_t)\nabla \log \pi_\theta(y_t)$ with $r_t = \pi_{\text{ref}}/\pi_\theta$. It is zero when the policy equals the reference and *changes sign* when the policy undershoots ($r_t > 1$ raises the token's probability again). It therefore pulls toward π_{ref} rather than away from the samples, and entropy returns to the reference's level.

Exercise 18.6 Per-token gradient weights for (short $+1$, long -1):

rule	per token (short)	per token (long)	total long
`seq_mean_token_mean`	$1/20 = 0.05$	$1/2000 = 0.0005$	0.5
`token_mean`	$1/1010$	$1/1010$	0.99
`seq_mean_token_sum_norm` ($L = 1000$)	$1/2000$	$1/2000$	0.5

Per-sequence averaging lets each token of the long wrong answer off 100 times more lightly than a token of the short right answer, which over many updates favours long wrong answers. Token-level and constant normalisation treat tokens equally.

Exercise 18.7 With std scaling, $\sum_i |A_i| = \frac{2Gp(1-p)}{s} = 2\sqrt{G(G-1)}\sqrt{p(1-p)}$. Without it, $2Gp(1-p)$. The ratio (with/without) is $1/s$, which is largest when $p(1-p)$ is small, i.e. for nearly-always-solved and nearly-never-solved prompts. For $G = 8$: at $p = \frac{1}{8}$ the scaled weight is 4.95 versus 1.75 unscaled; at $p = \frac{1}{2}$ it is 7.48 versus 4. Std scaling up-weights edge cases relative to medium-difficulty prompts. Dr. GRPO calls this a difficulty bias. One could argue the reverse, that rare successes on hard prompts are the most informative, which is why DeepSeek-V3.2 keeps it.

Exercise 18.8 Full `token_entropy` on a $(4096, 151{,}936)$ tensor creates float32 copies of the logits, log-probs and their product: about three times 2.5 GB. The chunked version holds $\approx 3 \times C \times |\mathcal{V}| \times 4\,\text{B}$, i.e. ≈ 0.47 GB at $C = 256$ and ≈ 1.9 GB at $C = 1024$, with run time almost unchanged above $C \approx 128$ (each chunk is still a large, efficient kernel). Below that, launch overhead dominates.

Exercise 18.9 The policy with AdamW needs $32 \times 10^9 \times 16\,\text{B}/16 = 32$ GB per GPU when FSDP-sharded. (a) A full bf16 reference adds 64 GB per GPU, 96 GB in total, which does not fit on 80 GB GPUs. (b) Sharding it like the policy adds 4 GB per GPU, plus an all-gather of ≈ 64 GB of reference weights per forward pass (each GPU receives $\frac{15}{16} \cdot 64$ GB per step): ≈ 0.15 s at 400 GB/s NVLink, over a second across 50 GB/s InfiniBand, overlappable layer by layer. (c) The rollout workers compute reference log-probs. That costs trainer memory nothing and ships only 4 bytes per token (4 MB per million tokens), but the rollout fleet must hold an extra 64 GB copy (or reuse the base weights if the policy is a LoRA adapter). Option (c) is common in disaggregated systems, and (b) in colocated ones.

Exercise 18.10 Casting to bf16 changes per-token log-probs by roughly 10^{-3}–10^{-2} nats, with heavy tails on low-probability tokens. On the toy task, training is stable either way, because sequences are short and the mismatch is small. The truncated weights mostly sit within $[0.99, 1.01]$, with a few capped outliers. The effect becomes important on long sequences (thousands of tokens, where small per-token differences compound) and with aggressive sampler optimisations such as FP8 KV caches or speculative decoding.

Chapter 19: RL Training Systems

Exercise 19.1 (Optimal split). Let $x = G_{\text{gen}}$. $T(x) = W_g/x + W_t/(G-x) + T_{\text{sync}}$; setting $T'(x) = 0$ gives $W_g/x^2 = W_t/(G-x)^2$, so $x^\star/G = \sqrt{W_g}/(\sqrt{W_g} + \sqrt{W_t})$ and $T^\star = (\sqrt{W_g} + \sqrt{W_t})^2/G + T_{\text{sync}} = (W_g + W_t + 2\sqrt{W_g W_t})/G + T_{\text{sync}}$. Since $2\sqrt{W_g W_t} \geq 0$, $T^\star \geq (W_g + W_t)/G = T_{\text{coloc}} - T_{\text{switch}}$ (ignoring T_{sync}), with equality iff $W_g W_t = 0$. The synchronous split wins iff $T_{\text{switch}} > 2\sqrt{W_g W_t}/G + T_{\text{sync}}$, i.e. only when switching is very expensive.

Exercise 19.2 (KV budget). Qwen3-4B has ≈4.02B parameters (≈8.0 GB in bf16). Its KV cost is the same as the 8B model's, $2 \cdot 36 \cdot 8 \cdot 128 \cdot 2 = 147$ KB per token, because layer count and KV heads match. Budget $\approx 80 - 8.0 - 8 = 64$ GB; at 6,000 live tokens per sequence, 64e9/(147e3 · 6000) ≈ 72 sequences. Each decode step streams ≈ 72 GB, ≈ 21.5 ms at 3.35 TB/s, so the bound is ≈ 72/0.0215 ≈ 3,400 tokens/s per GPU—long contexts make KV, not weights, the dominant traffic.

Exercise 19.3 (Staleness bound). Batch k starts generating at time ≥ the completion of training step $k - s - 1$, so at least $k - s$ updates have been applied; its version $v \geq k - s$. It is consumed by training step k, after which the policy has version $k + 1$; the update to θ_{k+1} uses data from θ_v, staleness $k - v \leq s$. With $s = 0$, batch k cannot start until step $k - 1$ finishes, which is the synchronous split schedule (Fig. 19.5a with separate pools).

Exercise 19.4 (Loss normalisation). (a) $(\ell_1 + \ell_2)/2$, each micro-batch weighted equally regardless of size. (b) $(100\ell_1 + 1000\ell_2)/1100$. Fix: before the accumulation loop all-reduce the total number of loss tokens N_{glob} over micro-batches and ranks, compute each micro-batch loss as `(tok_loss * mask).sum() / N_glob`, and do not divide by the number of micro-batches or ranks again (set the DDP gradient averaging accordingly, or multiply by the world size).

Exercise 19.5 (TIS bias). With $\log \rho \sim \mathcal{N}(-s^2/2, s^2)$ (so $\mathbb{E}[\rho] = 1$), $\mathbb{E}[\min(\rho, C)] = \Phi(\frac{\ln C - s^2/2}{s}) + C(1 - \Phi(\frac{\ln C + s^2/2}{s}))$. For $C = 2$ the bias $\mathbb{E}[\min(\rho, C)] - 1$ is ≈ 0 at $s = 0.1$, -0.0015 at $s = 0.3$, -0.026 at $s = 0.5$ and -0.19 at $s = 1$. Mismatch is usually small ($s \ll 1$), so the bias is negligible while the variance reduction from removing the heavy right tail is large; when s is large the bias is a warning that the engine and trainer disagree too much.

Exercise 19.6 (Packing efficiency). At most one bin in a first-fit packing is at most half full: if bins $i < j$ were both at most half full, the items of j would have fit into i when they were placed. Hence with k bins the total size exceeds $(k - 1) \cdot \text{cap}/2$, and OPT ≥ total/cap gives $k < 2\,\text{OPT} + 1$. Empirically, padded efficiency falls quickly with σ (e.g. ≈0.4 at $\sigma = 0.8$ for batches of 8), sorting recovers a large part, and FFD packing stays above 0.9.

Exercise 19.7 (Partial-rollout bias). Record `[r.length for r in fin]` per step. In the first steps the consumed mean length is well below the generated mean (short responses finish first); as carried-over long responses complete, later batches contain them and the consumed distribution approaches the generated one after a few steps. Larger `oversample` increases the bias in each step (more short responses compete for the B slots) but also increases throughput; the steady-state mixed-policy fraction grows with it.

Exercise 19.8 (Weight sync across sites). Qwen3-32B has ≈ 32.8B parameters: bf16 ≈ 65.5 GB, i.e. ≈ 5.2 s at 12.5 GB/s; fp8 ≈ 2.6 s. LoRA rank 64 on all seven projections of 64 layers is $64 \cdot 64 \cdot \sum(d_{\text{in}} + d_{\text{out}}) \approx 537$M parameters, ≈ 1.07 GB in bf16, ≈ 0.09 s. With full weights, use bounded-staleness asynchronous RL so the 5 s transfer overlaps generation; with LoRA, even a synchronous schedule pays little.

Exercise 19.9 (Attacking the grader). (1) `os._exit(0)` or `sys.exit(0)` before the tests run—run tests in a separate process that imports the candidate, and require a sentinel printed after all tests; (2) returning an object whose `__eq__` always returns `True`—compare with type

checks or serialise results; (3) monkeypatching `assert`-like helpers, `unittest` or builtins—execute tests in a fresh namespace after the candidate code and verify the runner's integrity; (4) reading expected outputs from disk or environment—keep tests outside the sandbox, mount nothing, empty environment; (5) exhausting memory/CPU so the grader crashes into a default—treat every abnormal exit as failure and count resources. Writing (1) and (2) as tests against Listing 19.4 shows that the teaching sandbox is vulnerable to both.

Exercise 19.10 (Measurement). An `async+partial` policy runs the partial-rollout generator in its own pool while the previous batch trains. Expect throughput above both `async` and `partial` for heavy tails, with mean staleness between 1 and 2 and a mixed-policy fraction similar to `partial`; throughput rises and saturates with `oversample` while staleness and length bias grow.

Exercise 19.11 (Design). One reasonable answer: 14B bf16 weights are $\approx 29\,\text{GB}$, KV $\approx 2 \cdot 40 \cdot 8 \cdot 128 \cdot 2 \approx 164\,\text{KB/token}$. Responses average 6k tokens and are capped at 32k, so the tail is severe: use split pools with bounded-staleness async plus partial rollouts. Generation: 12 GPUs as TP= 2 engines (6 replicas, $\sim$100 GB KV each for $\sim$100–150 concurrent long sequences per replica); training: 20 GPUs with FSDP (16 B/param $\Rightarrow$ 224 GB of state, ≈ 11 GB per GPU) plus activation checkpointing and token-budget packing. Balance the pools by measuring W_g and W_t in the first steps, drop the reference model (KL= 0) to save memory, enable TIS and log mismatch, and stream rewards to a CPU verifier pool.

Chapter 20: Distillation for Efficient Reasoning

Exercise 20.1 (Split balance). With 25 validation examples drawn at random from a heavy-tailed length distribution, the validation mean length can differ noticeably from the training mean. Validation loss is length-sensitive because per-token loss varies along a trace (early reasoning tokens are harder to predict than formulaic later ones) and long traces contribute more tokens. A stratified split sorts examples by length, cuts them into q quantile buckets, and draws $\lfloor 25/q \rfloor$ validation examples from each bucket with a fixed seed.

Exercise 20.2 (Without think tags). Without tags the target is the trace followed by the answer with no delimiter; validation loss is usually similar (the tags are two easy tokens), so differences show up in behaviour rather than loss: tagged students can be parsed and can be switched between thinking and non-thinking modes by pre-filling an empty `<think></think>` block, which untagged students never saw. The lab's `bench.py` compares tagged, untagged and answer-only students on the synthetic task.

Exercise 20.3 (Loss identity). `answer_only_loss` averages $-\log p_\theta(y_t \mid x, y_{<t})$ over the n positions whose targets are answer tokens (logit indices `prompt_len-1` to $T-2$), which by the chain rule is $-\frac{1}{n}\log p_\theta(y \mid x)$. For a batch, the token-level loss is $\sum_i n_i \bar{\ell}_i / \sum_i n_i$ and the average of per-example losses is $\frac{1}{B}\sum_i \bar{\ell}_i$; they are equal for all values of $\bar{\ell}_i$ iff all n_i are equal.

Exercise 20.4 (Off-by-one). Starting at `prompt_len` removes the target at the first answer position, so the model is never trained to predict `<think>` after `<assistant>`. Its first generated token is then whatever the untrained distribution favours, typically not `<think>`; greedy decoding starts in the wrong mode, and even when the model recovers, the answer format differs from training. Expect the rate of `<think>`-first outputs to drop from $\approx 100\%$ to near zero.

Exercise 20.5 (KD temperature). Let $q = \mathrm{softmax}(z/\tau)$ (student) and $p = \mathrm{softmax}(u/\tau)$ (teacher). $\mathrm{KL}(p\|q) = \sum_j p_j \log p_j - \sum_j p_j \log q_j$ and $\partial(-\sum_j p_j \log q_j)/\partial z_i = (q_i - p_i)/\tau$, so $\partial[\tau^2 \,\mathrm{KL}]/\partial z_i = \tau(q_i - p_i)$. Without the τ^2 factor the soft-target gradient would shrink as $1/\tau$ (and, for large τ, as $1/\tau^2$ after expanding $q - p$ in the logit difference), so mixing it with the unit-temperature CE term would silently change the relative weight α with temperature.

Exercise 20.6 (Top-k storage). For 3,000 tokens: full bf16 distributions $3000 \times 151{,}936 \times 2 \approx 912\,\mathrm{MB}$; top-8 at 6 bytes per entry $= 144\,\mathrm{KB}$; top-32 $= 576\,\mathrm{KB}$; top-128 $= 2.3\,\mathrm{MB}$; token ids $= 12\,\mathrm{KB}$. For a peaked teacher (most mass in a few tokens) the sparse-loss error is tiny already at $k = 8$; for a flat teacher it decays slowly with k because renormalising over the top-k support sharpens the target.

Exercise 20.7 (Forward vs. reverse KL). Minimising forward KL over Gaussians matches moments: for the mixture $\frac{1}{2}\mathcal{N}(-2.2, 0.6^2) + \frac{1}{2}\mathcal{N}(2.2, 0.6^2)$ the fit has mean 0 and variance $0.6^2 + 2.2^2 = 5.2$ (std ≈ 2.3), placing mass in the valley. Reverse KL has two local minima, each close to one component ($\mu \approx \pm 2.2$, $\sigma \approx 0.6$). A capacity-limited student trained with forward KL hedges between reasoning strategies and may produce incoherent mixtures; with reverse KL it commits to one strategy and may lose coverage of the other.

Exercise 20.8 (Logit memory). Weights+grads+Adam: $\approx 9.5\,\mathrm{GB}$ at every length. Activations without checkpointing: roughly 24k bf16 values per token per layer ($\approx 47\,\mathrm{KB}$), $\times 28$ layers $\approx 1.3\,\mathrm{MB}$/token, i.e. ≈ 2.7, 10.8 and 43 GB at 2k, 8k and 32k; with per-layer checkpointing about 0.1 MB/token (≈ 0.2, 0.8, 3.4 GB). Logits in fp32: 1.2, 5.0 and 19.9 GB (plus their gradient), capped at $\approx 0.6\,\mathrm{GB}$ by chunking. At 2k the optimizer state dominates; at 8k activations (without checkpointing); at 32k activations without checkpointing and the logits with it, which is why long-trace training needs both checkpointing and chunked/fused cross-entropy. (These are order-of-magnitude estimates; measure with `torch.cuda.max_memory_allocated`.)

Exercise 20.9 (Packing for SFT). Make a differentiable variant of `varlen_forward` by removing `@torch.no_grad()` and returning logits for the packed row; build targets per segment with `IGNORE` on prompt positions and on the last token of each segment (no cross-segment target), then compute `cross_entropy(..., reduction="sum")` divided by the total number of answer tokens. Because attention is block-diagonal and positions restart per segment, each segment's logits equal its unpacked logits, so the packed loss equals the padded token-level loss.

Exercise 20.10 (Cost per solved problem). A: $2100/0.33 \approx 6{,}364$ tokens per solved problem; B: $900/0.30 = 3{,}000$. B is $2.1\times$ cheaper. A wins only if B's accuracy falls below $900/6364 \approx 14.1\%$.

Exercise 20.11 (Teacher generation throughput). Qwen3-32B: $\approx 65.5\,\mathrm{GB}$ of bf16 weights, KV $= 2 \cdot 64 \cdot 8 \cdot 128 \cdot 2 \approx 262\,\mathrm{KB}$ per token. On 8 H100s (640 GB) with one TP$= 8$ replica and $\sim$64 GB overhead, $\approx 510\,\mathrm{GB}$ holds KV for ≈ 970 sequences at 2k average live context. A decode step streams weights plus KV, $\approx 576\,\mathrm{GB}$ at $8 \times 3.35\,\mathrm{TB/s} \approx 21\,\mathrm{ms}$, giving ≈ 45k tokens/s and $\approx 800\,\mathrm{s}$ (≈ 13 minutes) for 36M tokens as an upper bound. Compute is $\approx 2N \times 45\mathrm{k} \approx 3 \times 10^{15}\,\mathrm{FLOP/s}$, $\approx 38\%$ of peak, so the job is bandwidth-bound but not far from the ridge; in practice expect a few times longer because of prefill, scheduling, the length tail, and imperfect TP scaling.

Appendix A: GPU Kernels with Triton

Exercise A.1. $\lceil 10^6/1024 \rceil = 977$ programs, covering $977 \cdot 1024 = 1{,}000{,}448$ lanes, so 448 lanes are masked. `tl.arange` requires a power-of-two extent: register layouts distribute a block evenly over threads, warps and vector lanes, all of which come in powers of two.

Exercise A.2. With `other=0.0` the padded lanes take part in both reductions. The max is harmless (softmax is shift-invariant, though all-negative rows then get a needlessly shifted max), but the sum gains e^{0-m} per padded lane. For the row $(1, 2, 3)$ with `BLOCK`$= 4$: the true denominator is $e^{-2} + e^{-1} + 1 \approx 1.503$, but the kernel adds $e^{-3} \approx 0.050$. Every output is then too small by a factor 1.503/1.553 and the stored row sums to 0.968.

Exercise A.3. Loop over the row in chunks of `BLOCK` with running (m, ℓ) updated by online rescaling, then loop again computing e^{x-m}/ℓ and storing it. That is two reads and one write of the row. A single write is natural; a single *read* is possible only if the row is cached in SMEM between the passes, which the compiler cannot do for unbounded rows. For moderate lengths, rely on L2 to serve the second read.

Exercise A.4. $I = 2B_M B_N/((B_M + B_N)b)$. In bf16: 32, 64 and 85 FLOP/B; in fp8: 64, 128 and 171. The H100's ridge points are about 295 (bf16) and 591 (fp8), so none of these tiles could run at peak if every load came from HBM. The answer changes because panels of A and B are re-read by many programs: with good ordering most re-reads hit in L2, whose bandwidth is several times HBM's. The relevant intensity is then whole-problem FLOPs over HBM bytes, which is $n/3$ for a square GEMM.

Exercise A.5. With row-major order the first P programs cover $\lceil P/C \rceil$ rows and $\min(P, C)$ columns of tiles, which touches that many A and B panels. With group size G ($G \le R$, $P/G \le C$) they cover G rows and $\lceil P/G \rceil$ columns. The total $G + P/G$ is minimised at $G = \sqrt{P}$, with value $2\sqrt{P}$ instead of up to $P + 1$. For $P = 9$: $3 + 3 = 6$ panels instead of 8 for $C = 6$ (Fig. A.7).

Exercise A.6. Each stage buffers one A and one B slab: $s(B_M B_K + B_K B_N) \cdot 2$ bytes, i.e. 32 KB per stage for $128 \times 128 \times 64$. With 228 KB, $s \le 7$, leaving one program per SM. Typical choices are $s = 3$ (96 KB, two programs per SM) or $s = 4$ (128 KB, one program per SM). Remember that the epilogue may need SMEM for the output tile as well (it is usually aliased onto pipeline buffers).

Exercise A.7. Load the packed tile (`BK/2`×`BN` bytes), split nibbles with `& 0xF` and `» 4`, and interleave them back to a `BK`×`BN` tile of codes. When the group size equals `BK`, each K iteration has exactly one scale and zero point per output column. Load those two `BN`-vectors once per iteration. Either dequantize before the dot, $w = (q - z) \cdot s$, or compute `tl.dot(x, q - z)` and multiply the `BM`×`BN` partial product by s before adding it to `acc`. The latter applies the scale to $B_M B_N$ values instead of $B_K B_N$. The scale cannot be deferred to the epilogue as in W8A16, because it changes along K.

Exercise A.8. Q and O cost $2 \cdot H\,T\,d \cdot 2\,\text{B} = 134\,\text{MB}$. Each of the $H\,T/B_M$ programs streams all of K and V ($2S\,d \cdot 2\,\text{B} = 4.2\,\text{MB}$ per head), so without any cache hits the total is $(T/B_M) \cdot 134\,\text{MB}$: 17.2 GB for $B_M = 64$ and 8.6 GB for $B_M = 128$ (half of that with causal skipping). Naive

attention writes and re-reads a 4.3 GB score matrix and a 4.3 GB probability matrix, about 17 GB in total. So without L2 the tiled kernel would *not* save HBM traffic! It wins because one head's K and V (4.2 MB) fit comfortably in the 50 MB L2: programs for the same head run concurrently and reuse them. The real HBM traffic is then close to the 268 MB of Q, K, V, O. This is also why the grid puts the query-block index first (adjacent programs share a head).

Exercise A.9. With $T = 1$ there is exactly one query block per (batch, head), so the grid is $(1, B \cdot H)$, often fewer programs than SMs, each streaming a long KV. For split-KV, chunk c returns its running max m_c, normaliser ℓ_c and unnormalised accumulator $\boldsymbol{a}_c$. A second kernel computes $M = \max_c m_c$, $L = \sum_c e^{m_c - M} \ell_c$ and $\boldsymbol{o} = \sum_c e^{m_c - M} \boldsymbol{a}_c / L$. This is exact by Proposition A.1 applied to chunks instead of blocks. (Equivalently, store $\boldsymbol{a}_c / \ell_c$ and the log-sum-exp $m_c + \log \ell_c$.)

Exercise A.10. Interleave the columns of W_g and W_u so that every output tile contains matching gate and up columns, for example the first $B_N/2$ columns of the tile from W_g and the next $B_N/2$ from the same feature range of W_u. The epilogue then computes $\mathrm{SiLU}(g) \odot u$ and writes a $B_M \times B_N/2$ tile. The constraint is that B_N must cover pairs, so tiles see half as many output features per program. Per token and layer, the unfused path writes g and u ($2 \cdot 12288 \cdot 2\,\mathrm{B} = 49\,\mathrm{KB}$), reads them back (49 KB) and writes h (24.6 KB). The fused path only writes h, saving about 98 KB per token per layer, or ≈ 3.5 MB per token over 36 layers.

Exercise A.11. From $r = (\frac{1}{n} \sum_k x_k^2 + \epsilon)^{-1/2}$ we get $\partial r / \partial x_j = -r^3 x_j / n$. So $\partial \hat{x}_i / \partial x_j = r\delta_{ij} - x_i r^3 x_j / n = r(\delta_{ij} - \hat{x}_i \hat{x}_j / n)$, and $\partial \mathcal{L} / \partial x_j = \sum_i g_i \, \partial \hat{x}_i / \partial x_j = r(g_j - \hat{x}_j \frac{1}{n} \sum_i g_i \hat{x}_i)$ with $g_i = dy_i \, w_i$. Also $\partial \mathcal{L} / \partial w_i = \sum_{\text{rows}} dy_i \hat{x}_i$. Recomputing r is safe because it is a deterministic function of x, which the backward kernel loads anyway. It is cheaper because a row reduction on data already in registers is a few FLOPs per element, while saving r would cost an extra write in the forward, a read in the backward, and activation memory in between.

Exercise A.12. Our kernel layout is `(num_blocks, block_size, H, D)`. Per block, one program loads $16 \cdot 128 \cdot 2\,\mathrm{B} = 4\,\mathrm{KB}$ of K and as much of V: 8 KB, in 256-byte rows strided by $H \cdot D$. (A head-major block layout makes this fully contiguous.) With GQA, one program per (sequence, KV head) gives $2 \cdot 8 = 16$ programs, about 12% of a 132-SM GPU, each streaming 32k tokens. Split each sequence into 16–32 chunks of 1–2k tokens, giving 256–512 programs (2–4 per SM), and merge with the reduction from Exercise A.9.